

# 河南工业大学 ACM 校队选拔能力手册

知识体系版 | 基于公开新生周赛

---

78 场周赛 | 76 场公开 | 588 个题目位置 | 526 道唯一公开题

C++17 编译 526/526 | 公开样例/合法构造通过 496 | 核验日期 2026-07-25

基于 HAUTOJ 公开周赛、题面与可公开检索的题解整理。密码场次不绕过、未知选拔规则不臆测；训练分析不等同于校队官方大纲。

河南工业大学 ACM/ICPC 新生训练资料

# 收录边界与核验口径

- 核验日期：2026-07-25。
- 官网识别 78 场新生周赛，其中 76 场题目公开、2 场需要竞赛密码。
- 公开周赛共有 588 个题目出现位置，对应 526 道唯一题；重现赛题在时间版按原位置重现。
- CID 1045、CID 1044 只能确认比赛标题，公开页无法查看题目；本书不猜题、不绕过密码。
- 代码核验：526/526 道通过 C++17 编译；496 个可机读公开样例/合法构造通过；15 个特殊/不可机读样例跳过；15 道题无成对公开样例；已执行样例失败 0。
- 公开样例通过不等于隐藏数据必然 AC；应继续在原 OJ 提交验证。
- 难度、分类和训练优先级是教学分析，不是官方选拔大纲或入选结论。

# 数据画像

公开场次题数中位数为 8，范围为 6~11 题。接受率只反映官网历史快照，不是官方难度或分数线。

## 按年份覆盖

| 年份   | 场次 | 题目出现 |
|------|----|------|
| 2025 | 8  | 66   |
| 2024 | 7  | 60   |
| 2023 | 6  | 50   |
| 2022 | 8  | 66   |
| 2021 | 8  | 64   |
| 2020 | 5  | 42   |
| 2019 | 8  | 56   |
| 2018 | 10 | 61   |
| 2017 | 10 | 70   |
| 2016 | 8  | 53   |

## 按题号位置的作答画像

| 题号 | 出现 | 通过/提交       | 历史通过率 |
|----|----|-------------|-------|
| A  | 76 | 11077/34025 | 32.6% |
| B  | 76 | 8582/23129  | 37.1% |
| C  | 76 | 6427/20586  | 31.2% |
| D  | 76 | 5206/15333  | 34.0% |
| E  | 76 | 4420/12055  | 36.7% |
| F  | 76 | 4851/14385  | 33.7% |
| G  | 57 | 3070/10165  | 30.2% |
| H  | 49 | 2620/6690   | 39.2% |
| I  | 12 | 543/1319    | 41.2% |
| J  | 11 | 794/2416    | 32.9% |
| K  | 3  | 65/198      | 32.8% |

## 知识模块规模

| 模块           | 题数  |
|--------------|-----|
| 入门语法、输入输出与格式 | 105 |
| 模拟、枚举与构造     | 22  |
| 数组、字符串与双指针   | 120 |
| 排序、二分与区间     | 26  |
| 数学、数论与组合     | 94  |
| 贪心与博弈        | 67  |
| 数据结构         | 31  |
| 动态规划         | 21  |
| 搜索、图论与树      | 25  |
| 计算几何、概率与综合   | 15  |

## 命题风格与优先级

- 1. 基础实现决定下限：分支、字符串、数组、格式和模拟反复出现，先消灭本应拿到的失分。
- 2. 前中段常考观察加线性实现：奇偶、计数、排序、连续段、gcd 和简单贪心。
- 3. 后段以图论、搜索、动态规划、二分、堆/集合、线段树和综合模拟拉开差距。
- 4. 构造、ASCII 图形、中文标点和浮点题应使用专门检查方法，不能只与样例逐字比较。
- 5. 数据范围就是重要提示：小  $n$  关注枚举/状态压缩，大  $n$  寻找单调性与高效维护。

## 零基础工具箱

标准工作流：读范围和输入形式 → 手算最小样例 → 写朴素解 → 从重复计算或单调性优化 → 编译 → 边界测试 → 原题提交。

## 复杂度速查

| 规模                     | 常见可行复杂度                |
|------------------------|------------------------|
| $n \leq 20$            | $O(2^n \cdot n)$ ，状态压缩 |
| $n \leq 100$           | $O(n^3)$ 可能可行          |
| $n \leq 1000$          | $O(n^2)$ 常可行           |
| $n \leq 2 \times 10^5$ | $O(n \log n)$ 或 $O(n)$ |
| $n \approx 10^6$       | 线性或近线性                 |

## 必备 C++17 骨架

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     if (!(cin >> n)) return 0;
8 |     // 读入 -> 计算 -> 输出
9 |     return 0;
10 | }
```

## 浮点输出：printf 与 setprecision

两种写法都常见。固定保留 6 位小数可使用 printf，也可使用 fixed 与 setprecision；整份程序选择一套输入输出体系。

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```
4 | double x;
5 | scanf("%lf", &x);
6 | printf("%.6f\n", x);
7 | return 0;
8 | }

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     double x;
7 |     cin >> x;
8 |     cout << fixed << setprecision(6) << x << '\n';
9 |     return 0;
10 | }
```

- `setprecision(6)` 单独使用表示 6 位有效数字；加 `fixed` 才表示小数点后 6 位。
- 调用 `ios::sync_with_stdio(false)` 后不要混用 `printf` 与 `cout`，以免输出顺序异常。

## 16 周训练路线

| 周  | 主题                      | 达标                           |
|----|-------------------------|------------------------------|
| 1  | 输入输出、分支                 | 零编译错误                        |
| 2  | 循环、数组、计数                | 稳定线性扫描                       |
| 3  | 字符串、连续段                 | 掌握 <code>getline</code> /子序列 |
| 4  | 排序与并列规则                 | 写对比较器                        |
| 5  | 枚举、模拟、构造                | 会估复杂度与检查构造                   |
| 6  | 基础数学、奇偶                 | 从小数据推导公式                     |
| 7  | <code>gcd</code> 、质数、取模 | 处理整除与溢出                      |
| 8  | 前缀和、双指针                 | 减少重复计算                       |
| 9  | 贪心与区间                   | 写出证明                         |
| 10 | 栈、队列、集合、堆               | 选对数据结构                       |
| 11 | BFS、DFS、建图              | 处理访问与不可达                     |
| 12 | DP、背包                   | 状态转移完整                       |
| 13 | 二分、状态压缩                 | 识别单调性与小 <code>n</code>       |
| 14 | 树、最短路、综合                | 完成中后段题                       |
| 15 | 四场虚拟赛                   | 记录首 AC 与错因                   |
| 16 | 选拔模拟                    | 连续稳定输出                       |

## 全题知识体系详解

每题只在主模块中完整出现一次；出处列出全部周赛。多标签题可通过标题、PID 和文末周赛索引反查。

# 模块 1 | 入门语法、输入输出与格式

目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

本模块共 105 道唯一题。

## 1.1 | 总 1/526 A Simple Math Problem ( PID 1211 )

出处：2016级河南工大新生周赛（二）（B，CID 1007） | 训练定位：入门 | 入门语法、输入输出与格式

标签：定积分、多项式、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：134/181 ( 74.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：请计算  $\int (ax+bx^2+cx^3)dx$  在  $p$  到  $q$  上的定积分。

输入要点：输入一行  $a\ b\ c\ p\ q$  ( 保证  $p < q$  )

输出要点：一行答案，保留3位小数

### 零基础先修

- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：逐项积分： $\int (ax+bx^2+cx^3)dx = a \cdot x^2/2 + b \cdot x^3/3 + c \cdot x^4/4$ 。写成原函数  $F(x)$ ，答案就是  $F(q)-F(p)$ 。使用 `double` 并固定三位小数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double a, b, c, p, q;
5 |     cin >> a >> b >> c >> p >> q;
6 |     auto primitive = [&](double x) {
7 |         return a * x * x / 2.0 + b * x * x * x / 3.0 + c * x * x * x * x / 4.0;
8 |     };
9 |     cout << fixed << setprecision(3) << primitive(q) - primitive(p) << '\n';
10 |    return 0;
11 | }
```

### 公开样例

样例 1 输入

1.0 1.0 1.0 1.0 2.0

样例 1 输出

7.583

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.2 | 总 2/526 ykc的简单数学题 ( PID 1220 )

出处：2016级河南工大新生周赛（四）（A，CID 1009）| 训练定位：入门 | 入门语法、输入输出与格式

标签：数组、局部极值、趋势判断 | 限制：1.000 S / 128 MB | 历史统计：231/444 ( 52.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定  $n$  个整数  $a_1, a_2, \dots, a_n$  表示销售量，请计算出这些天总共有多少个折点。

输入要点：输入的第一行包含一个整数  $n$

第二行包含  $n$  个整数，用空格隔开，分别为  $a_1 \dots a_n$

输出要点：输出一个整数，表示折点出现的数量。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：折点只能出现在第  $2..n-1$  天。若  $(a[i]-a[i-1])$  与  $(a[i+1]-a[i])$  异号，走势便由升转降或由降转升，该天计一次。题目保证相邻值不同，所以无需处理差为 0 的平台。可直接用两个布尔比较，避免乘积潜在溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(n)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 | ios::sync_with_stdio(false);
5 | cin.tie(nullptr);
6 | int n;
7 | cin >> n;
8 | vector<int> sales(n);
9 | for (int& x : sales)
10 |     cin >> x;
11 | int turns = 0;
12 | for (int i = 1; i + 1 < n; ++i) {
13 |     bool mx = sales[i] > sales[i - 1] && sales[i] > sales[i + 1];
14 |     bool mn = sales[i] < sales[i - 1] && sales[i] < sales[i + 1];
15 |     turns += mx || mn;
16 | }
17 | cout << turns << '\n';
18 | return 0;
19 | }

```

## 公开样例

样例 1 输入

```

7
5 4 1 2 3 6 4

```

样例 1 输出

```

2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.3 | 总 3/526 1的个数 ( PID 1222 )

出处：2016级河南工大新生周赛（四）（C，CID 1009）| 训练定位：入门 | 入门语法、输入输出与格式

标签：二进制、位运算、进制转换 | 限制：1.000 S / 128 MB | 历史统计：202/353 ( 57.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组测试输出占一行，输出M的二进制表示中1的个数

输入要点：一个整数M(0<=M<=10000)

输出要点：每组测试输出占一行，输出M的二进制表示中1的个数

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：反复取最低二进制位 `M&1` 并把 M 右移，累加得到 1 的个数。C++ 也提供 `\_\_builtin\_popcount`，但手写循环更适合初学者理解十进制 转二进制的“除 2 取余”过程。M=0 时循环不执行，答案自然为 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\log M)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     unsigned int value;
5 |     cin >> value;
6 |     int ones = 0;
7 |     while (value > 0) {
8 |         ones += value & 1U;
9 |         value >>= 1;
10 |    }
11 |    cout << ones << '\n';
12 |    return 0;
13 | }
```

## 公开样例

样例 1 输入

4

样例 1 输出

1

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.4 | 总 4/526 大学生活 ( PID 1309 )

出处：2017级河南工大新生周赛（一）（B，CID 1028）| 训练定位：入门 | 入门语法、输入输出与格式

标签：条件分支、浮点累加、边界判断 | 限制：1.000 S / 128 MB | 历史统计：144/257 ( 56.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，重修科目的数量 $num1$ （假设小明补考肯定过），补考科目的数量 $num2$ ，补考和重修科目的总学分 $total$ 。 $num1, num2$ 为整数， $total$ 保留1位小数。

输入要点：第一行一个整数  $n$  ( $0 < n < 12$ )

接下来 $n$ 行数据，每一行两个数 $s, a$ 。

$s$  是该学科的学分, $s$ 可能为一位小数 ( $0 < s < 10$ )。

$a$  是小明该学科的成绩， $a$  为整数 ( $0 \leq a \leq 100$ )。



输出要点：重修科目的数量num1（假设小明补考肯定过），补考科目的数量num2，补考和重修科目的总学分total。

num1,num2为整数，total保留1位小数。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：成绩小于 30 的科目直接重修；30 到 59 分需要补考；60 分及以上 通过。扫描每科时分别增加重修/补考门数，并把所有低于 60 分科目的 学分加入 total。最后总学分固定一位小数。边界 30 属于补考，60 已通过。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     int retake = 0, makeup = 0;
9 |     double sum = 0.0;
10 |    for (int i = 0; i < n; ++i) {
11 |        double credit;
12 |        int score;
13 |        cin >> credit >> score;
14 |        if (score < 30) {
15 |            ++retake;
16 |            sum += credit;
17 |        } else if (score < 60) {
18 |            ++makeup;
19 |            sum += credit;
20 |        }
21 |    }
22 |    cout << retake << ' ' << makeup << ' ' << fixed << setprecision(1) << sum << '\n';
23 |    return 0;
24 | }
```

## 公开样例

样例 1 输入

```
5
1.5 0
2.0 29
3.0 30
4.5 59
2.0 60
```

样例 1 输出

```
2 2 11.0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.5 | 总 5/526 Choice学姐买糖果 I ( PID 1314 )

出处：2017级河南工大新生周赛（二）（A，CID 1029）| 训练定位：入门 | 入门语法、输入输出与格式

标签：数组扫描、最值、并列规则 | 限制：1.000 S / 128 MB | 历史统计：186/385 ( 48.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，第一行输出最贵那种糖果的编号和其价格，用空格隔开。第二行输出最便宜那种糖果的编号和其价格，用空格隔开。（如果最贵或者最便宜的糖果种类有多种，则按最小的那个编号输出）

输入要点：第一行输入一个整数  $n$  ( $1 \leq n \leq 1e5$ )，表示有  $n$  种糖果。

接下来一行  $n$  个空格分隔的整数  $p_i$  ( $p_i \leq 1e9$ )，表示第  $i$  种糖果的价格。

输出要点：第一行输出最贵那种糖果的编号和其价格，用空格隔开。

第二行输出最便宜那种糖果的编号和其价格，用空格隔开。

（如果最贵或者最便宜的糖果种类有多种，则按最小的那个编号输出）

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：单次扫描同时维护当前最高价、最低价及其 1 起编号。只有遇到严格更大 或 严格更小时才更新，这样价格并列时会自然保留最先出现、也就是编号 最小的种类。最后按题目要求先输出最贵，再输出最便宜。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 | ios::sync_with_stdio(false);
5 | cin.tie(nullptr);
6 | int n;
7 | cin >> n;
8 | long long mx = LLONG_MIN, mn = LLONG_MAX;
9 | int mxId = -1, mnId = -1;
10 | for (int index = 1; index <= n; ++index) {
11 |     long long price;
12 |     cin >> price;
13 |     if (price > mx) {
14 |         mx = price;
15 |         mxId = index;
16 |     }
17 |     if (price < mn) {
18 |         mn = price;
19 |         mnId = index;
20 |     }
21 | }
22 | cout << mxId << ' ' << mx << '\n';
23 | cout << mnId << ' ' << mn << '\n';
24 | return 0;
25 | }

```

## 公开样例

样例 1 输入

```

5
100 200 300 400 500

```

样例 1 输出

```

5 500
1 100

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.6 | 总 6/526 友好的第一题 ( PID 1382 )

出处：2018级河南工大新生周赛 ( 一 ) @徐向阳专场 ( A , CID 1040 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：434/902 ( 48.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，上述信息。

输入要点：无

输出要点：上述信息。

题面提示：样例输出可以复制粘贴！

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：友好的第一题 思路：打印字符的时候，尽量复制粘贴，这样既快，又不容易出错。参考代码
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

### 复杂度

O(1) 时间、O(1) 空间

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("学号、姓名、刷题数\n");
4 |     printf("201516030101、haut_cds、565\n");
5 |     printf("201516070302、*萌新瑟瑟发抖、488\n");
6 |     printf("201516030102、haut_张宇、433\n");
7 |     printf("201516010517、haut赵建伟、375\n");
8 |     printf("201516020517、Haut_袁珂晨、353\n");
9 |     printf("201516010101、haut_曾伟津、313\n");
10 |    printf("201516070106、Haut 胡文博、255\n");
11 |    printf("201516010421、haut.计科鱼坤、248\n");
12 |    printf("201516030122、x哈、245\n");
13 |    printf("201516920207、haut-SE-陈益旭、241");
14 |    return 0;
15 | }
```

### 公开样例

样例 1 输入

无

样例 1 输出

学号、姓名、刷题数  
201516030101、haut\_cds、565  
201516070302、\*萌新瑟瑟发抖、488  
201516030102、haut\_张宇、433  
201516010517、haut赵建伟、375  
201516020517、Haut\_袁珂晨、353  
201516010101、haut\_曾伟津、313  
201516070106、Haut 胡文博、255  
201516010421、haut.计科鱼坤、248  
201516030122、x哈、245  
201516920207、haut-SE-陈益旭、241

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.7 | 总 7/526 午饭问题 ( 二 ) ( PID 1385 )

出处：2018级河南工大新生周赛 ( 一 ) @徐向阳专场 ( D , CID 1040 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：136/242 ( 56.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对每行输入，输出其对应总挑选的方法，单独占一行。

输入要点：第一行一个正整数  $N$  (  $0 < N \leq 36$  )。

接下来  $N$  行数据，每行三个正整数  $s, h, z$  ( 以空格隔开，且  $1 < s < 7$  ,  $1 < h < 4$  ,  $1 < z < 3$  )。

输出要点：对每行输入，输出其对应总挑选的方法，单独占一行。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：午饭问题 ( 二 ) 思路：仔细读题之后，你会说最终结果和“菜品、主食、饮料”分别有多少种，这两件事没有半毛钱关系，好吧，确实是这样，最终结果只取决于“菜品、主食、饮料”总共有多少种。“菜品、主食、饮料”总共有  $kinds$  种，最终结果也就是：由二项式定理定理可知：所以最终结果为： $2$  的  $kinds$  次方 ( 即  $pow(2, kinds)$  )。参考代码
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     int i, n, s, h, z, sum;
5 |     scanf("%d", &n);
6 |     for (i = 0; i < n; i++) {
7 |         scanf("%d %d %d", &s, &h, &z);
8 |         printf("%d\n", (int)pow(2, (s + h + z)));
9 |     }
10 | }
```

### 公开样例

样例 1 输入

```
2
1 1 1
1 2 2
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.8 | 总 8/526 战舰 ( PID 1430 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( C , CID 1047 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：组合数学、排列组合、计数 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每个样例输出上色的方案数。(数据保证：计算过程产生的中间值，和计算的结果  $< \text{long long}$ )

输入要点：第一行输入一个样例数  $t(1 \leq t \leq 20)$

对于每个样例输入三个样例： $n, m, k$  ( $0 < n < 20, k < m < 20, k \leq (n/2)$ )

输出要点：对于每个样例输出上色的方案数。(数据保证：计算过程产生的中间值，和计算的结果  $< \text{long long}$ )

题面提示： $A(m, n) = (n!)/((n-m)!)$

$C(m, n) = A(m, n)/(m!)$

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：先从  $\text{floor}(n/2)$  艘偶数编号战舰中选择  $k$  艘，方案数  $C(\text{floor}(n/2), k)$ 。这  $k$  艘各用互不相同的颜色，剩余所有战舰再共用一种不同颜色，共需从  $m$  色中有序选择  $k+1$  色，即  $P(m, k+1)$ 。两部分相乘。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(k)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long combination(int n, int k) {
4 |     k = min(k, n - k);
5 |     long long res = 1;
6 |     for (int i = 1; i <= k; ++i)
7 |         res = res * (n - k + i) / i;
8 |     return res;
9 | }
10 | int main() {
11 |     int T;
12 |     cin >> T;
13 |     while (T--) {
14 |         int n, m, k;
15 |         cin >> n >> m >> k;
16 |         long long colors = 1;
17 |         for (int i = 0; i <= k; ++i)
18 |             colors *= m - i;
19 |         cout << combination(n / 2, k) * colors << '\n';
20 |     }
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

```
3
3 2 1
3 2 0
15 5 3
```

样例 1 输出

```
2
2
4200
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.9 | 总 9/526 小青的传统艺能 ( PID 1530 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（C，CID 1060）| 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：408/705 ( 57.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，%%%zcs

输入要点：此题无任何输入

输出要点：%%%zcs

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。

- 按题面给定关系完成常数计算：无输入，百分号在 `cout` 字符串中无需转义，按要求输出三个百分号 和 `zcs`。
- 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "%%zcs\n";
5 |     return 0;
6 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.10 | 总 10/526 卷卷签到成功 ( PID 1573 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（A，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（A，CID 1069） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：1517/1998 ( 75.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：卷卷的目标是AK这场比赛，卷卷首先要完成最简单的题（签到题）：请输出“>>签到成功<<”（不含引号）。

输入要点：本题无输入

输出要点：一行，“>>签到成功<<”（不含引号）

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：卷卷签到成功直接输出即可 `#include<stdio.h>`



3. 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf(">>签到成功<<");
4 |     return 0;
5 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.11 | 总 11/526 卷卷的空间限制 ( PID 1574 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（B，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（B，CID 1069）| 训练定位：入门 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：1317/1740 ( 75.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行一个整数，表示  $n$  MB能存放的`int`类型整数的个数

输入要点：一行一个非负整数 $n$ ，（ $n$ 小于1000）

输出要点：一行一个整数，表示  $n$  MB能存放的`int`类型整数的个数

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3. 核心处理采用：卷卷的空间限制由题目条件可知：1MB可以储存 $1024 * 1024 / 4 = 262144$ 个int类型整数。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     // 直接计算并输出即可
6 |     printf("%d", n * (1024 * 1024 / 4));
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

128

样例 1 输出

33554432

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.12 | 总 12/526 卷卷今天休息(?) ( PID 1576 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（D，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（D，CID 1069） | 训练定位：入门 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：898/1747 ( 51.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个整数表示聚聚能通过几道题

输入要点：第一行：时:分:秒（表示题目中“现在”的时间X时Y分Z秒）

第二行：时:分:秒（表示卷卷开始学习的时间A时B分C秒）

第三行：聚聚通过一道题需要的秒数m

输出要点：一行，一个整数表示聚聚能通过几道题

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：卷卷今天休息(?)总做题数 = 总时间 / 做一道题需要的时间
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a, b, c, x, y, z, m;
4 |     scanf("%d:%d:%d", &x, &y, &z);
5 |     scanf("%d:%d:%d", &a, &b, &c);
6 |     scanf("%d", &m);
7 |     // 计算总秒数
8 |     int sum = (a - x) * 3600 + (b - y) * 60 + (c - z);
9 |     // 答案=总秒数 / 通过一道题需要的秒数
10 |    int ans = sum / m;
11 |    printf("%d", ans);
12 |    return 0;
13 | }
```

## 公开样例

样例 1 输入

```
00:00:00
00:00:15
2
```

样例 1 输出

```
7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.13 | 总 13/526 牛牛的战斗力的 (PID 1594)

出处：2021级HAUT新生周赛（二）@李亚凯专场（C，CID 1070） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：266/481 ( 55.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出两位驾驶员战斗力之和的最低值，保留3位小数。

输入要点：一个正整数n，代表两位驾驶员战斗力之积。（ $n \leq 106$ ）。

输出要点：输出两位驾驶员战斗力之和的最低值，保留3位小数。

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：牛牛的战斗力的由高中的基本不等式 当 $a=b$ 时，取等时和最小，可用math.h里面的sqrt对n开方。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     double cnt = sqrt(n);
7 |     cnt *= 2;
8 |     printf("%.3lf", cnt);
9 | }
```

## 公开样例

样例 1 输入

1

样例 1 输出

2.000

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

1.14 | 总 14/526 我爱编程，编程使我快乐。（PID 1817）

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（A，CID 1091）；2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（A，CID 1086） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：772/1288（59.9%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：刚刚接触编程的你发现编程能给你带来许多乐趣，为了表达自己的快乐，请输出“我爱编程，编程使我快乐。”（只需要输出“”内的字符）。

输入要点：无

输出要点：我爱编程，编程使我快乐。

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：无输入固定输出题，按题面要求输出指定句子即可。
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "我爱编程，编程使我快乐。" << '\n';
5 |     return 0;
6 | }
```

公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

1.15 | 总 15/526 小明的人生难题 ( PID 1825 )

出处：2023级HAUT新生周赛（二）@曹瑞峰专场（A，CID 1087） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：197/243 ( 81.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一行，直接输出答案即可。

输入要点：本题无输入

输出要点：一行，直接输出答案即可。

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：小明的人生难题签到题，直接输出即可。 `#include<stdio.h>`
3. 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("%d", 520 * 1314 + 1024 / 2);
4 |     return 0;
5 | }
```

公开样例

|         |
|---------|
| 样例 1 输入 |
| 无       |
| 样例 1 输出 |
| 直接输出答案  |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.16 | 总 16/526 奖学金 ( PID 1839 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（C，CID 1088）| 训练定位：入门 | 入门语法、输入输出与格式

标签：条件分支、基础实现 | 限制：1.000 S / 128 MB | 历史统计：192/267 ( 71.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：卷卷的本学期的4科期末考试成绩出来了，奖学金的申请是看两学期的成绩，并且规定无挂科以及补考记录，现就这一条规定，卷卷想看一下自己目前是否有可能申请。

输入要点：输入四个正整数  $a, b, c, d$  ( $0 \leq a, b, c, d \leq 100$ ) 分别代表卷卷的高数，线代，英语，c语言程序设计四科成绩

输出要点：如果不能申请，输出"Next refueling!",如果有可能申请,输出"It's possible!"。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：申请条件中的这一关要求没有挂科记录，所以四科都必须至少 60 分。只要发现任意一科低于 60 就输出不能申请的提示。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, b, c, d;
5 |     cin >> a >> b >> c >> d;
6 |     if (min({a, b, c, d}) < 60)
7 |         cout << "Next refueling!\n";
8 |     else
9 |         cout << "It's possible!\n";
10 |    return 0;
11 | }
```

### 公开样例

样例 1 输入

95 97 80 90

样例 1 输出

It's possible!

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.17 | 总 17/526 依稀记得，那是一个烟雨蒙蒙的清晨 ( PID 1846 )

出处：2023级HAUT新生周赛（四）@牛浩然专场（B，CID 1089）| 训练定位：入门 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：148/195 ( 75.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数 $h$ ，表示屋檐到地面的高度。

输入要点：一个整数 $t$ 代表水滴落到地上的时间。（ $0 < t \leq 100$ ）。

输出要点：一个整数 $h$ ，表示屋檐到地面的高度。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：B.依稀记得，那是一个烟雨蒙蒙的早晨。签到题，根据自由落体公式输出 $5*t*t$ 即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int t;
4 |     scanf("%d", &t);
```



```
5 |     printf("%d\n", 5 * t * t);
6 |     return 0;
7 | }
```

### 公开样例

样例 1 输入

1

样例 1 输出

5

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.18 | 总 18/526 ACMer ( PID 1872 )

出处：河南工大2024新生周赛 ( 3 ) ——命题人：张宏泽 ( B , CID 1093 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：条件分支 | 限制：1.000 S / 128 MB | 历史统计：171/340 ( 50.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：本次比赛为参赛选手们提供了n个问题，对于每个问题，我们都知道是谁确定了解决方案。请帮助朋友们求出他们会写出解决方案的问题数量。

输入要点：第一行包含一个整数 n ( 1≤n≤1000 ) -- 竞赛中的问题数。

随后的n行分别包含三个整数，每个整数要么是 0 要么是 1。如果该行中的第一个数字等于 1，则表示 Zhz 确定问题的答案，否则表示他不确定。第二个数字表示Ly对解法的看法，第三个数字表示Lts的看法。一行中的数字之间用空格隔开。

输出要点：打印一个整数--朋友们在比赛中写出解决方案的问题数量。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：ACMer 判断每一组中1的个数是否大于2，如果大于2，则数量加一
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为 O(n)；若循环嵌套或迭代范围不同，应按代码重新核算

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, x, cnt = 0;
3 | void solve() {
4 |     int sum = 0;
5 |     for (int i = 0; i < 3; i++) // 读入三个数字
6 |     {
7 |         scanf("%d", &x);
8 |         sum += x;
9 |     }
10 |    if (sum >= 2)
11 |        cnt++;
12 | }
13 | int main() {
14 |     int T;
15 |     scanf("%d", &T);
16 |     while (T--)
17 |         solve();
18 |     printf("%d\n", cnt);
19 |     return 0;
20 | }
```

公开样例

样例 1 输入

```
3
1 1 0
1 1 1
1 0 0
```

样例 1 输出

```
2
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

1.19 | 总 19/526 Doki Doki Literature Club! ( PID 1879 )

出处：河南工大2024新生周赛（4）——命题人：马贺（A，CID 1094） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：217/258（84.1%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，Just Monika!

输入要点：无

输出要点：Just Monika!

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241118/20241118230846\\_50675.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241118/20241118230846_50675.png)]

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

思路推导

- 1. 确认题目没有输入，程序无需声明或读取测试数据。
- 2. 按题面给定关系完成常数计算：Doki Doki Literature Club! 直接输出就可以了。
- 3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 4. 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("Just Monika!");
4 |     return 0;
5 | }
```

公开样例

|              |
|--------------|
| 样例 1 输入      |
| 无            |
| 样例 1 输出      |
| Just Monika! |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

1.20 | 总 20/526 光 ( PID 1899 )

出处：河南工大2024新生周赛 ( 6 ) ——命题人：魏方 ( A , CID 1097 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：306/511 ( 59.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一段话

输入要点：本题无输入

输出要点：输出一段话

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

### 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：本题没有输入，严格按题面输出指定中文句子及英文感叹号。
3. 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "宏观困难但局部有光!\n";
5 |     return 0;
6 | }
```

### 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.21 | 总 21/526 计算罚时 ( PID 1923 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( B , CID 1100 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：基础运算、罚时规则 | 限制：1.000 S / 128 MB | 历史统计：458/617 ( 74.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，表示 `sy` 的 A 题罚时 ( 单位：分钟 )。

输入要点：输入一行，包含两个整数 `a` 和 `b`，用空格隔开：

-  
第一个整数 `a` ( $0 \leq a < 180$ ) 表示通过时间 ( 单位：分钟 )；

第二个整数  $b$  ( $0 \leq b < 1000$ ) 表示错误提交次数。

输出要点：输出一个整数，表示 sy 的 A 题罚时（单位：分钟）。

题面提示：sy 于比赛开始后 60 分钟通过 A 题，此前错误提交 3 次。

罚时 =  $60 + 3 \times 20 = 120$  分钟。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：ICPC 常见罚时为通过时刻加上此前错误提交数乘 20，因此直接计算  $a+20b$ 。使用 long long 可避免以后扩大数据范围时溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long acMin, wa;
5 |     cin >> acMin >> wa;
6 |     cout << acMin + 20 * wa << '\n';
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

60 3

样例 1 输出

120

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.22 | 总 22/526 诚信参赛 ( PID 1924 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( A , CID 1100 ) | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出、格式 | 限制：1.000 S / 128 MB | 历史统计：475/787 ( 60.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，sheng si kan dan, cheng xin xun lian!

输入要点：无

输出要点：sheng si kan dan, cheng xin xun lian!

### 零基础先修

- 固定输出题不需要读入；把目标字符串逐字写入 cout，并在本地比较标点与换行。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：这是固定输出题，不读输入，逐字输出题目要求的句子。注意大小写、空格、逗号和感叹号都属于答案的一部分。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "sheng si kan dan, cheng xin xun lian!\n";
5 |     return 0;
6 | }
```

### 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

# 1.23 | 总 23/526 编程霸总强制爱 ( PID 1981 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（C，CID 1110） | 训练定位：入门 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：242/314（77.1%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：如果你不喜欢编程，请输出“I love coding”。(输出不带引号)

输入要点：无

输出要点：按题目要求输出

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：编程霸总强制爱 按题意输出即可 `#include<bits/stdc++.h> using namespace std; int main() { cout<<"I love coding"; return 0; } D`
- 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "I love coding";
5 |     return 0;
6 | }
```

## 公开样例

样例 1 输入

无

样例 1 输出

无

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.24 | 总 24/526 四则运算 ( PID 1037 )

出处：2016级河南工大新生周赛（一）（C，CID 1006） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：条件分支、浮点输出、表达式 | 限制：1.000 S / 30 MB | 历史统计：222/681 ( 32.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给你一个简单的四则运算表达式，包含两个实数和一个运算符，请编程计算出结果

输入要点：表达式的格式为：s1 op s2，s1和s2是两个实数，op表示的是运算符(+,-,\*,/)，也可能是其他字符

输出要点：如果运算符合法，输出表达式的值；若运算符不合法或进行除法运算时除数是0，则输出"Wrong input"。最后结果小数点后保留两位。

题面提示：除数是0,用 $|s2| < 1e-10$ (即 $10$ 的 $-10$ 次方)判断

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 浮点题按误差要求输出足够位数，通常使用 fixed 与 setprecision。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：读取两个实数和运算符，分别处理加、减、乘、除。只有除法时才需要检查除数是否接近0；不能把“第二个操作数为0”误判为所有运算都非法。合法结果固定保留两位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 不要用字符串精确比较小数量例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double left, right;
5 |     char op;
6 |     cin >> left >> op >> right;
7 |     cout << fixed << setprecision(2);
8 |     if (op == '+')
9 |         cout << left + right << '\n';
10 |     else if (op == '-')
```



```

11 |         cout << left - right << '\n';
12 |     else if (op == '*')
13 |         cout << left * right << '\n';
14 |     else if (op == '/' && fabs(right) >= 1e-10) {
15 |         cout << left / right << '\n';
16 |     } else {
17 |         cout << "Wrong input\n";
18 |     }
19 |     return 0;
20 | }

```

## 公开样例

样例 1 输入

1.0 + 1.0

样例 1 输出

2.00

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.25 | 总 25/526 平均数计算 (PID 1200)

出处：2016级河南工大新生周赛（一）（B，CID 1006）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：平均数、最值、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：138/303 (45.5%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出题目中要求的平均数（保留1位小数）

**输入要点：**输入5个正整数，保证他们的和小于500

**输出要点：**输出题目中要求的平均数（保留1位小数）

## 零基础先修

- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：题目规定的平均数要去掉一个最大值和一个最小值，再对剩余三个数求平均。读五个整数时累计 `sum` 并维护 `minimum`、`maximum`，答案为 `(sum-minimum-maximum)/3.0`，固定一位小数。若最值重复，也只各去掉一个元素。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int sum = 0, mn = INT_MAX, mx = INT_MIN;
5 |     for (int i = 0; i < 5; ++i) {
6 |         int value;
7 |         cin >> value;
8 |         sum += value;
9 |         mn = min(mn, value);
10 |        mx = max(mx, value);
11 |    }
12 |    cout << fixed << setprecision(1) << (sum - mn - mx) / 3.0 << '\n';
13 |    return 0;
14 | }
```

## 公开样例

样例 1 输入

1 5 5 7 8

样例 1 输出

5.7

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.26 | 总 26/526 lolizlm的数字 ( PID 1207 )

出处：2016级河南工大新生周赛（二）（C，CID 1007）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：数组扫描、整除、最值 | 限制：1.000 S / 128 MB | 历史统计：100/242 ( 41.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：lolizlm学姐给你用随机函数生成了 $N$ 个非负整数字，但是学姐想知道它们加起来能不能被 $N$ 整除，如果能，请输出这个 $N$ 个数字中最大的数字，否则输出他们中最小的数字。

输入要点：第一行一个数字 $N$

第二行 $N$ 个数字分别用空格隔开

输出要点：输出一个数字

题面提示：各个点1s

$N \leq 1000$  每个输入的数字小于10000

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：扫描  $N$  个数时同时维护总和、最小值和最大值。若总和能被  $N$  整除，输出最大值；否则输出最小值。总和上界约  $10^7$ ，`int` 也够，但使用 `long long` 更稳妥。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     long long sum = 0;
9 |     int mn = INT_MAX, mx = INT_MIN;
10 |     for (int i = 0; i < n; ++i) {
11 |         int value;
12 |         cin >> value;
13 |         sum += value;
14 |         mn = min(mn, value);
15 |         mx = max(mx, value);
16 |     }
17 |     cout << (sum % n == 0 ? mx : mn) << '\n';
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
5
1 2 3 4 5
```

样例 1 输出

```
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.27 | 总 27/526 蚂蚁 ( PID 1215 )

出处：2016级河南工大新生周赛 ( 三 ) ( B , CID 1008 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：等价变换、蚂蚁问题、最值 | 限制：1.000 S / 128 MB | 历史统计：51/146 ( 34.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：n只蚂蚁以每秒1cm的速度在长为Lcm的竿子上爬行。当蚂蚁爬到竿子的端点时就会掉落。由于竿子太细，两只蚂蚁相遇时，它们不能交错通过，只能各自反向爬回去。对于每只蚂蚁，我们知道它距离竿子左端的距离x，但不知道它当前的朝向。请计算所有蚂蚁落下竿子可能需要的最长时间。

输入要点：输入第一行为两个正整数L和n，分别表示竿子长度和蚂蚁的数量。 $1 \leq L, n \leq 10^6$ 。

第二行输入n个数，表示n个蚂蚁距离竿子左端的距离

输出要点：输出一个正整数，表示所需最长时间。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：两只蚂蚁相遇后同时反向，与它们保持方向、彼此穿过的几何轨迹完全 等价，只是交换了身份。因此可把每只蚂蚁独立看待。为了让全部掉落时间 尽量长，每只蚂蚁可朝更远端爬，位置 x 的最远距离是  $\max(x, L-x)$ ；所有蚂蚁中的最大值就是最终时间。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long len;
7 |     int n;
8 |     cin >> len >> n;
9 |     long long ans = 0;
10 |    for (int i = 0; i < n; ++i) {
11 |        long long pos;
12 |        cin >> pos;
13 |        ans = max(ans, max(pos, len - pos));
14 |    }
15 |    cout << ans << '\n';
16 |    return 0;
17 | }
```

### 公开样例

样例 1 输入

10 3  
2 6 7

样例 1 输出

8

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.28 | 总 28/526 求和 ( PID 1219 )

出处：2016级河南工大新生周赛 ( 三 ) ( F , CID 1008 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：数位、循环、条件判断 | 限制：1.000 S / 128 MB | 历史统计：140/289 ( 48.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给出两个数a和b，判断它们的各位数之和是否相同。

输入要点：输入一行a，b。0<=a,b<=10^9。

输出要点：如果相同输出YES，否则输出NO。

### 零基础先修

- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：一个非负整数的数位和可通过反复取 `value%10` 后整除 10 得到。分别计算 a、b；0 的循环不执行，数位和仍正确为 0。比较两和，相同 输出 YES。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

O(log a+log b) 时间、O(1) 空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long digitSum(long long value) {
4 |     long long sum = 0;
5 |     while (value > 0) {
```

```

6 |         sum += value % 10;
7 |         value /= 10;
8 |     }
9 |     return sum;
10| }
11| int main() {
12|     long long a, b;
13|     cin >> a >> b;
14|     cout << (digitSum(a) == digitSum(b) ? "YES" : "NO") << '\n';
15|     return 0;
16| }

```

## 公开样例

样例 1 输入

12345 543210

样例 1 输出

YES

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.29 | 总 29/526 等式判断 ( PID 1229 )

出处：2016级河南工大新生周赛（五）（D，CID 1010）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：字符串流、表达式解析、除零判断 | 限制：1.000 S / 128 MB | 历史统计：80/337 ( 23.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：请判断等式是否正确。如果正确，输出YES，否则输出NO。

输入要点：输入第一行为一个数T，表示有T组数据。

每组数据输入一个等式，等式中只包含不大于10000的非负整数。

输出要点：对于每组数据输出YES或NO。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：用字符串流按`左操作数 运算符 右操作数 = 结果`读取。针对 +、-、\*、/ 计算左侧；除法时若除数为 0，等式直接错误。题目使用整数，所以`/`按 C/C++ 整数除法判断。最后比较计算值与等号右侧。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

每组  $O(|s|)$  时间、 $O(|s|)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string equation;
10 |         cin >> equation;
11 |         stringstream input(equation);
12 |         long long a, b, expected;
13 |         char op, equalSign;
14 |         input >> a >> op >> b >> equalSign >> expected;
15 |         bool valid = true;
16 |         long long actual = 0;
17 |         if (op == '+')
18 |             actual = a + b;
19 |         else if (op == '-')
20 |             actual = a - b;
21 |         else if (op == '*')
22 |             actual = a * b;
23 |         else if (op == '/') {
24 |             if (b == 0)
25 |                 valid = false;
26 |             else
27 |                 actual = a / b;
28 |         } else {
29 |             valid = false;
30 |         }
31 |         cout << (valid && actual == expected ? "YES" : "NO") << '\n';
32 |     }
33 |     return 0;
34 | }
```

## 公开样例

样例 1 输入

```
2
1+2=3
1*8=9
```

样例 1 输出

```
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.30 | 总 30/526 手机剩余电量 ( PID 1232 )

出处：2016级河南工大新生周赛 ( 五 ) ( G , CID 1010 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：字符画、整除取余、格式化输出 | 限制：1.000 S / 128 MB | 历史统计：32/89 ( 36.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每组实例，输出手机剩余电量。

输入要点：输入包含多组测试实例，每个实例为一个正整数 $n$ ，以 $n$ 等于-1结束。 $n \leq 100$

输出要点：对于每组实例，输出手机剩余电量。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：电池内部是  $10 \times 10$  字符格，共表示 100% 电量。从最底行、每行从 右向左填充 `!`：对从上到下的第  $row$  行，其对应电量区间可直接用全局 单元编号判断。更直观的实现是先计算  $fullRows = n / 10$  与  $remainder = n \% 10$ ：底部  $fullRows$  行填满，再上一行右侧填  $remainder$  个，其余为空。每个实例紧接输出，不插空行。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(100)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int battery;
7 |     while (cin >> battery && battery != -1) {
8 |         cout << "*-----*\n";
9 |         int fullRows = battery / 10;
10 |         int rem = battery % 10;
11 |         for (int rowFromTop = 0; rowFromTop < 10; ++rowFromTop) {
12 |             int row = 9 - rowFromTop;
13 |             int filled = 0;
14 |             if (row < fullRows)
15 |                 filled = 10;
16 |             else if (row == fullRows)
17 |                 filled = rem;
18 |             cout << '|' << string(10 - filled, ' ') << string(filled, '!') << "|\n";
19 |         }
20 |         cout << "*-----*\n";
21 |     }
22 |     return 0;
23 | }
```



## 公开样例

### 样例 1 输入

11  
66  
-1

### 样例 1 输出

[illegible]

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

### 1.31 | 总 31/526 又是郊游 (PID 1242)

出处：2016级河南工大新生周赛（七）（F，CID 1014） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：追及问题、相对速度、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：66/187 (35.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，输出狗跑的路径，结果保留小数点后两位。

输入要点：第一行输入一个整数N，表示测试数据的组数(N<100)

每组测试数据占一行，是四个正整数，分别为M,X,Y,Z (数据保证 $X<Y<Z$ )

输出要点：输出狗跑的路径，结果保留小数点后两位。

零基础先修

- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

- 核心处理采用：zy 出发时 cds 已领先  $M \cdot X$  米。两人的相对速度是  $Y - X$ ，所以追上所需时间为  $M \cdot X / (Y - X)$  分钟。狗在这整个时间内始终以  $Z$  米/分钟 奔跑，往返细节不会改变总路程，故答案为  $Z$  乘追赶时间。用 `double` 并保留两位小数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         double minutes, slowSpeed, fastSpeed, dogSpeed;
10 |         cin >> minutes >> slowSpeed >> fastSpeed >> dogSpeed;
11 |         double catchTime = minutes * slowSpeed / (fastSpeed - slowSpeed);
12 |         cout << fixed << setprecision(2) << catchTime * dogSpeed << '\n';
13 |     }
14 |     return 0;
15 | }
```

## 公开样例

样例 1 输入

```
1
5 10 15 20
```

样例 1 输出

```
200.00
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.32 | 总 32/526 Simple学长的史诗级难题 ( PID 1251 )

出处：2016级河南工大新生周赛总决赛 ( B, CID 1016 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：字符串计数、多数判断、题意辨析 | 限制：1.000 S / 128 MB | 历史统计：144/482 ( 29.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出决定吃什么的那个人，晴天或者Simple。

输入要点：输入一个T，表示有T组数据

每组数据一行字符，只包含Q，S，且一定为奇数个。Q表示这一局晴天赢，S表示Simple赢。

字符串长度小于10000

输出要点：输出决定吃什么的那个人，晴天或者Simple。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：字符串长度为奇数，因此Q与S的胜局数不会相等。Q表示晴天赢，S表示Simple赢，而题目约定“输的人决定吃什么”。所以 Q 更多时 Simple 输并输出 Simple；否则晴天输，输出中文“晴天”。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(|s|)$  时间、 $O(1)$  额外空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string res;
10 |        cin >> res;
11 |        int qWins = count(res.begin(), res.end(), 'Q');
12 |        int sWins = (int)res.size() - qWins;
13 |        cout << (qWins > sWins ? "Simple" : "晴天") << '\n';
14 |    }
15 |    return 0;
16 | }
```

## 公开样例

样例 1 输入

```
2
QQS
SQS
```

样例 1 输出

```
Simple
晴天
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)

## 1.33 | 总 33/526 第一个程序 ( PID 1308 )

出处：2017级河南工大新生周赛 ( 一 ) ( A , CID 1028 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出、转义字符、C 语言程序结构 | 限制：1.000 S / 128 MB | 历史统计：196/760 ( 25.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，`#include <stdio.h> int main() { printf("Hello world\n"); return 0; }`

输入要点：无

输出要点：`#include <stdio.h>`

```
int main()

{

printf("Hello world\n");

return 0;

}
```

### 零基础先修

- 固定输出题不需要读入；把目标字符串逐字写入 `cout`，并在本地比较标点与换行。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：程序没有输入，要求输出的内容本身是一段 C 源代码。关键在两层转义：要输出双引号，需要在当前 C++ 字符串里写 ``"`；目标代码中的 ``n`` 两个字符，需要在当前字符串里写 ``\\n``。为让零基础读者看清每一行，这里逐行输出目标文本。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "#include <stdio.h>\n";
5 |     cout << "int main()\n";
6 |     cout << "{\n";
7 |     cout << "printf(\"Hello world\\n\\n\");\n";
8 |     cout << "return 0;\n";
9 |     cout << "}\n";
10 |     return 0;
11 | }

```

## 公开样例

样例 1 输入

无

样例 1 输出

```

#include <stdio.h>
int main()
{
printf("Hello world\n");
return 0;
}

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.34 | 总 34/526 午饭问题 ( PID 1311 )

出处：2017级河南工大新生周赛（一）（D，CID 1028）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：计数数组、众数、并列规则 | 限制：1.000 S / 128 MB | 历史统计：27/58 ( 46.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：W：“给你一些数字，问你出现次数最多的数字，如果出现次数最多的数字有多个请输出最小的一个，不过最多可以有1000000个数字，这些数字都是整数，保证每个数字 大于等于 -5，小于等于5。”

输入要点：第一行一个数字 n (0< n <=1000000)。

接下来 n 个数字 d (-5 <= d <= 5),digit为整数。

输出要点：一个整数 ans (-5 <= ans <= 5)。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：数据值只可能是 -5..5，用长度 11 的计数数组即可，值 x 存在下标 x+5。读完后从 -5 到 5 递增扫描，只在次数严格更大时更新答案；次数相同则保留先扫描到的更小数字。这样既节省内存又自然处理并列规则。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     array<int, 11> freq{};
9 |     for (int i = 0; i < n; ++i) {
10 |         int value;
11 |         cin >> value;
12 |         ++freq[value + 5];
13 |     }
14 |     int ans = -5;
15 |     for (int value = -4; value <= 5; ++value)
16 |         if (freq[value + 5] > freq[ans + 5])
17 |             ans = value;
18 |     cout << ans << '\n';
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
5
-5
-5
0
5
5
```

样例 1 输出

```
-5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.35 | 总 35/526 Fate/Accepted ( PID 1325 )

出处：2017级河南工大新生周赛（三）（E，CID 1030）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：枚举因数、圆周数组、正多边形 | 限制：1.000 S / 128 MB | 历史统计：8/33 ( 24.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：那么问题来了，地上的宝石有一部分被圣杯污染了，其中的魔力变成了负数，但是我们召唤出的英灵强弱，和地上所有宝石的魔力值的和有关，魔力值的和越大，召唤出的英灵越强，所以我们可能要拿走一部分被污染的宝石。但是要求还留在地上的宝石必须仍然构成一个正多边形，才能召唤出英灵。并且地上剩余宝石的能量和越大，召唤出的英灵越强。因为可能性实在太多了现在就请你写个程序来帮助ma.....

输入要点：第一行，一个整数 $n$ ，代表刚开始是地上宝石的数量

数据范围：( $3 \leq n \leq 20000$ )

第二行， $n$ 个整数 $a[1], a[2], a[3], \dots, a[n]$ ，表示每个宝石蕴含的能量

数据范围：( $-1000 \leq a[i] \leq 1000$ )

输出要点：一个数字，召唤出最强英灵时，地上宝石的能量和

题面提示：样例：最优时，地上剩余宝石能量分别为，2，4，5，3。构成一个正4边形，能量和为14

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 3 核心处理采用：原 $n$ 个点等距分布在圆上。保留 $k$ 个点仍构成正多边形，当且仅当 $k \geq 3$ 且 $k$ 整除 $n$ ；相邻保留点的原编号间隔为 $\text{step} = n/k$ 。  
对每个合法 $k$ ，枚举起始余数 $\text{offset} = 0.. \text{step} - 1$ ，所选点就是 $\text{offset}, \text{offset} + \text{step}, \dots$ ，求其能量和并取全局最大。样例选择偶数编号四点。每个 $k$ 的全部 $\text{offset}$ 合计恰扫描 $n$ 个元素。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n \cdot \tau(n))$  时间、 $O(n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> energy(n);
9 |     for (long long& x : energy)
10 |         cin >> x;
11 |     long long ans = LLONG_MIN;
12 |     for (int vertices = 3; vertices <= n; ++vertices) {
13 |         if (n % vertices != 0)
14 |             continue;
15 |         int step = n / vertices;
16 |         for (int offset = 0; offset < step; ++offset) {
17 |             long long sum = 0;
18 |             for (int index = offset; index < n; index += step)
19 |                 sum += energy[index];
20 |             ans = max(ans, sum);
21 |         }
22 |     }
```

```
23 |     cout << ans << '\n';
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

```
8
1 2 -3 4 -5 5 2 3
```

样例 1 输出

```
14
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 1.36 | 总 36/526 辛普森积分与球面公式 ( PID 1366 )

出处：2017级河南工大新生周赛总决赛（重现）（F，CID 1049）；2017级河南工大新生周赛总决赛（重现）（F，CID 1039）；2017级河南工大新生周赛总决赛（F，CID 1038） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出、题意辨析 | 限制：1.000 S / 128 MB | 历史统计：126/550 ( 22.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，图片中的代码

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：图片中的代码

题面提示：看题目中第一句话！

## 零基础先修

- 固定输出题不需要读入；把目标字符串逐字写入 cout，并在本地比较标点与换行。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：第一句已经说明“题目是假的”。输出要求不是运行图片里的 C 程序，而是原样输出六个汉字“图片中的代码”。这是固定出题，图片、辛普森积分和球面公式都是干扰信息；不要多加引号或解释。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度



O(1) 时间、O(1) 空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "图片中的代码\n";
5 |     return 0;
6 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 1.37 | 总 37/526 大学生活 (二) (PID 1383)

出处：2018级河南工大新生周赛 (一) @徐向阳专场 (B, CID 1040) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：153/618 (24.8%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组数据输出占一行，两个数 F (挂科数)，G (绩点)。F 为一个正整数；G 为一个浮点数，保留两位小数。

输入要点：第一行一个正整数 N (0<N<=10)，接下来 N 组测试数据。

每组测试数据：第一行为一个正整数 M (0<M<=20)。接下来 M 行数据，每行一个正整数 S (0<=S<=100) 代表一科的分数。

输出要点：每组数据输出占一行，两个数 F (挂科数)，G (绩点)。

F 为一个正整数；G 为一个浮点数，保留两位小数。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：大学生活 (二) 思路：二重循环+判断语句，需要注意的是绩点的精度问题，保留两位小数，所以存储绩点要用 double 型。  
参考代码：ProblemC
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, m;
4 |     int i, j, f;
5 |     int sum;
6 |     double s, g;
7 |     scanf("%d", &n);
8 |     for (i = 0; i < n; i++) {
9 |         scanf("%d", &m);
10 |        f = sum = 0;
11 |        for (j = 0; j < m; j++) {
12 |            scanf("%lf", &s);
13 |            if (s < 60) {
14 |                f++;
15 |            } else if (s >= 60) {
16 |                sum = sum + (1 + (s - 60.0) / 10.0);
17 |            }
18 |        }
19 |        g = 1.0 * sum / m;
20 |        printf("%d %.2lf\n", f, g);
21 |    }
22 | }
```

## 公开样例

样例 1 输入

```
2
3
80
70
50
3
100
100
100
```

样例 1 输出

```
1 1.67
0 5.00
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

# 1.38 | 总 38/526 签签签签签到题 ( PID 1394 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( G , CID 1041 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：399/665 ( 60.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，n对匹配的括号，左右对称

输入要点：一个正整数n，代表输出的括号对数 (0 < n <1000)

输出要点：n对匹配的括号，左右对称

题面提示：仔细看看输出样例好不好：)

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：仔细看看输出样例好不好：)
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为 O(n)；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int i, n;
4 |     scanf("%d", &n);
5 |     for (i = 0; i < n; i++) {
6 |         printf("(");
7 |     }
8 |     printf(" ");
9 |     for (i = 0; i < n; i++) {
10 |         printf(")");
11 |     }
12 |     return 0;
13 | }
```

## 公开样例

样例 1 输入

4

样例 1 输出

(((()))

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.39 | 总 39/526 选数字 ( PID 1441 )

出处：2018级河南工大新生周赛 ( 七 ) @牛寅旭专场 ( E , CID 1046 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：回溯、组合枚举、字典序 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，按字典序输出所有可能的排列。若无法选取，则输出-1.

输入要点：第一行有两个数 $n$ 和 $m$ 。(  $0 < n, k < 16$  )

第二行为 $n$ 个数。

输出要点：按字典序输出所有可能的排列。

若无法选取，则输出-1.

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：样例表明题目所说的“排列”实际要求从  $n$  个数中选  $m$  个、每组内部保持升序的组合。先排序，再用递归从  $last+1$  开始选择一个下标，生成顺序天然就是字典序； $m > n$  时输出 -1。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(C(n,m) \cdot m)$  输出时间、 $O(n)$  递归空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int n, m;
4 | vector<long long> number, chosen;
5 | void search(int start) {
6 |     if ((int)chosen.size() == m) {
```

```

7 |         for (int i = 0; i < m; ++i) {
8 |             if (i)
9 |                 cout << ' ';
10 |             cout << chosen[i];
11 |         }
12 |         cout << '\n';
13 |         return;
14 |     }
15 |     int stillNeeded = m - chosen.size();
16 |     for (int i = start; i + stillNeeded <= n; ++i) {
17 |         chosen.push_back(number[i]);
18 |         search(i + 1);
19 |         chosen.pop_back();
20 |     }
21 | }
22 | int main() {
23 |     cin >> n >> m;
24 |     number.resize(n);
25 |     for (long long& x : number)
26 |         cin >> x;
27 |     if (m > n) {
28 |         cout << -1 << '\n';
29 |         return 0;
30 |     }
31 |     sort(number.begin(), number.end());
32 |     search(0);
33 |     return 0;
34 | }

```

## 公开样例

### 样例 1 输入

```

4 3
1 2 3 4

```

### 样例 1 输出

```

1 2 3
1 2 4
1 3 4
2 3 4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.40 | 总 40/526 cjk想打篮球 ( PID 1479 )

出处：2019级河南工大新生周赛（一）@陶福专场（A，CID 1053）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、转义字符、阅读理解 | 限制：1.000 S / 128 MB | 历史统计：279/714 ( 39.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：字段0：\$GPRMC，语句ID，表明该语句为Recommended Minimum Specific GPS/TRANSIT Data ( RMC ) 推荐最小定位信息

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

题面提示：注意题意

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：题面大段背景是干扰信息，样例输出要求的实际内容是：一个开双引号、26 个小写字母、一个反斜杠和一个闭双引号。C++ 中双引号与反斜杠 都要转义。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "\"abcdefghijklmnopqrstuvwxyz\\\\"<\/pre>

```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.41 | 总 41/526 AC小将 ( PID 1491 )

出处：2019级河南工大新生周赛 ( 三 ) @邓祎培专场 ( F , CID 1055 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现、循环、整数运算 | 限制：1.000 S / 128 MB | 历史统计：254/372 ( 68.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出较大的那四个数的和或者是较大的四个数的积

输入要点：第一行输入南宫问天说出的四个数，数据之间用空格隔开

第二行输入刁民说出的四个数，数据之间用空格隔开

输出要点：输出较大的那四个数的和或者是较大的四个数的积

题面提示：输入的四个数都小于等于20

若两数相等则任意输出其一即可

## 零基础先修

- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：第一行四个数按南宫问天的规则求乘积，第二行四个数按刁民的规则求和，输出二者较大值。最大乘积  $20^4$ ，int 也可容纳，使用 long long 更稳妥。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long product = 1, sum = 0, x;
5 |     for (int i = 0; i < 4; ++i) {
6 |         cin >> x;
7 |         product *= x;
8 |     }
9 |     for (int i = 0; i < 4; ++i) {
10 |         cin >> x;
11 |         sum += x;
12 |     }
13 |     cout << max(product, sum) << '\n';
14 |     return 0;
15 | }
```

## 公开样例

样例 1 输入

```
1 2 3 4
1 1 1 1
```

样例 1 输出

```
24
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.42 | 总 42/526 棋盘上的米粒 ( PID 1514 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( A , CID 1058 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：76/276 ( 27.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组测试数据包含一行，每行只有一个整数  $k$

输入要点：多组测试数据，每组只有一个整数  $n$  ( $1 \leq n \leq 1018$ )

测试数据以 EOF 结束

输出要点：每组测试数据包含一行，每行只有一个整数  $k$

题面提示：样例解释：

第一组数据：

一共 1 粒米

第一个格子放 1 粒米，米粒用完

$k = 1$

第二组数据：

一共 4 粒米

第一个格子放 1 粒米，此时还剩下 3 粒米

第二个格子放 2 粒米，此时还剩下 1 粒米

第三个格子放 1 粒米，米粒用完

$k = 3$

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：棋盘上的米粒模拟签到题 从第一个格子开始放米粒，如果米粒数量为正数，就代表当前位置的格子是可以放上米粒的，放不放满无所谓，维护一个 代表放到第几个格子。kkk C++版代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。



## C++17 参考实现

```
1 | #include <iostream>
2 | using namespace std;
3 | typedef long long LL;
4 | LL n;
5 | int sol() {
6 |     int k = 0;
7 |     LL tem = 1;
8 |     while (n > 0) {
9 |         k++;
10 |        n -= tem;
11 |        tem *= 2;
12 |    }
13 |    return k;
14 | }
15 | int main() {
16 |     while (cin >> n) {
17 |         cout << sol() << "\n";
18 |     }
19 |     return 0;
20 | }
```

### 公开样例

样例 1 输入

```
1
4
```

样例 1 输出

```
1
3
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.43 | 总 43/526 阿正的忐忑不安 ( PID 1539 )

出处：2020级HAUT新生周赛（二）@杨哲专场（A，CID 1061） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：条件分支、固定格式 | 限制：1.000 S / 128 MB | 历史统计：245/639 ( 38.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在给出阿正同学语数英综合四科的成绩和阿正心仪院校的录取分数线，如果阿正的总分达到（大于等于）了院校录取分数线，请输出文本“Blessings for your college career!”；否则请输出文本“Blessings for your "fourth grade" in high school!”。

输入要点：第一行四个正整数  $a, b, c, d$  ( $0 \leq a, b, c \leq 150$ ； $0 \leq d \leq 300$ ) 分别代表阿正的语文，数学，英语，综合四科成绩；

第二行一个正整数  $N$  ( $0 \leq N \leq 750$ ) 代表阿正心仪院校的录取分数线。

输出要点：如果阿正的总分大于等于心仪院校的录取分数线，那么输出“Blessings for your college career!”，否则输出“Blessings for your "fourth grade" in high school!”。

题面提示：在样例中，阿正的总分为  $116+82+121+0=319$ ，小于525；

所以输出“Blessings for your "fourth grade" in high school!”。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

思路推导

- 1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 3. 核心处理采用：四科相加后与录取线比较。未达到时输出文本内部还含一对双引号，C++ 字符串中要写成转义的 \。
- 4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(1) 时间、O(1) 空间

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, b, c, d, line;
5 |     cin >> a >> b >> c >> d >> line;
6 |     if (a + b + c + d >= line) {
7 |         cout << "Blessings for your college career!\n";
8 |     } else {
9 |         cout << "Blessings for your \"fourth grade\" in high school!\n";
10 |     }
11 |     return 0;
12 | }
```

公开样例

样例 1 输入

116 82 121 0  
525

样例 1 输出

Blessings for your "fourth grade" in high school!

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

1.44 | 总 44/526 聚聚的幸运数字 ( PID 1578 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（F，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（F，CID 1069） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：条件分支 | 限制：1.000 S / 128 MB | 历史统计：720/1560 ( 46.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，如果可以则输出“YES”，否则输出“NO”。（不含引号）

输入要点：一行，一个整数x ( 保证x不等于5141919 )

输出要点：如果可以则输出“YES”，否则输出“NO”。( 不含引号 )

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：聚聚的幸运数字每一个1都可以选择删或不删，共有种，排除自身后满足条件的只有7种情况 $2^3-1=7$ 即：5499、51499、54199、54919、514199、514919、541919
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     if (n == 5499 || n == 51499 || n == 54199 || n == 54919 || n == 514199 || n == 514919 ||
6 |         n == 541919)
7 |         printf("YES");
8 |     else
9 |         printf("NO");
10 |    return 0;
11 | }
```

## 公开样例

样例 1 输入

51499

样例 1 输出

YES

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.45 | 总 45/526 牛牛的签到题 ( PID 1579 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（A，CID 1070）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：375/1343 ( 27.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，This is an output question. Please output it"%%%\n\n\n".

输入要点：无输入

输出要点：This is an output question. Please output it"%%%\n\n\n".

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

### 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：牛牛的签到题格式控制符里输入%%即可输出一个%；输出\\即可输出一个\；printf("\\")即可输出双引号。
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

### 复杂度

O(1) 时间、O(1) 空间

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("This is an output question. Please output it\"%%%\n\n\n.");
4 | }
```

### 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.46 | 总 46/526 牛牛的循环次数 ( PID 1585 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（G，CID 1070）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：60/118 ( 50.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每一组数据，在一行中输出OP的操作次数。

输入要点：第一行输入1个整数T( $1 \leq T \leq 100$ ),代表有T组数据

接下来T行，每行一个正整数n( $1 \leq n \leq 1000$ )

输出要点：对于每一组数据，在一行中输出OP的操作次数。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：牛牛的循环次数//比赛时，评测姬出了一些问题，导致这题没卡到人// 这题自己跑循环,正常情况下1s肯定超时。用排列组合的方式思考，用n=5举个例子: 有编号为1~5的5个小球，取3个小球,则有 $A(3,5)=60$ 种情况，但每一组都出现了6次，如1, 2, 3这种组合有 $A(3,3)=6$ 种情况，但只有(1,2,3)这种是满足要求的，因此可以得到公式 $A(3,n)/A(3,3)$ 。也可以跑一些数据慢慢找规律.....。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, T;
4 |     scanf("%d", &T);
5 |     while (T--) {
6 |         scanf("%d", &n);
7 |         int cnt = 0;
8 |         cnt = n * (n - 1) * (n - 2) / 6;
9 |         printf("%d\n", cnt);
10 |    }
11 | }
```

### 公开样例

样例 1 输入

```
4
3
4
5
6
```

样例 1 输出

```
1
4
10
20
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.47 | 总 47/526 纸牌游戏 ( PID 1590 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（B，CID 1071）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：86/224 ( 38.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果新生可以赢得s学长，输出“%%%”；如果新生输了，输出“tql”；

输入要点：一个整数 $n(1 \leq n \leq 13)$ ，代表每次可以取走的最多纸牌数。

输出要点：如果新生可以赢得s学长，输出“%%%”；

如果新生输了，输出“tql”；

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：纸牌游戏 算是一个简单博弈吧；例有两种花色的纸牌：新生第一次取走了第一种纸牌 $m$ 张，则S学长取走另一种花色 $m$ 张；此后不管新生取走哪一种花色的纸牌，也不论取走多少张，S学长都会取走另一种花色的纸牌同样多张；这样，最后一张纸牌一定是S学长取走的。拓展到四种花色，也是同样道理。这样的话，无论如何，结果都是S学长必胜。
- 最后按题目要求输出；提交前检查最小值、最大值、空/元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 核对读入顺序、变量类型和输出格式。

- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     printf("tql\n");
6 |     return 0;
7 | }
```

### 公开样例

|         |
|---------|
| 样例 1 输入 |
| 10      |
| 样例 1 输出 |
| tql     |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.48 | 总 48/526 签到时间 ( PID 1591 )

出处：2021级HAUT新生周赛 ( 三 ) @李晨曦专场 ( C , CID 1071 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：124/262 ( 47.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出好方案与坏方案的差。

输入要点：一个整数 $n$ ，代表参赛人数。(  $n \leq 1e6$  )

输出要点：输出好方案与坏方案的差。

题面提示：好方案为：[1, 2]、[1, 3]、[1, 4]、[2, 3]、[2, 4]、[3, 4]、[1, 2, 3, 4]，有7个；

坏方案为：[1]、[2]、[3]、[4]、[1, 2, 3]、[1, 2, 4]、[1, 3, 4]、[2, 3, 4]，有8个；

所以好方案与坏方案的差为-1；

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：签到时间 这是一道简单的高中数学题；已知：二项式展开式的系数，奇数项之和等于偶数项之和。选偶数个人，是：选奇数个人，是： 则结果为：二项式展开式得偶数项多一个；则最后结果为；如果，想不起来这个定理的话，建议写几个试试，找规律。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     printf("-1\n");
6 |     return 0;
7 | }
```

### 公开样例

样例 1 输入

4

样例 1 输出

-1

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.49 | 总 49/526 游戏时间 ( PID 1593 )

出处：2021级HAUT新生周赛 ( 三 ) @李晨曦专场 ( E , CID 1071 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：181/567 ( 31.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，代表A同学达到自己的目标分数所需要的天数。

输入要点：一行给出一个非负整数n，代表你的目标分数，一个非负整数k，代表A同学第一天分数的增长值。(  $n < 1e6$  ,  $k < 1e6$  )

输出要点：一个整数，代表A同学达到自己的目标分数所需要的天数。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。



2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：游戏时间 本场最签到题；每次循环当没有达到目标的话，天数加加，分数增长值加加。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | int main() {
4 |     int n, k;
5 |     scanf("%d%d", &n, &k);
6 |     int ans = 0;
7 |     int day = 0;
8 |     while (ans < n)
9 |         ans += k, k++, day++;
10 |    printf("%d\n", day);
11 |    return 0;
12 | }
```

## 公开样例

样例 1 输入

100 10

样例 1 输出

8

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 1.50 | 总 50/526 博学的学长 ( PID 1704 )

出处：2021级HAUT新生周赛（四）@王一杰专场（E，CID 1072）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、字符串 | 限制：1.000 S / 128 MB | 历史统计：325/554 ( 58.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行字符串。

输入要点：无

输出要点：一行字符串。

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：博学的学长典型签到题，直接输出。当然，你也可以去查ASCII码表。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | int main() {
6 |     printf("%c%c%c%c%c%c%c", 67, 72, 70, 89, 89, 68, 83);
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

无

样例 1 输出

无

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.51 | 总 51/526 图书馆信息管理系统 ( PID 1710 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（B，CID 1073）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、条件分支、数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：42/185 ( 22.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，若要判断的书的代号是所属类别的书，则输出YES,否则输出NO。

输入要点：第一行输入书籍的类别的代号，第二行输入要判断的书的代号。两个都是长度不大于20的非空字符串且均由小写字母构成。

输出要点：若要判断的书的代号是所属类别的书，则输出YES,否则输出NO。

题面提示：第二个子串lwyj匹配到第一个子串的lwyj则匹配成功。

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：图书馆信息管理系统思路：这道题目的意思是要判断第一行的字符串能不能通过删除一些子串得到第二个字符串，本质理论是判断第二个字符串的所有元素是否可以按顺序出现在第一个串中即可。具体实现：循环+双指针维持即可。参考代码如下
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

### 复杂度

通常为  $O(n)$

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | char a[30], b[30];
4 | int main() {
5 |     gets(a);
6 |     gets(b);
7 |     if (strlen(b) > strlen(a)) // b的长度比a的长直接输出NO
8 |     {
9 |         puts("NO");
10 |        return 0;
11 |    } else {
12 |        int j = 0;
13 |        for (int i = 0; i < strlen(a); i++) // 判断b字符串中的字符是否按顺序出现在a中
14 |        {
15 |            if (a[i] == b[j]) {
```

```

16 |         j++;
17 |     }
18 | }
19 | if (j == strlen(b)) // 符合
20 | {
21 |     puts("YES");
22 | } else // 不符合
23 | {
24 |     puts("NO");
25 | }
26 | }
27 | }

```

## 公开样例

样例 1 输入

```
lwyjyyds
lwyj
```

样例 1 输出

```
YES
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.52 | 总 52/526 小C的快乐寒假 ( PID 1751 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（F，CID 1077）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：103/183 ( 56.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，见样例，输出小C用C语言、C++、C#、JAVA、Python、Go、PHP这七种语言输出的“寒假快乐！！！”语句

输入要点：无

输出要点：见样例，输出小C用C语言、C++、C#、JAVA、Python、Go、PHP这七种语言输出的“寒假快乐！！！”语句

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：小C的快乐寒假签到题，按着输出就行，注意一下“”的输出即可 上代码
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("printf(\"Happy Winter Holiday!!!\\n\\n\");\n");
4 |     printf("cout<<\"Happy Winter Holiday!!!\\n<<endl;\n");
5 |     printf("Console.WriteLine(\"Happy Winter Holiday!!!\\n\");\n");
6 |     printf("System.out.println(\"Happy Winter Holiday!!!\\n\");\n");
7 |     printf("print(\"Happy Winter Holiday!!!\\n\");\n");
8 |     printf("fmt.Print(\"Happy Winter Holiday!!!\\n\");\n");
9 |     printf("Happy Winter Holiday!!!\\n");
10 |     return 0;
11 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.53 | 总 53/526 真 签到题 ( PID 1778 )

出处：2022级HAUT新生周赛（四）@张子豪专场（E，CID 1082）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出、转义字符 | 限制：1.000 S / 128 MB | 历史统计：162/246 ( 65.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行字符串

输入要点：无

输出要点：一行字符串

## 零基础先修

- 固定输出题不需要读入；把目标字符串逐字写入 cout，并在本地比较标点与换行。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：没有输入，原样输出包含双引号的字符串 `"a % b = c"`。在 C++ 字符串字面量中双引号要写成 `""`；百分号在 ``cout`` 中无需转义（若使用 `printf` 才要写成 `“%%”`）。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "\\a % b = c\\n";
5 |     return 0;
6 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.54 | 总 54/526 调查员三宝之朋友 ( PID 1780 )

出处：2022级HAUT新生周赛（四）@张子豪专场（A，CID 1082）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：27/58 ( 46.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行，献祭队友的方法数量

输入要点：一行，正整数m。  $10 \leq m \leq 2000000$

输出要点：一行，献祭队友的方法数量

题面提示：### 样例说明：

当  $m = 10000$  时，调查员献祭队友的方法有4种：

- 1.从第18号队友献祭到第142号 (  $\sum_{i=18}^{142} i = 10000$  )
- 2.从第297号队友献祭到第328号 (  $\sum_{i=297}^{328} i = 10000$  )
- 3.从第388号队友献祭到第412号 (  $\sum_{i=388}^{412} i = 10000$  )
- 4.从第1998号队友献祭到第2002号 (  $\sum_{i=1998}^{2002} i = 10000$  )

( 只献祭第m号队友不符合题目要求 )

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：调查员三宝之朋友： 题意：求有多少个连续的数段，这些数段中的全部数之和为m 双指针做法：用i, j代表区间的左右端点，sum代表[i,j]这个区间所有数之和 当sum小于目标值M时，将右端点右移（j++），sum会变大 当sum大于目标值M时，将左端点右移（i++），sum会变小 在双指针移动的过程中，如果有sum==M的情况ans+1。因为两个指针都是单调向右移动，也只扫一遍，可以证明时间复杂度为O(n) 左端点大于m/2时即可停止，因为只要长度为2的连续序列和就一定大于m
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(n)

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int m, ans = 0, sum = 3;
4 |     scanf("%d", &m);
5 |     for (int i = 1, j = 2; i <= m / 2;) {
6 |         if (sum == m) {
7 |             ans++;
8 |             sum -= i;
9 |             i++;
10 |        } else if (sum < m) {
11 |            j++;
12 |            sum += j;
13 |        } else {
14 |            sum -= i;
15 |            i++;
16 |        }
17 |    }
18 |    printf("%d", ans);
19 |    return 0;
20 | }
```

## 公开样例

样例 1 输入

10000

样例 1 输出

4

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

1.55 | 总 55/526 达拉崩吧 ( PID 1791 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（F，CID 1083）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：48/76 ( 63.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个整数占一行，勇者名字出现的次数。分别输出勇者、公主、巨龙和城市的名字，每个名字占一行。

输入要点：无

输出要点：一个整数占一行，勇者名字出现的次数。

分别输出勇者、公主、巨龙和城市的名字，每个名字占一行。

题面提示：有样例，但我不想给(´·ω·´)

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：达拉崩吧斑得贝迪卜多比鲁翁\n米娅莫拉苏娜丹妮谢莉红\n昆图库塔卡提考特苏瓦西拉松\n蒙达鲁克疏斯伯古比奇巴勒城"); return 0; } G
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     printf(
5 |         "0\n达拉崩吧斑得贝迪卜多比鲁翁\n米娅莫拉苏娜丹妮谢莉红\n昆图库塔卡提考特苏瓦西拉松\n蒙达鲁克疏斯伯古比奇巴勒城");
6 |     return 0;
7 | }
```

公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

训练动作



- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.56 | 总 56/526 寝室的宝箱（简单版）（PID 1821）

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（F，CID 1091）；2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（E，CID 1086） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：0.200 S / 128 MB | 历史统计：512/1071（47.8%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，计算结果。

输入要点：分别给出 正整数 $x$ 、运算符 $ch$ 、正整数 $y$ 。

（ $x, y \leq 9, ch \in \{ '+', '-' \}$ ）

输出要点：计算结果。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：寝室的宝箱（简单版）简单的加减计算，对于输入的数字和字符调用相关的式子即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int x, y;
4 |     char ch;
5 |     scanf("%d %c %d", &x, &ch, &y);
6 |     printf("%d", (ch == '+' ? x + y : x - y));
7 |     return 0;
```

## 公开样例

样例 1 输入

```
1 + 1
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.57 | 总 57/526 长跑评级 ( PID 1868 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( D , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( E , CID 1092 )  
| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：条件分支、时间换算、哨兵输入 | 限制：1.000 S / 128 MB | 历史统计：348/1248 ( 27.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，对每个同学的成绩，在单独的一行中输出对应的评级。输出完所有同学的评级后，在一个新行中输出不及格人数。

**输入要点：**每行都包含一个同学的成绩，第一个数  $m$  ( $2 \leq m \leq 10$ ) 为分钟，第二个数  $s$  ( $0 \leq s \leq 59$ ) 为秒。

单独的一行 -1 代表输入完毕。

本题保证输入的成绩数量不超过 100。

**输出要点：**对每个同学的成绩，在单独的一行中输出对应的评级。

输出完所有同学的评级后，在一个新行中输出不及格人数。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把分秒换成总秒，依次按  $\leq 207$ 、 $\leq 222$ 、 $\leq 272$  和更慢四档输出；最后一档同时累计不及格人数。输入以单独的 -1 结束。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int minute, failCount = 0;
7 |     while (cin >> minute && minute != -1) {
8 |         int second;
9 |         cin >> second;
10 |         int total = minute * 60 + second;
11 |         if (total <= 207)
12 |             cout << "Outstanding\n";
13 |         else if (total <= 222)
14 |             cout << "Good\n";
15 |         else if (total <= 272)
16 |             cout << "Pass\n";
17 |         else {
18 |             cout << "Fail\n";
19 |             ++failCount;
20 |         }
21 |     }
22 |     cout << failCount << '\n';
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
3 27
3 31
4 5
4 35
4 56
-1
```

样例 1 输出

```
Outstanding
Good
Pass
Fail
Fail
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

# 1.58 | 总 58/526 踱步 ( PID 1869 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( C , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( F , CID 1092 )  
| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：坐标模拟、方向、累加 | 限制：1.000 S / 128 MB | 历史统计：391/987 ( 39.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：请统计他共走了多少米，并记录最后停在了哪个位置。

输入要点：第一行一个整数  $n$ ，代表这个同学要走动的次数。

接下来  $n$  行，每行有两个整数  $direction$  和  $length$ 。代表向某个方向走一段距离，长度为  $length$ 。

题目保证  $1 \leq n \leq 100$ ； $direction$  只可能取 1, 2, 3, 4；所有  $length$  之和不超过  $10^4$ 。

输出要点：输出共两行。

第一行两个整数  $x$  和  $y$ ，表示最后停在了某个坐标。

第二行输出走过的所有  $length$  之和。

题面提示：对样例的解释：

起点为 (0, 0)

向东走，距离为 1。走到了 (1, 0)。

向西走，距离为 2。走到了 (-1, 0)。

向南走，距离为 3。走到了 (-1, -3)。

向北走，距离为 2。走到了 (-1, -1)。

向东走，距离为 3。走到了 (2, -1)。

最后停在了 (2, -1)，输出。

走过的距离之和为  $1 + 2 + 3 + 2 + 3 = 11$ ，输出。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：从 (0,0) 出发，方向 1/2/3/4 分别让  $x$  加、 $x$  减、 $y$  减、 $y$  加  $length$ ；同时把每段  $length$  加入总路程。最后分两行输出坐标与路程。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(n) 时间、O(1) 空间

易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, x = 0, y = 0, total = 0;
7 |     cin >> n;
8 |     while (n--) {
9 |         int dir, len;
10 |         cin >> dir >> len;
11 |         total += len;
12 |         if (dir == 1)
13 |             x += len;
14 |         else if (dir == 2)
15 |             x -= len;
16 |         else if (dir == 3)
17 |             y -= len;
18 |         else
19 |             y += len;
20 |     }
21 |     cout << x << ' ' << y << '\n' << total << '\n';
22 |     return 0;
23 | }
```

公开样例

样例 1 输入

```
5
1 1
2 2
3 3
4 2
1 3
```

样例 1 输出

```
2 -1
11
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

1.59 | 总 59/526 Stock Exchange(Easy Version) ( PID 1873 )

出处：河南工大2024新生周赛（3）——命题人：张宏泽（C，CID 1093） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：112/272 ( 41.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，表示zhz可以从这笔交易中获取的最大利润。如果不能获取任何利润，返回 0。

输入要点：共有两行：

(1) 第一行包含一个整数  $n$  ( $1 \leq n \leq 2000$ )；

(2) 第二行包含  $n$  个正整数，其中第  $i$  个正整数表示的是该股票在第  $i$  天的价值（保证在  $\text{int}$  范围内）；

输出要点：输出一个整数，表示  $\text{zhz}$  可以从这笔交易中获取的最大利润。如果不能获取任何利润，返回 0。

题面提示：第三天股票价值为 1 的时候买入，在第六天股票价值为 6 的时候卖出。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：Stock Exchange(Easy Version) 观察题目我们发现数据的范围非常的小， $1 \leq n \leq 2000$ ，因此我们可以采取时间复杂度为  $O(n^2)$  的算法，也就是暴力的枚举每一个区间，寻找哪一对天数的股票价值的差最大。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n^2)$

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <iostream>
2 | const int N = 2010;
3 | int n, a[N];
4 | int max(int a, int b) // 比较 a b 大小并返回最大值的函数
5 | {
6 |     return a >= b ? a : b;
7 | }
8 | void solve() {
9 |     scanf("%d", &n);
10 |    for (int i = 1; i <= n; i++)
11 |        scanf("%d", &a[i]);
12 |    int ans = 0;
13 |    for (int i = 1; i <= n; i++)
14 |        for (int j = i + 1; j <= n; j++) {
15 |            ans = max(ans, a[j] - a[i]);
16 |        }
17 |    printf("%d\n", ans);
18 | }
19 | int main() {
20 |     int T = 1;
21 |     while (T--)
22 |         solve();
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
7
3 7 1 2 5 6 4
```

5

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.60 | 总 60/526 Stock Exchange(Hard Version) ( PID 1874 )

出处：河南工大2024新生周赛（3）——命题人：张宏泽（D，CID 1093）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：66/165（40.0%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，表示zhz可以从这笔交易中获取的最大利润。如果不能获取任何利润，返回 0。

输入要点：共有两行：

(1) 第一行包含一个整数  $n$ （ $1 \leq n \leq 10^6$ ）；

(2) 第二行包含  $n$  个正整数，其中第  $i$  个正整数表示的是该股票在第  $i$  天的价值（保证在 `int` 范围内）；

输出要点：输出一个整数，表示zhz可以从这笔交易中获取的最大利润。如果不能获取任何利润，返回 0。

题面提示：第三天股票价值为1的时候买入，在第六天股票价值为6的时候卖出。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：Stock Exchange(Hard Version) 观察题目我们发现  $n$  的数据范围非常的大，因此我们不能再使用时间复杂度为  $O(n^2)$  的算法，而只能使用时间复杂度为  $O(n)$  或者  $O(n \log n)$  的算法。而对于本题我们想到贪心，在一次遍历的过程中动态地修改最小值，并计算最大值。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n^2)$

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <iostream>
2 | const int N = 1e6 + 10;
3 | int n, a[N];
4 | int min(int a, int b) {
5 |     return a >= b ? b : a;
6 | }
7 | int max(int a, int b) {
8 |     return a >= b ? a : b;
9 | }
10 | int main() {
11 |     scanf("%d", &n);
12 |     int Min = 1e9, ans = 0;
13 |     for (int i = 1; i <= n; i++) {
14 |         scanf("%d", &a[i]);
15 |         Min = min(Min, a[i]);
16 |         ans = max(ans, a[i] - Min);
17 |     }
18 |     printf("%d", ans);
19 |     return 0;
20 | }
```

### 公开样例

样例 1 输入

```
7
3 7 1 2 5 6 4
```

样例 1 输出

```
5
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.61 | 总 61/526 ^ ( PID 1875 )

出处：河南工大2024新生周赛（3）——命题人：张宏泽（E，CID 1093） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、转义字符 | 限制：1.000 S / 128 MB | 历史统计：169/380 ( 44.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，^

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：^

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

### 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：反斜杠在 C++ 字符串中要写成转义形式 \\，最终输出才是一个 反斜杠。



- 3. 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
- 4. 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "/\\n";
5 |     return 0;
6 | }
```

公开样例

样例 1 输入

无

样例 1 输出

Λ

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

1.62 | 总 62/526 Haut校草 ( PID 1883 )

出处：河南工大2024新生周赛 ( 4 ) ——命题人：马贺 ( E , CID 1094 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：最值、数学推导、数组扫描 | 限制：1.000 S / 128 MB | 历史统计：55/123 ( 44.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：给定数组  $a$ ，求它任意重新排序后能够取得的最大得分。

输入要点：第一行  $n$  (  $1 \leq n \leq 1000$  )；第二行  $n$  个整数  $a_i$  (  $1 \leq a_i \leq 1000$  )。

输出要点：一个整数  $w$ ，代表数组  $a$  在任意排序后，得分的最大值。

题面提示：题面含一张示意图片，完整定义与图片请在原题页查看。

零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：最终全体达到同一身高时，最矮与最高之间每增加 1 个单位，需要让  $n-1$  个人发生一次相对变化，因此答案为  $(\max-\min)\times(n-1)$ 。扫描数组 只保留最大值和最小值即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  额外空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     long long mn = LLONG_MAX, mx = LLONG_MIN;
9 |     for (int i = 0; i < n; ++i) {
10 |         long long height;
11 |         cin >> height;
12 |         mn = min(mn, height);
13 |         mx = max(mx, height);
14 |     }
15 |     cout << (mx - mn) * (n - 1LL) << '\n';
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
5
1 1 1 2 2
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 1.63 | 总 63/526 密码锁的旋转密码 ( PID 1926 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( G , CID 1104 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：217/400 ( 54.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：输入目标密码 ( 4 位整数，如 1234 )，计算从初始 0000 到目标密码的最少操作次数。

输入要点：输入一个4 位整数。

输出要点：输出一个整数a，表示最少操作次数

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：密码锁的旋转密码 循环判断每一位，取操作最小值累加。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int a;
5 |     scanf("%d", &a);
6 |     int ans = 0;
7 |     while (a) {
8 |         int b = a % 10;
9 |         if (b < 10 - b)
10 |             ans += b;
11 |         else
12 |             ans += 10 - b;
13 |         a /= 10;
14 |     }
15 |     printf("%d", ans);
16 |     return 0;
17 | }
```

### 公开样例

样例 1 输入

1234

样例 1 输出

10

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

1.64 | 总 64/526 机器人的寻宝路径 ( PID 1929 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( F , CID 1104 ) | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、字符串、数学 | 限制：1.000 S / 128 MB | 历史统计：73/214 ( 34.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，两个整数  $x, y$ ，代表最终坐标。

输入要点：一行长度为 $n$ 的字符串（仅含 U/D/L/R）， $0 \leq n \leq 20$ 。

输出要点：两个整数  $x, y$ ，代表最终坐标。

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：机器人的寻宝路径 注意连续两次移动方向相同，第二次移动距离翻倍。
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

通常为  $O(n)$

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int x = 0, y = 0, p = 0;
```

```

5 |     char pre = '\0';
6 |     char c;
7 |     while ((c = getchar()) != '\n') {
8 |         if (c == pre)
9 |             p++;
10 |        else
11 |            p = 0;
12 |        int d = 1 << p;
13 |        switch (c) {
14 |            case 'U':
15 |                y += d;
16 |                break;
17 |            case 'D':
18 |                y -= d;
19 |                break;
20 |            case 'L':
21 |                x -= d;
22 |                break;
23 |            case 'R':
24 |                x += d;
25 |                break;
26 |            default:
27 |                break;
28 |        }
29 |        pre = c;
30 |    }
31 |    printf("%d %d\n", x, y);
32 |    return 0;
33 | }

```

## 公开样例

样例 1 输入

UUR

样例 1 输出

1 3

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.65 | 总 65/526 小唐的工作 ( PID 1934 )

出处：河南工大2025新生周赛（2）——命题人：王伟立（F，CID 1103）| 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式、条件分支 | 限制：1.000 S / 128 MB | 历史统计：291/563 ( 51.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，一个小数，保留18位小数。

**输入要点：**一个整数 $n$ ,表示公爵检查的天数， $n \leq 50$ 。

**输出要点：**一个小数，保留18位小数。

**题面提示：**样例一：第一天 0.5

第二天  $0.5 + 0.25 = 0.75$

第三天  $0.5 + 0.25 + 0.125 = 0.875$

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的工作  $n$  天完成的任务量可以简化为  $1 - \text{pow}(0.5, n+1)$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int n;
5 |     cin >> n;
6 |     double x = pow(0.5, n);
7 |     printf("%.18lf", 1 - x);
8 |     return 0;
9 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

0.875000000000000000

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.66 | 总 66/526 小唐的日历 ( PID 1938 )

出处：河南工大2025新生周赛（2）——命题人：王伟立（B，CID 1103） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：375/852 ( 44.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：小唐很好奇日历上的每个月有多少天，现在给出年份和月份，请输出这月的天数。

输入要点：输入两个正整数，分别表示年份 和月数，以空格隔开。

输出要点：输出一行一个正整数，表示这个月有多少天。

题面提示：不要忘了闰年的二月

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的日历 注意闰年 #include <bits/stdc++.h> #define int long long #define inf 1e18 #define maxn 400005
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

o(false)

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | #define int long long
3 | #define inf 1e18
4 | #define maxn 400005
5 | using namespace std;
6 | int const mod = 998244353;
7 | void solve() {
8 |     int n, m;
9 |     cin >> n >> m;
10 |    int r = (n % 400 == 0) || (n % 4 == 0 && n % 100 != 0);
11 |    // switch针对月份分支，直接输出对应天数
12 |    switch (m) {
13 |        case 1:
14 |        case 3:
15 |        case 5:
16 |        case 7:
17 |        case 8:
18 |        case 10:
19 |        case 12:
20 |            cout << 31;
21 |            break;
22 |        case 4:
23 |        case 6:
24 |        case 9:
25 |        case 11:
26 |            cout << 30;
27 |            break;
```

```
28 |         case 2:
29 |             cout << (r ? 29 : 28);
30 |             break;
31 |         }
32 |         return;
33 |     }
34 | signed main() {
35 |     ios::sync_with_stdio(false);
36 |     cin.tie(0);
37 |     //         int t; cin >> t;
38 |     //         while (t--)
39 |         solve();
40 |     return 0;
41 | }
```

## 公开样例

样例 1 输入

1926 8

样例 1 输出

31

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.67 | 总 67/526 双 11 红包大派送 ( PID 1939 )

出处：河南工大2025新生周赛（3）——命题人：路耀华（C，CID 1104） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：283/714 ( 39.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：为兼顾“手气”与“公平”，红包拆分有两个要求：一是每个红包金额均为正整数，二是最大红包与最小红包的金额差不超过 1，避免出现某一红包金额远高于其他的情况。

输入要点：一行两个整数  $m$  和  $k$  ( $2 \leq k \leq m \leq 1000$ )，分别表示总红包金额和红包个数。

输出要点：一行  $k$  个整数，用空格分隔，按非递减顺序输出每个红包的金额。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：双 11 红包大派送 由于最大红包与最小红包的金额差不超过 1，故输出最多有 2 个不同的数，可以先求平均值（向下取整），再将多余的金额以每个红包一元分配。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。



- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int m, k;
5 |     scanf("%d %d", &m, &k);
6 |     int cur = m / k;
7 |     m %= k;
8 |     for (int i = 1; i <= k; i++) {
9 |         if (i > k - m)
10 |             printf("%d", cur + 1);
11 |         else
12 |             printf("%d", cur);
13 |         if (i < k)
14 |             printf(" ");
15 |     }
16 |     return 0;
17 | }
```

公开样例

样例 1 输入

10 3

样例 1 输出

3 3 4

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

1.68 | 总 68/526 圣诞节前前前夕 ( PID 1947 )

出处：河南工大2025新生周赛（5）——命题人：袁林坤（B，CID 1106） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：297/641（46.3%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，无

输入要点：无

输出要点：无

题面提示：这只是一道签到题

零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：圣诞节前前后夕 签到题，注意int范围不够，开long long 即可，两种解法。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 2e5 + 10;
4 | typedef long long ll;
5 | int main() {
6 |     ll ans = 0;
7 |     for (ll i = 1; i <= 20251125; i++)
8 |         ans += i;
9 |     cout << ans;
10 |    return 0;
11 | }
```

## 公开样例

样例 1 输入

无

样例 1 输出

无

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.69 | 总 69/526 签到题？ ( PID 1980 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（A，CID 1110） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：固定输出与格式 | 限制：1.000 S / 128 MB | 历史统计：218/455 ( 47.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：作为奥尔特加科学院外聘的地球人类计算专家（Human Computational Specialist, HCS），请你计算出本次初始化操作  $\alpha \oplus \beta$  的最终结果值（以OSEU为单位）。你需要确保你的计算过程严格遵循MSEFM模型和泽塔稳定性第一定律，并考虑所有给定的已知条件（尤其是所有干扰因子均为零）

输入要点：无

输出要点：按题目要求计算并输出

题面提示：这真是一道签到题

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：签到题？读题目，注意到 $\alpha$ 和 $\beta$ 的值都为1，现要求二者相加的值（经典线性求和），输出2即可
- 使用 `cout` 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << 2;
5 |     return 0;
6 | }
```

## 公开样例

样例 1 输入

无

样例 1 输出

自行计算并输出结果

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

1.70 | 总 70/526 这是啥杯？ ( PID 1984 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（F，CID 1110） | 训练定位：基础提高 | 入门语法、输入输出与格式

标签：条件分支、状态枚举、固定映射 | 限制：1.000 S / 128 MB | 历史统计：191/410 ( 46.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：在昨日圆车中，我们一般用杯子的级别来形容干员的强度，比如中杯，大杯，超大杯，而干员强度又与许多方面有关，我们不考虑那么多，现在认为干员的强度只与dps，破甲线，总伤三个属性有关。而每一个属性现认为只有高或低之分。现在以dps，破甲线，总伤的顺序输入属性的高低，用1来代表高，用零来代表低。现在我们规定只有一个高属性为中杯干员，有两个高属性为大杯干员，有三个高属性.....

输入要点：输入三个0或1，以空格分开

输出要点：按要求输出，中杯为"medium cup"，大杯为"big cup"，超大杯为"super~ big~ cup~"，如果一个高属性都没有，则输出"zheshishabei?"。

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：三个 0/1 分别表示三个属性是否高，只需统计 1 的个数。0、1、2、3 个高属性分别对应题目规定的四个字符串；标点、空格和波浪线都必须 原样输出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(1) 时间、O(1) 空间

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, b, c;
5 |     cin >> a >> b >> c;
6 |     int highCount = a + b + c;
7 |     if (highCount == 0)
8 |         cout << "zheshishabei?\n";
9 |     else if (highCount == 1)
10 |         cout << "medium cup\n";
11 |     else if (highCount == 2)
12 |         cout << "big cup\n";
13 |     else
14 |         cout << "super~ big~ cup~\n";
```

```
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入

```
1 0 1
```

样例 1 输出

```
big cup
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.71 | 总 71/526 数字游戏 ( PID 1228 )

出处：2016级河南工大新生周赛（五）（C，CID 1010）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：哈希表、频率统计、哨兵输入 | 限制：1.000 S / 128 MB | 历史统计：32/296 ( 10.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：ykc和cds在玩一个数字游戏，ykc会给出n个数，cds需要告诉ykc有多少个数只出现过一次，由于数字太多，cds慌得不行。请你帮助cds快速求出答案。

输入要点：输入包括多组数据。以n等于0结束

每组数据中：

第一行为一个整数n，表示整数的数量。

第二行输入n个整数。

所有输入的数均小于100000。

输出要点：输出答案。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：为每个整数建立出现次数。读完一组后遍历频率表，统计值恰为 1 的 键数。输入以 n=0 结束而不是测试组数；使用 `unordered_map` 还能兼容 题面未明确给出下界的整数，不依赖数组偏移。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组期望  $O(n)$  时间、 $O(n)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n && n != 0) {
8 |         unordered_map<int, int> freq;
9 |         freq.reserve(n * 2);
10 |         for (int i = 0; i < n; ++i) {
11 |             int value;
12 |             cin >> value;
13 |             ++freq[value];
14 |         }
15 |         int ans = 0;
16 |         for (auto [value, count] : freq)
17 |             ans += count == 1;
18 |         cout << ans << '\n';
19 |     }
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
5
1 2 3 3 2
7
1 2 3 4 5 5 6
0
```

样例 1 输出

```
1
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.72 | 总 72/526 简单的数学问题 ( PID 1243 )

出处：2016级河南工大新生周赛（七）（G，CID 1014）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：因数枚举、平方根、最优化 | 限制：1.000 S / 128 MB | 历史统计：36/184 ( 19.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：，就是给你一个矩形的面积，求它的最小周长，是不是很简单，哈哈，可是俩就是答不出来我有什么办法，呢问题只好交给你们了。

输入要点：第一行输入一个数代表测试实例t

每组测试实例输入一个数s (  $1 \leq s \leq 10^9$  ) 表示矩形的面积

输出要点：输出每测试实例最小周长

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：设整数边长为 a、b 且  $ab=s$ ，周长为  $2(a+b)$ 。乘积固定时两数越接近，和越小，因此从  $\text{floor}(\sqrt{s})$  向下寻找第一个能整除 s 的 a，此时  $b=s/a$ ，二者是最接近的一对整数因数，周长最小。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(\sqrt{s})$  最坏时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long area;
10 |         cin >> area;
11 |         long long side = sqrt((long double)area);
12 |         while (area % side != 0)
13 |             --side;
14 |         cout << 2 * (side + area / side) << '\n';
15 |     }
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
1
24
```

样例 1 输出

```
20
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.73 | 总 73/526 Choice学姐的众数问题 ( PID 1321 )

出处：2017级河南工大新生周赛 ( 二 ) ( F, CID 1029 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：Boyer-Moore、众数、常数空间 | 限制：1.000 S / 4 MB | 历史统计：12/68 ( 17.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出mi中出现最多的那个数，占一行。

输入要点：第一行输入一个整数n (  $1 \leq n \leq 1e6$  )。

接下来一行n个整数mi( $1 \leq mi \leq 1e9$ )，表示第i种糖果的个数，整数之间用空格隔开。

(注意内存限制，众数出现的次数大于n/2)

输出要点：输出mi中出现最多的那个数，占一行。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：多数元素出现次数严格大于  $n/2$ 。Boyer-Moore 投票法维护候选者和 票数：票数为 0 时换成当前数；相同则加一，不同则减一。每次抵消掉 一对不同元素后，多数元素仍比其他元素总数更多，所以最终候选必是 答案。无需保存百万个数，也无需哈希表。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     long long cand = 0;
9 |     int votes = 0;
10 |     for (int i = 0; i < n; ++i) {
11 |         long long value;
12 |         cin >> value;
```



```

13 |         if (votes == 0) {
14 |             cand = value;
15 |             votes = 1;
16 |         } else if (value == cand) {
17 |             ++votes;
18 |         } else {
19 |             --votes;
20 |         }
21 |     }
22 |     cout << cand << '\n';
23 |     return 0;
24 | }

```

## 公开样例

样例 1 输入

```

5
10 10 10 20 30

```

样例 1 输出

```

10

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.74 | 总 74/526 魔法卡题少女 (PID 1323)

出处：2017级河南工大新生周赛（三）（D，CID 1030）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：思维题、字符串计数、代数化简 | 限制：1.000 S / 128 MB | 历史统计：9/70 (12.9%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，一个数，第一个串最后一个数字的下标

**输入要点：**第一行，一个  $n$ ，01 串的长度

**数据范围：** ( $2 \leq n \leq 1000000$ )

**第二行，**一个 01 串  $s$

**输出要点：**一个数，第一个串最后一个数字的下标

**题面提示：**样例：这个串可以从关键位置分割成两个串，串1:0000，串2:10111，为了方便表示关键位置，我们用第一个串的最后一个数字的下标代替关键位置输出。所以输出串1中最后一个0的位置，也就是4。

**特殊的：**如果一个串中没有出现过1（比如串：0000），请输出0。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：设切在前  $i$  位后，前缀中有  $\text{one}$  个 1，整串共有  $\text{totalOne}$  个 1。串 1 的 0 数是  $i - \text{one}$ ，串 2 的 1 数是  $\text{totalOne} - \text{one}$ 。两者相等时， $i - \text{one} = \text{totalOne} - \text{one}$ ，约掉  $\text{one}$  后直接得到  $i = \text{totalOne}$ 。所以关键位置就是整串 1 的总数；没有 1 时自然输出 0。无需真的枚举

分割点或做前缀和。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  额外空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     string s;
8 |     cin >> n >> s;
9 |     cout << count(s.begin(), s.end(), '1') << '\n';
10 |     return 0;
11 | }
```

## 公开样例

样例 1 输入

```
9
000010111
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 1.75 | 总 75/526 Hearthstone ( PID 1363 )

出处：2017级河南工大新生周赛总决赛（重现）（C，CID 1049）；2017级河南工大新生周赛总决赛（重现）（C，CID 1039）；2017级河南工大新生周赛总决赛（C，CID 1038） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：概率动态规划、期望与概率、边界状态 | 限制：1.000 S / 128 MB | 历史统计：12/135 ( 8.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个百分数表示QAQ获胜的概率，精确到小数点后1位

输入要点：多实例测试

每行输入两个整数A，B表示双方的血量（ $1 \leq A, B \leq 100$ ）

输出要点：输出一个百分数表示QAQ获胜的概率，精确到小数点后1位

题面提示：因为1次造成10点伤害，所以第一发炎爆一定会导致一方死亡，概率为50%

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把血量换成“还要挨多少次 10 点伤害”：QAQ 需要  $p = \text{ceil}(A/10)$  次死亡，对手需要  $q = \text{ceil}(B/10)$  次死亡。令  $dp[i][j]$  表示双方还分别能挨  $i, j$  次时 QAQ 最终获胜的概率。若  $i=0$  则已失败，概率 0；若  $j=0$  则已获胜，概率 1；其他状态下一发各有一半打中一方，因此  $dp[i][j] = (dp[i-1][j] + dp[i][j-1]) / 2$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(pq)$  时间、 $O(pq)$  空间，其中  $p, q \leq 10$

## 易错点

- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int A, B;
7 |     while (cin >> A >> B) {
8 |         int p = (A + 9) / 10;
9 |         int q = (B + 9) / 10;
10 |         vector<vector<double>> dp(p + 1, vector<double>(q + 1));
11 |         for (int i = 1; i <= p; ++i)
12 |             dp[i][0] = 1.0;
13 |         for (int i = 1; i <= p; ++i)
14 |             for (int j = 1; j <= q; ++j)
15 |                 dp[i][j] = (dp[i - 1][j] + dp[i][j - 1]) / 2.0;
16 |         cout << fixed << setprecision(1) << dp[p][q] * 100.0 << "%\n";
17 |     }
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
5 10
1 11
```

样例 1 输出

```
50.0%
25.0%
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 1.76 | 总 76/526 Greedy ( PID 1424 )

出处：2018级河南工大新生周赛总决赛（重现）（H，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（H，CID 1048） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：区间动态规划、表达式、最大最小状态 | 限制：3.000 S / 32 MB | 历史统计：2/23 ( 8.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：问如何才能使得这个表达式的值最大

输入要点：多组测试数据

对于每组测试数据，先输入一个正整数 $n$  ( $n \leq 120$ )

然后输入 $n$ 个数字以表示表达式中的每一项

举个例子：7 2 -5 6 -4 8 对应的表达式就为 $7+2-5+6-4+8$

输出要点：输出表达式的最大值（保证答案在int范围内）

题面提示：对于样例1： $1+2+3+4+5 = 15$ （不需要加任何括号，当然你也可以在任何地方加括号，最后答案都是15）

对于样例2： $1+2+3-(4-2) = 4$

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把输入中的正数看成前面的“+”，负数看成前面的“-”，括号只能改变结合顺序。令  $mx[i][r]$ 、 $mn[i][r]$  分别为子表达式能取得的最大、最小值。枚举最后一次把区间切在  $k$ ：若连接符是加号，最大加最大、最小加最小；若是减号，最大值用左最大减右最小，最小值用左最小减右最大。由短区间 向长区间计算， $mx[0][n-1]$  即答案。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(n^3)$  时间、 $O(n^2)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n) {
8 |         vector<long long> input(n), value(n);
9 |         vector<char> op(n, '+');
10 |         for (int i = 0; i < n; ++i) {
11 |             cin >> input[i];
12 |             value[i] = labs(input[i]);
13 |             if (i > 0 && input[i] < 0)
14 |                 op[i] = '-';
15 |         }
16 |         value[0] = input[0];
17 |         const long long INF = (1LL << 60);
18 |         vector<vector<long long>> mn(n, vector<long long>(n, INF));
19 |         vector<vector<long long>> mx(n, vector<long long>(n, -INF));
20 |         for (int i = 0; i < n; ++i)
21 |             mn[i][i] = mx[i][i] = value[i];
22 |         for (int len = 2; len <= n; ++len) {
23 |             for (int left = 0; left + len <= n; ++left) {
24 |                 int right = left + len - 1;
25 |                 for (int split = left; split < right; ++split) {
26 |                     if (op[split + 1] == '+') {
27 |                         mx[left][right] =
28 |                             max(mx[left][right], mx[left][split] + mx[split + 1][right]);
29 |                         mn[left][right] =
30 |                             min(mn[left][right], mn[left][split] + mn[split + 1][right]);
31 |                     } else {
32 |                         mx[left][right] =
33 |                             max(mx[left][right], mx[left][split] - mn[split + 1][right]);
34 |                         mn[left][right] =
35 |                             min(mn[left][right], mn[left][split] - mx[split + 1][right]);
36 |                     }
37 |                 }
38 |             }
39 |         }
40 |         cout << mx[0][n - 1] << '\n';
41 |     }
42 |     return 0;
43 | }
```

### 公开样例

#### 样例 1 输入

```
5
1 2 3 4 5
5
1 2 3 -4 -2
```

#### 样例 1 输出

```
15
4
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)

## 1.77 | 总 77/526 River ( PID 1427 )

出处：2018级河南工大新生周赛总决赛（重现）（K，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（K，CID 1048） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：数学建模、极坐标、微积分、浮点输出 | 限制：1.000 S / 16 MB | 历史统计：3/11 ( 27.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对于每组测试数据，输出你渡河所需的时间（四舍五入到小数点后2位），如果永远到不了对岸，输出“Infinity”

输入要点：先输入一个T表示T组测试数据（ $T \leq 1000$ ）

接下来T行，每行三个正整数a, v1, v2（ $1 \leq a, v1, v2 \leq 100$ ）表示你所在的位置(0, a)，你的漂流速度，以及当天的风速

输出要点：对于每组测试数据，输出你渡河所需的时间（四舍五入到小数点后2位），如果永远到不了对岸，输出“Infinity”

题面提示：建议使用double数据类型

## 零基础先修

- 浮点题按误差要求输出足够位数，通常使用 fixed 与 setprecision。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：设木筏相对水的速度为 v1、横风为 v2。若  $v1 \leq v2$ ，靠近原点的分量无法战胜横向漂移，永远不能精确到达。否则在极坐标中有  $dr/dt = -v1 + v2 \cdot \cos\theta$ ， $r \cdot d\theta/dt = -v2 \cdot \sin\theta$ 。消去 t 并从初始  $\theta = \pi/2$ 、 $r = a$  积分，可得总时间  $T = a \cdot v1 / (v1^2 - v2^2)$ 。最后固定输出两位小数。  
样例 2,4,3 得  $8/7 \approx 1.14$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         double a, v1, v2;
10 |         cin >> a >> v1 >> v2;
11 |         if (v1 <= v2) {
12 |             cout << "Infinity\n";
13 |         } else {
14 |             double time = a * v1 / (v1 * v1 - v2 * v2);
15 |             cout << fixed << setprecision(2) << time << '\n';
16 |         }
17 |     }
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
2
2 3 3
2 4 3
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.78 | 总 78/526 HJ病毒 ( PID 1516 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( C , CID 1058 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：5/73 ( 6.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组测试数据输出一行，每行包含一个整数

输入要点：多组测试数据，每组包含两个整数， $n, t (1 \leq n \leq 10000, 1 \leq t \leq 10000000000)$

输入以 EOF 结束

输出要点：每组测试数据输出一行，每行包含一个整数

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：HJ病毒快速幂基础题 由 $1+1=2$ 我们可以知道：答案是  $n \times 2^{tn} \times 2^{tn} \times 2^t$  然后用快速幂算出 即可，不要忘记取模  $2^{t2} \times t2^t$  C++ 版代码
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <iostream>
2 | #define mod 1000000007
3 | using namespace std;
4 | typedef long long LL;
5 | LL n, t;
6 | LL q_w(LL x, LL t) {
```

```

7 |     LL ans = 1;
8 |     while (t) {
9 |         if (t & 1)
10 |             ans *= x, ans %= mod;
11 |         x *= x;
12 |         x %= mod;
13 |         t >>= 1;
14 |     }
15 |     return ans;
16 | }
17 | int main() {
18 |     while (cin >> n >> t) {
19 |         cout << n * q_w(2, t) % mod << "\n";
20 |     }
21 |     return 0;
22 | }

```

## 公开样例

样例 1 输入

```

1 4
2 4
3 4
4 4
999 999999999

```

样例 1 输出

```

16
32
48
64
742187513

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

# 1.79 | 总 79/526 游戏 ( PID 1526 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( E , CID 1059 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：0/8 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，对于每一种情况，根据游戏胜利者的不同，打印出案例号和“CXKNB”或“AWNB”。

**输入要点：**输入以整数T( $\leq 200$ )开始，表示测试用例的数量。

每一种情况都从一行包含k( $1 \leq k \leq 100$ )开始，表示单元格中的灰块数。下一行包含2k个不同的整数(按升序排列)，表示球的位置。第一个整数表示灰球，第二个整数表示白球，下一个整数表示灰球，依此类推。所有整数都位于 $[0, 109]$ 范围内。假设n足够大。

**输出要点：**对于每一种情况，根据游戏胜利者的不同，打印出案例号和“CXKNB”或“AWNB”。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导



1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：游戏 nim 博弈，将灰色和白色棋子之间的距离看作是石头的数量，可以转化成简单的 nim 博弈，直接以后求解。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | #define maxn 10005
3 | #define INF 0x3f3f3f3f
4 | typedef long long ll;
5 | using namespace std;
6 | int main() {
7 |     int n;
8 |     int t, T = 1;
9 |     cin >> t;
10 |    while (t--) {
11 |        cin >> n;
12 |        int ans = 0;
13 |        for (int i = 0; i < n; i++) {
14 |            int x, y;
15 |            cin >> x >> y;
16 |            ans ^= (y - x - 1);
17 |        }
18 |        if (ans == 0)
19 |            printf("Case %d: AWNB\n", T++);
20 |        else
21 |            printf("Case %d: CXKNB\n", T++);
22 |    }
23 |    return 0;
24 | }
```

## 公开样例

样例 1 输入

```
2
2
0 3 7 9
2
1 3 7 9
```

样例 1 输出

```
Case 1: CXKNB
Case 2: AWNB
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)
- [题面图片 1](#)
- [题面图片 2](#)
- [题面图片 3](#)
- [题面图片 4](#)

## 1.80 | 总 80/526 x的n次幂 ( PID 1528 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( H , CID 1059 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：29/167 ( 17.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定 $x$ ， $n(0 \leq x, n \leq 109)$  求出 $x^n \% 1000$

输入要点：第一行输入一个整数 $t$ ，代表 $t$ 组数据。

接下来 $t$ 行，每行包含两个整数 $x, n$

输出要点：求出每组 $x^n \% 1000$

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用： $x$ 的 $n$ 次幂 快速幂的模板题 `#include <bits/stdc++.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long QuickPow(int x, int N) {
4 |     int res = x;
5 |     int ans = 1;
6 |     while (N) {
```

```

7 |         if (N & 1) {
8 |             ans = ans * res % 1000;
9 |         }
10 |         res = res * res % 1000;
11 |         N = N >> 1;
12 |     }
13 |     return ans % 1000;
14 | }
15 | int main() {
16 |     int t;
17 |     cin >> t;
18 |     while (t--) {
19 |         int x, n;
20 |         cin >> x >> n;
21 |         cout << QuickPow(x, n) << endl;
22 |     }
23 |     return 0;
24 | }

```

## 公开样例

### 样例 1 输入

```

2
2 2
2 3

```

### 样例 1 输出

```

4
8

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.81 | 总 81/526 阿正的换位排序 ( PID 1546 )

出处：2020级HAUT新生周赛（二）@杨哲专场（H，CID 1061）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：逆序对、观察、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：3/95 ( 3.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**阿正随机写下了若干个序列，你的任务是判断这若干个序列能否在  $[n*(n-1)]/2 - 1$  步内完成排序。

**输入要点：**多实例输入

**每组数据第一行**一个正整数  $n$ ，代表序列长度；（ $2 \leq n \leq 100000$ ）

**每组数据第二行**包含  $n$  个非负整数，依次代表序列各元素。

**测试数据保证**全部样例  $n$  的和不超过 100000，其他数据保证在整形范围内。

**输出要点：**每组数据对应一行。

若可以在规定步数内完成排序，请输入“Yes”，否则请输入“No”。（输出不带引号）

**题面提示：**对于第一组数据，可以按以下方案进行交换完成阿正的“换位排序”：

1 交换四号元素“1”与三号元素“2”：[5 3 2 1 4] -> [5 3 1 2 4]

2 交换三号元素“1”与二号元素“3” : [5 3 1 2 4]->[5 1 3 2 4]

3 交换二号元素“1”与一号元素“5” : [5 1 3 2 4]->[1 5 3 2 4]

4 交换四号元素“2”与三号元素“3” : [1 5 3 2 4]->[1 5 2 3 4]

5 交换三号元素“2”与二号元素“5” : [1 5 2 3 4]->[1 2 5 3 4]

6 交换四号元素“3”与三号元素“5” : [1 2 5 3 4]->[1 2 3 5 4]

7 交换五号元素“4”与三号元素“5” : [1 2 3 5 4]->[1 2 3 4 5]

按上述步骤可以在最小的步数 ( 共七步 ) 内完成排序, 由于 $n=5$ ,  $[n*(n-1)]/2-1=9$ , 七步小于九步, 故输出Yes。

## 零基础先修

- 多组数据以 EOF 结束时使用 while (cin >> ...), 不要额外输出测试数据。

## 思路推导

1. 先按输入说明读取数据, 并用最小样例手算一次, 确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描, 不引入题目没有要求的额外状态。
3. 核心处理采用: 相邻交换排成非递减序的最少次数等于逆序对数。其理论最大值  $n(n-1)/2$  只在序列严格递减时出现; 题目上限恰比该最大值少 1, 所以只有严格递减序列输出 No, 其余全部输出 Yes。
4. 最后按题目要求输出; 提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素, 分支覆盖它的全部可能情况, 并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次, 累计状态即为所求。

## 复杂度

每组  $O(n)$  时间、 $O(n)$  空间

## 易错点

- EOF 题不要先读不存在的 T, 也不要循环结束后多输出内容。
- 按题目上界估算中间量, 相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n) {
8 |         vector<long long> a(n);
9 |         for (long long& x : a)
10 |             cin >> x;
11 |         bool dec = true;
12 |         for (int i = 1; i < n; ++i) {
13 |             if (a[i - 1] <= a[i])
14 |                 dec = false;
15 |         }
16 |         cout << (dec ? "No" : "Yes") << '\n';
17 |     }
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
5
5 3 2 1 4
6
```

3 5 4 2 7 5

样例 1 输出

Yes  
Yes

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.82 | 总 82/526 吓得我赶紧出了个签到题 ( PID 1562 )

出处：2020级HAUT新生周赛（四）@张承树专场（H，CID 1063）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、代码阅读 | 限制：1.000 S / 128 MB | 历史统计：113/252 ( 44.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，两行，第一行四个选项（全为大写，无空格），第二行输出填空题答案

输入要点：无

输出要点：两行，第一行四个选项（全为大写，无空格），第二行输出填空题答案

题面提示：(也许你可以运行一下这串代码)

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

## 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：四道选择题依次为 A、C、D、A。把题面给出的程序原样编译运行，其七个字符输出为 lcltql!；本题只要求输出结论，不应附加空格或解释。
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "ACDA\n";
5 |     cout << "lcltql!\n";
```

```
6 |     return 0;
7 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 1.83 | 总 83/526 牛牛的导数 ( PID 1584 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（F，CID 1070）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：122/709 ( 17.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数代表 $f(c)$ 的值

输入要点：依次输入三个整数 $a, b, c$ 。

输出要点：输出一个整数代表 $f(c)$ 的值

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：牛牛的导数水题，没啥好说的，注意 $\text{int}$ 会爆，开 $\text{long long}$ 就行。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a, b, c;
4 |     scanf("%d%d%d", &a, &b, &c);
5 |     a = a * b;
6 |     b--;
```

```

7 |     long long cnt = 1;
8 |     int t = b;
9 |     while (t > 0)
10 |         cnt *= c, t--;
11 |     cnt *= a;
12 |     printf("%lld", cnt);
13 | }

```

## 公开样例

样例 1 输入

5 1 2

样例 1 输出

5

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.84 | 总 84/526 卷王之争 ( PID 1589 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（A，CID 1071） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：167/1381 ( 12.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果小H的学习时间多，输出"H is juanwang!"；如果小T的学习时间多，输出"T is juanwang!"；如果两人学习时间一样多，输出"We are juanwang!"；(输出不带引号)

输入要点：一行四个非负整数， $a, b, x, y$

( $a \leq b, x \leq y, < a, b, x, y \leq 100\,000\,000, b, y \neq 0$ )

输出要点：如果小H的学习时间多，输出"H is juanwang!"；

如果小T的学习时间多，输出"T is juanwang!"；

如果两人学习时间一样多，输出"We are juanwang!"；

(输出不带引号)

题面提示：注意，double不能直接判等号。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：卷王之争 比较两个分数大小；法一：直接通分时要注意数据范围。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <iostream>
2 | int main() {
3 |     long long a, b, x, y;
4 |     scanf("%d%d%d%d", &a, &b, &x, &y);
5 |     a *= y;
6 |     x *= b;
7 |     if (a > x)
8 |         printf("H is juanwang!\n");
9 |     else if (a < x)
10 |         printf("T is juanwang!\n");
11 |     else
12 |         printf("We are juanwang!\n");
13 |     return 0;
14 | }
```

### 公开样例

样例 1 输入

```
1 2 3 4
```

样例 1 输出

```
T is juanwang!
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.85 | 总 85/526 这年头难道有人不喜欢签到吗？ ( PID 1711 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（C，CID 1073） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、数论、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：17/236 ( 7.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个正整数表示1-n的十进制二进制数字的个数。

输入要点：输入一个正整数n。其中n<=1e9;

输出要点：输出一个正整数表示1-n的十进制二进制数字的个数。

题面提示：样例解释：从1到10中有1和10两个符合的数字，因此总个数为2。

### 零基础先修



- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- string 可按不下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：这年头难道有人不喜欢签到吗题意分析：这道题相当于是数字组合的问题，数字范围是 $1e9$ ，那么我们可以先把该范围的所有符合的数字先提前预处理存一下，方便程序的实现。具体分析：接下来的实现就是得到一个正整数 $n$ ，首先先加上比 $n$ 的位数少的符合的数字的个数，然后再对 $n$ 所在的位数符合的数字进行判断，判断该问题时，我们可以从头往后处理，模拟处理。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

$O(1)$

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```

1 | #include <stdio.h>
2 | #include <string.h>
3 | #include <math.h>
4 | char a[20];
5 | int v[100];
6 | signed main() {
7 |     int sum = 9; // 数据范围最大1e9;
8 |     int x = 1, xx = 0;
9 |     while (sum--) // 预处理存储相应位数数字的所有可能的情况
10 |     {
11 |         v[xx++] = x;
12 |         x *= 2;
13 |     }
14 |     gets(a);
15 |     int pd = a[0] - '0';
16 |     if (pd > 1) // 首位数字大于1的时候则可以生成该位数所有的情况
17 |     {
18 |         int ans = 0;
19 |         for (int i = 0; i < strlen(a); i++) {
20 |             ans += v[i];
21 |         }
22 |         printf("%d\n", ans);
23 |     } else // 首位数字等于1
24 |     {
25 |         int ans = 0;
26 |         for (int i = 0; i < strlen(a) - 1; i++) // 生成比该数字位数少1的所有情况
27 |         {
28 |             ans += v[i];
29 |         }
30 |         int flag = -1, xx, pd = 0;
31 |         for (int i = 1; i < strlen(a); i++) // 从第一位数字开始处理当前位数的数字
32 |         {
33 |             xx = a[i] - '0';
34 |             if (xx > 1) // 若处理位大于1则后续数字均可以由0或1代替，排列组合思想，总情况就是pow(2,
35 |                 // strlen(a) - i)
36 |             {
37 |                 pd = 1;
38 |                 ans += pow(2, strlen(a) - i);
39 |                 flag = 1;

```

```

40 |         break;
41 |     } else if (xx == 1) // 若处理位等于1则当前位不能替换但后续位的数字可以替换为0或1,总情况就是
42 |         // pow(2, strlen(a) - i - 1)
43 |     {
44 |         ans += pow(2, strlen(a) - i - 1);
45 |         flag = 1;
46 |     }
47 | }
48 | if (flag == -1 || !pd) // 特判输入数字第一位是1需要加上它本身; 特判输入数字为0
49 | {
50 |     ans++;
51 | }
52 | printf("%d\n", ans);
53 | }
54 | }

```

## 公开样例

样例 1 输入

10

样例 1 输出

2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.86 | 总 86/526 爱情废柴 ( PID 1729 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（D，CID 1075）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：条件分支 | 限制：1.000 S / 128 MB | 历史统计：29/182 ( 15.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果count不为0，输出count，否则输出"Thanks to LF's object!" 不带引号

输入要点：第一行输入次数n（ $2 \leq n \leq 10000$ ）

第二行输入n个数，第i次写出a[i]个对象（ $0 \leq a[i] \leq 100000$ ）

输出要点：如果count不为0，输出count，否则输出"Thanks to LF's object!" 不带引号

题面提示：从第二个数据开始判断，编写的第一次不会分享。

-

5->8 多了3个对象，送给朋友

-

8->6 少了2个 需要补上2个

-

6->1 少了5个 需要补的2个被聋妃划掉了，现在需要补上5个

-

1->7 多了6个 把之前缺的5个补上还剩1个，送给朋友

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：爱情废柴模拟 按照要求模拟即可 C #include<stdio.h>int main() { int n; int a,b; int lack=0,thank=0; scanf("%d",&n); scanf("%d",&b); for(int i=2;i<=n;i++) { scanf("%d",&a); if(a<b) { lack =b-a; } else if(lack!=0) { if(a-b>lack) { lack =0; thank++; } else if(a-b==lack) { lack =0; } else if(a-b<lack) { lack -=a-b; } } else if(a>b) { thank ++; } b=a; } if(thank==0) { printf("Thanks to LF's object!\n"); } else { printf("%d\n",thank); } } E
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     int a, b;
5 |     int lack = 0, thank = 0;
6 |     scanf("%d", &n);
7 |     scanf("%d", &b);
8 |     for (int i = 2; i <= n; i++) {
9 |         scanf("%d", &a);
10 |         if (a < b) {
11 |             lack = b - a;
12 |         } else if (lack != 0) {
13 |             if (a - b > lack) {
14 |                 lack = 0;
15 |                 thank++;
16 |             } else if (a - b == lack) {
17 |                 lack = 0;
18 |             } else if (a - b < lack) {
19 |                 lack -= a - b;
20 |             }
21 |         } else if (a > b) {
22 |             thank++;
23 |         }
24 |         b = a;
25 |     }
26 |     if (thank == 0) {
27 |         printf("Thanks to LF's object!\n");
28 |     } else {
29 |         printf("%d\n", thank);
30 |     }
31 | }
```

## 公开样例

样例 1 输入

```
5
5 8 6 1 7
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.87 | 总 87/526 小C和反转数字 ( PID 1752 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（H，CID 1077）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：94/242 ( 38.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出小C能否用这个数字换取一颗糖，能则输出"YES",不能则输出"no"(不包括引号)

输入要点：输入小C得到的整数 $n$  ( $0 \leq n \leq 1e9$ )

输出要点：输出小C能否用这个数字换取一颗糖，能则输出"YES",不能则输出"no"(不包括引号)

题面提示：另一组样例：

输入：180

输出：no

对于第一组样例，数字第一次反转后是7531，再反转即可回到与原来相等的1357，因此输出"YES"

对于第二组样例，数字第一次反转后是81，再反转是18，与原来不等，因此输出"no"

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小C和反转数字这题是次签到题 对于一个数字，如果它前后反转会出现不同，那他肯定就是0结尾的（第一次反转后0被当作前导零清掉了），此外，0反转后是能回到0的，需要特判输出YES就行 上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 核对读入顺序、变量类型和输出格式。

- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     if (n != 0 && n % 10 == 0)
6 |         printf("no");
7 |     else
8 |         printf("YES");
9 |     return 0;
10 | }
```

### 公开样例

|         |
|---------|
| 样例 1 输入 |
| 1357    |
| 样例 1 输出 |
| YES     |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.88 | 总 88/526 比赛模拟 ( PID 1753 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（H，CID 1085）；2022级HAUT新生周赛（零）熟悉周赛规则专场（G，CID 1078） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现、循环、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：368/802 ( 45.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每组数据需你输出一个整数，代表这场比赛的总用时。不同组数据之间用回车“\n”来进行分割。

输入要点：多组数据，文件尾结束

对于每组数据，第一行一个整数  $n$  ( $1 \leq n \leq 20$ ) 代表这场的题目个数

接下来  $n$  行，每行有两个整数，之间以空格分割，第  $i$  行有  $t_i$  ( $0 \leq t_i \leq 300$ ) 和  $cnt_i$  ( $0 \leq cnt_i \leq 100$ )，分别代表第  $i$  道题的第一次通过的时间和 第  $i$  道题在第一次通过前非正确提交的次数。

输出要点：对于每组数据需你输出一个整数，代表这场比赛的总用时。

不同组数据之间用回车“\n”来进行分割。

题面提示：这是一个多组测试样例的题目。意思为每一个测试文件中会放入多组测试数据（未知多少组），当你读取数据读到文件结尾时代表结束。

示例：c 语言

```
// 这里scanf当读取到文件尾就会返回 -1，

// EOF为一个宏，EOF(end of file)，文件结尾，EOF定义在stdio.h 文件中 #define EOF (-1)

// 当你写的程序在本地测试不是文件作为读取流，是在终端运行，可以按 windows下为ctrl+z，linux/unix下为ctrl+c或ctrl+d；
```

// 就是可以在终端中先复制测试样例，然后回车 按 'ctrl + z' 或者 'ctrl + d'

```
while(scanf("%d",&n) != EOF){  
  
}
```

c++ 语言

// 这里 cin 可以作为判断条件是发生了隐式转化，具体了解可以参考 [https://en.cppreference.com/w/cpp/io/basic\\_istream](https://en.cppreference.com/w/cpp/io/basic_istream)

```
while(std::cin >> n){  
  
}
```

## 零基础先修

- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。
- 多组数据以 EOF 结束时使用 while (cin >> ...)，不要额外输出测试数据。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每道最终通过的题贡献首次通过时间  $t_i$ ；此前每次错误提交再贡献 20 分钟。逐组读入  $n$ ，累加  $t_i + 20 * cnt_i$  即可。题目没有给数据组数，所以必须把  $cin >> n$  直接放在 while 条件里，读到文件尾自然停止。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  额外空间

## 易错点

- EOF 题不要先读不存在的 T，也不要再在循环结束后多输出内容。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>  
2 | using namespace std;  
3 | int main() {  
4 |     ios::sync_with_stdio(false);  
5 |     cin.tie(nullptr);  
6 |     int n;  
7 |     while (cin >> n) {  
8 |         long long ans = 0;  
9 |         for (int i = 0; i < n; ++i) {  
10 |             long long time, wrong;  
11 |             cin >> time >> wrong;  
12 |             ans += time + 20 * wrong;  
13 |         }  
14 |         cout << ans << '\n';  
15 |     }  
16 |     return 0;  
17 | }
```

## 公开样例

样例 1 输入

```
2  
28 2  
8 1  
3
```

```
28 0
22 0
6 0
4
11 0
26 0
28 2
9 0
4
14 1
29 1
13 2
22 1
```

样例 1 输出

```
96
56
114
178
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 1.89 | 总 89/526 I really love haut! ( PID 1754 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（A，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（A，CID 1079） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、字符串 | 限制：1.000 S / 128 MB | 历史统计：848/4051 ( 20.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，"I really love haut!\n";

输入要点：无

输出要点："I really love haut!\n";

## 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：I really love haut!\n";"); return 0; } B
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("\nI really love haut!\n");
4 |     return 0;
5 | }
```

### 公开样例

|         |
|---------|
| 样例 1 输入 |
| 无       |
| 样例 1 输出 |
| 无       |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.90 | 总 90/526 喂喂，你知道魔法少女的这个传闻吗？ ( PID 1762 )

出处：2022级HAUT新生周赛（二）@武其轩专场（H，CID 1080） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、整数运算 | 限制：1.000 S / 128 MB | 历史统计：330/487 ( 67.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，直接输出答案即可。

输入要点：本题无输入

输出要点：一行，直接输出答案即可。

题面提示：奇迹和魔法都是存在的~~~

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

### 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：先算整数个数，再乘一个 int 占用的 4 字节。本题无输入，只需把 确定的答案 497120 原样输出；固定输出题最重要的是不要添加解释文字。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明



- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

复杂度

O(1) 时间、O(1) 空间

易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << 497120 << '\n';
5 |     return 0;
6 | }
```

公开样例

样例 1 输入

无

样例 1 输出

直接输出答案

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

1.91 | 总 91/526 寝室的宝箱 ( 困难版 ) ( PID 1822 )

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（G，CID 1091）；2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（F，CID 1086） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：0.200 S / 128 MB | 历史统计：201/1638 ( 12.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，见题目描述。

输入要点：分别给出 正整数x、运算符ch、正整数y。

( x、y<=100000，ch={'+', '-', '\*', '/'} )

输出要点：见题目描述。

零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

- 核心处理采用：寝室的宝箱（困难版）如果选择整形进行计算需要考虑使用longlong，对于除法需要特判一下，另外使用浮点数进行计算由于没有卡精度所以也是可以过的。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     long long x, y;
4 |     char ch;
5 |     scanf("%lld %c %lld", &x, &ch, &y);
6 |     if (ch == '/') {
7 |         if (x % y)
8 |             printf("%.2lf", (double)x / y);
9 |         else
10 |            printf("%lld", x / y);
11 |     } else
12 |         printf("%lld", ch == '+' ? x + y : (ch == '-' ? x - y : x * y));
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

10 / 3

样例 1 输出

3.33

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.92 | 总 92/526 jxh和lhy的决斗 (PID 1824)

出处：2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（H，CID 1086）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：大模拟、结构体、状态机 | 限制：1.000 S / 128 MB | 历史统计：2/17 (11.8%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：jxh和lhy两位聚聚想要来若干场游戏中的决斗来证明自己比对方强。假设攻击力高的一方先手（攻击力相等时jxh聚聚先手），给你每一场战斗时两位聚聚角色的属性和他们的操作，请你判断每一场胜负。

输入要点：第一行一个正整数 $t$ ，代表两位聚聚的战斗场数（ $1 \leq t \leq 50$ ）。

接下来 $t$ 组数据，每组数据的格式如下：

第一行六个正整数：hp1, mp1, atk1, x1, y1, z1，分别代表jxh聚聚的血量上限，法力值上限，攻击力，选择回血时回复的血量值，选择防御时获得的护盾值，选择释放技能时消耗的法力值；

第二行六个正整数：hp2, mp2, atk2, x2, y2, z2，分别代表lhy聚聚的血量上限，法力值上限，攻击力，选择回血时回复的血量值，选择防御时获得的护盾值，选择释放技能时消耗的法力值；

（双方的hp, mp, atk, x, y, z都是 $1e5$ 内的正整数）

第三行一个正整数 $n$ ，代表回合数；（ $1 \leq n \leq 1e4$ ）

接下来 $n$ 行，每行两个正整数，代表先手行动方和后手行动方在该回合的操作。

输出要点：输出 $t$ 行，第 $i$ 行代表第 $i$ 场战斗的胜负情况、

若jxh聚聚获胜，输出“不愧是你啊，jxh！”；

若lhy聚聚获胜，输出“lhy！lhy！lhy！！！”；

若平局则输出“-”。（三个都不输出双引号）。（平局条件为所有回合结束，双方的血量都大于0）

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：先按攻击力决定先后手（相等时jxh先手），每回合输入的两个操作依次属于先手、后手。行动前令自己的护盾剩余回合减一，归零时清空护盾。操作1普攻；2回血但不超过上限；3将护盾重置为y、持续3；4在MP足够时扣z并连续普攻三次。每次攻击先扣护盾再扣血，先手行动造成死亡后该回合立即结束。模拟结束仍都存活即平局。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(T \cdot n)$  时间、 $O(1)$  额外空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | struct Player {
4 |     long long hp, maxHp, mp, attack, heal, shieldValue, skillCost;
5 |     long long shield = 0;
6 |     int shieldTurns = 0;
7 |     string name;
8 | };
9 | void beginTurn(Player& player) {
```

```

10 |     if (player.shieldTurns > 0)
11 |         --player.shieldTurns;
12 |     if (player.shieldTurns == 0)
13 |         player.shield = 0;
14 | }
15 | void hit(const Player& attacker, Player& defender) {
16 |     long long damage = attacker.attack;
17 |     long long absorbed = min(defender.shield, damage);
18 |     defender.shield -= absorbed;
19 |     defender.hp -= damage - absorbed;
20 | }
21 | void act(Player& actor, Player& opponent, int op) {
22 |     if (op == 1) {
23 |         hit(actor, opponent);
24 |     } else if (op == 2) {
25 |         actor.hp = min(actor.maxHp, actor.hp + actor.heal);
26 |     } else if (op == 3) {
27 |         actor.shield = actor.shieldValue;
28 |         actor.shieldTurns = 3;
29 |     } else if (op == 4 && actor.mp >= actor.skillCost) {
30 |         actor.mp -= actor.skillCost;
31 |         for (int repeat = 0; repeat < 3; ++repeat)
32 |             hit(actor, opponent);
33 |     }
34 | }
35 | int main() {
36 |     ios::sync_with_stdio(false);
37 |     cin.tie(nullptr);
38 |     int T;
39 |     cin >> T;
40 |     while (T--) {
41 |         Player jxh, lhy;
42 |         cin >> jxh.hp >> jxh.mp >> jxh.attack >> jxh.heal >> jxh.shieldValue >>
43 |             jxh.skillCost;
44 |         cin >> lhy.hp >> lhy.mp >> lhy.attack >> lhy.heal >> lhy.shieldValue >>
45 |             lhy.skillCost;
46 |         jxh.maxHp = jxh.hp;
47 |         lhy.maxHp = lhy.hp;
48 |         jxh.name = "jxh";
49 |         lhy.name = "lhy";
50 |         Player* first = &jxh;
51 |         Player* second = &lhy;
52 |         if (jxh.attack < lhy.attack)
53 |             swap(first, second);
54 |         int rounds;
55 |         cin >> rounds;
56 |         string winner;
57 |         for (int round = 0; round < rounds; ++round) {
58 |             int op1, op2;
59 |             cin >> op1 >> op2;
60 |             if (!winner.empty())
61 |                 continue;
62 |             beginTurn(*first);
63 |             act(*first, *second, op1);
64 |             if (second->hp <= 0) {
65 |                 winner = first->name;
66 |                 continue;
67 |             }
68 |             beginTurn(*second);
69 |             act(*second, *first, op2);
70 |             if (first->hp <= 0)
71 |                 winner = second->name;
72 |         }
73 |         if (winner.empty())
74 |             cout << "_-\n";
75 |         else if (winner == "jxh")
76 |             cout << "不愧是你啊 , jxh ! \n";
77 |         else
78 |             cout << "lhy ! lhy ! lhy ! ! \n";
79 |     }
80 |     return 0;
81 | }

```

## 公开样例

### 样例 1 输入

```

3
10 10 1 5 5 2
10 10 1 5 5 2
5
4 4
4 4
4 4
4 4
10 10 1 5 5 2

```

```
10 10 5 5 5 2
5
1 2
3 3
4 1
1 1
1 1
10 10 2 5 5 2
10 10 1 5 5 2
5
1 1
1 1
3 2
1 1
4 4
```

样例 1 输出

```
不愧是你啊，jxh！
lhy！lhy！lhy！！！！
~_~
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.93 | 总 93/526 小明的数学游戏 ( PID 1833 )

出处：2023级HAUT新生周赛（二）@曹瑞峰专场（G，CID 1087）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：条件分支 | 限制：1.000 S / 128 MB | 历史统计：2/48 ( 4.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：请你计算，进行若干次操作后，是否可以将第一个数变成a，第二个数变成b。

输入要点：第一行一个正整数t，表示有t组测试数据。(t<=100000)

接下来n行，每行两个正整数a，b。(1<=a,b<=1e9)

输出要点：每行输出一个结果,如果任务可以达成,输出Yes,否则输出No.

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：小明的数学游戏根据算术基本定理：一个数一定能够被分成若干个质数的乘积,所以只需要考虑k取质数的情况了;那么每次我们选择一个质数k；每次操作相当于对a与b的积乘k的立方，那么可以得出a\*b的积一定是一个完全立方数，设t\*t\*t=a\*b。对每个选取的质数进行分析：p1选了a1次，对于a：p1这个质因子出现的次数x必须要在(a1<=x<=2\*a1)这个范围内,那么对于b：p1这个质因子满足(a1 <= y <= 2 \* a1)，根据这个性质，可以得出t可以整除a，b；
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | #define int long long
3 | #define endl '\n'
4 | using namespace std;
5 | signed main() {
6 |     int t;
7 |     ios::sync_with_stdio(false), cin.tie(0), cout.tie(0);
8 |     cin >> t;
9 |     while (t--) {
10 |         int x, y;
11 |         cin >> x >> y;
12 |         int z = x * y;
13 |         int p = pow(z, 1.0 / 3.0);
14 |         int j = 0;
15 |         bool flag = false;
16 |         for (int i = -1; i <= 1; ++i) {
17 |             j = p + i;
18 |             if (z == j * j * j) {
19 |                 flag = true;
20 |                 break;
21 |             }
22 |         }
23 |         if (flag) {
24 |             if (x % j == 0 && y % j == 0) {
25 |                 cout << "Yes" << endl;
26 |             } else {
27 |                 cout << "No" << endl;
28 |             }
29 |         } else {
30 |             cout << "No" << endl;
31 |         }
32 |     }
33 |     return 0;
34 | }
```

## 公开样例

样例 1 输入

```
4
1 1
2 4
8 27
1000 1331
```

样例 1 输出

```
Yes
Yes
No
No
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.94 | 总 94/526 现在是，幻想时刻！（PID 1847）

出处：2023级HAUT新生周赛（四）@牛浩然专场（C，CID 1089）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：23/249（9.2%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示小A可以获得的最大收益。

输入要点：第一行两个整数 $n, m$ 表示已知未来 $n$ 个时刻的股票价格，小A当前还有 $m$ 元（ $0 < n \leq 106, 0 \leq m \leq 103$ ）。

第二行 $n$ 个整数 $a_1 a_2 \dots a_{n-1} a_n$ 表示未来 $n$ 个时刻的股票价格，其中 $a_i$ 表示第 $i$ 个时刻股票的价格（ $0 < a \leq 105$ ）。

输出要点：一个整数，表示小A可以获得的最大收益。

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：现在是幻想时刻。考虑贪心，假设在第 $i$ 个时刻卖出，那么一定在前 $i$ 天股票价格最低时买入最佳，注意，如果最小值都大于现在剩余的钱，那么这个时候无法买入。枚举天数，记录前缀中的最小值，记录这个过程中的最大值即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, k;
4 |     scanf("%d %d", &n, &k);
5 |     int ans = 0;
6 |     int min1 = 1e7;
7 |     int x;
8 |     for (int i = 0; i < n; ++i) {
9 |         scanf("%d", &x);
10 |         if (x <= min1)
11 |             min1 = x;
12 |         if (min1 <= k)
13 |             if (x - min1 >= ans)
14 |                 ans = x - min1;
15 |     }
16 |     printf("%d\n", ans);
17 |     return 0;
18 | }
```

### 公开样例

样例 1 输入

```
5 3
2 1 3 4 5
```

样例 1 输出

```
4
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.95 | 总 95/526 历经千辛万苦，只为得到你 ( PID 1850 )

出处：河南工大2024新生周赛 ( 1 ) ——命题人：程立老师，陈兰锴 ( I , CID 1091 ) ； 2023级HAUT新生周赛 ( 四 ) @牛浩然专场 ( F , CID 1089 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：480/531 ( 90.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，无

输入要点：无

输出要点：无

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

- 确认题目没有输入，程序无需声明或读取测试数据。
- 按题面给定关系完成常数计算：历经千辛万苦，只为得到你 签到题，直接输出即可。
- 使用 cout 原样输出结果；核对字符、标点和末尾换行。
- 该程序不依赖输入规模，不需要循环或额外数据结构。

### 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。



- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     printf("AC!!!\n");
4 |     return 0;
5 | }
```

### 公开样例

样例 1 输入

无

样例 1 输出

AC!!!

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 1.96 | 总 96/526 卡萨的阻拦 ( PID 1856 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（D，CID 1090）| 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：2/50 ( 4.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，元素能量和

输入要点：输入  $n$ ， $n \leq 1000000$ 。

输出要点：元素能量和

题面提示：2 是 1 的倍数

3 是 1 的倍数

4 是 1 和 2 的倍数

5 是 1 的倍数

6 是 1，2，3 的倍数

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3. 核心处理采用：卡萨的阻拦 中等题 直接计算约数的耗时较大，应该会超时，考虑到一个比较小的数会是多个数的约数，所以直接计算这个数会被计算多少次。对于输入的 $n$ ，对于某个 $i$ ， $i \leq n$ ，一共有 $n/i$ 个数有 $i$ 这个约数，其中包括 $i$ 本身所以要减去1，所以单个 $i$ 的贡献为 $i * (n/i - 1)$ ，循环求和即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | void solve() {
5 |     int n;
6 |     cin >> n;
7 |     ll ans = 0;
8 |     for (int i = 1; i <= n; ++i) {
9 |         ans = ans + i * (n / i - 1);
10 |     }
11 |     cout << ans;
12 | }
13 | int main() {
14 |     ios::sync_with_stdio(false);
15 |     cin.tie(0);
16 |     cout.tie(0);
17 |     int test = 1;
18 |     // cin >> test;
19 |     while (test--) {
20 |         solve();
21 |     }
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

6

样例 1 输出

12

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 1.97 | 总 97/526 我要看奥运！！！（PID 1862）

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（B，CID 1091） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：229/1447（15.8%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，巴黎时间

输入要点：北京时间

输出要点：巴黎时间

题面提示：输入

21:05

输出

15:05

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：我要看奥运！！！注意时间跨天以及scanf和printf的使用
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int hh, mm;
5 |     scanf("%d:%d", &hh, &mm);
6 |     hh = hh + 18;
7 |     hh %= 24;
8 |     printf("%02d:%02d", hh, mm);
9 | }
```

## 公开样例

样例 1 输入

16:40

样例 1 输出

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.98 | 总 98/526 ReLU 函数 ( PID 1863 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( H , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( A , CID 1092 )  
| 训练定位：进阶 | 入门语法、输入输出与格式

标签：条件分支、分段函数 | 限制：1.000 S / 128 MB | 历史统计：646/981 ( 65.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个数字：x 经过 f(x) 计算后的结果。

输入要点：输入一个整数 x ( $-10^4 \leq x \leq 10^4$ )，代表传递给 f(x) 的参数。

输出要点：输出一个数字：x 经过 f(x) 计算后的结果。

题面提示：样例输入 2

-1

样例输出 2

0

样例输入 3

1

样例输出 3

1

样例输入 4

2

样例输出 4

2

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：ReLU 定义为  $\max(0,x)$ ：x≤0 时输出 0，否则输出 x。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(1) 时间、O(1) 空间

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int x;
5 |     cin >> x;
6 |     cout << max(0, x) << '\n';
7 |     return 0;
8 | }
```

公开样例

样例 1 输入

-2

样例 1 输出

0

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1
- 公开题解/参考来源 2
- 题面图片 1

1.99 | 总 99/526 进位 ( PID 1870 )

出处：河南工大2024新生周赛（2）——命题人：马家硕（G，CID 1092） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：高精度减法、数位、进位 | 限制：1.000 S / 128 MB | 历史统计：31/334（9.3%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：寻找一个最小的数  $x$ ，使  $num+x$  刚好比  $num$  的位数多一位。

输入要点：第一行输入整数  $n$ 。

第二行从高位到低位依次输入  $num$  每一位上的数字，它们之间都有空格作为分隔。

输出要点：输出你找到的数  $x$ 。

题面提示：输入数字为 23567，计算出答案  $x$  为 76433。下面是计算过程：

23567 共 5 位， $23567+76433=100000$ 。

100000 共 6 位，刚好比 23567 多一位。

所以答案是 76433。

## 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：目标是  $x=10^n - \text{num}$ 。直接把  $10^n$  写成“1 后跟 n 个 0”，再从 低位到高位执行带借位减法；这样即使 num 的末位是 0，也不会出现把数值 10 错当成一个十进制字符的问题。最后删除前导 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> digit(n);
9 |     for (int& value : digit)
10 |         cin >> value;
11 |     string minuend = "1" + string(n, '0');
12 |     string ans(n+1, '0');
13 |     int borrow = 0;
14 |     for (int i = n; i >= 0; --i) {
15 |         int upper = minuend[i] - '0' - borrow;
16 |         int lower = (i == 0 ? 0 : digit[i-1]);
17 |         if (upper < lower) {
18 |             upper += 10;
19 |             borrow = 1;
20 |         } else {
21 |             borrow = 0;
22 |         }
23 |         ans[i] = char('0' + upper - lower);
24 |     }
25 |     size_t first = ans.find_first_not_of('0');
26 |     if (first == string::npos)
27 |         cout << "0\n";
28 |     else
29 |         cout << ans.substr(first) << '\n';
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

|                |
|----------------|
| 5<br>2 3 5 6 7 |
| 样例 1 输出        |
| 76433          |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.100 | 总 100/526 圆 ( PID 1908 )

出处：河南工大2024新生周赛 ( 7 ) ——命题人：刘义 ( A , CID 1098 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：35/257 ( 13.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：对于每组询问，给定一个整数  $n$ ，求  $n$  个圆 ( 半径可以不同 ) 可以分割的最大区域数为多少？

输入要点：第一行包含一个正整数  $t(1 \leq t \leq 103)$ -----测试用例的数量。

接下来一行， $t$  个用空格隔开的整数  $n(0 \leq n \leq 106)$ ，依次表示每组询问给定的圆形个数。

输出要点：一行，对于每次询问输出一个整数表示结果，每个整数之间用空格隔开

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：圆的个数， $b$  为区域数  $\text{cin} >> n$ ; while( $n--$ ) {  $\text{cin} >> a$ ; if( $a==0$ )  $b=1$ ; else  $b=a*a-a+2$ ;  $\text{cout} << b << " "$ ; } return 0; } 问题 B
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
```

```
3 | #define int long long
4 | int n;
5 | signed main() {
6 |     int a, b; // a为圆的个数, b为区域数
7 |     cin >> n;
8 |     while (n--) {
9 |         cin >> a;
10 |         if (a == 0)
11 |             b = 1;
12 |         else
13 |             b = a * a - a + 2;
14 |         cout << b << " ";
15 |     }
16 |     return 0;
17 | }
```

### 公开样例

样例 1 输入

```
4
0 1 2 3
```

样例 1 输出

```
1 2 4 8
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 1.101 | 总 101/526 魔法天平的反向称重 ( PID 1927 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( J , CID 1104 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：392/1032 ( 38.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，代表交换后的数字。

输入要点：一个整数 $n$ ， $0 \leq n \leq 999$ 。

输出要点：一个整数，代表交换后的数字。

题面提示：签到题

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：魔法天平的反向称重 注意0的位置。 `#include <stdio.h> #include <stdlib.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明



- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

常数级或一次线性读入；以代码中的循环层数为准

易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     int a = 0, b = 0, c = 0;
7 |     a = n % 10; // n 的个位
8 |     n /= 10;
9 |     b = n % 10; // n 的十位
10 |    n /= 10;
11 |    c = n; // n 的百位
12 |    printf("%d%d%d", a, b, c);
13 |    return 0;
14 | }
```

公开样例

|         |
|---------|
| 样例 1 输入 |
| 123     |
| 样例 1 输出 |
| 321     |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

1.102 | 总 102/526 日历格式化输出 ( PID 1931 )

出处：河南工大2025新生周赛（3）——命题人：路耀华（I，CID 1104） | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：100/436 ( 22.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，符合要求的日历。

输入要点：一行两个整数，分别表示 w 和 d ( 0≤w≤6 , 28≤d≤31 )。

输出要点：符合要求的日历。

题面提示：仔细观察样例中日期的对齐方式。

零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：日历格式化输出 注意日期起始位置，每个日期占两格，日期与日期之间有空格。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int w, d;
5 |     scanf("%d %d", &w, &d);
6 |     printf("日 一 二 三 四 五 六\n");
7 |     int p = (w + d + 6) / 7, id = 1, f = 0;
8 |     for (int i = 0; i < p; i++) {
9 |         for (int j = 0; j < 7; j++) {
10 |             if (i == 0 && j < w)
11 |                 printf(" ");
12 |             else {
13 |                 printf("%2d", id);
14 |                 id++;
15 |             }
16 |             if (id > d) {
17 |                 f = 1;
18 |                 break;
19 |             }
20 |             if (j < 6)
21 |                 printf(" ");
22 |         }
23 |         if (f == 1)
24 |             break;
25 |         printf("\n");
26 |     }
27 |     return 0;
28 | }
```

## 公开样例

样例 1 输入

0 31

样例 1 输出

日 一 二 三 四 五 六  
1 2 3 4 5 6 7  
8 9 10 11 12 13 14  
15 16 17 18 19 20 21  
22 23 24 25 26 27 28  
29 30 31

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

1.103 | 总 103/526 小唐的饼干 ( PID 1935 )

出处：河南工大2025新生周赛 ( 2 ) ——命题人：王伟立 ( G , CID 1103 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：条件分支 | 限制：1.000 S / 128 MB | 历史统计：198/990 ( 20.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出小唐原来有多少瓶汽水。答案可能很大，对114取余。

输入要点：输入一个正整数 n，表示天数。

输出要点：输出小唐原来有多少瓶汽水。

答案可能很大，对114取余。

题面提示：数据保证， $1 < n < 10000$ 。

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的饼干 从第n天，每往前一天需要先加二再乘二，注意每中间数可能会很大，每一步运算都需要对114取模。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int const mod = 114;
4 | signed main() {
5 |     int n;
6 |     cin >> n;
7 |     n--;
8 |     int re = 1;
9 |     while (n--)
10 |         re = ((re + 2) * 2) % mod;
11 |     cout << re;
12 |     return 0;
}
```

## 公开样例

样例 1 输入

```
4
```

样例 1 输出

```
36
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.104 | 总 104/526 小唐的升级 ( PID 1937 )

出处：河南工大2025新生周赛 ( 2 ) ——命题人：王伟立 ( H , CID 1103 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：基础实现 | 限制：1.000 S / 128 MB | 历史统计：248/484 ( 51.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示提升的级数。

输入要点：三个整数， $x, y, n$ 。表示一级需要的经验，每局获得的经验数，打的局数。

输出要点：一个整数，表示提升的级数。

题面提示：保证测试数据均在int范围内

### 零基础先修

- 目标是把题意准确翻译成顺序、分支和循环，并做到输出字符完全一致。校队训练的第一道门槛不是复杂算法，而是让所有本应拿到的分稳定 AC。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的升级 先计算获得的经验总和，在计算提升的等级。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。

- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int x, y, n;
5 |     cin >> x >> y >> n;
6 |     int m = y * n;
7 |     int re = 0;
8 |     while (m >= x) {
9 |         m -= x;
10 |        x *= 2;
11 |        re++;
12 |    }
13 |    cout << re << endl;
14 |    return 0;
15 | }
```

### 公开样例

样例 1 输入

1 100 100

样例 1 输出

13

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 1.105 | 总 105/526 编程大蛇dream ( PID 1948 )

出处：河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇 ( A , CID 1105 ) | 训练定位：进阶 | 入门语法、输入输出与格式

标签：固定输出与格式、原始字符串 | 限制：1.000 S / 128 MB | 历史统计：249/1660 ( 15.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，直接输出样例中的代码

输入要点：无

输出要点：直接输出样例中的代码

题面提示：注意格式和中英文符号哦，先在自己的编译器成功输出可以减少编译错误。每个空格需要对照

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。

### 思路推导

1. 确认题目没有输入，程序无需声明或读取测试数据。
2. 按题面给定关系完成常数计算：输出内容包含大量易混淆的全角符号。使用自定义分隔符的原始字符串能避免逐个转义；其中样例里的反斜杠必须作为一个真实字符保留。
3. 使用 cout 原样输出结果；核对字符、标点和末尾换行。
4. 该程序不依赖输入规模，不需要循环或额外数据结构。

## 正确性说明

- 题目没有输入，因此所有合法运行面对的是同一个固定任务。
- 程序输出的常量与题面要求的结果逐字一致。
- 所以程序对全部可能运行都给出正确答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << R"HAUT(#《includue std1o。h)
5 |     1nt man ( ){
6 |     print ( "holle word\n");
7 |     retrun o :
8 | } )HAUT";
9 |     return 0;
10 | }
```

## 公开样例

样例 1 输入

无

样例 1 输出

```
#《includue std1o。h)
1nt man ( ){
    print ( "holle word\n");
    retrun o :
}
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“入门语法、输入输出与格式”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 模块 2 | 模拟、枚举与构造

模拟要求程序状态与题面过程一一对应；枚举要求证明没有漏解；构造则要同时说明输出满足全部约束。练习时必须手推小样例并写清不  
变量。

本模块共 22 道唯一题。

### 2.1 | 总 106/526 迷宫 ( PID 1429 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( B , CID 1047 ) | 训练定位：入门 | 模拟、枚举与构造

标签：模拟、图形输出、二维坐标 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：你作为一个迷宫构建师。你的任务就是生成符合要求的迷宫。

输入要点：第一行输入三个整数：n, m, q。 ( $1 \leq n \leq 50, 1 \leq m \leq 50, 0 \leq q \leq 2 * n * m - n - m$ )

接下来的q行,每一行输入一个k, ch ( $1 \leq k \leq n * m, ch \in \{'W', 'R'\}$ )

输出要点：生成符合要求的迷宫

#### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

#### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：先把每个要求转换为“该房间右墙是否开放”和“下墙是否开放”的布尔标记，重复要求不会产生额外影响。逐行输出：房间内部行默认右边是 |，R 开口改为空格；每行下边默认 ---，W 开口改成三个空格。外围没有开口由题目保证。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

#### 复杂度

$O(nm+q)$  时间、 $O(nm)$  空间

#### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

#### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n, m, q;
5 |     cin >> n >> m >> q;
6 |     vector<bool> openRight(n * m + 1, false), openDown(n * m + 1, false);
7 |     while (q--) {
8 |         int room;
9 |         char dir;
10 |         cin >> room >> dir;
11 |         if (dir == 'R')
12 |             openRight[room] = true;
13 |         else
14 |             openDown[room] = true;
15 |     }
16 |     cout << string(m, '+').substr(0, 0);
```

```

17 |     for (int column = 0; column < m; ++column)
18 |         cout << "+---";
19 |     cout << "+\n";
20 |     for (int row = 0; row < n; ++row) {
21 |         cout << '|';
22 |         for (int column = 0; column < m; ++column) {
23 |             int room = row * m + column + 1;
24 |             cout << " " << (openRight[room] ? ' ' : '|');
25 |         }
26 |         cout << '\n';
27 |         for (int column = 0; column < m; ++column) {
28 |             int room = row * m + column + 1;
29 |             cout << '+' << (openDown[room] ? " " : "----");
30 |         }
31 |         cout << "+\n";
32 |     }
33 |     return 0;
34 | }

```

## 公开样例

### 样例 1 输入

```

4 6 6
16 W
11 R
11 R
16 W
1 R
22 R

```

### 样例 1 输出

```

+--+--+--+--+--+--+
|  |  |  |  |  |
+--+--+--+--+--+--+
|  |  |  |  |  |
+--+--+--+--+--+--+
|  |  |  |  |  |
+--+--+--+  +--+--+
|  |  |  |  |  |
+--+--+--+--+--+--+

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.2 | 总 107/526 欢天喜地 ( PID 1481 )

出处：2019级河南工大新生周赛 ( 二 ) @邹岩专场 ( B , CID 1054 ) | 训练定位：入门 | 模拟、枚举与构造

标签：模拟、循环、增长率 | 限制：1.000 S / 128 MB | 历史统计：378/619 ( 61.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：【欢】想成为最大的熊，或者至少比他的朋友ZP更大。如果达成目的【欢】会非常开心。

输入要点：你将输入两个数a,b(1<=a,b<=10)分别是【欢】和ZP的体重。

输出要点：打印一个整数，表示整数年后【欢】将变得大于ZP。

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。



## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：每过一年，欢的体重乘 3，ZP 的体重乘 2。循环到  $a > b$  为止，循环次数就是完整年数；等于还不算“更重”，所以条件必须是  $a \leq b$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(\text{答案年数})$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, b, years = 0;
5 |     cin >> a >> b;
6 |     while (a <= b) {
7 |         a *= 3;
8 |         b *= 2;
9 |         ++years;
10 |    }
11 |    cout << years << '\n';
12 |    return 0;
13 | }
```

## 公开样例

样例 1 输入

4 9

样例 1 输出

3

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.3 | 总 108/526 寻欢作乐 (PID 1483)

出处：2019级河南工大新生周赛 (二) @邹岩专场 (D, CID 1054) | 训练定位：入门 | 模拟、枚举与构造

标签：模拟、数位、循环 | 限制：1.000 S / 128 MB | 历史统计：289/339 (85.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数表示整数  $n$  经过  $k$  次操作后变成的值。

输入要点：输入两个数 $n, k(2 \leq n \leq 10^9, 1 \leq k \leq 50)$ ,  $n$ 为看门人LJY给出的数， $k$ 为操作次数。

输出要点：输出一个整数表示整数 $n$ 经过 $k$ 次操作后变成的值。

题面提示：对于样例 $512 \rightarrow 511 \rightarrow 510 \rightarrow 51 \rightarrow 50$

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：重复  $k$  次：末位为 0 就除以 10，否则减 1。题目描述里的“含有零”实际由样例可知是“最后一位为零”。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(k)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     int k;
6 |     cin >> n >> k;
7 |     while (k--) {
8 |         if (n % 10 == 0)
9 |             n /= 10;
10 |        else
11 |            --n;
12 |    }
13 |    cout << n << '\n';
14 |    return 0;
15 | }
```

## 公开样例

样例 1 输入

512 4

样例 1 输出

50

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.4 | 总 109/526 旅行 ( PID 1548 )

出处：2020级HAUT新生周赛（三）@张雍薛专场（B，CID 1062）| 训练定位：入门 | 模拟、枚举与构造

标签：枚举、全排列、旅行商模型 | 限制：1.000 S / 128 MB | 历史统计：2/4 ( 50.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：lcl从城市1出发，他的目标是除了出发的城市外，每个城市都到达并且只到达一次，在最后回到出发的城市，并且恰好用完预算，求一共有多少种不同的旅行方案

输入要点：第一行两个整数 $n, W$ 代表共有 $n$ 个城市和lcl的预算 $W$

接下来给出一个 $n \times n$ 的矩阵 $w$ ， $w[i][j]$ 代表城市 $i$ 到城市 $j$ 的花费

$1 \leq n \leq 8$

$1 \leq W \leq 1e9$

如果 $i = j$

$w[i][j] = 0$

否则

$1 \leq w[i][j] = w[j][i] \leq 1e8$

输出要点：一个整数，代表满足条件的方案数

题面提示：满足条件的路径

1 -> 2 -> 3 -> 4 -> 5 -> 1

1 -> 2 -> 5 -> 3 -> 4 -> 1

1 -> 4 -> 3 -> 5 -> 2 -> 1

1 -> 5 -> 4 -> 3 -> 2 -> 1

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
- 核心处理采用：起点固定为城市 1，枚举其余  $n-1$  个城市的所有排列。对每个排列 累加 1→排列各点→1 的完整环路费用，恰等于  $W$  就计数。方向相反但 访问顺序不同的路线按题目算不同方案，会由排列自然分别统计。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

$O((n-1)! \cdot n)$  时间、 $O(n^2)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long budget;
8 |     cin >> n >> budget;
9 |     vector<vector<long long>> cost(n, vector<long long>(n));
10 |     for (auto& row : cost)
11 |         for (long long& x : row)
12 |             cin >> x;
13 |     vector<int> order;
14 |     for (int city = 1; city < n; ++city)
15 |         order.push_back(city);
16 |     long long ans = 0;
17 |     do {
18 |         long long total = 0;
19 |         int pre = 0;
20 |         for (int city : order) {
21 |             total += cost[pre][city];
22 |             pre = city;
23 |         }
24 |         total += cost[pre][0];
25 |         if (total == budget)
26 |             ++ans;
27 |     } while (next_permutation(order.begin(), order.end()));
28 |     cout << ans << '\n';
29 |     return 0;
30 | }
```

## 公开样例

样例 1 输入

```
5 29
0 5 9 2 7
5 0 2 10 6
9 2 0 6 10
2 10 6 0 9
7 6 10 9 0
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.5 | 总 110/526 约瑟夫环 ( PID 1223 )

出处：2016级河南工大新生周赛（四）（D，CID 1009） | 训练定位：基础提高 | 模拟、枚举与构造

标签：约瑟夫环、递推、模运算 | 限制：1.000 S / 128 MB | 历史统计：38/87 ( 43.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，最后剩下的人的编号

输入要点：2个数n和k，表示n个人，数到k的出列

输出要点：最后剩下的人的编号

## 零基础先修

- 模拟要求程序状态与题面过程一一对应；枚举要求证明没有漏解；构造则要同时说明输出满足全部约束。练习时必须手推小样例并写清不变量。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：用 0 起编号。设 survivor 表示当前 i-1 个人时的最终幸存下标，扩展到 i 个人后，首个被删位置带来的编号旋转使新下标为  $(\text{survivor}+k)\%i$ 。从  $i=1$ 、 $\text{survivor}=0$  递推到 n，最后加 1 转回题目的 1 起编号。无需真正模拟每次出列。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long n, k;
7 |     cin >> n >> k;
8 |     long long survivor = 0;
9 |     for (long long people = 2; people <= n; ++people)
10 |         survivor = (survivor + k) % people;
11 |     cout << survivor + 1 << '\n';
12 |     return 0;
13 | }
```

## 公开样例

样例 1 输入

3 2

样例 1 输出

3

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

## 2.6 | 总 111/526 消灭怪物 ( PID 1342 )

出处：2017级河南工大新生周赛 ( 六 ) ( B , CID 1035 ) | 训练定位：基础提高 | 模拟、枚举与构造

标签：构造、下界证明、观察 | 限制：2.000 S / 256 MB | 历史统计：25/113 ( 22.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你要找出消灭所有怪物所需的最小炸弹数。

输入要点：第一行：整数  $T$ ，表示测试实例个数。

对于每组测试实例：

输入一个整数  $n$  ( $2 \leq n \leq 100\,000$ ) ——表示有  $n$  个格子。

输出要点：每组测试实例输出一行：包括一个整数——消灭所有怪物所需的最小炸弹数。

### 零基础先修

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：每个格子的原生怪物都至少要先受一次伤，因此每个格子至少要投一颗，共  $n$  颗。最后一次首次受伤的那批怪物还会移动到相邻格，它们必须再被炸一次，所以至少还需一颗，下界为  $n+1$ 。按格子从一端依次投弹，前一格移来的怪物会在当前格受到第二击；到末端后再在相邻格补一颗即可达到  $n+1$ ，所以下界可达。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long n;
10 |         cin >> n;
11 |         cout << n + 1 << '\n';
12 |     }
13 |     return 0;
14 | }
```

### 公开样例

样例 1 输入

```
2
2
3
```

样例 1 输出

```
3
4
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

2.7 | 总 112/526 时光跑得太快，溜了溜了 ( PID 1358 )

出处：2017级河南工大新生周赛（八）（G，CID 1037） | 训练定位：基础提高 | 模拟、枚举与构造

标签：日期、模拟、格式化输出、闰年 | 限制：1.000 S / 128 MB | 历史统计：2/9 ( 22.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，每对测试样例，输出格式如下。2017 年 12 月 MON TUE WED THU FRI SAT SUN 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31

输入要点：多实例输入。

每行两个正整数，Year，Month。

2015 <= Year <= 2018。

1 <= Month <= 12。

输出要点：每对测试样例，输出格式如下。

2017 年 12 月

MON TUE WED THU FRI SAT SUN

1 2 3

4 5 6 7 8 9 10

11 12 13 14 15 16 17

18 19 20 21 22 23 24

25 26 27 28 29 30 31

题面提示：MON 和 TUE 之间只有一个空格。

零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：以星期一为下标 0。已知 2015-01-01 是星期四（下标 3），从 2015 年 1 月累加到目标月即可得到该月 1 日的星期。闰年按“能被 400 整除，或能被 4 整除但不能被 100 整除”判断。每个日期左对齐占 4 个字符：先输出 weekday×4 个空格，再逐日输出，星期日或月末换行。多组月份之间没有额外空行。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每个询问  $O(\text{年差} + \text{月数} + \text{当月天数})$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | bool leapYear(int year) {
4 |     return year % 400 == 0 || (year % 4 == 0 && year % 100 != 0);
5 | }
6 | int daysInMonth(int year, int month) {
7 |     static const int days[] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
8 |     if (month == 2 && leapYear(year))
9 |         return 29;
10 |    return days[month];
11 | }
12 | int first(int year, int month) {
13 |     int weekday = 3;
14 |     for (int y = 2015; y < year; ++y)
15 |         weekday = (weekday + (leapYear(y) ? 366 : 365)) % 7;
16 |     for (int m = 1; m < month; ++m)
17 |         weekday = (weekday + daysInMonth(year, m)) % 7;
18 |     return weekday;
19 | }
20 | int main() {
21 |     ios::sync_with_stdio(false);
22 |     cin.tie(nullptr);
23 |     int year, month;
24 |     while (cin >> year >> month) {
25 |         cout << year << "年" << month << "月\n";
26 |         cout << "MON TUE WED THU FRI SAT SUN\n";
27 |         int weekday = first(year, month);
28 |         cout << string(weekday * 4, ' ');
29 |         int totalDays = daysInMonth(year, month);
30 |         for (int day = 1; day <= totalDays; ++day) {
31 |             cout << left << setw(4) << day;
32 |             if (weekday == 6 || day == totalDays)
33 |                 cout << '\n';
34 |             weekday = (weekday + 1) % 7;
35 |         }
36 |     }
37 |     return 0;
38 | }
```

## 公开样例

### 样例 1 输入

```
2017 12
2015 1
```

### 样例 1 输出

```
2017 年 12 月
MON TUE WED THU FRI SAT SUN
    1  2  3
4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30 31
2015 年 1 月
MON TUE WED THU FRI SAT SUN
```



```

    1 2 3 4
  5 6 7 8 9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.8 | 总 113/526 体育课 (二) (PID 1387)

出处：2018级河南工大新生周赛 (一) @徐向阳专场 (F, CID 1040) | 训练定位：基础提高 | 模拟、枚举与构造

标签：预处理、回文数、数位和、枚举 | 限制：1.000 S / 128 MB | 历史统计：17/63 (27.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对每行输入，从小到大输出其对应的一组正整数  $M$ ，如果没有对应的正整数  $M$ ，输出“无”（不含引号），第  $i$  组输出前要加上“Testi:”，具体样式见样例输出。（ $M$  的各位之和要等于  $D$ ，且要求从左边读和从右边读是一样的， $10000 \leq M < 1000000$ ，例如  $M=10001$ ， $M$  的各位之和为  $1+0+0+0+1$  为 2，从左边读  $M: 10001$ ，……

输入要点：第一行一个正整数  $N$  ( $0 < N \leq 18$ )，接下来  $N$  行，每行一个正整数  $D$  ( $0 < D \leq 54$ )

输出要点：对每行输入，从小到大输出其对应的一组正整数  $M$ ，如果没有对应的正整数  $M$ ，输出“无”（不含引号），第  $i$  组输出前要加上“Testi:”，具体样式见样例输出。（ $M$  的各位之和要等于  $D$ ，且要求从左边读和从右边读是一样的， $10000 \leq M < 1000000$ ，例如  $M=10001$ ， $M$  的各位之和为  $1+0+0+0+1$  为 2，从左边读  $M: 10001$ ，从右边读也是：10001）

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
3. 核心处理采用：先一次性枚举 10000 到 999999，判断十进制正读、反读是否相同，再按数位和把所有回文数分组。每次询问只需输出对应分组；组为空时输出“无”。预处理保证数字天然按从小到大排列。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

$O(10^6 + \text{答案总量})$  时间、 $O(\text{回文数总量})$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | bool pal(int number) {
4 |     int original = number, reversed = 0;
5 |     while (number > 0) {
6 |         reversed = reversed * 10 + number % 10;
7 |         number /= 10;
8 |     }
9 |     return original == reversed;
10 | }
11 | int digitSum(int number) {
12 |     int sum = 0;
13 |     while (number > 0) {
14 |         sum += number % 10;
15 |         number /= 10;
16 |     }
17 |     return sum;
18 | }
19 | int main() {
20 |     ios::sync_with_stdio(false);
21 |     cin.tie(nullptr);
22 |     vector<vector<int>> bySum(55);
23 |     for (int number = 10000; number < 1000000; ++number) {
24 |         if (pal(number))
25 |             bySum[digitSum(number)].push_back(number);
26 |     }
27 |     int T;
28 |     cin >> T;
29 |     for (int testCase = 1; testCase <= T; ++testCase) {
30 |         int targetSum;
31 |         cin >> targetSum;
32 |         cout << "Test" << testCase << ":\n";
33 |         if (bySum[targetSum].empty()) {
34 |             cout << "无\n";
35 |         } else {
36 |             for (int number : bySum[targetSum])
37 |                 cout << number << '\n';
38 |         }
39 |     }
40 |     return 0;
41 | }
```

## 公开样例

### 样例 1 输入

```
3
52
2
1
```

### 样例 1 输出

```
Test1:
899998
989989
998899
Test2 :
10001
100001
Test3:
无
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 2.9 | 总 114/526 three numbers and two operators ( PID 1400 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( E , CID 1042 ) | 训练定位：基础提高 | 模拟、枚举与构造

标签：枚举、全排列、表达式 | 限制：1.000 S / 128 MB | 历史统计：53/178 ( 29.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，Each group outputs the maximum and minimum values, separated by spaces in the middle, and each group takes one row.

输入要点：Each group of data consists of one row, including three integers until the end of input 0 0 0.

输出要点：Each group outputs the maximum and minimum values, separated by spaces in the middle, and each group takes one row.

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
- 核心处理采用：三个数可以任意换序，两个符号分别是一个加号和一个乘号。枚举 三数的 6 种排列，并计算  $x+y*z$  与  $x*y+z$  两种符号位置，更新全局 最大最小值；乘法按通常优先级先算。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     array<long long, 3> a;
5 |     while (cin >> a[0] >> a[1] >> a[2] && (a[0] || a[1] || a[2])) {
6 |         sort(a.begin(), a.end());
7 |         long long mn = LLONG_MAX, mx = LLONG_MIN;
8 |         do {
9 |             long long first = a[0] + a[1] * a[2];
10 |             long long second = a[0] * a[1] + a[2];
11 |             mn = min({mn, first, second});
12 |             mx = max({mx, first, second});
13 |         } while (next_permutation(a.begin(), a.end()));
14 |         cout << mx << ' ' << mn << '\n';
15 |     }
16 |     return 0;
17 | }
```

### 公开样例

样例 1 输入

```
1 2 3
1 1 1
0 0 0
```

样例 1 输出

7 5  
2 2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.10 | 总 115/526 Gaming ( PID 1420 )

出处：2018级河南工大新生周赛总决赛（重现）（D，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（D，CID 1048） | 训练定位：基础提高 | 模拟、枚举与构造

标签：模拟、二进制、淘汰赛 | 限制：1.000 S / 32 MB | 历史统计：12/41 ( 29.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每组测试数据，输出IG和FNC会在第几回合相遇 特殊的，如果他们能在总决赛相遇，输出"Final!"

输入要点：先输入一个T表示T组测试数据 (  $T \leq 50$  )

每组测试数据输入三个数n, a, b表示总共有n支队伍参赛 (  $2 \leq n \leq 1024$ ，且n为2的次方数 )，IG的队伍编号为a，FNC的队伍编号为b (  $2 \leq a, b \leq n$ ， $a < b$  )

输出要点：对于每组测试数据，输出IG和FNC会在第几回合相遇

特殊的，如果他们能在总决赛相遇，输出"Final!"

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：每结束一轮，相邻的两个编号区间会合并。把队伍编号先转成从 0 开始，每轮都除以 2；a 与 b 第一次变成同一个编号时，就是它们相遇的轮次。总决赛轮数为  $\log_2(n)$ ，若相遇轮次等于它便输出 Final!。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

每组  $O(\log n)$  时间、 $O(1)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, a, b;
10 |         cin >> n >> a >> b;
11 |         --a;
12 |         --b;
13 |         int round = 0;
14 |         while (a != b) {
15 |             a /= 2;
16 |             b /= 2;
17 |             ++round;
18 |         }
19 |         int finalRound = 0;
20 |         for (int teams = n; teams > 1; teams /= 2)
21 |             ++finalRound;
22 |         if (round == finalRound)
23 |             cout << "Final!\n";
24 |         else
25 |             cout << round << '\n';
26 |     }
27 |     return 0;
28 | }

```

## 公开样例

### 样例 1 输入

```

3
4 1 2
8 5 7
4 1 3

```

### 样例 1 输出

```

1
2
Final!

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.11 | 总 116/526 Minecraft ( PID 1423 )

出处：2018级河南工大新生周赛总决赛（重现）（G，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（G，CID 1048） | 训练定位：基础提高 | 模拟、枚举与构造

标签：网格模拟、四方向、单调过程 | 限制：2.000 S / 64 MB | 历史统计：19/64 ( 29.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：求无限长的时间之后，是否整个  $n \times m$  的区域都会被水布满

输入要点：先输入一个  $T$  表示  $T$  组测试数据（ $T \leq 100$ ）

每组测试数据输入两个正整数  $n, m$ （ $n, m \leq 8$ ）

接下来  $n$  行，每行  $m$  个数，只会为 0 或 1，为 1 说明当前位置是水，为 0 表示空地

输出要点：如果无限长时间后  $n \times m$  个方块都会变成水方块，输出 "YES"，否则输出 "NO"

题面提示：对于第一个样例，2 秒之后整个区域就会被水充满了

## 零基础先修

- 模拟要求程序状态与题面过程一一对应；枚举要求证明没有漏解；构造则要同时说明输出满足全部约束。练习时必须手推小样例并写清不变量。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：水只会增加不会消失，因此可以逐秒扫描：本轮把所有“当前至少有两个 四邻水格”的空地记下来，扫描完再统一变水，保证符合题目的同步更新。如果某轮没有新增格子便达到稳定状态；此时检查是否还存在 0。格子最多 64 个，所以最多发生 64 轮有效变化。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组最坏  $O((nm)^2)$  时间、 $O(nm)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     const int dx[4] = {-1, 1, 0, 0};
9 |     const int dy[4] = {0, 0, -1, 1};
10 |    while (T--) {
11 |        int n, m;
12 |        cin >> n >> m;
13 |        vector<vector<int>> water(n, vector<int>(m));
14 |        for (auto& row : water)
15 |            for (int& cell : row)
16 |                cin >> cell;
17 |        while (true) {
18 |            vector<pair<int, int>> added;
19 |            for (int i = 0; i < n; ++i) {
20 |                for (int j = 0; j < m; ++j) {
21 |                    if (water[i][j])
22 |                        continue;
23 |                    int neighbors = 0;
24 |                    for (int d = 0; d < 4; ++d) {
25 |                        int x = i + dx[d], y = j + dy[d];
26 |                        if (0 <= x && x < n && 0 <= y && y < m)
27 |                            neighbors += water[x][y];
28 |                    }
29 |                    if (neighbors >= 2)
30 |                        added.push_back({i, j});
31 |                }
32 |            }
33 |            if (added.empty())
34 |                break;
35 |            for (auto [i, j] : added)
36 |                water[i][j] = 1;
37 |        }
38 |        bool full = true;
39 |        for (const auto& row : water)
40 |            for (int cell : row)
41 |                full &= (cell == 1);
42 |        cout << (full ? "YES" : "NO") << '\n';
43 |    }
44 |    return 0;
45 | }
```

公开样例

样例 1 输入

```
2
5 5
1 0 1 0 1
1 1 0 1 1
0 0 0 0 0
1 1 1 1 1
1 1 1 1 0
4 4
1 1 1 1
0 0 0 0
0 0 0 0
1 1 1 1
```

样例 1 输出

```
YES
NO
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

2.12 | 总 117/526 成龙AC记 ( PID 1490 )

出处：2019级河南工大新生周赛 ( 三 ) @邓祎培专场 ( E , CID 1055 ) | 训练定位：基础提高 | 模拟、枚举与构造

标签：模拟、条件分支、时间计算 | 限制：1.000 S / 128 MB | 历史统计：65/98 ( 66.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一个数字，表示此次成龙上学所用的时间。

输入要点：输入的第一行包含空格分隔的三个正整数  $r$ 、 $y$ 、 $g$ ，表示红绿灯的设置。这三个数均不超过  $10^6$  次方。

输入的第二行包含一个正整数  $n$  (  $n \leq 100$  )，表示成龙总共经过的道路段数和看到的红绿灯数目。

接下来的  $n$  行，每行包含空格分隔的两个整数  $k$ 、 $t$ 。 $k=0$  表示经过了一段道路，耗时  $t$  秒，此处  $t$  不超过  $10^6$  次方； $k=1$ 、 $2$ 、 $3$  时，

分别表示看到了一个红灯、黄灯、绿灯，且倒计时显示牌上显示的数字是  $t$ ，此处  $t$  分别不会超过  $r$ 、 $y$ 、 $g$ 。

输出要点：输出一个数字，表示此次成龙上学所用的时间。

题面提示：成龙先经过第一段道路，用时 10 秒，然后等待 5 秒的红灯，再经过第二段道路，用时 11 秒，然后等待 2 秒的黄灯和 30 秒的红灯，再经过第三段、第四段

道路，分别用时6、3秒，然后通过绿灯，再经过最后一段道路，用时 3 秒。共计  $10 + 5 + 11 + 2 + 30 + 6 + 3 + 3=70$  秒。

零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- `if/else` 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。

- 核心处理采用：记录已经是到达路口时的灯色和倒计时，不必再模拟此前路程造成的 相位变化。道路  $k=0$  加  $t$ ；红灯  $k=1$  等  $t$ ；黄灯  $k=2$  还要等完当前 黄灯及下一段红灯，即  $t+r$ ；绿灯  $k=3$  直接通过。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long red, yellow, green;
5 |     int n;
6 |     cin >> red >> yellow >> green >> n;
7 |     long long ans = 0;
8 |     while (n--) {
9 |         int type;
10 |         long long time;
11 |         cin >> type >> time;
12 |         if (type == 0 || type == 1)
13 |             ans += time;
14 |         else if (type == 2)
15 |             ans += time + red;
16 |     }
17 |     cout << ans << '\n';
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
30 3 30
8
0 10
1 5
0 11
2 2
0 6
0 3
3 10
0 3
```

样例 1 输出

```
70
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)



## 2.13 | 总 118/526 zp与车费 ( PID 1508 )

出处：2019级河南工大新生周赛 ( 五 ) @潘世泽专场 ( B , CID 1057 ) | 训练定位：基础提高 | 模拟、枚举与构造

标签：递推、斐波那契、循环 | 限制：1.000 S / 128 MB | 历史统计：161/395 ( 40.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：zp要在新大陆的各个地区之间旅行，每个地区都有车站并且每个地区都能到任何一个地区，可是坐车是要付钱的，并且途中经过的地区越多，要付的钱就越多。车站付钱的规则是这样的：第一第二个经过的地区所要支付的钱固定，之后每经过一个地区都要支付前两个地区所要支付的钱的总和，你的任务时帮助zp算出经过n个地区所要支付的总金额是多少！

输入要点：第一行输入经过的地区数 $n(1 \leq n \leq 100)$ ，第二行输入经过第一第二个地区所要支付的钱数 $x_1, x_2(1 \leq x_1, x_2 \leq 500)$ 。

long long!!!

输出要点：总共要付多少钱！

### 零基础先修

- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：第 1、2 项分别为  $x_1, x_2$ ，之后每项是前两项之和。边生成下一项 边加入总费用；题面特别提示使用 long long，代码按该数据约定实现。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     long long first, second;
6 |     cin >> n >> first >> second;
7 |     if (n == 1) {
8 |         cout << first << '\n';
9 |         return 0;
10 |    }
11 |    long long sum = first + second;
12 |    for (int i = 3; i <= n; ++i) {
13 |        long long next = first + second;
14 |        sum += next;
15 |        first = second;
16 |        second = next;
17 |    }
18 |    cout << sum << '\n';
19 |    return 0;
20 | }
```

### 公开样例

样例 1 输入

```
5
2 2
```

样例 1 输出

```
24
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 2.14 | 总 119/526 数字钟 ( PID 1570 )

出处：2020级HAUT新生周赛（五）@许金龙专场（F，CID 1064） | 训练定位：基础提高 | 模拟、枚举与构造

标签：模拟、时间换算、格式化输出 | 限制：1.000 S / 128 MB | 历史统计：46/140 ( 32.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出格式为 XX:XX:XX（分别表示时，分，秒）

输入要点：输入三个非负整数  $x, y, t$

数据范围：

$0 \leq x \leq 1000$

$0 \leq y \leq 1000$

$0 \leq t \leq 1000000$

输出要点：输出格式为 XX:XX:XX（分别表示时，分，秒）

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：电路刚接通时为 00:00:00。左键每次加一小时，右键每次加一分钟，再加自然经过的  $t$  秒；把总秒数对 24·3600 取模，依次除出时、分、秒，并用 `setw(2)` 与 `setfill('0')` 补前导零。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long hours, minutes, seconds;
5 |     cin >> hours >> minutes >> seconds;
6 |     long long total = (hours * 3600 + minutes * 60 + seconds) % 86400;
7 |     int h = total / 3600;
8 |     int m = total / 60 % 60;
9 |     int s = total % 60;
10 |     cout << setfill('0') << setw(2) << h << ':' << setw(2) << m << ':' << setw(2) << s
11 |         << '\n';
12 |     return 0;
13 | }
```

公开样例

|          |
|----------|
| 样例 1 输入  |
| 13 15 57 |
| 样例 1 输出  |
| 13:15:57 |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

2.15 | 总 120/526 9nine~九次九日九重色 ( PID 1884 )

出处：河南工大2024新生周赛 ( 4 ) ——命题人：马贺 ( F, CID 1094 ) | 训练定位：基础提高 | 模拟、枚举与构造

标签：逆序对、枚举、标记数组 | 限制：1.000 S / 128 MB | 历史统计：35/142 ( 24.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：作业要求你求出给定的n个数中逆序对的数量，并且找到被"孤立"的数的数量。你并不明白逆序对被"孤立"的数是什么意思，于是打电话问了沙月老师。沙月老师告诉你：逆序对指的是两个数 $a_i$ 与 $a_j$ ，满足 $i < j$ 并且 $a_i > a_j$ ，例如 $a_1=2, a_2=1$ , 则 $a_1$ 与 $a_2$ 构成一个逆序对。被"孤立"的数指的是某个数不能与其他任何一个数构成逆序对。

输入要点：第一行一个整数 $n(1 \leq n \leq 1000)$ 。

第二行n个整数 $a_1, a_2, \dots, a_n(1 \leq a_i \leq 1000)$ 。（保证输入的数各不相同）

输出要点：两个整数。第一个为逆序对的数量，第二个为被孤立的数的数量。两个数中间用一个空格隔开。

题面提示：[图片：https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116161050\_46634.png]

零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
3. 核心处理采用：两层循环枚举  $i < j$ ；若  $a[i] > a[j]$  就得到一个逆序对，并把  $i, j$  标记为“参与过逆序对”。最后未被标记的元素就是孤立数字。  
 $n \leq 1000$ ， $O(n^2)$  可以通过，也最适合零基础理解定义。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

$O(n^2)$  时间、 $O(n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> value(n);
9 |     vector<bool> involved(n, false);
10 |     for (long long& x : value)
11 |         cin >> x;
12 |     long long inversions = 0;
13 |     for (int i = 0; i < n; ++i) {
14 |         for (int j = i + 1; j < n; ++j) {
15 |             if (value[i] > value[j]) {
16 |                 ++inversions;
17 |                 involved[i] = involved[j] = true;
18 |             }
19 |         }
20 |     }
21 |     int isolated = count(involved.begin(), involved.end(), false);
22 |     cout << inversions << ' ' << isolated << '\n';
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
5
1 2 3 4 5
```

样例 1 输出

```
0 5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 2.16 | 总 121/526 幂幂幂 ( PID 1903 )

出处：河南工大2024新生周赛（6）——命题人：魏方（F，CID 1097） | 训练定位：基础提高 | 模拟、枚举与构造

标签：枚举 | 限制：1.000 S / 128 MB | 历史统计：68/99（68.7%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：wf是一个特别中二的人，他非常喜欢2这个数字以及2的k次方，现在给你一个正整数n，求这个数最少能被多少个2的幂次方组成。

输入要点：本题有多组输入。

输入一个正整数t( $1 \leq t \leq 105$ )，表示本题有t组输入，接下来每一行输入一个正整数n( $1 \leq n \leq 231-1$ )

输出要点：总共输出n行，每一行一个整数，表示n能被2的幂次方组成的最小个数。

题面提示：第一组输入中，1可以被2的0次方表示，所以答案是1

第二组输入中，17可以被2的4次方+2的0次方=16+1表示，所以答案是2

第三组输入中，36可以被2的5次方+2的2次方=32+4表示，所以答案是2

第四组输入中，58=32+16+8+2表示，所以答案是4

本题的输入量很大请不要尝试暴力解题！！！！

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
- 核心处理采用：幂幂幂 本题运用了一点点贪心的策略，尽量使用较大的2的幂次方，但是当 $n \% 2 == 1$ 时，表示必须使用当前这个2，因为如果 $\% 2 == 0$ ，则表示可以别的2的幂次方所表示， $\% 2 == 1$ ，表示不能再被别的2的幂次方表示了。的幂次方，比如 $5 \% 2 == 1$ ，就必须使用1， $5 / 2 = 2$ ，此时此时总和为1， $2 \% 2 == 0$ ，所以当前这个2的1次方不选， $2 / 2 == 1$ ， $1 \% 2 == 1$ ，所以要使用4，总共用了两个2的幂次方数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | void solve() {
3 |     int n, count = 0;
4 |     scanf("%d", &n);
5 |     while (n) {
6 |         count += n % 2;
7 |         n /= 2;
8 |     }
9 |     printf("%d\n", count);
10 | }
```

```

11 | int main() {
12 |     int t;
13 |     scanf("%d", &t);
14 |     while (t--) {
15 |         solve();
16 |     }
17 |     return 0;
18 | }

```

## 公开样例

样例 1 输入

```

4
1
17
36
58

```

样例 1 输出

```

1
2
2
4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.17 | 总 122/526 Simple学长玩游戏 ( PID 1250 )

出处：2016级河南工大新生周赛总决赛 ( A , CID 1016 ) | 训练定位：进阶 | 模拟、枚举与构造

标签：模拟、数组前缀交换、不等式 | 限制：1.000 S / 128 MB | 历史统计：0/9 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你的任务是在每次换防之后，计算出若此时勇者要击杀某一支护卫队恶魔并见到大魔王，至少需要多少点初始防御力。

输入要点：输入一个  $n$ ，代表有  $n$  支恶魔队伍 (  $n \leq 100$  )

接下来的  $n$  行，第一个数表示第  $i$  支队伍有多少只恶魔 (  $m \leq 100$  )，然后后面跟  $m$  个数， $a_i$  表示第  $i$  个恶魔的攻击力 (  $a_i \leq 100$  )

然后一个  $q$ ，表示有  $q$  次换防 (  $q \leq 100$  )

接下来的  $q$  行，每行有  $x_i, a_i, y_i, b_i$ ，表示第  $x_i$  支护卫队的前  $a_i$  只恶魔会与第  $y_i$  支护卫队的前  $b_i$  只恶魔交换。

输出要点：输出有  $q$  行，请在每次换防后，输出若此时勇者要 击杀 某一支护卫队恶魔并 存活，至少需要的初始防御力点数。

题面提示：对于样例，第一个队伍初始状态是 2 5 1 5 6，第二个队伍的初始状态是 5 6 7 8

经过第一次换防后，第一个队伍变成了 5 6 5 6，第二个队伍变成了 2 5 1 7 8，所以应该选择攻击第二支队伍，需要的最少防御力为 4

经过第二次换防后，第一个队伍空了 ( 空的队伍不能被攻击了 )，第二个队伍变成了 5 6 5 6 2 5 1 7 8，所以只能攻击第二支队伍，需要的防御力为 5

经过第三次换防后，第一个队伍变成了 5 6 5 6 2，第二个队伍变成了 5 1 7 8，这次不论选择第一个队伍还是第二个队伍需要的最小防御力都是 5

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：若勇者以初始防御  $D$  攻打一队，第  $i$  只恶魔（从 0 计）出现前已经击杀  $i$  只，当前防御为  $D+i$ 。要存活需  $D+i \geq \text{attack}[i]$ ，所以该队最低初防是  $\max(\text{attack}[i]-i)$ 。每次换防把两队指定长度的前缀取出并交换，各自剩余后缀顺序不变；随后扫描所有非空队伍，取其最低初防的最小值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每次换防  $O(\text{总恶魔数})$  时间、 $O(\text{总恶魔数})$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int need(const vector<int>& team) {
4 |     int required = 0;
5 |     for (int i = 0; i < (int)team.size(); ++i)
6 |         required = max(required, team[i] - i);
7 |     return required;
8 | }
9 | int main() {
10 |     ios::sync_with_stdio(false);
11 |     cin.tie(nullptr);
12 |     int n;
13 |     cin >> n;
14 |     vector<vector<int>> team(n);
15 |     for (auto& monsters : team) {
16 |         int count;
17 |         cin >> count;
18 |         monsters.resize(count);
19 |         for (int& attack : monsters)
20 |             cin >> attack;
21 |     }
22 |     int q;
23 |     cin >> q;
24 |     while (q--) {
25 |         int x, prefixX, y, prefixY;
26 |         cin >> x >> prefixX >> y >> prefixY;
27 |         --x;
28 |         --y;
29 |         vector<int> oldX = team[x];
30 |         vector<int> oldY = team[y];
31 |         vector<int> newX, newY;
32 |         newX.insert(newX.end(), oldY.begin(), oldY.begin() + prefixY);
33 |         newX.insert(newX.end(), oldX.begin() + prefixX, oldX.end());
34 |         newY.insert(newY.end(), oldX.begin(), oldX.begin() + prefixX);
35 |         newY.insert(newY.end(), oldY.begin() + prefixY, oldY.end());
36 |         team[x] = move(newX);
37 |         team[y] = move(newY);
38 |         int ans = INT_MAX;
39 |         for (const auto& monsters : team)
40 |             if (!monsters.empty())
41 |                 ans = min(ans, need(monsters));
42 |         cout << ans << '\n';
43 |     }
44 |     return 0;
45 | }
```

## 公开样例

样例 1 输入

```
2
5 2 5 1 5 6
```

```
4 5 6 7 8
3
1 3 2 2
1 4 2 0
1 0 2 5
```

样例 1 输出

```
4
5
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.18 | 总 123/526 趣味游戏 ( 三 ) : 剑圣 ( PID 1331 )

出处：2017级河南工大新生周赛 ( 四 ) ( C , CID 1031 ) | 训练定位：进阶 | 模拟、枚举与构造

标签：模拟、分类讨论、溢出伤害 | 限制：1.000 S / 128 MB | 历史统计：6/73 ( 8.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：听说你们比较喜欢宗师级别的人物，LOL中就有很多这样的人，剑圣正是其中之一。剑圣在刚成为剑道宗师时获得一个附加被动：“第4k刀会打出双倍攻击力 (  $k=1,2,3,4,\dots$  )”，之后剑圣为了稳固自己的剑道独自闯入敌人的一个中型营地，经过激烈的战斗剑圣带着满身伤痕回到了联盟。经过蒙多医生的治疗之后剑圣开始了自己的战后总结 ( 总结很重要 )，由于战斗过于激烈剑圣只记得.....

输入要点：第一行 一个整数 T 表示测试样例的数量

对于每组测试样例：

第一行：一个整数 n 表示剑圣出刀次数 (  $0 < n \leq 5000$  )

第二行：n 个整数，第 i 个整数表示 第 i 次出刀的攻击力 (  $0 < \text{攻击力} \leq 10000$  ) ( 不算被动 )。

第三行：n 个整数，第 i 个整数表示 第 i 次被击打敌人的防御力 (  $0 < \text{防御力} \leq 10000$  )

第四行：n 个整数，第 i 个整数表示 第 i 次被击打敌人的血量 (  $0 < \text{血量} \leq 10000$  )

输出要点：每组样例输出用一个空格隔开的两个整数，分别表示失去战斗能力的敌人的数量和造成的总伤害。

题面提示：注意剑圣的被动、直接伤害、持续伤害、全额伤害 ( 忽略血量的影响 )、溢出伤害。

<1>伤害和血量的关系：剑圣造成一点伤害相应敌人减去一点血量。

<2>溢出伤害为超过敌人血量的那一部分直接伤害

<3>持续伤害不会杀死敌人也不会使敌人失去战斗能力，但是会造成全额伤害。

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。



2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：第 4、8、12...刀先把基础攻击力翻倍。若最终攻击力严格大于防御力，敌人失去战斗能力，直接伤害为  $2 \cdot (\text{攻击} - \text{防御})$ ，但计入总伤害时不能超过其血量。否则是持续伤害`防御-攻击`：它不会让敌人失去战斗能力，却按全额计入，即使数值大于血量也不截断。按刀次逐项模拟并区分这两个分支。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(n)$  时间、 $O(n)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        vector<long long> attack(n), defense(n), health(n);
12 |        for (long long& x : attack)
13 |            cin >> x;
14 |        for (long long& x : defense)
15 |            cin >> x;
16 |        for (long long& x : health)
17 |            cin >> x;
18 |        int disabled = 0;
19 |        long long totalDamage = 0;
20 |        for (int i = 0; i < n; ++i) {
21 |            long long atk = attack[i] * ((i + 1) % 4 == 0 ? 2 : 1);
22 |            if (atk > defense[i]) {
23 |                ++disabled;
24 |                long long direct = 2 * (atk - defense[i]);
25 |                totalDamage += min(direct, health[i]);
26 |            } else {
27 |                totalDamage += defense[i] - atk;
28 |            }
29 |        }
30 |        cout << disabled << ' ' << totalDamage << '\n';
31 |    }
32 |    return 0;
33 | }
```

## 公开样例

样例 1 输入

```
2
4
10 12 14 16
5 7 9 20
100 100 100 2
4
5 6 7 8
1 2 3 4
2 3 4 5
```

样例 1 输出

```
4 32
4 14
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.19 | 总 124/526 文件 ( PID 1435 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( H , CID 1047 ) | 训练定位：进阶 | 模拟、枚举与构造

标签：栈、模拟、线性查找 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：自己想要文件需要移动其他文件的最小数量。如果想拿的文件不存在或者之前已经拿过了

输入要点：多样例测试

一行输入一个整数T表示样例数( $1 < T \leq 20$ )

对于每个样例：

第一行输入n , m , q ; ( $0 < n, m < 200, 0 < q < 500$ )

第二行输入n个整数 $a_i$ 表示"A"堆的文件从下到上的编号 ( $0 < a_i < 10^5$ )

第三行输入m个整数 $b_i$ 表示"B"堆的文件从下到上的编号( $0 < b_i < 10^5$ )

接下来的q行，每行输入一个整数 $c_i$  表示想拿的书的编号( $0 < c_i < 10^5$ )

注意：桌上文件的编号都是唯一的。

输出要点：对于每一个 $c_i$ 。输出要拿到这个文件需要移动其他文件的最小数量。

题面提示：样例解释：

'A'堆：(1 2 3) 'B'堆：(5 7 9)

想拿7

移动9：'A'堆：(1 2 3 9) 'B'堆：(5 7)

拿到7(移动一个文件)

'A'堆：(1 2 3 9) 'B'堆：(5)

想拿7

7已经拿过输出-1

'A'堆：(1 2 3 9) 'B'堆：(5)

想拿1

'A'堆：(1 2 3 9) 'B'堆：(5)

移动9：'A'堆：(1 2 3) 'B'堆：(5 9)

移动3 : 'A'堆 : (1 2) 'B'堆 : (5 9 3)

移动2 : 'A'堆 : (1) 'B'堆 : (5 9 3 2)

拿到1(移动3个文件)

'A'堆 : (1 2 3 9) 'B'堆 : (5)

想拿6

6不存在输出-1

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：两个 vector 按从下到上存成栈。查询文件时先在线性范围内定位；若在 A 的位置 p，就把 A 中 p 上方的文件逐个 pop 并 push 到 B，移动数为上方数量，随后删除目标；在 B 中同理。找不到（含已取走）输出 -1。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每次  $O(n+m)$ ，总空间  $O(n+m)$

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, m, q;
10 |         cin >> n >> m >> q;
11 |         vector<int> a(n), b(m);
12 |         for (int& x : a)
13 |             cin >> x;
14 |         for (int& x : b)
15 |             cin >> x;
16 |         while (q--) {
17 |             int target;
18 |             cin >> target;
19 |             auto ia = find(a.begin(), a.end(), target);
20 |             auto ib = find(b.begin(), b.end(), target);
21 |             if (ia == a.end() && ib == b.end()) {
22 |                 cout << -1 << '\n';
23 |             } else if (ia != a.end()) {
24 |                 int moves = a.end() - ia - 1;
25 |                 while (a.back() != target) {
26 |                     b.push_back(a.back());
27 |                     a.pop_back();
28 |                 }
29 |                 a.pop_back();
30 |                 cout << moves << '\n';
31 |             } else {
32 |                 int moves = b.end() - ib - 1;
```

```

33 |         while (b.back() != target) {
34 |             a.push_back(b.back());
35 |             b.pop_back();
36 |         }
37 |         b.pop_back();
38 |         cout << moves << '\n';
39 |     }
40 | }
41 | }
42 | return 0;
43 | }

```

## 公开样例

### 样例 1 输入

```

1
3 3 4
1 2 3
5 7 9
7
7
1
6

```

### 样例 1 输出

```

1
-1
3
-1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 2.20 | 总 125/526 阿正的排球测试 ( PID 1545 )

出处：2020级HAUT新生周赛（二）@杨哲专场（G，CID 1061）| 训练定位：进阶 | 模拟、枚举与构造

标签：位运算、模拟、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：1/24 ( 4.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**现在你的任务是算出经过坏心眼的阿树捣乱后阿正同学接到Mickey传过来球的概率，这个概率以阿正可以接到的球的数量比上所有球可能落点的数量来表示。

**输入要点：**第一行一个十进制的正整数N，代表Mickey和阿正传球的场地。

第二行三个整数pos，r和t，分别代表阿正的位置，阿正的接球半径和坏心眼的阿树使用魔法的次数。

随后t行每行两个整数，分别代表坏心眼阿树使用魔法的种类和魔法处理的位置。（1代表与操作，2代表或操作，3代表异或操作）。

输入保证 $0 \leq \text{pos} \leq 32$ ， $0 \leq r \leq 32$ ， $0 \leq t \leq 100000$ 。

**输出要点：**一个百分数代表阿正能否接到Mickey传过来球的概率，采用四舍五入取整法，保留两位小数。

**题面提示：**对于1000000666转换为32位的二进制字符串后为 [00111011100110101100110010011010]；

阿正同学位于这个字符串下标为16的位置，即下划线标明的位置：[00111011100110101100110010011010]；

阿正同学的接球区间为：[00111011100110101100110010011010]；

阿树同学施展第一次魔法，使得 $S[10]=S[10]\&0$ ；字符串变为 `[00111011100110101100110010011010]`；

阿树同学施展第二次魔法，使得 $S[12]=S[12]\&0$ ；字符串变为 `[00111011100110101100110010011010]`；

阿树同学施展第三次魔法，使得 $S[8]=S[8]\&0$ ；字符串变为 `[00111011000110101100110010011010]`；

阿树同学施展……

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：将十进制数展开为 32 位数组，题面下标 0 对应最高位。魔法 1 将该位清零，魔法 2 与 0 按位或所以不改变，魔法 3 翻转该位。最后分别统计全部 1 和接球区间内的 1，输出百分比两位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(32+t)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     uint32_t n;
7 |     cin >> n;
8 |     int pos, radius, t;
9 |     cin >> pos >> radius >> t;
10 |     array<int, 32> bit{};
11 |     for (int i = 0; i < 32; ++i)
12 |         bit[i] = (n >> (31 - i)) & 1U;
13 |     while (t--) {
14 |         int type, index;
15 |         cin >> type >> index;
16 |         if (index < 0 || index >= 32)
17 |             continue;
18 |         if (type == 1)
19 |             bit[index] = 0;
20 |         else if (type == 3)
21 |             bit[index] ^= 1;
22 |     }
23 |     int total = accumulate(bit.begin(), bit.end(), 0);
24 |     int caught = 0;
25 |     for (int i = max(0, pos - radius); i <= min(31, pos + radius); ++i) {
26 |         caught += bit[i];
27 |     }
28 |     double probability = total ? 100.0 * caught / total : 0.0;
29 |     cout << fixed << setprecision(2) << probability << "\n";
30 |     return 0;
31 | }
```

公开样例

样例 1 输入

1000000666  
16 7 4  
1 10  
2 12  
1 8  
3 17

样例 1 输出

40.00%

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

2.21 | 总 126/526 求序列和 ( PID 1865 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( G , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( B , CID 1092 )  
| 训练定位：进阶 | 模拟、枚举与构造

标签：递推、数位、累加 | 限制：1.000 S / 128 MB | 历史统计：386/2079 ( 18.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，在一行中输出序列和。

输入要点：在一行中给出  $a$  和  $n$  (  $0 \leq a \leq 9, 1 \leq n \leq 16$  ) 。

输出要点：在一行中输出序列和。

题面提示：灵感来源：C语言程序设计 ( 第4版 )

零基础先修

- 模拟要求程序状态与题面过程一一对应；枚举要求证明没有漏解；构造则要同时说明输出满足全部约束。练习时必须手推小样例并写清不变量。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：递推构造  $a, aa, aaa \dots$  :  $term=term*10+a$ ，再把  $term$  加入  $sum$ 。  $n \leq 16$ ，题目保证 long long 范围适用。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

$O(n)$  时间、 $O(1)$  空间

易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, n;
5 |     cin >> a >> n;
6 |     long long term = 0, sum = 0;
7 |     for (int i = 0; i < n; ++i) {
8 |         term = term * 10 + a;
9 |         sum += term;
10 |     }
11 |     cout << sum << '\n';
12 |     return 0;
13 | }
```

### 公开样例

|         |
|---------|
| 样例 1 输入 |
| 2 3     |
| 样例 1 输出 |
| 246     |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 2.22 | 总 127/526 时间加速器 ( PID 1911 )

出处：河南工大2024新生周赛（7）——命题人：刘义（D，CID 1098） | 训练定位：进阶 | 模拟、枚举与构造

标签：枚举 | 限制：1.000 S / 128 MB | 历史统计：5/28（17.9%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：要你求出完成所有任务花费的时间。

输入要点：第一行三个整数，分别为 n,a,b。

接下来 2 到 n+1 行，第 i行输入 w<sub>i</sub>。

输出要点：一行，完成所有作业的最少时间。

题面提示：1≤w<sub>i</sub>,n,a,b≤5×10<sup>5</sup>

数据比较大，请不要直接暴力

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。

3. 核心处理采用：时间加速器使用总数是否小于等于mid即可。#include<iostream> #define int long long

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <iostream>
2 | #define int long long
3 | using namespace std;
4 | int n, a, b, x[600006];
5 | bool check(int mid) // 判断函数
6 | {
7 |     int tot = 0;
8 |     for (int i = 1; i <= n; i++) {
9 |         if (x[i] <= mid * a)
10 |             continue;
11 |         else {
12 |             if ((x[i] - mid * a) % b == 0)
13 |                 tot += (x[i] - mid * a) / b;
14 |             else
15 |                 tot += ((x[i] - mid * a) / b + 1);
16 |         }
17 |     }
18 |     return tot <= mid;
19 | }
20 | signed main() {
21 |     cin >> n >> a >> b;
22 |     for (int i = 1; i <= n; i++)
23 |         cin >> x[i];
24 |     int l = 0, r = 1e9, mid;
25 |     while (l <= r) // 二分搜索
26 |     {
27 |         mid = (l + r) / 2;
28 |         if (check(mid))
29 |             r = mid - 1;
30 |         else
31 |             l = mid + 1;
32 |     }
33 |     cout << l << endl;
34 |     return 0;
35 | }
```

## 公开样例

样例 1 输入

```
3 2 1
1
2
3
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“模拟、枚举与构造”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 3 | 数组、字符串与双指针

这一模块训练线性扫描、下标边界、字符处理、连续段和左右指针。它们频繁出现在周赛前中段，也是后续数据结构与动态规划的语言基础。

本模块共 120 道唯一题。

## 3.1 | 总 128/526 Crazy Computer ( PID 1226 )

出处：2016级河南工大新生周赛（五）（A，CID 1010）| 训练定位：入门 | 数组、字符串与双指针

标签：模拟、连续段、时间差 | 限制：1.000 S / 128 MB | 历史统计：104/179 ( 58.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，Print a single positive integer, the number of words that remain on the screen after all  $n$  words was typed, in other words, at the second  $t_n$ .

**输入要点：**The first line contains two integers  $n$  and  $c$  ( $1 \leq n \leq 100000, 1 \leq c \leq 10^9$ ) — the number of words ZS the Coder typed and the crazy computer delay respectively. The first line contains two integers  $n$  and  $c$  ( $1 \leq n \leq 100000, 1 \leq c \leq 10^9$ ) — the number of words ZS the Coder typed and the crazy computer delay respectively.

The next line contains  $n$  integers  $t_1, t_2, \dots, t_n$  ( $1 \leq t_1 < t_2 < \dots < t_n \leq 10^9$ ), where  $t_i$  denotes the second when ZS the Coder ty.....

**输出要点：**Print a single positive integer, the number of words that remain on the screen after all  $n$  words was typed, in other words, at the second  $t_n$ .

**题面提示：**For the first sample, after typing the first word at the second 1, it disappears because the next word is typed at the second 3 and  $3-1>1$ . Similarly, only 1 word will remain at the second 9. Then, a word is typed at the second 10, so there will be two words on the screen, as the old word won't disappear because  $10-9 \leq 1$ .

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：若相邻两次输入单词的时间差大于延迟  $c$ ，之前屏幕上的所有单词都会消失，当前可见数重置为 1；否则旧单词尚未清空，当前可见数加一。从第一个单词的计数 1 开始扫描所有相邻时间差，最后的计数就是  $t_n$  时刻屏幕上的单词数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long delay;
8 |     cin >> n >> delay;
9 |     long long pre;
10 |    cin >> pre;
11 |    int visible = 1;
12 |    for (int i = 1; i < n; ++i) {
13 |        long long cur;
14 |        cin >> cur;
15 |        if (cur - pre > delay)
16 |            visible = 1;
17 |        else
18 |            ++visible;
19 |        pre = cur;
20 |    }
21 |    cout << visible << '\n';
22 |    return 0;
23 | }
```

### 公开样例

样例 1 输入

```
6 1
1 3 5 7 9 10
```

样例 1 输出

```
2
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.2 | 总 129/526 和小埋一起做准备运动 ( PID 1324 )

出处：2017级河南工大新生周赛 ( 三 ) ( C , CID 1030 ) | 训练定位：入门 | 数组、字符串与双指针

标签：字符串、逆序、大整数读入 | 限制：1.000 S / 128 MB | 历史统计：134/210 ( 63.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，1行，逆序输出这个数字（直接输出，不需要判断前缀0）

输入要点：第一行n, 这个数字的长度。

数据范围：( 1 <= n <= 1000 )

第二行，一个很长的数字s，数的位数小于1000

输出要点：1行，逆序输出这个数字（直接输出，不需要判断前缀0）

题面提示：int main(void)

```

{

int n , myNumber[1010];

char myString[1010];

scanf("%d", &n);

scanf("%s", myString);//这样就读入了那一长串数字。

for(int i=0;i<n;i++){

myNumber[i] = myString[i] - '0';//这样就把读入的字符的转化成了数字，保存到了数组里

}

...//输出

}

```

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：数字位数接近 1000，不能读入整数类型，但题目只要求逐字符逆序且保留 前导 0。把它作为字符串读取，调用 reverse 后直接输出即可；例如 1908 变成 8091。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     string number;
8 |     cin >> n >> number;
9 |     reverse(number.begin(), number.end());
10 |     cout << number << '\n';
11 |     return 0;
12 | }

```

## 公开样例

样例 1 输入

```

4
1908

```

样例 1 输出

```

8091

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.3 | 总 130/526 最大增区间 (一) (PID 1438)

出处：2018级河南工大新生周赛 (七) @牛寅旭专场 (B, CID 1046) | 训练定位：入门 | 数组、字符串与双指针

标签：数组与序列、双指针、最长区间 | 限制：1.000 S / 128 MB | 历史统计：0/0 (无公开统计)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给你  $n$  个数字，求相邻数字的最大增区间。

输入要点：第一行有一个数字  $n$ 。(  $n \leq 10^5$  )

第二行有  $n$  个数字。

输出要点：输出最大增区间的起止范围。(区间下标从 1 开始)

若有多个，输出下标最小的区间。

题面提示：增区间内，数字可以相等

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：扫描所有极大的连续非递减段。维护当前段起点；遇到下降就结束 当前段并与最优长度比较，只有严格更长才更新，从而在长度相同时保留 下标更小的区间。循环结束后还要检查最后一段。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(n)$  空间

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     vector<long long> a(n);
7 |     for (long long& x : a)
8 |         cin >> x;
9 |     int cur = 0, bestLeft = 0, bestRight = 0;
10 |    for (int i = 1; i <= n; ++i) {
11 |        if (i == n || a[i - 1] > a[i]) {
12 |            if (i - cur > bestRight - bestLeft + 1) {
13 |                bestLeft = cur;
14 |                bestRight = i - 1;
15 |            }
16 |            cur = i;
17 |        }
18 |    }
19 |    cout << bestLeft + 1 << ' ' << bestRight + 1 << '\n';
20 |    return 0;
21 | }
```

## 公开样例

样例 1 输入

```
10
1 5 3 6 4 8 9 1 5 7
```

样例 1 输出

```
5 7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.4 | 总 131/526 那年那题那些AC ( PID 1486 )

出处：2019级河南工大新生周赛 ( 三 ) @邓祎培专场 ( A , CID 1055 ) | 训练定位：入门 | 数组、字符串与双指针

标签：矩阵、模拟、三重循环 | 限制：1.000 S / 128 MB | 历史统计：158/262 ( 60.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出：输出答案矩阵，输出是x行，每行z个数据，数据之间用空格隔开。

输入要点：输入：

第一行输入三个整数x,y,z (  $x \leq 10, y \leq 10, z \leq 10$  ) 用来表示第一个矩阵M是x行y列，第二个矩阵N是y行z列；

后面的x行是第一个矩阵M的数，中间用空格隔开

接着是y行是第二个矩阵N的数，中间用空格隔开

输出要点：输出：

输出答案矩阵，输出是x行，每行z个数据，数据之间用空格隔开。

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：矩阵  $M$  为  $x \times y$ ， $N$  为  $y \times z$ ，结果  $C$  的第  $i$  行第  $j$  列等于  $\sum_{k=0..y-1} M[i][k] \cdot N[k][j]$ 。三重循环按定义计算，并控制每行元素之间有一个空格、行末无多余空格。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(xyz)$  时间、 $O(xy+yz)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int x, y, z;
5 |     cin >> x >> y >> z;
6 |     vector<vector<long long>> a(x, vector<long long>(y));
7 |     vector<vector<long long>> b(y, vector<long long>(z));
8 |     for (auto& row : a)
9 |         for (long long& value : row)
10 |             cin >> value;
11 |     for (auto& row : b)
12 |         for (long long& value : row)
13 |             cin >> value;
14 |     for (int i = 0; i < x; ++i) {
15 |         for (int j = 0; j < z; ++j) {
16 |             long long value = 0;
17 |             for (int k = 0; k < y; ++k)
18 |                 value += a[i][k] * b[k][j];
19 |             if (j)
20 |                 cout << ' ';
21 |             cout << value;
22 |         }
23 |         cout << '\n';
24 |     }
25 |     return 0;
26 | }
```

## 公开样例

样例 1 输入

```
2 2 3
1 2
3 4
1 2 3
4 5 6
```

样例 1 输出

```
9 12 15
19 26 33
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.5 | 总 132/526 对小19的亲切关怀 ( PID 1501 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( D , CID 1056 ) | 训练定位：入门 | 数组、字符串与双指针

标签：字符串、图形输出、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：24/42 ( 57.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，[图片：[https://acm.haut.edu.cn/upload/image/20191124/20191124120744\\_10903.png](https://acm.haut.edu.cn/upload/image/20191124/20191124120744_10903.png)]

输入要点：1

2

3

输出要点：[图片：[https://acm.haut.edu.cn/upload/image/20191124/20191124120744\\_10903.png](https://acm.haut.edu.cn/upload/image/20191124/20191124120744_10903.png)]

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 多组数据以 EOF 结束时使用 while (cin >> ...)，不要额外输出测试数据。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：图形可拆为宽  $8N$ 、高  $4N$  的正面矩形，以及向右上偏移  $N+1$  列、 $N+1$  行的背面。先画背面上边与  $N$  条斜边，再画正面上边；中间先输出  $3N-2$  行三条竖边，接着输出  $N$  行逐列左移的右下斜边，最后画正面下边。多组  $N$  读到文件尾，图形之间不插空行。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(N^2)$  输出时间、 $O(N)$  临时空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- EOF 题不要先读不存在的 T，也不要再在循环结束后多输出内容。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
```

```

6 |     int n;
7 |     while (cin >> n) {
8 |         int width = 8 * n;
9 |         cout << string(n + 1, ' ') << string(width, '+') << "\n";
10 |        for (int i = 1; i <= n; ++i) {
11 |            cout << string(n + 1 - i, ' ') << '/' << string(width - 2, ' ') << '/'
12 |                << string(i - 1, ' ') << "+\n";
13 |        }
14 |        cout << string(width, '+') << string(n, ' ') << "+\n";
15 |        for (int i = 0; i < 3 * n - 2; ++i) {
16 |            cout << '+' << string(width - 2, ' ') << '+' << string(n, ' ') << "+\n";
17 |        }
18 |        for (int i = 1; i <= n; ++i) {
19 |            cout << '+' << string(width - 2, ' ') << '+' << string(n - i, ' ') << "/\n";
20 |        }
21 |        cout << string(width, '+') << "\n";
22 |    }
23 |    return 0;
24 | }

```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 3.6 | 总 133/526 你是个好人 ( PID 1718 )

出处：2021级HAUT新生周赛（六）@李志豪专场（B，CID 1074）| 训练定位：入门 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：239/311 ( 76.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，她的回复。

输入要点：无

输出要点：她的回复。

题面提示：可以复制到自己的编译器上运行一下qwq

(感谢zhj提供的程序)

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：你是个好人 标签：签到题 easy直接提交给出的代码就行了。项目地址在这里 [Change\\_code\\_underline](#)感兴趣的可以去看看。C.
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。



## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | // C
2 | #include <string.h>
3 | #include <stdio.h>
4 | #include <math.h>
5 | int t, i, j, la, lb;
6 | char a[1005], b[1005], th[1005];
7 | int bj[1005];
8 | int main() {
9 |     scanf("%d", &t);
10 |    while (t--) {
11 |        scanf("%s%s", a, b, th);
12 |        la = strlen(a);
13 |        lb = strlen(b);
14 |        for (int i = 0; i < la; i++)
15 |            bj[i] = 0;
16 |        for (i = 0; i < la - lb + 1; i++) {
17 |            int pp = 1;
18 |            for (j = 0; j < lb; j++) {
19 |                if (a[i + j] != b[j]) {
20 |                    pp = 0;
21 |                    break;
22 |                }
23 |            }
24 |            if (pp == 1) {
25 |                bj[i] = 1;
26 |                i += lb - 1;
27 |            }
28 |        }
29 |        for (i = 0; i < la; i++) {
30 |            if (bj[i]) {
31 |                printf("%s", th);
32 |                i += lb - 1;
33 |            } else
34 |                printf("%c", a[i]);
35 |        }
36 |        printf("\n");
37 |    }
38 |    return 0;
39 | }
```

## 公开样例

官网未提供可成对提取、可机读的公开样例；请在原题页核对题面图片或特殊格式。

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.7 | 总 134/526 关键词 ( PID 1728 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（C，CID 1075）| 训练定位：入门 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/5 ( 60.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：有一天，小明突然忘记了他手机的解锁密码（可能是把记录密码的突触丢进回收站了），小明尝试了五次，均输入错误，但是小明知道每次输入的密码中一定是有且仅有两位数字正确并且位置不对的。请你帮助小明求出他的四位手机密码。

输入要点：五行四位数字密码

输出要点：四个数字

题面提示：数据保证只有一个正确密码

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：关键词枚举暴力从0123开始查找，查找到9876，区间内必出答案 C
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | char s[5][5];
3 | char t[5];
4 | int isok() {
5 |     int cnt;
6 |     for (int i = 0; i < 5; i++) {
7 |         cnt = 0;
8 |         for (int j = 0; j < 4; j++) {
9 |             for (int k = 0; k < 4; k++) {
10 |                 if (s[i][k] == t[j]) {
11 |                     if (j == k)
12 |                         return 0;
13 |                     cnt++;
14 |                 }
15 |             }
16 |         }
17 |         if (cnt != 2)
18 |             return 0;
19 |     }
20 |     return 1;
21 | }
22 | int main() {
23 |     int temp;
24 |     for (int i = 0; i < 5; i++) {
25 |         scanf("%s", s[i]);
26 |     }
27 |     for (int i = 1; i < 10000; i++) {
```

```

28 |         temp = i;
29 |         for (int j = 3; j >= 0; j--) {
30 |             t[j] = temp % 10 + '0';
31 |             temp /= 10;
32 |         }
33 |         if (isok()) {
34 |             printf("%04d", i);
35 |         }
36 |     }
37 | }

```

## 公开样例

样例 1 输入

```

6087
5173
1358
3825
2531

```

样例 1 输出

```

8712

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.8 | 总 135/526 调查员三宝之旧印 ( PID 1782 )

出处：2022级HAUT新生周赛（四）@张子豪专场（C，CID 1082）| 训练定位：入门 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：49/84 ( 58.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：打开旧印的密码是三个未知正整数 $x, y, z (x \leq y \leq z)$ 。旧印上的提示有七个正整数，是 $x, y, z, x+y, x+z, y+z, x+y+z$ 这七个数的某种排列。请你帮调查员求出 $x, y, z$ 分别是什么。

输入要点：一行，七个正整数

输出要点：一行，输出 $x, y, z$ 的值，以空格分开

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：调查员三宝之旧印：分析： $x \leq y \leq z$ ，显然七个数中，最小的是 $x$ ，次小的是 $y$ ，最大的是 $x+y+z$  将7个数排序后，依次输出最小的，次小的，最大的减去最小的和次小的
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a[10];
4 |     for (int i = 1; i <= 7; i++) {
5 |         scanf("%d", &a[i]);
6 |     }
7 |     for (int i = 1; i <= 7; i++) {
8 |         for (int j = i + 1; j <= 7; j++) {
9 |             if (a[i] > a[j]) {
10 |                 int t = a[i];
11 |                 a[i] = a[j];
12 |                 a[j] = t;
13 |             }
14 |         }
15 |     }
16 |     printf("%d %d %d", a[1], a[2], a[7] - a[1] - a[2]);
17 |     return 0;
18 | }
```

## 公开样例

样例 1 输入

2 2 11 4 9 7 9

样例 1 输出

2 2 7

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.9 | 总 136/526 矩阵乘法 ( PID 1230 )

出处：2016级河南工大新生周赛（五）（E，CID 1010）| 训练定位：基础提高 | 数组、字符串与双指针

标签：矩阵、三重循环、线性代数 | 限制：1.000 S / 128 MB | 历史统计：11/46 ( 23.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给你两个  $n \times n$  的矩阵，请计算并输出它们相乘的结果。

输入要点：第1行：1个数  $N$ ，表示矩阵的大小 ( $2 \leq N \leq 100$ )

第2 -  $N + 1$  行，每行  $N$  个数，对应  $M1$  的1行 ( $0 \leq M1[i] \leq 1000$ )

第  $N + 2$  -  $2N + 1$  行，每行  $N$  个数，对应  $M2$  的1行 ( $0 \leq M2[i] \leq 1000$ )

输出要点：输出共  $N$  行，每行  $N$  个数，对应  $M1 * M2$  的结果的一行。

## 零基础先修

- 这一模块训练线性扫描、下标边界、字符处理、连续段和左右指针。它们频繁出现在周赛前中段，也是后续数据结构与动态规划的语言基础。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：矩阵乘法中结果  $C[i][j]$  是 A 第  $i$  行与 B 第  $j$  列的点积：`Σ A[i][k]\*B[k][j]`。用三层循环枚举行、列和公共下标。每行输出时仅在第 2 个及以后元素前加空格，避免多余分隔。累加使用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n^3)$  时间、 $O(n^2)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<vector<long long>> a(n, vector<long long>(n));
9 |     vector<vector<long long>> b(n, vector<long long>(n));
10 |     for (auto& row : a)
11 |         for (long long& x : row)
12 |             cin >> x;
13 |     for (auto& row : b)
14 |         for (long long& x : row)
15 |             cin >> x;
16 |     for (int i = 0; i < n; ++i) {
17 |         for (int j = 0; j < n; ++j) {
18 |             long long value = 0;
19 |             for (int k = 0; k < n; ++k)
20 |                 value += a[i][k] * b[k][j];
21 |             if (j)
22 |                 cout << ' ';
23 |             cout << value;
24 |         }
25 |         cout << '\n';
26 |     }
27 |     return 0;
28 | }
```

## 公开样例

样例 1 输入

```
2
1 0
0 1
0 1
1 0
```

样例 1 输出

```
0 1
1 0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.10 | 总 137/526 加密工作 ( PID 1312 )

出处：2017级河南工大新生周赛 ( 一 ) ( E, CID 1028 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组映射、ASCII | 限制：1.000 S / 128 MB | 历史统计：22/68 ( 32.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，四个整数，以空格隔开。

输入要点：每一行 给你一个选项  $c$  ( $1 \leq c \leq 4$ ) 和一个长度为 4 的字符串。

输出要点：四个整数，以空格隔开。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：先把每个字母统一转成 1..26 的值。四个选项对应原下标排列分别为 [0,1,2,3]、[3,2,1,0]、[2,3,0,1]、[1,0,3,2]。选出相应排列后，用单个空格连接四个整数，不在行尾额外输出空格。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int option;
7 |     string s;
8 |     while (cin >> option >> s) {
9 |         const int order[4][4] = {{0, 1, 2, 3}, {3, 2, 1, 0}, {2, 3, 0, 1}, {1, 0, 3, 2}};
10 |         for (int i = 0; i < 4; ++i) {
11 |             if (i)
12 |                 cout << ' ';
13 |             char c = s[order[option - 1][i]];
14 |             cout << tolower((unsigned char)c) - 'a' + 1;
15 |         }
16 |         cout << '\n';
17 |     }
```

```
17 |     }
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

1 Haut

样例 1 输出

8 1 21 20

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.11 | 总 138/526 QAQ ( PID 1345 )

出处：2017级河南工大新生周赛 ( 六 ) ( E , CID 1035 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：子序列计数、前缀统计、字符串 | 限制：1.000 S / 256 MB | 历史统计：27/53 ( 50.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组测试实例输出一行：包含一个整数，子序列"QAQ"的个数。

输入要点：第一行：一个整数T，表示测试实例个数。

对于每组测试实例：包含一个长度为 n ( $1 \leq n \leq 100$ ) 的字符串。

输出要点：每组测试实例输出一行：包含一个整数，子序列"QAQ"的个数。

题面提示：如第一组样例：共有4个子序列"QAQ"，分别如下：

"QAQAQYSYIOIWIN", "QAQAQYSYIOIWIN", "QAQAQYSYIOIWIN", "QAQAQYSYIOIWIN".

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：把每个 A 当作子序列中间字符。它左侧任取一个 Q、右侧任取一个 Q 就形成一组 QAQ，因此贡献为“左侧 Q 数×右侧 Q 数”。预先统计总 Q，扫描时维护左侧 Q 数，遇到 A 就累加乘积。也可用三状态子序列 DP。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(n)$  时间、 $O(1)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string s;
10 |        cin >> s;
11 |        long long totalQ = count(s.begin(), s.end(), 'Q');
12 |        long long leftQ = 0, ans = 0;
13 |        for (char c : s) {
14 |            if (c == 'Q')
15 |                ++leftQ;
16 |            else if (c == 'A')
17 |                ans += leftQ * (totalQ - leftQ);
18 |        }
19 |        cout << ans << '\n';
20 |    }
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
2
QAQAQYSYIOIWIN
QAQQZZYNOIWIN
```

样例 1 输出

```
4
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.12 | 总 139/526 Choice 的字符串 ( PID 1352 )

出处：2017级河南工大新生周赛（七）（F，CID 1036） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、模拟、游程编码 | 限制：1.000 S / 128 MB | 历史统计：49/123 ( 39.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组输出一行，表示展开后的字符串。

输入要点：第一行输入一个整数T，表示测试组数。

每组数据包括一行，输入一个Choice写的字符串，长度小于100（第一位不会出现数字）

输出要点：每组输出一行，表示展开后的字符串。

## 零基础先修



- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：从左到右扫描。遇到字母就原样加入结果，并把它记作最近字符；遇到一位数字 d，就把最近字符额外追加 d 次。数字不会成为新的最近字符，因而 `c22` 是原有一个 c 再追加 2+2 个，共五个 c。第一位保证是字母。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(\text{输入长度} + \text{输出长度})$  时间与空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string encoded, decoded;
10 |         cin >> encoded;
11 |         char latest = '\0';
12 |         for (char c : encoded) {
13 |             if (isdigit((unsigned char)c)) {
14 |                 decoded.append(c - '0', latest);
15 |             } else {
16 |                 latest = c;
17 |                 decoded.push_back(c);
18 |             }
19 |         }
20 |         cout << decoded << '\n';
21 |     }
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

```
1
a3b4c22
```

样例 1 输出

```
aaaabbbbcccc
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.13 | 总 140/526 可以说是签到题 ( PID 1354 )

出处：2017级河南工大新生周赛（八）（A，CID 1037） | 训练定位：基础提高 | 数组、字符串与双指针

标签：固定输出、词频统计、字符串 | 限制：1.000 S / 128 MB | 历史统计：65/197 ( 33.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，26 个整数，中间以空格隔开。

输入要点：Some of us are having problems with our parents , as they often look into our school bags or read our diaries . I fully understand why we are not comfortable about it , but there's no need to feel too sad. Our parents are checking our bags or diaries to make sure we are not getting into any trouble . They have probably heard some horrible stories about other kids and thought we might do the same . Or perhaps they jus.....

输出要点：26 个整数，中间以空格隔开。

### 零基础先修

- 固定输出题不需要读入；把目标字符串逐字写入 cout，并在本地比较标点与换行。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：这是一道只有固定文章、没有输入的统计题。把文章按英文单词切分，`don't` 仍算一个单词；统一把每个单词首字母转成小写，再统计 a 到 z。得到的 26 个计数依次为 17,5,4,5,0,2,1,4,9,1,1,5,4,4,11,5,0,2,8,26,4,0,13,0,2,0。提交时直接输出这行即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间 ( 计数过程对文章长度为  $O(L)$  )

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     cout << "17 5 4 5 0 2 1 4 9 1 1 5 4 4 11 5 0 2 8 26 4 0 13 0 2 0\n";
5 |     return 0;
6 | }
```

### 公开样例

官网未提供可成对提取、可机读的公开样例；请在原网页核对题面图片或特殊格式。

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.14 | 总 141/526 加密工作 (二) (PID 1386)

出处：2018级河南工大新生周赛 (一) @徐向阳专场 (E, CID 1040) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：49/96 (51.0%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对每行输入，输出其对应的加密结果，单独占一行。

输入要点：第一行正整数N，接下来N ( $0 < N \leq 52$ ) 行字母 c (c 可以为大写，也可以为小写)

输出要点：对每行输入，输出其对应的加密结果，单独占一行。

题面提示：每行输入的行尾会有一个回车字符，例如：8，其实是 8'\n'

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：加密工作 (二) 思路：这题难度其实和第二题差不多，放在第五道是因为 每行输入的行尾有个回车字符。怕大家以前没遇到过这种情况，所以就放在了后面。处理这种情况的方法在参考代码中已经给出。参考代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <ctype.h>
3 | int main() {
4 |     int i, j, n, d;
5 |     int x, y; // x---*,y---#;
6 |     char c;
7 |     scanf("%d", &n);
8 |     for (i = 0; i < n; i++) {
9 |         scanf("%c", &c);
10 |         if (islower(c)) {
11 |             d = c - 'a' + 1;
12 |         } else {
13 |             d = c - 'A' + 1;
14 |         }
```

```

15 |         x = d % 5;
16 |         y = d / 5;
17 |         for (j = 0; j < y; j++) {
18 |             printf("#");
19 |         }
20 |         for (j = 0; j < x; j++) {
21 |             printf("*");
22 |         }
23 |         printf("\n");
24 |     }
25 | }

```

## 公开样例

### 样例 1 输入

```

8
A
a
E
e
F
f
Z
z

```

### 样例 1 输出

```

*
*
#
#
#*
#*
#####
#####

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.15 | 总 142/526 逻辑判断 ( PID 1399 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( D , CID 1042 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、逻辑运算、模拟 | 限制：1.000 S / 128 MB | 历史统计：67/160 ( 41.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，每组数据输出最终的逻辑值，每组占一行

**输入要点：**第1行是一个整数n,代表下面有n组数据。(n<=12)

第2~n+1行，每行一个合法的简单的逻辑表达式 ( 仅包含两个逻辑值和两者间的一个逻辑连接词 )

**输出要点：**每组数据输出最终的逻辑值，每组占一行

题面提示：逻辑连接词均为英文符号

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：表达式只有两个逻辑值和一个运算符。首尾字符转为布尔值，中间 若含 && 做与，若为 | 做或，否则做异或，最后转回 T/F。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     while (n--) {
7 |         string expr;
8 |         cin >> expr;
9 |         bool a = expr.front() == 'T';
10 |        bool b = expr.back() == 'T';
11 |        bool res;
12 |        if (expr.find("&&") != string::npos)
13 |            res = a && b;
14 |        else if (expr.find('|') != string::npos)
15 |            res = a || b;
16 |        else
17 |            res = a ^ b;
18 |        cout << (res ? 'T' : 'F') << '\n';
19 |    }
20 |    return 0;
21 | }
```

## 公开样例

样例 1 输入

```
3
T&&F
T|F
T^F
```

样例 1 输出

```
F
T
T
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.16 | 总 143/526 寻找遗迹 ( PID 1406 )

出处：2018级河南工大新生周赛 ( 四 ) @杜锐专场 ( E , CID 1043 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：二维数组、模拟、图形输出 | 限制：1.000 S / 128 MB | 历史统计：60/117 ( 51.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，绘制后的示意图，字符占10行。 每行10个字符，字符之间用空格隔开。 其中，用减号“-”表示未划线的部分，用星号“\*”来代表划线。

输入要点：第一行是一个整数N (  $N \leq 5$  )

之后的N行，每行有4个整数，分别代表每个矩形左下的x、y坐标和右上的x、y坐标 (  $1 \leq x, y \leq 10$  ) 用空格隔开

输出要点：绘制后的示意图，字符占10行。

每行10个字符，字符之间用空格隔开。

其中，用减号“-”表示未划线的部分，用星号“\*”来代表划线。

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：用  $mark[x][y]$  表示坐标纸格点是否画线。每个矩形把上下两条横边 的所有 x、左右两条竖边的所有 y 标为星号。输出从  $y=10$  到  $y=1$ ，每行  $x=1..10$ ，正好对应以左下角为原点的坐标系。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(N \cdot 10 + 100)$  时间、 $O(100)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     bool mark[11][11] = {};
7 |     while (n--) {
8 |         int x1, y1, x2, y2;
9 |         cin >> x1 >> y1 >> x2 >> y2;
10 |         for (int x = x1; x <= x2; ++x)
11 |             mark[x][y1] = mark[x][y2] = true;
12 |         for (int y = y1; y <= y2; ++y)
13 |             mark[x1][y] = mark[x2][y] = true;
14 |     }
15 |     for (int y = 10; y >= 1; --y) {
16 |         for (int x = 1; x <= 10; ++x) {
17 |             if (x > 1)
18 |                 cout << ' ';
```

```

19 |         cout << (mark[x][y] ? '*' : '-');
20 |     }
21 |     cout << '\n';
22 | }
23 | return 0;
24 | }

```

## 公开样例

样例 1 输入

```

3
1 1 2 4
2 4 10 7
5 5 8 9

```

样例 1 输出

```

-----
----*---
----*---
_*****
_***_*_*
_***_*_*
_***_*_*
*****_
*****
**-----
**-----
**-----

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.17 | 总 144/526 神秘代码 ( PID 1407 )

出处：2018级河南工大新生周赛 ( 四 ) @杜锐专场 ( F , CID 1043 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：斐波那契、查表、字符串 | 限制：1.000 S / 128 MB | 历史统计：191/349 ( 54.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**雷顿教授在一处遗迹中发现了一连串神秘的数字！经过分析、辨认，这串数字都属于斐波那契数列，并且最大的数字是317811，正好是斐波那契数列中除了两个1之外的第26个数。雷顿教授大胆猜想——这些数字与26英文字母表上大写字母的排序一一对应，如8是斐波那契数列除了1以外的第四个数 ( 2, 3, 5, 8, 13, …… )，就对应着第四个大写字母“D”。“零”表示空格。现在有……

**输入要点：**第一行是一个整数N(N<100)。第二行是N个整数An(A<= 317811)，每个整数之间用空格隔开。

**输出要点：**翻译后的英文语句，占一行。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：建立从斐波那契值到字母的映射：2 对应 A、3 对应 B，随后递推 到第 26 个值；输入 0 输出空格，其他数查表输出对应大写字母。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n+26)$  时间、 $O(26)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     map<int, char> decode;
5 |     int first = 2, second = 3;
6 |     decode[first] = 'A';
7 |     decode[second] = 'B';
8 |     for (int index = 2; index < 26; ++index) {
9 |         int next = first + second;
10 |        decode[next] = char('A' + index);
11 |        first = second;
12 |        second = next;
13 |    }
14 |    int n;
15 |    cin >> n;
16 |    while (n--) {
17 |        int value;
18 |        cin >> value;
19 |        cout << (value == 0 ? ' ' : decode[value]);
20 |    }
21 |    cout << '\n';
22 |    return 0;
23 | }
```

## 公开样例

样例 1 输入

```
11
55 13 377 377 1597 0 75025 1597 6765 377 8
```

样例 1 输出

```
HELLO WORLD
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.18 | 总 145/526 矿产含量 (PID 1432)

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( E , CID 1047 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：二维数组、矩形并面积、模拟 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，计算矿产的总含量。

输入要点：多样例测试



第一行输入一个整数T (  $1 \leq T \leq 100$  )

接下来输入T个样例，每个样例第一行输入一个n (  $1 \leq n \leq 100$  )。

接下来的n行，每行输入i, j, x, y用空格隔开，表示一个矩形 (  $0 \leq i < x \leq 10^3, 0 \leq j < y \leq 10^3$  )。

输出要点：计算矿产的总含量。

题面提示：样例解释：对于第二个样例

[图片：[http://47.106.196.181/upload/image/20181213/20181213190728\\_83523.png](http://47.106.196.181/upload/image/20181213/20181213190728_83523.png)]

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：坐标可能按左上、右下给出，所以先分别取 x、y 的最小最大值。矩形覆盖的单位方格为  $x \in [xmin, xmax)$ 、 $y \in [ymin, ymax)$ ；用  $1000 \times 1000$  布尔网格置 1，重叠处仍只计一次，最后统计真值个数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(\text{矩形覆盖格数} + 10^6)$  时间、 $O(10^6)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         vector<vector<bool>> covered(1000, vector<bool>(1000, false));
12 |         while (n--) {
13 |             int x1, y1, x2, y2;
14 |             cin >> x1 >> y1 >> x2 >> y2;
15 |             int left = min(x1, x2), right = max(x1, x2);
16 |             int bottom = min(y1, y2), top = max(y1, y2);
17 |             for (int x = left; x < right; ++x) {
18 |                 for (int y = bottom; y < top; ++y)
19 |                     covered[x][y] = true;
20 |             }
21 |         }
22 |         long long ans = 0;
23 |         for (const auto& row : covered) {
24 |             ans += count(row.begin(), row.end(), true);
25 |         }
26 |         cout << ans << '\n';
27 |     }
28 |     return 0;
29 | }
```

## 公开样例

样例 1 输入

```
3
1
1 2 2 1
3
1 3 3 1
2 4 4 2
6 5 8 2
2
1 3 3 1
1 2 2 1
```

样例 1 输出

```
1
13
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.19 | 总 146/526 cxx 的进制转换问题 ( PID 1478 )

出处：2019 级河南工大新生周赛 ( 一 ) @陶福专场 ( E , CID 1053 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：进制转换、字符串、循环 | 限制：1.000 S / 128 MB | 历史统计：115/158 ( 72.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：cxx 的进制转换问题

输入要点：输入一行，包含两个整数  $n$   $m$  用空格隔开

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：反复取  $n\%m$  得到从低位到高位各位，把它们反转后输出。 $n=0$  时循环不会执行，需单独输出 0； $m\leq 9$ ，所以每一位可直接用字符表示。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\log_m n)$  时间、 $O(\log_m n)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     int base;
6 |     cin >> n >> base;
7 |     if (n == 0) {
8 |         cout << 0 << '\n';
9 |         return 0;
10 |    }
11 |    string ans;
12 |    while (n > 0) {
13 |        ans += char('0' + n % base);
14 |        n /= base;
15 |    }
16 |    reverse(ans.begin(), ans.end());
17 |    cout << ans << '\n';
18 |    return 0;
19 | }
```

## 公开样例

样例 1 输入

4 2

样例 1 输出

100

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.20 | 总 147/526 CXK想要篮球 ( PID 1505 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场（B，CID 1059） | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：35/76 ( 46.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：CXK有很多乒乓球，他想用乒乓球换AW的篮球。已知AW的篮球价值X元，而CXK有分别价值1、5、10、50、100、500元的乒乓球各A、B、C、D、E、F个，CXK想知道最少需要多少个乒乓球能刚好换取篮球。假设本题至少存在一种支付方式。

输入要点：输入6个数分别代表A，B，C，D，E，F。最后输入一个数代表X。

输出要点：输出需要支付的乒乓球数

题面提示：500元乒乓球1个，50元乒乓球2个，10元乒乓球1个，5元乒乓球2个。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：CXK想要篮球 一个贪心的入门题，采用“尽可能多的采用面值较大的硬币”的贪心思想
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int v[6] = {1, 5, 10, 50, 100, 500};
4 | int c[6];
5 | int main() {
6 |     int A;
7 |     int ans = 0;
8 |     for (int i = 0; i < 6; i++)
9 |         cin >> c[i];
10 |    cin >> A;
11 |    for (int i = 5; i >= 0; i--) {
12 |        int t = min(A / v[i], c[i]);
13 |        A -= t * v[i];
14 |        ans += t;
15 |    }
16 |    cout << ans << endl;
17 |    return 0;
18 | }
```

## 公开样例

样例 1 输入

```
3 2 1 3 0 2
620
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.21 | 总 148/526 读书 ( PID 1523 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( F , CID 1059 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：7/27 ( 25.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：AW正在读一本书，其中包括 $n$ 页从1到 $n$ 的页码。每次读到的页码数字能被 $m$ 整除时，他都会写下该页码的最后一位。例如，如果 $n = 15$ 且 $m = 5$ ，则可被 $m$ 整除的页面为5,10,15。它们的最后一位数字分别为5,0,5，它们的总和为10。您的任务是计算AW写下的所有数字的总和。

输入要点：输入的第一行包含一个整数 $q$  ( $1 \leq q \leq 1000$ ) 代表查询数。以下 $q$ 行包含查询，每行一个。每个查询均以两个整数 $n$ 和 $m$  ( $1 \leq n, m \leq 1016$ ) 给出-分别是书中的页数和所需的除数。

输出要点：为每个查询打印答案— AW写下的数字总和。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：读书 思维题，假如 $n$ 为45， $m$ 为5,那么在1~45这个范围内，5的倍数为：5，10，15，20，25，30，35，40，45，再来看这些数的个位数，分别是：5，0，5，0，5，0，5，0，5。发现规律了没有！5，0循环重复出现！循环的长度为2。所以问题就有了突破口，使用for循环（这个循环最多有 $n/m$ 次）查找 $m$ 的倍数，然后开一个数组记录每次出现的个位数，同时声明一个变量 $len$ 来记录这个循环长度并计算这些个位数之和存到 $sum$ 中，当某个数字重复出现时就退出for循环。接下来就是计算能有多少个这样的循环，计算出次数并乘以刚刚计算出来的 $sum$ ，并将这个数值赋给 $sum$ 。可能这时候还有漏网之鱼，就像上面举的例子里面的最后一个5，这时候就要将构不成一个个位数出现规律循环的数加在 $sum$ 中，然后输出。分析结束
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回  $int$ 。

### C++17 参考实现

```
1 | #include <algorithm>
2 | #include <cstdio>
3 | #include <cstring>
4 | #include <iostream>
5 | using namespace std;
6 | long long int n, m; // n为书的页数, m表示每读m页就记录一次个位数
7 | int q; // 总循环的次数
8 | int i, j; // 控制循环
9 | int s[50]; // 记录循环过的数据
10 | int a[50]; // 记录循环过的次数
11 | int main() {
12 |     cin >> q;
13 |     while (q--) {
14 |         memset(s, 0, sizeof(s)); // 初始化数组, 每循环一次就要清空一下数组, 要不然就凉了
15 |         memset(a, 0, sizeof(a)); // 初始化数组
16 |         long long int sum = 0; // 记录数据之和
17 |         int temp = 0; // 记录个位数
```

```

18 |         int len = 0;           // 记录循环的长度
19 |         cin >> n >> m;
20 |         long long int cs = n / m; // cs表示共可循环多少次
21 |         if (m > n) {
22 |             cout << 0 << endl;
23 |             continue;
24 |         } else {
25 |             for (i = 1; i <= cs; i++) {
26 |                 temp = (m * i) % 10;
27 |                 s[i] = temp;
28 |                 a[temp]++;
29 |                 if (a[temp] == 2)
30 |                     break;
31 |                 sum += temp;
32 |                 len++;
33 |             }
34 |         }
35 |         sum *= (cs / len);
36 |         if (cs / len != 0) {
37 |             for (i = 1; i <= cs % len; i++)
38 |                 sum += s[i];
39 |         }
40 |         cout << sum << endl;
41 |     }
42 |     return 0;
43 | }

```

## 公开样例

### 样例 1 输入

```

7
1 1
10 1
100 3
1024 14
998244353 1337
123 144
1234312817382646 13

```

### 样例 1 输出

```

1
45
153
294
3359835
0
427262129093995

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.22 | 总 149/526 小青的班委竞选 ( PID 1531 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（A，CID 1060）| 训练定位：基础提高 | 数组、字符串与双指针

标签：查表、文件尾输入、字符串 | 限制：1.000 S / 128 MB | 历史统计：297/746 ( 39.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，对于每组输入，输出答案占一行，代表该张卡片背面的英文

**输入要点：**本题为多实例输入，不知道多实例输入是什么的同学可以去看一下HAUTOJ的1078，1079，1080学习一下，输入有若干行，代表每次询问给出卡片的正面的数字 $n$ （ $1 \leq n \leq 7$ ）

输出要点：对于每组输入，输出答案占一行，代表该张卡片背面的英文

## 零基础先修

- 多组数据以 EOF 结束时使用 `while (cin >> ...)`，不要额外输出测试数据。
- `string` 可按下标访问字符；注意长度、空串、大小写、标点和 `getline` 与 `cin` 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：数字 1 到 7 与星期英文一一对应，建立下标从 1 开始的查表数组。输入没有给组数，使用 `while(cin>>n)` 读到文件尾。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- EOF 题不要先读不存在的 T，也不要循环结束后多输出内容。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const string weekday[8] = {"", "monday", "tuesday", "wednesday",
7 |                               "thursday", "friday", "saturday", "sunday"};
8 |     int n;
9 |     while (cin >> n)
10 |         cout << weekday[n] << '\n';
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

```
2
4
5
```

样例 1 输出

```
tuesday
thursday
friday
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.23 | 总 150/526 聚聚的小游戏 ( PID 1577 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（E，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（E，CID 1069） | 训练定位：基础提高 | 数组、字符串与双指针

标签：条件分支、字符串 | 限制：1.000 S / 128 MB | 历史统计：652/1679 ( 38.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：聚聚写课程设计突发奇想做了一个益智（简单）小游戏。游戏中间有一部分是使用键盘W、A、S、D（分别代表上、左、下、右）来控制角色移动，现在游戏角色在原点，聚聚给你n个大写字母'W' 'A' 'S' 'D'，如果角色经过这n个操作后依然在原点请输出"YES"，否则输出"NO"（不带引号）。

输入要点：第一行，一个正整数n，且 $n \leq 100$

第二行，n个字符，每个字符是大写字母'W'或'A'或'S'或'D'

输出要点：一行，如果角色还在原点输出"YES"（不带引号），否则输出"NO"（不带引号）

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- string 可按下表访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：聚聚的小游戏循环输入，统计四种字符的出现次数。W的数量和S相等且A的数量和D相等时输出YES，否则输出NO。注意变量的初始化和行末换行字符的控制
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int w = 0, a = 0, s = 0, d = 0; // 统计WASD的出现次数
4 |     int n, i;
5 |     char op;
6 |     scanf("%d%c", &n); // 使用%c吃掉行末换行
7 |     for (i = 1; i <= n; i++) {
8 |         scanf("%c", &op);
9 |         // 统计各种字符出现次数
10 |         if (op == 'W')
11 |             w++;
12 |         else if (op == 'A')
13 |             a++;
14 |         else if (op == 'S')
15 |             s++;
16 |         else if (op == 'D')
17 |             d++;
18 |     }
```



```

18 |     }
19 |     // 如果上下次数相等 且 左右次数相等，停在原点
20 |     if (w == s && a == d)
21 |         printf("YES");
22 |     else
23 |         printf("NO");
24 |     return 0;
25 | }

```

## 公开样例

样例 1 输入

```

8
WASDWASD

```

样例 1 输出

```

YES

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.24 | 总 151/526 聚聚的子串 ( PID 1587 )

出处：2021级HAUT新生周赛（一）@张君毅专场（G，CID 1069）| 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：107/254 ( 42.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：聚聚手上有  $n$  个字符，希望你能求出这  $n$  个字符所组成的串  $S$  中出现次数最多的非空子串的出现次数，记作  $p$ 。

输入要点：第一行，一个正整数  $n$ 。（ $0 < n \leq 100$ ）

第二行， $n$  个只含有小写字母 'a' 'b' 'c' 'd' 'e' 的长度为  $n$  的串  $S$

输出要点：一行，一个整数  $p$ 。

题面提示：子串：是字符串中任意个连续的字符组成的子序列称为该串的子串。

样例解释："abcdea" 中，出现最多的非空子串是 "a"，因此输出 2。

提示：尝试简化问题，不要想复杂了。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：聚聚的子串首先考虑子串长度为 1 的情况。这类子串就是 a~e 的字符。那么出现次数最多的子串就是出现次数最多的字符。接下来考虑子串长度为 2 的情况。这类子串一定由某一字符（长度为 1 的子串）加一个新字符组成。因此，这种子串的出现一次一定小于等于第一类子串的出现次数。例如：串 abab 中，ab 是一个这类子串。而 a 出现的次数一定不会比 ab 出现次数少。因此，我们的思路是：单个字符的子串中出现次数最多的一定是所有子串中出现次数最多的。统计出现的字母的数量即可，最大值即为答案。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a = 0, b = 0, c = 0, d = 0, e = 0; // 分别用来存储 5 个字符出现的次数
4 |     int n, i;
5 |     char ch;
6 |     scanf("%d%c", &n); // %c用来吃掉行末空格，也可以用 getchar();
7 |     for (i = 1; i <= n; i++) {
8 |         scanf("%c", &ch);
9 |         // 统计字符出现次数
10 |         if (ch == 'a')
11 |             a++;
12 |         else if (ch == 'b')
13 |             b++;
14 |         else if (ch == 'c')
15 |             c++;
16 |         else if (ch == 'd')
17 |             d++;
18 |         else if (ch == 'e')
19 |             e++;
20 |     }
21 |     // 找到出现次数的最大值
22 |     int max = a;
23 |     if (b > max)
24 |         max = b;
25 |     if (c > max)
26 |         max = c;
27 |     if (d > max)
28 |         max = d;
29 |     if (e > max)
30 |         max = e;
31 |     printf("%d", max);
32 |     return 0;
33 | }
```

## 公开样例

样例 1 输入

```
6
abcdea
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.25 | 总 152/526 没有名字 ( PID 1597 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（G，CID 1071） | 训练定位：基础提高 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：122/298 ( 40.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，一行输出两个整数以空格分开。 分别为UU和学长的得分。

输入要点：第一行一个整数 $t$  ( $1 \leq t \leq 1e6$ )，表示游戏进行的轮数。

接下来  $t$ 行，每行两个整数分别表示这一轮UU和学长说的数。

输出要点：一行输出两个整数以空格分开。

分别为UU和学长的得分。

#### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

#### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：没有名字 简单分支；依照题意可得，如下：`#include <iostream> #include <cstdio> int a[1000010];`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

通常为  $O(n)$

#### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

#### C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | int a[1000010];
4 | int main() {
5 |     int n;
6 |     scanf("%d", &n);
7 |     int ans = 0, sum = 0;
8 |     while (n--) {
9 |         int x, y;
10 |         scanf("%d%d", &x, &y);
11 |         if (x > y) {
12 |             if (x == 40 && (y == 10 || y == 20))
13 |                 sum += 5;
14 |             else
15 |                 ans += (x - y);
16 |         } else {
17 |             if (y == 40 && (x == 10 || x == 20))
18 |                 ans += 5;
19 |             else
20 |                 sum += (y - x);
```

```

21 |     }
22 | }
23 | printf("%d %d", ans, sum);
24 | return 0;
25 | }

```

## 公开样例

样例 1 输入

```

2
10 20
20 10

```

样例 1 输出

```

10 10

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.26 | 总 153/526 拔河比赛 ( PID 1702 )

出处：2021级HAUT新生周赛（四）@王一杰专场（C，CID 1072）| 训练定位：基础提高 | 数组、字符串与双指针

标签：条件分支、字符串 | 限制：1.000 S / 128 MB | 历史统计：183/399 ( 45.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：HF中学一年一度的拔河比赛又开始了，小锋被邀请去做裁判，在观看比赛的同时他也在分析选手们的状态。他发现，除了体重，选手们的站位、握绳姿势、站姿等也是取胜的关键。当双方选手又一次陷入了僵持时，他选中一位选手，将他的重心定为原点(0,0,0)对他进行空间上的受力分析，判断他是否受力平衡。由于计算量太大，请你编写程序帮助他判断。

输入要点：第一行一个整数 $n(1 \leq n \leq 100)$ 代表选手受力个数；随后的 $n$ 行，每行三个整数：第 $i$ 个力的 $X_i$ 坐标, $Y_i$ 坐标, $Z_i$ 坐标。  
( $-100 \leq X_i, Y_i, Z_i \leq 100$ )

输出要点：如果处于受力平衡，输出"YES"，否则输出"NO"。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：拔河比赛这道题很简单，只需要判断三个方向上的合力是否都为零，如果都为0，则受力平衡。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | int main() {
6 |     int n, x = 0, y = 0, z = 0, a, b, c; // x,y,z要赋值为0
7 |     scanf("%d", &n);
8 |     for (int i = 1; i <= n; i++) {
9 |         scanf("%d%d%d", &a, &b, &c);
10 |        x += a, y += b, z += c; // 累加
11 |    }
12 |    if (!x && !y && !z)
13 |        printf("YES\n"); // 简单的判断三个方向的合力是否都为0
14 |    else
15 |        printf("NO\n");
16 |    return 0;
17 | }
```

## 公开样例

样例 1 输入

```
4
-2 81 83
5 -98 11
-6 71 -9
7 8 89
```

样例 1 输出

```
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.27 | 总 154/526 爬山 (PID 1721)

出处：2021级HAUT新生周赛（六）@李志豪专场（D，CID 1074）| 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：57/249 (22.9%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：所以你的任务是，在依次给出的所有海拔值中，回答山峰的坐标是多少(坐标从1开始)

输入要点：第一行一个T，代表有T组测试样例  $1 \leq T \leq 100$ 。

每组测试样例包含一个整数n，代表给出n个海拔值。(  $n \leq 1e5$  )

接下来n个整数 $h_i$ ，两个整数之间用空格隔开。(  $0 \leq h_i \leq 1e9$  )

输出要点：输出一行包含m个整数 $a_i$ ，m代表山峰的数量， $a_i$ 为山峰的坐标。请按照从小到大的顺序输出各个下标

如果没有一个山峰的话请在那一行输出0。

题面提示：第一个测试样例 $3 > 1$ 并且 $3 > 2$ ，所以3是山峰海拔，下标为2，没有其他的山峰了，所以输出2。

第二个测试样例没有山峰，所以输出0。

第三个测试样例2为山峰，下标为2，5为山峰，下标为4，7为山峰，下标为6，所以输出2 4 6。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：爬山 标签：模拟 easy题意：给出n个值，找到所有数大于它相邻左边的值和相邻右边的值，输出这些数的下标。思路：按照题目模拟。标程
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define ll long long
4 | #define endl "\n"
5 | const int max_n = 1e5 + 100;
6 | const int inf = 0x3f3f3f3f;
7 | const int mod = 1e9 + 7;
8 | int h[max_n];
9 | int ans[max_n];
10 | int main() {
11 |     ios::sync_with_stdio(false);
12 |     cin.tie(0);
13 |     cout.tie(0);
14 |     int T;
15 |     cin >> T;
16 |     while (T--) {
17 |         int n;
18 |         cin >> n;
19 |         for (int i = 1; i <= n; i++)
20 |             cin >> h[i];
21 |         int id = 0;
22 |         for (int i = 2; i < n; i++) {
23 |             if (h[i] > h[i - 1] && h[i] > h[i + 1])
24 |                 ans[id++] = i;
25 |         }
26 |         if (!id)
27 |             cout << "0" << endl;
28 |         else {
29 |             for (int i = 0; i < id; i++)
30 |                 cout << ans[i] << " ";
31 |             cout << endl;
32 |         }
33 |     }
34 |     return 0;
}
```

## 公开样例

样例 1 输入

```
3
5
1 3 2 7 8
10
3 2 3 4 5 6 7 8 9 10
7
1 2 1 5 3 7 4
```

样例 1 输出

```
2
0
2 4 6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.28 | 总 155/526 交换余生 ( PID 1732 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（G，CID 1075）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：5/16 ( 31.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每行输出一个正整数或者一个字符串

输入要点：每行随机一个小于1e6的正整数或者一个长度小于5的字符串

输出要点：每行输出一个正整数或者一个字符串

题面提示：题目中所有字符均为大写字符

本题为多样例输入，不会的可以去参考HAUTOJ1079题

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：交换余生字符串，进制 先判断输入的是字符串还是整形 然后进行不同的操作 字符串转换为数字需要对进位处进行一些处理 C
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <ctype.h>
3 | #include <string.h>
4 | char s[10];
5 | char c[10];
6 | void reverse() {
7 |     char temp;
8 |     int len = strlen(c);
9 |     for (int i = 0; i < len / 2; i++) {
10 |         temp = c[i];
11 |         c[i] = c[len - i - 1];
12 |         c[len - i - 1] = temp;
13 |     }
14 | }
15 | void numtostring() {
16 |     int num = 0;
17 |     for (int i = 0; i < strlen(s); i++) {
18 |         num *= 10;
19 |         num += s[i] - '0';
20 |     }
21 |     memset(c, 0, sizeof(c));
22 |     int cnt = 0;
23 |     while (num) {
24 |         if (num % 26) {
25 |             c[cnt++] = num % 26 + 'A' - 1;
26 |             num /= 26;
27 |         } else {
28 |             c[cnt++] = 'Z';
29 |             num = num / 26 - 1;
30 |         }
31 |     }
32 |     reverse();
33 | }
34 | void stringtonum() {
35 |     int num = 0;
36 |     int len = strlen(s);
37 |     for (int i = 0; i < len; i++) {
38 |         num *= 26;
39 |         num += s[i] - 'A' + 1;
40 |     }
41 |     memset(c, 0, sizeof(c));
42 |     int cnt = 0;
43 |     while (num) {
44 |         c[cnt++] = num % 10 + '0';
45 |         num /= 10;
46 |     }
47 |     reverse();
48 | }
49 | int main() {
50 |     while (scanf("%s", s) != EOF) {
51 |         if (isdigit(s[0])) {
52 |             numtostring();
53 |         } else {
54 |             stringtonum();
55 |         }
56 |         printf("%s\n", c);
57 |     }
58 | }
```

## 公开样例

样例 1 输入

```
LCL
17861
27
E
```

样例 1 输出

```
8202
ZJY
```



## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.29 | 总 156/526 那真是太令人高兴了 ( PID 1735 )

出处：2021级HAUT新生周赛（八）@孔维飒专场（B，CID 1076）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 32 MB | 历史统计：63/228 ( 27.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，请你在某些出现过“problem”的聊天后新加一行“学到了” 输出所有聊天记录

输入要点：第一行一个整数 $n$ 代表共有 $n$ 条聊天记录

接下来 $n$ 行，每行是一个字符串(每个字符都是除空格外的可见字符)代表聊天记录

数据范围 $1 \leq n \leq 1000$ ，字符串长度小于1000。

输出要点：请你在某些出现过“problem”的聊天后新加一行“学到了”

输出所有聊天记录

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：那真是太令人高兴了（传送门）题目其实就是问你那个字符串中含有problem这个序列，那么只需要暴力判断一下就行了。那么就是暴力匹配串 代码实现
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

- 确认 0/1 起下标、首尾元素和 n=1。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int n;
4 | char a[1100];
5 | char tag[] = {"problem"};
6 | int main() {
7 |     scanf("%d", &n);
8 |     for (int i = 0; i < n; i++) {
9 |         scanf("%s", a); // 读入字符串
10 |         int m = strlen(a);
11 |         int flag = 0;
12 |         for (int j = 0; j + 6 < m; j++) {
13 |             flag = 1;
14 |             for (int k = 0; k < 7; k++) { // 判断从当前j位开始是否出现过problem
15 |                 if (a[j + k] != tag[k])
16 |                     flag = 0; // 不匹配置为0
17 |             }
18 |             if (flag)
19 |                 break;
20 |         }
21 |         puts(a); // 输出原来的串
22 |         if (flag)
23 |             puts("学到了"); // 如果flag没有置为0，则说明有problem这个串，那么输出
24 |     }
25 |     return 0;
26 | }
```

## 公开样例

### 样例 1 输入

```
3
%%%%%%%%
problem1001???
orz
```

### 样例 1 输出

```
%%%%%%%%
problem1001???
学到了
orz
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)
- [题面图片 2](#)
- [题面图片 3](#)
- [题面图片 4](#)
- [题面图片 5](#)
- [题面图片 6](#)

## 3.30 | 总 157/526 你能面对真正的内心吗 ( PID 1740 )

出处：2021级HAUT新生周赛（八）@孔维斌专场（G，CID 1076）| 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：85/197 ( 43.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，小圆做出了多少道题

输入要点：第一行两个整数 $n, m$

第二行 $m$ 个整数 $s_1, s_2, \dots, s_i, \dots, s_m$ 代表小圆的提交序列。

数据范围 $1 \leq n, m \leq 1e6, 1 \leq s_i \leq n$

输出要点：一个整数，小圆做出了多少道题

题面提示：小圆会从1开始依次做题，开始提交 $a_1 = 1$ ，与题目编号匹配，做出，提交 $a_2 = 3$ ，此时题目编号是2，所以不匹配，然后提交 $a_3 = 4$ ，还是与题目编号不匹配，然后提交 $a_4 = 2$ ，匹配，做出， $a_5 = 3$ ，匹配，做出， $a_6 = 3$ ，不匹配。最后做出了1, 2, 3三道题，所以最后输出3。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：你能面对真正的内心吗(传送门)题目大意：小圆会按照顺序交题，那么用一个数来记录当前交到了第几题，那么进行一次循环即可。时间复杂度:  $O(n)O(n)O(n)$ 参考代码
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, m;
4 |     scanf("%d%d", &n, &m);
5 |     int cnt = 0; // 记录当前做到第几题了
6 |     for (int i = 1; i <= m; i++) {
7 |         int x;
8 |         scanf("%d", &x);
9 |         if (x == cnt + 1)
10 |             cnt++; // 判断x 是否是正确的题目号，当前做完的下一道
11 |     }
12 |     printf("%d\n", cnt);
13 |     return 0;
14 | }
```

### 公开样例

样例 1 输入

```
4 6
1 3 4 2 3 3
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.31 | 总 158/526 挑选礼物，但是收礼人 (PID 1756)

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（C，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（C，CID 1079）| 训练定位：基础提高 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：223/622 (35.9%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示最后聚聚想要的礼物所在的纸杯编号。

输入要点：输入一共  $n+2$  行

第一行输入一个正整数  $n$  ( $1 \leq n \leq 20$ ) 代表交换次数，一个正整数  $p$  ( $p=1,2,3$ ) 代表聚聚想要的礼物

第二行输入三个正整数  $a_1, a_2, a_3$  代表礼物最初的摆放位置

之后 $n$ 行每行输入两个正整数  $b_1, b_2$  代表交换的纸杯

（要注意的是：最左边的杯子编号永远为 1，最右边的编号永远为 3，不会随着交换而改变）

输出要点：一个整数，表示最后聚聚想要的礼物所在的纸杯编号。

题面提示： $1 \leq n \leq 20$

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：挑选礼物，但是收礼人 简单的模拟，用数组(也可以不用)记录初始位置，然后根据输入交换就行
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, p;
4 |     scanf("%d%d", &n, &p);
5 |     int a[4];
6 |     scanf("%d%d%d", &a[1], &a[2], &a[3]);
7 |     for (int i = 0; i < n; i++) {
8 |         int b1, b2;
9 |         scanf("%d%d", &b1, &b2);
10 |         int k = a[b1];
11 |         a[b1] = a[b2];
12 |         a[b2] = k;
13 |     }
14 |     for (int i = 1; i <= 3; i++) {
15 |         if (a[i] == p) {
16 |             printf("%d", i);
17 |         }
18 |     }
19 |     return 0;
20 | }
```

### 公开样例

样例 1 输入

```
3 1
1 2 3
2 1
3 1
1 3
```

样例 1 输出

```
2
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.32 | 总 159/526 表白 ( PID 1757 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（D，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（D，CID 1079） | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：294/929 ( 31.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个整数

输入要点：输入一个字符串，仅由大小写字母组成

输出要点：一行，一个整数

题面提示：字符串可以看做结尾字符为'\0'的特殊字符数组，字符串读入操作为scanf("%s",str);

可以利用循环，读到 '\0' 字符作为终止条件

或者也可以选择一个字符一个字符的读入

字符串大小不超过 100000

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：表白 输入字符串然后直接循环相加即可 #include <stdio.h>
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n = 0;
4 |     char str[100010];
5 |     scanf("%s", str);
6 |     for (int i = 0; str[i] != '\0'; i++) {
7 |         n += str[i];
8 |     }
9 |     printf("%d", n);
10 |    return 0;
11 | }
```

## 公开样例

样例 1 输入

hgihi

样例 1 输出

521

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

## 3.33 | 总 160/526 好数 ( PID 1760 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（F，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（G，CID 1079）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、相邻比较、比例、整数化 | 限制：1.000 S / 128 MB | 历史统计：135/336 ( 40.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行 如果摆摆可以继续开摆，输出 "YES ^\_^" 否则，输出 "no!!! X\_X" 输出均不含引号

输入要点：一个整数  $n$ ，聚聚给摆摆的数

输出要点：一行

如果摆摆可以继续开摆，输出 "YES ^\_^"

否则，输出 "no!!! X\_X"

输出均不含引号

题面提示：10 ≤  $n$  ≤ 1018

样例解释：

递增共有3组，"14","45","14",递减有1组，"51"，总共4组，递增组占比75%，即概率为75%，大于70%

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：从最高位向最低位比较相邻字符：后一位较大记一次递增，较小记一次 递减，相等不计。概率大于 70% 等价于 10×递增数>7×有效比较数，用整数比较避免浮点误差；没有有效比较时条件不成立。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(|n|)$  时间、 $O(|n|)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
```

```

3 | int main() {
4 |     string number;
5 |     cin >> number;
6 |     int increasing = 0, decreasing = 0;
7 |     for (size_t i = 1; i < number.size(); ++i) {
8 |         if (number[i] > number[i - 1])
9 |             ++increasing;
10 |        else if (number[i] < number[i - 1])
11 |            ++decreasing;
12 |    }
13 |    int comparisons = increasing + decreasing;
14 |    bool canRest = comparisons > 0 && 10 * increasing > 7 * comparisons;
15 |    cout << (canRest ? "YES ^_^" : "no!!! X_X") << '\n';
16 |    return 0;
17 | }

```

## 公开样例

样例 1 输入

114514

样例 1 输出

YES ^\_^

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.34 | 总 161/526 绝对不能回邮件哦 ( PID 1766 )

出处：2022级HAUT新生周赛（二）@武其轩专场（E，CID 1080）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、查表、模拟 | 限制：1.000 S / 128 MB | 历史统计：125/226 ( 55.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出占一行，为经转换后的二进制文字。

输入要点：输入一个长度不超过1000的字符串，只包含大写英文字母和空格。

输出要点：输出占一行，为经转换后的二进制文字。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：题面已经按 A 到 Z 给出 26 个摩斯编码，直接存入字符串数组。逐字符扫描原文：空格原样输出，大写字母 c 用表中 c-'A' 的位置替换。不要在相邻字母编码之间自行加分隔符，因为样例和题意都要求直接拼接。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。



- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(|s| + \text{输出长度})$  时间、 $O(1)$  额外空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const vector<string> morse = {"01", "1000", "1010", "100", "0", "0010", "110",
7 |                                   "0000", "00", "0111", "101", "0100", "11", "10",
8 |                                   "111", "0110", "1101", "010", "000", "1", "001",
9 |                                   "0001", "011", "1001", "1011", "1100"};
10 |
11 |     string s;
12 |     getline(cin, s);
13 |     for (char c : s) {
14 |         if (c == ' ')
15 |             cout << ' ';
16 |         else
17 |             cout << morse[c - 'A'];
18 |     }
19 |     cout << '\n';
20 |     return 0;
}
```

### 公开样例

样例 1 输入

HELLO WORLD

样例 1 输出

0000001000100111 0111110100100100

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 3.35 | 总 162/526 我的名字 ( PID 1767 )

出处：2022级HAUT新生周赛（二）@武其轩专场（F，CID 1080） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、字符计数、模拟 | 限制：1.000 S / 128 MB | 历史统计：167/275（60.7%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，在一行中按题目要求输出排序后的字符串。题目保证输出非空。

输入要点：在一行中给出一个长度不超过10000的、仅由英文字母构成的非空字符串。

输出要点：在一行中按题目要求输出排序后的字符串。题目保证输出非空。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：只统计 m、t、l、f 四种字母，先用 tolower 消除大小写差异。然后不断按 m→t→l→f 的顺序尝试输出：某种字母还有剩余就输出并减一，已用完则跳过。一轮若一个字符也没输出，说明全部计数都已归零。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(|s| + \text{答案长度})$  时间、 $O(1)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     string s;
7 |     cin >> s;
8 |     const string order = "mtlff";
9 |     array<int, 4> count{};
10 |    for (unsigned char c : s) {
11 |        char x = static_cast<char>(tolower(c));
12 |        for (int i = 0; i < 4; ++i) {
13 |            if (x == order[i])
14 |                ++count[i];
15 |        }
16 |    }
17 |    while (true) {
18 |        bool printed = false;
19 |        for (int i = 0; i < 4; ++i) {
20 |            if (count[i] > 0) {
21 |                cout << order[i];
22 |                --count[i];
23 |                printed = true;
24 |            }
25 |        }
26 |        if (!printed)
27 |            break;
28 |    }
29 |    cout << '\n';
30 |    return 0;
31 | }
```

## 公开样例

样例 1 输入

```
dfhmyrijetrijkhtrimjihkrtmMIUJIDNGIU
```

样例 1 输出

```
mtlffmtlmtm
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.36 | 总 163/526 棋子反转 ( PID 1784 )

出处：2022级HAUT新生周赛（四）@张子豪专场（G，CID 1082）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：32/67 ( 47.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你可以进行如下操作：选择一个数字  $i$  ( $1 \leq i \leq n$ )，将前  $i$  个棋子同时变色（黑色变白色，白色变黑色），求如果要将所有棋子变成白色，最少要进行这样的操作多少次？

输入要点：一个字符串，由 0 和 1 组成，表示每个棋子的颜色

输出要点：一个整数，表示要进行操作的最少次数

题面提示：样例说明：

第 1 次翻转：把第一个棋子变为黑色，字符串为 00

第 2 次翻转：把第一个第二个棋子一起变为白色，字符串为 11，翻转完成，输出 2

$1 \leq n \leq 1e6$ ;

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：棋子反转：题意：读入表示棋子状态的字符串，要把棋子都翻到 1。翻转有一定的规则，如果你要翻第  $i$  枚棋子，必须把  $1 \sim i$  枚都翻转。问怎么翻才能用最少的次数得到全部为 1 的棋子序列。分析：从左往右扫，如果这个棋子与下个棋子状态不同，就说明要翻一次。最后还得判断最后一个是不是 0，如果是 0 还要再翻一次，因为题目说要全部正面朝上，就是全部为 1，最后一位为 0 就说明翻完后全是 0，还要再翻一次。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | char s[1000005];
4 | int main() {
5 |     scanf("%s", s);
6 |     int ans = 0;
7 |     int n = strlen(s);
8 |     for (int i = 0; i < n - 1; i++) {
9 |         if (s[i] != s[i + 1])
10 |             ans++;
11 |     }
12 |     if (s[n - 1] == '0')
13 |         ans++;
14 |     printf("%d", ans);
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入

10

样例 1 输出

2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.37 | 总 164/526 夏虫 ( PID 1786 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（A，CID 1083）| 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：23/87 ( 26.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，蝉获得的最多的营养。

输入要点：第1行三个整数 $n, m, L$  ( $1 \leq n \leq 103$ ) ( $1 \leq m \leq 109$ ) ( $1 \leq L \leq 109$ )

第2行 $n$ 个整数，第 $i$ 个数字是 $a_i$  ( $1 \leq a_i \leq 106$ )

输出要点：一个整数，蝉获得的最多的营养。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：夏虫 题意：求前  $m/L$  大的值的和 解法：把所有树枝按照营养从大到小排序，然后计算最多可以飞到几个树枝上，然后把营养最多的前几个树枝的营养加起来即为答案。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[1005];
3 | int main() {
4 |     int n, m, l;
5 |     scanf("%d%d%d", &n, &m, &l);
6 |     for (int i = 0; i < n; i++)
7 |         scanf("%d", &a[i]);
8 |     for (int i = 0; i < n - 1; i++)
9 |         for (int j = i + 1; j < n; j++)
10 |             if (a[i] < a[j]) {
11 |                 int temp = a[i];
12 |                 a[i] = a[j];
13 |                 a[j] = temp;
14 |             }
15 |     int ans = 0;
16 |     for (int i = 0; i < n && i < m / l; i++)
17 |         ans += a[i];
18 |     printf("%d", ans);
19 | }
```

## 公开样例

样例 1 输入

```
5 10 3
1 2 3 4 5
```

样例 1 输出

```
12
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.38 | 总 165/526 比较大小 ( PID 1800 )

出处：2022级HAUT新生周赛（六）@张鑫专场（G，CID 1084）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：131/305 ( 43.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给你两个字符，可能是阿拉伯数字也可能是其他字符，现在让你比较这两个字符的大小并输出较小的数字，如果无法比较（有非数字的字符）请输出-1。

输入要点：两个字符。

输出要点：一个数字

题面提示：一定是单个字符，数字大于等于0小于等于9。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：比较大小签到题 `#include<stdio.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     char a, b;
4 |     scanf("%c %c", &a, &b);
5 |     if (a < '0' || a > '9' || b < '0' || b > '9')
6 |         printf("-1");
7 |     else
8 |         printf("%c", a < b ? a : b);
9 |     return 0;
10 | }
```

## 公开样例

样例 1 输入

a 1

样例 1 输出

-1

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.39 | 总 166/526 简单统计数字 ( PID 1838 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（B，CID 1088）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、字符计数、排序思想 | 限制：1.000 S / 128 MB | 历史统计：73/285 ( 25.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，对 N 中每一种不同的个位数字，以 D:M 的格式在一行中输出该位数字 D 及其在 N 中出现的次数 M。要求按 D 的降序输出。

输入要点：每个输入包含 1 个测试用例，即一个不超过 1000 位的正整数 N。

输出要点：对 N 中每一种不同的个位数字，以 D:M 的格式在一行中输出该位数字 D 及其在 N 中出现的次数 M。要求按 D 的降序输出。

#### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

#### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：统计十进制数字 0 到 9 的出现次数，再从 9 降到 0 输出计数非零的项。把输入当字符串读取可以自然保留前导零。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

$O(|N|)$  时间、 $O(1)$  空间

#### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

#### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string s;
5 |     cin >> s;
6 |     array<int, 10> count{};
7 |     for (char c : s)
8 |         ++count[c - '0'];
9 |     for (int digit = 9; digit >= 0; --digit) {
10 |         if (count[digit])
11 |             cout << digit << ':' << count[digit] << '\n';
12 |     }
13 |     return 0;
14 | }
```

#### 公开样例

样例 1 输入

100311

样例 1 输出

3:1  
1:3  
0:2

#### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题

3.40 | 总 167/526 输出WAGD ( PID 1843 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（G，CID 1088）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、字符计数、模拟 | 限制：1.000 S / 128 MB | 历史统计：55/111 ( 49.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，在一行中按题目要求输出排序后的字符串。题目保证输出非空。

输入要点：输入在一行中给出一个长度不超过10000的、仅由英文字母构成的非空字符串。

输出要点：在一行中按题目要求输出排序后的字符串。题目保证输出非空。

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：与 mtf 型循环输出相同：不区分大小写统计 W、A、G、D，随后按 W→A→G→D 循环尝试输出，耗尽的字母跳过，直至一整轮无输出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

复杂度

O(|s|+答案长度) 时间、O(1) 空间

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string s;
5 |     cin >> s;
6 |     const string order = "WAGD";
7 |     array<int, 4> count{};
8 |     for (unsigned char c : s) {
9 |         char x = static_cast<char>(toupper(c));
10 |         for (int i = 0; i < 4; ++i) {
11 |             if (x == order[i])
```



```
12 |         ++count[i];
13 |     }
14 | }
15 | while (true) {
16 |     bool printed = false;
17 |     for (int i = 0; i < 4; ++i) {
18 |         if (count[i]) {
19 |             cout << order[i];
20 |             --count[i];
21 |             printed = true;
22 |         }
23 |     }
24 |     if (!printed)
25 |         break;
26 | }
27 | cout << '\n';
28 | return 0;
29 | }
```

公开样例

样例 1 输入

pcTclnGloRgLrtLhgljkLhGFauPewSKgt

样例 1 输出

WAGGGGG

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

3.41 | 总 168/526 连续五天早八 ( PID 1866 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( F , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( C , CID 1092 )  
| 训练定位：基础提高 | 数组、字符串与双指针

标签：连续段、数组扫描、模拟 | 限制：1.000 S / 128 MB | 历史统计：355/1042 ( 34.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：大学生连续上 5 天早八会精神焕发，输入某位大学生接下来 n 天是否需要上早八，判断他会不会精神焕发。

输入要点：第一行包含一个整数 n (5 ≤ n ≤ 100)。

第二行给出 n 个以空格分隔的整数，0 代表今天没早八，1 代表今天有早八。

输出要点：如果这个大学生会精神焕发，输出 Yes，否则输出 No。

题面提示：样例输入 2

7  
1 0 0 1 1 1 1

样例输出 2

No

零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：扫描 0/1 序列，1 使连续计数加一，0 使其清零；计数达到 5 即存在 连续五天早八。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, consecutive = 0;
7 |     bool found = false;
8 |     cin >> n;
9 |     while (n--) {
10 |         int value;
11 |         cin >> value;
12 |         consecutive = value ? consecutive + 1 : 0;
13 |         found |= consecutive >= 5;
14 |     }
15 |     cout << (found ? "Yes" : "No") << '\n';
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
6
1 1 1 1 1 0
```

样例 1 输出

```
Yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

### 3.42 | 总 169/526 找坐标 ( PID 1867 )

出处：河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕 ( E , CID 1100 ) ；河南工大2024新生周赛 ( 2 ) ——命题人：马家硕 ( D , CID 1092 )  
| 训练定位：基础提高 | 数组、字符串与双指针

标签：二维数组、坐标、遍历 | 限制：1.000 S / 128 MB | 历史统计：327/687 ( 47.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，整数  $k$  所在的行和列，这两个数字用空格分隔。

输入要点：第一行两个整数分别是  $n$  和  $k$ ， $1 \leq n \leq 100$ ， $0 \leq k \leq 9$ 。

接下来输入  $n$  行整数，每行有  $n$  列。其中的每个数  $x$  都满足  $0 \leq x \leq 9$ 。

题目保证要寻找的数字存在且只出现一次。

输出要点：整数  $k$  所在的行和列，这两个数字用空格分隔。

#### 零基础先修

- 这一模块训练线性扫描、下标边界、字符处理、连续段和左右指针。它们频繁出现在周赛前中段，也是后续数据结构与动态规划的语言基础。

#### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：按 1 起行列遍历  $n \times n$  矩阵，读到唯一的  $k$  时记录并输出其坐标。即使已经找到仍可继续完成输入，写法更稳健。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

$O(n^2)$  时间、 $O(1)$  空间

#### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

#### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, target, answerRow = -1, col = -1;
7 |     cin >> n >> target;
8 |     for (int row = 1; row <= n; ++row) {
9 |         for (int column = 1; column <= n; ++column) {
10 |             int value;
11 |             cin >> value;
12 |             if (value == target) {
13 |                 answerRow = row;
14 |                 col = column;
15 |             }
16 |         }
17 |     }
18 |     cout << answerRow << ' ' << col << '\n';
19 |     return 0;
20 | }
```

公开样例

样例 1 输入

```
5 3
8 2 5 5 8
0 1 9 8 6
2 7 2 3 9
2 9 9 0 1
1 1 1 1 1
```

样例 1 输出

```
3 4
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

3.43 | 总 170/526 简单的运算 ( PID 1891 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭铄 ( E , CID 1095 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：153/401 ( 38.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出仅包含一行，第一行一个字符串是符合要求的符号式子，符号之间不需要空格隔开

输入要点：输入分为两行

第一行为字符串的长度

第二行为该字符串

输出要点：输出仅包含一行，第一行一个字符串是符合要求的符号式子，符号之间不需要空格隔开

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：简单的运算 简单的字符串题目，输入后遍历字符串，找出不是数字的元素即可
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int n;
4 | string s;
5 | int main() {
6 |     cin >> n;
7 |     cin >> s;
8 |     vector<char> c;
9 |     for (int i = 0; i < s.size(); i++) {
10 |         if (s[i] < '0' || s[i] > '9')
11 |             c.push_back(s[i]);
12 |     }
13 |     for (int i = 0; i < c.size(); i++)
14 |         cout << c[i];
15 |     return 0;
16 | }
```

公开样例

样例 1 输入

```
7
1+2+3+1
```

样例 1 输出

```
+++
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.44 | 总 171/526 小唐的密码 ( PID 1919 )

出处：河南工大2025新生周赛（2）——命题人：王伟立（C，CID 1103） | 训练定位：基础提高 | 数组、字符串与双指针

标签：条件分支、字符串 | 限制：1.000 S / 128 MB | 历史统计：165/686 ( 24.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：凯撒密码由字符串中的每个字母向后移动  $n$  位形成的。z 的下一个字母是 a，如此循环。现在给出移动前的原文字符串及  $n$ ，请你求出密码。

输入要点：第一行： $n$ 。

第二行：原前的密码。

输出要点：一行，移动后的密码

题面提示：字符串长度  $\leq 50$ ， $1 \leq n \leq 26$ 。

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的密码  $n$  次循环录入字符，并进行移动后依次输出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int n;
5 |     cin >> n;
6 |     char x;
7 |     while (cin >> x) {
8 |         int y = (x - 'a' + n) % 26;
9 |         cout << (char)(y + 'a');
10 |     }
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

```
1
qwe
```

样例 1 输出

```
rxl
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.45 | 总 172/526 银行账单的数字美化 ( PID 1930 )

出处：河南工大2025新生周赛（3）——命题人：路耀华（D，CID 1104） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：112/344 ( 32.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一行带逗号分隔的字符串，表示输出的整数。

输入要点：输入一个整数 $n$ ， $-100000000 \leq n \leq 100000000$ 。

输出要点：输出一行带逗号分隔的字符串，表示输出的整数。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：银行账单的数字美化 注意到数据范围不大，最多可添加两个“，”，可以直接将原数据以1000为基准拆成三部分，判断最高位，并添加“，”。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     int a = 0, b = 0, c = 0, f = 1;
7 |     if (n < 0) {
8 |         f = -1;
9 |         n = -n;
10 |    }
11 |    c = n % 1000;
12 |    n /= 1000;
13 |    if (n > 0) {
14 |        b = n % 1000;
15 |        n /= 1000;
16 |    }
17 |    if (n > 0)
18 |        a = n;
19 |    if (a != 0) {
20 |        printf("%d,%03d,%03d", a * f, b, c);
21 |    } else if (b != 0) {
22 |        printf("%d,%03d", b * f, c);
23 |    } else {
24 |        printf("%d", c * f);
25 |    }
26 |    return 0;
27 | }
```

### 公开样例

样例 1 输入

12345

样例 1 输出

12,345

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.46 | 总 173/526 小唐的真名 ( PID 1933 )

出处：河南工大2025新生周赛 ( 2 ) ——命题人：王伟立 ( E , CID 1103 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：169/383 ( 44.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，找到真名时输出 yes 找不到时输出 no

输入要点：第一行一个数n，n<=2000.

第二行n个字符 ( 仅包含小写英文字母 )

输出要点：找到真名时输出 yes

找不到时输出 no

题面提示：注意七个字符必须连续，如 sbangqiu 不符合要求。

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小唐的真名 循环遍历，匹配字符。 #include <bits/stdc++.h>
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为 O(n)

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。



## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int n;
5 |     cin >> n;
6 |     if (n < 7) {
7 |         cout << "no\n";
8 |         return 0;
9 |     }
10 |     char t0 = 's', t1 = 'a', t2 = 'n', t3 = 'g', t4 = 'q', t5 = 'i', t6 = 'u';
11 |     int k = 0;
12 |     char c;
13 |     for (int i = 0; i < n; i++) {
14 |         cin >> c;
15 |         switch (k) {
16 |             case 0:
17 |                 c == t0 && (k = 1);
18 |                 break;
19 |             case 1:
20 |                 c == t1 ? k = 2 : (k = c == t0 ? 1 : 0);
21 |                 break;
22 |             case 2:
23 |                 c == t2 ? k = 3 : (k = c == t0 ? 1 : 0);
24 |                 break;
25 |             case 3:
26 |                 c == t3 ? k = 4 : (k = c == t0 ? 1 : 0);
27 |                 break;
28 |             case 4:
29 |                 c == t4 ? k = 5 : (k = c == t0 ? 1 : 0);
30 |                 break;
31 |             case 5:
32 |                 c == t5 ? k = 6 : (k = c == t0 ? 1 : 0);
33 |                 break;
34 |             case 6:
35 |                 if (c == t6) {
36 |                     cout << "yes\n";
37 |                     return 0;
38 |                 } else
39 |                     k = c == t0 ? 1 : 0;
40 |                 break;
41 |             }
42 |         } // 会使用数组的话，可以减少大量码量。
43 |         cout << "no\n";
44 |         return 0;
45 |     }
```

## 公开样例

样例 1 输入

```
5
wwwwl
```

样例 1 输出

```
no
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.47 | 总 174/526 小圈子 ( PID 1945 )

出处：河南工大2025新生周赛（7）——命题人：牛见青（E，CID 1109） | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 32 MB | 历史统计：20/31（64.5%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：你的任务就是：给出同学之间的“互相认识”情况后，帮班长算一算，至少要分成几桌。

输入要点：输入以一个整数  $T$ （ $1 \leq T \leq 25$ ）开头，表示有  $T$  组活动安排。

接下来是  $T$  组数据，每组数据格式如下：

-

第一行是两个整数  $N$  和  $M$ （ $1 \leq N, M \leq 1000$ ）。

-

$N$  表示这次活动有多少位同学参加，同学编号为 1 到  $N$ （可以理解成学校给的出席名单编号）。

-

$M$  表示接下来会给出多少条“他们两个互相认识”的信息。

-

接下来  $M$  行，每行有两个整数  $A$  和  $B$ （ $A \neq B$ ），表示编号为  $A$  的同学和编号为  $B$  的同学彼此认识，因此属于同一个圈子。

两组数据之间有一个空行。

输出要点：对每一组数据，输出一个整数，表示至少需要分成多少桌。

不要输出多余空格。

#### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

#### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小圈子 本题考察并查集，数据范围不需要使用路径压缩、按秩合并。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

$O(\text{false})$

#### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int parent[1010];
4 | int findRoot(int x) {
5 |     while (parent[x] != x) {
6 |         x = parent[x];
7 |     }
8 |     return x;
9 | }
10 | bool unite(int a, int b) {
11 |     int ra = findRoot(a);
12 |     int rb = findRoot(b);
13 |     if (ra == rb)
14 |         return false;
15 |     parent[rb] = ra;
16 |     return true;
17 | }
18 | void solve() {
19 |     int N, M;
20 |     cin >> N >> M;
21 |     for (int i = 1; i <= N; ++i) {
22 |         parent[i] = i;
23 |     }
24 |     int groups = N;
25 |     for (int i = 0; i < M; ++i) {
26 |         int A, B;
27 |         cin >> A >> B;
28 |         if (unite(A, B)) {
29 |             groups--;
30 |         }
31 |     }
32 |     cout << groups << "\n";
33 | }
34 | int main() {
35 |     ios::sync_with_stdio(false);
36 |     cin.tie(nullptr);
37 |     int T;
38 |     cin >> T;
39 |     while (T--)
40 |         solve();
41 |     return 0;
42 | }
```

## 公开样例

### 样例 1 输入

```
2
6 4
1 2
2 3
3 4
1 4

8 10
1 2
2 3
5 6
7 5
4 6
3 6
6 7
2 5
2 4
4 3
```

### 样例 1 输出

```
3
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.48 | 总 175/526 Hello,World! ( PID 1955 )

出处：河南工大2025新生周赛（6）——命题人：张康发（D，CID 1107） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、子序列、双指针 | 限制：1.000 S / 128 MB | 历史统计：113/355（31.8%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：给定一个字符串，字符串中不包含空格，以换行结束。你需要判断是否存在按顺序排列的“Hello,World!”字符序列，序列中的字符不要求连续，但必须保持原有顺序。

输入要点：输入为一行长度不超过 1000 的字符串，字符串由 ASCII 字符组成(不包含空格)。

输出要点：如果输入字符串中存在按顺序排列的“Hello,World!”字符序列（字符无需连续），则输出“Yes”；否则输出“No”。

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：用目标串指针 matched 表示已经按顺序匹配了多少字符；扫描输入串，当前字符等于目标的下一字符就推进指针。到行末若指针走完整个目标串 就输出 Yes。公开样例含多行，因此按 EOF 逐行处理，更兼容实际数据。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(输入总长度) 时间、O(1) 额外空间

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const string target = "Hello,World!";
7 |     string text;
8 |     while (cin >> text) {
9 |         size_t matched = 0;
10 |         for (char ch : text) {
11 |             if (matched < target.size() && ch == target[matched])
12 |                 ++matched;
13 |         }
14 |         cout << (matched == target.size() ? "Yes" : "No") << '\n';
15 |     }
16 |     return 0;
}
```

## 公开样例

### 样例 1 输入

```
Hello,World!  
Hewllo,World!
```

### 样例 1 输出

```
Yes  
Yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.49 | 总 176/526 NASA的食物补给 ( PID 1960 )

出处：河南工大2025新生周赛 ( 5 ) ——命题人：袁林坤 ( C , CID 1106 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：73/187 ( 39.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：NASA ( 美国航空航天局 ) 因为航天飞机的隔热瓦等其他安全技术问题一直大伤脑筋，因此在各方压力下终止了航天飞机的历史，但是此类事情会不会在以后发生，谁也无法保证。所以，在遇到这类航天问题时，也许只能让航天员出仓维修。但是过多的维修会消耗航天员大量的能量，因此 NASA 便想设计一种食品方案，使体积和承重有限的条件下多装载一些高卡路里的食物。航天飞机的体积有限，.....

输入要点：第一行 2 个整数，分别代表体积最大值 H 和质量最大值 T。

第二行 1 个整数代表食品总数 n。

接下来 n 行每行 3 个数 体积 hi，质量 ti，所含卡路里 ki。

输出要点：一个数，表示所能达到的最大卡路里 ( int 范围内 )

题面提示：0<=H,T<=500

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：NASA的食物补给 01背包题 `#include<bits/stdc++.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

$O(1)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define endl '\n'
4 | #define ull unsigned long long
5 | #define ll long long
6 | #define PII pair<int, int>
7 | const int N = 2e3 + 10;
8 | const int mod = 1e9 + 7;
9 | const int P = 1;
10 | int n, m;
11 | int h, t;
12 | int f[N][N], c[N];
13 | int v[N], w[N];
14 | void solve() {
15 |     cin >> h >> t;
16 |     cin >> n;
17 |     for (int i = 1; i <= n; i++) {
18 |         int hh, tt, kk;
19 |         cin >> hh >> tt >> kk;
20 |         for (int j = h; j >= hh; j--)
21 |             for (int k = t; k >= tt; k--)
22 |                 f[j][k] = max(f[j][k], f[j - hh][k - tt] + kk);
23 |     }
24 |     cout << f[h][t];
25 | }
26 | int main() {
27 |     ios::sync_with_stdio(0);
28 |     cin.tie(0);
29 |     cout.tie(0);
30 |     int T = 1;
31 |     // cin >> T;
32 |     while (T--)
33 |         solve();
34 |     return 0;
35 | }
```

## 公开样例

样例 1 输入

```
320 350
4
160 40 120
80 110 240
220 70 310
40 400 220
```

样例 1 输出

```
550
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.50 | 总 177/526 时空快递的“跨天送达”(PID 1961)

出处：河南工大2025新生周赛（3）——命题人：路耀华（B，CID 1104）| 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：94/290（32.4%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，输出一行字符串，代表送达时间戳，格式与输入一致。

输入要点：输入两行，第一行输入出发时间戳，格式为"YYYY-MM-DD HH:MM:SS"，

第二行输入一个整数 $t, 1 \leq t \leq 100000000$ 。

输出要点：输出一行字符串，代表送达时间戳，格式与输入一致。

题面提示：注意特殊日期

#### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

#### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：时空快递的“跨天送达”从出发时间开始，循环判断并进位，推断出送达时间，注意闰年2月，注意格式。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

通常为  $O(n)$

#### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

#### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int isLeapYear(int year) {
4 |     return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
5 | }
6 | int daysInMonth(int year, int month) {
7 |     if (month == 2)
8 |         return isLeapYear(year) ? 29 : 28;
9 |     else if (month == 4 || month == 6 || month == 9 || month == 11)
10 |         return 30;
11 |     else
12 |         return 31;
13 | }
14 | int main() {
15 |     int year, month, day;
16 |     int hour, minute, second;
17 |     int t;
18 |     scanf("%d-%d-%d %d:%d:%d", &year, &month, &day, &hour, &minute, &second);
19 |     scanf("%d", &t);
20 |     t += second;
21 |     second = t % 60;
22 |     minute += t / 60;
23 |     hour += minute / 60;
24 |     minute %= 60;
25 |     day += hour / 24;
26 |     hour %= 24;
```

```
27 |         while (day > daysInMonth(year, month)) {
28 |             day -= daysInMonth(year, month);
29 |             month++;
30 |             if (month > 12) {
31 |                 month = 1;
32 |                 year++;
33 |             }
34 |         }
35 |         printf("%04d-%02d-%02d %02d:%02d:%02d\n", year, month, day, hour, minute, second);
36 |         return 0;
37 |     }
```

公开样例

样例 1 输入

2025-11-11 00:00:00  
100

样例 1 输出

2025-11-11 00:01:40

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.51 | 总 178/526 接水 ( PID 1964 )

出处：河南工大2025新生周赛（5）——命题人：袁林坤（D，CID 1106） | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：47/108 ( 43.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示接水所需的总时间。

输入要点：第一行两个整数 n 和 m，用一个空格隔开，分别表示接水人数和龙头个数。

第二行 n 个整数 w1,w2,...,wn，每两个整数之间用一个空格隔开，wi 表示 i 号同学的接水量。

输出要点：一个整数，表示接水所需的总时间。

题面提示：样例说明：

第 1 秒，3 人接水。第 1 秒结束时，1,2,3 号同学每人的已接水量为 1,3 号同学接完水，4 号同学接替 3 号同学开始接水。

第 2 秒，3 人接水。第 2 秒结束时，1,2 号同学每人的已接水量为 2,4 号同学的已接水量为 1。

第 3 秒，3 人接水。第 3 秒结束时，1,2 号同学每人的已接水量为 3,4 号同学的已接水量为 2。4 号同学接完水，5 号同学接替 4 号同学开始接水。

第 4 秒，3 人接水。第 4 秒结束时，1,2 号同学每人的已接水量为 4,5 号同学的已接水量为 1。1,2,5 号同学接完水，即所有人完成接水的总接水时间为 4 秒。

1≤n≤104，1≤m≤100，m≤n；

1≤wi≤100。

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导



1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：接水过程即可。 `#include<iostream> #include<algorithm>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <algorithm>
3 | using namespace std;
4 | int a[10005];
5 | int n, m;
6 | int s[105];
7 | int t;
8 | int main(void) {
9 |     cin >> n >> m;
10 |    for (int i = 1; i <= n; i++)
11 |        cin >> a[i];
12 |    for (int i = 1; i <= n; i++) {
13 |        t = 1;
14 |        for (int j = 2; j <= m; j++)
15 |            if (s[t] > s[j])
16 |                t = j;
17 |        s[t] += a[i];
18 |    }
19 |    int mx = 0;
20 |    for (int i = 1; i <= m; i++) {
21 |        mx = max(s[i], mx);
22 |    }
23 |    cout << mx;
24 | }
```

## 公开样例

样例 1 输入

```
5 3
4 4 1 2 1
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.52 | 总 179/526 阳台接雨 ( PID 1966 )

出处：河南工大2025新生周赛 ( 5 ) ——命题人：袁林坤 ( E , CID 1106 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：28/107 ( 26.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示这排花架最多能储存的立方米雨水。

输入要点：第一行一个整数 $n$ ，表示 $n$ 块挡板。

接下来是挡板高度数组 $H=[a_1,a_2,\dots,a_n]$ 。

输出要点：一个整数，表示这排花架最多能储存的立方米雨水。

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251118/20251118134849\\_98431.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251118/20251118134849_98431.png)]

$1 \leq n \leq 2 * 10^4$

$0 \leq a[i] \leq 10^5$

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：阳台接雨 线性dp,两个数组分别存储前后最大值，两边最大值的最小值与当前值相减即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(0)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define endl '\n'
4 | #define ull unsigned long long
5 | #define ll long long
6 | #define PII pair<int, int>
7 | const ll N = 1e4 + 10;
8 | const int mod = 80112002;
9 | const int P = 1;
10 | void solve() {
11 |     int n;
12 |     cin >> n;
13 |     vector<int> h(n);
14 |     for (int i = 0; i < n; i++)
15 |         cin >> h[i];
16 |     vector<int> ma1(n + 1, 0), ma2(n + 1, 0);
17 |     int sum1 = 0, sum2 = 0;
18 |     for (int i = 0; i < n; i++) {
```

```

19 |         if (!i)
20 |             ma1[i] = h[i];
21 |         else
22 |             ma1[i] = max(ma1[i - 1], h[i]);
23 |     }
24 |     for (int i = n - 1; i >= 0; i--) {
25 |         ma2[i] = max(ma2[i + 1], h[i]);
26 |     }
27 |     for (int i = 0; i < n; i++) {
28 |         sum1 += min(ma1[i], ma2[i]);
29 |         sum2 += h[i];
30 |     }
31 |     cout << sum1 - sum2 << endl;
32 | }
33 | int main() {
34 |     ios::sync_with_stdio(0);
35 |     cin.tie(0);
36 |     cout.tie(0);
37 |     int T = 1;
38 |     // cin >> T;
39 |     while (T--)
40 |         solve();
41 |     return 0;
42 | }

```

## 公开样例

样例 1 输入

```

12
0 1 0 2 1 0 1 3 2 1 2 1

```

样例 1 输出

```

6

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.53 | 总 180/526 校园文化节宣传标语 ( PID 1967 )

出处：河南工大2025新生周赛（5）——命题人：袁林坤（G，CID 1106） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、双指针、回文 | 限制：1.000 S / 128 MB | 历史统计：50/200（25.0%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**请你编写一个程序，接收这条宣传标语字符串，判断其是否满足上述回文条件，返回true或false。

**输入要点：**一段字符串。

**输出要点：**true或false。

题面提示：-

输入字符串长度范围： $0 \leq \text{len}(s) \leq 10000$ ；

-  
字符串仅包含：大写英文字母（A-Z）、小写英文字母（a-z）、空格（ ）、英文标点（,;!.）；

-  
验证规则：忽略大小写（A 和 a 视为相同）、忽略所有空格和标点，仅对比字母序列是否对称。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：用 getline 读取整行；双指针从两端向中间移动，跳过非英文字母，比较时统一转成小写。任一对有效字符不同就不是回文，否则是回文。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  额外空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string text;
5 |     getline(cin, text);
6 |     int left = 0, right = int(text.size()) - 1;
7 |     bool palindrome = true;
8 |     while (left < right) {
9 |         while (left < right && !isalpha(static_cast<unsigned char>(text[left])))
10 |             ++left;
11 |         while (left < right && !isalpha(static_cast<unsigned char>(text[right])))
12 |             --right;
13 |         if (tolower(static_cast<unsigned char>(text[left])) !=
14 |             tolower(static_cast<unsigned char>(text[right]))) {
15 |             palindrome = false;
16 |             break;
17 |         }
18 |         ++left;
19 |         --right;
20 |     }
21 |     cout << (palindrome ? "true" : "false") << '\n';
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

A man, a plan, a canal: Panama

样例 1 输出

true

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

3.54 | 总 181/526 这是一道签到题 ( PID 1975 )

出处：河南工大2025新生周赛 ( 6 ) ——命题人：张康发 ( B , CID 1107 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：126/273 ( 46.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一行一个整数，表示最少还需要向小 S 借几张牌才能凑成一副完整的扑克牌。

输入要点：输入的第一行包含一个整数 n 表示牌数。

接下来 n 行：每行包含一个长度为 2 的字符串描述一张牌，其中第一个字符描述其花色，第二个字符描述其点数。

输出要点：输出一行一个整数，表示最少还需要向小 S 借几张牌才能凑成一副完整的扑克牌。

题面提示：对于所有测试数据，保证：1≤n≤52，输入的 n 个字符串每个都代表一张合法的扑克牌，即字符串长度为 2，且第一个字符为 D C H S 中的某个字符，第二个字符为 A 2 3 4 5 6 7 8 9 T J Q K 中的某个字符。

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：这是一道签到题 对于每一种牌，用数字映射到二维数组中，用来记录是否已经借过这张牌，若之前没有借过，则记录，并将牌总数减一。最后输出剩余牌的数量即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为 O(n)

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和 n=1。
- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | typedef long long ll;
3 | using namespace std;
4 | const int N = 1e3 + 10;
5 | int n;
6 | int cnt[N][N];
7 | int main() {
```

```

8 |     cin >> n;
9 |     int ans = 52;
10 |     for (int i = 1; i <= n; i++) {
11 |         string op;
12 |         cin >> op;
13 |         // cout << op << endl;
14 |         int f = -1, s = -1;
15 |         if (op[1] == 'A') {
16 |             f = 1;
17 |         } else if (op[1] == '2') {
18 |             f = 2;
19 |         } else if (op[1] == '3') {
20 |             f = 3;
21 |         } else if (op[1] == '4') {
22 |             f = 4;
23 |         } else if (op[1] == '5') {
24 |             f = 5;
25 |         } else if (op[1] == '6') {
26 |             f = 6;
27 |         } else if (op[1] == '7') {
28 |             f = 7;
29 |         } else if (op[1] == '8') {
30 |             f = 8;
31 |         } else if (op[1] == '9') {
32 |             f = 9;
33 |         } else if (op[1] == 'T') {
34 |             f = 10;
35 |         } else if (op[1] == 'J') {
36 |             f = 11;
37 |         } else if (op[1] == 'Q') {
38 |             f = 12;
39 |         } else if (op[1] == 'K') {
40 |             f = 13;
41 |         }
42 |         if (op[0] == 'D') {
43 |             s = 1;
44 |         } else if (op[0] == 'C') {
45 |             s = 2;
46 |         } else if (op[0] == 'H') {
47 |             s = 3;
48 |         } else if (op[0] == 'S') {
49 |             s = 4;
50 |         }
51 |         if (cnt[f][s] == 0)
52 |             ans--;
53 |         cnt[f][s]++;
54 |     }
55 |     cout << ans << endl;
56 |     return 0;
57 | }

```

## 公开样例

样例 1 输入

```

4
DQ
H3
DQ
DT

```

样例 1 输出

```

49

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.55 | 总 182/526 回声 ( PID 1985 )

出处：河南工大2025新生周赛（7）——命题人：牛见青（A，CID 1109） | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、词法扫描、模拟 | 限制：1.000 S / 128 MB | 历史统计：66/212（31.1%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，一行，相应回声的单词，保持原本的大小写，中间用空格隔开。

输入要点：一行，一句英文，可能有标点符号。

输出要点：一行，相应回声的单词，保持原本的大小写，中间用空格隔开。

题面提示：题目保证只有字母、符号与空格，且空格不出现在首尾。

可能出现连字符，如 twenty-one，视为一个单词。

#### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

#### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：把字母和位于单词内部的连字符保留，其他符号都当作分隔符。扫描得到所有单词后，从 max(0,数量-3) 开始输出，因少于三个单词时也能自然输出全部内容；大小写保持原样。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

#### 复杂度

O(|line|) 时间、O(|line|) 空间

#### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

#### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string line;
5 |     getline(cin, line);
6 |     vector<string> words;
7 |     string cur;
8 |     for (int i = 0; i < (int)line.size(); ++i) {
9 |         unsigned char c = line[i];
10 |         bool letter = isalpha(c);
11 |         bool innerHyphen = line[i] == '-' && i > 0 && i + 1 < (int)line.size() &&
12 |             isalpha((unsigned char)line[i - 1]) &&
13 |             isalpha((unsigned char)line[i + 1]);
14 |         if (letter || innerHyphen)
15 |             cur += line[i];
16 |         else if (!cur.empty()) {
17 |             words.push_back(cur);
18 |             cur.clear();
19 |         }
20 |     }
```

```
21 |     if (!cur.empty())
22 |         words.push_back(cur);
23 |     int start = max(0, (int)words.size() - 3);
24 |     for (int i = start; i < (int)words.size(); ++i) {
25 |         if (i > start)
26 |             cout << ' ';
27 |         cout << words[i];
28 |     }
29 |     cout << '\n';
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

Hello, Echo!!

样例 1 输出

Hello Echo

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.56 | 总 183/526 身份证校验 ( PID 1986 )

出处：河南工大2025新生周赛 ( 7 ) ——命题人：牛见青 ( C , CID 1109 ) | 训练定位：基础提高 | 数组、字符串与双指针

标签：字符串、查表、模拟 | 限制：1.000 S / 128 MB | 历史统计：173/387 ( 44.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，18位的完整身份证号。

输入要点：一串数字，代表身份证的前17位。

输出要点：18位的完整身份证号。

题面提示：本题不考虑身份证号是否可能真实存在。如前17位可能是0000000000000000，而在现实中不存在这种情况。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：前 17 位分别乘固定权重并求和，对 11 取模。用字符串 "10X98765432" 直接按余数下标查校验码，再接到原 17 位之后输出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。



- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

O(17) 时间、O(1) 空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string id;
5 |     cin >> id;
6 |     const int weight[17] = {7, 9, 10, 5, 8, 4, 2, 1, 6, 3, 7, 9, 10, 5, 8, 4, 2};
7 |     const string check = "10X98765432";
8 |     int sum = 0;
9 |     for (int i = 0; i < 17; ++i)
10 |         sum += (id[i] - '0') * weight[i];
11 |     cout << id << check[sum % 11] << '\n';
12 |     return 0;
13 | }
```

### 公开样例

样例 1 输入

12345678901234567

样例 1 输出

123456789012345677

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.57 | 总 184/526 多项式相加 (多实例) (PID 1199)

出处：河南工大2025新生周赛（6）——命题人：张康发（G，CID 1107）；2016级河南工大新生周赛（一）（A，CID 1006）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：21/449（4.7%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：程序要处理的幂最大为100。

输入要点：第一行输入一个整数t，表示有t组数据。

对于每组数据：

总共要输入两个多项式，每个多项式的输入格式如下：

每行输入两个数字，第一个表示幂次，第二个表示该幂次的系数，所有的系数都是整数。第一行一定是最高幂，最后一行一定是0次幂。

注意第一行和最后一行之间不一定按照幂次降低顺序排列；如果某个幂次的系数为0，就不出现在输入数据中了；0次幂的系数为0时还是会出现在输入数据中。

输出要点：对于每组数据：

从最高幂开始依次降到0幂，如：

$2x^6+3x^5+12x^3-6x+20$

注意其中的x是小写字母x，而且所有的符号之间都没有空格，如果某个幂的系数为0则不需要有那项。

题面提示：`%+d` 可输出符号，正号或负号

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：多项式相加（多实例）输入多项式求和，高次到0次规范输出结果。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int t, i, j, x, y;
5 |     scanf("%d", &t);
6 |     for (i = 1; i <= t; i++) {
7 |         int a[110] = {0};
8 |         int b[110] = {0};
9 |         int c[110] = {0};
10 |         while (scanf("%d %d", &x, &y), x != -1) {
11 |             a[x] += y;
12 |         }
13 |         while (scanf("%d %d", &x, &y), x != -1) {
14 |             b[x] += y;
15 |         }
16 |         for (j = 0; j <= 100; j++) {
17 |             c[j] = b[j] + a[j];
18 |         }
19 |         int z = 0;
20 |         for (j = 100; j > 1; j--) {
21 |             if (z == 0 && c[j] != 0) {
22 |                 if (c[j] == 1) {
23 |                     printf("x%d", j);
24 |                 } else if (c[j] == -1) {
25 |                     printf("-x%d", j);
26 |                 } else {
27 |                     printf("%dx%d", c[j], j);
28 |                 }
29 |                 z++;
30 |             } else if (c[j] != 0) {
31 |                 if (c[j] == 1) {
32 |                     printf("+x%d", j);
```

```

33 |         } else if (c[j] == -1) {
34 |             printf("-x%d", j);
35 |         } else {
36 |             printf("%+dx%d", c[j], j);
37 |         }
38 |     }
39 | }
40 | if (c[1] != 0) {
41 |     if (c[1] == 1) {
42 |         if (z == 0)
43 |             printf("x");
44 |         else
45 |             printf("+x");
46 |     } else if (c[1] == -1) {
47 |         printf("-x");
48 |     } else {
49 |         if (z == 0)
50 |             printf("%dx", c[1]);
51 |         else
52 |             printf("%+dx", c[1]);
53 |     }
54 |     z++;
55 | }
56 | if (c[0] != 0) {
57 |     if (z == 0)
58 |         printf("%d", c[0]);
59 |     else
60 |         printf("%+d", c[0]);
61 |     z++;
62 | }
63 | if (z == 0) {
64 |     printf("0");
65 | }
66 | printf("\n");
67 | }
68 | return 0;
69 | }

```

## 公开样例

### 样例 1 输入

```

1
6 2
5 3
3 12
1 6
0 20
-1 -1
6 2
5 3
2 12
1 6
0 20
-1 -1

```

### 样例 1 输出

```

4x6+6x5+12x3+12x2+12x+40

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.58 | 总 185/526 txt的号码 ( PID 1208 )

出处：2016级河南工大新生周赛 ( 二 ) ( D , CID 1007 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、校验码、加权求和 | 限制：1.000 S / 128 MB | 历史统计：7/69 ( 10.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你的任务是编写程序判断输入的ISBN号码中识别码是否正确，如果正确，则仅输出“Right”；如果错误，则输出你认为是正确的ISBN号码。

输入要点：输入只有一行，是一个字符序列，表示一本书的ISBN号码（保证输入符合ISBN号码的格式要求）。

输出要点：输出共一行，假如输入的ISBN号码的识别码正确，那么输出“Right”，否则，按照规定的格式，输出正确的ISBN号码（包括分隔符“-”）。

题面提示：各个测试点1s

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：忽略三个连字符，取前 9 个数字，按从 1 到 9 的权重相乘求和再模 11。余数 0..9 对应字符 '0'..'9'，10 对应大写 'X'。若它与输入 最后一位相同输出 Right；否则只替换最后一个字符，保留原有连字符格式 后输出整个 ISBN。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(|s|)$  时间、 $O(|s|)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string isbn;
5 |     cin >> isbn;
6 |     int sum = 0, pos = 1;
7 |     for (char c : isbn) {
8 |         if (isdigit((unsigned char)c) && pos <= 9) {
9 |             sum += (c - '0') * pos;
10 |             ++pos;
11 |         }
12 |     }
13 |     int rem = sum % 11;
14 |     char correct = rem == 10 ? 'X' : char('0' + rem);
15 |     if (isbn.back() == correct) {
16 |         cout << "Right\n";
17 |     } else {
18 |         isbn.back() = correct;
19 |         cout << isbn << '\n';
20 |     }
21 |     return 0;
22 | }
```

### 公开样例

样例 1 输入

0-670-82162-4

样例 1 输出

Right

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.59 | 总 186/526 kx的压缩 ( PID 1209 )

出处：2016级河南工大新生周赛 ( 二 ) ( F , CID 1007 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：游程编码、字符串、模拟 | 限制：1.000 S / 128 MB | 历史统计：1/28 ( 3.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，压缩码。

输入要点：N表示为N\*N的矩阵 (  $3 \leq N \leq 100$  )

汉字点阵图 ( 点阵符号之间不留空格 ) 一行。

输出要点：一行，压缩码。

题面提示：N<=100

【认真读题哦】

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：把 N×N 点阵按题面给出的行优先顺序视作一个长串，依次统计连续 0、连续 1、连续 0...的长度。编码规定第一项总表示 0 的段，因此若输入一开始就是 1，要先输出长度 0。之后每次字符变化就输出上一段计数，最后别忘了输出末段。N 本身不属于样例输出的压缩码。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

O(N²) 时间、O(段数) 空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     string bitmap;
8 |     cin >> n >> bitmap;
9 |     vector<int> runs;
10 |     char expected = '0';
11 |     int len = 0;
12 |     for (char bit : bitmap) {
13 |         if (bit == expected) {
14 |             ++len;
15 |         } else {
16 |             runs.push_back(len);
17 |             expected = bit;
18 |             len = 1;
19 |         }
20 |     }
21 |     runs.push_back(len);
22 |     for (int i = 0; i < (int)runs.size(); ++i) {
23 |         if (i)
24 |             cout << ' ';
25 |         cout << runs[i];
26 |     }
27 |     cout << '\n';
28 |     return 0;
29 | }
```

## 公开样例

样例 1 输入

```
7
0001000000100000011110001000000100000010001111111
```

样例 1 输出

```
3 1 6 1 6 4 3 1 6 1 6 1 3 7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.60 | 总 187/526 Simple学长喝咖啡 ( PID 1235 )

出处：2016级河南工大新生周赛 ( 六 ) ( F , CID 1012 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：大数取模、字符串、整除 | 限制：1.000 S / 128 MB | 历史统计：14/277 ( 5.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：Simple学长的咖啡都有一个编号，但是Simple学长只喝编号能被6整除的咖啡，Simple学长有好多咖啡，所以他的咖啡的编号长度可达1000位，现在给你一个编号，问你Simple学长是否会喝这杯咖啡。

输入要点：输入一个T，表示有T组数据

每组数据输入一个数字，数字的长度不大于1000

输出要点：如果Simple学长喝这杯咖啡的话，输出YES，否者输出NO

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：编号长达 1000 位，要在读数字的同时取模。若当前余数为  $r$ ，追加一位  $d$  后新余数是  $(r*10+d)\%6$ 。扫描完整个字符串后余数为 0 就能被 6 整除。也可用“末位偶数且数位和为 3 的倍数”，但逐位取模可推广到任何除数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(|s|)$  时间、 $O(1)$  额外空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string number;
10 |        cin >> number;
11 |        int rem = 0;
12 |        for (char digit : number)
13 |            rem = (rem * 10 + digit - '0') % 6;
14 |        cout << (rem == 0 ? "YES" : "NO") << '\n';
15 |    }
16 |    return 0;
17 | }
```

## 公开样例

样例 1 输入

```
5
6
66
666
6666
66667
```

样例 1 输出

```
YES
YES
YES
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.61 | 总 188/526 神奇的井 ( PID 1244 )

出处：2016级河南工大新生周赛（七）（H，CID 1014） | 训练定位：进阶 | 数组、字符串与双指针

标签：单调预处理、双指针、模拟 | 限制：1.000 S / 128 MB | 历史统计：0/10 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出最终落到井内的盘子数量。

输入要点：第1行：2个数N, M中间用空格分隔，N为井的深度，M为盘子的数量( $1 \leq N, M \leq 50000$ )。

第2 - N + 1行，每行1个数，对应井的宽度 $W_i$ ( $1 \leq W_i \leq 10^9$ )。

第N + 2 - N + M + 1行，每行1个数，对应盘子的宽度 $D_i$ ( $1 \leq D_i \leq 10^9$ )

输出要点：输出最终落到井内的盘子数量。

### 零基础先修

- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：盘子下落时能否经过某层，取决于从井口到该层的最窄宽度。因此先把 井宽改为前缀最小值，使其单调不增。维护下一个可放置的最深层 position；对每只盘子从该位置向上找首个宽度不小于盘宽的层，放下后 position 减一。若越过井口，后续盘子都无法进入，停止。指针总共只向上走 N 次。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(N+M)$  时间、 $O(N)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int depth, plateCount;
7 |     cin >> depth >> plateCount;
8 |     vector<long long> width(depth);
9 |     for (long long& x : width)
10 |         cin >> x;
```



```

11 |     for (int i = 1; i < depth; ++i)
12 |         width[i] = min(width[i], width[i - 1]);
13 |     int pos = depth - 1;
14 |     int placed = 0;
15 |     for (int i = 0; i < plateCount; ++i) {
16 |         long long plate;
17 |         cin >> plate;
18 |         if (pos < 0)
19 |             continue;
20 |         while (pos >= 0 && width[pos] < plate)
21 |             --pos;
22 |         if (pos >= 0) {
23 |             ++placed;
24 |             --pos;
25 |         }
26 |     }
27 |     cout << placed << '\n';
28 |     return 0;
29 | }

```

## 公开样例

样例 1 输入

```

7 5
5
6
4
3
6
2
3
2
3
5
2
4

```

样例 1 输出

```

4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.62 | 总 189/526 Simple学长的a+b ( PID 1256 )

出处：2016级河南工大新生周赛总决赛（G，CID 1016） | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串解析、表达式、模拟 | 限制：1.000 S / 128 MB | 历史统计：14/125 ( 11.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**给出一个表达式,其中运算符仅包含+,-,要求求出表达式的最终值.

**输入要点：**输入一个T，表示有T组数据

每一行有一个表达式，数值只有正值，数的个数小于1000，每个数的数值小于1000

**输出要点：**输出一行，表达式的值

### 零基础先修

- 解析字符串时先确定分隔符、字符类别和多位数边界，再逐段转换。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：表达式只有非负整数以及 '+'、'-'。逐字符累计当前数字；遇到运算符 时，用此前保存的符号把当前数字加入答案，再清零并更新符号。字符串结尾 可人为补一个 '+'，从而让最后一个数也走同一套结算逻辑。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(|s|)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string expr;
10 |        cin >> expr;
11 |        expr.push_back('+');
12 |        long long ans = 0, number = 0;
13 |        int sign = 1;
14 |        for (char c : expr) {
15 |            if (isdigit((unsigned char)c)) {
16 |                number = number * 10 + (c - '0');
17 |            } else {
18 |                ans += sign * number;
19 |                number = 0;
20 |                sign = c == '+' ? 1 : -1;
21 |            }
22 |        }
23 |        cout << ans << '\n';
24 |    }
25 |    return 0;
26 | }
```

## 公开样例

样例 1 输入

```
1
1+1-1
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.63 | 总 190/526 大大大大大扫除 ( PID 1393 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( F , CID 1041 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、位置映射、模拟 | 限制：1.000 S / 128 MB | 历史统计：4/41 ( 9.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，第一行  $k$  个整数，表示第  $k$  次操作移动的书的总数 第二行，如果能能把全部的书挪走，则输出yes，反之输出no

输入要点：第一行一个数  $n$ ，代表  $n$  本书 (  $0 < n \leq 200000$  )

第二行  $n$  个数，代表书的编号和拜访次序

第三行一个数  $k$ ，代表  $k$  次操作 (  $0 < k \leq n$  )

第四行  $k$  个数，代表第  $k$  次操作的书的编号

输出要点：第一行  $k$  个整数，表示第  $k$  次操作移动的书的总数

第二行，如果能能把全部的书挪走，则输出yes，反之输出 no

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：记录每本书在原排列中的位置  $pos$ 。由于每次都取当前目标及其之前的全部书，已移走部分始终是一个原排列前缀。维护前缀终点  $removed$ ：若  $pos[w] > removed$ ，本次移动  $pos[w] - removed$  本并更新终点；否则为 0。最终  $removed = n$  就已全部移走。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(n+k)$  时间、 $O(n)$  空间

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> pos(n + 1);
```

```

9 |         for (int i = 1, book; i <= n; ++i) {
10 |             cin >> book;
11 |             pos[book] = i;
12 |         }
13 |         int k, removed = 0;
14 |         cin >> k;
15 |         for (int i = 0; i < k; ++i) {
16 |             int book;
17 |             cin >> book;
18 |             int moved = max(0, pos[book] - removed);
19 |             removed = max(removed, pos[book]);
20 |             if (i)
21 |                 cout << ' ';
22 |             cout << moved;
23 |         }
24 |         cout << '\n' << (removed == n ? "yes" : "no") << '\n';
25 |         return 0;
26 |     }

```

## 公开样例

### 样例 1 输入

```

5
3 1 4 2 5
5
4 5 1 3 2

```

### 样例 1 输出

```

3 2 0 0 0
yes

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.64 | 总 191/526 Poem ( PID 1426 )

出处：2018级河南工大新生周赛总决赛（重现）（J，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（J，CID 1048） | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、回文串、分类讨论 | 限制：1.000 S / 16 MB | 历史统计：29/185 ( 15.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果修改后可以成为回文字符串，输出"YES"，否则输出"NO!"

输入要点：先输入一个T表示有T组测试数据（ $T \leq 30$ ）

每组测试数据为一个只包含大写字母的字符串，长度不超过30

输出要点：如果修改后可以成为回文字符串，输出"YES"，否则输出"NO!"

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：比较每一对对称字符，统计不相等的对数。超过一对不相等时，一次改动 不可能全部修好；恰有一对时，改其中一个字符即可。若原串已经回文，题目还要求新名字与原名不同：奇数长度可以只改中间字符且仍然回文，偶数长度改任何一个字符都会破坏对称

, 所以分别为 YES 与 NO。注意失败 文本是带感叹号的 'NO!'。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(|s|)$  时间、 $O(1)$  额外空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string s;
10 |        cin >> s;
11 |        int mismatches = 0;
12 |        for (int left = 0, right = (int)s.size() - 1; left < right; ++left, --right) {
13 |            mismatches += (s[left] != s[right]);
14 |        }
15 |        bool possible = mismatches == 1 || (mismatches == 0 && s.size() % 2 == 1);
16 |        cout << (possible ? "YES" : "NO!") << '\n';
17 |    }
18 |    return 0;
19 | }
```

### 公开样例

样例 1 输入

```
2
AABA
QB
```

样例 1 输出

```
YES
YES
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)

## 3.65 | 总 192/526 cxx下棋 ( PID 1475 )

出处：2019级河南工大新生周赛（一）@陶福专场（C，CID 1053）| 训练定位：进阶 | 数组、字符串与双指针

标签：枚举、二维数组、井字棋 | 限制：1.000 S / 128 MB | 历史统计：35/174 ( 20.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：cxx下棋

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

## 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
- 核心处理采用：枚举 3 行、3 列和 2 条对角线，若三个字符相同就得到赢家标记。题目保证必有一人获胜，因此最终按 O 或 X 输出对应文本。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     vector<string> board(3);
5 |     for (string& row : board)
6 |         cin >> row;
7 |     char winner = '?';
8 |     for (int i = 0; i < 3; ++i) {
9 |         if (board[i][0] == board[i][1] && board[i][1] == board[i][2]) {
10 |             winner = board[i][0];
11 |         }
12 |         if (board[0][i] == board[1][i] && board[1][i] == board[2][i]) {
13 |             winner = board[0][i];
14 |         }
15 |     }
16 |     if (board[0][0] == board[1][1] && board[1][1] == board[2][2]) {
17 |         winner = board[1][1];
18 |     }
19 |     if (board[0][2] == board[1][1] && board[1][1] == board[2][0]) {
20 |         winner = board[1][1];
21 |     }
22 |     cout << (winner == 'O' ? "CXKNB" : "AWNBN") << '\n';
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
OXO
XXO
OXX
```

样例 1 输出

```
AWNBN
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.66 | 总 193/526 郁郁寡欢 ( PID 1482 )

出处：2019级河南工大新生周赛 ( 二 ) @邹岩专场 ( C , CID 1054 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：摩尔投票、数组与序列、计数 | 限制：1.000 S / 64 MB | 历史统计：9/192 ( 4.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数即出现次数超过一半的数；

输入要点：第一行输入一个 $n(1 \leq n \leq 1e6)$ ；

第二行是 $n$ 个整数 $m$  ( $m \leq 1e18$ ) 代表每个朋友的评分；( 必定有一个数字出现次数超过一半 )

输出要点：输出一个整数即出现次数超过一半的数；

题面提示：每次输入都符合要求

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：题目保证存在严格过半元素，可用 Boyer-Moore 投票：相同票加一，不同票抵消，票数归零时更换候选。多数元素无法被其余元素全部抵消，最终候选必为答案；评分需用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, votes = 0;
7 |     long long cand = 0;
8 |     cin >> n;
9 |     while (n--) {
10 |         long long x;
```

```

11 |         cin >> x;
12 |         if (votes == 0)
13 |             cand = x, votes = 1;
14 |         else if (x == cand)
15 |             ++votes;
16 |         else
17 |             --votes;
18 |     }
19 |     cout << cand << '\n';
20 |     return 0;
21 | }

```

## 公开样例

样例 1 输入

```

5
2 2 2 3 3

```

样例 1 输出

```

2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.67 | 总 194/526 如果能重来我要选Jinx ( PID 1496 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( B , CID 1056 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、差分、模拟 | 限制：1.000 S / 128 MB | 历史统计：6/179 ( 3.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给了两个大小相同的数组a,b。现判断是否能通过一次给a中某个区间的元素都加上相同值（该值大于等于0），从而使数组a,b相等。

输入要点：第一行输入一个n  $n \leq 10000$

第二行，第三行各输入n个值表示数组a,b。

数组内最大值为1000。

输出要点：输出YES或NO

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：令  $d[i]=b[i]-a[i]$ 。一次给连续区间加非负常数，意味着 d 必须 形如“若干 0、若干个相同的正数、若干 0”。扫描时记录首个正差值，进入正段后若遇到不同正数或离开正段后又遇正数，就不可能；任意  $d < 0$  也立即失败。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明



- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     vector<long long> a(n);
7 |     for (long long& x : a)
8 |         cin >> x;
9 |     long long value = -1;
10 |    bool ended = false, possible = true;
11 |    for (int i = 0; i < n; ++i) {
12 |        long long b;
13 |        cin >> b;
14 |        long long diff = b - a[i];
15 |        if (diff < 0)
16 |            possible = false;
17 |        else if (diff == 0) {
18 |            if (value != -1)
19 |                ended = true;
20 |        } else {
21 |            if (ended)
22 |                possible = false;
23 |            if (value == -1)
24 |                value = diff;
25 |            else if (value != diff)
26 |                possible = false;
27 |        }
28 |    }
29 |    cout << (possible ? "YES" : "NO") << '\n';
30 |    return 0;
31 | }
```

## 公开样例

样例 1 输入

```
6
3 7 1 4 1 2
3 7 3 6 3 2
```

样例 1 输出

```
YES
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.68 | 总 195/526 CXK的篮球数 ( 加强版 ) ( PID 1504 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( A , CID 1059 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：22/120 ( 18.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输入第x筐中篮球数

输入要点：第一行输入n,m。第二行输入n个数，代表初始每筐中篮球数，接下来m行，每行包括三个数L,R,W。最后一行输入一个数x。(  $1 \leq x \leq n$  )

输出要点：输入第x筐中篮球数

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：A: CXK的篮球 ( 加强版 ) 由于本题数据较大，所以直接暴力会超时 ( 差分我在新生宣讲的时候讲过 )，可以采用“差分”的算法思想。加一个讲解的链接
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int SIZE = 1e6;
4 | int a[SIZE], c[SIZE * 10];
5 | long long ans = 0, ask, n, m, x, y, w;
6 | int main() {
7 |     cin >> n >> m; // 输入数列长度和修改次数
8 |     for (long long i = 1; i <= n; i++)
9 |         cin >> a[i]; // 输入数列
10 |    for (long long i = 1; i <= m; i++) {
11 |        cin >> x >> y >> w; // 从x到y全部加w
12 |        c[x] += w;           // x处标记+w
13 |        c[y + 1] -= w;       // y+1处标记-w
14 |    }
15 |    cin >> ask; // 输入查询下标
16 |    for (long long i = 1; i <= ask; i++)
17 |        ans += c[i];
18 |    cout << a[ask] + ans; // 不要忘记加原数列值
19 |    return 0;
20 | }
```

### 公开样例

样例 1 输入

```
3 2
1 2 3
1 3 1
1 2 1
2
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.69 | 总 196/526 Help Me ! ! ! ! ! ! ( PID 1520 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( C , CID 1059 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：1/5 ( 20.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，The output consists of two lines. The first line consists of the string "White: ", followed by the description of positions of the pieces of the white player. The second lin.....

输入要点：The input consists of an ASCII-art picture of a chessboard with chess pieces on positions described by the input. The pieces of the white player are shown in upper-case letters, while the black player's pieces are lower-case letters. The letters are one of "K" (King), "Q" (Queen), "R" (Rook), "B" (Bishop), "N" (Knight), or "P" (Pawn). The chessboard outline is made of plus ("+"), minus ("-"), and pipe ("|") character.....

输出要点：The output consists of two lines. The first line consists of the string "White: ", followed by the description of positions of the pieces of the white player. The second line consists of the string "Black: ", followed by the description of positions of the pieces of the black player.

The description of the position of.....

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：Help Me ! ! ! ! ! 一个基本的模拟题但是代码量巨大，需要认真看题。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | using namespace std;
4 | char t[50];
5 | char a[15][50];
6 | int main() {
7 |     for (int i = 1; i <= 8; ++i) {
8 |         scanf("%s", &t);
9 |         scanf("%s", (a[i] + 1));
10 |    }
11 |    scanf("%s", &t);
12 |    int bn = 0, wn = 0;
13 |    for (int i = 1; i <= 8; ++i)
14 |        for (int j = 3; j <= 32; j += 4) {
15 |            if ('a' <= a[i][j] && a[i][j] <= 'z')
16 |                bn++;
17 |            else if ('A' <= a[i][j] && a[i][j] <= 'Z')
18 |                wn++;
19 |        }
20 |    cout << "White: ";
21 |    int wcnt = 1;
22 |    for (int i = 8; i >= 1; --i)
23 |        for (int j = 3; j <= 32; j += 4) {
24 |            if (a[i][j] == 'K') {
25 |                char c = (char)(((j + 1) / 4) + 96);
26 |                cout << 'K' << c << (9 - i);
27 |                if (wcnt < wn)
28 |                    cout << ',';
29 |            }
30 |            else
31 |                cout << endl;
32 |            wcnt++;
33 |        }
34 |    }
35 |    for (int i = 8; i >= 1; --i)
36 |        for (int j = 3; j <= 32; j += 4) {
37 |            if (a[i][j] == 'Q') {
38 |                char c = (char)(((j + 1) / 4) + 96);
39 |                cout << 'Q' << c << (9 - i);
40 |                if (wcnt < wn)
41 |                    cout << ',';
42 |            }
43 |            else
44 |                cout << endl;
45 |            wcnt++;
46 |        }
47 |    }
48 |    for (int i = 8; i >= 1; --i)
49 |        for (int j = 3; j <= 32; j += 4) {
50 |            if (a[i][j] == 'R') {
51 |                char c = (char)(((j + 1) / 4) + 96);
52 |                cout << 'R' << c << (9 - i);
53 |                if (wcnt < wn)
54 |                    cout << ',';
55 |            }
56 |            else
57 |                cout << endl;
58 |            wcnt++;
59 |        }
60 |    }
61 |    for (int i = 8; i >= 1; --i)
62 |        for (int j = 3; j <= 32; j += 4) {
63 |            if (a[i][j] == 'B') {
64 |                char c = (char)(((j + 1) / 4) + 96);
65 |                cout << 'B' << c << (9 - i);
66 |                if (wcnt < wn)
67 |                    cout << ',';
68 |            }
69 |            else
70 |                cout << endl;
71 |            wcnt++;
72 |        }
73 |    }
74 |    for (int i = 8; i >= 1; --i)
75 |        for (int j = 3; j <= 32; j += 4) {
76 |            if (a[i][j] == 'N') {
77 |                char c = (char)(((j + 1) / 4) + 96);
78 |                cout << 'N' << c << (9 - i);
79 |                if (wcnt < wn)
80 |                    cout << ',';
81 |            }
82 |            else
83 |                cout << endl;
84 |        }
85 |    }
```

```

79 |         wcnt++;
80 |     }
81 | }
82 | for (int i = 8; i >= 1; --i)
83 |     for (int j = 3; j <= 32; j += 4) {
84 |         if (a[i][j] == 'p') {
85 |             char c = (char)(((j + 1) / 4) + 96);
86 |             cout << c << (9 - i);
87 |             if (wcnt < wn)
88 |                 cout << ',';
89 |             else
90 |                 cout << endl;
91 |             wcnt++;
92 |         }
93 |     }
94 | cout << "Black: ";
95 | int bcnt = 1;
96 | for (int i = 1; i <= 8; ++i)
97 |     for (int j = 3; j <= 32; j += 4) {
98 |         if (a[i][j] == 'k') {
99 |             cout << 'K';
100 |            char c = (char)(((j + 1) / 4) + 96);
101 |            cout << c << (9 - i);
102 |            if (bcnt < bn)
103 |                cout << ',';
104 |            else
105 |                cout << endl;
106 |            bcnt++;
107 |        }
108 |    }
109 | for (int i = 1; i <= 8; ++i)
110 |     for (int j = 3; j <= 32; j += 4) {
111 |         if (a[i][j] == 'q') {
112 |             cout << 'Q';
113 |            char c = (char)(((j + 1) / 4) + 96);
114 |            cout << c << (9 - i);
115 |            if (bcnt < bn)
116 |                cout << ',';
117 |            else
118 |                cout << endl;
119 |            bcnt++;
120 |        }
121 |    }
122 | for (int i = 1; i <= 8; ++i)
123 |     for (int j = 3; j <= 32; j += 4) {
124 |         if (a[i][j] == 'r') {
125 |             cout << 'R';
126 |            char c = (char)(((j + 1) / 4) + 96);
127 |            cout << c << (9 - i);
128 |            if (bcnt < bn)
129 |                cout << ',';
130 |            else
131 |                cout << endl;
132 |            bcnt++;
133 |        }
134 |    }
135 | for (int i = 1; i <= 8; ++i)
136 |     for (int j = 3; j <= 32; j += 4) {
137 |         if (a[i][j] == 'b') {
138 |             cout << 'B';
139 |            char c = (char)(((j + 1) / 4) + 96);
140 |            cout << c << (9 - i);
141 |            if (bcnt < bn)
142 |                cout << ',';
143 |            else
144 |                cout << endl;
145 |            bcnt++;
146 |        }
147 |    }
148 | for (int i = 1; i <= 8; ++i)
149 |     for (int j = 3; j <= 32; j += 4) {
150 |         if (a[i][j] == 'n') {
151 |             cout << 'N';
152 |            char c = (char)(((j + 1) / 4) + 96);
153 |            cout << c << (9 - i);
154 |            if (bcnt < bn)
155 |                cout << ',';
156 |            else
157 |                cout << endl;
158 |            bcnt++;
159 |        }
160 |    }
161 | for (int i = 1; i <= 8; ++i)
162 |     for (int j = 3; j <= 32; j += 4) {
163 |         if (a[i][j] == 'p') {
164 |             char c = (char)(((j + 1) / 4) + 96);
165 |             cout << c << (9 - i);

```

```

166 |         if (bcnt < bn)
167 |             cout << ',';
168 |         else
169 |             cout << endl;
170 |         bcnt++;
171 |     }
172 | }
173 | }

```

## 公开样例

### 样例 1 输入

```

+--+--+--+--+--+--+--+--+--+
|.r.|::|.b.|:q|.k.|::|.n.|:r.|
+--+--+--+--+--+--+--+--+--+
|p|.p.|:p|.p.|:p|.p.|::|.p.|
+--+--+--+--+--+--+--+--+--+
|...|:|.n.|::|.n.|:|.p.|
+--+--+--+--+--+--+--+--+--+
|::|...|:|.n.|:|.n.|:|.n.|
+--+--+--+--+--+--+--+--+--+
|...|:|.n.|:|.P.|:|.n.|:|.n.|
+--+--+--+--+--+--+--+--+--+
|:P.|:|.n.|:|.n.|:|.n.|:|.n.|
+--+--+--+--+--+--+--+--+--+
|.P.|:|.P.|:|.P.|:|.P.|:|.P.|
+--+--+--+--+--+--+--+--+--+
|:R.|:|.N.|:|.B.|:|.K.|:|.B.|:|.R.|
+--+--+--+--+--+--+--+--+--+

```

### 样例 1 输出

```

White: Ke1,Qd1,Ra1,Rh1,Bc1,Bf1,Nb1,a2,c2,d2,f2,g2,h2,a3,e4
Black: Ke8,Qd8,Ra8,Rh8,Bc8,Ng8,Nc6,a7,b7,c7,d7,e7,f7,h7,h6

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.70 | 总 197/526 ACM脱单大法 ( PID 1557 )

出处：2020级HAUT新生周赛（四）@张承树专场（C，CID 1063）| 训练定位：进阶 | 数组、字符串与双指针

标签：前缀后缀、数组与序列、枚举 | 限制：1.000 S / 128 MB | 历史统计：1/14 ( 7.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**可能会存在多个符合题目要求的下标 $x$ ，请输出最小的 $x$ 。（下标从1-n）

**输入要点：**第一行一个 $T$ ，代表有 $T$ 组样例

接下来 $T$ 组

每一组第一行一个 $n$ ，代表序列长度

第二行 $n$ 个数， $a[1],a[2],a[3].....a[n-1],a[n]$ 。

$a[i]$ 在int范围内

$T \leq 100$

$2 \leq n \leq 1e5$

输出要点：一个x，意义如题目所描述

可能存在多个答案，请输出最小的x ( $x < n$ )

题面提示：第一组样例中，符合条件的区间为下面两个

这样划分区间

[1,2][2,5] 前者区间的最大最小值=后者区间的最大最小值

[1,3][3,5] 前者区间的最大最小值=后者区间的最大最小值

2比3小，故输出2

第二组样例中我们找不到这个x值

所以输出-1

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
3. 核心处理采用：预处理 prefixMin/Max 与 suffixMin/Max。切点 x 合法，当且仅当  $[1, x]$  与  $[x+1, n]$  的最小值相同且最大值相同。按 x 从小到大检查，首个合法值就是题目要求的最短第一部分。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

每组  $O(n)$  时间、 $O(n)$  空间

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        vector<long long> a(n), preMin(n), preMax(n), sufMin(n), sufMax(n);
12 |        for (long long& x : a)
13 |            cin >> x;
14 |        preMin[0] = preMax[0] = a[0];
15 |        for (int i = 1; i < n; ++i) {
16 |            preMin[i] = min(preMin[i - 1], a[i]);
17 |            preMax[i] = max(preMax[i - 1], a[i]);
```

```

18 |     }
19 |     sufMin[n - 1] = sufMax[n - 1] = a[n - 1];
20 |     for (int i = n - 2; i >= 0; --i) {
21 |         sufMin[i] = min(sufMin[i + 1], a[i]);
22 |         sufMax[i] = max(sufMax[i + 1], a[i]);
23 |     }
24 |     int ans = -1;
25 |     for (int i = 0; i + 1 < n; ++i) {
26 |         if (preMin[i] == sufMin[i + 1] && preMax[i] == sufMax[i + 1]) {
27 |             ans = i + 1;
28 |             break;
29 |         }
30 |     }
31 |     cout << ans << '\n';
32 | }
33 | return 0;
34 | }

```

## 公开样例

样例 1 输入

```

2
5
1 5 3 1 5
6
1 2 3 4 5 6

```

样例 1 输出

```

2
-1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 3.71 | 总 198/526 牛牛的数独 ( PID 1595 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（H，CID 1070）| 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：6/22 ( 27.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：牛牛遇到了一个9×9的数独，你能帮牛牛判断数独是否有效。只需要根据以下规则，验证已经填入的数字是否有效即可。

输入要点：输入有9行，每行有9个区间为[0,9]的整数，其中用0表示空白，整数之间用空格隔开。

输出要点：如果数独有效输出yes，否则则输出no。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：牛牛的数独这题用数组模拟判断，判断行，列，九宫格时可以开一个数组进行标记。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。



## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int board[9][9];
4 |     for (int i = 0; i < 9; i++)
5 |         for (int j = 0; j < 9; j++)
6 |             scanf("%d", &board[i][j]);
7 |     // 检查行
8 |     for (int i = 0; i < 9; i++) {
9 |         int a[10] = {0};
10 |         for (int j = 0; j < 9; j++) {
11 |             if (board[i][j] == 0)
12 |                 continue;
13 |             if (a[board[i][j]] != 0) {
14 |                 printf("no");
15 |                 return 0;
16 |             }
17 |             a[board[i][j]] = 1;
18 |         }
19 |     }
20 |     // 检查列
21 |     for (int i = 0; i < 9; i++) {
22 |         int a[10] = {0};
23 |         for (int j = 0; j < 9; j++) {
24 |             if (board[j][i] == 0)
25 |                 continue;
26 |             if (a[board[j][i]] != 0) {
27 |                 printf("no");
28 |                 return 0;
29 |             }
30 |             a[board[j][i]] = 1;
31 |         }
32 |     }
33 |     // 检查九宫格
34 |     for (int i = 0; i < 3; i++)
35 |         for (int j = 0; j < 3; j++) {
36 |             int a[10] = {0};
37 |             for (int ii = i * 3; ii < i * 3 + 3; ii++)
38 |                 for (int jj = j * 3; jj < j * 3 + 3; jj++) {
39 |                     if (board[ii][jj] == 0)
40 |                         continue;
41 |                     if (a[board[ii][jj]] != 0) {
42 |                         printf("no");
43 |                         return 0;
44 |                     }
45 |                     a[board[ii][jj]] = 1;
46 |                 }
47 |         }
48 |     printf("yes");
49 | }
```

## 公开样例

### 样例 1 输入

```
530070000
600195000
098000060
800060003
400803001
700020006
060000280
000419005
000080079
```

样例 1 输出

```
yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.72 | 总 199/526 周赛榜单 ( PID 1596 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（F，CID 1071） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：13/137 ( 9.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：于是，好奇的某人想知道，选出的这m个人成绩之和最大是多少。

输入要点：一个整数 $n$  ( $n \leq 1e6$ )，代表参加新生赛的新生数。

接下来按榜单顺序输入n个新生的成绩。成绩不大于100。

一个整数 $t$  ( $t \leq 1e6$ )，代表某人询问的次数。

接下来t个整数。

输出要点：每次询问，给出前m个新生的成绩之和。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：周赛榜单 如果对每个 $t$ 都过一遍 $1 \sim n$ 的话，时间复杂度达到了 $1e12$ ，这样的话时间超限，所以这个题得要先处理一下。SO，这个只要把前缀和处理一下，这个题就变成了签到题。例：原数组: 前缀和 为数组的前项和 前缀和: 前缀和算法能够快速的求出某段区间的和；出题人说：前缀和更深度的学习看这里。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[1000010];
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     a[0] = 0;
7 |     for (int i = 1; i <= n; i++) {
8 |         int x;
9 |         scanf("%d", &x);
10 |         a[i] = a[i - 1] + x; // a[i]表示前i个人的分数的和
11 |     }
12 |     int t;
13 |     scanf("%d", &t);
14 |     while (t--) {
15 |         int x;
16 |         scanf("%d", &x);
17 |         printf("%d\n", a[x]);
18 |     }
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
5
10 7 5 4 3
2
1 3
```

样例 1 输出

```
10
22
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.73 | 总 200/526 三角图形 ( PID 1598 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（H，CID 1071）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：20/176 ( 11.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，能组成三角形，输出"Yes"；反之，输出"No"；

输入要点：六个数，代表阿H六根棍子的长度。(长度小于100)

输出要点：能组成三角形，输出"Yes"；

反之，输出"No"；

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：三角图形 简单模拟；六根棍子拼出两个三角形。法一：枚举，枚举每种可能拼成三角形的情况，判断即可；
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | int is_right(int a, int b, int c) {
4 |     if (c + b > a)
5 |         return 1;
6 |     return 0;
7 | }
8 | int main() {
9 |     int a[10];
10 |    for (int i = 0; i < 6; i++)
11 |        scanf("%d", &a[i]);
12 |    int flag = 0;
13 |    for (int i = 0; i < 6; i++)
14 |        for (int j = i + 1; j < 6; j++)
15 |            for (int k = j + 1; k < 6; k++) {
16 |                int b[5];
17 |                int c = 0;
18 |                for (int p = 0; p < 6; p++)
19 |                    if (p != i && p != j && p != k)
20 |                        b[c++] = a[p];
21 |                if (is_right(a[i], a[j], a[k]) == 1 && is_right(b[0], b[1], b[2]) == 1) {
22 |                    flag = 1;
23 |                    break;
24 |                }
25 |            }
26 |    if (flag == 0)
27 |        printf("No\n");
28 |    else
29 |        printf("Yes\n");
30 |    return 0;
31 | }
```

## 公开样例

样例 1 输入

1 1 1 1 1 1

样例 1 输出

Yes

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.74 | 总 201/526 小锋的书 ( PID 1700 )

出处：2021级HAUT新生周赛（四）@王一杰专场（A，CID 1072）| 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：57/415 ( 13.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：作为学霸的小锋从小就喜欢看书，每本书都要读上99999999遍，但是看得多了书就容易破损，为了让书能够保存完好，所以他买了好多本同一种书，并放在书架上。每一层书架上只放两种书"1"和"0"，他只读连续摆放的数量不小于7的书，如果没有，那么他就不能读书。现在给你一层书架的放书情况，请你判断他能不能读这层书架的书。

输入要点：一个由"1"和"0"组成的非空字符串，表示书架上的书，字符串长度不超过100个字符。

输出要点：如果可以读，输出YES,否则输出NO。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小锋的书问一个字符串中有没有连续的七个及以上的"1"或"0"，只需要定义两个变量记录连续最大的"1"和"0"的数量即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | char st[1000];
6 | int main() {
7 |     scanf("%s", st);
8 |     int zero = 0, one = 0, maz = 0, mao = 0;
9 |     // 这里有四个变量，zero和one记录遍历过程中，连续的1和0的数量
10 |    // maz和mao则记录连续的0串和1串最大长度
11 |    for (int i = 0; i < strlen(st); i++) { // 遍历字符串
12 |        if (st[i] == '1') {
13 |            one++; // 如果是1,"1"串长度++
14 |            if (one > mao)
15 |                mao = one; // 同时如果大于最大长度，更新mao
16 |            zero = 0; // 遇到1的话说明0串中断了，zero 归零
```

```

17 |         } else {
18 |             zero++; // 这一部分和上面同理
19 |             if (zero > maz)
20 |                 maz = zero;
21 |             one = 0;
22 |         }
23 |     }
24 |     if (mao >= 7 || maz >= 7)
25 |         printf("YES"); // 简单的输出
26 |     else
27 |         printf("NO");
28 |     return 0;
29 | }

```

## 公开样例

样例 1 输入

001001

样例 1 输出

NO

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.75 | 总 202/526 人生的抉择 ( PID 1705 )

出处：2021级HAUT新生周赛（四）@王一杰专场（F，CID 1072）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串 | 限制：1.000 S / 128 MB | 历史统计：4/134 ( 3.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果小锋要左转，输出LEFT;要一直往前走，输出TOWARDS;要右转，输出RIGHT。

输入要点：第一行，上一个阶段点A的坐标(x1,y1); 第二行，当前所在点B的坐标(x2,y2); 第三行，“非常卷”点C的坐标(x3,y3)。|x|,|y|<=1e9，此时小锋站在B,背对A。

输出要点：如果小锋要左转，输出LEFT;要一直往前走，输出TOWARDS;要右转，输出RIGHT。

## 零基础先修

- string 可按标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：人生的抉择题目看似复杂，其实原理很简单，用BA, BC向量的叉乘就可判断。BAxBC(叉乘)=|BA|\*|BC|\*sinθ，θ为两向量的夹角。（注意叉乘的顺序如果调换，判断原则也要变换）叉乘的计算为两向量交叉相乘再相减。夹角为90°时，向右转，叉乘的乘积大于0；夹角为180°时，直走，叉乘乘积为0；夹角为270°时，向左转，乘积小于0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | int main() {
6 |     long long xa, ya, xb, yb, xc, yc; // 三个点，记得用long long
7 |     scanf("%lld%lld%lld%lld%lld%lld", &xa, &ya, &xb, &yb, &xc, &yc);
8 |     long long ans = (xa - xb) * (yc - yb) - (xc - xb) * (ya - yb);
9 |     if (ans == 0)
10 |         printf("TOWARDS");
11 |     else if (ans < 0)
12 |         printf("LEFT");
13 |     else
14 |         printf("RIGHT");
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入

```
0 0
0 1
1 1
```

样例 1 输出

```
RIGHT
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.76 | 总 203/526 小锋卖瓜 ( PID 1706 )

出处：2021级HAUT新生周赛（四）@王一杰专场（G，CID 1072） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：0/7 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：小锋暑假社会实践时来到了一个瓜摊应聘，击败544个竞争者后他成功上岗。现在他需要解决一个难题，他的瓜摊上每天会有特定的顾客来买特定种类的瓜，瓜的种类命名为1、2、3、、、他要在 $n$ 天的实习期内满足每天的顾客的需求。老板给了他一种特殊的容器，可以无限储存同种类的瓜，但瓜的种类有一定数量限制。当容器满时（指瓜的种类储存达到上限），如果顾客需要的瓜不在容器中，.....

输入要点：输入有两行：

第1行两个整数 $n, k$ 表示一共有 $n$ 天实习期，容器的储存种类上限为 $k$  ( $1 \leq n, k \leq 80, 1 \leq n, k \leq 80$ )。

第2行包括n个整数，第i个数表示在第i天会有一个人来买瓜的种类 $a_i(1 \leq a_i \leq n)$ 。

输出要点：输出一个整数：

小锋最少要花的路费。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小锋卖瓜简单的贪心思想 三种情况：1.已经有了顾客需求种类的瓜，直接下一个；2.没有顾客需要的瓜，容器还没有满，直接去进货，路费+1；3.没有顾客需要的瓜且容器已满，这是需要扔掉某种瓜，这里就要用到贪心的思想，需要 扔掉最晚用到的一种瓜，才能使总的路费最少。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[100], have[100], vis[100][100], rem[100], num[100];
3 | int main() {
4 |     int n, k, sum = 0, total = 0;
5 |     scanf("%d%d", &n, &k);
6 |     for (int i = 1; i <= n; i++) {
7 |         scanf("%d", &a[i]);
8 |         vis[a[i]][rem[a[i]]] = i; // 保存每种瓜出现的天数
9 |         rem[a[i]]++;
10 |        // rem记录每种瓜已经出现几天，保证vis数组能按顺序存储a[i]种瓜都在哪一天出现
11 |    }
12 |    for (int i = 1; i <= n; i++) {
13 |        vis[i][rem[i]] = 0x3f3f3f3f; // 将每种瓜的最后一位置为最大值
14 |    }
15 |    for (int i = 1; i <= n; i++) {
16 |        num[a[i]]++; // 记录当前循环a[i]种瓜已经出现几次
17 |        if (have[a[i]])
18 |            continue; // have记录容器中已有的瓜，1为有
19 |        else if (total < k) { // 第二种情况，直接去进货
20 |            total++; // 容器已用空间+1
21 |            have[a[i]] = 1;
22 |        } else {
23 |            int ma = -1, flag;
24 |            for (int j = 1; j <= n; j++) {
25 |                if (have[j] && vis[j][num[j]] > ma) {
26 |                    // 如果容器中有这种瓜，并且这种瓜出现的下一个天数大于当前最大值
27 |                    // 因为上面已经将每种瓜的最后一位置为最大数，即后面已经不需要该种瓜，可以丢弃
28 |                    ma = vis[j][num[j]], flag = j; // 更新最大值，记录瓜的种类
29 |                }
30 |            }
31 |            have[a[i]] = 1, have[flag] = 0; // 丢弃瓜，买新瓜
32 |        }
33 |        sum++; // sum只在需要买瓜的时候+1
34 |    }
35 |    printf("%d", sum);
36 |    return 0;
37 | }
```



公开样例

样例 1 输入

```
4 80
1 2 2 1
```

样例 1 输出

```
2
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.77 | 总 204/526 对称括号序列 ( PID 1708 )

出处：2021级HAUT新生周赛 ( 四 ) @王一杰专场 ( H , CID 1072 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、构造、字符串 | 限制：1.000 S / 128 MB | 历史统计：1/24 ( 4.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一行一个正整数代表由n对括号可以构造出多少种不同的合法对称括号序列。

输入要点：一行一个正整数t，代表有t组数据；

随后t行每行一个正整数n，代表你有n对括号可以使用；

测试数据保证t组数据n的和<=1e6。

输出要点：一行一个正整数代表由n对括号可以构造出多少种不同的合法对称括号序列。

题面提示：按照题目描述的规则，由3对括号构成的合法的对称序列共有以下三组：

()()

((()))

((()))

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：对称括号序列的数量； $f[k]f[k]f[k]$ 根据规则2，由k-1对括号序列在外侧增加一个括号，形如为 ( P )，其数量为 个； $f[k-1]f[k-1]f[k-1]$ 根据规则3，由k-2对括号序列分别在左侧和右侧增加一个括号，形如 ( ) P ( )，其数量为 个； $f[k-2]f[k-2]f[k-2]$ 综上，得到递推方程： $f[i] = f[i-1] + f[i-2]f[i] = f[i-1] + f[i-2]f[i] = f[i-1] + f[i-2]$ 但是形如 (( ))(( )) 之类的括号序列虽然是对称的，但是不能由题目给出的规则构造得到，所以不计入答案数量中，最后注意取模即可。同时，这道题的结论也是斐波那契数列，直接手推前几项找规律也可以，前提是要读懂构造规则~~ ( 读不懂建议remake ) ~~。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | const int MOD = 1e9 + 7;
6 | int t, n;
7 | long long F[1000005];
8 | int main() {
9 |     scanf("%d", &t);
10 |    F[1] = 1, F[2] = 2;
11 |    for (int i = 3; i <= 1000004; i++)
12 |        F[i] = (F[i - 1] % MOD + F[i - 2] % MOD) % MOD;
13 |    while (t--) {
14 |        scanf("%d", &n);
15 |        printf("%lld\n", F[n]);
16 |    }
17 |    return 0;
18 | }
```

## 公开样例

样例 1 输入

```
3
1
2
3
```

样例 1 输出

```
1
2
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.78 | 总 205/526 爱因斯坦光电效应 ( PID 1709 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（A，CID 1073）| 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：9/153 ( 5.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：但本题则主要是以经典物理学光的波动力学说为基础，即电子获得光子能量成为光电子并打到阳极板上的过程不一定是瞬时的（也可能是瞬时的当且仅当所需光照时间无穷小的时候），也就是说可能需要一定的时间的累积才可发生光电效应(本质是光作为一种电磁波在周围产生的场的能量的一个转化)，也就说阴极板受到的光照的时间刚好和所需的时间相等的时候即可发生光电效应，每次输入给出5个时间段.....

输入要点：输入包含六行，前五行每行一个时间段，一天中的五个时间段不会跨到另一天,每个时间段有两个时刻，第一个时刻是开始时刻，第二个时刻是结束时刻，题目保证单个时间段的时间不会超过24个小时，最后一行给出所需的光照总时间,总时间的表示形式是秒数。

输出要点：在经典物理学的理论基础下判断是否恰可以发生光电效应，恰可以发生则输出"Yes"(没有引号),否则输出"No"(没有引号)。

题面提示：五个时间段加起来的总时间刚好是五个小时也就是 $5 \times 3600 = 18000$ 秒故可以发生光电效应。

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：爱因斯坦光电效应 思路：本题给出五个用字符串表示的时间段，需要分别对五个时间段进行处理然后算出五个时间段用秒表示的时间然后存储一下五个时间段的值，之后再循环模拟出所有情况的答案看是否有匹配的答案就可以了，如果有输出Yes，如果没有符合的就输出No。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int v[30], a, b, c, x, y, z;
3 | int main() {
4 |     for (int i = 1; i <= 5; i++) // 存储每个时间段用秒表示的总时间
5 |     {
6 |         scanf("%d:%d:%d %d:%d:%d", &a, &b, &c, &x, &y, &z);
7 |         int start, end;
8 |         end = x * 3600 + y * 60 + z;
9 |         start = a * 3600 + b * 60 + c;
10 |         v[i] = end - start;
11 |     }
12 |     int ans, f = 0;
13 |     scanf("%d", &ans);
14 |     for (int i = 0; i <= 1; i++) // 判断第一个时间段选不选
15 |     {
```

```

16 |         for (int j = 0; j <= 1; j++) // 判断第二个时间段选不选
17 |         {
18 |             for (int k = 0; k <= 1; k++) // 判断第三个时间段选不选
19 |             {
20 |                 for (int q = 0; q <= 1; q++) // 判断第四个时间段选不选
21 |                 {
22 |                     for (int w = 0; w <= 1; w++) // 判断第五个时间段选不选
23 |                     {
24 |                         int res = 0;
25 |                         if (i == 1)
26 |                             res += v[1];
27 |                         if (j == 1)
28 |                             res += v[2];
29 |                         if (k == 1)
30 |                             res += v[3];
31 |                         if (q == 1)
32 |                             res += v[4];
33 |                         if (w == 1)
34 |                             res += v[5];
35 |                         if (res == ans) // 有符合的方案
36 |                         {
37 |                             f = 1;
38 |                             printf("Yes\n");
39 |                             return 0;
40 |                         }
41 |                     }
42 |                 }
43 |             }
44 |         }
45 |     }
46 |     if (!f)
47 |         printf("No\n"); // 没有符合的方案
48 | }

```

## 公开样例

### 样例 1 输入

```

01:00:00 02:00:00
03:00:00 04:00:00
05:00:00 06:00:00
07:00:00 08:00:00
09:00:00 10:00:00
18000

```

### 样例 1 输出

```

Yes

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.79 | 总 206/526 lwjq与国际象棋 ( PID 1713 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（E，CID 1073）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/28 ( 10.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**经过21年的高考，lwjq上大学了，这是个十分让人快乐的日子，刚上大学的他结识了他的第一个伙伴lzl，二人十分的投缘并且lwjq发现lzl也喜欢下棋，因此为了测试lzl是否可以成为自己的下棋对手，lwjq给lzl出了这样的一道题目：给定一个8×8的棋盘，在刚开始的时候棋盘的64个方块都是白色的，这时候lwjq对这个棋盘做了些手脚，他把一些行变成了黑色……

**输入要点：**输入包含8行，每行包含8个字符。即lwjq做过手脚之后的棋盘。W字符表示白色的正方形，B字符表示涂成黑色的正方形。

输出要点：输出一个非负整数，即lwjq做手脚的最小步数。

题面提示：lwjq分别对第二行、第四列、第七列进行染色，故操作步数为3。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：lwjq与国际象棋 思路：先理解下题意，这道题同样是给了一个类似于棋盘的模型，但该棋盘是8\*8限定棋盘，每次操作是可以让一行全变为黑色，或者让一列全变为黑色，最终让统计所做的操作数。那么既然每次可以影响一行或者一列，那么我们只需判断第一行的元素和判断第一列的元素即可。具体分析：也就是我们分别判断第一行的每个元素，判断的实现是判断该元素所在的列是否全染色，同样分别判断第一列的每个元素的时候的实现也是同样的思路。但要注意的是要特判一下整个棋牌都被染成黑色的情况，不然上述的模拟会多加答案的数目。ok，分析完这些直接上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | char a[10][10];
3 | int main() {
4 |     int pd = 1;
5 |     for (int i = 1; i <= 8; i++) {
6 |         for (int j = 1; j <= 8; j++) {
7 |             scanf("%c", &a[i][j]);
8 |             if (a[i][j] != 'B') {
9 |                 pd = 0;
10 |            }
11 |        }
12 |    }
13 |    if (pd) // 判断所有字符全为B
14 |    {
15 |        printf("8");
16 |        return 0;
17 |    }
18 |    int ans = 0;
19 |    for (int j = 1; j <= 8; j++) {
20 |        if (a[1][j] == 'B') // 判断第一行字符如果有B则判断该B所在的列
21 |        {
22 |            int pd = 1;
23 |            for (int k = 1; k <= 8; k++) {
24 |                if (a[k][j] != 'B') // 有不符合的情况
25 |                {
26 |                    pd = 0;
27 |                    break;
28 |                }
29 |            }
30 |            if (pd)
31 |                ans++;
32 |        }
33 |    }
34 |    for (int i = 1; i <= 8; i++) {
35 |        if (a[i][1] == 'B') // 判断第一列字符如果有B则判断该B所在的行
```

```

36 |         {
37 |             int pd = 1;
38 |             for (int k = 1; k <= 8; k++) {
39 |                 if (a[i][k] != 'B') // 有不符合的情况
40 |                 {
41 |                     pd = 0;
42 |                     break;
43 |                 }
44 |             }
45 |             if (pd)
46 |                 ans++;
47 |         }
48 |     }
49 |     printf("%d\n", ans);
50 | }

```

## 公开样例

样例 1 输入

```

WWWBWWBW
BBBBBBBB
WWWBWWBW
WWWBWWBW
WWWBWWBW
WWWBWWBW
WWWBWWBW
WWWBWWBW

```

样例 1 输出

```

3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.80 | 总 207/526 爱你呦 ( PID 1719 )

出处：2021级HAUT新生周赛（六）@李志豪专场（C，CID 1074）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：6/55 ( 10.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：所以你的任务是，写一个检查接收一个含有目标子串，并将所有子串替换成相应的回复吧。

输入要点：每个测试包含多组测试样例。第一行输入一个T(1 <= T <= 100)。代表接下来又T组测试样例。

每个测试样例包含三个字符串s1,s2,s3，s1为发送的字符串，s2为需要替换的字符串，s3为替换之后的字符串，两个字符串之间用空格隔开。

0< |s1| , |s2|, |s3| <1e3 。

|s|表示字符串s的长度。

输出要点：输出T行，每行一个字符串代表替换之后的字符串。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：爱你呦 标签:字符串匹配 mid字符串暴力匹配替换就行了。标程：`//C`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认  $0/1$  起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | // C
2 | #include <string.h>
3 | #include <stdio.h>
4 | #include <math.h>
5 | int t, i, j, la, lb;
6 | char a[1005], b[1005], th[1005];
7 | int bj[1005];
8 | int main() {
9 |     scanf("%d", &t);
10 |    while (t--) {
11 |        scanf("%s%s", a, b, th);
12 |        la = strlen(a);
13 |        lb = strlen(b);
14 |        for (int i = 0; i < la; i++)
15 |            bj[i] = 0;
16 |        for (i = 0; i < la - lb + 1; i++) {
17 |            int pp = 1;
18 |            for (j = 0; j < lb; j++) {
19 |                if (a[i + j] != b[j]) {
20 |                    pp = 0;
21 |                    break;
22 |                }
23 |            }
24 |            if (pp == 1) {
25 |                bj[i] = 1;
26 |                i += lb - 1;
27 |            }
28 |        }
29 |        for (i = 0; i < la; i++) {
30 |            if (bj[i]) {
31 |                printf("%s", th);
32 |                i += lb - 1;
33 |            } else
34 |                printf("%c", a[i]);
35 |        }
36 |        printf("\n");
37 |    }
38 |    return 0;
39 | }
```

## 公开样例

样例 1 输入

```
2
a./ct ./c ainiyou
./dqwq ./d ./dqqq
```

样例 1 输出

```
aainiyout
./dqqqqwq
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.81 | 总 208/526 游戏 ( PID 1722 )

出处：2021级HAUT新生周赛（六）@李志豪专场（E，CID 1074）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 256 MB | 历史统计：0/10 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在你的问题是，看看是否能够达到门的位置取得游戏的胜利。

输入要点：第一行输入一个T代表有T组测试样例。(1<= T <= 100)

每个测试样例第一行输入n,atk,def,exp，n代表地图的大小，atk代表初始攻击力，def代表初始防御力，exp代表升级需要的经验值。(1<= n <= 1e6, 0 <= atk <= 500, 0 <= def <= 500, 1 <= exp <= 1000)

第二行输入n个用空格隔开的整数，代表有n个地图的情况。各个数字的代表情况如下：

0：代表是空格，即什么都没有。

1：代表是有怪。

2：代表是有能力药。

3：代表是有经验值药。

4：代表玩家初始位置。

5：代表终点的位置。

第三行输入k个用空格隔开的整数，为地图上从左到右的怪或者道具的值，如果是怪的话就是怪的攻击力，如果是道具的话，就是相应的提升值。 $k = \text{sum}(1) + \text{sum}(2) + \text{sum}(3)$  sum(i)表示数字为i的数的个数。

输出要点：如果能够逃出，输出YES。

如果不能够逃出，输出NO。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：游戏 标签：贪心 双指针 模拟 mid题目为了简化问题，减少了很多东西。读完之后分析一下发现，给的攻击力和防御力只有一个值有用，我们只需要取其中最大的那一个，在以后可以提升自己的能力的时候选择那一个就行了，并且又不会扣血，也没有让找最优



解，所以我们只需要把所有能走的都走一遍进行模拟看看能不能到达门那里就行了。标程如下

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define ll long long
4 | #define endl "\n"
5 | const int max_n = 1e6 + 100;
6 | const int inf = 0x3f3f3f3f;
7 | const int mod = 1e9 + 7;
8 | int arr[max_n], buf[max_n], vi[max_n];
9 | int n, a, d, e, ex;
10 | bool ans = false;
11 | bool cal(int x) {
12 |     if (arr[x] == 2) {
13 |         a += vi[x];
14 |         return true;
15 |     } else if (arr[x] == 3) {
16 |         ex += vi[x];
17 |         a += ex / e;
18 |         ex %= e;
19 |         return true;
20 |     } else if (arr[x] == 5) {
21 |         ans = true;
22 |         return true;
23 |     } else if (arr[x] == 1 && a > vi[x]) {
24 |         ex += vi[x] / 2;
25 |         a += ex / e;
26 |         ex %= e;
27 |         return true;
28 |     } else if (arr[x] == 0 || arr[x] == 4)
29 |         return true;
30 |     return false;
31 | }
32 | int main() {
33 |     ios::sync_with_stdio(false);
34 |     cin.tie(0);
35 |     cout.tie(0);
36 |     int T;
37 |     cin >> T;
38 |     while (T--) {
39 |         ans = false;
40 |         cin >> n >> a >> d >> e;
41 |         a = max(a, d);
42 |         int st = 0, ed = 0, id = 0;
43 |         for (int i = 0; i < n; i++) {
44 |             cin >> arr[i];
45 |             if (arr[i] == 1 || arr[i] == 2 || arr[i] == 3)
46 |                 buf[id++] = i;
47 |             if (arr[i] == 4)
48 |                 st = i;
49 |             if (arr[i] == 5)
50 |                 ed = i;
51 |         }
52 |         for (int i = 0; i < id; i++)
53 |             cin >> vi[buf[i]];
54 |         int l, r, ex;
55 |         l = r = st;
56 |         ex = 0;
57 |         bool isok = true;
58 |         while (isok) {
59 |             isok = false;
60 |             if (l >= 0) {
```

```

61 |         if (cal(l)) {
62 |             isok = true;
63 |             l--;
64 |         }
65 |     }
66 |     if (r < n) {
67 |         if (cal(r)) {
68 |             isok = true;
69 |             r++;
70 |         }
71 |     }
72 |     if (ans) {
73 |         cout << "YES" << endl;
74 |         break;
75 |     }
76 | }
77 | if (!ans)
78 |     cout << "NO" << endl;
79 | }
80 | return 0;
81 | }

```

## 公开样例

样例 1 输入

```

1
10 1 1 5
4 2 2 1 1 3 1 0 1 5
1 1 2 2 1 1 4 5

```

样例 1 输出

```

YES

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.82 | 总 209/526 我，真是笨蛋 ( PID 1741 )

出处：2021级HAUT新生周赛（八）@孔维斌专场（H，CID 1076）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：34/94 ( 36.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：使得这  $m$  个数或(位运算或，c语言可以直接使用|，没学过可自行百度位运算或)的值最大。请输出这个最大值。

输入要点：第一行一个整数  $n$

第二行  $n$  个整数  $a_1, a_2, a_3, \dots, a_{n-1}, a_n$ ，空格隔开

数据范围  $1 \leq n \leq 1e6, 1 \leq a_i \leq 1e8$ 。

输出要点：输出最大或的值。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：我，真是个笨蛋( 传送门)题目大意：选出m个数，将选出的数或起来，得到或的最大值，那么由于一个数或上一个数会使原先的更大，那么由于m没有限制，那么将所有的数全部或起来就是最大值。所以输出所有数或起来的最大值。时间复杂度： $O(n)O(n)O(n)$ 参考代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     int ans = 0;
6 |     for (int i = 0; i < n; i++) {
7 |         int x;
8 |         scanf("%d", &x);
9 |         ans |= x;
10 |    }
11 |    printf("%d\n", ans);
12 |    return 0;
13 | }
```

## 公开样例

样例 1 输入

```
5
1 2 3 4 5
```

样例 1 输出

```
7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.83 | 总 210/526 小C的又一个矩阵 ( PID 1742 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（J，CID 1077）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、字符串、构造 | 限制：1.000 S / 128 MB | 历史统计：5/17 ( 29.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，第一行输出合理的操作次数 $m$  ( $m \leq 12$ )；接下来 $m$ 行，输出每次操作改变的3个数字位置标号。（答案可能有多种，输出任意一种操作次数不大于12的合理答案即可，可以证明，答案一定存在。）

输入要点：输入一个两行两列的01矩阵

输出要点：第一行输出合理的操作次数 $m$  ( $m \leq 12$ )；

接下来 $m$ 行，输出每次操作改变的3个数字位置标号。

（答案可能有多种，输出任意一种操作次数不大于12的合理答案即可，可以证明，答案一定存在。）

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：小C的又一个矩阵这题属于简单思维题，需要想清楚矩阵中0、1需要怎么变换，才能很快的做出这题。这题这里提供两种方法：法一：这题我们需要做的，是将0转换成1，那么我们可以想想如何将一个0变成1，而其他几个数字不改变。要做到这一点，对于每个0，我们只需要3步就能实现了。这里直接上代码：上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, cnt;
3 | int a[5];
4 | int main() {
5 |     for (int i = 1; i <= 4; i++) {
6 |         scanf("%d", &a[i]);
7 |         if (a[i] == 0)
8 |             cnt++;
9 |     }
10 |    printf("%d\n", cnt * 3);
11 |    for (int i = 1; i <= 4; i++) {
12 |        if (a[i] == 0) {
13 |            if (i == 1)
14 |                printf("1 2 3\n1 3 4\n1 2 4\n");
15 |            else if (i == 2)
16 |                printf("1 2 3\n2 3 4\n2 1 4\n");
17 |            else if (i == 3)
18 |                printf("1 2 3\n2 3 4\n1 3 4\n");
19 |            else
20 |                printf("1 2 4\n1 3 4\n2 3 4\n");
21 |        }
22 |    }
23 |    return 0;
}
```

## 公开样例

样例 1 输入

```
0 1
1 1
```

样例 1 输出

```
3
1 2 3
1 2 4
1 3 4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.84 | 总 211/526 小C买泡芙 ( easy version ) ( PID 1746 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（D，CID 1077）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：8/57 ( 14.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在，小C来到了面包店，他希望以最少的价钱买齐每一种口味的泡芙，那么他至少得花多少钱呢

输入要点：第一行输入三个整数 $n, m, k$ ，表示一共有 $m$ 种口味（每种口味用数字1到 $m$ 表示）的 $n$ 个泡芙，每个泡芙售价为 $k$ 元

第二行输入 $n$ 个整数，表示这 $n$ 个泡芙，每个泡芙的口味

数据范围：  $1 \leq n \leq 1000, 1 \leq m \leq 100, 1 \leq k \leq 20$

输出要点：输出一个整数，即为小C买齐每种口味的泡芙至少需要多少钱

题面提示：对于样例，因为购买的泡芙必须连续，所以想买齐所有3种口味，至少得买4个泡芙，即【2 1 1 3】，所以至少得花20块钱

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小C买泡芙 ( easy version ) 作为easy version，这题的数据量特别小，因此这题只需模拟以每一个位置作为开始，买齐所有种类的泡芙需要买多少个即可，需要买的个数再乘价格即是要付的钱数 这里用 $v$ 数组记录的是当前哪些种类的泡芙已经买过，记得每次换到下一个位置开始模拟前，需要清空一次 $v$ 数组 上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int i, j, n, m, k, ans = 0x3f3f3f3f, a[1005], v[105];
3 | int main() {
4 |     scanf("%d%d%d", &n, &m, &k);
5 |     for (i = 1; i <= n; i++)
6 |         scanf("%d", &a[i]);
7 |     for (i = 1; i <= n; i++) {
8 |         int s = 0; // s记录当前已经买了多少种类，每次一旦买齐所有种类就可以退出循环了
9 |         for (j = 1; j <= m; j++) // 清空v数组
10 |             v[j] = 0;
11 |         for (j = i; j <= n; j++) {
12 |             if (v[a[j]] == 0) // 如果这个种类的此前没有出现，则记录一下
13 |             {
14 |                 v[a[j]] = 1;
15 |                 s++;
16 |             }
17 |             if (s == m) // 种类已经买齐
18 |             {
19 |                 int mon = (j - i + 1) * k; // 计算价钱
20 |                 if (mon < ans)
21 |                     ans = mon;
22 |                 break;
23 |             }
24 |         }
25 |     }
26 |     printf("%d", ans);
27 |     return 0;
28 | }
```

## 公开样例

样例 1 输入

```
5 3 5
2 2 1 1 3
```

样例 1 输出

```
20
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.85 | 总 212/526 小C买泡芙 ( hard version) ( PID 1747 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（E，CID 1077）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/16 ( 12.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在，小C来到了面包店，他希望以最少的价钱买齐每一种口味的泡芙，那么他至少得花多少钱呢

输入要点：第一行输入三个整数 $n, m, k$ ，表示一共有 $m$ 种口味（每种口味用数字1到 $m$ 表示）的 $n$ 个泡芙，每个泡芙售价为 $k$ 元

第二行输入 $n$ 个整数，表示这 $n$ 个泡芙，每个泡芙的口味

数据范围：  $1 \leq n \leq 2e5, 1 \leq m \leq 2e5, 1 \leq k \leq 2e5$

输出要点：输出一个整数，即为小C买齐每种口味的泡芙至少需要多少钱

题面提示：对于样例，因为购买的泡芙必须连续，所以想买齐所有3种口味，至少得买4个泡芙，即【2 1 1 3】，所以至少得花20块钱

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小C买泡芙（hard version）对于这题的数据量，前一题的时间复杂度的做法，在这就肯定会超时了。因此，这题需要换一个做法，用双指针来做。 $O(n^2)$  $O(n^2)$ 双指针，指的是用两个指针（左指针、右指针）表示出一个区间，每次都要尽可能保证区间是符合题目条件的，并用区间的答案去更新最优的答案。对于这题，双指针的做法即是：在每次更新答案之前，需要将右指针不断右移，直到区间内所有种类的泡芙都已存在，保证了所有种类的泡芙都已存在，这个区间才算是符合条件的，便可以用于更新最终答案了。当区间符合条件后，左指针便也开始向右移，每次只移动一位，如果区间仍符合条件，则可以继续用区间的结果更新最终的答案；如果这个区间泡芙种类已不够，则返回至前面移动右指针。以此反复，直到把所有情况都考虑到，也就算完成了。这道题也需要 $v$ 数组对当前区间内的每种泡芙的数量进行统计，当一种泡芙的数量从0变成1，则区间内的泡芙种类则+1；当一种泡芙的数量从1变成0，则区间内的泡芙种类则-1。知道了双指针怎么操作，也知道怎么统计区间内的泡芙种类数后，这题就可以算是能完成了。上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n^2)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #define ll long long
3 | int a[200005], v[200005];
4 | int main() {
5 |     int i, n, m, k, s = 0;
6 |     ll ans = 1e15;
7 |     scanf("%d%d%d", &n, &m, &k);
8 |     for (i = 1; i <= n; i++)
9 |         scanf("%d", &a[i]);
10 |    int l = 1, r = 0; // 左右两个指针
11 |    while (l < n) {
12 |        while (s < m && r + 1 <= n) // 当种类数不够的时候，右移右指针
13 |        {
14 |            v[a[++r]]++; // 记录当前种类的泡芙个数
15 |            if (v[a[r]] == 1)
16 |                s++; // 当前种类数量从0到1，区间种类数+1
17 |        }
18 |        if (s == m) // 判断区间是否符合条件，符合即可更新答案
19 |        {
20 |            ll mon = 1ll * (r - l + 1) * k;
21 |            if (mon < ans)
22 |                ans = mon;
23 |        }
24 |        v[a[l]]--;
```

```

25 |         if (v[a[l]] == 0)
26 |             s--;
27 |             l++;
28 |             // 将左指针右移一位，相对应的泡芙数量也要进行处理
29 |         }
30 |         printf("%lld", ans);
31 |         return 0;
32 |     }

```

## 公开样例

样例 1 输入

```

5 3 5
2 2 1 1 3

```

样例 1 输出

```

20

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.86 | 总 213/526 小C的cf上分历程 ( PID 1749 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（K，CID 1077）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：62/187 ( 33.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**在个人主页中，我们可以清楚的了解到小C一共参加了 $n$ 场比赛，还能了解到他每一场比赛赛后的rating分。假设小C通过自己的上分历程能得到的最大开心值是他这 $n$ 场比赛中，连续上分的最大场次数，请问小C能得到的最大开心值是多少？

**输入要点：**第一行输入一个整数 $n$  ( $1 \leq n \leq 2e5$ )，表示小C之前一共参加了 $n$ 场比赛

**第二行输入** $n$ 个整数，表示小C这 $n$ 场比赛，每场比赛赛后的rating分( $1 \leq a[i] \leq 2e5$ )

**输出要点：**输出一个整数，即小C通过自己的上分历程能得到的最大开心值

**题面提示：**在样例中，我们能找到两个较长的连续上分的序列【4 6 8 10 13】和【11 15 17 20】；因为前一次的序列连续上分的场次数是5，而第二次为4，所以小C能得到的最大快乐值应该是 $\max(5,4)$ ，即为5。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：K.小C的cf上分之路本题是道简单模拟，模拟分数变化过程，需要注意对于上升中断的时候，计数并非归0，而是置为1即可 上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。



- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int i, n, len, ans, a[20005];
3 | int main() {
4 |     scanf("%d", &n);
5 |     for (i = 1; i <= n; i++)
6 |         scanf("%d", &a[i]);
7 |     for (i = 1; i <= n; i++) {
8 |         if (a[i] > a[i - 1])
9 |             len++;
10 |        else {
11 |            if (len > ans)
12 |                ans = len;
13 |            len = 1;
14 |        }
15 |    }
16 |    printf("%d", ans);
17 |    return 0;
18 | }
```

## 公开样例

样例 1 输入

```
10
5 4 6 8 10 13 11 15 17 20
```

样例 1 输出

```
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.87 | 总 214/526 小C的回文矩阵 ( PID 1750 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（I，CID 1077）| 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：3/30 ( 10.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对于每组数据，如果矩阵为回文矩阵，则输出“Yes”，否则输出“No”，双引号无需输出

输入要点：第一行输入一个整数  $t$ ，代表有  $t$  组数据（ $1 \leq t \leq 100$ ）

对于每组数据，读入一个整数  $n$ ，和一个  $n \times n$  的矩阵（只包含英文字母），每组数据之间有空行（ $1 \leq n \leq 500$ ）

输出要点：对于每组数据，如果矩阵为回文矩阵，则输出“Yes”，否则输出“No”，双引号无需输出

题面提示：顺时针走回原点时，原点都会再次计入字符串

对于第一组数据：

顺时针走回原点形成的字符串为aaaaa，逆时针走回原点形成的字符串为aaaaa，两次形成的字符串相同，所以为Yes

对于第二组数据：

第一圈顺时针走回原点形成的字符串都为bbbbbbbbbbbb，因此第一圈满足回文条件，继续判断第二圈。

第二圈顺时针走回原点形成的字符串都为fgjif，逆时针走回原点形成的字符串为fijgf，两次形成的字符串不同，已经不能满足回文条件了，因此不需要再继续判断了，该组样例答案为No

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小C的回文矩阵这题题目已经把回文矩阵的判断方法已经表达的十分清楚了，直接模拟题意判断每一层的是否回文即可。不过，当我们把回文串的对应关系画图表示出来，这题就更简单了。不同的颜色，对应着不同层的判断，而在同一层中，我们可以发现，回文串中需要判断相等的两个字符，其实是已矩阵的对角线对称的！那么，我们只需要对每个字符判断 即可。  
`s[i][j]==s[j][i]s[i][j]==s[j][i]s[i][j]==s[j][i]s[i][j]==s[j][i]` 上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int t, n;
4 |     char a[505][505];
5 |     scanf("%d", &t);
6 |     while (t--) {
7 |         int i, j, judge = 1;
8 |         scanf("%d", &n);
9 |         for (i = 0; i < n; i++)
10 |             scanf("%s", a[i]);
11 |         for (i = 0; i < n; i++)
12 |             for (j = 0; j < n; j++) {
13 |                 if (a[i][j] != a[j][i]) // 如果一旦存在不相等，即可说明不是回文矩阵
14 |                 {
15 |                     judge = 0;
16 |                     break;
17 |                 }
18 |             }
19 |         if (judge)
20 |             printf("Yes\n");
```

```
21 |         else
22 |             printf("No\n");
23 |     }
24 | }
```

## 公开样例

样例 1 输入

```
2

2
aa
aa

4
bbbb
bfgb
bijb
bbbb
```

样例 1 输出

```
Yes
No
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.88 | 总 215/526 免费餐食 ^o^y ( PID 1759 )

出处：2022级HAUT新生周赛（一）@卡子骏专场（F，CID 1079）| 训练定位：进阶 | 数组、字符串与双指针

标签：连续段、数组扫描、最值 | 限制：1.000 S / 128 MB | 历史统计：45/300 ( 15.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个整数  $x$  代表连续次数最多的打分，一个整数  $y$  代表连续次数，以空格分隔 如果没有连续的打分(即没有两个数字连续出现两次及以上)，输出 "0 0"

输入要点：输入共两行

第一行输入一个整数  $n$  ( $2 \leq n \leq 1000$ ) 代表一共发了  $n$  次饭

第二行输入  $n$  个整数  $a_1 - a_n$ ,  $a_i$  ( $1 \leq a_i \leq 9$ ) 代表第  $i$  次饭的质量

输出要点：一行，一个整数  $x$  代表连续次数最多的打分，一个整数  $y$  代表连续次数，以空格分隔

如果没有连续的打分(即没有两个数字连续出现两次及以上)，输出 "0 0"

## 零基础先修

- 这一模块训练线性扫描、下标边界、字符处理、连续段和左右指针。它们频繁出现在周赛前中段，也是后续数据结构与动态规划的语言基础。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

- 核心处理采用：扫描每个相同数字的连续段，记录当前段的数字和长度。段长度至少为 2 才参与比较；先比较长度，长度相同再选评分较大的段。初始化答案为 0 0，便自然覆盖“没有连续重复”的情况。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, bestValue = 0, bestLength = 0;
7 |     cin >> n;
8 |     int pre = -1, len = 0;
9 |     for (int i = 0; i < n; ++i) {
10 |         int value;
11 |         cin >> value;
12 |         if (i > 0 && value == pre)
13 |             ++len;
14 |         else
15 |             len = 1;
16 |         if (len >= 2 && (len > bestLength || (len == bestLength && value > bestValue))) {
17 |             bestLength = len;
18 |             bestValue = value;
19 |         }
20 |         pre = value;
21 |     }
22 |     cout << bestValue << ' ' << bestLength << '\n';
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
15
999523447621586
```

样例 1 输出

```
93
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.89 | 总 216/526 lycoris ( PID 1761 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（G，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（H，CID 1079）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：60/173 ( 34.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，三个整数，以空格分隔，代表三位密码

输入要点：一行，一个字符串，仅由如题三种括号组成

输出要点：一行，三个整数，以空格分隔，代表三位密码

题面提示：题目保证每个数字密码都在 0 - 9999 范围内

注意密码序号

注意：括号的配对符合常用语法

o(\*\* ^ \_ ^ \*\*)ブ sakana~~~

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：lycoris 括号配对符合基本语法，即对于相同括号，前括号在前，后括号在后就算匹配成功（同种括号）记录非匹配括号数量和匹配括号数量，出现前括号非匹配次数++，出现后括号非匹配次数--并且匹配次数++，如果非匹配数量为0，则出现后括号不做变动
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     char str[10010];
4 |     scanf("%s", str);
5 |     int s = 0, m = 0, l = 0;
6 |     int ss = 0, mm = 0, ll = 0;
7 |     for (int i = 0; str[i] != '\0'; i++) {
8 |         if (str[i] == '(')
9 |             s++;
10 |        else if (str[i] == '[')
11 |            m++;
12 |        else if (str[i] == '{')
13 |            l++;
14 |        else if (str[i] == ')') {
```

```

15 |         if (s != 0) {
16 |             s--;
17 |             ss++;
18 |         }
19 |     } else if (str[i] == ']') {
20 |         if (m != 0) {
21 |             m--;
22 |             mm++;
23 |         }
24 |     } else if (str[i] == '}') {
25 |         if (l != 0) {
26 |             l--;
27 |             ll++;
28 |         }
29 |     }
30 | }
31 | printf("%d %d %d", mm, ss, ll);
32 | return 0;
33 | }

```

## 公开样例

样例 1 输入

```
[(D)]{
```

样例 1 输出

```
2 1 0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.90 | 总 217/526 我们之间没有你插手的余地哦？ ( PID 1764 )

出处：2022级HAUT新生周赛（二）@武其轩专场（C，CID 1080）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、模拟、整行输入 | 限制：1.000 S / 128 MB | 历史统计：36/192 ( 18.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**给出一个长度为 $n$ 的字符串，该字符串只有英文字母，数字字符以及空格组成，其中，字符'1'表示为一个人，空格表示为“空隙”。现在你要写出一个程序，判断是否有两个人之间存在且只存在空隙。

**输入要点：**第一行输入一个整数 $T(1 \leq T \leq 1145)$ ，表示有 $T$ 组数据。

每组数据的第一行输入一个整数 $n(2 \leq n \leq 100)$ ，第二行输入一个长度为 $n$ 的字符串（字符串至少包含一个非空格字符）。

**输出要点：**对于每组数据，如果存在两个人之间只存在空隙则输出“Y”，否则输出“N”（均不包含双引号）。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。

- 核心处理采用：一对合法的“人”必须是两个字符 1，且它们中间至少有一个字符，中间每个字符都必须是空格。枚举左端的 1，再向右越过连续空格；若越过至少一个空格后正好遇到另一个 1，就找到了答案。读取整行前要先处理上一行留下的换行。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  额外空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        cin.ignore(numeric_limits<streamsize>::max(), '\n');
12 |        string s;
13 |        getline(cin, s);
14 |        bool found = false;
15 |        for (int i = 0; i < n && !found; ++i) {
16 |            if (s[i] != '1')
17 |                continue;
18 |            int j = i + 1;
19 |            while (j < n && s[j] == ' ')
20 |                ++j;
21 |            if (j > i + 1 && j < n && s[j] == '1')
22 |                found = true;
23 |        }
24 |        cout << (found ? "Y" : "N") << '\n';
25 |    }
26 |    return 0;
27 | }
```

## 公开样例

样例 1 输入

```
2
5
1 101
3
101
```

样例 1 输出

```
Y
N
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 3.91 | 总 218/526 我愿意为你跨越世界的尽头 ( PID 1774 )

出处：2022级HAUT新生周赛（三）@焦小航专场（E，CID 1081）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、计数、标记数组 | 限制：1.000 S / 128 MB | 历史统计：75/375 ( 20.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出总统计次数

输入要点：第一行给出一个  $t$ ，代表有  $t$  组询问。（ $1 \leq t \leq 100$ ）

接下来  $t$  组，每组第一行给出一个  $|S|$ ，第二个给出 字符串  $S$ 。（ $1 \leq |S| \leq 50$ ）， $|S|$  代表字符串  $S$  的长度。

输出要点：输出总统计次数

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每组先读题目给出的字符串长度，再读字符串。seen[26] 标记大写 字母是否出现过：第一次贡献 2，以后贡献 1；非大写 字符不计入。长度字段本身不能误当成字符串处理。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(\sum |S|)$  时间、 $O(1)$  额外空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int len;
10 |         string text;
11 |         cin >> len >> text;
12 |         array<bool, 26> seen{};
13 |         int ans = 0;
14 |         for (char ch : text) {
15 |             if (ch < 'A' || ch > 'Z')
16 |                 continue;
17 |             int id = ch - 'A';
18 |             ans += seen[id] ? 1 : 2;
19 |             seen[id] = true;
20 |         }
21 |         cout << ans << '\n';
22 |     }
23 |     return 0;
24 | }
```

### 公开样例



样例 1 输入

```
1
4
ABDD
```

样例 1 输出

```
7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 3.92 | 总 219/526 一半一半 ( PID 1793 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（H，CID 1083）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：11/23 ( 47.8% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：请计算这些时空裂缝将时空分成了几个部分。

输入要点：第一行一个整数N

接下来N行每行两个整数,第i行分别是Ai和Bi

输出要点：一个整数代表答案。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：一半一半 题意：求出所给直线把整个平面分割成的块数 解法
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | #define eps 1e-5
4 | struct line {
5 |     int a, b;
6 | } lines[505];
7 | typedef struct point {
8 |     double x, y;
9 | } point;
10 | // 用来标记这条直线是否和前面的直线重合
11 | int st[505];
12 | int main() {
13 |     long long ans = 0;
14 |     int n;
15 |     // 读入
16 |     scanf("%d", &n);
17 |     for (int i = 0; i < n; i++) {
18 |         // 读入直线
19 |         scanf("%d%d", &lines[i].a, &lines[i].b);
20 |         // 创建一个数组用来存放这条直线与前面几条直线的交点
21 |         point points[505] = {0};
22 |         int cnt = 0;
23 |         for (int j = 0; j < i; j++) {
24 |             // 如果这条直线与其他直线重复 (重合) 就跳过
25 |             if (st[j])
26 |                 continue;
27 |             // 与第j条线平行
28 |             if (lines[i].a == lines[j].a) {
29 |                 // 与第j条线重合
30 |                 if (lines[i].b == lines[j].b) {
31 |                     // 标记为重复, 没有增加任何分块, 直接break
32 |                     st[i] = 1;
33 |                     break;
34 |                 } else {
35 |                     // 仅平行, 没有焦点
36 |                     continue;
37 |                 }
38 |             }
39 |             point p;
40 |             // 计算出直线i和直线j的交点坐标
41 |             p.x = (lines[j].b - lines[i].b) * 1.0 / (lines[i].a - lines[j].a);
42 |             p.y = (lines[i].a * p.x + lines[i].b);
43 |             int flag = 1;
44 |             for (int k = 0; k < cnt; k++) {
45 |                 // 如果与之前计算的交点重复则不计入 (这里有误差判断)
46 |                 if (fabs(points[k].x - p.x) < eps && fabs(points[k].y - p.y) < eps) {
47 |                     flag = 0;
48 |                     break;
49 |                 }
50 |             }
51 |             if (flag) {
52 |                 points[cnt++] = p;
53 |             }
54 |         }
55 |         // 与前面的所有直线有cnt个交点, 即增加了cnt+1个分块
56 |         if (!st[i])
57 |             ans += cnt + 1;
58 |     }
59 |     // 输出ans+1(因为初始时相当于一整块区域)
60 |     printf("%lld", ans + 1);
61 |     return 0;
62 | }
```

## 公开样例

### 样例 1 输入

```
3
1 1
2 2
3 3
```

### 样例 1 输出

```
6
```

## 训练动作

- 先遮住代码, 用 3~5 句话复述状态、步骤和复杂度, 再独立实现。
- 自行补至少三个边界: 最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”, 一周后限时重做; 若 20 分钟仍无方向, 只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

## 3.93 | 总 220/526 数字统计 ( PID 1796 )

出处：2022级HAUT新生周赛（六）@张鑫专场（C，CID 1084）| 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：23/113 ( 20.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，请输出一行，每行有10个正整数，分别为0~9的出现次数

输入要点：输入六个正整数y、m、d、h、t、s，分别表示年月日和时分秒。

输出要点：请输出一行，每行有10个正整数，分别为0~9的出现次数

题面提示：1<=y<=9999，1<=m<=12，0<=h<=23

假如现在是今年的第1234秒，那么1、2、3、4各出现一次其他数字出现零次。

（小心边界情况，注意判断闰年）

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：数字统计 只需要注意当秒数为0时0的个数为1，我特意提示判断边界但是很多同学还是掉坑里了。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int num[20];
3 | int time(int y, int m, int d, int h, int t, int s) // 计算并返回秒数
4 | {
5 |     int a[20] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
6 |     int ans = d - 1;
7 |     if ((y % 4 == 0 && y % 100 != 0) || (y % 400 == 0))
8 |         a[2] = 29; // 判断是否是闰年，二月要变化
9 |     for (int i = 1; i < m; i++)
10 |         ans += a[i]; // 总天数
```

```

11 |     ans = ans * 3600 * 24;
12 |     ans += (h * 3600 + t * 60 + s);
13 |     return ans;
14 | }
15 | int main() {
16 |     int y, m, d, h, t, s, sum;
17 |     scanf("%d %d %d %d %d %d", &y, &m, &d, &h, &t, &s);
18 |     sum = time(y, m, d, h, t, s);
19 |     if (sum == 0)
20 |         num[0] = 1;
21 |     while (sum) {
22 |         num[sum % 10]++;
23 |         sum /= 10;
24 |     }
25 |     for (int i = 0; i <= 9; i++)
26 |         printf("%d ", num[i]);
27 |     return 0;
28 | }

```

## 公开样例

样例 1 输入

2022 10 18 18 22 13

样例 1 输出

0 2 3 2 0 1 0 0 0 0

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.94 | 总 221/526 握手 ( PID 1801 )

出处：2022级HAUT新生周赛（六）@张鑫专场（H，CID 1084）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：6/128 ( 4.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个正整数。

输入要点：第一行输入两个正整数分别是 $t, n$ ，第二行给你 $n$ 个正整数为每个人的友谊值 $a_i$ 。(输入的友谊值保证是严格非递减序列)

输出要点：输出一个正整数。

题面提示： $2 \leq n \leq 1e5$ ， $1 \leq a_i \leq 1e6$ ， $t \leq 1e9$

结果需要开longlong，不能自己和自己组合且 $(a_i, a_j)$ 和 $(a_j, a_i)$ 为同一对。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：握手（二分）对每一个数 $a[i]$ 二分查找对应的 $t-a[i]$
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | typedef long long int LL;
3 | int a[100005];
4 | int num[1000005] = {0}; // num记录每个数字的出现次数
5 | int func(int l, int r, int x) // 二分答案，返回匹配的数量
6 | {
7 |     int mid;
8 |     while (l <= r) {
9 |         mid = l + r >> 1; //(l+r)/2
10 |         if (a[mid] == x)
11 |             return num[x]; // 如果找到直接返回
12 |         else if (a[mid] > x)
13 |             r = mid - 1; // 太大缩小右边界
14 |         else
15 |             l = mid + 1; // 太小缩小左边界
16 |     }
17 |     return 0;
18 | }
19 | int main() {
20 |     int t, n;
21 |     scanf("%d %d", &t, &n);
22 |     for (int i = 1; i <= n; i++)
23 |         scanf("%d", &a[i]), num[a[i]]++;
24 |     LL sum = 0; // 会爆int
25 |     for (int i = 1; i <= n; i++) {
26 |         num[a[i]]--; // 每次剔除数字防止重复
27 |         int f = func(i + 1, n, t - a[i]); // 检查x=t-a[i]的数量
28 |         sum += f;
29 |     }
30 |     printf("%lld", sum);
31 |     return 0;
32 | }
```

## 公开样例

样例 1 输入

```
2 4
1 1 1 1
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

### 3.95 | 总 222/526 我的矩形 ( PID 1815 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（I，CID 1085） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：2.500 S / 128 MB | 历史统计：47/154 ( 30.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，x行y列的整数矩阵，矩阵中的每个数字都需要占五位，即可能需要前导0。

输入要点：第一行三个正整数依次为n、x、y。

第二行n个正整数

(  $n \leq 1e6$  ,  $x * y \leq n$  ,  $a[i] < 1e5$  )

输出要点：x行y列的整数矩阵，矩阵中的每个数字都需要占五位，即可能需要前导0。

#### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

#### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：我的矩形 在读取数字的时候同时输出，输出数量为y的倍数时输出回车即可。代码：c c++
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

#### 复杂度

$O(1)$

#### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

#### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n, x, y;
4 |     scanf("%d%d%d", &n, &x, &y);
5 |     for (int i = 1; i <= n; i++) {
6 |         int x;
7 |         scanf("%d", &x);
8 |         printf("%05d ", x);
9 |         if (i % y == 0)
10 |             printf("\n");
11 |     }
12 |     return 0;
13 | }
```

#### 公开样例

样例 1 输入

```
6 2 3
1 2 3 4 5 99999
```

样例 1 输出

```
00001 00002 00003
00004 00005 99999
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.96 | 总 223/526 睡过去的jxh仍在学习 ( PID 1820 )

出处：2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（D，CID 1086）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：23/298 ( 7.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个正整数，表示jxh聚聚至少需要多久才能回到图书馆。

输入要点：第一行两个正整数n和m，表示答题空间内的题目数量和jxh聚聚回到图书馆前必须解决的问题数量（ $1 \leq n \leq 1e5, 1 \leq m \leq n$ ）。

第二行n个正整数， $a_i$ 表示jxh聚聚解决第i道题需要的时间（ $1 \leq a_i \leq 1e5$ ）。

请注意数据范围！！！！

输出要点：一行，一个正整数，表示jxh聚聚至少需要多久才能回到图书馆。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：睡过去的jxh仍在学习 对数组a进行排序，然后把前m个小的数相加即可，注意题目的数据范围，不能用 $n^2$ 的排序方法，可以用快排、归并之类的，使用c++的话，可以直接用sort函数。如果不会这些话，注意到数组a中的元素都是 $1e5$ 以内的正整数，可以开一个数组b来记录数组a中元素出现的个数，即 $b[x]$ 代表x在a中出现的次数，x为0时，没有出现。这样就可以从1开始遍历b，找到a中前m个小的数，把它们相加即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[100005];
3 | int main() {
4 |     int n, m, cnt = 0;
5 |     scanf("%d%d", &n, &m);
6 |     for (int i = 0; i < n; i++) {
7 |         int x;
8 |         scanf("%d", &x);
9 |         a[x]++;
10 |    }
11 |    long long ans = 0;
12 |    for (int i = 1; i <= 100000; i++) {
13 |        long long x = a[i];
14 |        if (x > m - cnt)
15 |            x = m - cnt;
16 |        cnt += x;
17 |        ans += i * x;
18 |        if (cnt == m)
19 |            break;
20 |    }
21 |    printf("%lld", ans);
22 |    return 0;
23 | }
```

## 公开样例

样例 1 输入

```
5 3
6 9 5 100 3
```

样例 1 输出

```
14
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.97 | 总 224/526 排队 ( PID 1823 )

出处：2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（G，CID 1086）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：15/86 ( 17.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入， $n$ 个正整数，用空格分隔。

输入要点：第一行：一个正整数 $n$ ，表示队伍人数。

由于第一个人前面没有人，所以接下来 $n-1$ 行，每行给出两个正整数 $x$ 和 $y$ ， $x$ 表示当前学生的编号， $y$ 表示编号为 $x$ 的学生前面的学生的编号。

( $1 \leq n \leq 5000$  ,  $1 \leq x, y \leq n$ )

输出要点： $n$ 个正整数，用空格分隔。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。



## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：排队 单向链表的简单模拟，反向记录一下每个人后面的人即可，可以标记一下前面有人的人的编号没有标记的就是队首，另外有n可能为1赛时很多人wa这个点了。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[5005], st[5005];
3 | int main() {
4 |     int n, h;
5 |     scanf("%d", &n);
6 |     for (int i = 1; i < n; i++) {
7 |         int x, y;
8 |         scanf("%d%d", &x, &y);
9 |         a[y] = x;
10 |        st[x] = 1;
11 |    }
12 |    for (int i = 1; i <= n; i++)
13 |        if (!st[i]) {
14 |            h = i;
15 |            break;
16 |        }
17 |    while (h) {
18 |        printf("%d ", h);
19 |        h = a[h];
20 |    }
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
4
4 3
2 1
3 2
```

样例 1 输出

```
1 2 3 4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.98 | 总 225/526 小明的内卷日 ( PID 1827 )

出处：2023级HAUT新生周赛（二）@曹瑞峰专场（C，CID 1087）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：23/347 ( 6.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个正整数，表示小明最多可以解决多少道题。

输入要点：第一行两个整数 $n$ 和 $m$ ，表示练习册上题目数量和小明学习的时间。 $(1 \leq n \leq 1e5, 0 \leq m \leq 1e10)$

第二行 $n$ 个正整数， $a[i]$ 表示解决第 $i$ 道题需要花费的时间。 $(1 \leq a[i] \leq 1e5)$

输出要点：一行，一个正整数，表示小明最多可以解决多少道题。

题面提示：注意数据范围

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：小明的内卷日用数组存入写题花费的时间，并用`qsort`从小到大排序，然后每次将 $m$ 减去需要花费最少时间的题目，直到 $m$ 减少到小于零时退出，便可得到最优解。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int cmp(const void* a, const void* b) {
4 |     return *(int*)a > *(int*)b;
5 | }
6 | int a[100010];
7 | int main() {
8 |     int n;
9 |     long long m;
10 |    scanf("%d %lld", &n, &m);
11 |    for (int i = 0; i < n; i++) {
12 |        scanf("%d", &a[i]);
13 |    }
14 |    qsort(a, n, sizeof(int), cmp);
15 |    int ans = 0;
16 |    for (int i = 0; i < n; i++) {
17 |        m -= a[i];
18 |        if (m >= 0)
19 |            ans++;
20 |        else
21 |            break;
22 |    }
```

```
23 |     printf("%d", ans);
24 | }
```

## 公开样例

样例 1 输入

```
5 10
4 6 5 2 3
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.99 | 总 226/526 小明的数字 ( PID 1828 )

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（D，CID 1091）；2023级HAUT新生周赛（二）@曹瑞峰专场（D，CID 1087）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：88/700（12.6%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：小明有一个长度为 $n(1 \leq n \leq 100)$ 的正整数 $a$ 和一个正整数 $b(0 \leq b \leq 9)$ ，现在小明需要将 $b$ 插入到 $a$ 中，使得变化后的 $a$ 最大。请你输出变化后的 $a$ 。

输入要点：第一行：正整数 $n$ ，表示 $a$ 的长度( $1 \leq n \leq 100$ )。

第二行：正整数 $a$ 和 $b(0 \leq b \leq 9)$ 。

输出要点：一行，直接输出变化后的 $a$ 。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：小明的数字 先用 $\text{char}$ 数组读入字符串 $s$ ，再遍历判断 $s$ 中的每一个字符，从第一个字符开始，当第 $i$ 个字符比 $m$ 小时，将 $m$ 插入到第 $i$ 个字符前面，就可以保证最后得到的值是最大值。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

易错点

- 确认 0/1 起下标、首尾元素和 n=1。

C++17 参考实现

```
1 | #include <stdio.h>
2 | char s[105];
3 | int main() {
4 |     int n;
5 |     char m;
6 |     scanf("%d", &n);
7 |     scanf("%s %c", s, &m);
8 |     int f = 0;
9 |     for (int i = 0; i < n; i++) {
10 |         if (m > s[i] && f == 0) {
11 |             printf("%c", m);
12 |             f = 1;
13 |         }
14 |         printf("%c", s[i]);
15 |     }
16 |     if (f == 0)
17 |         printf("%c", m);
18 |     return 0;
19 | }
```

公开样例

样例 1 输入

5  
12345 5

样例 1 输出

512345

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

3.100 | 总 227/526 小明的幸运数 ( PID 1832 )

出处：2023级HAUT新生周赛（二）@曹瑞峰专场（H，CID 1087） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：2/39 ( 5.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个正整数，表示幸运序列的第k位是什么。

输入要点：一个数字k，表示需要寻找的位置。(1<=k<=1e14)

输出要点：一个正整数，表示幸运序列的第k位是什么。

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：小明的幸运数该题本质上是十进制数字转换为九进制数字。将输入进的十进制数字逐位计算得到九进制数字的每一位，最后再判断里面是否有4。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | void solve() {
4 |     long long s;
5 |     cin >> s;
6 |     string ans = "";
7 |     while (s) {
8 |         char op = s % 9 + '0';
9 |         if (op >= '4')
10 |             op++;
11 |         ans += op;
12 |         s /= 9;
13 |     }
14 |     reverse(ans.begin(), ans.end());
15 |     cout << ans;
16 | }
17 | int main() {
18 |     cin.tie(0);
19 |     cout.tie(0);
20 |     int t = 1;
21 |     // cin>>t;
22 |     while (t--) {
23 |         solve();
24 |     }
25 |     return 0;
26 | }
```

## 公开样例

样例 1 输入

14

样例 1 输出

16

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.101 | 总 228/526 纵向排版 ( PID 1844 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（H，CID 1088）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、二维下标、格式化输出 | 限制：1.000 S / 128 MB | 历史统计：20/125 ( 16.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，按纵向排版格式排版给定的字符串，每列N个字符（除了最后一列可能不足N个）。

**输入要点：**输入在第一行给出一个正整数N（ $<100$ ），是每一列的字符数。第二行给出一个长度不超过1000的非空字符串，以回车结束。

**输出要点：**按纵向排版格式排版给定的字符串，每列N个字符（除了最后一列可能不足N个）。

**题面提示：**注意每列都要有N个字符，不足的部分要用空格补充。

不要使用getchar()吸收换行符，否则将会出现错误。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：原串按“每列 N 个字符”切成若干列，输出时外层枚举行号 0..N-1，内层枚举列号。原串下标为  $column * N + row$ ；越界说明最后一列不足，应输出空格占位。第一行数字后的换行用 ignore 清除，再 getline。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(N \cdot \text{列数})$  时间、 $O(|s|)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     cin.ignore(numeric_limits<streamsize>::max(), '\n');
9 |     string s;
10 |    getline(cin, s);
11 |    int columns = (s.size() + n - 1) / n;
12 |    for (int row = 0; row < n; ++row) {
13 |        for (int column = 0; column < columns; ++column) {
14 |            int index = column * n + row;
15 |            cout << (index < (int)s.size() ? s[index] : ' ');
16 |        }
17 |        cout << '\n';
18 |    }
19 |    return 0;
20 | }
```

公开样例

样例 1 输入

```
4
This is a test case
```

样例 1 输出

```
T asa
hi ts
ist e
s ec
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

3.102 | 总 229/526 火柴的奇怪用途 ( PID 1851 )

出处：2023级HAUT新生周赛（四）@牛浩然专场（G，CID 1089） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：4/48 ( 8.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个整数，能拼成的不同等式的数目。

输入要点：一个整数，表示还剩下n根火柴 ( 1<=n<=24)。

输出要点：一个整数，能拼成的不同等式的数目。

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20231119/20231119144748\\_37293.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20231119/20231119144748_37293.png)]

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：火柴的奇怪用途. 由于n很小，考虑暴力，x，y，那么当n=24的时候，x，y最大可能是多少呢？由于1花费的火柴数最少，那么让x,y均为1111时，此时已经消耗16火柴，而且此时z的位数最小，但仍一共需要28根火柴，那么x，y一定不大于1111，从0开始枚举x，y时间复杂度为n方，可以通过这道题。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为 O(n)

易错点

- 确认 0/1 起下标、首尾元素和 n=1。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a[2333] = {0}, b;
4 |     int c[10] = {6, 2, 5, 5, 4, 5, 6, 3, 7, 6}; // 预处理每个个数需要多少火柴。
5 |     int ans = 0, i, p;
6 |     scanf("%d", &b);
7 |     for (i = 0; i <= 2222; i++) {
8 |         p = i;
9 |         if (p == 0)
10 |             a[p] = 6;
11 |         while (p > 0) // 求每个数所用的火柴棒
12 |             {
13 |                 int d = p % 10;
14 |                 a[i] = a[i] + c[d];
15 |                 p = p / 10;
16 |             }
17 |     }
18 |     for (int i = 0; i <= 1111; i++) {
19 |         for (int j = 0; j <= 1111; j++)
20 |             if (a[i] + a[j] + a[i + j] + 4 <= b)
21 |                 ans++; // X,Y,Z,加号与等号。
22 |     }
23 |     printf("%d", ans);
24 |     return 0;
25 | }
```

### 公开样例

样例 1 输入

14

样例 1 输出

3

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.103 | 总 230/526 奇怪的魔法书 ( PID 1852 )

出处：2023级HAUT新生周赛（四）@牛浩然专场（H，CID 1089）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：9/64 ( 14.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示有多少种方案触发咒语。

输入要点：第一行一个字符串，代表s2（仅包含小写字母，0<s2的长度<=100000）。

第二行一个字符串，代表s1（仅包含小写字母，0<s1的长度<=100）。

输出要点：一个整数，表示有多少种方案触发咒语。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。



- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：奇怪的魔法书 s2串很小，直接暴力匹配即可。如果ba可以等于s2，那么一定有ba的长度=s2的长度。可以枚举后缀b，然后求出前缀a。然后把ba串与s2比较，如果一样，那么就让ans++即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int main() {
4 |     char arr[110000];
5 |     char brr[110];
6 |     scanf("%s %s", arr, brr);
7 |     int m = strlen(brr);
8 |     int n = strlen(arr);
9 |     int ans = 0;
10 |    for (int i = 1; i < m; ++i) // 枚举后缀长度
11 |    {
12 |        int p = m - i;
13 |        char drr[1100] = {0};
14 |        int cnt = 0;
15 |        for (int j = 0; j < i; ++j) {
16 |            drr[cnt] = arr[n - i + j];
17 |            cnt++;
18 |        }
19 |        for (int j = 0; j < p; ++j) {
20 |            drr[cnt] = arr[j];
21 |            cnt++;
22 |        }
23 |        if (strcmp(drr, brr) == 0) {
24 |            ans++;
25 |        }
26 |    }
27 |    printf("%d\n", ans);
28 |    return 0;
29 | }
```

## 公开样例

样例 1 输入

```
aabaabaabaab
aabaabaab
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

3.104 | 总 231/526 元素中和之力 ( PID 1854 )

出处：河南工大2024新生周赛（ 1 ）——命题人：程立老师，陈兰锴（ J，CID 1091 ）；2023级HAUT新生周赛（ 五 ）@陈兰锴专场（ B，CID 1090 ） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：18/152（ 11.8% ）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出 t行，每行一个整数 表示 最大原能

输入要点：输入 第一行 t，表示有 t组样例，接下来 t行,每行 七个整数 (分别表示 草 火 水 风 冰 雷 岩 元素 数量)  $x_i, 0 \leq x_i \leq 1000000, t \leq 1000000$ ;

输出要点：输出 t行，每行一个整数 表示 最大原能

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：元素中和之力 观察发现草或风只有一种中和反应方法，所以从草或风开始依次计算即可 反应时取反应元素中较小的元素值
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

$O(1)$

易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | void solve() {
5 |     ll x[8] = {0};
6 |     int t;
7 |     cin >> t;
8 |     while (t--) {
9 |         for (int i = 1; i <= 7; ++i) {
10 |             cin >> x[i];
11 |         }
12 |         ll ans = 0;
13 |         swap(x[4], x[7]); // 交换
14 |         for (int i = 7; i >= 1; --i) {
15 |             ll mi = min(x[i], x[i - 1]);
```

```

16 |         if (mi > 0) {
17 |             ans += mi;
18 |             x[i - 1] -= mi;
19 |         }
20 |     }
21 |     cout << ans << "\n";
22 | }
23 | }
24 | int main() {
25 |     ios::sync_with_stdio(false);
26 |     cin.tie(0);
27 |     cout.tie(0);
28 |     int test = 1;
29 |     // cin >> test;
30 |     while (test--) {
31 |         solve();
32 |     }
33 |     return 0;
34 | }

```

## 公开样例

样例 1 输入

```

2
1 1 1 1 1 1 1
0 3 6 6 5 1 9

```

样例 1 输出

```

3
12

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 3.105 | 总 232/526 旅行传送门 ( PID 1855 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（C，CID 1090）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：1/9 ( 11.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**不需要花费任何时间，每一个时空之球只能使用一次，为了尽快到达目的地，可莉希望花费最少的时间到达目的地，但派蒙并没有帮助可莉，笨蛋可莉再次向你寻求帮助，你能帮助她解决问题吗？

**输入要点：**第一行  $n$

第二行  $n$  个整数，第  $i$  个数  $a_i$  表示  $i$  位置传送门类型

**输出要点：**输出可以少花费的时间

题面提示：1 ≤  $a_i$  ≤ 1000000,  $n$  ≤ 5000; 派蒙不够聪明，（所以她不会走回头路）

注意要输出的结果是什么

样例解释，在位置 2 使用 时空之球 到 位置 7，可节省 5 个单位时间

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：旅行传送门 简单题 因为不会回头，考虑枚举中间点，在两边使用传送门，使用两个大空间数组，记录数字出现的位置，每枚举一次中间点都要将本次标记清除。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 4e6 + 10;
4 | typedef long long ll;
5 | ll ma1[N], ma2[N];
6 | ll a[N];
7 | void solve() {
8 |     int n;
9 |     cin >> n;
10 |    for (int i = 1; i <= n; ++i)
11 |        cin >> a[i];
12 |    ll ans = 0;
13 |    for (int i = 1; i <= n; ++i) {
14 |        ll l = 0, r = 0;
15 |        for (int j = i; j >= 1; --j) {
16 |            if (ma1[a[j]] == 0) {
17 |                ma1[a[j]] = j;
18 |            } else
19 |                l = max(l, ma1[a[j]] - j);
20 |        }
21 |        for (int j = i + 1; j <= n; ++j) {
22 |            if (ma2[a[j]] == 0) {
23 |                ma2[a[j]] = j;
24 |            } else
25 |                r = max(r, j - ma2[a[j]]);
26 |        }
27 |        // 清空数组
28 |        for (int j = i; j >= 1; --j) {
29 |            ma1[a[j]] = 0;
30 |        }
31 |        for (int j = i + 1; j <= n; ++j) {
32 |            ma2[a[j]] = 0;
33 |        }
34 |        ans = max(ans, l + r);
35 |    }
36 |    cout << ans;
37 | }
38 | int main() {
39 |     ios::sync_with_stdio(false);
40 |     cin.tie(0);
41 |     cout.tie(0);
42 |     int test = 1;
43 |     while (test--) {
44 |         solve();
45 |     }
46 |     return 0;
47 | }
```

公开样例

样例 1 输入

```
7
1 5 6 2 1 3 5
```

样例 1 输出

```
5
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.106 | 总 233/526 怎么了，你累啦？ ( PID 1858 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（F，CID 1090） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/36 ( 8.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，剩余原石能量值和

输入要点：第一行 一个整数  $n$

接下来一行， $n$  个整数 第  $i$  个整数  $x_i$  表示 第  $i$  块原石的原石值， $0 \leq x_i \leq 1e9$ ， $n \leq 4e6$ ;

输出要点：剩余原石能量值和

题面提示：第一块原石和最后一块不会消失

样例解释

原石消失后变为：

1 3 4 5

能量和为 13

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：怎么了，你累啦？ 难题 提前记录对于位置  $i$  的后面和前面的最大值，如果位置  $i$  的值比这两个最大值都要小，那么这个值就一定会消失。也可以用 一个队列 解决 本问题，要注意判别条件
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

$O(1)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | void solve() {
5 |     int n;
6 |     cin >> n;
7 |     vector<ll> a(n + 10); // 数组
8 |     vector<ll> r(n + 10); // 数组 // 记录后缀最大值
9 |     r[n + 1] = 0;
10 |    a[0] = 0;
11 |    ll ma = 0; // 记录前缀最大值
12 |    ll ans = 0;
13 |    for (int i = 1; i <= n; ++i) {
14 |        cin >> a[i];
15 |    }
16 |    for (int i = n; i >= 1; --i) {
17 |        r[i] = max(r[i + 1], a[i]);
18 |    }
19 |    for (int i = 1; i <= n; ++i) {
20 |        ma = max(ma, a[i - 1]);
21 |        if (a[i] >= ma || a[i] >= r[i + 1]) {
22 |            ans = ans + a[i];
23 |        }
24 |    }
25 |    cout << ans;
26 | }
27 | int main() {
28 |     ios::sync_with_stdio(false);
29 |     cin.tie(0);
30 |     cout.tie(0);
31 |     int test = 1;
32 |     // cin >> test;
33 |     while (test--) {
34 |         solve();
35 |     }
36 |     return 0;
37 | }
```

## 公开样例

样例 1 输入

```
5
1 3 2 4 5
```

样例 1 输出

```
13
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.107 | 总 234/526 你就是被上天选中的人 ( PID 1860 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（H，CID 1090）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：0/26 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出若干行，表示 每次操作3的结果

输入要点：第一行  $n$ ， $m$ （ $n$ 个数， $m$ 次魔法施加）

第二行  $n$  个数， $a_1 - a_n$

接下来  $m$  行，每行表示 一种魔法施加

输出要点：输出若干行，表示 每次操作3的结果

题面提示：说明  $1 \leq n \leq 1000000, 1 \leq m \leq 1000000, |a_i| \leq 1000000, 1 \leq y \leq 1000000, 1 \leq x \leq n$ , 操作 3 不超过 10000 次

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：你就是被上天选中的人 中等题 对于每次操作1和2，使用额外数组记录改变的值，对与操作3，枚举  $x$  的约数，因为操作3次数有限制，但要注意枚举到约数  $j$  时，另一个约数  $x/j$  也要计算。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

$O(\text{false})$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 4e6 + 10;
4 | typedef long long ll;
5 | ll a[N], add[N]; // add 是额外数组
6 | int n, m;
7 | void solve() {
8 |     ll ans = 0;
9 |     cin >> n >> m;
10 |     for (int i = 1; i <= n; ++i) {
11 |         cin >> a[i];
12 |     }
13 |     for (int i = 1; i <= m; ++i) {
14 |         string s;
15 |         int x, y;
```

```

16 |         cin >> s;
17 |         if (s[0] == 'u') {
18 |             cin >> x >> y;
19 |             add[x] += y;
20 |         } else if (s[0] == 'd') {
21 |             cin >> x >> y;
22 |             add[x] -= y;
23 |         } else {
24 |             cin >> x;
25 |             ll ans = a[x];
26 |             for (int i = 1; i * i <= x; ++i) {
27 |                 if (x % i == 0) {
28 |                     ans += add[i];
29 |                     if (i * i != x) {
30 |                         ans += add[x / i];
31 |                     }
32 |                 }
33 |             }
34 |             cout << ans << "\n";
35 |         }
36 |     }
37 | }
38 | int main() {
39 |     ios::sync_with_stdio(false);
40 |     cin.tie(0);
41 |     cout.tie(0);
42 |     int test = 1;
43 |     // cin >> test;
44 |     while (test--) {
45 |         solve();
46 |     }
47 |     return 0;
48 | }

```

## 公开样例

### 样例 1 输入

```

4 3
2 3 4 5
down 1 3
up 2 9
is 4

```

### 样例 1 输出

```

11

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.108 | 总 235/526 数学考试 ( PID 1877 )

出处：河南工大2024新生周赛（3）——命题人：张宏泽（G，CID 1093） | 训练定位：进阶 | 数组、字符串与双指针

标签：条件分支、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：43/198（21.7%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，对于每个测试用例，如果能使整个数组的和等于 0，则输出 "YES"，否则输出 "NO"。

**输入要点：**第一行包含一个整数  $t$  ( $1 \leq t \leq 100$ )

- 测试用例数。

每个测试用例的唯一一行包含两个整数  $n$  和  $m$  ( $0 \leq n, m < 10$ ) - 数组中 "1" 和 "2" 的个数。



输出要点：对于每个测试用例，如果能使整个数组的和等于 0，则输出 "YES"，否则输出 "NO"。

题面提示：-

$n = 0, m = 1$  :这意味着数组是 [2] - 不可能通过添加符号 "+" 或 "-" 得到结果 0；

-

$n = 0, m = 3$  :这意味着数组为 [2,2,2] - 不可能通过添加符号 "+" 或 "-" 得到结果 0；

-

$n = 2, m = 0$  :这意味着数组是 [1,1] - 可以通过添加符号 "+" 或 "-" 得到结果 0 ( $+1 -1 = 0$ )；

-

$n = 2, n = 3$  :这意味着数组为 [1,1,2,2,2] - 可以通过添加符号 "+" 或 "-" 得到结果 0 ( $+1 +1 -2 -2 +2 = 0$ )；

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：数学考试 思维题。- 首先要能发现，当  $\backslash(n)$ 、 $\backslash(m)$  是偶数时，即可以一半  $\backslash(1)$  为正数，一半  $\backslash(1)$  的负数，组内消化掉。  
- 当  $\backslash(n)$  是奇数时，即当  $\backslash(1)$  的个数是奇数的时，显然  $\backslash(n - 1)$  是偶数，那么必然能把  $\backslash(n - 1)$  个  $\backslash(1)$  分成  $\backslash(n - 1) / 2$  组，那么每组中的两个  $\backslash(1)$ ，一个前面是 "+"，一个前面是 "-"，则一定可以把  $\backslash(n - 1)$  个  $\backslash(1)$  抵消掉，无论  $\backslash(2)$  有多少个，也无法与最后剩下的那个奇数  $\backslash(1)$  相消，得证：当  $\backslash(n)$  为奇数是，一定无法使数组之和等于  $\backslash(0)$ 。- 当  $\backslash(2)$  的个数  $\backslash(m)$  是奇数时，当且仅当  $\backslash(1)$  的个数为偶数，且  $\backslash(1)$  个数不为  $\backslash(0)$  才可以存在两个  $\backslash(1)$  和多出来的那个  $\backslash(2)$  相消。而除此之外的所有情况都存在一种可能性使得数组的元素之和等于  $\backslash(0)$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | void solve() {
3 |     int n, m;
4 |     scanf("%d%d", &n, &m);
5 |     if (n & 1) // 判断是奇数
6 |     {
7 |         puts("NO");
8 |         return;
9 |     }
10 |    if (m % 2 && !n) // n == 0 而且 m 是奇数
11 |    {
12 |        puts("NO");
13 |        return;
14 |    }
15 |    puts("YES");
16 | }
17 | int main() {
18 |     int T;
19 |     scanf("%d", &T);
20 |     while (T--)
21 |         solve();
```

```
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

```
5
0 1
0 3
2 0
2 3
3 1
```

样例 1 输出

```
NO
NO
YES
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 3.109 | 总 236/526 200个数字 ( PID 1878 )

出处：河南工大2024新生周赛 ( 3 ) ——命题人：张宏泽 ( H , CID 1093 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：50/102 ( 49.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给定一个长度为 $n$ 数组  $a$ ，请你输出该数组中出现次数最多的数字是哪个。

输入要点：第一行包含一个整数  $n$  (  $1 \leq n \leq 10^5$  )

- 测试用例数。

第二行包含 $n$ 个整数 $a_i$ ，(  $-100 \leq a_i \leq 100$  )，每两个数字之间用空格隔开- 数组中的数字。

输出要点：仅输出一个数字 $a_i$ ，表示数组中出现次数最多的数字，如果有多个数字出现的次数均为最多，

则输出较大的那个。

题面提示：样例中，35和-8 均出现了两次，那么输出较大的35

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：200个数字 首先想到使用数组  $cnt[i]$  记录 数字 $i$  出现的个数。其次我们发现数字 $i$ 可能是负数，那么增加一个偏移量就行了，譬如 偏移量  $C = 110$ ，那么此时  $cnt[10]$  记录就是 -100 出现的个数 计算公式如  $-100 + C = 10$ ；此时从高向低遍历一遍，找到出现次数最多的数字，由于从高向低遍历，则一定满足该数字是出现次数一样多的数字里面的最大的。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

C++17 参考实现

```
1 | #include <stdio.h>
2 | const int C = 110; // 偏移量
3 | int cnt[400], n;
4 | void solve() {
5 |     scanf("%d", &n);
6 |     for (int i = 1; i <= n; i++) {
7 |         int x;
8 |         scanf("%d", &x);
9 |         cnt[x + C]++;
10 |     }
11 |     int Max = 0, ans = -1e9;
12 |     for (int i = -100 + C; i <= 100 + C; i++) {
13 |         if (Max <= cnt[i]) {
14 |             Max = cnt[i];
15 |             ans = i - C;
16 |         }
17 |     }
18 |     printf("%d\n", ans);
19 | }
20 | int main() {
21 |     int T = 1;
22 |     while (T--)
23 |         solve();
24 |     return 0;
25 | }
```

公开样例

|                                      |
|--------------------------------------|
| 样例 1 输入                              |
| 10<br>-1 -19 35 97 -8 -8 33 35 100 3 |
| 样例 1 输出                              |
| 35                                   |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

3.110 | 总 237/526 Raging Loop ( PID 1881 )

出处：河南工大2024新生周赛（4）——命题人：马贺（C，CID 1094） | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数位、条件分支 | 限制：1.000 S / 128 MB | 历史统计：51/518（9.8%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，“Hai”或者“chi ga u”

输入要点：一个整数 $x(1 \leq x \leq 100000)$

输出要点：“Hai”或者“chi ga u”

题面提示：形如12345，23456，78910这种都不是哦。

[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116180453\\_94916.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116180453_94916.png)]

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把整数当作十进制字符串检查。公开题解给出的判定是：前两位必须为“10”；第三位大于1时成立，第三位等于1时还必须存在第四位。其余情况均输出 chi ga u。字符串写法避免了数值位数判断的边界错误。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(|x|)$  时间、 $O(|x|)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string number;
5 |     cin >> number;
6 |     bool raging = number.size() >= 3 && number[0] == '1' && number[1] == '0' &&
7 |         (number[2] > '1' || (number[2] == '1' && number.size() > 3));
8 |     cout << (raging ? "Hai" : "chi ga u") << '\n';
9 |     return 0;
10 | }
```

## 公开样例

样例 1 输入

1005

样例 1 输出

chi ga u

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1
- 题面图片 1

3.111 | 总 238/526 NEEDY GIRL OVERDOSE ( PID 1886 )

出处：河南工大2024新生周赛（4）——命题人：马贺（H，CID 1094）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：19/44 ( 43.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：为了保证副作用带来的伤害最低并且药效最好，糖糖让阿p你来帮助她分药，要求，分成的k段中，每段药物剂量的最大值最小，比如：4 2 4 5 1中，要求分成三次吃完，若分成[4 2][4 5][1]，最大值为4+5=9；若分成 [4][2 4][5 1]，最大值为2+4或者5+1=6；并且，其他任何分段方案的最大值，没有比6更小的，故对于4 2 4 5 1，每段.....

输入要点：第一行两个整数n和k，代表药物总片数和要求分的次数（段数）， $1 \leq n \leq 100000, 1 \leq k \leq n$ 。

第二行n个整数 $a_i$ ，代表每片的剂量， $1 \leq a_i \leq 1000000000$ ）。

保证答案不会超过1000000000。

输出要点：一个整数，代表每段药物剂量的最大值最小为多少。

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175019\\_10411.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175019_10411.png)]

[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175030\\_34163.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175030_34163.png)]

糖糖提醒你：大学诱惑很多，请保护好自己，做到自爱自重自强。

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：NEEDY GIRL OVERDOSE 注意到如果分段数增加，最大值会减小；反之最大值会增大。并且，对于一个最大值，如果对应的段数小于要求的段数，那么比这个最大值还大的最大值对应段数一定也小于要求的段数。对于这种满足单调性的答案，考虑二分。每次选出来一个数作为预估的最大值，如果预估值对应的段数  $\leq$  要求的段数，则这个预估答案是合理的，但不一定是最优的，需要考虑有没有更小的
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int max(int a, int b) {
3 |     return a > b ? a : b;
4 | }
5 | int main() {
6 |     int n, k;
7 |     scanf("%d %d", &n, &k);
8 |     int num[100005] = {0};
9 |     int rt = 0;
10 |    int lt = 0;
11 |    for (int i = 0; i < n; ++i) {
12 |        scanf("%d", &num[i + 1]);
13 |        rt += num[i + 1]; // 答案最大的时候，一次吃完全部的药物
14 |        lt = max(lt, num[i + 1]); // 答案最小也应该是所有药物中剂量最大的一篇
15 |    }
16 |    while (lt < rt) {
17 |        int mid = (lt + rt) / 2;
18 |        int flag;
19 |        int segments = 1; // 当前答案下分的段数
20 |        int sum = 0;
21 |        for (int i = 1; i <= n; ++i) {
22 |            if (sum + num[i] > mid) {
23 |                segments++;
24 |                sum = 0;
25 |            }
26 |            sum += num[i];
27 |        }
28 |        if (segments > k)
29 |            flag = 1; // 说明这种分法不可行 需要的段数超过了要求
30 |        else
31 |            flag = 0;
32 |        if (flag)
33 |            lt = mid + 1;
34 |        else
35 |            rt = mid;
36 |    }
37 |    printf("%d", lt);
38 |    return 0;
39 | }
```

## 公开样例

样例 1 输入

```
53
42451
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 3.112 | 总 239/526 大大大 (PID 1888)

出处：河南工大2024新生周赛（5）——命题人：刘庭钰（B，CID 1095） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：21/159（13.2%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：请问，进行第几次操作时使得式  $S$  最大？若存在多种答案，请输出操作次数的最小值

输入要点：输入包含 2 行

第一行两个整数  $n, t$  ( $1 \leq n \leq 2e5, 0 \leq t \leq 1e9$ ) 表示数组  $a$  的长度和最多的操作次数

第二行  $n$  个实数  $a_i$  ( $1 \leq a_i \leq 1e9$ ) 表示数组  $a$  的元素

输出要点：输出包含一行一个整数，表示使得  $S$  最大的最小操作次数

题面提示：注意除法的性质

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：大大大只需要保证第一位数字最大即可使式  $S$  最大 注意：如果将最大值右移到数组头部的次数大于  $t$ ，从后往前遍历找  $t$  次内能一刀数组头部的最大值
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 2e5 + 10;
4 | int n, t, maxx = -1, idx = -1;
5 | int a[N];
6 | int main() {
7 |     cin >> n >> t;
8 |     for (int i = 1; i <= n; i++) {
9 |         cin >> a[i];
10 |         if (a[i] >= maxx) // 存储最大值的位置与值
11 |         {
12 |             maxx = a[i];
13 |             idx = i;
14 |         }
15 |     }
16 |     if (idx == 1) // 如果最大值在第一位
17 |     {
18 |         cout << 0;
19 |         return 0;
20 |     } else {
21 |         if (n - idx + 1 <= t) {
22 |             cout << n - idx + 1;
23 |             return 0;
24 |         } else {
25 |             maxx = a[n]; // 从后往前遍历，假设数组尾部的元素为最大值进行比较
26 |             idx = n; // 位置为 n
27 |             for (int i = n; i >= n - t + 1; i--) {
28 |                 if (a[i] > maxx) // 如果有更大的值，记录
29 |                 {
30 |                     maxx = a[i];
31 |                     idx = i;
32 |                 }
33 |             }
```

```
34 |         cout << n - idx + 1; // 输出次数
35 |         return 0;
36 |     }
37 | }
38 | }
```

## 公开样例

样例 1 输入

```
5 10
5 3 4 1 2
```

样例 1 输出

```
0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.113 | 总 240/526 玫瑰花的葬礼 ( PID 1889 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭铄 ( C , CID 1095 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/59 ( 3.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在他拿到了一个数组，数组之灵告诉他，如果可以让该数组的记忆落痕值最大，他就可以回到495天前挽回那位友人。学长最多可以进行1次操作：选择一个元素，使其+1。请你帮助学长求出最多1次操作后，数组的记忆落痕值最大为多少

输入要点：第一行输入一个正整数 $n$ ，代表每个数组的大小

第二行输入 $n$ 个正整数 $a_i$ ，代表数组的元素

$1 \leq n, a_i \leq 400000$ ;

输出要点：一个整数，代表最多一次操作后，数组的最大记忆落痕值

题面提示：悟已往之不鉴 知来者之可追

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：玫瑰花的葬礼 由于元素组合个数最大为  $495 * 495 \approx 2.5 * 10^5$ ，所以直接预处理出所有的组合 对于数组中的每个元素，如果两个元素大于495，则对于该元素，对495取模的效果与用该元素直接寻找方案数效果相同，即取模后存入  $0 \sim 495$  的哈希表即可（桶排序）再对hash表中已经存入的数字进行遍历，由于只能操作一次使得某个元素+1，则对每个元素判断+1和不变两种情况时，方案数最大值即可 还有一种方法是通过分解495的质因子为  $3 * 3 * 5 * 11$ ，因此若两个数含有  $3 * 3 * 5 * 11$  这四个质因子，则相乘后的数一定是495的倍数，这是官方正解 详情可以去b站搜索，这里不再赘述 原题链接：D-红魔馆的馆主 ( 二 ) \_牛客周赛 Round 68
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明



- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define int long long
4 | const int res = 495;
5 | int n;
6 | int hashes[res + 10];
7 | int solve() // 预处理
8 | {
9 |     int ans = 0;
10 |    for (int i = 0; i < res; i++)
11 |        for (int j = 0; j < res; j++) {
12 |            // 如果是 i * j 是 495 的倍数 且哈希表中存在 i 与 j 的值 (数组中出现过 i, j)
13 |            if (i * j % res == 0 && hashes[i] > 0 && hashes[j] > 0) {
14 |                // 每个 hashes[i] 与其余个 hashes[j] 配对相乘
15 |                if (i == j)
16 |                    ans += hashes[i] * (hashes[i] - 1);
17 |                // 值为 i 的元素个数与值为 j 的元素个数可以进行组合，组合方案数为 i * j
18 |                else
19 |                    ans += (hashes[i] * hashes[j]);
20 |            }
21 |        }
22 |    return ans;
23 | }
24 | signed main() {
25 |    cin >> n;
26 |    for (int i = 1; i <= n; i++) {
27 |        int a;
28 |        cin >> a;
29 |        hashes[a % res]++; // 存入 hash 表中，表示出现次数;
30 |    }
31 |    int maxx = solve(); // 定义最开始时的记忆落痕最大值
32 |    for (int i = 0; i < res; i++) {
33 |        if (hashes[i]) // 如果数组中存在该值或存在取模后为该值的元素
34 |        {
35 |            // 选择 +1 后判断方案数
36 |            hashes[i]--;
37 |            hashes[(i + 1) % res]++;
38 |            maxx = max(maxx, solve());
39 |            // 回溯，复原该元素未 +1 前的 hash 表状态
40 |            hashes[i]++;
41 |            hashes[(i + 1) % res]--;
42 |        }
43 |    }
44 |    // 因为每个元素都进行了重复使用，对结果除以二后输出即可
45 |    cout << maxx / 2;
46 |    return 0;
47 | }
```

## 公开样例

样例 1 输入

```
4
1 9 55 494
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

3.114 | 总 241/526 倾盆的雨下了一整夜 ( PID 1894 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭铄 ( H , CID 1095 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：81/217 ( 37.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：请你帮红月大人求出有多少个 $2 * 2$ 的区域满足该区域内全部都在降雨

输入要点：第一行输入两个正整数  $n, m$ ，代表地图的行数和列数

接下来的  $n$  行，每行输入一个仅包含'0'和'1'的字符串，用来表示地图的降雨情况

$1 \leq n, m \leq 100$

输出要点：一个正整数，代表 $2 * 2$ 的区域都在降雨的区域数量

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：倾盆的雨下了一整夜 二维数组在避免边界越界的情况下进行遍历即可
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 1e3 + 10;
4 | int n, m;
5 | char g[N][N];
6 | int main() {
7 |     cin >> n >> m;
8 |     for (int i = 1; i <= n; i++)
9 |         for (int j = 1; j <= m; j++)
10 |             cin >> g[i][j];
11 |     // 字符型或字符串型才可输入，否则若定义g[N][N]为int型
```

```

12 | // 会出现只在 g[i][j] 中输入了一个大整数的情况
13 | int res = 0;
14 | // 避免边界, i, j 从 2 开始, 每次与上一行/上一列进行判断
15 | for (int i = 2; i <= n; i++)
16 |     for (int j = 2; j <= m; j++)
17 |         if (g[i][j] == '1' && g[i-1][j] == '1' && g[i][j-1] == '1' &&
18 |             g[i-1][j-1] == '1')
19 |             res++;
20 | cout << res;
21 | return 0;
22 | }

```

## 公开样例

### 样例 1 输入

```

3 4
0010
0111
1110

```

### 样例 1 输出

```

1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 3.115 | 总 242/526 切巧克力 (PID 1912)

出处：河南工大2024新生周赛（7）——命题人：刘义（E，CID 1098）| 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列 | 限制：1.000 S / 128 MB | 历史统计：4/24 (16.7%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**现在，给出从不同的线切割所要花的代价，求把整块巧克力分割成  $1 \times 1$  块小方块所需要耗费的最小代价。

**输入要点：**第一行  $N, M$ ，表示  $N \times M$  的巧克力；第二行  $N-1$  个非负整数，表示各条横切线的代价；第三行  $M-1$  个非负整数，表示各条竖切线的代价。

**输出要点：**输出一个整数，表示最小代价。

题面提示： $1 \leq N, M \leq 2000$ 。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：切巧克力：输入 4 2 4 5 6 1 输出 19 如果先切横的，结果就是  $1+2 \times 4+2 \times 5+2 \times 6=31$  如果先切竖的，结果就是  $4+5+6+1 \times 4=19$ 。显然先切竖的更优。由此，可以得出一个结论：先切代价大的。于是，思路就出来了。先把两个数组从大到小排序，然后两个数组挨个比较，较大的就在答案上加上这个代价（要乘上切了多少刀），再把指针往后挪一位，直到全部算完即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <cmath>
3 | #include <algorithm>
4 | using namespace std;
5 | int n, m;
6 | int a[10005], b[10005]; // 横线和竖线的代价
7 | int main() {
8 |     cin >> n >> m;
9 |     int s1 = 1, s2 = 1; // s1和s2可以是a数组和b数组的指针，也可以是横纵分成的块数
10 |    for (int i = 1; i <= n - 1; i++) {
11 |        cin >> a[i];
12 |    }
13 |    for (int i = 1; i <= m - 1; i++) {
14 |        cin >> b[i];
15 |    }
16 |    for (int i = 1; i <= n - 1; i++) {
17 |        for (int j = 1; j <= m - 1 - i; j++) {
18 |            if (a[j] < a[j + 1]) {
19 |                int t;
20 |                t = a[j];
21 |                a[j] = a[j + 1];
22 |                a[j + 1] = t;
23 |            }
24 |        }
25 |    }
26 |    for (int i = 1; i <= m - 1; i++) {
27 |        for (int j = 1; j <= n - 1 - i; j++) {
28 |            if (b[j] < b[j + 1]) {
29 |                int t;
30 |                t = b[j];
31 |                b[j] = b[j + 1];
32 |                b[j + 1] = t;
33 |            }
34 |        }
35 |    }
36 |    long long ans = 0;
37 |    for (int i = 2; i < n + m; i++) {
38 |        if (a[s1] > b[s2])
39 |            ans += s2 * a[s1++]; // 选择大的，这里s2表示块数，s1指针后移一位
40 |        else
41 |            ans += s1 * b[s2++]; // 同理，和上面相反
42 |    }
43 |    cout << ans << endl;
44 |    return 0;
45 | }
```

## 公开样例

样例 1 输入

```
2 2
3
3
```

样例 1 输出

```
9
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.116 | 总 243/526 A\*B ( PID 1913 )

出处：河南工大2024新生周赛 ( 7 ) ——命题人：刘义 ( C , CID 1098 ) | 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：28/206 ( 13.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个非负整数表示乘积。

输入要点：输入共两行，每行一个非负整数。

输出要点：输出一个非负整数表示乘积。

题面提示：每个非负整数不超过  $10^{2000}$ 。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：A\*B：经典的高精度乘法：`#include <stdio.h> #include <string.h> //翻转函数 void reverse (int d[510], int len) { int temp, mid = len/2; for (int i = 0, j = len - 1; i < mid, j >= mid; i++, j--) { temp = d[i]; d[i] = d[j]; d[j] = temp; } }`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | // 翻转函数
4 | void reverse(int d[510], int len) {
5 |     int temp, mid = len / 2;
6 |     for (int i = 0, j = len - 1; i < mid, j >= mid; i++, j--) {
7 |         temp = d[i];
8 |         d[i] = d[j];
9 |         d[j] = temp;
10 |    }
11 | }
```

```

12 | int main() {
13 |     char s1[2005], s2[2005];
14 |     int a[2005], b[2005], c[4010] = {0};
15 |     scanf("%s", s1);
16 |     scanf("%s", s2);
17 |     int lena = strlen(s1);
18 |     int lenb = strlen(s2);
19 |     int lenc = lena + lenb;
20 |     // 将char的1转为int的1
21 |     for (int i = 0; i < lena; i++) {
22 |         a[i] = s1[i] - '0';
23 |     }
24 |     for (int i = 0; i < lenb; i++) {
25 |         b[i] = s2[i] - '0';
26 |     }
27 |     reverse(a, lena);
28 |     reverse(b, lenb);
29 |     // 核心代码
30 |     for (int i = 0; i < lenb; i++) {
31 |         for (int j = 0; j < lena; j++) {
32 |             c[i + j] += b[i] * a[j];
33 |             c[i + j + 1] += c[i + j] / 10;
34 |             c[i + j] = c[i + j] % 10;
35 |         }
36 |     }
37 |     // 处理前导0
38 |     lenc = lenc - 1;
39 |     while (c[lenc] == 0 && lenc >= 1) {
40 |         lenc--;
41 |     }
42 |     // 输出
43 |     for (int i = lenc; i >= 0; i--) {
44 |         printf("%d", c[i]);
45 |     }
46 |     return 0;
47 | }

```

## 公开样例

样例 1 输入

```

1
2

```

样例 1 输出

```

2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.117 | 总 244/526 A-B ( PID 1957 )

出处：河南工大2025新生周赛（5）——命题人：袁林坤（A，CID 1106）| 训练定位：进阶 | 数组、字符串与双指针

标签：字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：69/591 ( 11.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，结果(是负数要输出负号)

输入要点：两个整数A,B(第二个可能比第一个大)

输出要点：结果(是负数要输出负号)

题面提示：-

100% 数据  $0 < a, b \leq 1010086$ 。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：A-B 简单的的高精度减法，数组模拟算术减法即可，注意范围。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 1e6;
4 | int a[N], b[N], c[N];
5 | int main() {
6 |     string s1, s2;
7 |     cin >> s1 >> s2;
8 |     if (s1.size() < s2.size() || (s1.size() == s2.size() && s1 < s2)) {
9 |         string s = s1;
10 |        s1 = s2;
11 |        s2 = s;
12 |        cout << '-';
13 |    }
14 |    reverse(s1.begin(), s1.end());
15 |    reverse(s2.begin(), s2.end());
16 |    int lena = s1.size(), lenb = s2.size();
17 |    int lenc = max(lena, lenb) + 1;
18 |    for (int i = 0; i < lena; i++)
19 |        a[i] = s1[i] - '0';
20 |    for (int i = 0; i < lenb; i++)
21 |        b[i] = s2[i] - '0';
22 |    for (int i = 0; i < lenc; i++) {
23 |        if (a[i] < b[i])
24 |            a[i + 1]--, a[i] += 10;
25 |        c[i] = a[i] - b[i];
26 |    }
27 |    while (lenc > 1 && !c[lenc - 1])
28 |        lenc--;
29 |    for (int i = lenc - 1; i >= 0; i--)
30 |        cout << c[i];
31 |    return 0;
32 | }
```

## 公开样例

样例 1 输入

```
2
1
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.118 | 总 245/526 这真是一道签到题吗？(PID 1971)

出处：河南工大2025新生周赛(6)——命题人：张康发(C, CID 1107) | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、字符串 | 限制：1.000 S / 128 MB | 历史统计：9/131 (6.9%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**在一次操作中可以指定相邻的两个数，将它们一起加 1 或减 1；也可以只指定一个数加 1 或减 1，请问最少需要操作多少次能把这个数组变成回文数组？

**输入要点：**输入的第一行包含一个正整数  $n$ 。

第二行包含  $n$  个整数  $a_1, a_2, \dots, a_n$ ，相邻整数之间使用一个空格分隔。

**输出要点：**输出一行包含一个整数表示答案。

题面提示： $1 \leq n \leq 105$ ,

$-106 \leq a_i \leq 106$

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：这真是一道签到题吗？题目要求变为回文数组，那么对于数组两边对称的数来说，左边的数加一相当于右边的数减一，左边的数减一相当于右边的数加一。因此可以先对数组两边初始化，都减去较小的数，使得数组的元素全部变为非负整数。最后用贪心的方法计算答案。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。



- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | const ll N = 2e5 + 10;
5 | int n;
6 | int a[N];
7 | int main() {
8 |     cin >> n;
9 |     for (int i = 1; i <= n; i++)
10 |         cin >> a[i];
11 |     for (int i = 1, j = n; i <= j; i++, j--) {
12 |         int t = min(a[i], a[j]);
13 |         a[i] -= t;
14 |         if (i != j)
15 |             a[j] -= t;
16 |     }
17 |     ll res = 0;
18 |     for (int i = 1; i <= n; i++) {
19 |         int t = a[i];
20 |         res += t;
21 |         a[i + 1] -= min(t, a[i + 1]);
22 |     }
23 |     cout << res;
24 |     return 0;
25 | }
```

### 公开样例

样例 1 输入

```
4
1 2 3 4
```

样例 1 输出

```
3
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.119 | 总 246/526 这是一道签到题吗？ ( PID 1974 )

出处：河南工大2025新生周赛（6）——命题人：张康发（A，CID 1107） | 训练定位：进阶 | 数组、字符串与双指针

标签：数组与序列、构造 | 限制：1.000 S / 128 MB | 历史统计：155/910（17.0%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出最终矩阵，具体形式参照输出样例。

输入要点：共一行，包含两个整数 N 和 M。

输出要点：输出最终矩阵，具体形式参照输出样例。

题面提示：0≤n，m≤100。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：这是一道签到题吗？用双层循环模拟即可，注意特殊输出的判断。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | const int P = 1e9 + 7;
5 | const int N = 1e2 + 10;
6 | int a[N][N];
7 | int main() {
8 |     int n, m;
9 |     cin >> n >> m;
10 |     int num = 0;
11 |     for (int i = 1; i <= n; i++) {
12 |         for (int j = 1; j <= m - 1; j++) {
13 |             num++;
14 |             cout << num << ' ';
15 |         }
16 |         num++;
17 |         if (i % 2 == 1)
18 |             cout << "HAUT" << endl;
19 |         else
20 |             cout << num << endl;
21 |     }
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

7 4

样例 1 输出

```
1 2 3 HAUT
5 6 7 8
9 10 11 HAUT
13 14 15 16
17 18 19 HAUT
21 22 23 24
25 26 27 HAUT
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 3.120 | 总 247/526 进制转换 ( PID 1988 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（H，CID 1110）| 训练定位：进阶 | 数组、字符串与双指针

标签：进制转换、字符串、数位 | 限制：1.000 S / 128 MB | 历史统计：36/85 ( 42.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个正整数，表示转换之后的  $m$  进制数。

输入要点：共三行，第一行是一个正整数，表示需要转换的数的进制  $n$  ( $2 \leq n \leq 16$ )，第二行是一个  $n$  进制数，若  $n > 10$  则用大写字母 A~F 表示数码 10~15，并且该  $n$  进制数对应的十进制的值不超过 109，第三行也是一个正整数，表示转换之后的数的进制  $m$  ( $2 \leq m \leq 16$ )。

输出要点：一个正整数，表示转换之后的  $m$  进制数。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：先逐位把源进制串转成十进制整数： $value = value \times n + digit$ ；再反复 对目标进制  $m$  取余得到从低到高的数字，最后逆序输出。数值范围不超过  $10^9$ ，long long 足够；特别处理  $value = 0$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(|s| + \log_m \text{value})$  时间、 $O(\log_m \text{value})$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int digitValue(char character) {
4 |     if (isdigit(static_cast<unsigned char>(character)))
5 |         return character - '0';
6 |     return toupper(static_cast<unsigned char>(character)) - 'A' + 10;
7 | }
8 | char ch(int value) {
9 |     return value < 10 ? char('0' + value) : char('A' + value - 10);
10 | }
11 | int main() {
12 |     int srcBase, dstBase;
13 |     string number;
14 |     cin >> srcBase >> number >> dstBase;
15 |     long long value = 0;
16 |     for (char character : number) {
```

```

17 |         value = value * srcBase + digitValue(character);
18 |     }
19 |     if (value == 0) {
20 |         cout << "0\n";
21 |         return 0;
22 |     }
23 |     string ans;
24 |     while (value > 0) {
25 |         ans.push_back(ch(value % dstBase));
26 |         value /= dstBase;
27 |     }
28 |     reverse(ans.begin(), ans.end());
29 |     cout << ans << '\n';
30 |     return 0;
31 | }

```

## 公开样例

样例 1 输入

```

16
FF
2

```

样例 1 输出

```

11111111

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数组、字符串与双指针”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 4 | 排序、二分与区间

排序把无序信息转成可利用的单调结构；二分依赖单调性；区间题常考端点、闭开边界与贪心选择。

本模块共 26 道唯一题。

## 4.1 | 总 248/526 Meeting Friends ( PID 1213 )

出处：2016级河南工大新生周赛（三）（A，CID 1008）| 训练定位：入门 | 排序、二分与区间

标签：中位数、绝对值距离、排序 | 限制：1.000 S / 128 MB | 历史统计：196/371 ( 52.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，Print one integer — the minimum total distance the friends need to travel in order to meet together.

**输入要点：**The first line of the input contains three distinct integers  $x_1$ ,  $x_2$  and  $x_3$  ( $1 \leq x_1, x_2, x_3 \leq 100$ ) — the coordinates of the houses of the first, the second and the third friends respectively.

**输出要点：**Print one integer — the minimum total distance the friends need to travel in order to meet together.

**题面提示：**In this sample, friends should meet at the point 4. Thus, the first friend has to travel the distance of 3 (from the point 7 to the point 4), the second friend also has to travel the distance of 3 (from the point 1 to the point 4), while the third friend should not go anywhere because he lives at the point 4.

## 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：三人在数轴上一点会合时，绝对距离和在中位数处最小。把三个坐标 排序为  $x \leq y \leq z$ ，选  $y$  后总路程为  $(y-x)+(z-y)=z-x$ ；直接输出最大 坐标减最小坐标即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     array<int, 3> x{};
5 |     cin >> x[0] >> x[1] >> x[2];
6 |     sort(x.begin(), x.end());
7 |     cout << x[2] - x[0] << '\n';
8 |     return 0;
9 | }
```

## 公开样例

样例 1 输入

7 1 4

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.2 | 总 249/526 崩崩崩 ( PID 1322 )

出处：2017级河南工大新生周赛 ( 三 ) ( B , CID 1030 ) | 训练定位：入门 | 排序、二分与区间

标签：模拟、前缀和、单位换算 | 限制：1.000 S / 128 MB | 历史统计：189/379 ( 49.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，Stone最早第几天得到自己心仪的女武神。

输入要点：第一行，两个数 $n$ 和 $t$ ， $n \rightarrow$ 距离期末考试的天数， $t \rightarrow$ 收集所有八重樱碎片需要的总时间（单位是秒）。

数据范围：(  $1 \leq n \leq 100$  ) (  $1 \leq t \leq 1000000$  )

第二行， $n$  个数字  $a[1], a[2] \dots a[n]$ ，每个数字用空格隔开， $a[i] \rightarrow$  每天 Stone 被其他琐事所占用的时间（单位是秒）。

数据范围：( $0 \leq a[i] \leq 86400$ )

输出要点：Stone 最早第几天得到自己心仪的女武神。

题面提示：一天有 86400 秒

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- $\text{prefix}[i]$  表示前  $i$  项之和，区间  $[l, r]$  的和为  $\text{prefix}[r] - \text{prefix}[l-1]$ 。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：一天共有 86400 秒，第  $i$  天可用于收集的时间是  $86400 - a[i]$ 。从第一天开始累加空闲时间，第一次达到所需总时间  $t$  的下标就是最早天数。也可以每读一天便从  $t$  中减去当天空闲时间，首次  $t \leq 0$  时输出。题目数据按其设定会在给定天数内完成；代码仍为异常输入保留 -1。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long required;
8 |     cin >> n >> required;
9 |     int ans = -1;
10 |    for (int day = 1; day <= n; ++day) {
11 |        long long occupied;
12 |        cin >> occupied;
13 |        if (ans == -1) {
14 |            required -= 86400 - occupied;
15 |            if (required <= 0)
16 |                ans = day;
17 |        }
18 |    }
19 |    cout << ans << '\n';
20 |    return 0;
21 | }
```

## 公开样例

样例 1 输入

```
2 2
86400 86398
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 4.3 | 总 250/526 数字分组 ( PID 1437 )

出处：2018级河南工大新生周赛 ( 七 ) @牛寅旭专场 ( A , CID 1046 ) | 训练定位：入门 | 排序、二分与区间

标签：前缀和与差分、构造、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出有一行，为各组数字的数量。若无结果，输出-1.

输入要点：第一行为两个数字n和k，表示数字的数量和要分的组数。

第二行为n个数字，每个数字都在int范围内。

$0 < n \leq 10^6$

$k \leq n$

输出要点：输出有一行，为各组数字的数量。

若无结果，输出-1.

### 零基础先修

- 前缀和让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：每组元素和必须等于  $total/k$ ，所以  $total$  不能整除  $k$  时直接失败。用前缀和依次寻找  $target$ 、 $2target$ 、...、 $(k-1)target$  的边界，且每个边界下标必须严格递增并早于  $n$ ；找到后输出相邻边界之差。这个写法对负数和  $target=0$  也成立。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

### 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, k;
7 |     cin >> n >> k;
8 |     vector<long long> a(n);
9 |     long long total = 0;
10 |    for (long long& x : a) {
11 |        cin >> x;
12 |        total += x;
13 |    }
14 |    if (total % k != 0) {
15 |        cout << -1 << '\n';
16 |        return 0;
17 |    }
18 |    long long target = total / k, prefix = 0;
19 |    vector<int> cuts;
20 |    int needed = 1;
21 |    for (int i = 1; i < n && needed < k; ++i) {
22 |        prefix += a[i - 1];
23 |        if (prefix == target * needed) {
24 |            cuts.push_back(i);
25 |            ++needed;
26 |        }
27 |    }
28 |    if (needed != k) {
29 |        cout << -1 << '\n';
30 |        return 0;
31 |    }
32 |    int pre = 0;
33 |    for (int cut : cuts) {
34 |        cout << cut - pre << ' ';
35 |        pre = cut;
36 |    }
37 |    cout << n - pre << '\n';
38 |    return 0;
39 | }
```

## 公开样例

样例 1 输入

```
5 3
1 4 5 2 3
```

样例 1 输出

```
2 1 2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)



## 4.4 | 总 251/526 cxx的篮球数 ( PID 1477 )

出处：2019级河南工大新生周赛 ( 一 ) @陶福专场 ( D , CID 1053 ) | 训练定位：入门 | 排序、二分与区间

标签：前缀和与差分、区间修改、数组 | 限制：1.000 S / 128 MB | 历史统计：84/165 ( 50.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：cxx的篮球数

输入要点：第一行输入n,m。第二行输入n个数，代表初始每筐中篮球数，接下来m行，每行包括三个数L,R,W。最后一行输入一个数x。

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

### 零基础先修

- 前缀和与让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：区间  $[L,R]$  同加  $W$  可用差分数组： $\text{difference}[L] += W$ ， $\text{difference}[R+1] -= W$ 。所有操作后做一次前缀和即可恢复每个位置的总增量，再添加到初值；虽然只询问一个  $x$ ，这种写法也便于迁移到多询问。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n+m)$  时间、 $O(n)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n, m;
5 |     cin >> n >> m;
6 |     vector<long long> a(n + 1), diff(n + 2, 0);
7 |     for (int i = 1; i <= n; ++i)
8 |         cin >> a[i];
9 |     while (m--) {
10 |         int l, r;
11 |         long long w;
12 |         cin >> l >> r >> w;
13 |         diff[l] += w;
14 |         diff[r + 1] -= w;
15 |     }
16 |     int target;
17 |     cin >> target;
18 |     long long added = 0;
19 |     for (int i = 1; i <= target; ++i)
20 |         added += diff[i];
21 |     cout << a[target] + added << '\n';
22 |     return 0;
23 | }
```

### 公开样例

样例 1 输入

```
3 2
1 2 3
1 3 1
1 2 1
2
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.5 | 总 252/526 阿正的快乐源泉 ( PID 1541 )

出处：2020级HAUT新生周赛（二）@杨哲专场（C，CID 1061）| 训练定位：入门 | 排序、二分与区间

标签：二分查找、模拟、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：68/138 ( 49.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在给出若干组阿正情绪指标取值范围的上界  $Rightmax$  和场合的合理指标  $H$ ，你的任务是算出每个场合中阿正通过药丸稳定情绪所花费的时间。

输入要点：多实例输入，每组数据占一行，包含两个正整数，分别代表阿正情绪指标的上界  $Rightmax$ ，一个场合的合理指标  $H$ 。

(  $0 < H < Rightmax \leq 100000$  )

输出要点：每组数据一行一个数代表阿正通过药丸稳定情绪所花费的时间。

题面提示：对第一组样例，阿正通过服用药丸恢复使情绪指标恢复稳定的过程为：

1. 服用一枚红色药丸， $Left=Mid=(100+0)/2=50$ ， $Right=100$ ；
2. 服用一枚红色药丸， $Left=Mid=(100+50)/2=75$ ， $Right=100$ ；
3. 服用一枚蓝色药丸， $Left=75$ ， $Right=Mid=(100+75)/2=87$ ；

至此，共花费了3个单位时间使得  $Mid=(Left+Right)/2=81$ ，与  $H=81$  相等。

对第二组样例，在不服用药丸时， $Mid=(Left+Right)/2=81=H$ ；

所以阿正不需要服用药丸，时间花费为0。

对第三组样例，阿正通过服用药丸恢复使情绪指标恢复稳定的过程为：

1. 服用一枚蓝色药丸， $Left=0$ ， $Right=Mid=(0+163)/2=81$ ；
2. 服用一枚蓝色药丸， $Left=0$ ， $Right=Mid=(0+81)/2=40$ ；
3. 服用一枚蓝色药丸， $Left=0$ ， $Right=Mid=(0+40)/2=20$ ；
4. 服用一枚蓝色药丸， $Left=0$ ， $Right=Mid=(0+20)/2=10$ ；
5. 服用一枚蓝色药丸， $Left=0$ ， $Right=Mid=(0+10)/2=5$ ；
6. 服用一枚红色药丸， $Left=Mid=(0+5)/2=2$ ， $Right=5$ ；

至此，共花费了6个单位时间使得  $Mid=(Left+Right) \dots$

## 零基础先修

- lower\_bound 找第一个不小于目标的位置，upper\_bound 找第一个大于目标的位置。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- 多组数据以 EOF 结束时使用 while (cin >> ...)，不要额外输出测试数据。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：每次服药都把当前闭区间的一端替换为中点，这正是二分查找。当前 mid 等于 H 时停止；H 更小就令 Right=mid，否则令 Left=mid。每次区间至少缩小一半，记录操作次数。多组数据读到文件尾。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

每组  $O(\log \text{Rightmax})$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- EOF 题不要先读不存在的 T，也不要再在循环结束后多输出内容。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int mx, target;
7 |     while (cin >> mx >> target) {
8 |         int left = 0, right = mx, steps = 0;
9 |         while ((left + right) / 2 != target) {
10 |             int middle = (left + right) / 2;
11 |             if (target < middle)
12 |                 right = middle;
13 |             else
14 |                 left = middle;
15 |             ++steps;
16 |         }
17 |         cout << steps << '\n';
18 |     }
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
100 81
162 81
163 3
```

样例 1 输出

```
3
0
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.6 | 总 253/526 最小的距离之和 ( PID 1248 )

出处：2016级河南工大新生周赛（七）（C，CID 1014）| 训练定位：基础提高 | 排序、二分与区间

标签：中位数、绝对值、排序 | 限制：1.000 S / 128 MB | 历史统计：40/171 ( 23.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：X轴上有N个点，求X轴上一点使它到这N个点的距离之和最小，输出这个最小的距离之和。

输入要点：第1行：点的数量N。( $2 \leq N \leq 10000$ )

第2 - N + 1行：点的位置。( $-10^9 \leq P[i] \leq 10^9$ )

输出要点：输出这个最小的距离之和。

题面提示：注意int的范围。。。

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：一维绝对距离和在中位数处最小。排序后选 `position[n/2]` 作为中位数（偶数个点时中间两点之间任意位置都同样最优），累加每个点与它的绝对距离。坐标和距离和可能超过 int，要用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> pos(n);
9 |     for (long long& x : pos)
10 |         cin >> x;
11 |     sort(pos.begin(), pos.end());
12 |     long long median = pos[n / 2];
13 |     long long ans = 0;
```

```

14 |     for (long long x : pos)
15 |         ans += llabs(x - median);
16 |     cout << ans << '\n';
17 |     return 0;
18 | }

```

## 公开样例

样例 1 输入

```

5
-1
-3
0
7
9

```

样例 1 输出

```

20

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.7 | 总 254/526 体育课 ( PID 1313 )

出处：2017级河南工大新生周赛 ( 一 ) ( F , CID 1028 ) | 训练定位：基础提高 | 排序、二分与区间

标签：排序、平均数、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：27/94 ( 28.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，输出一个浮点数，保留两位小数。

**输入要点：**第一行一个整数  $n$  ( $50 < n < 60$ )，接下来  $n$  个整数，代表每个同学的身高，身高数据保证合理(单位厘米)，但是数据无序。

**输出要点：**输出一个浮点数，保留两位小数。

题面提示：样例 1 中去掉的数据，211，211，163，162。

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 浮点题按误差要求输出足够位数，通常使用 fixed 与 setprecision。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：将身高排序，去掉最小两个和最大两个后，对中间  $n-4$  人求平均数。题目说“前两个最高、后两个最低”均按人数去除，即便最高值相同也仍只 去掉两个元素。使用 long long 求和、double 做除法，保留两位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 按题目上界估算中间量，相关变量不要退回 `int`。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> height(n);
9 |     for (int& x : height)
10 |         cin >> x;
11 |     sort(height.begin(), height.end());
12 |     long long sum = 0;
13 |     for (int i = 2; i + 2 < n; ++i)
14 |         sum += height[i];
15 |     cout << fixed << setprecision(2) << (double)sum / (n - 4) << '\n';
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
50
211 211 211 211 208 207 206 205 204 203 202 201 200 199 198 197 196 195 194 193 192 191 190 189 188 187
186 185 184 183 182 181 180 179 178 177 176 175 174 173 172 171 170 169 168 167 166 165 164 163
162
```

样例 1 输出

```
186.54
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.8 | 总 255/526 加密工作 (一) (PID 1360)

出处：2017级河南工大新生周赛（八）（E，CID 1037） | 训练定位：基础提高 | 排序、二分与区间

标签：字符串、排序、编码、结构体思想 | 限制：1.000 S / 128 MB | 历史统计：27/39 (69.2%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一串数字。每个字符要转换成 3 个整数：第一个整数：字符的类型，1 代表小写，2 代表大写。第二个整数：第三个数的长度。第三个整数：该字符转换成数字的值， $A = 1$   $a = 1$ ,  $B = 2$ ,  $b = 2 \dots Z = 26$ ,  $z = 26$ 。

输入要点：多实例测试，每行一个整数  $op$  ( $1 \leq op \leq 4$ )，和一个长度为 4 的字符串。

输出要点：一串数字。

每个字符要转换成 3 个整数：

第一个整数：字符的类型，1 代表小写，2 代表大写。

第二个整数：第三个数的长度。

第三个整数：该字符转换成数字的值，A = 1 a = 1, B = 2, b = 2 ... Z = 26, z = 26。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：先根据 op 调整四个字符的顺序：1 保持原序，2 完全逆序，3 按字母值降序，4 按字母值升序（排序比较时忽略大小写）。  
输出先写 op。每个字符随后编码为三段：小写类型 1/大写类型 2；字母值位数 1 或 2；字母值 A/a=1...Z/z=26。三段之间没有分隔符。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  额外空间（字符串长度固定为 4）

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int op;
7 |     string s;
8 |     while (cin >> op >> s) {
9 |         if (op == 2) {
10 |             reverse(s.begin(), s.end());
11 |         } else if (op == 3 || op == 4) {
12 |             sort(s.begin(), s.end(), [&](char a, char b) {
13 |                 int x = tolower((unsigned char)a) - 'a';
14 |                 int y = tolower((unsigned char)b) - 'a';
15 |                 return op == 3 ? x > y : x < y;
16 |             });
17 |         }
18 |         cout << op;
19 |         for (char c : s) {
20 |             int type = islower((unsigned char)c) ? 1 : 2;
21 |             int value = tolower((unsigned char)c) - 'a' + 1;
22 |             int len = value >= 10 ? 2 : 1;
23 |             cout << type << len << value;
24 |         }
25 |         cout << '\n';
26 |     }
27 |     return 0;
28 | }
```

公开样例

样例 1 输入

```
1 haut
2 haut
3 hAut
4 HAUT
```

样例 1 输出

```
111811112211220
212201221111118
312211220118211
421121822202221
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

4.9 | 总 256/526 小明的出题日 ( PID 1830 )

出处：2023级HAUT新生周赛 ( 二 ) @曹瑞峰专场 ( F , CID 1087 ) | 训练定位：基础提高 | 排序、二分与区间

标签：排序、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：10/29 ( 34.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：问题是：为了达到主办方的要求，小明最少要删除多少题目。

输入要点：第一行一个正整数 $n$ 和 $k$ ，表示小明准备的题目数量和连续题目之间的难度差值上限。 $(1 \leq n \leq 1e5, 1 \leq k \leq 1e9)$

第二行 $n$ 个正整数 $a_1, a_2, a_3 \dots a_n$ ，表示问题的难度。 $(1 \leq a[i] \leq 1e9)$ 。

输出要点：一个整数，表示最少需要删除的问题数。

零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：小明的出题日这是一个寻找连续子串的问题，先将题目难度按照从大到小进行排序，然后从第一位开始遍历，当 $a_i+1-a_i$ 的值小于等于 $k$ 时，不断增大当前连续子串长度的值，当 $a_i+1-a_i$ 的值大于 $k$ 时或者循环结束时，更新连续子串长度的最大值，再将当前连续子串长度归零。最后找到最长连续子串长度，用 $n$ 将其减去就得到了最少需要删除题目数量。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。



- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | int cmp(const void* a, const void* b) {
5 |     return *(int*)a - *(int*)b;
6 | }
7 | int arr[110000];
8 | int brr[110000];
9 | int main() {
10 |     int n, m;
11 |     scanf("%d %d", &n, &m);
12 |     for (int i = 0; i < n; ++i) {
13 |         scanf("%d", &arr[i]);
14 |     }
15 |     qsort(arr, n, sizeof(int), cmp);
16 |     for (int i = 1; i < n; ++i) {
17 |         brr[i] = arr[i] - arr[i - 1];
18 |     }
19 |     int ans = 1;
20 |     int l = 1;
21 |     for (int i = 1; i < n; ++i) {
22 |         if (arr[i] - arr[i - 1] > m) {
23 |             l = 1;
24 |         } else
25 |             l++;
26 |         if (l > ans)
27 |             ans = l;
28 |     }
29 |     printf("%d\n", n - ans);
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

```
5 1
1 2 4 5 6
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.10 | 总 257/526 AK路上的绊脚山 ( PID 1954 )

出处：河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇 ( D , CID 1105 ) | 训练定位：基础提高 | 排序、二分与区间

标签：二分答案、数组与序列、单调性 | 限制：1.000 S / 128 MB | 历史统计：39/164 ( 23.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：幸运的是，他可以使用魔术技巧删除每个山峰高度 $>h$ 的部分。这种魔术技巧的体力消耗与 $h$ 负相关，为节省体力，请你帮他确定越过山脉所需要的最大 $h$ 。

输入要点：输入共2行

第1行两个非负整数 $n, k$ , 表示 $n$ 座山与 $k(n \leq 1e4, k \leq 1e9)$

第2行有 $n$ 个整数 $a_1 \sim a_n$ , 分别表示第 $i$ 座山的高度, 每个输入之间存在一个空格。(  $a_i \leq 1e8$  )

输出要点：输出共1行, 请在一行中输出最大的 $h$ . ( $0 \leq h \leq \max(a)$ )

### 零基础先修

- 二分答案要求判定函数随答案单调；先写清可行区间与 true/false 的方向。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：给定截断高度  $h$ ，越过所有山所需体力为  $\sum(\min(a_i, h))$ 。这个值 随  $h$  单调不减，因此可二分最大的可行  $h$ ：若总和 $\leq k$ ，说明还能尝试 更高；否则降低上界。求和中一旦超过  $k$  就可提前停止并避免溢出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n \log \max(a_i))$  时间、 $O(n)$  空间

### 易错点

- 检查左右边界、循环条件和最终返回的是第一个可行还是最后一个不可行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long k;
8 |     cin >> n >> k;
9 |     vector<long long> height(n);
10 |     long long low = 0, high = 0;
11 |     for (long long& x : height) {
12 |         cin >> x;
13 |         high = max(high, x);
14 |     }
15 |     while (low < high) {
```

```

16 |         long long middle = low + (high - low + 1) / 2;
17 |         long long cost = 0;
18 |         for (long long x : height) {
19 |             cost += min(x, middle);
20 |             if (cost > k)
21 |                 break;
22 |         }
23 |         if (cost <= k)
24 |             low = middle;
25 |         else
26 |             high = middle - 1;
27 |     }
28 |     cout << low << '\n';
29 |     return 0;
30 | }

```

## 公开样例

样例 1 输入

```

3 5
29 2 14

```

样例 1 输出

```

1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.11 | 总 258/526 Simple学长数咖啡 ( PID 1237 )

出处：2016级河南工大新生周赛（六）（C，CID 1012）| 训练定位：进阶 | 排序、二分与区间

标签：前缀和、区间查询、格式化输出 | 限制：1.000 S / 128 MB | 历史统计：14/253 ( 5.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，输出有m行，每一行代表一次询问的结果。每一组数据后额外输出一个空行。

**输入要点：**输入一个T，表示有T组数据

每一个数据的第一行输入两个数，n，m，n表示有Simple学长有n个箱子，m表示有m次询问（n,m <= 100000）

接下来的一行有n个数，ai代表第i个箱子里的咖啡数(ai <= 100)

再接下来的m行每一行有两个数l,r，表示区间[l,r]（1<= l <= r <= n）

**输出要点：**输出有m行，每一行代表一次询问的结果。

每一组数据后额外输出一个空行。

### 零基础先修

- prefix[i] 表示前 i 项之和，区间 [l,r] 的和为 prefix[r]-prefix[l-1]。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3. 核心处理采用：建立前缀和  $\text{prefix}[i]$  = 前  $i$  个箱子的咖啡总数。任意闭区间  $[l, r]$  的答案就是  $\text{prefix}[r] - \text{prefix}[l-1]$ ，把每次  $O(n)$  累加降为  $O(1)$ 。题目特别要求每一大组数据后再输出一个空行，包括最后一组。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(n+m)$  时间、 $O(n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, queries;
10 |        cin >> n >> queries;
11 |        vector<long long> prefix(n + 1, 0);
12 |        for (int i = 1; i <= n; ++i) {
13 |            long long value;
14 |            cin >> value;
15 |            prefix[i] = prefix[i - 1] + value;
16 |        }
17 |        while (queries--) {
18 |            int left, right;
19 |            cin >> left >> right;
20 |            cout << prefix[right] - prefix[left - 1] << '\n';
21 |        }
22 |        cout << '\n';
23 |    }
24 |    return 0;
25 | }
```

## 公开样例

样例 1 输入

```
2
3 1
1 2 3
1 3
3 2
1 2 3
1 2
2 3
```

样例 1 输出

```
6
3
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.12 | 总 259/526 Simple学长发咖啡 ( PID 1240 )

出处：2016级河南工大新生周赛 ( 六 ) ( G , CID 1012 ) | 训练定位：进阶 | 排序、二分与区间

标签：排序、字符串、配对 | 限制：1.000 S / 128 MB | 历史统计：3/27 ( 11.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：大家都知道，Simple学长是一位非常慷慨大方的人，他总喜欢给他身边的人发福利。喏~，Simple学长准备了很多咖啡，要请他的朋友们喝（其实大家都知道，因为Simple学长只喝特定编号的咖啡，所以才把咖啡分给大家）。每一瓶咖啡都有他的编号，每一个人都有一个名字（这不是废话吗），然而Simple学长规定了按自己名字的字典序先后，依次选择一个最小编号的咖啡。

输入要点：输入一个T，表示有T组数据

每组数据第一行有一个数n，表示有n个人和n杯咖啡( $n \leq 1000$ )

接下来n行有n个名字（名字都是小写字母，且长度小于等于10）

再接下来n行有n个编号（在int范围内）

输出要点：按名字字典序从小到大输出n行

每行一个名字，后面跟他得到咖啡的编号

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：规则是按名字字典序依次取当前最小编号的咖啡。把所有名字升序排序，所有编号也升序排序，然后同一下标一一配对输出即可。名字和编号之间用一个空格，每人一行。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
```

```

10 |         cin >> n;
11 |         vector<string> name(n);
12 |         vector<int> coffee(n);
13 |         for (string& s : name)
14 |             cin >> s;
15 |         for (int& number : coffee)
16 |             cin >> number;
17 |         sort(name.begin(), name.end());
18 |         sort(coffee.begin(), coffee.end());
19 |         for (int i = 0; i < n; ++i)
20 |             cout << name[i] << ' ' << coffee[i] << '\n';
21 |     }
22 |     return 0;
23 | }

```

## 公开样例

### 样例 1 输入

```

1
5
sunnying
cds
yk
zy
ykc
1
2
3
4
5

```

### 样例 1 输出

```

cds 1
sunnying 2
yk 3
ykc 4
zy 5

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.13 | 总 260/526 Simple学长的数字游戏 ( PID 1252 )

出处：2016级河南工大新生周赛总决赛（C，CID 1016） | 训练定位：进阶 | 排序、二分与区间

标签：排序、自定义比较器、字符串拼接 | 限制：1.000 S / 128 MB | 历史统计：1/56 ( 1.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**比如  $n = 3$  时，123，23，312组成的最大编号就是31223123.

**输入要点：**输入一个  $T$ ，表示有  $T$  组数据

每组数据第一个数为  $n$ ，表示有  $n$  个多位数 ( $n \leq 1000$ )

接下来一行输入  $n$  个多位数，多位数的长度小于等于 30

**输出要点：**输出那个最大的多位数

## 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：决定两个字符串  $a$ 、 $b$  谁在前面时，不能只比首位或数值，而要比比较 拼接  $a+b$  与  $b+a$ ：若前者更大，就让  $a$  在前。按这个比较器排序后 依次连接，任意相邻对都选择了更优顺序，因此整体字典序（也就是同长度 十进制数值）最大。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(n \log n \cdot L)$  时间、 $O(nL)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        vector<string> number(n);
12 |        for (string& s : number)
13 |            cin >> s;
14 |        sort(number.begin(), number.end(), [](const string& a, const string& b) {
15 |            return a + b > b + a;
16 |        });
17 |        for (const string& s : number)
18 |            cout << s;
19 |        cout << '\n';
20 |    }
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
2
3
13 312 343
4
7 13 4 246
```

样例 1 输出

```
34331213
7424613
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.14 | 总 261/526 文物鉴定 ( PID 1402 )

出处：2018级河南工大新生周赛 ( 四 ) @杜锐专场 ( A , CID 1043 ) | 训练定位：进阶 | 排序、二分与区间

标签：排序、字符串解析、时间轴 | 限制：1.000 S / 128 MB | 历史统计：4/109 ( 3.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组输入对应一行输出，两个时间较早的年份在前，你只需要在年份数字后面加上AD或者BC即可。

输入要点：第一行是一个整数N (  $1 \leq N \leq 100$  )，其后有N组数据。

每组数据由6个字符组成，如2018AD ( 公元2018年 )，0450BC ( 公元前450年 )，故年份的范围为9999BC~9999AD，时间之间用确保没有完全相同的时间，用空格隔开。年份的总数目不超过20个

每组数据以一个“/”结束。

输出要点：每组输入对应一行输出，两个时间较早的年份在前，你只需要在年份数字后面加上AD或者BC即可。

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 解析字符串时先确定分隔符、字符类别和多位数边界，再逐段转换。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：把 AD 年记为正数、BC 年记为负数，按时间轴排序。最近的一对一定在排序后相邻位置中；依次比较间隔，严格更小时更新，间隔相同保留先遇到的更早一对。输出时取绝对值并还原 AD/BC，整数输出会自然去掉输入中的前导零。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(k \log k)$  时间、 $O(k)$  空间， $k \leq 20$

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int parseYear(const string& s) {
4 |     int year = stoi(s.substr(0, 4));
5 |     return s.substr(4) == "BC" ? -year : year;
6 | }
7 | string formatYear(int year) {
8 |     return to_string(abs(year)) + (year < 0 ? "BC" : "AD");
9 | }
10 | int main() {
11 |     int T;
12 |     cin >> T;
13 |     while (T--) {
14 |         vector<int> years;
15 |         string token;
16 |         while (cin >> token && token != "/")
17 |             years.push_back(parseYear(token));
18 |         sort(years.begin(), years.end());
19 |         int best = 0;
```



```

20 |         for (int i = 1; i + 0 < (int)years.size(); ++i) {
21 |             if (years[i] - years[i - 1] < years[best + 1] - years[best]) {
22 |                 best = i - 1;
23 |             }
24 |         }
25 |         cout << formatYear(years[best]) << ' ' << formatYear(years[best + 1]) << '\n';
26 |     }
27 |     return 0;
28 | }

```

## 公开样例

样例 1 输入

```

3
2018AD 2015AD 2016AD /
0230BC 0450AD 1500AD 3077AD /
0009AD 0010BC /

```

样例 1 输出

```

2015AD 2016AD
230BC 450AD
10BC 9AD

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.15 | 总 262/526 Invictus ( PID 1421 )

出处：2018级河南工大新生周赛总决赛（重现）（E，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（E，CID 1048） | 训练定位：进阶 | 排序、二分与区间

标签：ASCII、代码阅读、二分答案、边界处理 | 限制：1.000 S / 16 MB | 历史统计：36/213 ( 16.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，作为答案

输入要点：多组测试数据

每组输入两个整数A和B (  $|A|, |B| < 32768000$  )

输出要点：一个整数，作为答案

## 零基础先修

- 二分答案要求判定函数随答案单调；先写清可行区间与 true/false 的方向。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：先把题面中的十进制 ASCII 逐个转成字符，会得到一张代码图片的网址。图片里的 C 程序在区间  $[1, 10000]$  上二分第一个满足  $m - A \geq B$  的整数。因而答案就是把  $A+B$  限制在  $[1, 10000]$ ：小于 1 得 1，大于 10000 得 10000。题目为文件尾多组输入。手册同时展示解码链，避免把它误当成普通  $A+B$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 检查左右边界、循环条件和最终返回的是第一个可行还是最后一个不可行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long A, B;
7 |     while (cin >> A >> B) {
8 |         cout << min(10000LL, max(1LL, A + B)) << '\n';
9 |     }
10 |     return 0;
11 | }
```

## 公开样例

样例 1 输入

2 0

样例 1 输出

2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 4.16 | 总 263/526 干饭时间 ( PID 1592 )

出处：2021级HAUT新生周赛（三）@李晨曦专场（D，CID 1071）| 训练定位：进阶 | 排序、二分与区间

标签：排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：4/47 ( 8.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**请你计算，聪明的小R买完所有他想吃的东西，所用的最短时间。

**输入要点：**一行一个整数  $n$  ( $n \leq 1000$ )，代表小R想买的东西的个数。

$n$  个整数  $a_1$ ， $a_2$ ， $a_3 \dots a_n$ ，均不大于1000；

**输出要点：**一行一个整数，代表最短的时间。

## 零基础先修

- `sort` 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：干饭时间 简单贪心；每次只能去买两种东西，必须在第一种东西做好之前回来；则，每次选取得两个数字不能相等；最终结果等于每次选择的两个数字的最大值再加和；为了最后的时间加和最小，则需要让每两个时间中最小的那个时间值，尽可能的大；做法：先排序，每次选择两个不同的数，消耗的时间为较大的那个数值；
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | int a[1010], b[1010];
4 | void Sort(int a[], int n) {
5 |     for (int i = n - 1; i > 0; i--)
6 |         for (int j = 0; j < i; j++)
7 |             if (a[j] > a[j + 1]) {
8 |                 int t = a[j];
9 |                 a[j] = a[j + 1];
10 |                a[j + 1] = t;
11 |             }
12 | }
13 | int main() {
14 |     int n;
15 |     scanf("%d", &n);
16 |     for (int i = 0; i < n; i++)
17 |         scanf("%d", &a[i]);
18 |     Sort(a, n);
19 |     int ans = 0;
20 |     for (int i = n - 1; i >= 0; i--) {
21 |         if (b[i])
22 |             continue;
23 |         ans += a[i];
24 |         for (int j = i - 1; j >= 0; j--) {
25 |             if (!b[j] && a[j] < a[i]) {
26 |                 b[j] = 1;
27 |                 break;
28 |             }
29 |         }
30 |     }
31 |     printf("%d\n", ans);
32 |     return 0;
33 | }
```

## 公开样例

样例 1 输入

```
5
4 1 5 2 3
```

样例 1 输出

```
9
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.17 | 总 264/526 大海我来了 ( PID 1701 )

出处：2021级HAUT新生周赛（四）@王一杰专场（B，CID 1072）| 训练定位：进阶 | 排序、二分与区间

标签：字符串、排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：10/46 ( 21.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出 $n$ 行，第 $i$ 行是第 $i$ 个离开的船员的名字。

输入要点：第一行输入整数 $n(1 \leq n \leq 100)$ ；第2到 $n+1$ 行是每个人的名字和所属人群。名字由拉丁字母组成，第一个字母大写，其他字母都是小写，长度不大于10个字符。rat代表老鼠，woman代表女人，child代表小孩，man代表男人。

输出要点：输出 $n$ 行，第 $i$ 行是第 $i$ 个离开的船员的名字。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：大海我来了根据每个人所属的类别先后输出不同类的人名，可以按照题目要求的顺序，进行四次循环判断。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | struct po // 定义结构体存储每个人的名字和类别
6 | {
7 |     char name[19], rank[19];
8 | } per[109];
9 | int vis[109]; /* 定义一个记录输出情况的数组，在这里仅用这个数组判断哪个是未输出的船长。*/
10 | /* 我们也可以利用这个数组实现一种可能更简单的方法，在读入时将这四类人记为 1, 2, 3, 4，然后分别存入数组，之后每次判断输出时只需要用这个数组进行判断即可。*/
11 | int main() {
```

```

12 |     int n;
13 |     scanf("%d", &n);
14 |     for (int i = 1; i <= n; i++) {
15 |         scanf("%s %s", per[i].name, per[i].rank);
16 |     }
17 |     for (int i = 1; i <= n; i++) {
18 |         if (!strcmp(per[i].rank, "rat")) {
19 |             /*这里使用的是strcmp()函数，不懂使用方法可百度*/
20 |             vis[i] = 1; // 这个数组把已经输出的第i个人记为1，而其他未输出为0
21 |             printf("%s\n", per[i].name);
22 |         }
23 |     }
24 |     for (int i = 1; i <= n; i++) {
25 |         if (!strcmp(per[i].rank, "child") || !strcmp(per[i].rank, "woman")) {
26 |             vis[i] = 1;
27 |             printf("%s\n", per[i].name);
28 |         }
29 |     }
30 |     for (int i = 1; i <= n; i++) {
31 |         if (!strcmp(per[i].rank, "man")) {
32 |             vis[i] = 1;
33 |             printf("%s\n", per[i].name);
34 |         }
35 |     }
36 |     for (int i = 1; i <= n; i++) {
37 |         if (!vis[i]) { // 最后1个仍未输出的即是船长
38 |             printf("%s\n", per[i].name);
39 |             break;
40 |         }
41 |     }
42 |     return 0;
43 | }

```

## 公开样例

### 样例 1 输入

```

6
Chfyyds captain
Charlie man
Alice woman
Teddy rat
Bob child
Julia woman

```

### 样例 1 输出

```

Teddy
Alice
Bob
Julia
Charlie
Chfyyds

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.18 | 总 265/526 你又是个好人 ( PID 1723 )

出处：2021级HAUT新生周赛（六）@李志豪专场（F，CID 1074）| 训练定位：进阶 | 排序、二分与区间

标签：排序、二分答案、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/68 (2.9%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出m+1行，第i行输出一个正整数，代表第i次询问的结果。(1<= i <= m) 如果你觉得自己写对了的话，请在第m+1行输出一行，“Wo Tai Qiang Le” 如果你觉得自己写错了的话，请在第m+1行输出一行，“YOU ARE A GOOD MAN TOO”。

输入要点：第一行输入一个n代表给你的n个数( $1 \leq n \leq 1e5$ )

第二行输入一个m代表m次查询( $1 \leq m \leq 1e5$ )

接下来m行输入一个数t, ( $1 \leq t \leq 1e9$ )

输出要点：输出m+1行，第i行输出一个正整数，代表第i次询问的结果。( $1 \leq i \leq m$ )

如果你觉得自己写对了的话，请在第m+1行输出一行，“Wo Tai Qiang Le”

如果你觉得自己写错了的话，请在第m+1行输出一行，“YOU ARE A GOOD MAN TOO”。

## 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 二分答案要求判定函数随答案单调；先写清可行区间与 true/false 的方向。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：你又是个好人 标签：二分 mid题意是给你n个数，这n个数刚开始是乱序的。进行m次询问，问你询问的那个数是第几小的数。方法一：有n个数还要求第几小的话，我们可以先对他排序，对于m次查询，我们可以二分查找。n的范围是，所以排序的算法需要是 $n \log n$ 的，这里就推荐直接使用C++的sort函数，或者自己手写一个快速排序。二分查找也可以使用STL里面的lower\_bound函数，或者自己手写一个都行。e 5 1e5 1e5需要注意的是，这里面可能会出现重复的数，所以我们应该在排过序之后对数组进行去重，去重的话对C可能会麻烦一些，如果借助STL的一些函数就会方便很多。方法二：使用STL的map进行映射。//方法一
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\text{false})$

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 检查左右边界、循环条件和最终返回的是第一个可行还是最后一个不可行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | // 方法一：
2 | #include <bits/stdc++.h>
3 | using namespace std;
4 | #define ll long long
5 | #define endl "\n"
6 | const int max_n = 1e5 + 100;
7 | const int inf = 0x3f3f3f3f;
8 | const int mod = 1e9 + 7;
9 | vector<int> nums;
10 | int get(int x) {
11 |     return lower_bound(nums.begin(), nums.end(), x) - nums.begin() + 1;
12 | }
13 | int main() {
14 |     int n;
15 |     cin >> n;
16 |     int m;
```

```

17 |     cin >> m;
18 |     for (int i = 0; i < n; i++) {
19 |         int tmp;
20 |         cin >> tmp;
21 |         nums.push_back(tmp);
22 |     }
23 |     sort(nums.begin(), nums.end());
24 |     nums.erase(unique(nums.begin(), nums.end()), nums.end());
25 |     while (m--) {
26 |         int x;
27 |         cin >> x;
28 |         cout << get(x) << endl;
29 |     }
30 |     cout << "Wo Tai Qiang Le" << endl;
31 |     return 0;
32 | }

```

## 公开样例

样例 1 输入

```

6
3
123 321 1 3 2 1
3
2
321

```

样例 1 输出

```

3
2
5
Wo Tai Qiang Le

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.19 | 总 266/526 零食采购员小C ( PID 1748 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（G，CID 1077）| 训练定位：进阶 | 排序、二分与区间

标签：差分数组、前缀和、区间覆盖 | 限制：1.000 S / 128 MB | 历史统计：3/33 ( 9.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出一个整数 $x$ ，即小C给每个同学 $x$ 份零食时，才能使得班里快乐的人数最多。若出现多个 $x$ 值能使得班里快乐的人数最多，请输出其中最小的 $x$ 值

**输入要点：**第一行输入一个整数 $n(1 \leq n \leq 1e6)$ ，表示班里共有 $n$ 个同学

后面 $n$ 行，每行两个整数 $a, b$ （ $1 \leq a \leq b \leq 1e5$ ），表示班里每个同学希望得到的零食数量范围

**输出要点：**输出一个整数 $x$ ，即小C给每个同学 $x$ 份零食时，才能使得班里快乐的人数最多。

若出现多个 $x$ 值能使得班里快乐的人数最多，请输出其中最小的 $x$ 值

题面提示：样例中，如果每个人都分到3份零食，则班里所有人都会快乐，快乐的人数最多

## 零基础先修

- $\text{prefix}[i]$  表示前  $i$  项之和，区间  $[l, r]$  的和为  $\text{prefix}[r] - \text{prefix}[l - 1]$ 。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每名同学在整数区间  $[a, b]$  内都会快乐。用差分数组在  $a$  加一、 $b+1$  减一，做前缀和后  $\text{count}[x]$  就是统一发  $x$  份时的快乐人数。从小 到大扫描，只在人数严格增加时更新答案，自动保留并列时最小的  $x$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n+10^5)$  时间、 $O(10^5)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     const int LIMIT = 100000;
9 |     vector<int> diff(LIMIT + 2, 0);
10 |    while (n--) {
11 |        int left, right;
12 |        cin >> left >> right;
13 |        ++diff[left];
14 |        --diff[right + 1];
15 |    }
16 |    int cur = 0, bestCount = -1, ans = 1;
17 |    for (int value = 1; value <= LIMIT; ++value) {
18 |        cur += diff[value];
19 |        if (cur > bestCount) {
20 |            bestCount = cur;
21 |            ans = value;
22 |        }
23 |    }
24 |    cout << ans << '\n';
25 |    return 0;
26 | }
```

## 公开样例

样例 1 输入

```
6
15
24
17
13
33
26
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.20 | 总 267/526 工作罢了 (PID 1755)

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（B，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（B，CID 1079）| 训练定位：进阶 | 排序、二分与区间

标签：固定输出与格式、排序 | 限制：1.000 S / 128 MB | 历史统计：339/1707 ( 19.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一共两行 第一行输出排名第一的总成绩 (如果有相同的情况，优先输出先输入的) 第二行按顺序输出所有人的语文、数学、英语、理综平均成绩，以空格分隔，保留2位小数

输入要点：输入共  $n+1$  行

第一行输入一个整数  $n$  代表有  $n$  名同学

接下来  $n$  行每行输入 4 个整数分别代表每个人的语文，数学，英语，理综成绩

输出要点：输出一共两行

第一行输出排名第一的总成绩 (如果有相同的情况，优先输出先输入的)

第二行按顺序输出所有人的语文、数学、英语、理综平均成绩，以空格分隔，保留2位小数

题面提示： $3 \leq n \leq 1000$

注意题目要求的排名方式  $\psi(\dots)$

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- `sort` 默认排序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：工作罢了 只需要找到数学成绩最高并且输入最早的，并且再计算四项平均成绩即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     double ch, ma, en, sc;
6 |     double s_ch = 0, s_ma = 0, s_en = 0, s_sc = 0;
7 |     int max_sum = 0, max_math = 0;
8 |     for (int i = 0; i < n; i++) {
9 |         scanf("%lf%lf%lf%lf", &ch, &ma, &en, &sc);
10 |         if (ma > max_math) {
11 |             max_math = ma;
12 |             max_sum = ch + ma + en + sc;
13 |         }
14 |         s_ch += ch, s_ma += ma, s_en += en, s_sc += sc;
15 |     }
16 |     printf("%d\n", max_sum);
17 |     printf("%.2f %.2f %.2f %.2f", s_ch / n, s_ma / n, s_en / n, s_sc / n);
18 |     return 0;
19 | }
```

## 公开样例

样例 1 输入

```
5
10 20 30 40
20 30 50 60
60 90 110 210
10 30 50 40
51 40 40 50
```

样例 1 输出

```
470
30.20 42.00 56.00 80.00
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.21 | 总 268/526 这就是绝交证明书 ( PID 1763 )

出处：2022级HAUT新生周赛 ( 二 ) @武其轩专场 ( B , CID 1080 ) | 训练定位：进阶 | 排序、二分与区间

标签：前缀和与差分、数组与序列、快速输入输出 | 限制：1.000 S / 128 MB | 历史统计：49/333 ( 14.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，共 $t$ 行，每行输出一个答案，即从第 $l$ 级到第 $r$ 级阶梯上共写有多少个名字。

**输入要点：**第一行一个整数 $n(1 \leq n \leq 100000)$ ，表示楼梯的级数。

第二行 $n$ 个不大于1000的自然数，表示从1到 $n$ 级每级阶梯上名字的个数。

第三行一个整数 $t(1 \leq t \leq 100000)$ ，表示询问次数。接下来 $t$ 行，每行包含两个整数 $l$ 和 $r$  (  $1 \leq l \leq r \leq n$  )。

**输出要点：**共 $t$ 行，每行输出一个答案，即从第 $l$ 级到第 $r$ 级阶梯上共写有多少个名字。

题面提示：请使用较快的输入输出

## 零基础先修

- 前缀和与让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：建立前缀和  $\text{prefix}[i]$ ，表示第 1 级到第  $i$  级楼梯的名字总数。区间  $[l, r]$  的和等于  $\text{prefix}[r] - \text{prefix}[l-1]$ ，这样每次询问只做两次数组访问和一次减法。总和最大约  $1e8$ ， $\text{int}$  足够，使用  $\text{long long}$  也能避免以后改范围时溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

预处理  $O(n)$ ，每次询问  $O(1)$ ，空间  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> prefix(n + 1, 0);
9 |     for (int i = 1; i <= n; ++i) {
10 |         long long x;
11 |         cin >> x;
12 |         prefix[i] = prefix[i - 1] + x;
13 |     }
14 |     int q;
15 |     cin >> q;
16 |     while (q--) {
17 |         int l, r;
18 |         cin >> l >> r;
19 |         cout << prefix[r] - prefix[l - 1] << '\n';
20 |     }
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

```
5
1 2 3 4 5
2
1 3
2 4
```

样例 1 输出

```
6
9
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 4.22 | 总 269/526 我愿付出一切，只为再次看到你的笑容 ( PID 1770 )

出处：2022级HAUT新生周赛（三）@焦小航专场（A，CID 1081）| 训练定位：进阶 | 排序、二分与区间

标签：字符串、排序、字符处理 | 限制：1.000 S / 128 MB | 历史统计：159/723 ( 22.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，判断是否"haut"四个字母出现且仅出现一次。如果是，则输出"YeS"，否则输出"No"。（输出不包括""）**注意：**符合要求的字符串不一定是"haut"，还可能是"Haut"，即不区分大小写，大小写字母视为同一个字母。

**输入要点：**第一行给出  $t$  (  $1 \leq t \leq 1e3$  )

接下来  $t$  行，每行包括一个长度为四的字符串。

**输出要点：**判断是否"haut"四个字母出现且仅出现一次。如果是，则输出"YeS"，否则输出"No"。（输出不包括""）

**注意：**符合要求的字符串不一定是"haut"，还可能是"Haut"，即不区分大小写，大小写字母视为同一个字母。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：先把四个字符统一转成小写，再排序。若四个字符恰好各出现一次 h、a、u、t，排序结果就必然等于 ahtu；反之一定不相等。注意目标串也必须写成排序后的形式，而不是原顺序 haut。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         string s;
10 |         cin >> s;
```

```

11 |         for (char& c : s)
12 |             c = static_cast<char>(tolower((unsigned char)c));
13 |         sort(s.begin(), s.end());
14 |         cout << (s == "ahtu" ? "YeS" : "No") << '\n';
15 |     }
16 |     return 0;
17 | }

```

## 公开样例

样例 1 输入

```

2
Hauz
hauz

```

样例 1 输出

```

No
No

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 4.23 | 总 270/526 最大值 ( PID 1783 )

出处：2022级HAUT新生周赛（四）@张子豪专场（F，CID 1082）| 训练定位：进阶 | 排序、二分与区间

标签：排序、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：9/63（14.3%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：zzh可以按照任意种顺序释放技能。请你求出zzh所能造成伤害的最大值。

输入要点：第一行，一个正整数 $n$ ，技能的数量

第二行， $n$ 个整数（0或1，0代表技能是火属性，1代表技能是冰属性），每个技能的属性

第三行， $n$ 个整数，每个技能造成的伤害

输出要点：输出一个数字，zzh所能造成伤害的最大值

题面提示：样例说明：

在测试用例中，我们可以按[3,1,4,2]对技能进行排序，总伤害为 $100+2\times 1+2\times 1000+10=2112$ 。（第三个技能先释放，造成 $1\times 100$ 点伤害，接着释放第一个技能，触发翻倍造成 $2\times 1$ 点伤害，接着释放第4个技能，造成 $2\times 1000$ 点伤害，最后释放第二个技能，造成 $1\times 10$ 点伤害）

数据范围：

$1 \leq n \leq 1000$ ;

$0 \leq$  每个技能造成的伤害  $\leq 1e9$

### 零基础先修

- sort 默认排序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：最大值：分析：如果火技能的数量与冰技能的数量相等，则除初始伤害最小的技能外，所有技能的伤害加倍。否则，设k为两种技能数量的最小值，然后将每种技能中初始伤害最大的k个技能的伤害加倍。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int t[1005], fire[1005], ice[1005], fl, il;
4 | long long ans;
5 | void sort(int a[], int n) {
6 |     for (int i = 1; i <= n; i++) {
7 |         for (int j = i + 1; j <= n; j++) {
8 |             if (a[i] < a[j]) {
9 |                 int t = a[i];
10 |                 a[i] = a[j];
11 |                 a[j] = t;
12 |             }
13 |         }
14 |     }
15 | }
16 | int main() {
17 |     int n;
18 |     scanf("%d", &n);
19 |     for (int i = 1; i <= n; i++)
20 |         scanf("%d", &t[i]);
21 |     for (int i = 1; i <= n; i++) {
22 |         int x;
23 |         scanf("%d", &x);
24 |         if (t[i]) {
25 |             ice[++il] = x;
26 |         } else {
27 |             fire[++fl] = x;
28 |         }
29 |         ans += x;
30 |     }
31 |     sort(ice, il);
32 |     sort(fire, fl);
33 |     int temp = il < fl ? il : fl;
34 |     for (int i = 1; i <= temp; i++) {
35 |         ans += fire[i] + ice[i];
36 |     }
37 |     if (fl == il) {
38 |         ans -= ice[il] < fire[fl] ? ice[il] : fire[fl];
39 |     }
40 |     printf("%lld", ans);
41 |     return 0;
42 | }
```

## 公开样例

样例 1 输入

```
4
0 1 1 1
1 10 100 1000
```

样例 1 输出

```
2112
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.24 | 总 271/526 小明的打怪游戏 ( PID 1829 )

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（E，CID 1091）；2023级HAUT新生周赛（二）@曹瑞峰专场（E，CID 1087）| 训练定位：进阶 | 排序、二分与区间

标签：排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：20/209（9.6%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：小明正在电脑上玩一款打怪游戏，一共有 $n$ 只怪物，编号从1到 $n$ ，第 $i$ 只怪物的血量为 $a[i]$ 。小明的角色攻击力为 $k$ ，对怪物进行攻击时，怪物的血量减少 $k$ ，当怪物的血量小于等于零时，怪物死亡。但是他每次只能攻击血量最高的怪物，如果几只怪物血量相同，则攻击编号小的那个。你的目标是求出怪物死亡的顺序。

输入要点：第一行两个整数 $n$ 和 $k$ 。 $(1 \leq n \leq 100000, 1 \leq k \leq 1e9)$ 表示怪物的数量和角色攻击力。

第二行 $n$ 个正整数 $a_1, a_2, \dots, a_n$ 。表示 $n$ 个怪物的血量 $(1 \leq a[i] \leq 1e9)$ 。

输出要点：一行，按怪物死亡顺序排列的编号。

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
- 核心处理采用：小明的打怪游戏 由于先攻击血量高的怪物，所以在读取时可以先对血量 $a_i$ 大于 $k$ 的值对 $k$ 取模。取模后这个题就变成了一个排序问题，将血量高的排在前面，因为优先攻击血量高的怪物，血量相同时，按找下标排序，将下表较小的放在前面。就可以得到怪物死亡顺序了。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 1e5 + 10;
4 | struct xp {
5 |     int i, k;
6 | } a[N];
7 | bool cmp(xp a, xp b) {
8 |     if (a.k != b.k) {
9 |         return a.k > b.k;
10 |    }
11 |    return a.i < b.i;
12 | }
13 | int main() {
14 |     int n, k;
15 |     cin >> n >> k;
16 |     for (int i = 1; i <= n; i++) {
17 |         a[i].i = i;
18 |         cin >> a[i].k;
19 |         if (a[i].k % k == 0)
20 |             a[i].k = k;
21 |         else
22 |             a[i].k %= k;
23 |     }
24 |     sort(a + 1, a + n + 1, cmp);
25 |     for (int i = 1; i <= n; i++) {
26 |         cout << a[i].i << ' ';
27 |     }
28 |     return 0;
29 | }
```

## 公开样例

样例 1 输入

```
3 2
1 2 3
```

样例 1 输出

```
2 1 3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 4.25 | 总 272/526 是心碎的声音 (PID 1859)

出处：2023级HAUT新生周赛（五）@陈兰锴专场（G，CID 1090）| 训练定位：进阶 | 排序、二分与区间

标签：排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：1/24 (4.2%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数 表示收集的原石个数

输入要点：第一行



输入两个整数  $n, k$

第二行

$n$  个整数  $a_1 - a_n$

输出要点：一个整数 表示收集的原石个数

题面提示：提示  $1 \leq n \leq 1000000, 1 \leq k \leq n, 1 \leq a_i \leq 1e9$ .

样例解释

1 5 3 4 2

第一次选取 2

能量衰减为：0 2 1 1 1

第二次选取 1

能量衰减为：0 1 0 0 0

第三次选取 1

能量衰减为：0 0 0 0 0

没有可选择的

## 零基础先修

- `sort` 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：是心碎的声音 中等题 首先如果  $k \leq 2$ ，所有原石都不会消失，直接输出 其次 将 原石从小到大排序，如果小的能拿，一定优先拿小的，这将使用一次机会，最后计数即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n \log n)$

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 确认 0/1 起下标、首尾元素和  $n-1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | void solve() {
5 |     int n, k;
```

```

6 |     cin >> n >> k;
7 |     vector<int> a(n);
8 |     for (int i = 0; i < n; ++i) {
9 |         cin >> a[i];
10 |     }
11 |     if (k <= 2) {
12 |         cout << n << "\n";
13 |         return;
14 |     }
15 |     sort(a.begin(), a.end()); // 排序
16 |     ll ans = 0;
17 |     ll time = 0;
18 |     for (int i = 0; i < n; ++i) {
19 |         ll op = 0;
20 |         while (1) {
21 |             a[i] = (1.0 * a[i] / k + 0.5);
22 |             op++;
23 |             if (op >= time + 1)
24 |                 break;
25 |             if (a[i] == 0)
26 |                 break;
27 |         }
28 |         if (op >= time + 1) {
29 |             time++;
30 |             ans++;
31 |         }
32 |     }
33 |     cout << ans << "\n";
34 | }
35 | int main() {
36 |     ios::sync_with_stdio(false);
37 |     cin.tie(0);
38 |     cout.tie(0);
39 |     int test = 1;
40 |     // cin >> test;
41 |     while (test--) {
42 |         solve();
43 |     }
44 |     return 0;
45 | }

```

## 公开样例

样例 1 输入

```

5 3
1 5 3 4 2

```

样例 1 输出

```

3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 4.26 | 总 273/526 神奇的数字 ( PID 1895 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭钰 ( I , CID 1095 ) | 训练定位：进阶 | 排序、二分与区间

标签：排序 | 限制：1.000 S / 128 MB | 历史统计：33/156 ( 21.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对于每个询问输出一行，该行包含一个整数，表示  $1 \sim n$  中“神奇的数字”个数

输入要点：第一行包含一个数  $T$  (  $1 \leq T \leq 1000$  )，表示询问数

接下来  $T$  行，每行包含一个整数  $n$  ( $1 \leq n \leq 1000$ )；

输出要点：对于每个询问输出一行，该行包含一个整数，表示  $1 \sim n$  中“神奇的数字”个数

题面提示：请认真阅读题干，其实本题才是签到题

## 零基础先修

- `sort` 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：神奇的数字“个数” `#include <bits/stdc++.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int T;
4 | int main() {
5 |     cin >> T;
6 |     while (T--) {
7 |         int n;
8 |         cin >> n;
9 |         cout << n << endl;
10 |     }
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

```
1
1
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“排序、二分与区间”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 5 | 数学、数论与组合

先从定义和小数据找规律，再用整除、奇偶、最大公约数、取模或组合计数证明公式。特别注意整数溢出与浮点误差。

本模块共 94 道唯一题。

## 5.1 | 总 274/526 最大公约数 ( PID 1217 )

出处：2016级河南工大新生周赛 ( 三 ) ( D , CID 1008 ) | 训练定位：入门 | 数学、数论与组合

标签：最大公约数、欧几里得算法、模运算 | 限制：1.000 S / 128 MB | 历史统计：243/399 ( 60.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给出两个数a和b，输出它们的最大公约数。

输入要点：输入一行a和b， $0 < a, b < 10^9$ 。

输出要点：输出它们的最大公约数。

### 零基础先修

- `std::gcd(a,b)` 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：欧几里得算法利用  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。不断把较大问题替换为 余数问题，直到  $b=0$ ，此时  $a$  就是最大公约数。  
C++17 的 `std::gcd` 已实现同一算法。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(\log \min(a,b))$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long a, b;
5 |     cin >> a >> b;
6 |     while (b != 0) {
7 |         long long rem = a % b;
8 |         a = b;
9 |         b = rem;
10 |    }
11 |    cout << a << '\n';
12 |    return 0;
13 | }
```

### 公开样例

样例 1 输入

24 36

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.2 | 总 275/526 Choice学姐买糖果 II ( PID 1320 )

出处：2017级河南工大新生周赛 ( 二 ) ( B , CID 1029 ) | 训练定位：入门 | 数学、数论与组合

标签：循环、格式化输出、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：185/337 ( 54.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出Choice学姐心里所想的，占一行(不要输出多余的空格)。

输入要点：第一行输入一个整数 $n$  ( $1 \leq n \leq 100$ ),代表有 $n$ 种种糖果。

输出要点：输出Choice学姐心里所想的，占一行(不要输出多余的空格)。

### 零基础先修

- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：共输出  $n$  个短语，第 1、3、5...项是 `I love`，第 2、4、6...项是 `I hate`。相邻项之间输出 ` or `，最后一项后输出 ` it`。把分隔符放在 第 2 项及以后之前，可避免首尾多余空格。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
```

```

6 |     for (int i = 1; i <= n; ++i) {
7 |         if (i > 1)
8 |             cout << " or ";
9 |         cout << (i % 2 == 1 ? "I love" : "I hate");
10 |    }
11 |    cout << " it\n";
12 |    return 0;
13 | }

```

## 公开样例

样例 1 输入

3

样例 1 输出

I love or I hate or I love it

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.3 | 总 276/526 超超超超超越数 ( PID 1391 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( D, CID 1041 ) | 训练定位：入门 | 数学、数论与组合

标签：数论、阶乘、观察 | 限制：1.000 S / 128 MB | 历史统计：68/135 ( 50.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，一个数字，0或1，即刘维尔数小数点后第n位的数字

**输入要点：**一个正整数n，表示刘维尔数小数点后第n位 (0 < n <= 1000 000 000)

**输出要点：**一个数字，0或1，即刘维尔数小数点后第n位的数字

题面提示：“超越数，那些1844年之前甚至不能证明其存在性的数，其实占了实数的绝大部分。事实上，它们几乎占满了所有的实数。

千百年来，我们对于数的概念完全是偏颇和扭曲的。我们总是重视整洁、秩序、模式，然而我们又生活在一个折中与近似的世界中。我们只关注那些对我们有意义的数字。为了数农场里的动物，我们发明了自然数；为了测量，我们发明了有理数；而在高等数学中，我们又发明了代数数。我们从连续统中挖出了所有这些数，却完全无视了实数海洋中其他有如微生物一般繁多的数。我们活在一种很安逸的幻觉中：有理数比无理数多得多，代数数比超越数多得多，当然这仅仅是我们的一厢情愿。然而事实上，在连续统的世界中几乎每一个数都是超越数。”

——《图灵的秘密》

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：刘维尔数在小数点后 k! 位放 1，其余位为 0。n≤1e9，只需从 1! 开始不断乘下一个整数，若命中 n 输出 1，超过 n 则输出 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。

- 因此所得数值正是题目定义的答案。

## 复杂度

$O(\log n)$  级循环、 $O(1)$  空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, factorial = 1;
5 |     cin >> n;
6 |     bool isOne = false;
7 |     for (long long k = 1; factorial <= n; ++k) {
8 |         if (factorial == n) {
9 |             isOne = true;
10 |            break;
11 |        }
12 |        if (factorial > n / (k + 1))
13 |            break;
14 |        factorial *= k + 1;
15 |    }
16 |    cout << (isOne ? 1 : 0) << '\n';
17 |    return 0;
18 | }
```

## 公开样例

样例 1 输入

2

样例 1 输出

1

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.4 | 总 277/526 签到题 ( PID 1428 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( A , CID 1047 ) | 训练定位：入门 | 数学、数论与组合

标签：二分查找、位运算、对数 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在B想知道自己一定能够猜中key的最小的猜次数(猜中：B必须说出：key与k相等)。

输入要点：多样例测试

第一行输出一个T表示样例数 ( $1 \leq T \leq 10000$ )

接下来的T行每行输入一个n ( $1 \leq n \leq 10^{15}$  注意数据范围!!!)

输出要点：对于每一个n, 输出B一定能够猜中这个数字的最小的猜次数。

## 零基础先修

- lower\_bound 找第一个不小于目标的位置，upper\_bound 找第一个大于目标的位置。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：一次比较可按二分排除约一半候选。区间含  $n$  个数时，最坏猜测次数是  $\text{floor}(\log_2 n) + 1$ ，也就是  $n$  的二进制位数；用循环右移避免浮点  $\log$  在 2 的幂附近产生误差。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(\log n)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int T;
5 |     cin >> T;
6 |     while (T--) {
7 |         unsigned long long n;
8 |         cin >> n;
9 |         int ans = 0;
10 |         do {
11 |             ++ans;
12 |             n >>= 1;
13 |         } while (n);
14 |         cout << ans << '\n';
15 |     }
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
1
2
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)



## 5.5 | 总 278/526 cxx 的苹果总数 ( PID 1476 )

出处：2019 级河南工大新生周赛 ( 一 ) @陶福专场 ( B , CID 1053 ) | 训练定位：入门 | 数学、数论与组合

标签：高精度、字符串加法、累加 | 限制：1.000 S / 128 MB | 历史统计：256/519 ( 49.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：cxx 的苹果总数

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

题面提示：注意数据范围

使用 long long 读入数据是用 %lld

### 零基础先修

- 超过 64 位范围时用字符串逐位运算，或使用题目允许的大整数工具。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每筐苹果数可达  $1e18$ 、筐数近一万，总和可能超过 64 位。用十进制 字符串保存累计和，从末位向前逐位相加并处理进位，就不依赖非标准大数 库。输入中的每个  $a_i$  也按字符串读取。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(nD)$  时间、 $O(D)$  空间， $D$  为答案位数

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | string add(string a, string b) {
4 |     int i = (int)a.size() - 1, j = (int)b.size() - 1, carry = 0;
5 |     string res;
6 |     while (i >= 0 || j >= 0 || carry) {
7 |         int x = i >= 0 ? a[i--] - '0' : 0;
8 |         int y = j >= 0 ? b[j--] - '0' : 0;
9 |         int sum = x + y + carry;
10 |         res.push_back(char('0' + sum % 10));
11 |         carry = sum / 10;
12 |     }
13 |     reverse(res.begin(), res.end());
14 |     return res;
15 | }
16 | int main() {
17 |     ios::sync_with_stdio(false);
18 |     cin.tie(nullptr);
19 |     int n;
20 |     cin >> n;
21 |     string total = "0";
22 |     while (n--) {
23 |         string apples;
```

```
24 |         cin >> apples;
25 |         total = add(total, apples);
26 |     }
27 |     cout << total << '\n';
28 |     return 0;
29 | }
```

## 公开样例

样例 1 输入

```
4
1 2 3 4
```

样例 1 输出

```
10
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.6 | 总 279/526 小青的报道 ( PID 1529 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（B，CID 1060）| 训练定位：入门 | 数学、数论与组合

标签：数论、质因数分解、组合计数 | 限制：1.000 S / 128 MB | 历史统计：340/526 ( 64.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个正整数，表示问题的答案。

输入要点：一个正整数 $n$  ( $1 \leq n \leq 10$ )

输出要点：一个正整数，表示问题的答案。

题面提示： $4! = 24$ ，显然仅有 1,2,3,4,6,8,12,24 为 24 的正因数，所以我们输出 8 就好了

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：若  $n!$  的质因数分解为各质数  $p$  的  $ep$  次方，正因子个数就是  $\prod (ep+1)$ 。 $n \leq 10$ ，可逐个分解  $2..n$  并累计每个质因数的指数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(n\sqrt{n})$  时间、 $O(\text{质数个数})$  空间

### 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     map<int, int> exponent;
7 |     for (int value = 2; value <= n; ++value) {
8 |         int x = value;
9 |         for (int p = 2; p * p <= x; ++p) {
10 |             while (x % p == 0) {
11 |                 ++exponent[p];
12 |                 x /= p;
13 |             }
14 |         }
15 |         if (x > 1)
16 |             ++exponent[x];
17 |     }
18 |     int ans = 1;
19 |     for (auto [prime, power] : exponent)
20 |         ans *= power + 1;
21 |     cout << ans << '\n';
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

4

样例 1 输出

8

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 5.7 | 总 280/526 递归式 ( PID 1547 )

出处：2020级HAUT新生周赛（三）@张雍舜专场（A，Cid 1062）| 训练定位：入门 | 数学、数论与组合

标签：数学、递推、周期性 | 限制：1.000 S / 128 MB | 历史统计：80/163 ( 49.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，T行,每行一个实数,答案保留三位小数

输入要点：第一行为T,代表有T组数据

接下来T行, 每行 两个实数a,b 和一个整数n  
接下来T行,每行两个实数a , b和一个整数n

$1 \leq T \leq 1e5$

$1 \leq a,b,n \leq 1e5$

输出要点：T行,每行一个实数,答案保留三位小数

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：按定义先算  $f_0=a$ 、 $f_1=b$ 。把递推式连续展开可发现  $f_5=a$ 、 $f_6=b$ ，因而整个序列以 5 为周期；每组只需算  $f_0..f_4$ ，再取  $n\%5$  对应项。这也是  $T$  和  $n$  都可达  $1e5$  时不能逐项递推的关键。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 `int`。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     cout << fixed << setprecision(3);
9 |     while (T--) {
10 |         double a, b;
11 |         long long n;
12 |         cin >> a >> b >> n;
13 |         double f[5];
14 |         f[0] = a;
15 |         f[1] = b;
16 |         for (int i = 2; i < 5; ++i)
17 |             f[i] = (1.0 * f[i - 1]) / f[i - 2];
18 |         cout << f[n % 5] << '\n';
19 |     }
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
10
11.552433642296053 9.84536372979228 21
9.127610802358296 11.061421019305289 19
5.352034655215464 11.492644371125753 22
1.035865637269147 7.451481779826788 18
9.97708237621654 4.398241324723653 16
10.916270799722476 7.38201491477216 23
9.711021627018521 7.311157633768417 23
10.550401414216932 5.1938809719988175 25
9.997828367794625 10.951453033276481 16
3.806909182276062 4.780967251081492 18
```

样例 1 输出

```
9.845
0.916
2.334
1.229
4.398
0.239
0.254
```

10.550  
10.951  
0.527

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 5.8 | 总 281/526 Isjp和他的小伙伴们 ( PID 1712 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（D，CID 1073）| 训练定位：入门 | 数学、数论与组合

标签：固定输出与格式、数学 | 限制：1.000 S / 128 MB | 历史统计：174/319 ( 54.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：经过百天的奋战，21年的高考算是圆满结束了，对于生活十分有仪式感的Isjp来说来一场说走就走的旅行最合适不过了，他和他的家人们率先来了场美妙的旅行。又过了不久就开学了，Isjp在haut结识了不少新的伙伴，其中的两个就是lwjq和lcby，因此他们三个也计划着在中秋节或者国庆节来一场说走就走的旅行，就在这个时候有个问题就出现了，三人都有不同的想去的地方，Is.....

输入要点：输入一行两个整数为lwjq和lcby率先掷出的骰子的点数

输出要点：以不可约分数形式输出Isjp获胜的概率，输出格式为a/b,如果概率为0则输出0/1;如果概率为1则输出1/1。

题面提示：Isjp掷出4、5、6均可获胜，故获胜概率是1/2。

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：Isjp和他的小伙伴们题意理解：这道题目主要就是概率和最大公约数的相关知识点，只要找出大于等于前两位点数的情况再约分即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int gcd(int x, int y) // 最大公约数用于约分
3 | {
4 |     if (x % y == 0)
5 |         return y;
6 |     else
7 |         return gcd(y, x % y);
8 | }
9 | int main() {
10 |     int a, b;
11 |     scanf("%d %d", &a, &b);
12 |     int maxx = a > b ? a : b; // 求a,b中的最大的那个
13 |     int ans = 6 - maxx + 1;
14 |     int x = gcd(ans, 6);
15 |     printf("%d/%d\n", ans / x, 6 / x);
16 | }
```

## 公开样例

样例 1 输入

4 2

样例 1 输出

1/2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.9 | 总 282/526 已经没有什么好怕的了 ( PID 1736 )

出处：2021级HAUT新生周赛（八）@孔维飒专场（C，CID 1076）| 训练定位：入门 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：20/32 ( 62.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出 $m$ 行，每行输出一个查询的结果（有多少对学生的亲密值等于 $x$ ）。

输入要点：第一行两个整数 $n, m$ ， $n$ 代表 $n$ 个学生， $m$ 代表 $m$ 个询问

第二行有 $n$ 个整数 $w_i$  ( $1 \leq i \leq n$ )， $w_i$  代表第 $i$ 个学生的心理状态

第三行有 $m$ 个整数 $x$ 代表 $m$ 个询问

数据范围： $1 \leq n \leq 1000, 1 \leq m \leq 10, 0 \leq w_i \leq 214, 0 \leq x \leq 14$ 。

输出要点：输出 $m$ 行，每行输出一个查询的结果（有多少对学生的亲密值等于 $x$ ）。

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：已经没有什么好怕的了(传送门)题目大意，就是找到异或值中二进制中1的个数等于询问个数的对的值。由于数据量比较小，可以保留找。保留枚举所有的对(由于这里说i,j和j,i是同一个对，那么你枚举的时候第二个循环就可以写成j = i + 1开始，这样保证j > i那么就保证不会重复枚举了，，，，如果你是重复枚举的化，统计结果/2也是可以的。)那么接下来就是如何求出当前数的二进制中1的个数(1).可以使用位运算  $x \gg i \& 1$  ,就可以来判断x这个数第i位是不是1(不会位运算参考：博客中四右移运算符的应用，三取第k位的值)(2).可以将十进制转化位二进制，然后自然得到二进制中1的个数，oj有将十进制转化位二进制的原题传送门时间复杂度  $O(n^2)O(n^2)O(n^2)$  参考代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n^2)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和 n=1。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[1100];
3 | int get_count_1(
4 |     int x) { // 得到x中1的数量的函数，这里我采用位运算方式，你也可以使用转化为二进制的方式来求
5 |     int cnt = 0;
6 |     for (int i = 0; i <= 15; i++) { // 只需要找到15即可，因为小于2的14次方
7 |         if (x >> i & 1)
8 |             cnt++; // 判断第i位是不是1
9 |     }
10 |    return cnt;
11 | }
12 | int main() {
13 |     int n, m;
14 |     scanf("%d%d", &n, &m);
15 |     for (int i = 0; i < n; i++)
16 |         scanf("%d", &a[i]);
17 |     while (m--) {
18 |         int t;
19 |         scanf("%d", &t);
20 |         int cnt = 0;
21 |         for (int i = 0; i < n; i++) {
22 |             for (int j = i + 1; j < n;
23 |                 j++) { // 枚举对，这里j = i + 1,开始就是为了不重复枚举
24 |                 if (get_count_1(a[i] ^ a[j]) == t)
25 |                     cnt++;
26 |             }
27 |         }
28 |         printf("%d\n", cnt);
29 |     }
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

```
5 2
1 2 3 4 5
2 3
```

样例 1 输出

```
4
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

5.10 | 总 283/526 为了再见你一面，我愿意重启整个世界 ( PID 1773 )

出处：2022级HAUT新生周赛 ( 三 ) @焦小航专场 ( D , CID 1081 ) | 训练定位：入门 | 数学、数论与组合

标签：数学、公式推导 | 限制：1.000 S / 128 MB | 历史统计：309/363 ( 85.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出  $c$ ，使得  $b$  为  $a$  和  $c$  的等差中项。

输入要点：给出两个正整数  $a, b$  (  $1 \leq a, b \leq 100$  )，数据保证  $a + c$  是偶数。

输出要点：输出  $c$ ，使得  $b$  为  $a$  和  $c$  的等差中项。

零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：由  $b=(a+c)/2$  移项，直接得到  $c=2b-a$ 。范围很小，普通 `int` 足够。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

$O(1)$  时间、 $O(1)$  空间

易错点

- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, b;
5 |     cin >> a >> b;
6 |     cout << 2 * b - a << '\n';
7 |     return 0;
8 | }
```

公开样例

样例 1 输入

2 5



样例 1 输出

8

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

5.11 | 总 284/526 君と彼女と彼女の恋 ( PID 1880 )

出处：河南工大2024新生周赛 ( 4 ) ——命题人：马贺 ( B , CID 1094 ) | 训练定位：入门 | 数学、数论与组合

标签：数论 | 限制：1.000 S / 128 MB | 历史统计：157/324 ( 48.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：美雪给葵一个整数  $x$ 。葵的任务是找出任意一个整数  $y$ ，使得  $\gcd(x,y)+y$  最大。 ( $1 \leq y \leq x$ )

输入要点：一个整数  $x$ 。 ( $1 \leq x \leq 500000$ )

输出要点： $\gcd(x,y)+y$  的最大值。

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175833\\_18328.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116175833_18328.png)]

零基础先修

- 数论常用整除、 $\gcd/\text{lcm}$ 、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：君と彼女と彼女の恋  $\because y$  最大可以等于  $x \therefore \gcd(x,y)$  最大 =  $x$  直接输出  $2 \times x$  即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

常数级或一次线性读入；以代码中的循环层数为准

易错点

- $\gcd/\text{lcm}$  的 0 值、负数和乘法溢出都要单独核对。

C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int x;
4 |     scanf("%d", &x);
5 |     printf("%d", x * 2);
6 |     return 0;
7 | }
```

公开样例

|         |
|---------|
| 样例 1 输入 |
| 1       |
| 样例 1 输出 |
| 2       |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

5.12 | 总 285/526 cds的大大大阶乘 ( PID 1245 )

出处：2016级河南工大新生周赛 ( 七 ) ( E , CID 1014 ) | 训练定位：基础提高 | 数学、数论与组合

标签：高精度、阶乘、数组模拟 | 限制：1.000 S / 128 MB | 历史统计：34/78 ( 43.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：cds：哦？是么，那我给你一个数n，你能立刻求出它的阶乘么？

输入要点：单实例测试，输入一个自然数n ( n<=2000)

输出要点：输出n的阶乘

零基础先修

- 超过 64 位范围时用字符串逐位运算，或使用题目允许的大整数工具。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：2000! 约有 5736 位，必须高精度。用低位在前的十进制数组表示结果，初始为 1；依次乘 2..n，逐位计算 `digit[i]*factor+carry`，留下个位并向高位传递进位。最后从最高位倒序输出。0! 与 1! 都会保持为 1。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(n·位数) 时间、O(位数) 空间

易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> digit(1, 1);
9 |     for (int factor = 2; factor <= n; ++factor) {
10 |         int carry = 0;
11 |         for (int& value : digit) {
12 |             int product = value * factor + carry;
13 |             value = product % 10;
14 |             carry = product / 10;
15 |         }
16 |         while (carry > 0) {
17 |             digit.push_back(carry % 10);
18 |             carry /= 10;
19 |         }
20 |     }
21 |     for (auto it = digit.rbegin(); it != digit.rend(); ++it)
22 |         cout << *it;
23 |     cout << '\n';
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

80

样例 1 输出

7156945704626380229481153372318653216558465734236575257710944505822703925548014884266894486728081408  
000000000000000000

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.13 | 总 286/526 ykc's easy exam ( PID 1246 )

出处：2016级河南工大新生周赛（七）（D，CID 1014）| 训练定位：基础提高 | 数学、数论与组合

标签：快速幂思想、个位周期、取模 | 限制：1.000 S / 128 MB | 历史统计：111/339 ( 32.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，Print single integer — the last digit

输入要点：The single line of input contains one integer  $n$  ( $0 \leq n \leq 10^9$ ).

输出要点：Print single integer — the last digit

## 零基础先修

- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3. 核心处理采用：只关心  $1378^n$  的个位，即  $8^n$  的个位。正指数时按 8,4,2,6 周期 4 循环； $n=0$  按定义结果为 1。可用数组按  $n\%4$  查询，其中余数 0 对应 6。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     cin >> n;
6 |     if (n == 0) {
7 |         cout << 1 << '\n';
8 |     } else {
9 |         const int lastDigit[4] = {6, 8, 4, 2};
10 |        cout << lastDigit[n % 4] << '\n';
11 |    }
12 |    return 0;
13 | }
```

## 公开样例

样例 1 输入

1

样例 1 输出

8

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.14 | 总 287/526 偶数 ( PID 1337 )

出处：2017级河南工大新生周赛 ( 五 ) ( C , CID 1033 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学、奇偶性、条件判断 | 限制：1.000 S / 128 MB | 历史统计：234/540 ( 43.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：小智特别喜欢偶数，所以它希望一个数可以由两个偶数相加得来。小智给你一个数  $n$ ，你来判断  $n$  是否可以由两个偶数(正整数)相加得来，如果可以

输入要点：第一行：测试实例个数T。

第二行：一个整数n (  $1 \leq n \leq 1000$  )

输出要点：输出“Happy QAQ”或者输出“Sad TAT”。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：两个正偶数的最小值都是 2，所以它们的和至少为 4，且仍是偶数。反过来，任何不小于 4 的偶数 n 都能写成  $2+(n-2)$ ，两项均为正偶数。因此判断 `n>=4 && n%2==0`。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         cout << (n >= 4 && n % 2 == 0 ? "Happy QAQ" : "Sad TAT") << '\n';
12 |     }
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

```
5
12
8
7
9
6
```

样例 1 输出

```
Happy QAQ
Happy QAQ
Sad TAT
Sad TAT
Happy QAQ
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.15 | 总 288/526 移动坐标 ( PID 1340 )

出处：2017级河南工大新生周赛 ( 五 ) ( F , CID 1033 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学、坐标、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：20/65 ( 30.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给你两个点A、B，和固定移动距离 ( x,y )，问是否可以通过移动A点若干次最终到B点。

输入要点：第一行：T，测试实例个数：

第二行：四个整数。x1,y1,x2,y2。分别是A点的坐标和B点的坐标。  $-10^5 \leq x1, y1, x2, y2 \leq 10^5$

第三行：两个整数。x,y是固定移动距离 (  $1 \leq x, y \leq 105$  )

输出要点：“YES”或者“NO”。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：设横、纵方向分别需要净走  $p=(x2-x1)/x$ 、 $q=(y2-y1)/y$  个单位步。首先两个差必须能整除固定步长。走 k 次时，每一维都是 k 个  $\pm 1$  的和，所以其绝对净步数不超过 k，且与 k 同奇偶。k 可以任意增大，因此只需  $|p|$  与  $|q|$  同奇偶：此时选足够大的同奇偶 k，两维都能安排出相应数量的正负步。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
```

```

7 |     cin >> T;
8 |     while (T--) {
9 |         long long x1, y1, x2, y2, xStep, yStep;
10 |         cin >> x1 >> y1 >> x2 >> y2;
11 |         cin >> xStep >> yStep;
12 |         long long dx = x2 - x1;
13 |         long long dy = y2 - y1;
14 |         bool possible = dx % xStep == 0 && dy % yStep == 0;
15 |         if (possible) {
16 |             long long horizontal = labs(dx / xStep);
17 |             long long vertical = labs(dy / yStep);
18 |             possible = horizontal % 2 == vertical % 2;
19 |         }
20 |         cout << (possible ? "YES" : "NO") << '\n';
21 |     }
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

2
0 0 0 6
2 3
1 1 3 6
1 5

```

样例 1 输出

```

YES
NO

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 5.16 | 总 289/526 比赛 ( PID 1341 )

出处：2017级河南工大新生周赛 ( 六 ) ( A , CID 1035 ) | 训练定位：基础提高 | 数学、数论与组合

标签：枚举、组合、奇偶性 | 限制：2.000 S / 256 MB | 历史统计：86/361 ( 23.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：你的任务是判断是否可能组成两个队使两个队的分数相等。

输入要点：第一行：整数  $T$ ，测试实例个数。

对于每组测试实例：

输入一行：包含6个整数  $a_1, \dots, a_6$  ( $0 \leq a_i \leq 1000$ )，代表每个同学的分数

输出要点：每组测试实例输出一行：如果可能组成两个队使两个队的分数相等，则输出"YES"；否则输出"NO"。

## 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。
- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。

- 核心处理采用：六人必须分成两个三人队。先求总分；若为奇数必不可能。否则枚举  $C(6,3)=20$  个三人组合，只要某组三人之和等于总分的一半，其余三人自然也是同一分数。数据固定只有六个数，直接三重循环最清楚。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         array<int, 6> score{};
10 |         int total = 0;
11 |         for (int& x : score) {
12 |             cin >> x;
13 |             total += x;
14 |         }
15 |         bool possible = false;
16 |         if (total % 2 == 0) {
17 |             for (int i = 0; i < 6; ++i)
18 |                 for (int j = i + 1; j < 6; ++j)
19 |                     for (int k = j + 1; k < 6; ++k)
20 |                         possible |= score[i] + score[j] + score[k] == total / 2;
21 |         }
22 |         cout << (possible ? "YES" : "NO") << '\n';
23 |     }
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

```
2
1 3 2 1 2 1
1 1 1 1 1 99
```

样例 1 输出

```
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)



## 5.17 | 总 290/526 数学 ( 一 ) ( PID 1356 )

出处：2017级河南工大新生周赛 ( 八 ) ( C , CID 1037 ) | 训练定位：基础提高 | 数学、数论与组合

标签：排列数、取模、循环 | 限制：1.000 S / 128 MB | 历史统计：14/59 ( 23.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出  $A(N, M)$ ，结果对 1000000007 取余。第  $i$  组数据输出前加上，“Test:”，不含引号。

输入要点：第一行一个整数  $N$  ( $0 < N < 210$ )

接下来  $N$  行数据，每行两个整数  $N, M$  ( $0 \leq M \leq N \leq 40$ )。

输出要点：输出  $A(N, M)$ ，结果对 1000000007 取余。

第  $i$  组数据输出前加上，“Test:”，不含引号。

题面提示：部分 整数类型 的大致数值范围：

short -32768 ~ 32767

int -2147483648 ~ 2147483647

long long -9223372036854774808 ~ 9223372036854774807

详情请自行百度。

### 零基础先修

- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。
- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：排列数  $A(N,M)=N\cdot(N-1)\cdot\ldots\cdot(N-M+1)$ 。因为只要求模 1000000007，不必先求可能溢出的阶乘；直接把这  $M$  个因子逐个乘入，每一步都取模即可。 $M=0$  时空乘积为 1。注意样例规定的编号格式为 `Test1: 120`。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(M)$  时间、 $O(1)$  空间

### 易错点

- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const long long MOD = 1000000007LL;
```

```

7 |     int T;
8 |     cin >> T;
9 |     for (int test = 1; test <= T; ++test) {
10 |         int n, m;
11 |         cin >> n >> m;
12 |         long long ans = 1;
13 |         for (int value = n - m + 1; value <= n; ++value)
14 |             ans = ans * value % MOD;
15 |         cout << "Test" << test << " : " << ans << '\n';
16 |     }
17 |     return 0;
18 | }

```

## 公开样例

样例 1 输入

```

3
5 5
7 0
34 3

```

样例 1 输出

```

Test1 : 120
Test2 : 1
Test3 : 35904

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 5.18 | 总 291/526 数学 ( 二 ) ( PID 1384 )

出处：2018级河南工大新生周赛 ( 一 ) @徐向阳专场 ( C , CID 1040 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数论 | 限制：1.000 S / 128 MB | 历史统计：78/167 ( 46.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对每行输入，输出其对应那个的八进制数，单独占一行。

输入要点：第一行一个正整数  $N$  (  $0 < N \leq 1024$  )

接下来  $N$  行正二进制数  $M$  (  $0 \leq M \leq 1111111111$  )。

输出要点：对每行输入，输出其对应那个的八进制数，单独占一行。

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：数学 ( 二 ) 思路：从  $M$  的个位开始，每次取三位，转化成8进制数。举例：  $M=1001010111$  第一步：取出111，  $M=1001010$ ，结果为7第二步：取出010，  $M=1001$ ，结果为27第三步：取出001，  $M=1$ ，结果为127第四步：取出1，  $M=0$  (  $M=0$ 则结束 )，结果为1127最终结果为：1127参考代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int i, n, m;
4 |     scanf("%d", &n);
5 |     for (i = 1; i <= n; i++) {
6 |         int reg = 0;
7 |         int x = 1;
8 |         ;
9 |         scanf("%d", &m);
10 |         if (m == 0) {
11 |             printf("0\n");
12 |         } else {
13 |             while (m > 0) {
14 |                 int t = m % 1000;
15 |                 int a, b, c; // 个, 十, 百位。
16 |                 a = t % 10;
17 |                 b = (t / 10) % 10;
18 |                 c = (t / 100) % 10;
19 |                 reg = reg + (a + b * 2 + c * 4) * x;
20 |                 x = x * 10;
21 |                 m = m / 1000;
22 |             }
23 |             printf("%d\n", reg);
24 |         }
25 |     }
26 | }
```

## 公开样例

样例 1 输入

```
3
111
10101
0
```

样例 1 输出

```
7
25
0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 5.19 | 总 292/526 曼曼曼曼曼哈顿 ( PID 1389 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( B , CID 1041 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：228/613 ( 37.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，即乌拉拉经过两个点所需的最短距离

输入要点：输入一共两行，每行包含两个整数  $x,y$  代表两个需要经过的点的坐标 (  $-2000 \leq x, y \leq 2000$  )

输出要点：一个整数，即乌拉拉经过两个点所需的最短距离

题面提示：所求距离为曼哈顿距离

如点 ( 0 , 0 ) 和点 ( 3 , 4 )

这两个点的直线距离 ( 欧几里得距离 ) 为5

这两个点的曼哈顿距离为  $3+4=7$

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：所求距离为曼哈顿距离 如点 ( 0 , 0 ) 和点 ( 3 , 4 ) 这两个点的直线距离 ( 欧几里得距离 ) 为5 这两个点的曼哈顿距离为  $3+4=7$
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     int min1, min2;
5 |     int xa, ya, xb, yb;
6 |     scanf("%d %d\n%d %d", &xa, &ya, &xb, &yb);
7 |     min1 = abs(xa) + abs(ya) + abs(xa - xb) + abs(ya - yb);
8 |     min2 = abs(xb) + abs(yb) + abs(xa - xb) + abs(ya - yb);
9 |     if (min1 < min2) {
10 |         printf("%d", min1);
11 |     } else {
12 |         printf("%d", min2);
13 |     }
14 | }
```

### 公开样例

样例 1 输入

```
1 1
2 2
```

样例 1 输出

```
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.20 | 总 293/526 学学学学学素数 ( PID 1390 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( C , CID 1041 ) | 训练定位：基础提高 | 数学、数论与组合

标签：质数筛、个位数、计数 | 限制：1.000 S / 128 MB | 历史统计：141/404 ( 34.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，两个整数  $a, b$ 。分别是个位数是 1, 3, 7, 9 的正整数的数量  $a$  和 其中素数的个数  $b$

输入要点：一个正整数  $n$ 。(  $1 < n \leq 1000000$  )

输出要点：两个整数  $a, b$ 。分别是个位数是 1, 3, 7, 9 的正整数的数量  $a$  和 其中素数的个数  $b$

## 零基础先修

- 先从定义和小数据找规律，再用整除、奇偶、最大公约数、取模或组合计数证明公式。特别注意整数溢出与浮点误差。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：先用埃氏筛预处理  $0..n$  的素数性，明确把 0 和 1 标为非素数。然后 遍历  $1..n$ ：个位属于 1、3、7、9 时增加  $a$ ；若筛表还表明它是素数，再增加  $b$ 。样例中的候选为 1、3、7、9，其中只有 3、7 是素数，所以输出 4 2。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n \log \log n)$  时间、 $O(n)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 | ios::sync_with_stdio(false);
5 | cin.tie(nullptr);
6 | int n;
7 | cin >> n;
8 | vector<bool> prime(n + 1, true);
9 | prime[0] = false;
10 | if (n >= 1)
11 |     prime[1] = false;
12 | for (int p = 2; 1LL * p * p <= n; ++p)
13 |     if (prime[p])
14 |         for (int multiple = p * p; multiple <= n; multiple += p)
15 |             prime[multiple] = false;
16 | int candList = 0, cand = 0;
17 | for (int value = 1; value <= n; ++value) {
18 |     int last = value % 10;
19 |     if (last == 1 || last == 3 || last == 7 || last == 9) {
20 |         ++candList;
21 |         cand += prime[value];
22 |     }
23 | }
24 | cout << candList << ' ' << cand << '\n';
25 | return 0;
26 | }

```

## 公开样例

样例 1 输入

10

样例 1 输出

4 2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.21 | 总 294/526 存储单位进率 ( PID 1396 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( A , CID 1042 ) | 训练定位：基础提高 | 数学、数论与组合

标签：查表、数学、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：84/197 ( 42.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组输出一个数X，代表后一个单位是前一个单位的2的X次方倍,每组占一行。

输入要点：每组输入占一行，每行包括两个字符，代表两个单位，直到输入两个都为0停止。

输出要点：每组输出一个数X，代表后一个单位是前一个单位的2的X次方倍,每组占一行。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 多组数据以 EOF 结束时使用 while (cin >> ...)，不要额外输出测试数据。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：把所有单位统一换成“相对 1 bit 的二进制指数”：b=0、B=3，K=13、M=23、G=33、T=43。后一个单位是前一个单位的  $2^{(exp2-exp1)}$  倍，直接做指数相减；读到 0 0 停止。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- EOF 题不要先读不存在的 T，也不要再在循环结束后多输出内容。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int exponent(char unit) {
4 |     if (unit == 'b')
5 |         return 0;
6 |     if (unit == 'B')
7 |         return 3;
8 |     string larger = "KMGT";
9 |     return 13 + 10 * larger.find(unit);
10 | }
11 | int main() {
12 |     char first, second;
13 |     while (cin >> first >> second && !(first == '0' && second == '0')) {
14 |         cout << exponent(second) - exponent(first) << '\n';
15 |     }
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
b B
B K
M K
G M
G T
0 0
```

样例 1 输出

```
3
10
-10
-10
10
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.22 | 总 295/526 Math ( PID 1422 )

出处：2018级河南工大新生周赛总决赛（重现）（F，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（F，CID 1048） | 训练定位：基础提高 | 数学、数论与组合

标签：数论、循环小数、最大公约数、字符串解析 | 限制：2.000 S / 16 MB | 历史统计：4/13 ( 30.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给你一个小于1的非负有理数的小数形式，求出它的分数形式，当然这个小数可能是无限循环小数

输入要点：先输入一个T表示T组测试数据（ $T \leq 15$ ）

接下来每行输入一个字符串表示一个大于等于0小于1的有理数，长度不会超过10

输出要点：每一个样例输出对应的最简分数形式，如果分子是分母的倍数，直接输出一个整数

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- std::gcd(a,b) 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。
- 解析字符串时先确定分隔符、字符类别和多位数边界，再逐段转换。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：设小数点后非循环部分有  $k$  位、循环节有  $r$  位。若不存在循环节，分数就是  $A/10^k$ ；若存在，令  $A$  为非循环数字形成的整数、 $AB$  为“非循环+一遍循环节”形成的整数，则  $x=(AB-A)/(10^{k+r}-10^k)$ 。最后用最大公约数约分；分母约成 1 时只输出整数。这个公式来自把  $x$  分别乘  $10^k$  和  $10^{k+r}$  后相减，循环尾部会完全抵消。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(|s|+\log V)$  时间、 $O(|s|)$  空间

### 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long power10(int exponent) {
4 |     long long value = 1;
5 |     while (exponent--)
6 |         value *= 10;
7 |     return value;
8 | }
9 | long long digitsValue(const string& s) {
10 |     long long value = 0;
11 |     for (char c : s)
12 |         value = value * 10 + (c - '0');
13 |     return value;
14 | }
15 | int main() {
16 |     ios::sync_with_stdio(false);
17 |     cin.tie(nullptr);
```



```

18 |     int T;
19 |     cin >> T;
20 |     while (T--) {
21 |         string s;
22 |         cin >> s;
23 |         size_t dot = s.find('.');
24 |         size_t leftBracket = s.find('[');
25 |         string nonRepeat;
26 |         string repeat;
27 |         if (dot != string::npos) {
28 |             size_t end = leftBracket == string::npos ? s.size() : leftBracket;
29 |             nonRepeat = s.substr(dot + 1, end - dot - 1);
30 |         }
31 |         if (leftBracket != string::npos) {
32 |             size_t r = s.find(']', leftBracket);
33 |             repeat = s.substr(leftBracket + 1, r - leftBracket - 1);
34 |         }
35 |         long long p, q;
36 |         if (repeat.empty()) {
37 |             p = digitsValue(nonRepeat);
38 |             q = power10((int)nonRepeat.size());
39 |         } else {
40 |             long long A = digitsValue(nonRepeat);
41 |             long long AB = digitsValue(nonRepeat + repeat);
42 |             p = AB - A;
43 |             q = power10((int)(nonRepeat.size() + repeat.size())) -
44 |                 power10((int)nonRepeat.size());
45 |         }
46 |         long long g = gcd(p, q);
47 |         p /= g;
48 |         q /= g;
49 |         if (q == 1)
50 |             cout << p << '\n';
51 |         else
52 |             cout << p << '/' << q << '\n';
53 |     }
54 |     return 0;
55 | }

```

## 公开样例

样例 1 输入

```

3
0.8[3]
0.[3]
0.125

```

样例 1 输出

```

5/6
1/3
1/8

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.23 | 总 296/526 数学家 ( PID 1433 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( F , CID 1047 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数论、最大公约数、字符串解析 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：他希望你能帮他找到满足条件的最小正整数 $b$ ，并且用 $c/b$ 来表示这个小数 $0.a$ 。

输入要点：多样例测试

第一行输入一个整数  $T$  ( $1 \leq T \leq 10^5$ )

接下来的  $n$  行每行输入一个小数  $a$  ( $0 < a < 10^{18}$ ,  $a$  不含前导 0)

输出要点：输出  $c/b$  来表示这个小数 ( $b$  为满足条件的最小正整数)

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- std::gcd(a,b) 返回最大公约数，并满足  $\text{gcd}(a,b) = \text{gcd}(b, a \bmod b)$ 。
- 解析字符串时先确定分隔符、字符类别和多位数边界，再逐段转换。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：把输入当字符串，取小数点后的数字作为分子  $a$ ，分母是  $10^{\text{小数位数}}$ 。用  $\text{gcd}(a, \text{denominator})$  同除，所得分母最小；字符串方式还能准确保留小数位数而不受浮点误差影响。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(\text{小数位数} + \log \text{值域})$  时间、 $O(\text{小数位数})$  空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int T;
5 |     cin >> T;
6 |     while (T--) {
7 |         string decimal;
8 |         cin >> decimal;
9 |         string digits = decimal.substr(decimal.find('.') + 1);
10 |         unsigned long long p = stoull(digits);
11 |         unsigned long long q = 1;
12 |         for (char ignored : digits)
13 |             q *= 10;
14 |         unsigned long long divisor = gcd(p, q);
15 |         cout << p / divisor << '/' << q / divisor << '\n';
16 |     }
17 |     return 0;
18 | }
```

## 公开样例

样例 1 输入

```
2
0.2
0.5678
```

样例 1 输出

```
1/5
2839/5000
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.24 | 总 297/526 优化算法 ( PID 1434 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( G , CID 1047 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学、复杂度估算、向上取整 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：速度为( $v \cdot 10^{18}$ 次/s)的计算机。用这个优化后的算法能在几秒内计算出最优解。

输入要点：多样例测试

第一行输入样例数：T

对于每个样例：输入S n v ( $0 \leq s, n < 10^7, 0 < v < 10^7$ )

输出要点：对于每个样例输出：计算机能在几秒内计算出最优解(对于小数部分向上取整)。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：算法运算量为  $(S \cdot 10^9)(n \cdot 10^9) = Sn \cdot 10^{18}$ ，计算机每秒完成  $v \cdot 10^{18}$  次，秒数就是  $Sn/v$  向上取整，即  $(Sn+v-1)/v$ 。题目给定 范围内 Sn 可用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int T;
5 |     cin >> T;
```

```
6 |         while (T--) {
7 |             long long capacity, number, speed;
8 |             cin >> capacity >> number >> speed;
9 |             cout << (capacity * number + speed - 1) / speed << '\n';
10 |         }
11 |         return 0;
12 |     }
```

## 公开样例

样例 1 输入

```
2
3 3 9
3 2 5
```

样例 1 输出

```
1
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.25 | 总 298/526 cxk 的数学难题 ( PID 1474 )

出处：2019级河南工大新生周赛 ( 一 ) @陶福专场 ( F , CID 1053 ) | 训练定位：基础提高 | 数学、数论与组合

标签：模拟、奇偶性、Collatz | 限制：1.000 S / 128 MB | 历史统计：128/507 ( 25.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：cxk 的数学难题

输入要点：官网没有可提取的文字输入说明，请以原题页为准。

输出要点：官网没有可提取的文字输出说明，请以原题页为准。

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：按 Collatz 规则直接模拟：奇数变为  $3n+1$ ，偶数除以 2，每变化一次计数，直到  $n=1$ 。使用 64 位无符号整数覆盖中间值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(\text{变换次数})$  时间、 $O(1)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     unsigned long long n;
5 |     cin >> n;
6 |     long long steps = 0;
7 |     while (n != 1) {
8 |         if (n & 1ULL)
9 |             n = 3 * n + 1;
10 |        else
11 |            n /= 2;
12 |        ++steps;
13 |    }
14 |    cout << steps << '\n';
15 |    return 0;
16 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

7

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.26 | 总 299/526 悲欢离合 ( PID 1484 )

出处：2019级河南工大新生周赛（二）@邹岩专场（E，CID 1054）| 训练定位：基础提高 | 数学、数论与组合

标签：字符串、构造、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：154/270 ( 57.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出【欢】的感受。

输入要点：输入一个整数为【欢】的情感层数  $n$  ( $1 \leq n \leq 100$ )。

输出要点：输出【欢】的感受。

## 零基础先修

- `string` 可按下标访问字符；注意长度、空串、大小写、标点和 `getline` 与 `cin` 混用。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：第  $i$  层从 1 开始，奇数层输出 I hate，偶数层输出 I love；最后一层后接 it，其余层后接 that。严格控制空格即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     for (int i = 1; i <= n; ++i) {
7 |         cout << (i % 2 ? "I hate " : "I love ");
8 |         cout << (i == n ? "it" : "that ");
9 |     }
10 |    cout << '\n';
11 |    return 0;
12 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

I hate that I love that I hate it

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.27 | 总 300/526 欢迎来到LDX广场 ( PID 1503 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( F , CID 1056 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学、铺砖、构造 | 限制：1.000 S / 128 MB | 历史统计：236/397 ( 59.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：欢迎来到LDX广场，广场大小 $n*m$ ，由于广场新建，需要用 $2*1$ 大小的方块去最大限度的填满这个广场。请问需要多少方块。

输入要点：输入 $n,m$ 表示矩形广场的边

$1 \leq n \leq m \leq 5000$

输出要点：输出用多少方块可以尽可能铺满广场。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：每块覆盖两个格子，所以任何方案至多使用  $\text{floor}(nm/2)$  块。矩形网格可以沿任意偶数方向成对铺设；即使总格数为奇数，也只剩一个格子，因此这个上界总能达到。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, m;
5 |     cin >> n >> m;
6 |     cout << n * m / 2 << '\n';
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

2 4

样例 1 输出

4

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.28 | 总 301/526 小青的宿舍 ( PID 1534 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（G，CID 1060）| 训练定位：基础提高 | 数学、数论与组合

标签：枚举、数位统计、取模 | 限制：1.000 S / 128 MB | 历史统计：149/315 ( 47.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个正整数，表示小青的房间号。

输入要点：一个正整数  $n$ ，(  $1 \leq n \leq 10000$  )

输出要点：一个正整数，表示小青的房间号。

题面提示：2，12，20，21存在一个2，22存在两个2，(  $1^4+2^1$  ) %2020=6

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。
- 取模用于周期或大数；减法后可写  $(x-y+mod)\%mod$  防止出现负数。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
- 核心处理采用： $n \leq 10000$ ，直接枚举  $1..n$ ，把每个数逐位拆开并统计数字 2。只需输出模 2020 的结果，计数过程中随时取模即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

$O(n \log n)$  时间、 $O(1)$  空间

### 易错点

- C++ 负数取模仍可能为负；减法后补模数。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     int ans = 0;
7 |     for (int x = 1; x <= n; ++x) {
8 |         int value = x;
9 |         while (value) {
10 |             if (value % 10 == 2)
11 |                 ans = (ans + 1) % 2020;
12 |             value /= 10;
13 |         }
14 |     }
15 |     cout << ans << '\n';
16 |     return 0;
17 | }
```

### 公开样例

样例 1 输入

22

样例 1 输出

6



## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.29 | 总 302/526 求和 ( PID 1553 )

出处：2020级HAUT新生周赛（三）@张雍蕻专场（G，CID 1062）| 训练定位：基础提高 | 数学、数论与组合

标签：数学、等差数列、公式推导 | 限制：1.000 S / 128 MB | 历史统计：58/263 ( 22.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，T行，每行一个整数，表示结果

输入要点：多组数据，第一行为T,代表有T组

接下来T行,每行两个正整数x和n，用空格隔开

$1 \leq T \leq 20$

$1 \leq x \leq 10$

$1 \leq n \leq 1e8$

输出要点：T行，每行一个整数，表示结果

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：题图定义  $f(n) = \sum_{i=0..n} x \cdot i$ 。提出常数  $x$ ，再用等差数列求和  $0+1+\dots+n = n(n+1)/2$ ，得到  $f(n) = x \cdot n(n+1)/2$ 。乘法使用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long x, n;
10 |         cin >> x >> n;
11 |         cout << x * n * (n + 1) / 2 << '\n';
12 |     }
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

```
5
5 7
2 10
2 4
1 8
10 100000000
```

样例 1 输出

```
140
110
20
36
50000000500000000
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 5.30 | 总 303/526 结束了(easy version) ( PID 1725 )

出处：2021级HAUT新生周赛（六）@李志豪专场（G，CID 1074） | 训练定位：基础提高 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：45/186 ( 24.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在你的任务就变成了求能把从1到n所有整数整除的最小整数。

输入要点：一个正整数n。1<= n <= 20

输出要点：一个整数，表示答案。由于答案可能过大，请把答案取模1e9+7。

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。

3. 核心处理采用：G.H. 结束了 标签：数论 G-easy H-hard 题目意思是寻找  $[1, n]$  的所有数的最小公倍数。在这道题之前，先说一个数学定理：唯一分解定理又叫算术基本定理。定理中提到：对于任何一个大于1的正整数，都存在唯一的分解式：，其中  $n = p_1^{k_1} * p_2^{k_2} * \dots * p_k^{k_k}$   $n = p_1^{k_1} * p_2^{k_2} * \dots * p_n^{k_n}$  都是素数。  $p_i$  知道了这个定理之后，再去求1到n的最小公倍数就变成了求出小于n的质数的最高幂次的乘积。由于数据过大，所以需要使用线性筛法，为了照顾大家线性筛的写法都给大家了。筛出所有的素数之后，进行暴力枚举最高次幂就行了。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #pragma GCC optimize("O2")
4 | #define ll long long
5 | #define endl "\n"
6 | const int max_n = 1e8 + 100;
7 | const int inf = 0x3f3f3f3f;
8 | const int mod = 1e9 + 7;
9 | int primes[6000000], id = 0;
10 | bool used[max_n];
11 | void init(int n) {
12 |     for (int i = 2; i <= n; ++i) {
13 |         if (!used[i])
14 |             primes[id++] = i;
15 |         for (int j = 0; j < id && i * primes[j] <= n; ++j) {
16 |             used[i * primes[j]] = true;
17 |             if (i % primes[j] == 0)
18 |                 break;
19 |         }
20 |     }
21 | }
22 | int main() {
23 |     ios::sync_with_stdio(false);
24 |     cin.tie(0);
25 |     cout.tie(0);
26 |     int n;
27 |     cin >> n;
28 |     init(n);
29 |     ll ans = 1;
30 |     for (int i = 0; i < id; ++i) {
31 |         if (primes[i] > n)
32 |             break;
33 |         ll t = 1;
34 |         while (t <= n)
35 |             t *= primes[i];
36 |         t /= primes[i];
37 |         ans = ans * t % mod;
38 |     }
39 |     cout << ans << endl;
40 |     return 0;
41 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.31 | 总 304/526 错位时空 ( PID 1730 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（E，CID 1075） | 训练定位：基础提高 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：86/379 ( 22.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：在这个赛季的 $n$ 场ycpc亚洲区域赛中，kws 要带领他的攻壳机动队拿得这个赛季的奖牌。如果打的第一场kws拿到了奖牌，他们就不会打下一场了(要放松一下)，如果打的第一场没拿到奖牌，他们也再不会打了(心态炸了)。lzh和zjy表示要打比赛非常开心和激动，但是也十分害怕打铁，于是他们找到了空间大师sjp，想要知道是否会打铁，但sjp是空间大师，不知道是否会打铁，.....

输入要点：第一行输入这个赛季的比赛次数 $n(8 \leq n \leq 10)$

第二行输入 $n$ 个整数 $a_1, a_2, \dots, a_n$ ，表示这一场kws拿奖的概率( $0 \leq a_i \leq 100$ )

输出要点：输出一个整数 $t$ 表示kws这个赛季打铁的概率( $t\%$ ) ( $t$ 为 $0 \sim 100$ 的整数)

题面提示：1.最终结果保留到整数位（四舍五入）

2.打铁就是没拿到奖

样例说明

十场比赛，每一场的拿奖率都是50%，所以总的拿奖率也是50%，打铁率也是50%

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：错位时空概率期望 结果要进行四舍五入 计算时不能计算出最终拿奖率后用100减，在每一个给出拿奖率后要直接转换为打铁率后再进行计算
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     int temp;
5 |     double sum;
6 |     scanf("%d", &n);
7 |     for (int i = 1; i <= n; i++) {
8 |         scanf("%d", &temp);
9 |         sum += 100 - temp;
10 |     }
11 |     printf("%.f", sum / n);
12 | }
```

### 公开样例

|                                     |
|-------------------------------------|
| 样例 1 输入                             |
| 10<br>50 50 50 50 50 50 50 50 50 50 |
| 样例 1 输出                             |
| 50                                  |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.32 | 总 305/526 夜间出租车 ( PID 1792 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（G，CID 1083）| 训练定位：基础提高 | 数学、数论与组合

标签：字符串、数学、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/6 ( 33.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每次询问输出一个时间点占一行，格式与输入相同。

输入要点：第一行一个整数 n,q表示路口的数量和询问的数量，

第二行一个时间表示当前时间点，格式为 `hh:mm` ,二十四小时制，例`08:08`。

第三行 n个字符表示当前 `n` 个路口红绿灯的状态，`r` 代表红灯 `g` 代表绿灯 `y` 代表黄灯，

接下来 q 行每行一个整数 x ，询问通过第x个路口的时间点。

(1 <= n,q <=105)

输出要点：对于每次询问输出一个时间点占一行，格式与输入相同。

题面提示：为了方便理解，这里放另一组数据

输入：

5 2  
18:00  
rgyrg

5  
1

输出：

18:09  
18:01

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：夜间出租车 题意：快速计算出出租车通过某个路口的时间 解法：模拟题，按照题意，通过绿灯不耗时间，但等待绿灯的过程需要时间，所以从前往后模拟每经过一个红绿灯就把经过这个红绿灯消耗的时间记录一下加在时刻上（由题意知最多等待两分钟），且所有本题需要预处理一遍通过每个红绿灯的时间，然后查询的时候直接回答即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, q, h, m;
3 | char str[100005];
4 | int ti[100005];
5 | // 这里记录分别为绿灯黄灯红灯时要等的时间
6 | // a[0]代表绿灯的情况，a[1]代表黄灯的情况，a[2]代表红灯的情况
7 | // 因为路灯三分钟一循环，所以记录三个值就行
8 | int a[3][3] = {{0, 2, 1}, {2, 1, 0}, {1, 0, 2}};
9 | int main() {
10 |     scanf("%d%d", &n, &q);
11 |     scanf("%d:%d", &h, &m);
12 |     scanf("%s", str);
13 |     int now = 0;
14 |     for (int i = 0; i < n; i++) {
15 |         if (str[i] == 'g') {
16 |             now += a[0][now % 3];
17 |         } else if (str[i] == 'y') {
18 |             now += a[1][now % 3];
19 |         } else if (str[i] == 'r') {
20 |             now += a[2][now % 3];
21 |         }
22 |         ti[i] = now;
23 |     }
24 |     for (int i = 1; i <= q; i++) {
25 |         int x;
26 |         scanf("%d", &x);
27 |         printf("%02d:%02d\n", ((ti[x - 1] + m) / 60 + h) % 24, (ti[x - 1] + m) % 60);
28 |     }
```

```
29 |         return 0;
30 |     }
```

公开样例

样例 1 输入

```
5 1
18:00
99999
5
```

样例 1 输出

```
18:00
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.33 | 总 306/526 二进制翻转 (PID 1818)

出处：河南工大2024新生周赛 ( 1 ) ——命题人：程立老师，陈兰锴 ( C, CID 1091 ) ；2023级HAUT新生周赛 ( 一 ) @21级学长专场 ( 张子豪，张鑫，李昊阳 ) ( B, CID 1086 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数组与序列、数论 | 限制：1.000 S / 128 MB | 历史统计：224/821 ( 27.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：你每次可以选择连续的一些0翻转成1，请计算把数组中所有元素变成1所需的最小翻转次数。

输入要点：一个整数n，表示数组长度 ( 1<=n<=200000 )

一行n个0或1，用空格分隔。

输出要点：一个整数，表示最小翻转次数

零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：二进制翻转 根据题意只需要计算数组中连续的0的段数即是答案，注意到每个连续的0的段前面必定有一个1 ( 如果起始为0单独计算 )，所以记录前1后0的位置的数量即是连续的0的段的数量。c c++
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int n;
3 | int a[200005];
4 | int main() {
5 |     scanf("%d", &n);
6 |     for (int i = 0; i < n; i++)
7 |         scanf("%d", &a[i]);
8 |     int ans = 0;
9 |     if (a[0] == 0)
10 |         ans++;
11 |     for (int i = 1; i < n; i++) {
12 |         if (a[i] == 0 && a[i - 1] == 1)
13 |             ans++;
14 |     }
15 |     printf("%d\n", ans);
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
5
0 1 0 0 1
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 5.34 | 总 307/526 神奇的谕示裁定枢机 ( PID 1841 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（E，CID 1088）| 训练定位：基础提高 | 数学、数论与组合

标签：数学、信息量、对数 | 限制：1.000 S / 128 MB | 历史统计：30/51 ( 58.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对于每组数据，输出保证一定能找到较重物品的最少称量次数。

输入要点：输入两个整数  $n$ 、 $m$ ， $2 \leq n \leq 10^9$ ， $2 \leq m \leq 10^9$ 。

输出要点：对于每组数据，输出保证一定能找到较重物品的最少称量次数。

题面提示：对于样例：

首先把9件物品分成三堆，每堆3件，然后将其中两堆放入谕示裁定枢机。若一样重则表示重的物品在没放入的那堆中，否则通过谕示裁定枢机我们可以找到重的那堆。



把重的那堆三件物品中的其中两件再次放入谕示裁定枢机，若一样重则表示剩下的那件物品就是重的物品，否则通过谕示裁定枢机我们可以找到重的那件物品。

那维莱特：水龙不会，水龙哭哭~ TAT

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：一次称量最多产生  $m+1$  种结果： $m$  份中某份最重，或  $m$  份一样重从而重物在未上秤的一份。做  $k$  次最多区分  $(m+1)^k$  个候选，因此 答案是最小的  $k$  使该值不少于  $n$ 。循环乘  $m+1$ ，并用除法上界避免 64 位溢出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(\log_{m+1} n)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     unsigned long long n, m;
5 |     cin >> n >> m;
6 |     unsigned long long capacity = 1;
7 |     int ans = 0;
8 |     while (capacity < n) {
9 |         ++ans;
10 |         if (capacity > n / (m + 1))
11 |             capacity = n;
12 |         else
13 |             capacity *= m + 1;
14 |     }
15 |     cout << ans << '\n';
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

92

样例 1 输出

2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 5.35 | 总 308/526 旅行者的抛弃 ( PID 1853 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（A，CID 1090）| 训练定位：基础提高 | 数学、数论与组合

标签：固定输出与格式、数学 | 限制：1.000 S / 128 MB | 历史统计：12/30 ( 40.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个小数（保留3位小数）

输入要点：第一行输入两个整数

m M

输出要点：一个小数（保留3位小数）

题面提示：所有星球都是用一种叫“稀”的元素形成，表明密度相同，（可以使用pow函数求立方根）

考察高中物理，万有引力公式

### 零基础先修

- 输出也是答案的一部分：大小写、空格、换行、标点和小数位数均需与题面一致。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：旅行者的抛弃 签到题 根据重量比求出半径比 要注意重量比是  $M + 1$   $m_1$  是可莉的重量， $m_2$  是要求的结果  $m_1 g_1 / 2 = m_2 g_2$  根据万有引力公式： $g = GM / (r * r)$  得出  $g_1 / g_2$  的值 最后带入 可得  $m_2$  最后直接输出
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$

### 易错点

- 复制样例时核对中英文标点、尾随空格和末尾换行。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | void solve() {
4 |     double m, M;
5 |     cin >> m >> M;
6 |     double r = pow(M + 1, 1.0 / 3);
7 |     cout << fixed << setprecision(3) << r * r / (M + 1) * m / 2;
8 | }
```

```
9 | int main() {
10 |     ios::sync_with_stdio(false);
11 |     cin.tie(0);
12 |     cout.tie(0);
13 |     int test = 1;
14 |     // cin >> test;
15 |     while (test--) {
16 |         solve();
17 |     }
18 |     return 0;
19 | }
```

公开样例

样例 1 输入

1 2

样例 1 输出

0.347

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.36 | 总 309/526 分解质因数 ( PID 1893 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭钰 ( G , CID 1095 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数论 | 限制：1.000 S / 128 MB | 历史统计：50/160 ( 31.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：给出一个数字  $n$  要求你分解他的质因数，且分解出来的质因数要求个数最少。

输入要点：输入包含 1 行

输入一个数字  $n$  (  $1 \leq n \leq 1e6$  )

输出要点：输出包含一行

输出分解出来的  $n$  的质因数

题面提示：18 = 2 \* 3 \* 3

零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：分解质因数 模拟操作即可 #include<bits/stdc++.h>
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。

- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define int long long
4 | // 判断是否为质数的函数
5 | bool is_prime(int x) {
6 |     if (x < 2)
7 |         return false;
8 |     for (int i = 2; i <= x / i; i++)
9 |         if (x % i == 0)
10 |             return false;
11 |     return true;
12 | }
13 | // 输出函数
14 | void printPrime(int n) {
15 |     // 如果n本身就是质数，输出n即可
16 |     if (is_prime(n)) {
17 |         cout << n;
18 |         return;
19 |     } else {
20 |         int x = n;
21 |         for (int i = 2; i <= x / i; i++) {
22 |             // 直到x = 1 或者 x 为质数时结束循环
23 |             if (x == 1 || is_prime(x))
24 |                 break; // ①
25 |             // 如果i是x的因数
26 |             if (x % i == 0) {
27 |                 if (is_prime(i)) // 如果i是x的质因数
28 |                 {
29 |                     while (x % i == 0) // 循环对x提取质因子i
30 |                     {
31 |                         x /= i;
32 |                     }
33 |                     // 输出i
34 |                     cout << i << ' ';
35 |                 }
36 |                 // 如果i不是x的质因数，重新循环
37 |                 else {
38 |                     continue;
39 |                 }
40 |             }
41 |         }
42 |         // 如果x不为1，输出x（上方①处判断过x，必为质数或1，不为1时x必为质数，
43 |         // 输出即可）
44 |         if (x != 1)
45 |             cout << x;
46 |     }
47 | }
48 | signed main() {
49 |     int n;
50 |     cin >> n;
51 |     printPrime(n);
52 |     return 0;
53 | }

```

## 公开样例

样例 1 输入

18

样例 1 输出

2 3

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.37 | 总 310/526 天杀的二进制！（PID 1897）

出处：河南工大2024新生周赛（6）——命题人：魏方（B，CID 1097） | 训练定位：基础提高 | 数学、数论与组合

标签：字符串、数论 | 限制：1.000 S / 128 MB | 历史统计：116/456（25.4%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出包含t行，每一行输出一个十进制数字

输入要点：输入一个t，表示有t组测试用例

接着输入t个字符串，表示t个需要转换的二进制数字

输出要点：输出包含t行，每一行输出一个十进制数字

题面提示：从左到右表示二进制为从低到高

零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：天杀的二进制 本题也算是签到题，就是将2的n次方进行相加，简单的循环遍历读取字符即可。 注意要处理换行符。在多组输入输出中，可以构造solve函数，一组测试用例一次调用，降低耦合度，提高函数可读性
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

通常为  $O(n)$

易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

C++17 参考实现

```
1 | #include <stdio.h>
2 | void solve() {
3 |     int sum = 0;
4 |     getchar();
5 |     for (int i = 0; i <= 31; i++) {
6 |         char ch;
```

## 公开样例

21  
7

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

- HAUTOJ 原题
- 公开题解/参考来源 1

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 第 446 页

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：you love 1203! 本题就是一个暴力遍历，我们将遍历地毯的所有层，并将每一层中遇到的1203记录数添加到答案中，为此，我们可以遍历例如每层左上角的单元格，其形式为(i,i),i的范围是[1,min(n,m)/2];然后使用简单算法遍历该层，将遇到的数字写入某个数组，然后遍历数组，计算该层中出现的1203，此外，在遍历数组是，我们应该考虑该层的循环性质，记得检查包含起始单元的1203是否可能出现，以及数组越界的情况。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | char a[1005][1005];
3 | char layer[4005];
4 | void solve() {
5 |     int n, m;
6 |     scanf("%d %d", &n, &m);
7 |     for (int i = 0; i < n; ++i)
8 |         scanf("%s", a[i]);
9 |     int count = 0;
10 |    for (int i = 0; (i + 1) * 2 <= n && (i + 1) * 2 <= m; ++i) {
11 |        int pos = 0;
12 |        for (int j = i; j < m - i; ++j)
13 |            layer[pos++] = a[i][j];
14 |        for (int j = i + 1; j < n - i - 1; ++j)
15 |            layer[pos++] = a[j][m - i - 1];
16 |        for (int j = m - i - 1; j >= i; --j)
17 |            layer[pos++] = a[n - i - 1][j];
18 |        for (int j = n - i - 2; j >= i + 1; --j)
19 |            layer[pos++] = a[j][i];
20 |        for (int j = 0; j < pos; ++j)
21 |            if (layer[j] == '1' && layer[(j + 1) % pos] == '2' &&
22 |                layer[(j + 2) % pos] == '0' && layer[(j + 3) % pos] == '3')
23 |                count++;
24 |    }
25 |    printf("%lld\n", count);
26 | }
27 | int main() {
28 |     int t;
29 |     scanf("%d", &t);
30 |     while (t--)
31 |         solve();
32 | }
```

## 公开样例

样例 1 输入

```
7
2 4
1203
7777
2 4
7120
8903
2 4
3021
8888
2 2
20
13
2 2
```

```
51
43
2 6
032012
212030
4 4
3021
1203
5518
7634
```

样例 1 输出

```
1
1
0
1
0
2
0
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 5.39 | 总 312/526 山雨欲来！（PID 1904）

出处：河南工大2024新生周赛（6）——命题人：魏方（G，CID 1097）| 训练定位：基础提高 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：42/91（46.2%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一个正整数，表示能够接到雨水的最大体积

输入要点：输入一个 $n(1 \leq n \leq 2 \times 10^4)$ ，表示柱子的数量，接下来输入 $n$ 个整数 $a(1 \leq a \leq 105)$ ，表示 $n$ 个柱子的高度。

输出要点：输出一个正整数，表示能够接到雨水的最大体积

零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：山雨欲来 动态规划 对于下标  $i$ ，下雨后水能到达的最大高度等于下标  $i$  两边的最大高度的最小值，下标  $i$  处能接的雨水量等于下标  $i$  处的水能到达的最大高度减去  $height[i]$ 。朴素的做法是对于数组  $height$  中的每个元素，分别向左和向右扫描并记录左边和右边的最大高度，然后计算每个下标位置能接的雨水量。假设数组  $height$  的长度为  $n$ ，该做法需要对每个下标位置使用  $O(n)$  的时间向两边扫描并得到最大高度 上述做法的时间复杂度较高是因为需要对每个下标位置都向两边扫描。如果已经知道每个位置两边的最大高度，则可以在  $O(n)$  的时间内得到能接的雨水总量。使用动态规划的方法，可以在  $O(n)$  的时间内预处理得到每个位置两边的最大高度。 创建两个长度为  $n$  的数组  $leftMax$  和  $rightMax$ 。对于  $0 \leq i < n$ ， $leftMax[i]$  表示下标  $i$  及其左边的位置中， $height$  的最大高度， $rightMax[i]$  表示下标  $i$  及其右边的位置中， $height$  的最大高度。显然， $leftMax[0]=height[0]$ ， $rightMax[n-1]=height[n-1]$ 。两个数组的其余元素的计算如下：当  $1 \leq i \leq n-1$  时， $leftMax[i]=\max(leftMax[i-1],height[i])$ ；当  $0 \leq i \leq n-2$  时， $rightMax[i]=\max(rightMax[i+1],height[i])$ 。因此可以正向遍历数组  $height$  得到数组  $leftMax$  的每个元素值，反向遍历数组  $height$  得到数组  $rightMax$  的每个元素值。在得到数组  $leftMax$  和  $rightMax$  的每个元素值之后，



对于  $0 \leq i < n$ ，下标  $i$  处能接的雨水量等于  $\min(\text{leftMax}[i], \text{rightMax}[i]) - \text{height}[i]$ 。遍历每个下标位置即可得到能接的雨水总量。本题与c语言版本不一样的仅有vector容器和max函数(取两个数的最大值)，与c语言中的数组起到的作用一样，所以不再提供c语言版本答案，知道vector相当与数组即可。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <vector>
3 | using namespace std;
4 | int trap(vector<int>& height) {
5 |     int n = height.size();
6 |     if (n == 0)
7 |         return 0;
8 |     vector<int> leftMax(n);
9 |     leftMax[0] = height[0];
10 |    for (int i = 1; i < n; ++i) {
11 |        leftMax[i] = max(leftMax[i - 1], height[i]);
12 |    }
13 |    vector<int> rightMax(n);
14 |    rightMax[n - 1] = height[n - 1];
15 |    for (int i = n - 2; i >= 0; --i) {
16 |        rightMax[i] = max(rightMax[i + 1], height[i]);
17 |    }
18 |    int ans = 0;
19 |    for (int i = 0; i < n; i++)
20 |        ans += min(leftMax[i], rightMax[i]) - height[i];
21 |    return ans;
22 | }
23 | int main() {
24 |     int n;
25 |     cin >> n;
26 |     vector<int> height(n);
27 |     for (int i = 0; i < n; i++) {
28 |         int a;
29 |         cin >> a;
30 |         height[i] = a;
31 |     }
32 |     cout << trap(height);
33 |     return 0;
34 | }
```

## 公开样例

样例 1 输入

```
12
0 1 0 2 1 0 1 3 2 1 2 1
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

5.40 | 总 313/526 数字躲猫猫 ( PID 1928 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( E , CID 1104 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数论 | 限制：1.000 S / 128 MB | 历史统计：82/332 ( 24.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，代表数字之和。

输入要点：输入两个整数 a 和 b， $0 \leq a, b \leq 1000000$ 。

输出要点：输出一个整数，代表数字之和。

零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：数字躲猫猫 注意a,b大小不确定，所以要先判断谁在前，然后循环把不带3的数字相加，注意每次相加都要取模。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | const int mod = 1e9 + 7;
4 | int main() {
5 |     int a, b;
6 |     scanf("%d %d", &a, &b);
7 |     if (a >= b) {
8 |         int t;
9 |         t = a;
10 |        a = b;
11 |        b = t;
12 |    }
13 |    int ans = 0;
14 |    for (int i = a; i <= b; i++) {
15 |        int x = i, f = 1;
16 |        while (x) {
17 |            int y = x % 10;
18 |            if (y == 3) {
19 |                f = 0;
20 |                break;
21 |            }
22 |            x /= 10;
```

```
23 |     }
24 |     if (f)
25 |         ans = (ans + i) % mod;
26 |     }
27 |     printf("%d", ans);
28 |     return 0;
29 | }
```

公开样例

样例 1 输入

3 5

样例 1 输出

9

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.41 | 总 314/526 小唐的运气 ( PID 1932 )

出处：河南工大2025新生周赛 ( 2 ) ——命题人：王伟立 ( D , CID 1103 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：333/757 ( 44.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，能够追上则输出yes 追不上时输出no

输入要点：第一行三个数x,a,b

第二行一个数t

输出要点：能够追上则输出yes

追不上时输出no

题面提示：所有数均保证在int范围内。

零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：小唐的运气  $(a-b)*t > x$  时能追上。 `#include <bits/stdc++.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

常数级或一次线性读入；以代码中的循环层数为准

易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | signed main() {
4 |     int x, a, b, t;
5 |     cin >> x >> a >> b >> t;
6 |     if ((a - b) * t >= x) {
7 |         cout << "yes" << endl;
8 |         return 0;
9 |     } else
10 |         cout << "no" << endl;
11 |     return 0;
12 | }
```

公开样例

样例 1 输入

100 1 1  
1000

样例 1 输出

no

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.42 | 总 315/526 小唐的签到 ( PID 1936 )

出处：河南工大2025新生周赛（2）——命题人：王伟立（A，CID 1103） | 训练定位：基础提高 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：411/1101 ( 37.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，能签到成功输出qiandao 不能成功输出bu

输入要点：一行数分别是a,x,n,b,y.

保证输入数据均在int范围内

输出要点：能签到成功输出qiandao

不能成功输出bu

题面提示：这是一道签到题

零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：小唐的签到 小唐到达教室的时间等于路上所用时间和上楼时间之和，注意如果教室在n楼,只需要上n-1层。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int a, x, n, b, y;
5 |     cin >> a >> x >> n >> b >> y;
6 |     long long res = (x + a - 1) / a + (n - 1) * b;
7 |     if (y >= res)
8 |         cout << "qiandao";
9 |     else
10 |         cout << "bu";
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

100 100 1 1000 1

样例 1 输出

qiandao

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.43 | 总 316/526 加and乘 ( PID 1969 )

出处：河南工大2025新生周赛 ( 6 ) ——命题人：张康发 ( F , CID 1107 ) | 训练定位：基础提高 | 数学、数论与组合

标签：数组与序列、数论 | 限制：1.000 S / 128 MB | 历史统计：32/99 ( 32.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对每个操作 3，按照它在输入中出现的顺序，依次输出一行一个整数表示询问结果（和模P的值）。

输入要点：第一行两个整数N和P；

第二行含有N个非负整数，从左到右依次为 $a_1, a_2, \dots, a_N$ ；

第三行一个整数M，表示操作总数；

从第四行开始每行描述一个操作，格式为1 t g c、2 t g c或3 t g。

输出要点：对每个操作3，按照它在输入中出现的顺序，依次输出一行一个整数表示询问结果（和模P的值）。

题面提示： $1 \leq N, M \leq 105$

$1 \leq t \leq g \leq N$

$0 \leq c, a_i \leq 109$

$1 \leq P \leq 109$

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：加and乘 根据题目模拟即可。#include <iostream> #include <vector>
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <vector>
3 | using namespace std;
4 | int main() {
5 |     int N, P;
6 |     cin >> N >> P;
7 |     vector<long long> a(N + 1);
8 |     for (int i = 1; i <= N; ++i) {
9 |         cin >> a[i];
10 |         a[i] %= P;
11 |     }
12 |     int M;
13 |     cin >> M;
14 |     while (M--) {
15 |         int op;
16 |         cin >> op;
17 |         if (op == 1) {
18 |             int t, g, c;
19 |             cin >> t >> g >> c;
```

```

20 |         c %= P;
21 |         for (int i = t; i <= g; ++i) {
22 |             a[i] = (a[i] * c) % P;
23 |         }
24 |     } else if (op == 2) {
25 |         int t, g, c;
26 |         cin >> t >> g >> c;
27 |         c %= P;
28 |         for (int i = t; i <= g; ++i) {
29 |             a[i] = (a[i] + c) % P;
30 |             if (a[i] < 0)
31 |                 a[i] += P;
32 |         }
33 |     } else if (op == 3) {
34 |         int t, g;
35 |         cin >> t >> g;
36 |         long long sum = 0;
37 |         for (int i = t; i <= g; ++i) {
38 |             sum = (sum + a[i]) % P;
39 |         }
40 |         cout << sum % P << endl;
41 |     }
42 | }
43 | return 0;
44 | }

```

## 公开样例

### 样例 1 输入

```

7 43
1 2 3 4 5 6 7
5
1 2 5 5
3 2 4
2 3 7 9
3 1 3
3 4 7

```

### 样例 1 输出

```

2
35
8

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.44 | 总 317/526 数组求和 ( PID 1978 )

出处：河南工大2025新生周赛（6）——命题人：张康发（E，CID 1107） | 训练定位：基础提高 | 数学、数论与组合

标签：数组与序列、数论、字符串 | 限制：1.000 S / 128 MB | 历史统计：83/173 ( 48.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：输入一个二维数组 $M[12][12]$ ，根据输入要求，求出二维数组的左下部分元素的平均值或元素的和。

输入要点：第一行输入一个大写字母，若为 S，则表示需要求出左下半部分的元素的和，若为 M，则表示需要求出左下半部分的元素的平均值。

接下来 12 行，每行包含 12 个用空格隔开的浮点数，表示这个二维数组，其中第  $i+1$  行的第  $j+1$  个数表示数组元素  $M[i][j]$ 。

输出要点：输出一个数，表示所求的平均数或和的值，保留一位小数。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：数组求和 遍历求和即可。#include <stdio.h> #include <string.h>
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int main() {
4 |     char s[2];
5 |     double t, ans = 0.0;
6 |     scanf("%s", s);
7 |     for (int i = 0; i < 12; i++) {
8 |         for (int j = 0; j < 12; j++) {
9 |             scanf("%lf", &t);
10 |             if (i > j) {
11 |                 ans += t;
12 |             }
13 |         }
14 |     }
15 |     if (strcmp(s, "S") == 0) {
16 |         printf("%.1f\n", ans);
17 |     } else {
18 |         printf("%.1f\n", ans / 66);
19 |     }
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
S
8.7 5.6 -2.0 -2.1 -7.9 -9.0 -6.4 1.7 2.9 -2.3 8.4 4.0
-7.3 -2.1 0.6 -9.8 9.6 5.6 -1.3 -3.8 -9.3 -8.0 -2.0 2.9
-4.9 -0.5 -5.5 -0.2 -4.4 -6.1 7.6 6.9 -8.0 6.8 9.1 -8.5
-1.3 5.5 4.6 6.6 8.1 7.9 -9.3 9.6 4.6 0.9 -3.5 -4.3
-7.0 -1.2 7.0 7.1 -5.7 7.8 -2.3 4.3 0.2 -0.4 -6.6 7.6
-3.2 -5.4 -4.7 4.7 3.6 8.8 5.1 -3.1 -2.9 2.8 -4.3 -1.4
-1.8 -3.3 -5.6 1.8 8.3 -0.5 2.0 -3.9 -1.0 -8.6 8.0 -3.3
-2.5 -9.8 9.2 -0.8 -9.4 -0.5 1.6 1.5 3.4 -0.1 7.0 -6.2
-1.0 4.9 2.2 -8.7 -0.9 4.8 2.3 2.0 -3.2 -7.5 -4.0 9.9
-1.1 -2.9 8.7 3.6 7.4 7.8 10.0 -9.0 1.6 8.3 6.3 -5.8
-9.9 0.6 2.0 -3.8 -6.3 0.6 7.3 3.8 -7.1 9.5 2.2 1.3
-2.8 -9.1 7.1 -0.2 0.6 -6.5 1.1 -0.1 -3.6 4.0 -5.4 1.1
```

样例 1 输出



## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

## 5.45 | 总 318/526 韩信点兵 (PID 1202)

出处：2016级河南工大新生周赛（一）（E，CID 1006）| 训练定位：进阶 | 数学、数论与组合

标签：同余、最小公倍数、\_\_int128 | 限制：1.000 S / 128 MB | 历史统计：4/202 (2.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出最小答案，如果无法得出科学的答案，则输出"Wrong count!"

输入要点：输入四个正整数对应题目中的  $a, b, c, n$  ( $2 \leq a, b, c \leq 28000, n < 1000000000$ )

输出要点：输出最小答案，如果无法得出科学的答案，则输出"Wrong count!"

## 零基础先修

- \_\_int128 可保存约 38 位有符号整数，但输入输出需要自行转换。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：人数  $x$  对  $a, b, c$  都余 1，等价于  $x-1$  同时被三数整除，所以  $x=1+k \cdot \text{lcm}(a, b, c)$ 。军队至少 2 人，最小取  $k=1$ ，即  $\text{lcm}(a, b, c)+1$ 。若它超过给定上限  $n$ ，则没有科学答案。计算最小公倍数时使用 \_\_int128，避免中间乘法溢出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\log \max(a, b, c))$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using i128 = __int128_t;
4 | i128 lcm128(i128 a, i128 b) {
5 |     long long x = (long long)a;
6 |     long long y = (long long)b;
7 |     return a / gcd(x, y) * b;
8 | }
9 | int main() {
```

```

10 |     long long a, b, c, limit;
11 |     cin >> a >> b >> c >> limit;
12 |     i128 common = lcm128(lcm128(a, b), c);
13 |     i128 ans = common + 1;
14 |     if (ans > limit)
15 |         cout << "Wrong count!\n";
16 |     else
17 |         cout << (long long)ans << '\n';
18 |     return 0;
19 | }

```

## 公开样例

样例 1 输入

2 3 5 100000

样例 1 输出

31

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.46 | 总 319/526 Simple学长翻硬币 ( PID 1253 )

出处：2016级河南工大新生周赛总决赛（D，CID 1016） | 训练定位：进阶 | 数学、数论与组合

标签：因数、完全平方数、数学 | 限制：1.000 S / 128 MB | 历史统计：11/158 ( 7.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出正面朝上硬币的编号（编号从1开始），每两个编号用空格隔开。

输入要点：一个T，表示有T组数据

每组数据一个n，表示Simple学长会翻n次（ $0 \leq n \leq 1e6$ ）

输出要点：输出正面朝上硬币的编号（编号从1开始），每两个编号用空格隔开。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：编号 i 的硬币会在 i 的每个正因数轮被翻一次，因此总翻转次数等于 i 的因数个数。因数通常成对出现 d 与 i/d，只有完全平方数的平方根会落单，所以恰好完全平方数被翻奇数次、最终正面朝上。直接输出  $1^2, 2^2, \dots, n$ ；用标志控制空格，避免行尾多余分隔。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(\sqrt{n})$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long n;
10 |        cin >> n;
11 |        bool first = true;
12 |        for (long long root = 1; root * root <= n; ++root) {
13 |            if (!first)
14 |                cout << ' ';
15 |            cout << root * root;
16 |            first = false;
17 |        }
18 |        cout << '\n';
19 |    }
20 |    return 0;
21 | }
```

## 公开样例

样例 1 输入

```
1
5
```

样例 1 输出

```
14
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.47 | 总 320/526 Simple学长的礼物 ( PID 1254 )

出处：2016级河南工大新生周赛总决赛 ( E , CID 1016 ) | 训练定位：进阶 | 数学、数论与组合

标签：分类讨论、奇偶性、构造 | 限制：1.000 S / 128 MB | 历史统计：32/231 ( 13.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，In a single line print "YES" (without the quotes) if it is possible to divide all the apples between his friends. Otherwise print "NO" (without the quotes).

**输入要点：**The first T indicates there are T groups of date.

The first line of every group contains an integer n ( $1 \leq n \leq 100$ ) — the number of apples. The second line contains n integers  $w_1, w_2, \dots, w_n$  ( $w_i = 100$  or  $w_i = 200$ ), where  $w_i$  is the weight of the i-th apple.

输出要点：In a single line print "YES" (without the quotes) if it is possible to divide all the apples between his friends. Otherwise print "NO" (without the quotes).

## 零基础先修

- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：把重量除以 100，只剩 1 和 2。总重量必须能平分，因此 100 克苹果 个数必须为偶数。若至少有两只 100 克苹果，它们还能修正一只 200 克带来的奇偶差；若没有 100 克苹果，则 200 克苹果本身必须是偶数，才能两边各放一半。故条件为 `count100 为偶数` 且 `count100>0 或 count200 为偶数`。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  空间

## 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, count100 = 0, count200 = 0;
10 |         cin >> n;
11 |         for (int i = 0; i < n; ++i) {
12 |             int weight;
13 |             cin >> weight;
14 |             if (weight == 100)
15 |                 ++count100;
16 |             else
17 |                 ++count200;
18 |         }
19 |         bool possible = count100 % 2 == 0 && (count100 > 0 || count200 % 2 == 0);
20 |         cout << (possible ? "YES" : "NO") << '\n';
21 |     }
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

```
2
3
100 200 100
4
100 100 100 200
```

样例 1 输出

```
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.48 | 总 321/526 Simple学长的数学题 ( PID 1255 )

出处：2016级河南工大新生周赛总决赛 ( F , CID 1016 ) | 训练定位：进阶 | 数学、数论与组合

标签：解析几何、分数化简、\_\_int128、格式化输出 | 限制：1.000 S / 128 MB | 历史统计：1/37 ( 2.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：Simple学长在学数学的时候遇到了一个难题，根据点A(X1,Y1),点B(X2,Y2)的坐标求出直线AB的函数表达式，Simple学长半天也没做出来，希望你们帮他一下。

输入要点：共2\*n行(n<=10)

每两行两个整数对，分别表示A,B的坐标

A,B在X轴,Y轴上且 $-2^{32} \leq \text{坐标值} \leq 2^{32}$

输出要点：输出格式见样例(即 $y=kx+b(k,b \text{ 为常数})$ )，你的程序必须保证：一次项写在前面；

遇到小数按照最简分数输出

如果数据出错(即没有或多个的一次函数表达式，请输出"Error"不含引号)

题面提示：斜率不存在或者斜率为0输出Error

### 零基础先修

- \_\_int128 可保存约 38 位有符号整数，但输入输出需要自行转换。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3核心处理采用：两点式给出斜率 $k=(y_2-y_1)/(x_2-x_1)$ 和截距 $b=(x_2 \cdot y_1 - x_1 \cdot y_2)/(x_2 - x_1)$ 。竖线 $dx=0$ 或水平线 $dy=0$ 按题意输出Error。其余把 $k$ 、 $b$ 的分子分母分别用 gcd 约成最简分数，统一让分母 为正；系数  $1/-1$  分别省写为  $'x'/'-x'$ ，截距为 0 时省略。坐标乘积 可能越过有符号 64 位，使用 \_\_int128。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(\log C)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using i128 = __int128_t;
4 | i128 abs128(i128 x) {
5 |     return x < 0 ? -x : x;
6 | }
7 | i128 gcd128(i128 a, i128 b) {
8 |     a = abs128(a);
9 |     b = abs128(b);
10 |    while (b != 0) {
11 |        i128 r = a % b;
12 |        a = b;
13 |        b = r;
14 |    }
15 |    return a;
16 | }
17 | string toStr(i128 x) {
18 |     if (x == 0)
19 |         return "0";
20 |     bool neg = x < 0;
21 |     x = abs128(x);
22 |     string s;
23 |     while (x > 0) {
24 |         s.push_back(char('0' + x % 10));
25 |         x /= 10;
26 |     }
27 |     if (neg)
28 |         s.push_back('-');
29 |     reverse(s.begin(), s.end());
30 |     return s;
31 | }
32 | struct Frac {
33 |     i128 p, q;
34 |     Frac(i128 n, i128 d) {
35 |         if (d < 0)
36 |             n = -n, d = -d;
37 |         i128 g = gcd128(n, d);
38 |         p = n / g;
39 |         q = d / g;
40 |     }
41 |     string str() const {
42 |         i128 n = abs128(p);
43 |         if (q == 1)
44 |             return toStr(n);
45 |         return toStr(n) + "/" + toStr(q);
46 |     }
47 | };
48 | int main() {
49 |     ios::sync_with_stdio(false);
50 |     cin.tie(nullptr);
51 |     long long xa, ya, xb, yb;
52 |     while (cin >> xa >> ya >> xb >> yb) {
53 |         i128 x1 = xa, y1 = ya, x2 = xb, y2 = yb;
54 |         i128 dx = x2 - x1, dy = y2 - y1;
55 |         if (dx == 0 || dy == 0) {
56 |             cout << "Error\n";
57 |             continue;
58 |         }
59 |         Frac k(dy, dx);
60 |         Frac bias(x2 * y1 - x1 * y2, dx);
61 |         cout << "y=";
62 |         if (k.p < 0)
63 |             cout << '-';
64 |         if (abs128(k.p) != k.q)
65 |             cout << k.str();
66 |         cout << 'x';
67 |         if (bias.p != 0) {
68 |             cout << (bias.p > 0 ? '+' : '-');
69 |             cout << bias.str();
70 |         }
71 |         cout << '\n';
72 |     }
73 |     return 0;
74 | }
```

## 公开样例

样例 1 输入

```
0 1
1 0
0 2
-1 0
```

样例 1 输出

```
y=-x+1
y=2x+2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.49 | 总 322/526 趣味游戏 ( 一 ) : 喝饮料 ( PID 1329 )

出处：2017级河南工大新生周赛 ( 四 ) ( A , CID 1031 ) | 训练定位：进阶 | 数学、数论与组合

标签：最大公约数、最小公倍数、建模 | 限制：1.000 S / 128 MB | 历史统计：118/657 ( 18.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组样例输出一个整数表示需要买的饮料数目

输入要点：第一行一个整数  $T$  表示测试样例的组数。

每组样例第一行两个数字  $n$  和  $m$ ；(  $0 \leq n \leq 100000, 0 < m \leq 100000$  )

输出要点：每组样例输出一个整数表示需要买的饮料数目

题面提示：包括小华本身 喝饮料的有  $n+1$  人

仔细读题,注意数据范围

## 零基础先修

- `std::gcd(a,b)` 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：喝饮料的共有  $n+1$  人。买  $x$  瓶可得  $x \cdot m$  杯；既要每人分到相同的正整数杯数，又不能剩余，就要求  $x \cdot m$  是  $n+1$  的倍数。  
最小总杯数是  $\text{lcm}(n+1,m)$ ，所以最少瓶数为  $\text{lcm}/m=(n+1)/\gcd(n+1,m)$ 。用这个化简 还能避免先求最小公倍数时的乘法溢出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(\log \min(n,m))$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 | ios::sync_with_stdio(false);
5 | cin.tie(nullptr);
6 | int T;
7 | cin >> T;
8 | while (T--) {
9 |     long long friends, per;
10 |    cin >> friends >> per;
11 |    long long people = friends + 1;
12 |    cout << people / gcd(people, per) << '\n';
13 | }
14 | return 0;
15 | }

```

## 公开样例

样例 1 输入

```

2
16
26

```

样例 1 输出

```

1
1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.50 | 总 323/526 趣味游戏 (二) : 铺地板 (PID 1330)

出处：2017级河南工大新生周赛 (四) (B, CID 1031) | 训练定位：进阶 | 数学、数论与组合

标签：向上取整、数学、面积覆盖 | 限制：1.000 S / 128 MB | 历史统计：46/214 (21.5%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：听说大家喜欢玩富含创造性的《我的世界》游戏，并且立志要成为一个可以在《我的世界》中做出超级计算机的大神级玩家，想要成为一位可以做出超级计算机的大神首先要从小事做起。现在某玩家遇到一个装修的问题，他知道你是一位可以搭载出超级计算机的大神，所以他认为你可以帮助他解决问题。他想为自己现实中长度为  $n$  宽为  $m$  的小屋铺上精美的地板。由于这个玩家不想看到一块地板砖……

输入要点：第一行一个整数  $T$  表示测试样例的数量

每组样例第一行 两个数字  $n$  和  $m$ ， $n$  ( $0 < n \leq 1000000000$ ) 表示小屋的长  $m$  ( $0 < m \leq 1000000000$ ) 表示小屋的宽

每组样例第二行 一个数字  $a$  ( $0 < a \leq 10$ ) 表示地板的长度

输出要点：每组样例输出一个整数表示需要地板的数量

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。



- 核心处理采用：每块地板是  $a \times a$  且不能重复使用。长度方向至少需要  $\text{ceil}(n/a)$  块，宽度方向至少需要  $\text{ceil}(m/a)$  块，按规则网格铺设可以同时达到这两个下界，因此相乘就是答案。整数向上取整用  $\lceil (length+a-1)/a \rceil$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long n, m, a;
10 |         cin >> n >> m >> a;
11 |         long long rows = (n + a - 1) / a;
12 |         long long columns = (m + a - 1) / a;
13 |         cout << rows * columns << '\n';
14 |     }
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入

```
2
10 10
3
15 10
3
```

样例 1 输出

```
16
20
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.51 | 总 324/526 多米诺骨牌 ( PID 1338 )

出处：2017级河南工大新生周赛（五）（D，CID 1033） | 训练定位：进阶 | 数学、数论与组合

标签：数学、构造、整数除法 | 限制：1.000 S / 128 MB | 历史统计：77/351 ( 21.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示放入的多米诺骨牌的最大数量。

输入要点：第一行：T，测试实例数。

第二行两个整数M和N。 ( $1 \leq M \leq N \leq 10^9$ )。

输出要点：一个整数，表示放入的多米诺骨牌的最大数量。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：每块多米诺覆盖两个单位格，所以数量不可能超过  $\text{floor}(MN/2)$ 。沿行（或列）成对铺放即可达到这个上界：面积为偶数时铺满，面积为奇数时只剩一个格。因此答案就是整数除法  $M*N/2$ 。乘积最大  $10^{18}$ ，要用 long long。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long m, n;
10 |         cin >> m >> n;
11 |         cout << m * n / 2 << '\n';
12 |     }
13 |     return 0;
14 | }
```

### 公开样例

样例 1 输入

```
2
2 4
3 3
```

样例 1 输出

```
4
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.52 | 总 325/526 Choice的加法 ( PID 1347 )

出处：2017级河南工大新生周赛 ( 七 ) ( A , CID 1036 ) | 训练定位：进阶 | 数学、数论与组合

标签：高精度、字符串、进位 | 限制：1.000 S / 128 MB | 历史统计：19/297 ( 6.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出A+B的结果

输入要点：输入数据有多组。

每组一行输入两个整数A，B

A和B的长度小于1000 ( A，B都是非负数 )

输出要点：输出A+B的结果

### 零基础先修

- 超过 64 位范围时用字符串逐位运算，或使用题目允许的大整数工具。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：从两个十进制字符串的末尾开始逐位相加，同时维护进位 carry。某个字符串已经用完时把该位当 0；当前位写入  $(x+y+carry)\%10$ ，新进位为整除 10 的结果。最后若仍有进位再补一位，并将倒序收集的 数字翻转。这样可处理近 1000 位的非负整数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(\max(|A|,|B|))$  时间与空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | string addStrings(const string& a, const string& b) {
4 |     int i = (int)a.size() - 1;
5 |     int j = (int)b.size() - 1;
6 |     int carry = 0;
7 |     string reversed;
8 |     while (i >= 0 || j >= 0 || carry) {
9 |         int x = i >= 0 ? a[i--] - '0' : 0;
10 |        int y = j >= 0 ? b[j--] - '0' : 0;
11 |        int sum = x + y + carry;
12 |        reversed.push_back(char('0' + sum % 10));
13 |        carry = sum / 10;
14 |    }
15 |    reverse(reversed.begin(), reversed.end());
16 |    return reversed;
17 | }
18 | int main() {
19 |     ios::sync_with_stdio(false);
20 |     cin.tie(nullptr);
21 |     string A, B;
22 |     while (cin >> A >> B)
23 |         cout << addStrings(A, B) << '\n';
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

```
1 1
1 2
```

样例 1 输出

```
2
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 5.53 | 总 326/526 Choice的心情 ( PID 1348 )

出处：2017级河南工大新生周赛（七）（B，CID 1036） | 训练定位：进阶 | 数学、数论与组合

标签：组合计数、快速幂、取模 | 限制：1.000 S / 128 MB | 历史统计：1/27 ( 3.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出Choice不开心的状态数，答案模100003取余

输入要点：输入数据有多组。

每组一行输入两个整数M，N.  $1 \leq M \leq 10^5, 1 \leq N \leq 10^9$

输出要点：输出Choice不开心的状态数，答案模100003取余

## 零基础先修

- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：全部放法有  $M^N$  种。Choice 开心意味着任意相邻箱子不同：第一个箱子有  $M$  种，之后每个箱子只有  $M-1$  种，所以开心方案为  $M \cdot (M-1)^{(N-1)}$ 。不开心数是两者之差。N 达  $10^9$ ，使用二进制快速幂取模 100003；作差后再加模数防止负数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(\log N)$  时间、 $O(1)$  空间

## 易错点

- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const long long MOD = 100003;
4 | long long modPower(long long base, long long exponent) {
5 |     long long res = 1;
6 |     base %= MOD;
7 |     while (exponent > 0) {
8 |         if (exponent & 1)
9 |             res = res * base % MOD;
10 |         base = base * base % MOD;
11 |         exponent >>= 1;
12 |     }
13 |     return res;
14 | }
15 | int main() {
16 |     ios::sync_with_stdio(false);
17 |     cin.tie(nullptr);
18 |     long long m, n;
19 |     while (cin >> m >> n) {
20 |         long long total = modPower(m, n);
21 |         long long happy = m % MOD * modPower(m - 1, n - 1) % MOD;
22 |         cout << (total - happy + MOD) % MOD << '\n';
23 |     }
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

2 3

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.54 | 总 327/526 Choice的箱子 ( PID 1349 )

出处：2017级河南工大新生周赛（七）（C，CID 1036） | 训练定位：进阶 | 数学、数论与组合

标签：容斥原理、最大公约数、最小公倍数 | 限制：1.000 S / 128 MB | 历史统计：1/39 ( 2.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出需要检查箱子的数目。

输入要点：第一行输入整数T，代表测试组数。（T不小于10）

每组一行输入三个整数n,a,b,c( $1 \leq n \leq 1e8, 1 \leq a,b,c \leq 1e5$ )

输出要点：输出需要检查箱子的数目。

题面提示：案例中最后打开箱子的编号为2,3,4,5,8,9，所以有6个。

### 零基础先修

- std::gcd(a,b) 返回最大公约数，并满足  $\text{gcd}(a,b)=\text{gcd}(b,a \bmod b)$ 。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：箱子最后打开，当且仅当它被 a、b、c 三组操作覆盖了奇数次。设 A、B、C 为相应倍数集合。把单集计数相加后，两集合交集的元素 被算了两次但应为 0，故每个两两交集减 2 次；三重交集经历三次操作 应保留，而前三步合计给它系数 -3，所以再加 4 次。答案为  $|A|+|B|+|C|-2(|AB|+|AC|+|BC|)+4|ABC|$ ，交集用最小公倍数计数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(\log \max(a,b,c))$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long safeLcm(long long a, long long b, long long limit) {
4 |     __int128 value = (__int128)(a / gcd(a, b)) * b;
5 |     return value > limit ? limit + 1 : (long long)value;
6 | }
7 | long long multiples(long long n, long long divisor) {
8 |     return divisor > n ? 0 : n / divisor;
9 | }
10 | int main() {
11 |     ios::sync_with_stdio(false);
12 |     cin.tie(nullptr);
13 |     int T;
14 |     cin >> T;
15 |     while (T--) {
16 |         long long n, a, b, c;
17 |         cin >> n >> a >> b >> c;
18 |         long long ab = safeLcm(a, b, n);
19 |         long long ac = safeLcm(a, c, n);
20 |         long long bc = safeLcm(b, c, n);
```

```

21 |         long long abc = safeLcm(ab, c, n);
22 |         long long ans = multiples(n, a) + multiples(n, b) + multiples(n, c) -
23 |             2 * (multiples(n, ab) + multiples(n, ac) + multiples(n, bc)) +
24 |             4 * multiples(n, abc);
25 |         cout << ans << '\n';
26 |     }
27 |     return 0;
28 | }

```

## 公开样例

样例 1 输入

```

1
10 2 3 5

```

样例 1 输出

```

6

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 5.55 | 总 328/526 N ! ( PID 1353 )

出处：2017级河南工大新生周赛（七）（G，CID 1036） | 训练定位：进阶 | 数学、数论与组合

标签：阶乘、对数、数值处理、预处理 | 限制：1.000 S / 128 MB | 历史统计：0/18 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，每组案例占一行，输出 $n!$ 的第一个非零数和最后一个非零数用空格隔开。

**输入要点：**输入多组案例。

每组案例一行，输入一个正整数 $n$  ( $1 \leq n \leq 10000$ )

**输出要点：**每组案例占一行，输出 $n!$ 的第一个非零数和最后一个非零数用空格隔开。

题面提示：第二个案例 $5! \rightarrow 120$ ，所以答案为1 2

## 零基础先修

- 先从定义和小数据找规律，再用整除、奇偶、最大公约数、取模或组合计数证明公式。特别注意整数溢出与浮点误差。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：首位用对数求： $\log_{10}(n!) = \sum \log_{10}(i)$ ，其小数部分 $f$ 满足首位为 $\text{floor}(10^f)$ 。末位非零数用缩减乘积：依次乘 $i$ ，每步把末尾的0全部除掉，再只保留较低若干位，最后个位就是所求。 $n \leq 10000$ ，可以预处理1..10000的两个答案，之后每个文件尾询问 $O(1)$ 输出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

预处理  $O(10000)$ ，每个询问  $O(1)$ ，空间  $O(10000)$

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const int LIMIT = 10000;
7 |     vector<int> firstDigit(LIMIT + 1), lastDigit(LIMIT + 1);
8 |     long double logarithm = 0.0L;
9 |     long long tail = 1;
10 |    for (int n = 1; n <= LIMIT; ++n) {
11 |        logarithm += log10((long double)n);
12 |        long double fractional = logarithm - floor(logarithm);
13 |        firstDigit[n] = (int)(pow((long double)10.0, fractional) + 1e-12L);
14 |        tail *= n;
15 |        while (tail % 10 == 0)
16 |            tail /= 10;
17 |        tail %= 100000000LL;
18 |        lastDigit[n] = tail % 10;
19 |    }
20 |    int n;
21 |    while (cin >> n)
22 |        cout << firstDigit[n] << ' ' << lastDigit[n] << '\n';
23 |    return 0;
24 | }
```

## 公开样例

样例 1 输入

```
1
5
```

样例 1 输出

```
1 1
1 2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.56 | 总 329/526 午饭问题 (一) (PID 1357)

出处：2017级河南工大新生周赛（八）（D，CID 1037）| 训练定位：进阶 | 数学、数论与组合

标签：高精度、字符串、竖式乘法 | 限制：1.000 S / 128 MB | 历史统计：4/20 (20.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，多实例测试，输出  $A*B$  的结果。

输入要点：两个整数  $A$  和  $B$ ， $A$ 和 $B$ 的长度 $l$  ( $0 \leq l \leq 1000$ )。

输出要点：多实例测试，输出  $A*B$  的结果。

## 零基础先修

- 超过 64 位范围时用字符串逐位运算，或使用题目允许的大整数工具。



- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：两个整数最多 1000 位，内置整数装不下。按小学竖式建立长度  $n+m$  的数组：从最低位开始枚举  $A[i]$ 、 $B[j]$ ，把乘积累加到对应位；再从低位向高位统一进位。去掉最高端多余的 0 后反向输出。输入按 两行一对一直读到文件尾，任何一个因数为 0 时自然得到 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(|A| \cdot |B|)$  时间、 $O(|A| + |B|)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | string multiply(const string& a, const string& b) {
4 |     vector<int> digit(a.size() + b.size(), 0);
5 |     for (int i = (int)a.size() - 1; i >= 0; --i) {
6 |         for (int j = (int)b.size() - 1; j >= 0; --j) {
7 |             int pos = (int)a.size() - 1 - i + (int)b.size() - 1 - j;
8 |             digit[pos] += (a[i] - '0') * (b[j] - '0');
9 |         }
10 |    }
11 |    for (int i = 0; i + 1 < (int)digit.size(); ++i) {
12 |        digit[i + 1] += digit[i] / 10;
13 |        digit[i] %= 10;
14 |    }
15 |    while (digit.size() > 1 && digit.back() == 0)
16 |        digit.pop_back();
17 |    string res;
18 |    for (auto it = digit.rbegin(); it != digit.rend(); ++it)
19 |        res.push_back(char('0' + *it));
20 |    return res;
21 | }
22 | int main() {
23 |     ios::sync_with_stdio(false);
24 |     cin.tie(nullptr);
25 |     string A, B;
26 |     while (cin >> A >> B)
27 |         cout << multiply(A, B) << '\n';
28 |     return 0;
29 | }
```

## 公开样例

### 样例 1 输入

```
4038
5255
55718
7221
8160
104906
3511906
8767574
```

### 样例 1 输出

```
21219690
402339678
856032960
30790895736044
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.57 | 总 330/526 铺铺铺铺铺地板 ( PID 1388 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( A , CID 1041 ) | 训练定位：进阶 | 数学、数论与组合

标签：向上取整、数学、大整数 | 限制：1.000 S / 128 MB | 历史统计：298/1387 ( 21.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：乌拉拉喜欢旅游，最新的一项意大利研究发现，每年旅游3次，会使人心花怒放，神龙马壮，精神抖擞，六脉调和。一次，乌拉拉到了一个还没有完工的广场，这个广场的面积为  $n*m$ ，工人们要用面积为  $a*a$  的正方形大理石板铺满广场，大理石板可以超出广场的面积范围，乌拉拉想计算出铺满这个广场至少需要多少大理石板？

输入要点：输入一行，包含三个整数： $n, m, a$  ( $1 \leq n, m, a \leq 1000\ 000\ 000$ )

输出要点：一个整数，即至少需要多少大理石板

题面提示：注意看数据范围！

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：长度和宽度方向分别至少需要  $\text{ceil}(n/a)$ 、 $\text{ceil}(m/a)$  块边长为  $a$  的石板；允许越界，因此规则网格铺放能达到这个下界。两者相乘即可，数据到  $1e9$ ，乘积要用 `long long`。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
```

```

5 |     cin.tie(nullptr);
6 |     long long n, m, a;
7 |     cin >> n >> m >> a;
8 |     cout << ((n + a - 1) / a) * ((m + a - 1) / a) << '\n';
9 |     return 0;
10| }

```

## 公开样例

样例 1 输入

6 6 4

样例 1 输出

4

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.58 | 总 331/526 容斥原理 ( PID 1398 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( C , CID 1042 ) | 训练定位：进阶 | 数学、数论与组合

标签：容斥原理、最小公倍数、数论 | 限制：1.000 S / 128 MB | 历史统计：5/217 ( 2.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出 1~n 中能被 a 或 b 或 c 整除的数的个数，每组占一行

输入要点：每组数据占一行，依次包括整数 n, a, b, c，直到输入 0 0 0 0 为止

(1 ≤ a, b, c ≤ n ≤ 1e9)

输出要点：输出 1~n 中能被 a 或 b 或 c 整除的数的个数，每组占一行

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：用容斥原理：三个单集计数相加，减去三个两两交集，再加三者交集。能同时被若干数整除等价于被它们的最小公倍数整除。计算 lcm 时使用 \_\_int128 并在超过 n 后截断，避免乘法溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(\log \text{ 值域})$  时间、 $O(1)$  空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using i128 = __int128_t;
4 | long long cappedLcm(long long a, long long b, long long cap) {
5 |     i128 value = (i128)(a / gcd(a, b)) * b;
6 |     return value > cap ? cap + 1 : (long long)value;
7 | }
8 | int main() {
9 |     long long n, a, b, c;
10 |    while (cin >> n >> a >> b >> c && (n || a || b || c)) {
11 |        long long ab = cappedLcm(a, b, n);
12 |        long long ac = cappedLcm(a, c, n);
13 |        long long bc = cappedLcm(b, c, n);
14 |        long long abc = cappedLcm(ab, c, n);
15 |        auto divisible = [&](long long d) {
16 |            return d > n ? 0LL : n / d;
17 |        };
18 |        long long ans = n / a + n / b + n / c - divisible(ab) - divisible(ac) -
19 |            divisible(bc) + divisible(abc);
20 |        cout << ans << '\n';
21 |    }
22 |    return 0;
23 | }
```

## 公开样例

样例 1 输入

```
1000 2 3 5
100 2 3 5
0 0 0 0
```

样例 1 输出

```
734
74
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.59 | 总 332/526 点赞分能量 ( PID 1401 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( F , CID 1042 ) | 训练定位：进阶 | 数学、数论与组合

标签：数学、比例分配、边界条件 | 限制：1.000 S / 128 MB | 历史统计：98/751 ( 13.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出占一行，分别输出每个人分得的能量（向下取整），中间用空格隔开

输入要点：第一行是战队的总能量整数  $n$  (  $100 \leq n \leq 100000$  )

第2~6行分别是每个人 为战队贡献的点赞数  $x_i$ 。 (  $0 \leq x_i \leq 100000$  )

输出要点：输出占一行，分别输出每个人分得的能量（向下取整），中间用空格隔开

题面提示：若两个人贡献的点赞相等，则他们分得的能量值相等。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：点赞总数非零时，第  $i$  人得到  $\text{floor}(n * \text{likes}[i] / \text{sumLikes})$ ，乘法 先用 long long。若五人点赞全为 0，大家贡献相等，按均分规则每人 得到  $\text{floor}(n/5)$ ，这也是官方题解特别提醒的零分母边界。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long energy;
5 |     cin >> energy;
6 |     array<long long, 5> likes{};
7 |     for (long long& x : likes)
8 |         cin >> x;
9 |     long long total = accumulate(likes.begin(), likes.end(), 0LL);
10 |    for (int i = 0; i < 5; ++i) {
11 |        if (i)
12 |            cout << ' ';
13 |        if (total == 0)
14 |            cout << energy / 5;
15 |        else
16 |            cout << energy * likes[i] / total;
17 |    }
18 |    cout << '\n';
19 |    return 0;
20 | }
```

## 公开样例

样例 1 输入

```
8000
50
30
10
10
0
```

样例 1 输出

```
4000 2400 800 800 0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.60 | 总 333/526 Balance ( PID 1419 )

出处：2018级河南工大新生周赛总决赛（重现）（C，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（C，CID 1048） | 训练定位：进阶 | 数学、数论与组合

标签：信息论、天平问题、数学 | 限制：1.000 S / 128 MB | 历史统计：8/114 ( 7.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，对于每一个测试数据，输出至少需要称量多少次，如果永远无法得到劣质小球是哪一个，输出-1

输入要点：多组测试数据（不超过25组）

每行输入一个数字 $n$ 表示小球的数量（ $1 \leq n \leq 25$ ）

输出要点：对于每一个测试数据，输出至少需要称量多少次，如果永远无法得到劣质小球是哪一个，输出-1

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：劣质球可能偏重也可能偏轻，但只需确定“是哪一只”。一次天平称量有左重、平衡、右重三种结果。 $k$ 次称量能球的身份提供的最大容量是  $(3^k - 1)/2$ ；逐步扩大  $k$ ，找到容量首次不少于  $n$  的值即可。两个球是唯一例外：互相称只能知道一重一轻，却无法判断哪只才是异物，又没有第三只已知正常球作参照，所以永远无法确定。一个球无需称量。在本题  $n \leq 25$  内也可核对为：1→0，2→-1，3..4→2，5..13→3，14..25→4。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(\log n)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n) {
8 |         if (n == 1) {
9 |             cout << 0 << '\n';
10 |             continue;
11 |         }
12 |         if (n == 2) {
13 |             cout << -1 << '\n';
```

```

14 |         continue;
15 |     }
16 |     int weighings = 0;
17 |     long long p3 = 1;
18 |     while ((p3 - 1) / 2 < n) {
19 |         p3 *= 3;
20 |         ++weighings;
21 |     }
22 |     cout << weighings << '\n';
23 | }
24 | return 0;
25 | }

```

## 公开样例

样例 1 输入

```

3
4
12

```

样例 1 输出

```

2
2
3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.61 | 总 334/526 Nineteen ( PID 1425 )

出处：2018级河南工大新生周赛总决赛（重现）（I，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（I，CID 1048） | 训练定位：进阶 | 数学、数论与组合

标签：质数筛、预处理、二分查找、数位 | 限制：1.000 S / 32 MB | 历史统计：5/13 ( 38.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给你一个数字  $n$ ，求出  $\leq n$  最大的一个象征绝望的数字

输入要点：多组测试数据

每组测试数据为一个整数  $n$  ( $2 \leq n \leq 138888$ )

可能会有 100000 组以上的数据

输出要点：输出满足  $\leq n$ ，最大的象征绝望的数字

题面提示：PS：什么是前缀和翻转后的前缀？举个例子就明白了

54318 的前缀有 5 个：54318, 5431, 543, 54, 5，翻转后前缀为：8, 81, 813, 8134, 81345

## 零基础先修

- `lower_bound` 找第一个不小于目标的位置，`upper_bound` 找第一个大于目标的位置。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

- 核心处理采用：数据上限只有 138888，但询问超过十万，所以先一次性筛出所有“绝望 数字”。对每个候选  $x$ ，反复除以 10 检查所有普通前缀；再把十进制数字 反转，对反转数同样反复除以 10，就检查了题面定义的全部“翻转后前缀”。每个值必须是 1 或质数。保存全部合格数并排序，每次询问用 `upper_bound` 找第一个大于  $n$  的位置，向前一格就是答案。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

预处理  $O(U \log \log U + U \cdot \text{位数})$ ，每次询问  $O(\log K)$

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int rev(int x) {
4 |     int res = 0;
5 |     while (x > 0) {
6 |         res = res * 10 + x % 10;
7 |         x /= 10;
8 |     }
9 |     return res;
10 | }
11 | int main() {
12 |     ios::sync_with_stdio(false);
13 |     cin.tie(nullptr);
14 |     const int LIMIT = 138888;
15 |     vector<bool> prime(LIMIT + 1, true);
16 |     prime[0] = prime[1] = false;
17 |     for (int i = 2; i * i <= LIMIT; ++i)
18 |         if (prime[i])
19 |             for (int j = i * i; j <= LIMIT; j += i)
20 |                 prime[j] = false;
21 |     auto validPrefix = [&](int x) {
22 |         while (x > 0) {
23 |             if (x != 1 && !prime[x])
24 |                 return false;
25 |             x /= 10;
26 |         }
27 |         return true;
28 |     };
29 |     vector<int> despair;
30 |     for (int x = 1; x <= LIMIT; ++x) {
31 |         if (validPrefix(x) && validPrefix(rev(x)))
32 |             despair.push_back(x);
33 |     }
34 |     int n;
35 |     while (cin >> n) {
36 |         auto it = upper_bound(despair.begin(), despair.end(), n);
37 |         --it;
38 |         cout << *it << '\n';
39 |     }
40 |     return 0;
41 | }
```

## 公开样例

样例 1 输入

```
5
6
7
22
```

样例 1 输出

```
5
5
7
17
```



## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 5.62 | 总 335/526 AC之子 ( PID 1487 )

出处：2019级河南工大新生周赛 ( 三 ) @邓祎培专场 ( C , CID 1055 ) | 训练定位：进阶 | 数学、数论与组合

标签：高精度、斐波那契递推、字符串加法 | 限制：1.000 S / 128 MB | 历史统计：71/515 ( 13.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个数，是前t项的总和

输入要点：第一行输入三个数m, n, t, 分别代表第一项，第二项与求前t项的和 (  $m \leq 1000, n \leq 1000, t \leq 1000$  )

输出要点：输出一个数，是前t项的总和

## 零基础先修

- 超过 64 位范围时用字符串逐位运算，或使用题目允许的大整数工具。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：数列从第三项起为前两项之和。t 可达 1000，项和会远超 64 位，因此用十进制字符串实现加法。分别维护前两项 current/next 与总和，每生成一项便加入总和；t=1、2 单独由同一循环自然覆盖。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(tD) 时间、O(D) 空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | string add(string a, string b) {
4 |     int i = (int)a.size() - 1, j = (int)b.size() - 1, carry = 0;
5 |     string res;
6 |     while (i >= 0 || j >= 0 || carry) {
7 |         int x = i >= 0 ? a[i--] - '0' : 0;
8 |         int y = j >= 0 ? b[j--] - '0' : 0;
9 |         int sum = x + y + carry;
10 |         res.push_back(char('0' + sum % 10));
11 |         carry = sum / 10;
12 |     }
13 |     reverse(res.begin(), res.end());
14 |     return res;
```

```

12 |     }
13 |     reverse(res.begin(), res.end());
14 |     return res;
15 | }
16 | int main() {
17 |     ios::sync_with_stdio(false);
18 |     cin.tie(nullptr);
19 |     string first, second;
20 |     int t;
21 |     cin >> first >> second >> t;
22 |     string sum = "0";
23 |     if (t >= 1)
24 |         sum = add(sum, first);
25 |     if (t >= 2)
26 |         sum = add(sum, second);
27 |     for (int index = 3; index <= t; ++index) {
28 |         string next = add(first, second);
29 |         sum = add(sum, next);
30 |         first = second;
31 |         second = next;
32 |     }
33 |     cout << sum << '\n';
34 |     return 0;
35 | }

```

## 公开样例

样例 1 输入

2 3 4

样例 1 输出

18

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.63 | 总 336/526 你的AC。（PID 1495）

出处：2019级河南工大新生周赛（三）@邓祎培专场（B，CID 1055）| 训练定位：进阶 | 数学、数论与组合

标签：图着色、奇偶性、计数 | 限制：1.000 S / 128 MB | 历史统计：20/98（20.4%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，对于每一个测试数据，输出最小的组队数量，每组数据之间用换行符隔开

**输入要点：**第一行输入q代表有q组测试数据（ $1 \leq q \leq 100$ ）

每一组测试数据的第一行包括一个整数n（ $1 \leq n \leq 100$ ）代表这个测试数据有n个学生，

第二行有n个整数代表每个学生所对应的技能 $a_1, a_2, \dots, a_n$ （ $1 \leq a_i \leq 100$ ）

**输出要点：**对于每一个测试数据，输出最小的组队数量，每组数据之间用换行符隔开

题面提示：队伍数量应尽可能的小

## 零基础先修

- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：如果不存在技能值相差 1 的两名学生，所有人可放一组。否则一组 不可能，但按技能值奇偶分成两组一定可行：同奇偶数的差不可能为 1。因此答案只需判断是否同时出现某个  $v$  和  $v+1$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(n + \text{值域})$  时间、 $O(\text{值域})$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int q;
5 |     cin >> q;
6 |     while (q--) {
7 |         int n;
8 |         cin >> n;
9 |         array<bool, 102> exists{};
10 |         for (int i = 0, x; i < n; ++i) {
11 |             cin >> x;
12 |             exists[x] = true;
13 |         }
14 |         int ans = 1;
15 |         for (int value = 1; value < 100; ++value) {
16 |             if (exists[value] && exists[value + 1])
17 |                 ans = 2;
18 |         }
19 |         cout << ans << '\n';
20 |     }
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

```
4
4
2 10 1 20
2
3 6
5
2 3 4 99 100
1
42
```

样例 1 输出

```
2
1
2
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 5.64 | 总 337/526 第k大分数 ( 加强版 ) ( PID 1517 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( F , CID 1058 ) | 训练定位：进阶 | 数学、数论与组合

标签：排序、数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：4/31 ( 12.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：请帮助小EF求出其中第 k 大的分数

输入要点：第一行包含两个整数 n 和 k

第二行包含 n 个质数  $p_i$  ( $1 \leq n \leq 1000, 1 \leq k \leq 1000000000, 2 \leq p_i \leq 1000000000$ )

输出要点：输出一个分数表示答案

题面提示：样例解释：

2得到的分数有  $1/2$

3得到的分数有  $1/3, 2/3$

5得到的分数有  $1/5, 2/5, 3/5, 4/5$

分数排序后有： $4/5, 2/3, 3/5, 1/2, 2/5, 1/3, 1/5$

所以第3大分数是 $3/5$

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
- 核心处理采用：第k大分数 ( 加强版 ) 二分提高题 因为给出的是质数，所以每一个分数对应的小数都是独一无二的，就像只能有  $\frac{1}{2}$  而不会有  $\frac{1}{2}$  一样。 $\frac{2}{4}$  而且当分母一定时，分子的大小顺序就是对应的分数的大小顺序，就像  $45 > 35 > 25 > 15$   $\frac{4}{5} > \frac{3}{5} > \frac{2}{5} > \frac{1}{5}$   $54 > 53 > 52 > 51$  那么如果我们知道 大于小数，且分母为 midmidmid 的分数个数PPP的话，我们就可以枚举 cntcntcnt 个分母求得所有大于 nnn 的分数个数 midmidmid，通过比较这个 cntcntcnt 和 cntcntcnt 的大小就可以知道 kkk 是太大了还是太小了，二分夹逼，直到最后 midmidmid。cnt=kcnt = kcnt=k那么怎么获取这个呢？我们知道当分母为 cntcntcnt 时，构成的分数是： $555$ ，假设  $45, 35, 25$  和  $15 \frac{4}{5}, \frac{3}{5}, \frac{2}{5}$  和  $\frac{1}{5}$   $54, 53, 52$  和  $51$  此时为 midmidmid，那么把  $0.50.50.5$  转化为一个分数且分母为  $0.50.50.5$  的最简单的方式就是分子分母同时乘以  $555$ ： $555$ 。所以得到的分数就是  $0.5 = 5 \times 0.550.5 = \frac{5}{10} \times \frac{1}{2} = \frac{5}{20}$   $0.5 = 55 \times 0.5$ ，显然，分子大于  $2.55 \frac{2.5}{5} 52.5$  的有  $2.52.52.5$  个，即  $(222)$  ( 符号意为向下取整 )；同理当分母为  $5 - \lfloor P \cdot \text{mid} \rfloor - 15 - \lfloor \text{floor} 2.5 \rfloor - 15 - \lfloor 2.5 \rfloor - 1$  时，大于 PPP 的分数个数就是 midmidmid， $P - \lfloor P \cdot \text{mid} \rfloor - 1P - \lfloor \text{floor} P \rfloor - \lfloor \text{mid} \rfloor - 1$  而每一次求时我们都保存最接近 cntcntcnt 的分数的分子和分母，那么二分结束时的分子和分母就是要输出的答案了。midmidmid C++版代码
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。

- 因此所得数值正是题目定义的答案。

## 复杂度

$O(1)$

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <iostream>
2 | #include <algorithm>
3 | #define maxn 1005
4 | #define _for(i, a) for (LL i = 0; i < (a); ++i)
5 | using namespace std;
6 | typedef long long LL;
7 | LL n, k;
8 | LL P[maxn];
9 | LL p, q; // 最接近mid的分子和分母
10 | LL find(double mid) {
11 |     LL cnt = 0;
12 |     _for(i, n) {
13 |         LL num = mid * P[i]; // 向下取整
14 |         cnt += P[i] - num - 1; // 比mid小的分数
15 |         if (p == -1 && q == -1 || p * P[i] > (num + 1) * q) { // 保存分子和分母
16 |             p = num + 1;
17 |             q = P[i];
18 |         }
19 |     }
20 |     return cnt;
21 | }
22 | void sol() {
23 |     cin >> n >> k;
24 |     _for(i, n) cin >> P[i];
25 |     double l = 0, r = 1;
26 |     while (1) {
27 |         p = -1, q = -1;
28 |         double mid = (l + r) / 2;
29 |         LL cnt = find(mid);
30 |         if (cnt == k)
31 |             break;
32 |         if (cnt < k)
33 |             r = mid;
34 |         else
35 |             l = mid;
36 |     }
37 |     cout << p << "/" << q << "\n";
38 | }
39 | int main() {
40 |     ios::sync_with_stdio(false);
41 |     cin.tie(0);
42 |     cout.tie(0);
43 |     // freopen("in.txt", "r", stdin);
44 |     sol();
45 |     return 0;
46 | }
```

## 公开样例

样例 1 输入

```
3 3
2 3 5
```

样例 1 输出

```
3/5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.65 | 总 338/526 第k大分数 ( 简单版 ) ( PID 1519 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( E , CID 1058 ) | 训练定位：进阶 | 数学、数论与组合

标签：排序、数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：12/74 ( 16.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：请帮助小EF求出其中第  $k$  大的分数

输入要点：第一行包含两个整数  $n$  和  $k$

第二行包含  $n$  个质数  $p_i$  ( $1 \leq n \leq 1000, 1 \leq k \leq 1000, 2 \leq p_i \leq 1000$ )

输出要点：输出一个分数表示答案

题面提示：样例解释：

2得到的分数有  $1/2$

3得到的分数有  $1/3$  ,  $2/3$

5得到的分数有  $1/5$  ,  $2/5$  ,  $3/5$  ,  $4/5$

分数排序后有： $4/5$  ,  $2/3$  ,  $3/5$  ,  $1/2$  ,  $2/5$  ,  $1/3$  ,  $1/5$

所以第3大分数是 $3/5$

### 零基础先修

- `sort` 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 `long long` 或 `__int128`。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：第k大分数 ( 简单版 ) 暴力基础题 先把所有的形成的分数保存起来，然后按从大到小快排一遍，最后输出第  $k$  个即可  
C++版代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```

1 | #include <iostream>
2 | #include <algorithm>
3 | #include <vector>
4 | #define _for(i, a) for (int i = 0; i < (a); ++i)
5 | #define _rep(i, a, b) for (int i = (a); i <= (b); ++i)
6 | using namespace std;
7 | typedef long long LL;
8 | struct poi {
9 |     LL n, d;
10 |     poi() {}
11 |     poi(int n, int d) : n(n), d(d) {}
12 |     bool operator<(const poi& b) const {
13 |         return 1.0 * n / d > 1.0 * b.n / b.d;
14 |     }
15 | };
16 | LL n, k;
17 | vector<poi> a;
18 | void sol() {
19 |     _for(i, n) {
20 |         LL tem;
21 |         cin >> tem;
22 |         _rep(i, 1, tem - 1) a.push_back(poi(i, tem));
23 |     }
24 |     sort(a.begin(), a.end());
25 |     cout << a[k - 1].n << "/" << a[k - 1].d << "\n";
26 | }
27 | int main() {
28 |     while (cin >> n >> k) {
29 |         sol();
30 |     }
31 |     return 0;
32 | }

```

## 公开样例

样例 1 输入

```

3 3
2 3 5

```

样例 1 输出

```

3/5

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.66 | 总 339/526 小青的高等数学 ( PID 1532 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（E，CID 1060）| 训练定位：进阶 | 数学、数论与组合

标签：数学、观察、质数 | 限制：1.000 S / 128 MB | 历史统计：34/171 ( 19.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：第一节高数课高数老师就给小青来了一个惊喜（xia），他给了小青一个长度为 $n$ 的序列 $A$ ，其中序列中每个元素的范围都是 $1\sim m$ ，同时保证这 $m$ 个整数都出现在了序列里。现在高数老师要求小青选取序列中的两个数，使得两个数的和最小，同时还需要保证这两个数的和为质数，你可以告诉小青这个最小的和是多少吗？

输入要点：第一行为两个整数  $n, m$ ，为整个序列的长度和序列中元素的范围。

第二行为  $n$  个整数，其中第  $i$  个数代表  $A_i$ 。

$2 \leq m \leq n \leq 1000000$

输出要点：只有一个数，为这个质数的最小值。

题面提示：选取两个 1，即可满足条件

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：题目保证  $1..m$  每个数都出现，且  $m \geq 2$ 。因此若 1 至少出现两次，可选  $1+1=2$ ；否则一定可以选 1 和 2，得到质数 3。不存在更小的正质数，答案只可能是 2 或 3。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m, ones = 0;
7 |     cin >> n >> m;
8 |     for (int i = 0, x; i < n; ++i) {
9 |         cin >> x;
10 |         if (x == 1)
11 |             ++ones;
12 |     }
13 |     cout << (ones >= 2 ? 2 : 3) << '\n';
14 |     return 0;
15 | }
```

## 公开样例

样例 1 输入

```
5 3
1 1 2 2 3
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- [HAUTOJ 原题](#)

## 5.67 | 总 340/526 小青的线性代数 ( PID 1533 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（H，CID 1060）| 训练定位：进阶 | 数学、数论与组合

标签：数学、矩阵、公式模拟 | 限制：1.000 S / 128 MB | 历史统计：169/307 ( 55.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，代表行列式的值

输入要点：第一行为一个整数  $n$  ( $1 \leq n \leq 3$ )，代表行列式的阶数。

第2到 $n+1$ 行每行有 $n$ 个整数，表示行列式的各个项，保证行列式的每一项的绝对值小于100。

输出要点：一个整数，代表行列式的值

题面提示： $1*3-2*7=-11$

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：阶数只有 1、2、3，分别套用行列式公式。三阶采用第一行展开： $a(ei-fh)-b(di-fg)+c(dh-eg)$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(n^2)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     long long a[3][3] = {};
7 |     for (int i = 0; i < n; ++i) {
8 |         for (int j = 0; j < n; ++j)
9 |             cin >> a[i][j];
10 |    }
11 |    long long ans;
12 |    if (n == 1)
13 |        ans = a[0][0];
14 |    else if (n == 2)
15 |        ans = a[0][0] * a[1][1] - a[0][1] * a[1][0];
16 |    else {
17 |        ans = a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1]) -
```

```

18 |         a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0]) +
19 |         a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0]);
20 |     }
21 |     cout << ans << '\n';
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

2
17
23

```

样例 1 输出

```

-11

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.68 | 总 341/526 小青与nana的不解之谜 ( PID 1535 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（J，CID 1060）| 训练定位：进阶 | 数学、数论与组合

标签：字符串、循环、取模 | 限制：1.000 S / 128 MB | 历史统计：279/486 ( 57.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**当然，仅此显然是不够的，小青还要设置文件名，当然，文件名的规则也很简单，假设变量名为s，只需n个s连接到一起即可组成文件名。下面给你小青得到的随机数，请你输出小青设置的文件名。

**输入要点：**一个正整数n (  $1 \leq n \leq 6$  )

**输出要点：**一行字符，代表小青的文件名

**题面提示：** $2\%3=2$ ，所以小青的变量名为nananana,文件名为2个变量名，所以文件名为nananananananana

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- for 适合已知次数，while 适合由条件或 EOF 决定的次数；每轮都要保证状态向结束推进。
- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：变量名由若干个 na 组成：余数 0、1、2 时分别重复 2、3、4 次，即  $2+n\%3$  次。文件名又把变量名连接 n 次，所以总共输出  $n \cdot (2+n\%3)$  个 na。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\text{输出长度})$  时间、 $O(1)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- C++ 负数取模仍可能为负；减法后补模数。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     int times = n * (2 + n % 3);
7 |     while (times--)
8 |         cout << "na";
9 |     cout << '\n';
10 |    return 0;
11 | }
```

## 公开样例

样例 1 输入

2

样例 1 输出

nanananananana

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.69 | 总 342/526 小青和四火的肥宅生活 ( PID 1537 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（I，CID 1060）| 训练定位：进阶 | 数学、数论与组合

标签：数论、最大公约数、构造 | 限制：1.000 S / 128 MB | 历史统计：11/139 ( 7.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：愉快的寒假来临了，小青跟四火迎来了快乐的肥宅生活，他们买了冰阔落与小零食，还有一个大蛋糕，随后四火展现了自己的迪拜刀法，将大蛋糕切成了 $n$ 块，每块的大小依次为 $1, 2, 3, 4, \dots, n-1, n$ ，众所周知，小青吃蛋糕会得到快乐，我们把这个快乐程度量化为快乐值，快乐值越高代表着小青越快乐。这个大蛋糕有一个神奇的魔力，小青获得的快乐值并不是由他吃下去的蛋糕的总体积……

输入要点：两个正整数  $n, k$  (  $1 \leq k \leq n \leq 1000000$  )

定义一个数的最大公约数就是他本身

输出要点：一个正整数，代表小青可以获得的最大快乐值

题面提示：小青能且仅能吃1块大小为1的蛋糕，获得的最大快乐值为1

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- `std::gcd(a,b)` 返回最大公约数，并满足 `gcd(a,b)=gcd(b,a mod b)`。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：要让选出的  $k$  个不同整数的最大公约数至少为  $g$ ， $1..n$  中至少要有  $k$  个  $g$  的倍数，即  $\text{floor}(n/g) \geq k$ 。因此  $g$  最大为  $\text{floor}(n/k)$ ；选择  $g, 2g, \dots, kg$  可证明该上界能达到。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, k;
5 |     cin >> n >> k;
6 |     cout << n / k << '\n';
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

```
1 1
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.70 | 总 343/526 阿正的子母序列 ( PID 1544 )

出处：2020级HAUT新生周赛（二）@杨哲专场（F，CID 1061） | 训练定位：进阶 | 数学、数论与组合

标签：数组与序列、数学、连续子数组 | 限制：1.000 S / 128 MB | 历史统计：1/8 ( 12.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在学长给阿正了一个长度为N的序列A，不仅要让阿正求出A中不相等的连续非递减子序列的数量，同时，学长将一个序列的权值定义为该序列的长度，要让阿正求出A的所有不相同的连续非递减子序列权值总和。

输入要点：第一行一个正整数N，代表序列A的长度；(0<N<=10000)

第二行N个整形范围内的整数，依次代表序列A中的各元素。

输出要点：第一行一个正整数代表A的不同的连续非递减子序列个数。

第二行一个正整数代表A的所有连续非递减子序列权值总和。

题面提示：对于样例中的序列：1 2 4 5 3 的连续非递减子序列，有 ( 每个[]为一个 )

[1]、[1 2]、[1 2 4]、[1 2 4 5]

[2]、[2 4]、[2 4 5]

[4]、[4 5]

[5]

[3]

共11个；

对上述非递减子序列，权值依次为：

1、2、3、4

1、2、3

1、2

1

1

总和为21。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：把序列划分成若干个极大的连续非递减段。长度为 L 的段包含  $L(L+1)/2$  个连续子序列；这些子序列的长度总和为  $L(L+1)(L+2)/6$ 。由于题目把起点位置也纳入相等定义，不必对相同值的子序列去重。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> a(n);
9 |     for (long long& x : a)
10 |         cin >> x;
11 |     long long count = 0, weight = 0;
12 |     for (int start = 0; start < n;) {
13 |         int end = start;
14 |         while (end + 1 < n && a[end] <= a[end + 1])
15 |             ++end;
16 |         long long len = end - start + 1;
17 |         count += len * (len + 1) / 2;
18 |         weight += len * (len + 1) * (len + 2) / 6;
19 |         start = end + 1;
20 |     }
21 |     cout << count << '\n' << weight << '\n';
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

```
5
1 2 4 5 3
```

样例 1 输出

```
11
21
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.71 | 总 344/526 别看了 这是水题 ( PID 1556 )

出处：2020级HAUT新生周赛（四）@张承树专场（B，CID 1063）| 训练定位：进阶 | 数学、数论与组合

标签：构造、数论、最大公约数 | 限制：1.000 S / 128 MB | 历史统计：0/9 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果存在这样的序列，请输出这个序列（可能会很多种序列符合情况，请输出任意一组）如果不存在，请输出 -1

输入要点：第一行一个T代表有T组样例

接下来T行每行一个两个整数代表n，k

输出要点：如果存在这样的序列，请输出这个序列（可能会很多种序列符合情况，请输出任意一组）

如果不存在，请输出-1

题面提示：T ≤ 100

n, k ≤ 10000

n, k ≥ 1

第一组样例有很多种情况，比如：[3,5,1,4,6,2]、[1,6,3,5,2,4]等，符合条件的序列均正确，输出任意一个即可。

## 零基础先修

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- std::gcd(a,b) 返回最大公约数，并满足 gcd(a,b)=gcd(b,a mod b)。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：n=1 时单元素序列总是合法。n>1 时存在解当且仅当 n≥2k。令 g=gcd(n,k)，对起点 g,g-1,...,1 依次沿“加 k、超过 n 就减 n”走完一个模 n 环。环内相邻差为 k 或 n-k，均不少于 k；前一环末尾到下一环起点的差为 n-k+1，也合法，且所有数恰出现一次。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

每组 O(n) 时间、O(n) 空间

## 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, k;
10 |         cin >> n >> k;
11 |         if (n == 1) {
12 |             cout << 1 << '\n';
13 |             continue;
14 |         }
15 |         if (n < 2 * k) {
16 |             cout << -1 << '\n';
17 |             continue;
18 |         }
19 |         int g = gcd(n, k);
20 |         vector<int> ans;
21 |         vector<bool> used(n + 1, false);
22 |         for (int start = g; start >= 1; --start) {
23 |             int cur = start;
24 |             while (!used[cur]) {
25 |                 used[cur] = true;
26 |                 ans.push_back(cur);
27 |                 cur += k;
28 |                 if (cur > n)
29 |                     cur -= n;
30 |             }
31 |         }
32 |         for (int i = 0; i < n; ++i) {
```

```

33 |         if (i)
34 |             cout << ' ';
35 |             cout << ans[i];
36 |         }
37 |         cout << '\n';
38 |     }
39 |     return 0;
40 | }

```

## 公开样例

样例 1 输入

```

2
7 2
2 2

```

样例 1 输出

```

3 5 1 4 6 2 7
-1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.72 | 总 345/526 我劝你们这些年轻人，不要轻易碰此题 ( PID 1558 )

出处：2020级HAUT新生周赛（四）@张承树专场（D，CID 1063）| 训练定位：进阶 | 数学、数论与组合

标签：字符串、大整数、高精度比较 | 限制：1.000 S / 128 MB | 历史统计：4/61 ( 6.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，对于每轮游戏来说，每行输出一个结果。如果阿正的数比Love\_Jacques的数大，输出"AZNB" 如果Love\_Jacques的数比阿正的数大，输出"YZNB" 如果阿正的数跟Love\_Jacques的数一样大，输出"EASY GAME!" 输出不带引号

**输入要点：**第一行一个T，代表要做T组游戏

接下来T行，每行一个x，y（之间用空格隔开）分别代表阿正和Love\_Jacques给出的实数

x，y的小数位最多会有1000位

保证给出的实数均为合法数据，即（0.203不会写成.203）、（1也不会写成1.）的情况

**输出要点：**对于每轮游戏来说，每行输出一个结果。

如果阿正的数比Love\_Jacques的数大，输出"AZNB"

如果Love\_Jacques的数比阿正的数大，输出"YZNB"

如果阿正的数跟Love\_Jacques的数一样大，输出"EASY GAME!"

输出不带引号

题面提示：一定有你没有想到的情况

[图片：（原题中的内网资源地址已省略）]

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。



- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：不能转成浮点数，因为小数可有 1000 位。把整数部分与小数部分分开，先比较整数位；相同后把小数末尾无意义的 0 删除，再补齐到相同长度按字典序比较。这样 1.2 与 1.20 会被判为相等。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(|x|+|y|)$  时间、 $O(|x|+|y|)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | pair<string, string> splitNumber(string s) {
4 |     size_t dot = s.find('.');
5 |     string integer = dot == string::npos ? s : s.substr(0, dot);
6 |     string fraction = dot == string::npos ? "" : s.substr(dot + 1);
7 |     while (!fraction.empty() && fraction.back() == '0')
8 |         fraction.pop_back();
9 |     return {integer, fraction};
10 | }
11 | int cmp(const string& a, const string& b) {
12 |     auto [ai, af] = splitNumber(a);
13 |     auto [bi, bf] = splitNumber(b);
14 |     if (ai != bi)
15 |         return ai < bi ? -1 : 1;
16 |     size_t len = max(af.size(), bf.size());
17 |     af.resize(len, '0');
18 |     bf.resize(len, '0');
19 |     if (af == bf)
20 |         return 0;
21 |     return af < bf ? -1 : 1;
22 | }
23 | int main() {
24 |     ios::sync_with_stdio(false);
25 |     cin.tie(nullptr);
26 |     int T;
27 |     cin >> T;
28 |     while (T--) {
29 |         string x, y;
30 |         cin >> x >> y;
31 |         int relation = cmp(x, y);
32 |         if (relation > 0)
33 |             cout << "AZNB\n";
34 |         else if (relation < 0)
35 |             cout << "YZNB\n";
36 |         else
37 |             cout << "EASY GAME!\n";
38 |     }
39 |     return 0;
40 | }
```

## 公开样例

样例 1 输入

```
3
1.0 2.32
1.65 0.23
1.123 1.123
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 5.73 | 总 346/526 牛牛的x数 ( PID 1582 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（D，CID 1070）| 训练定位：进阶 | 数学、数论与组合

标签：大整数 | 限制：1.000 S / 128 MB | 历史统计：50/265 ( 18.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：牛牛定义了一种新的数，叫做“x数”，一个整数a，其所有位数中出现的最大数的次数若为1，则把这个最大数定义为a的“x数”。现在给你a，让你找出a的“x数”。

输入要点：一个整数a。（ $-1000000 < a < 1000000$ ）

输出要点：若存在则输出a的“x数”，不存在则输出NO

### 零基础先修

- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：牛牛的x数处理每一位求最大值及其出现的次数即可 `#include<stdio.h> #include<math.h>`
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     int m, cnt, n; // m代表当前最大值, cnt表示其出现的次数
5 |     scanf("%d", &n);
6 |     n = abs(n), cnt = 0, m = -1;
7 |     if (n == 0) {
```

```

8 |         printf("0");
9 |         return 0;
10 |     }
11 |     while (n) {
12 |         int t = n % 10; // 得到n的末位
13 |         n /= 10;
14 |         if (t == m)
15 |             cnt++;
16 |         if (t > m) {
17 |             cnt = 1;
18 |             m = t;
19 |         }
20 |     }
21 |     if (cnt != 1)
22 |         printf("NO");
23 |     else
24 |         printf("%d", m);
25 | }

```

## 公开样例

样例 1 输入

54213

样例 1 输出

5

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.74 | 总 347/526 牛牛的素数判断 ( PID 1583 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（E，CID 1070）| 训练定位：进阶 | 数学、数论与组合

标签：条件分支、数论 | 限制：1.000 S / 128 MB | 历史统计：19/422 ( 4.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：牛牛最近学会了素数判断，但牛牛的朋友不相信想考考他，便问他：给你两个数a,b，判断a×b是否是素数。

输入要点：第一行输入一个整数T，代表T组数据。接下来T行(1≤T≤10)，每行两个整数a，b(1≤a,b≤1e11)。

输出要点：对于每一行的输入若，a×b是素数则输出一行YES,否则输出一行NO。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：牛牛的素数判断//这题数据弱了，有些a×b的也过了(a×b会爆掉)；由于评测姬出了一些问题，时间复杂度也没卡到// 由素数定义可知若a×b为素数则必定其中一个数为1且另一个数必须为素数，因此本题只需判断不为1那个数是否为素数即可。时间限制为1s的话，如果你在判断素数时从2挨个求余会超时，判断到sqrt(n)即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int prime(long long x) {
4 |     if (x == 1)
5 |         return 0;
6 |     for (int i = 2; i <= sqrt(x); i++)
7 |         if (x % i == 0)
8 |             return 0;
9 |     return 1;
10| }
11| int main() {
12|     int T;
13|     scanf("%d", &T);
14|     while (T--) {
15|         long long a, b;
16|         scanf("%lld%lld", &a, &b);
17|         if (a == 1 && prime(b) == 1 || b == 1 && prime(a) == 1)
18|             printf("YES\n");
19|         else
20|             printf("NO\n");
21|     }
22| }
```

## 公开样例

样例 1 输入

```
3
2 3
1 7
1 4
```

样例 1 输出

```
NO
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.75 | 总 348/526 聚聚的Rating目标 ( PID 1588 )

出处：2021级HAUT新生周赛（一）@张君毅专场（H，CID 1069）| 训练定位：进阶 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：121/335 ( 36.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个正整数，表示当前Rating距离下一个Rating目标有多少分。

输入要点：一行，一个正整数 $n$ ，且 $n \leq 100000$

输出要点：一行，一个正整数，表示当前Rating距离下一个Rating目标有多少分。

题面提示：样例解释：2021的下一个Rating目标是3000，相差979分

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：聚聚的Rating目标举几个例子：1234的下一个目标是2000  $((1234 / 1000) + 1) * 1000 = 2000$  20的下一个目标是30  $((20 / 10) + 1) * 10 = 30$  因此，下一个Rating目标=当前数字的最高位上的数+1，其余各位变成0
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     int rat, r, ans, k, t = 0;
5 |     scanf("%d", &rat);
6 |     r = rat;
7 |     // 求位数
8 |     while (rat) {
9 |         t++;
10 |         rat = rat / 10;
11 |     }
12 |     // 求出10的(长度-1)次方
13 |     k = pow(10, t - 1);
14 |     // 求出结果
15 |     ans = ((r / k) + 1) * k - r;
16 |     printf("%d", ans);
17 |     return 0;
18 | }
```

### 公开样例

样例 1 输入

2021

样例 1 输出

979

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.76 | 总 349/526 子序列和 ( PID 1707 )

出处：2021级HAUT新生周赛（四）@王一杰专场（D，CID 1072）| 训练定位：进阶 | 数学、数论与组合

标签：数组与序列、字符串、数论 | 限制：1.000 S / 128 MB | 历史统计：5/121 ( 4.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：由于结果可能过大，请输出结果对 $1e9+7$ 取模后的值。

输入要点：第一行一个正整数  $n$  ( $1 \leq n \leq 1e5$ ) 代表序列长度；

第二行为  $n$  个整数代表序列，每个数小于  $1e4$ 。

输出要点：一行一个正整数，代表所有子序列和的和。

题面提示：对于样例中的序列有以下子序列：

序列 {1} 的和为1

序列 {3} 的和为3

序列 {4} 的和为4

序列 {1,3} 的和为4

序列 {1,4} 的和为5

序列 {3,4} 的和为7

序列 {1,3,4} 的和为8

所以所有子序列和的和为32

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：子序列和枚举子序列计算总和的方式显然不现实，考虑枚举每位  $a[i]$  最总和造成的贡献。假设包含  $a[i]$  的子序列共有  $k$  个，那么  $a[i]$  的贡献值便为  $a[i] * k$ ；假设对于当前子序列， $a[i]$  已经选了，那么其他所有元素都有选和不选两种情况，最后包含的  $a[i]$  的子序列便有一个， $n$  为序列总长度。 $2^{n-1}$  例如，求包含  $a_1, a_2, a_3, a_4$  的子序列共有多少个，那么  $a_1$  选不选会分出来两种情况为  $a_2 a_3 a_4$  和  $\{a_1\}, \{a_1, a_2\}, \{a_1, a_3\}, \{a_1, a_4\}, \{a_1, a_2, a_3\}, \{a_1, a_2, a_4\}, \{a_1, a_3, a_4\}, \{a_1, a_2, a_3, a_4\}$  同理，所以除了  $a_4 a_3 a_2$  以外其他元素要考

考虑  $n-1$  次，最后包含  $a_i$  的子序列共有  $a_i a_i a_i$  个。 $2^{n-1} 2^{n-1} 2^{n-1}$  另一方面，计算 次方本题中可以在算每一位的贡献之前预处理出来，即开个  $\text{for}$  循环  $n$  次用一个数组  $2^{n-1} 2^{n-1} 2^{n-1}$ ， $p[p][p][p]$  存  $\text{pipipi}$  的值，或者可以直接用快速幂计算，不了解快速幂的可以学一下。 $2^{i_1 i_2 i_3}$  此处为了照顾大多数人就不用快速幂了。 $\text{ps} : \text{ps} : \text{ps}$

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次  $\text{cin}$  留下的换行。
- $\text{gcd/lcm}$  的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <string.h>
4 | #include <math.h>
5 | const int MOD = 1e9 + 7, maxn = 1e6 + 5;
6 | int n, a[1000005];
7 | long long BS[1000005];
8 | void init() {
9 |     BS[0] = 1;
10 |     for (int i = 1; i <= maxn - 1; i++) {
11 |         BS[i] = (BS[i - 1] % MOD * 2) % MOD;
12 |     }
13 | }
14 | int main() {
15 |     init();
16 |     scanf("%d", &n);
17 |     long long res = 0;
18 |     for (int i = 1; i <= n; i++) {
19 |         scanf("%d", &a[i]);
20 |         res = res % MOD + (a[i] % MOD * BS[n - 1] % MOD) % MOD;
21 |     }
22 |     printf("%lld\n", res);
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
3
1 3 4
```

样例 1 输出

```
32
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.77 | 总 350/526 lzl的三角形绝技 ( PID 1714 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（F，CID 1073）| 训练定位：进阶 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：11/52 ( 21.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：21年了，lzl上大学了，来到了美丽的haut，他十分开心的进入新的学期，有天高数老师讲到了三角形并留下了一个关于三角形的作业，这道题目是这样的：在一行中给定四根小棍子的长度，挑选其中的三根问是否可以组成一个三角形，如果不能，又是否可以组成一个退化三角形(其中一个角度是180度或者有两个角都是90度,面积为0的三角形),不允许折断棍子或使用它们的部分长度,.....

输入要点：输入的第一行包含四个由空格分隔的不超过100的正整数分别表示四根小棍子的长度。

输出要点：如果给定的四根小棍子挑选其中三根可以组成一个三角形，则在一行中输出TRIANGLE，如果不能组成一个三角形但可以组成一个退化三角形那么则输出SEGMENT，如果三角形和退化三角形都无法组成，那么在一行中输出IMPOSSIBLE。

题面提示：只需选择2、3、4三个长度为边即可构成三角形。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：lzl的三角形绝技思路：这道题目给出了退化三角形的定义，也就是一个角度是180度或者两个角度都是90度，面积为0的三角形，关于角度的处理是不太方便的，那么我们可以进行转化，也就是说，一个角度是180度意思是有两边之和等于第三边，相当于等腰三角形腰边和底边重合了，两个角度都是90度的可以转化为有一条边是0；有了这个思路之后我们就可以按照题目给的数据进行判断了。  
参考代码如下

- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int a, b, c, d;
4 |     scanf("%d %d %d %d", &a, &b, &c, &d);
5 |     int sum = 0; // 判断有多少个0
6 |     if (a == 0)
7 |         sum++;
8 |     if (b == 0)
9 |         sum++;
10 |    if (c == 0)
11 |        sum++;
12 |    if (d == 0)
13 |        sum++;
14 |    if ((a + b > c) && (a + c > b) && (b + c > a) ||
15 |        (a + b > d) && (a + d > b) && (b + d > a) ||
16 |        (a + c > d) && (a + d > c) && (c + d > a) ||
17 |        (b + c > d) && (b + d > c) && (c + d > b)) {
18 |        puts("TRIANGLE"); // 判断可以组成三角形
19 |    } else if (((a + b == c) || (a + c == b) || (b + c == a) || (a + b == d) ||
20 |        (a + d == b) || (b + d == a) || (a + c == d) || (a + d == c) ||
```



```

21 |             (c + d == a) || (b + c == d) || (b + d == c) || (c + d == b)) ||
22 |             (sum > 0 && sum <= 2)) {
23 |                 puts("SEGMENT"); // 判断可以组成退化三角形:两边相加等于第三边或有0边
24 |             } else {
25 |                 puts("IMPOSSIBLE");
26 |             }
27 | }

```

## 公开样例

样例 1 输入

4 2 1 3

样例 1 输出

TRIANGLE

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 5.78 | 总 351/526 突然之间 ( PID 1733 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（H，CID 1075）| 训练定位：进阶 | 数学、数论与组合

标签：二分答案、对数、数位 | 限制：1.000 S / 128 MB | 历史统计：31/208 ( 14.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：使得Xx达到或超过 n 位数字的最小正整数 x 是多少？

输入要点：一个正整数n

输出要点：使得Xx达到n位数字的最小正整数x

题面提示：n<2e9

## 零基础先修

- 二分答案要求判定函数随答案单调；先写清可行区间与 true/false 的方向。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用： $x^x$  的十进制位数是  $\text{floor}(x \cdot \log_{10}(x)) + 1$ 。该值随正整数 x 单调增加，在  $[1, 2e9]$  二分第一个位数不少于 n 的 x。使用 long double 减少边界处的浮点误差。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(\log 2e9)$  时间、 $O(1)$  空间

## 易错点

- 检查左右边界、循环条件和最终返回的是第一个可行还是最后一个不可行。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long n;
7 |     cin >> n;
8 |     long long left = 1, right = 2000000000LL;
9 |     auto enough = [&](long long x) {
10 |         long double logarithm = x * log10((long double)x);
11 |         return logarithm + 1e-12L >= (long double)(n - 1);
12 |     };
13 |     while (left < right) {
14 |         long long middle = left + (right - left) / 2;
15 |         if (enough(middle))
16 |             right = middle;
17 |         else
18 |             left = middle + 1;
19 |     }
20 |     cout << left << '\n';
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

11

样例 1 输出

10

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.79 | 总 352/526 这种事绝对很奇怪啊 ( PID 1739 )

出处：2021级HAUT新生周赛（八）@孔维斌专场（F，CID 1076） | 训练定位：进阶 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 32 MB | 历史统计：27/162 ( 16.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在需要你修建多条隧道，使得总隧道的长度最小，并且任意两个城市之间可以经过隧道可以到达（可以以其他城市作为中转站）。

输入要点：第一行为一个整数 $n$ ，第二行为 $n$ 个整数，分别代表第 $i$ 个城市的数轴坐标。

数据范围： $1 \leq n \leq 1e5$ , 城市坐标在int范围内

输出要点：输出所有所修建的所有隧道的总长度。

题面提示：城市坐标在int范围内

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：这种事绝对很奇怪啊(传送门)题目大意是修建隧道可以使两两到达，任意两座城市间的修建费用是坐标距离。显然如果x到y中间有其他的城市z，那么先修建x到z，再修建z到y的费用想等或者更低。那么其实就是求出最右端点和最左端点的差值。(注意肯会超过int范围，所以使用long long) 时间复杂度： $O(n)O(n)O(n)$ 参考代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | typedef long long ll;
3 | int main() {
4 |     ll n;
5 |     scanf("%lld", &n);
6 |     ll l, r; // l记录最左端，r记录最右端
7 |     for (int i = 0; i < n; i++) {
8 |         int x;
9 |         scanf("%d", &x);
10 |         if (!i)
11 |             l = x, r = x;
12 |         else {
13 |             if (x < l)
14 |                 l = x;
15 |             if (x > r)
16 |                 r = x;
17 |         }
18 |     }
19 |     printf("%lld", r - l);
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
6
7 9 -3 4 -6 1
```

样例 1 输出

```
15
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 5.80 | 总 353/526 即使世界毁灭，我也想再见你一面 ( PID 1777 )

出处：2022级HAUT新生周赛（三）@焦小航专场（H，CID 1081）| 训练定位：进阶 | 数学、数论与组合

标签：构造、数学、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：62/167 ( 37.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出  $t$  行，每行输出符合题目要求的一种答案，如果存在多种答案，输出任意一种即可。如果无解，输出  $k$  个 0 即可。

输入要点：第一行给出一个  $t$  (  $1 \leq t \leq 1000$  )，代表有  $t$  组询问。

每组包括两个数  $n$  和  $k$ 。 (  $1 \leq n \leq 1e9, 1 \leq k \leq 100$  )

输出要点：输出  $t$  行，每行输出符合题目要求的一种答案，如果存在多种答案，输出任意一种即可。如果无解，输出  $k$  个 0 即可。

### 零基础先修

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：正整数按奇偶只可能用 1 或 2 作为前  $k-1$  项。先尝试  $k-1$  个 1，若末项  $n-k+1$  为正奇数就成立；否则尝试  $k-1$  个 2，若末项  $n-2(k-1)$  为正偶数就成立。两种都不行时按题意输出  $k$  个 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

### 复杂度

每组  $O(k)$  时间、 $O(1)$  额外空间

### 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
```

```

5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long n;
10 |        int k;
11 |        cin >> n >> k;
12 |        if (n >= k && (n - k) % 2 == 0) {
13 |            for (int i = 1; i < k; ++i)
14 |                cout << 1 << ' ';
15 |            cout << n - (k - 1) << '\n';
16 |        } else if (n >= 2LL * k && (n - 2LL * k) % 2 == 0) {
17 |            for (int i = 1; i < k; ++i)
18 |                cout << 2 << ' ';
19 |            cout << n - 2LL * (k - 1) << '\n';
20 |        } else {
21 |            for (int i = 0; i < k; ++i) {
22 |                if (i)
23 |                    cout << ' ';
24 |                cout << 0;
25 |            }
26 |            cout << '\n';
27 |        }
28 |    }
29 |    return 0;
30 | }

```

## 公开样例

样例 1 输入

```

1
100 5

```

样例 1 输出

```

20 20 20 20 20

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

# 5.81 | 总 354/526 调查员三宝之撬棍 ( PID 1781 )

出处：2022级HAUT新生周赛（四）@张子豪专场（B，CID 1082）| 训练定位：进阶 | 数学、数论与组合

标签：数论、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：20/114 ( 17.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，一行，二进制表示下的  $17 \times N$ 。

**输入要点：**一行，一个不超过 1000 位的二进制数字  $N$ 。

**输出要点：**一行，二进制表示下的  $17 \times N$ 。

**题面提示：**样例解释：

给定数字 10110111 在十进制表示下为 183。

$183 \times 17 = 3111$ ，在二进制表示下为 110000100111。

**提示：**

直接用进制转换的话，数据会超出int和long long的范围

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：调查员三宝之撬棍：分析：一个二进制数变为原来的17倍，相当于这个数乘以16再加上原来的数 二进制下，数的末尾添加一个零，相当于这个数乘以2，所以将这个数的末尾加上4个0，再加上原来的数 N最大是1000位，可以用高精加来进行计算(用数组来模拟加法)
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int main() {
4 |     char s[1100] = {0};
5 |     scanf("%s", s);
6 |     int n = strlen(s);
7 |     int t1[1100] = {0}, t2[1100] = {0}; // t1是原本的数, t2是原来的数乘以16
8 |     for (int i = 0; i < n; i++) {
9 |         t2[i + 4] = t1[i] = s[n - i - 1] - '0';
10 |     }
11 |     n += 4;
12 |     for (int i = 0; i < n; i++) {
13 |         t2[i] += t1[i];
14 |         if (t2[i] > 1) {
15 |             t2[i] -= 2;
16 |             t2[i + 1] += 1;
17 |         }
18 |     }
19 |     while (t2[n] > 1) {
20 |         n++;
21 |     }
22 |     for (int i = n - 1; i >= 0; i--)
23 |         printf("%d", t2[i]);
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

10110111

样例 1 输出

110000100111

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.82 | 总 355/526 神之天平 ( PID 1785 )

出处：2022级HAUT新生周赛（四）@张子豪专场（H，CID 1082）| 训练定位：进阶 | 数学、数论与组合

标签：大整数、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：12/30 ( 40.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行，一个正整数，被释放的调查员灵魂的最大数量

输入要点：第一行，一个正整数 $n$ ，调查员的数量。  $1 \leq n \leq 1e5$

第二行， $n$ 个正整数，代表第 $i$ 个调查员灵魂的重量。  $1 \leq \text{灵魂重量} \leq 1e6$ ;

输出要点：一行，一个正整数，被释放的调查员灵魂的最大数量

题面提示：提示：

灵魂重量之和可能会爆int

样例说明：

从左往右拿前三个和后四个，左边和右边重量均为30，此时释放的灵魂最多，为7个

### 零基础先修

- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：神之天平：分析：用 $i, j$ 分别表示左端点和右端点， $suml, sumr$ 表示目前左托盘和右托盘的灵魂重量。 $ans$ 为答案， $res$ 为距离上一次天平平衡，左右托盘新增灵魂数量 两个指针相遇前，进行循环，每次循环进行判断：  
1.  $suml < sumr$  将指针 $i$ 右移， $suml$ 加上 $w[i]$   
2.  $suml > sumr$  将指针 $j$ 左移， $sumr$ 加上 $w[j]$  3.  $suml = sumr$  更新 $ans$ 和 $res$ ，且将指针 $i$ 右移， $suml$ 加上 $w[i]$ 或将指针 $j$ 左移， $sumr$ 加上 $w[j]$
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int w[100005];
3 | int n;
4 | int ans;
5 | int main() {
6 |     scanf("%d", &n);
7 |     for (int i = 1; i <= n; i++) {
8 |         scanf("%d", &w[i]);
9 |     }
10 |     long long suml = w[1], sumr = w[n], res = 2;
11 |     for (int i = 1, j = n; i < j;) {
12 |         while (suml < sumr && i < j) {
13 |             i++;
14 |             suml += w[i];
15 |             res++;
16 |         }
17 |         while (suml > sumr && i < j) {
18 |             j--;
19 |             sumr += w[j];
20 |             res++;
21 |         }
22 |         if (suml == sumr) {
23 |             i++;
24 |             suml += w[i];
25 |             ans += res;
26 |             res = 1;
27 |         }
28 |     }
29 |     printf("%d\n", ans);
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

```
9
7 3 20 5 15 1 11 8 10
```

样例 1 输出

```
7
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.83 | 总 356/526 寻找相似度 (PID 1794)

出处：2022级HAUT新生周赛（六）@张鑫专场（A，CID 1084）| 训练定位：进阶 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：1/6 (16.7%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：尤里和尤安是一对亲兄弟，在他们的家乡有一种非常有趣的游戏，两个人分别说出一个非负整数 $x$ ， $y$ ，比较每个对应的位数 $i$  ( $i \geq 1$ )，如果第 $i$ 位上的值 $a[i]$   $b[i]$ 相同且大于0则第 $i$ 位的相似度为 $i$ ，反之则为0，而两个数字 $x$ ， $y$ 的相似度为各个位数相似度的和（比如：123和145， $i=3$ 时 $a[i]=b[i]$ ，所以第3位相似度为3，其他位数相似度为0，总相似度为3）……

输入要点：第一行输入一个正整数 $t$ ，表示有 $t$ 组测试样例，接下来 $t$ 行每行两个正整数 $x$ ， $y$ 。

输出要点：输出 $t$ 行，每行两个整数 $sum1$ ， $sum2$ ，分别表示交换总次数的最小值、最大相似度。



题面提示：x,y<=1e9,t<=1e5

样例：8，9的二进制表示分别为（1000）（1001）无需交换发现第四位相似度为4，其他位数（无法匹配）相似度为0

2，9的二进制表示分别为（10）（1001）9交换第2位和第4位（0011）（10）发现第二位相似度为2，其他位数（无法匹配）相似度为0

（注意：不存在的位数不可以交换）

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：寻找相似度（模拟）计算出两个数的二进制形式1的个数并选择最小的个数为总匹配对数，从最小的数字的二进制最高位开始一一匹配，匹配过程中如果某个位置不为1则需要移动。（其实就是从最高可以匹配的位置开始把1尽量堆到前面）
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int fun(int x, int a[], int* ans) // 转换成二进制，储存1的个数并返回最高位数
3 | {
4 |     int n = 0;
5 |     while (x) { // x&1就是判断二进制最低位是否为1即是否是偶数
6 |         if (x & 1)
7 |             a[++n] = 1, (*ans)++;
8 |         else
9 |             a[++n] = 0;
10 |         x >>= 1; // 右移一位，就是x/=2;
11 |     }
12 |     return n;
13 | }
14 | int min(int x, int y) // 返回最小值
15 | {
16 |     if (x > y)
17 |         return y;
18 |     else
19 |         return x;
20 | }
21 | int main() {
22 |     int t;
23 |     int a[100], b[100];
24 |     scanf("%d", &t);
25 |     while (t--) {
26 |         int x, y;
27 |         int sum1 = 0, sum2 = 0, n1 = 0, n2 = 0, ans1 = 0, ans2 = 0;
28 |         // sum1记录交换次数，sum2记录相似度//n1,n2记录x,y二进制最高位数//ans1,ans2分别记录x,y
29 |         // 数字1的个数
30 |         scanf("%d%d", &x, &y);
31 |         n1 = fun(x, a, &ans1), n2 = fun(y, b, &ans2);
32 |         for (int i = min(n1, n2), j = min(ans1, ans2); i >= 1 && j >= 1;
33 |             i--, j--) { // i从最高的有效位数开始，j从x,y最多有效匹配的对数开始
34 |             sum2 += i; // 尽量让最高位匹配
```

```

35 |         if (a[i] == 0)
36 |             sum1++;
37 |         if (b[i] == 0)
38 |             sum1++; // 如果为0则交换
39 |     }
40 |     printf("%d %d\n", sum1, sum2);
41 | }
42 | return 0;
43 | }

```

## 公开样例

样例 1 输入

```

2
8 9
2 9

```

样例 1 输出

```

0 4
1 2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.84 | 总 357/526 最大数 ( PID 1797 )

出处：2022级HAUT新生周赛（六）@张鑫专场（D，CID 1084）| 训练定位：进阶 | 数学、数论与组合

标签：字符串、数学、大整数、排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：9/110（8.2%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**一天汉斯教授收到了一个陌生的邮件，发现有绑匪绑架了他的孙女并且他们在邮件中留下了一些线索，汉斯教授需要根据这些线索救出他的孙女。线索是一个很大的正整数，数字越大线索的有效性就越大，现在汉斯教授需要重新排列这些线索并让它们首尾相连后组成的总线索的有效性尽量大，请问有效性最大的总线索是多少？

**输入要点：**第一行输入一个正整数 $n$ ，第二行输入 $n$ 个正整数 $a_i(1 \leq i \leq n)$ 。

**输出要点：**输出最大的组合数字。

题面提示： $n \leq 30, a_i \leq 1e31$ （不含前导0）

（输入的数据很大，考虑用字符串储存）

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。

3. 核心处理采用：最大数也就是最大的约数 } for (int i = maxn; i >= 1 && ans <= n; i--)//i从maxn开始以满足每种项链的美丽值最大 { if(num[i]>=ans) { sum[i]++;//sum记录对应约数(美丽值)的出现次数 if(sum[i]>res) w = i, res = sum[i];//res记录次数的最大值，w记录对应的美丽值 ans++;//ans为你需要选择的宝石数，如果num[i]成功则组成的
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | char a[100][100]; // 由于数字很大所以选择用字符串储存
4 | void swap(char x[], char y[]) // 字符串交换
5 | {
6 |     char c[100];
7 |     strcpy(c, x), strcpy(x, y), strcpy(y, c);
8 | }
9 | int judge(char x[], char y[]) {
10 |     char x1[100], y1[100]; // x1代表不交换的组合, y1代表交换的组合
11 |     strcpy(x1, x), strcpy(y1, y); // x,y复制到x1,y1
12 |     strcat(x1, y), strcat(y1, x); // 构建x+y和y+x形式的字符串
13 |     return strcmp(x1, y1); // 比较大小并返回, 判断交换位置是否有利
14 | }
15 | void sort(int n) // 选择排序
16 | {
17 |     for (int i = 1; i <= n; i++)
18 |         for (int j = i + 1; j <= n; j++)
19 |             if (judge(a[i], a[j]) < 0)
20 |                 swap(a[i], a[j]);
21 | }
22 | int main() {
23 |     int n;
24 |     scanf("%d", &n);
25 |     for (int i = 1; i <= n; i++)
26 |         scanf("%s", a[i]);
27 |     sort(n); // 排序
28 |     for (int i = 1; i <= n; i++)
29 |         printf("%s", a[i]); // 直接输出
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

```
3
123 32 9
```

样例 1 输出

```
932123
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.85 | 总 358/526 宝石 ( PID 1798 )

出处：2022级HAUT新生周赛（六）@张鑫专场（E，CID 1084）| 训练定位：进阶 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/14（14.3%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：在遥远的大麦哲伦星系有一颗盛产宝石的行星，这种宝石可以发出美丽的光芒因此广受宇宙公民的喜爱，现在一支地球的采矿小队来到了这颗神奇的行星并且成功开采了 $n$ 颗宝石，他们准备利用这些宝石制作项链，每个项链都有一个美丽值，他们希望获得美丽值最大的项链，每一颗宝石都有一个光芒值 $a_i(i=1..n)$ ，项链的美丽值等于组成项链的宝石的光芒值的最大公约数（比如由光芒值为 6.....

输入要点：第一行输入一个正整数 $n$ ，第二行输入 $n$ 个正整数 $a_i(i=1..n)$ 。

输出要点：一个正整数 $w$ ，表示出现次数最多的 $f_i$ 的值。

题面提示： $n \leq 1e4$ ， $a_i \leq 1e6$

一个数的最大公约数即本身。

样例： $i=1$ ,选 ( 6 )  $f_1=6$ ；

$i=2$ ,选 ( 6,3 )  $f_2=3$ ；

$i=3$ ,选 ( 2,2,4 )  $f_3=2$ ；

$i=4$ ,选 ( 2,2,4,6 )  $f_4=2$ ；

$i=5$ ,选 ( 1,2,2,3,4 )  $f_5=1$ ；

$i=6$ ,选 ( 1,2,2,4,3,6 )  $f_6=1$ ；

发现 $i=3$ 和 $i=4$ 有 $f_i=2$ ，出现两次所以 $w=f_i=2$ 。

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：宝石数加一 } else i--; //如果这个约数无法满足要求则遍历其他约数 } printf("%d", w); return 0; }
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

通常为  $O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和 n=1。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int num[1000010], sum[1000010];
3 | void func(int x) // 求每个数的约数
4 | {
5 |     for (int i = 1; i <= x / i; i++)
6 |         if (x % i == 0) {
7 |             num[i]++; // 标记每种约数的出现次数
8 |             if (x / i != i)
9 |                 num[x / i]++;
10 |        }
11 |    }
12 | int max(int x, int y) // 返回最大值
13 | {
14 |     if (x > y)
15 |         return x;
16 |     else
17 |         return y;
18 | }
19 | int main() {
20 |     int n, maxn = 0, ans = 1, w = 0, res = 0;
21 |     scanf("%d", &n);
22 |     for (int i = 1; i <= n; i++) {
23 |         int x;
24 |         scanf("%d", &x);
25 |         func(x);
26 |         maxn = max(x, maxn); // maxn记录最大数也就是最大的约数
27 |     }
28 |     for (int i = maxn; i >= 1 && ans <= n;) // i从maxn开始以满足每种项链的美丽值最大
29 |     {
30 |         if (num[i] >= ans) {
31 |             sum[i]++; // sum记录对应约数(美丽值)的出现次数
32 |             if (sum[i] > res)
33 |                 w = i, res = sum[i]; // res记录次数的最大值, w记录对应的美丽值
34 |             ans++; // ans为你需要选择的宝石数, 如果num[i]成功则组成的宝石数加一
35 |         } else
36 |             i--; // 如果这个约数无法满足要求则遍历其他约数
37 |     }
38 |     printf("%d", w);
39 |     return 0;
40 | }
```

## 公开样例

样例 1 输入

```
6
1 2 2 4 3 6
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.86 | 总 359/526 二进制扩列 ( PID 1819 )

出处：2023级HAUT新生周赛（一）@21级学长专场（张子豪，张鑫，李昊阳）（C，CID 1086）| 训练定位：进阶 | 数学、数论与组合

标签：字符串、数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：35/249 ( 14.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，最初的字符串可能的最短长度。

输入要点：一行一个整数 $n$ 表示最终的二进制字符串长度（ $1 \leq n \leq 100000$ ）

一行一个字符串仅由0和1构成。

输出要点：一个整数，最初的字符串可能的最短长度。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：二进制扩列 由题意知原先的字符串经过扩展后必定是一边一个0另一边一个1，所以使用两个变量标记左右边界，如果符和一边是0另一边是1则符和条件，可以继续收缩。由于你要得到最短的最初的字符串，所以你只要找到不符合的位置，然后输出此时字符串的长度即可。c c++
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

通常为  $O(n)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | char s[100005];
3 | int main() {
4 |     int n;
5 |     scanf("%d%s", &n, s);
6 |     int l = 0, r = n - 1, ans = 0;
7 |     while (s[l] != s[r] && l < r) {
8 |         l++, r--, ans++;
9 |     }
10 |    printf("%d\n", n - ans * 2);
11 |    return 0;
12 | }
```

### 公开样例

样例 1 输入

```
6
110000
```

样例 1 输出

```
2
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

5.87 | 总 360/526 小明的午餐难题 ( PID 1826 )

出处：河南工大2024新生周赛（1）——命题人：程立老师，陈兰锴（H，CID 1091）；2023级HAUT新生周赛（二）@曹瑞峰专场（B，CID 1087） | 训练定位：进阶 | 数学、数论与组合

标签：数学 | 限制：1.000 S / 128 MB | 历史统计：458/912 ( 50.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：9:00,10:00,11:00,11:55...终于快放学了，伴随着时钟的滴答声，小明的思绪早已飞向餐厅，今天是吃一餐的烤鱼，还是二餐的烤鸡，又或是三餐的牛肉面。。。咕噜~咕噜~饥饿的肚子容不得小明多想，他必须奔向最近的餐厅来拯救他的肚子。学校的四个餐厅距离小明分别为a米、b米、c米、d米、小明的速度是1m/s。那么，小明最少花多长时间才能拯救他的肚子？

输入要点：依次输入四个整数a，b，c，d，代表四个餐厅与小明的距离。（a,b,c,d<=1000000）

输出要点：输出一个整数，代表小明最短需要多久来拯救他的肚子。

零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：小明的午餐难题 找到a,b,c,d中的最小值，并将其输出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

常数级或一次线性读入；以代码中的循环层数为准

易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

C++17 参考实现

```

1 | #include <stdio.h>
2 | int main() {
3 |     int a, b, c, d;
4 |     scanf("%d %d %d %d", &a, &b, &c, &d);
5 |     int minnum = a;
6 |     if (minnum > b)
7 |         minnum = b;
8 |     if (minnum > c)
9 |         minnum = c;
10 |    if (minnum > d)
11 |        minnum = d;
12 |    printf("%d", minnum);
13 |    return 0;
14 | }

```

## 公开样例

样例 1 输入

516 334 769 392

样例 1 输出

334

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 5.88 | 总 361/526 派蒙爱吃素 ( PID 1840 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（D，CID 1088）| 训练定位：进阶 | 数学、数论与组合

标签：数论、素数判定、快速幂 | 限制：0.100 S / 128 MB | 历史统计：23/385 (6.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：也就是说，请你判断 $a \times b$ 是否是素数。

输入要点：输入两个整数  $a, b$  ( $1 \leq a, b \leq 10^{14}$ )。

输出要点：若输入满足 $a \times b$ 是素数，输出一行"YES"，否则输出一行"NO"（没有引号）

题面提示：本题时间要求非常苛刻

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：两个正整数的乘积是素数，当且仅当其中一个等于 1，另一个本身是素数。于是先做这个必要条件判断，再对最多  $1e14$  的另一个数做 64 位确定性 Miller-Rabin。乘法用 \_\_int128，避免模乘时溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明



- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(\log^3 n)$  量级时间、 $O(1)$  空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using u64 = unsigned long long;
4 | using u128 = __uint128_t;
5 | u64 powerMod(u64 a, u64 e, u64 mod) {
6 |     u64 res = 1;
7 |     while (e) {
8 |         if (e & 1)
9 |             res = (u128)res * a % mod;
10 |         a = (u128)a * a % mod;
11 |         e >>= 1;
12 |     }
13 |     return res;
14 | }
15 | bool isPrime(u64 n) {
16 |     if (n < 2)
17 |         return false;
18 |     for (u64 p :
19 |         {2ULL, 3ULL, 5ULL, 7ULL, 11ULL, 13ULL, 17ULL, 19ULL, 23ULL, 29ULL, 31ULL, 37ULL}) {
20 |         if (n % p == 0)
21 |             return n == p;
22 |     }
23 |     u64 d = n - 1, s = 0;
24 |     while ((d & 1) == 0)
25 |         d >>= 1, ++s;
26 |     for (u64 a : {2ULL, 325ULL, 9375ULL, 28178ULL, 450775ULL, 9780504ULL, 1795265022ULL}) {
27 |         if (a % n == 0)
28 |             continue;
29 |         u64 x = powerMod(a % n, d, n);
30 |         if (x == 1 || x == n - 1)
31 |             continue;
32 |         bool composite = true;
33 |         for (u64 r = 1; r < s; ++r) {
34 |             x = (u128)x * x % n;
35 |             if (x == n - 1) {
36 |                 composite = false;
37 |                 break;
38 |             }
39 |         }
40 |         if (composite)
41 |             return false;
42 |     }
43 |     return true;
44 | }
45 | int main() {
46 |     u64 a, b;
47 |     cin >> a >> b;
48 |     bool ans = (a == 1 && isPrime(b)) || (b == 1 && isPrime(a));
49 |     cout << (ans ? "YES" : "NO") << '\n';
50 |     return 0;
51 | }
```

## 公开样例

样例 1 输入

2 3

样例 1 输出

NO

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

5.89 | 总 362/526 命运之夜 ( PID 1848 )

出处：2023级HAUT新生周赛（四）@牛浩然专场（D，CID 1089） | 训练定位：进阶 | 数学、数论与组合

标签：数论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/26 ( 11.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示小A有多少种方案召唤出最强英灵。

输入要点：第一行两个整数 $n, k$ ， $n$ 表示小A拥有的圣遗物的数量， $k$ 表示召唤出最强英灵需要的条件 ( $2 \leq n \leq 100000, 0 \leq k \leq 60$ )。

第二行 $n$ 个整数 ( $a_1, a_2, a_3 \dots a_n$ ) 第 $i$ 个数表示第 $i$ 件圣遗物的价值 ( $0 \leq a_i \leq 109$ )；

输出要点：一个整数，表示小A有多少种方案召唤出最强英灵。

题面提示：样例解释：

- 1.选择第一个和第三个， $1 \text{ AND } 3 = 2, 1 \text{ OR } 3 = 3$ ，2的二进制位中有一个1，3的二进制位中有两个1，二者之和大于等于3。
- 2.选择第三个和第四个， $1 \text{ AND } 3 = 2, 1 \text{ OR } 3 = 3$ ，2的二进制位中有一个1，3的二进制位中有两个1，二者之和大于等于3。
- 3.选择第二个和第三个， $2 \text{ AND } 3 = 2, 2 \text{ OR } 3 = 3$ ，2的二进制位中有一个1，3的二进制位中有两个1，二者之和大于等于3。

除此之外，没有别的组合了。

零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：命运之夜 与运算：如果两个数 $a, b$ 的第 $i$ 位都为1，那么与运算之后，答案的第 $i$ 位也为1. 或运算：如果两个数 $a, b$ 的第 $i$ 位中存在1个1，那么或运算之后，答案的第 $i$ 位为1；考虑 $a, b$ 第 $i$ 位的四种情况 1 1：与运算结果第 $i$ 位为1，或运算结果第 $i$ 位为1. 1 0：与运算结果第 $i$ 位为0，或运算结果第 $i$ 位为1. 0 1：与运算结果第 $i$ 位为0，或运算结果第 $i$ 位为1. 0 0：与运算结果第 $i$ 位为0，或运算结果第 $i$ 位为0. 不难发现，与运算和或运算的答案内1的个数和与运算前 $a, b$ 两数的1的个数和相同，那么，我们可以先预处理出来每个数的二进制位含多少1，cnt数组记录一下，然后枚举以它为 $i$ 时，大于 $i$ 的部分有多少下标满足条件（有多少数的二进制中1的个数大于等于 $k - cnt[i]$ ）。让 $ans +=$ 满足 $k - cnt[i]$ 的个数即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

复杂度

通常为  $O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和 n=1。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | long long arr[110000];
3 | int cnt[90]; // cnt[i]表示有多少数的二进制位中1的个数等于i。
4 | int crr[110000]; //crr[i]表示第i个数的二进制位中有多少1。
5 | long long sum[90]; // sum[i]表示有多少数的二进制位中1的个数大于等于i。
6 | int main() {
7 |     int n, k;
8 |     scanf("%d %d", &n, &k);
9 |     for (int i = 1; i <= n; ++i) {
10 |         scanf("%lld", &arr[i]);
11 |         int cnt1 = 0;
12 |         for (int j = 0; j <= 30; ++j) // 计算每个数中有多少1
13 |         {
14 |             int c = (1 << j);
15 |             if (c & arr[i]) {
16 |                 cnt1++;
17 |             }
18 |         }
19 |         crr[i] = cnt1;
20 |         cnt[cnt1]++;
21 |     }
22 |     for (int i = 70; i >= 0; --i) {
23 |         sum[i] = sum[i + 1] + cnt[i];
24 |     }
25 |     int c = 0;
26 |     long long ans = 0;
27 |     for (int i = 1; i <= n; ++i) {
28 |         int t = k - crr[i];
29 |         for (
30 |             int j = crr[i]; j >= 0;
31 |             --j) // 将自身减去。保证当前sum[j]表示的是大于i中有多少数的二进制位中1的个数大于等于i。避免重复。
32 |         {
33 |             sum[j]--;
34 |         }
35 |         if (t < 0)
36 |             t = 0;
37 |         // cout<<t<<endl;
38 |         ans += sum[t];
39 |     }
40 |     printf("%lld\n", ans);
41 |     return 0;
42 | }
```

## 公开样例

样例 1 输入

```
4 3
1 2 3 1
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.90 | 总 363/526 星空列车与白的旅行 ( PID 1885 )

出处：河南工大2024新生周赛（4）——命题人：马贺（G，CID 1094） | 训练定位：进阶 | 数学、数论与组合

标签：奇偶性、区间计数、数学 | 限制：0.200 S / 128 MB | 历史统计：17/213（8.0%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：诺瓦是个很可爱的女孩，有着猫尾猫耳，而且总是呆呆的样子。诺瓦有一个可爱值的属性，在时间 $t$ 秒的时候，诺瓦的可爱值会增加 $t^t$ ，而这一秒增加的可爱值会持续 $k$ 秒。如果在时间等于 $n$ 秒的时候，诺瓦总的可爱值为奇数，那么你将不受控制，会被诺瓦吸引导致你被其他乘客当成hentai。你不想被当成hentai，所以请你判断，在时间为 $n$ 秒的时候，诺瓦的可爱值是否为奇数。如果可爱.....

输入要点：两个整数， $n$ 和 $k$ 。

(  $1 \leq n \leq 1000000000$  ) (  $1 \leq k \leq n$  )

输出要点："YES"或者"NO"

( 可爱值为偶数输出"YES"，反之输出"NO" )

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116172451\\_82893.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116172451_82893.png)]

诺瓦小提示：不要遍历哦，会超时的，这是数学题，好好思考一下吧（^^）

### 零基础先修

- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用： $t^t$  的奇偶性只由  $t$  决定： $t$  为奇数时  $t^t$  为奇数，否则为偶数。因而只需数区间  $[n-k+1, n]$  中有多少奇数；奇数项个数为偶数时总和为偶数，输出 YES，否则输出 NO。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, k;
5 |     cin >> n >> k;
6 |     long long left = n - k + 1;
7 |     long long oddCount = (n + 1) / 2 - left / 2;
8 |     cout << (oddCount % 2 == 0 ? "YES" : "NO") << '\n';
9 |     return 0;
10 | }
```

公开样例

|         |
|---------|
| 样例 1 输入 |
| 1 1     |
| 样例 1 输出 |
| NO      |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

5.91 | 总 364/526 哥德巴赫猜想 ( PID 1892 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭钰 ( F , CID 1095 ) | 训练定位：进阶 | 数学、数论与组合  
标签：条件分支、数论、大整数、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/708 ( 0.4% )  
代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：现在不需要你验证哥德巴赫猜想，只需要验证该数是否为质数即可。如果是，输出“YeS”，如果不是，输出“NO”

输入要点：输入包含 1 行

输入一个数字 n ( 1 <= n <= 1e18 )

输出要点：如果 n 是质数 输出“YeS”；

否则输出 "NO"；

题面提示：注意数据范围哦

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 估算最大值：若乘积可能超过约  $9\times10^{18}$ ，就需 \_\_int128 或高精度。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：哥德巴赫猜想 验证输入的n是否为质数 需要用到Miller\_Rabin算法 注意数据范围 本题了解即可
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。

- 因此所得数值正是题目定义的答案。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define int long long
4 | int prime[10] = {2, 3, 5, 7, 11, 13, 17, 19, 23, 29};
5 | int mul(int a, int b, int c) // 快速积 (和快速幂差不多)
6 | {
7 |     long long ans = 0, res = a;
8 |     while (b) {
9 |         if (b & 1)
10 |             ans = (ans + res) % c;
11 |         res = (res + res) % c;
12 |         b >>= 1;
13 |     }
14 |     return (int)ans;
15 | }
16 | int Quick_Power(int a, int b, int c) // 快速幂, 这里就不赘述了
17 | {
18 |     int ans = 1, res = a;
19 |     while (b) {
20 |         if (b & 1)
21 |             ans = mul(ans, res, c);
22 |         res = mul(res, res, c);
23 |         b >>= 1;
24 |     }
25 |     return ans;
26 | }
27 | bool Miller_Rabin(int x) // 判断素数
28 | {
29 |     int i, j, k;
30 |     int s = 0, t = x - 1;
31 |     if (x == 2)
32 |         return true; // 2是素数
33 |     if (x < 2 || !(x & 1))
34 |         return false; // 如果x是偶数或者是0,1, 那它不是素数
35 |     while (!(t & 1)) // 将x分解成(2^s)*t的样子
36 |     {
37 |         s++;
38 |         t >>= 1;
39 |     }
40 |     for (i = 0; i < 10 && prime[i] < x; ++i) // 随便选一个素数进行测试
41 |     {
42 |         int a = prime[i];
43 |         int b = Quick_Power(a, t, x); // 先算出a^t
44 |         for (j = 1; j <= s; ++j) // 然后进行s次平方
45 |         {
46 |             k = mul(b, b, x); // 求b的平方
47 |             if (k == 1 && b != 1 && b != x - 1) // 用二次探测判断
48 |                 return false;
49 |             b = k;
50 |         }
51 |         if (b != 1)
52 |             return false; // 用费马小定律判断
53 |     }
54 |     return true; // 如果进行多次测试都是对的, 那么x就很有可能是素数
55 | }
56 | signed main() {
57 |     int x;
58 |     cin >> x;
59 |     if (Miller_Rabin(x))
60 |         cout << "Yes";
61 |     else
62 |         cout << "No";
63 |     return 0;
64 | }

```

## 公开样例

样例 1 输入

4

样例 1 输出

NO

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.92 | 总 365/526 魔法水晶的三角形测试 ( PID 1940 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( H , CID 1104 ) | 训练定位：进阶 | 数学、数论与组合

标签：条件分支、数学 | 限制：1.000 S / 128 MB | 历史统计：110/477 ( 23.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，若不能构成三角形：输出一行 “It cannot form an energy triangle.”；若能构成三角形：输出两行，第一行输出魔法属性（按上述优先级选择），第二行输出能量值（保留两位小数）。

输入要点：一行三个正整数 $a, b, c$  ( $1 \leq a, b, c \leq 1000$ )，整数之间用空格分隔。

输出要点：若不能构成三角形：输出一行 “It cannot form an energy triangle.”；

若能构成三角形：输出两行，第一行输出魔法属性（按上述优先级选择），第二行输出能量值（保留两位小数）。

题面提示：[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251111/20251111133204\\_27828.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251111/20251111133204_27828.png)]

### 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：魔法水晶的三角形测试 根据所给边判断是否能构成的三角形，如果可以构成三角形，求三角形类型并求面积。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | #include <math.h>
4 | int main() {
5 |     int a, b, c;
6 |     scanf("%d %d %d", &a, &b, &c);
7 |     if (a + b > c && b + c > a && a + c > b) {
8 |         if (a == b && b == c)
9 |             printf("Perfect equilateral crystal!!!");
10 |        else {
11 |            int f1 = 0, f2 = 0;
12 |            if (a == b || a == c || b == c)
13 |                f1 = 1;
14 |            if (a * a + b * b == c * c || a * a + c * c == b * b || b * b + c * c == a * a)
15 |                f2 = 1;
16 |            if (f1 + f2 == 2)
17 |                printf("Isosceles right-angle crystal!!");
18 |            else if (f1 == 1)
19 |                printf("Isosceles crystal!");
20 |            else if (f2 == 1)
21 |                printf("Right-angle crystal!");
22 |            else
23 |                printf("Common triangular crystal.");
24 |        }
25 |        printf("\n");
26 |        double p = (a + b + c) / 2.0;
27 |        double ans = sqrt(p * (p - a) * (p - b) * (p - c));
28 |        printf("%.2lf", ans);
29 |    } else
30 |        printf("It cannot form an energy triangle.");
31 |    return 0;
32 | }
```

### 公开样例

样例 1 输入

```
3 4 5
```

样例 1 输出

```
Right-angle crystal!
6.00
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 5.93 | 总 366/526 Crychic的解散 ( PID 1949 )

出处：河南工大2025新生周赛（4）——命题人：张梦淇（H，CID 1105） | 训练定位：进阶 | 数学、数论与组合

标签：数论、大整数、字符串 | 限制：1.000 S / 128 MB | 历史统计：68/290（23.4%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：祥子想要退出Crychic，为了阻止她，素世给了她一个整数n。只有她求出n的每一位之积的末尾0的个数才能退出。素世不想祥子退出，因此写的整数非常大(0<=|n|<=10100000)。

输入要点：只有一行，为一个整数n。



输出要点：请输出n的每一位数字乘积的末尾0的个数。

题面提示：这个数真的很大！

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 \_\_int128 或高精度。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：一位数字的质因数只可能为 2、3、5、7；乘积末尾零的个数等于 因子 2 与 因子 5 总数的较小值。逐位累加即可，不需要保存或转换这个 超大整数。题目样例把单个数字 0 的乘积视为 0，并规定答案为 1。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(|n|)$  时间、 $O(|n|)$  读入空间

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     string n;
7 |     cin >> n;
8 |     if (n.find('0') != string::npos) {
9 |         cout << 1 << '\n';
10 |         return 0;
11 |     }
12 |     long long twos = 0, fives = 0;
13 |     for (char c : n) {
14 |         int digit = c - '0';
15 |         while (digit % 2 == 0)
16 |             ++twos, digit /= 2;
17 |         while (digit % 5 == 0)
18 |             ++fives, digit /= 5;
19 |     }
20 |     cout << min(twos, fives) << '\n';
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

0

样例 1 输出

1

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 5.94 | 总 367/526 卡厄斯兰娜的33550336次轮回 ( PID 1951 )

出处：河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇 ( C , CID 1105 ) | 训练定位：进阶 | 数学、数论与组合

标签：数论、完美数、查表 | 限制：1.000 S / 128 MB | 历史统计：9/307 ( 2.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，请输出第 $n$ 世承载火种的是第几世的卡厄斯兰娜。

输入要点：给出一个正整数 $n$ ，表示第 $n$ 世( $0 \leq n \leq 1e10$ )的卡厄斯兰娜。

输出要点：请输出第 $n$ 世承载火种的是第几世的卡厄斯兰娜。

题面提示：完美数：一个数的所有真因数(除自身之外的因数)之和等于其自身。

当 $p \leq 210$ ，只有 $2^p - 1$ 为质数时，才会有 $(2^p - 1) \cdot 2^{p-1}$ 为一个完美数。

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：承载者只会在“完成一个完美数轮回”之后更新，因此需要找严格小于  $n$  的最大完美数；在第 0 世或尚未经过第一个完美数时输出 0。 $1e10$  内的偶完美数可由欧几里得—欧拉公式列出，逐个更新最近值即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     unsigned long long n;
5 |     cin >> n;
6 |     const vector<unsigned long long> perfect = {6ULL,      28ULL,      496ULL,
7 |                                                  8128ULL, 33550336ULL, 8589869056ULL};
```

```
8 | unsigned long long ans = 0;
9 | for (auto x : perfect) {
10 |     if (x < n)
11 |         ans = x;
12 |     else
13 |         break;
14 | }
15 | cout << ans << '\n';
16 | return 0;
17 | }
```

## 公开样例

样例 1 输入

0

样例 1 输出

0

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数学、数论与组合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 6 | 贪心与博弈

贪心不能只凭直觉，需要交换论证、下界与可达构造或不变量；博弈题先找终止条件、必胜态和奇偶性。

本模块共 67 道唯一题。

## 6.1 | 总 368/526 Simple学长买咖啡 ( PID 1238 )

出处：2016级河南工大新生周赛 ( 六 ) ( B , CID 1012 ) | 训练定位：入门 | 贪心与博弈

标签：贪心、向上取整、数学 | 限制：1.000 S / 128 MB | 历史统计：165/336 ( 49.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，Print the minimum number of steps that Simple needs to make to get from point 0 to point x.

输入要点：The first T indicates there are T groups of date.

The first line of every group contains an integer x ( $1 \leq x \leq 1\,000\,000$ ) — The coordinate of the shop.

输出要点：Print the minimum number of steps that Simple needs to make to get from point 0 to point x.

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：每步最多走 5 个单位，为减少步数应尽量每步走满 5。到距离 x 至少需要  $\text{ceil}(x/5)$  步，而用若干个 5 加上最后一个 1..4 的余量可以达到，所以下界可实现。整数公式为  $\lceil (x+4)/5 \rceil$ 。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long x;
10 |         cin >> x;
11 |         cout << (x + 4) / 5 << '\n';
12 |     }
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

```
2
5
12
```

样例 1 输出

```
1
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.2 | 总 369/526 Choice学姐买糖果 IV ( PID 1317 )

出处：2017级河南工大新生周赛（二）（D，CID 1029）| 训练定位：入门 | 贪心与博弈

标签：鸽巢原理、最坏情况、贪心 | 限制：1.000 S / 128 MB | 历史统计：41/85 ( 48.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出要买的糖果个数，答案占一行。

输入要点：第一行输入两个整数 $n, k$  ( $1 \leq n, k \leq 1e4$ )。

接下来一行 $n$ 个空格分隔的整数 $m_i$ , 表示第 $i$ 种糖果的个数。 ( $1 \leq m_i \leq 1e9$ )。 (数据保证有解)

输出要点：输出要买的糖果个数，答案占一行。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：在尚未有任何一种达到 $k$ 个的前提下，第 $i$ 种最多能买 $\min(m_i, k-1)$ 个。把各类这个上限相加，就是“仍可能不满足要求”的最大总数；再多买一个，无论它属于哪一类，必有一类达到 $k$ 。数据保证至少有一种 $m_i \geq k$ ，所以答案是该和加1。这是带库存上限的鸽巢原理。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回`int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long k;
8 |     cin >> n >> k;
9 |     long long safeMaximum = 0;
10 |    for (int i = 0; i < n; ++i) {
11 |        long long count;
12 |        cin >> count;
13 |        safeMaximum += min(count, k - 1);
14 |    }
15 |    cout << safeMaximum + 1 << '\n';
16 |    return 0;
17 | }
```

## 公开样例

样例 1 输入

```
5 60
100 200 300 400 500
```

样例 1 输出

```
296
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.3 | 总 370/526 决胜的抓硬币！（PID 1403）

出处：2018级河南工大新生周赛（四）@杜锐专场（B，CID 1043）| 训练定位：入门 | 贪心与博弈

标签：博弈论、取硬币、取模 | 限制：1.000 S / 128 MB | 历史统计：310/639 (48.5%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你的任务是，判断当有  $N$  ( $N < 1 \times 10^6$ ) 个硬币时，谁会获胜。

输入要点：一个整数  $N$  ( $N < 10^6$ )。

输出要点：如果雷顿获胜，输出。

Hershel Layton will win.

否则：

Hershel Layton will lose.

题面提示：当有三个硬币时，无论雷顿抓取几个，布洛涅夫都可以拿到最后一个硬币。

### 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每次可取 1 或 2 枚，3 的倍数是必败态：无论先手取多少，后手 都能补成一轮合计 3。非 3 的倍数先取余数，即可把 3 的倍数留给 对手。因此  $n \% 3 \neq 0$  时雷顿获胜。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     cin >> n;
6 |     if (n % 3)
7 |         cout << "Hershel Layton will win.\n";
```

```
8 |     else
9 |         cout << "Hershel Layton will lose.\n";
10 |     return 0;
11 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

Hershel Layton will lose.

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.4 | 总 371/526 取物品 ( PID 1440 )

出处：2018级河南工大新生周赛（七）@牛寅旭专场（D，CID 1046）| 训练定位：入门 | 贪心与博弈

标签：博弈论、威佐夫博弈、黄金比例 | 限制：1.000 S / 128 MB | 历史统计：0/0（无公开统计）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出胜者。

输入要点：输入有一行， $x, y$ 。

$x$ 为第一堆物品的数量。

$y$ 为第二堆物品的数量。

$0 < x, y < 10^9$

输出要点：输出胜者。

### 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：这是威佐夫博弈。令  $x \leq y$ 、差  $z = y - x$ ，黄金比例  $\varphi = (1 + \sqrt{5})/2$ ；冷态恰为  $(\text{floor}(z\varphi), \text{floor}(z\varphi) + z)$ 。命中冷态时后手 B 必胜，否则先手 A 能一步转入某个冷态。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long x, y;
5 |     cin >> x >> y;
6 |     if (x > y)
7 |         swap(x, y);
8 |     long long diff = y - x;
9 |     long long cold = (long long)(diff * ((1.0L + sqrtl(5.0L)) / 2.0L));
10 |    cout << (cold == x ? 'B' : 'A') << '\n';
11 |    return 0;
12 | }
```

## 公开样例

样例 1 输入

5 2

样例 1 输出

A

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

## 6.5 | 总 372/526 一觉醒来成为算法高手 ( PID 1982 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（D，CID 1110）| 训练定位：入门 | 贪心与博弈

标签：贪心、区间调度、排序 | 限制：1.000 S / 128 MB | 历史统计：82/138 ( 59.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，表示最多能参加的活动个数。

输入要点：输入共  $n+1$  行。

第一行只有一个正整数  $n$ ，表示一共有  $n$  个活动。 ( $1 \leq n \leq 10000$ )

接下来  $n$  行每行 2 个整数  $a_i, b_i$ 。  $a_i$  和  $b_i$  表示第  $i$  个活动的开始时间和结束时间。 ( $0 \leq a_i \leq 1000$ ) , ( $0 \leq b_i \leq 1000$ )

输出要点：输出一个整数，表示最多能参加的活动个数。

题面提示：我真的睡醒了吗QAQ

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导



1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：这是区间调度：按结束时间从小到大排序，每次选择开始时间不早于 上一个已选区间结束时间的区间。结束越早，留给后续课程的时间越多，这一局部选择可通过交换论证得到最优解。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<pair<long long, long long>> seg(n);
9 |     for (auto& interval : seg) {
10 |         long long start, finish;
11 |         cin >> start >> finish;
12 |         interval = {finish, start};
13 |     }
14 |     sort(seg.begin(), seg.end());
15 |     long long last = LLONG_MIN;
16 |     int ans = 0;
17 |     for (auto [finish, start] : seg) {
18 |         if (start >= last) {
19 |             ++ans;
20 |             last = finish;
21 |         }
22 |     }
23 |     cout << ans << '\n';
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

```
5
14
35
06
57
89
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

6.6 | 总 373/526 取石子游戏 ( PID 1203 )

出处：2016级河南工大新生周赛（一）（F，CID 1006） | 训练定位：基础提高 | 贪心与博弈

标签：博弈论、必胜态、模 3 | 限制：1.000 S / 128 MB | 历史统计：20/50 ( 40.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，如果A获胜输出A，如果B获胜输出B

输入要点：输入一个正整数N ( N<100 )

输出要点：如果A获胜输出A，如果B获胜输出B

题面提示：对于样例：一开始该A取石子，很显然A可以取走1颗,2颗或4颗，如果A选择拿2颗，那么会剩下3颗，这个时候B无论拿1颗石子还是2颗石子，A都可以取走最后1颗赢得游戏

零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：2 的幂模 3 只会在 1、2 间交替。若石子数是 3 的倍数，无论当前 玩家取哪个合法幂，都会把非 3 倍数局面留给对手；若不是 3 的倍数，当前玩家可按余数取 1 或 2，把 3 的倍数留给对手。于是 3 的倍数为 必败态，先手 A 在其他局面必胜。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(1) 时间、O(1) 空间

易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     cout << (n % 3 == 0 ? "B" : "A") << '\n';
7 |     return 0;
8 | }
```

公开样例

样例 1 输入

5

样例 1 输出

A

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.7 | 总 374/526 Let's play ( PID 1210 )

出处：2016级河南工大新生周赛（二）（E，CID 1007）| 训练定位：基础提高 | 贪心与博弈

标签：博弈论、棋盘染色、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：19/40 ( 47.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：一个  $n \times n$  的棋盘，一开始的时候棋子在一个角落的格子里，每次可以移动棋子到上下左右的相邻格子中（不能超过边界，而且不能走走过的格子）；当其中一人不能走时输，现在问你先手必胜还是后手必胜？如果先手胜请输出“FF”，后手胜请输出“AA”。

输入要点：输入若干个数字，每行一个数字  $N$  表示  $N \times N$  的矩阵，输入到  $N=0$  表示输入结束。

你可以用这样的代码输入：

```
while(scanf("%d",&N)&&N!=0)
```

```
{
```

```
....//对你的输入操作
```

```
}
```

输出要点：对于每个输入的非零数字  $N$  输出一行答案。详见样例：

题面提示：  $N \leq 1000000000$

### 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：棋盘按黑白二染色，每一步必在两种颜色间交替。起点在角落。 $n$  为奇数时起点颜色比另一色多一个，后手可用配对策略应对，使路径最终停在起点同色、轮到先手无路，故后手 AA 胜； $n$  为偶数时两色数量相同，相同配对思想由先手取得最后一次移动，输出 FF。结论只取决于  $n$  的奇偶，支持  $10^9$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。

- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long n;
7 |     while (cin >> n && n != 0)
8 |         cout << (n % 2 == 0 ? "FF" : "AA") << '\n';
9 |     return 0;
10 | }
```

## 公开样例

样例 1 输入

```
11377
1
0
```

样例 1 输出

```
AA
AA
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.8 | 总 375/526 零钱问题 ( PID 1225 )

出处：2016级河南工大新生周赛（四）（F，CID 1009）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、找零、整除取余 | 限制：1.000 S / 128 MB | 历史统计：186/338 ( 55.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：假设老板那边有面值 50，10，5，2，1 无数张，请你帮 zy 算算老板应该找他多少钱，钱张数最少是多少张。

输入要点：输入一个整数  $t$ ，表示 zy 花了  $t$  块钱

输出要点：输出 2 个整数  $m$  和  $n$ ，用空格隔开。表示老板找 zy  $m$  块钱，最少的张数是  $n$ 。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。

- 核心处理采用：找零额为  $100-t$ 。面额 50、10、5、2、1 构成规范系统，从最大面额起尽量使用：整除得到该面额张数，余数交给更小面额。若用若干小面额能替代一张更大面额，张数不会更少，因此贪心最优。输出找零额和张数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int spent;
5 |     cin >> spent;
6 |     int change = 100 - spent;
7 |     int remaining = change;
8 |     int notes = 0;
9 |     const int coin[] = {50, 10, 5, 2, 1};
10 |    for (int value : coin) {
11 |        notes += remaining / value;
12 |        remaining %= value;
13 |    }
14 |    cout << change << ' ' << notes << '\n';
15 |    return 0;
16 | }
```

## 公开样例

样例 1 输入

16

样例 1 输出

84 6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.9 | 总 376/526 此为背景 ( PID 1236 )

出处：2016级河南工大新生周赛（六）（A，CID 1012）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、前缀最小值、提前购买 | 限制：1.000 S / 128 MB | 历史统计：96/392 ( 24.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：Simple学长是一位集帅气与智慧于一身并且很奇怪的人，因为自从Simple学长打过某一场比赛之后，就有一个了怪癖，那就是每天都要喝很多瓶拿铁咖啡，明知道拿铁咖啡是禁忌品，但是嘴上却说着要什么以毒攻毒。所以如果他没喝够他那天想喝的咖啡，那么他

就写不出代码。但是我们学校的商店非常的奇怪，咖啡的价格每天都不一样。因为这个学长忙于敲代码，所以现在我们需要帮助这个学  
.....

输入要点：输入一个T，表示有T组数据

每组数据的第一行是一个n，表示有n天  $n \leq 100000$

接下来n行每一行有两个数，第一个数表示Simple学长第i天需要喝的咖啡数 $a_i$ ，第二个数表示第i天每瓶咖啡的价格 $p_i$

(  $1 \leq a_i, p_i \leq 100$  )

输出要点：输出一个数，表示n天需要的最少花费

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：咖啡可以提前购买且没有储存限制，所以第  $i$  天应按前  $i$  天出现过的 最低价格购买当天所需数量。扫描时维护 `minimumPrice`，并累加  $a_i \cdot \text{minimumPrice}$ 。若在更贵的某天购买，改为此前最低价时提前买入只会 降低费用，因此该策略最优。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        long long ans = 0;
12 |        int mn = INT_MAX;
13 |        for (int day = 0; day < n; ++day) {
14 |            int needed, price;
15 |            cin >> needed >> price;
16 |            mn = min(mn, price);
17 |            ans += 1LL * needed * mn;
18 |        }
19 |        cout << ans << '\n';
20 |    }
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
2
3
1 3
2 2
3 1
3
1 3
2 1
3 2
```

样例 1 输出

```
10
8
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.10 | 总 377/526 XXX班的团事活动 ( PID 1241 )

出处：2016级河南工大新生周赛（七）（B，CID 1014）| 训练定位：基础提高 | 贪心与博弈

标签：双指针、贪心、排序 | 限制：1.000 S / 128 MB | 历史统计：12/38 ( 31.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：一月一度的团事活动又来了，这次的活动是去郊游，可是呢，团支书立马就泼了一发冷水，说是我们的目的地在一个隔海的小岛上，需要乘独木舟才能到该小岛上，一条独木舟最多只能乘坐两个人，且乘客的总重量不能超过独木舟的最大承载量。并且独木舟的租费很贵，班费又有限。。我们要尽量减少这次活动中的花销，所以要找出可以安置所有学生的最少的独木舟条数，ykc真的很想去这次郊游，你能……

输入要点：第一行输入s,表示测试数据的组数；

每组数据的第一行包括两个整数w，n， $80 \leq w \leq 200$ ,  $1 \leq n \leq 300$ ，w为一条独木舟的最大承载量,n为人数；

接下来的一组数据为每个人的重量（不能大于船的承载量）；

输出要点：每组所需要租的最小独木舟数

## 零基础先修

- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：排序后用双指针。最重的学生无论如何都要占一条船：若他能与当前最轻 学生同船，就同时安排两人；否则最轻者不可能与他同船（其他人更重），只能让最重者单独乘。每轮船数加一并移走最重者，直到所有人安排完。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int capacity, n;
10 |         cin >> capacity >> n;
11 |         vector<int> weight(n);
12 |         for (int& x : weight)
13 |             cin >> x;
14 |         sort(weight.begin(), weight.end());
15 |         int left = 0, right = n - 1, boats = 0;
16 |         while (left <= right) {
17 |             if (left < right && weight[left] + weight[right] <= capacity)
18 |                 ++left;
19 |             --right;
20 |             ++boats;
21 |         }
22 |         cout << boats << '\n';
23 |     }
24 |     return 0;
25 | }
```

## 公开样例

样例 1 输入

```
3
85 6
5 84 85 80 84 83
90 3
90 45 60
100 5
50 50 90 40 6
```

样例 1 输出

```
5
3
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)



## 6.11 | 总 378/526 Choice学姐买糖果III ( PID 1316 )

出处：2017级河南工大新生周赛（二）（C，CID 1029）| 训练定位：基础提高 | 贪心与博弈

标签：向上取整、贪心、装载 | 限制：1.000 S / 128 MB | 历史统计：65/216 ( 30.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出至少要拿多少次，答案占一行。

输入要点：第一行输入两个整数 $n, w$  ( $1 \leq n, w \leq 1e4$ )，用空格隔开

接下来一行 $n$ 个空格分隔的整数 $m_i$ 表示第 $i$ 种糖果的个数。( $1 \leq m_i \leq 1e3$ )。

输出要点：输出至少要拿多少次，答案占一行。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：一个纸袋容量 $w$ 且一次只能装一种糖果，所以第 $i$ 种至少需要 $\text{ceil}(m_i/w)$ 个“袋次”。这些袋次可以任意两两配成一次搬运，因为 Choice 有两个袋子；最后若总袋次数为奇数，再单独搬一次。因此先求 $\text{bags} = \sum \text{ceil}(m_i/w)$ ，答案为 $\text{ceil}(\text{bags}/2)$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回`int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long capacity;
8 |     cin >> n >> capacity;
9 |     long long bags = 0;
10 |    for (int i = 0; i < n; ++i) {
11 |        long long count;
12 |        cin >> count;
13 |        bags += (count + capacity - 1) / capacity;
14 |    }
15 |    cout << (bags + 1) / 2 << '\n';
16 |    return 0;
17 | }
```

### 公开样例

样例 1 输入

```
6 5
3 2 6 5 4 4
```

样例 1 输出

4

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.12 | 总 379/526 Choice学姐买糖果 V ( PID 1318 )

出处：2017级河南工大新生周赛（二）（E，CID 1029）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、货币系统、整除取余 | 限制：1.000 S / 128 MB | 历史统计：73/132 ( 55.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出至少拿了多少张纸币，答案占一行。

输入要点：第一行输入一个整数 $n$  ( $1 \leq n \leq 1e9$ )。

输出要点：输出至少拿了多少张纸币，答案占一行。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：要让纸币张数最少，应尽量使用面额最大的纸币。依次对 100、50、20、10、5、1 做整除，商加入张数、余数继续处理。  
若某方案使用多张小面额而能换成一张更大面额，张数会减少，所以该贪心最优。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long amount;
5 |     cin >> amount;
6 |     const int coin[] = {100, 50, 20, 10, 5, 1};
7 |     long long notes = 0;
```

```

8 |         for (int value : coin) {
9 |             notes += amount / value;
10 |             amount %= value;
11 |         }
12 |         cout << notes << '\n';
13 |         return 0;
14 |     }

```

## 公开样例

样例 1 输入

110

样例 1 输出

2

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 6.13 | 总 380/526 炮姐的硬币 ( PID 1326 )

出处：2017级河南工大新生周赛 ( 三 ) ( F , CID 1030 ) | 训练定位：基础提高 | 贪心与博弈

标签：博弈论、巴什博弈、取模 | 限制：1.000 S / 128 MB | 历史统计：52/102 ( 51.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：黑子今天又跑去姐姐大人的寝室玩了，今天黑子和姐姐大人玩取硬币，有一堆硬币共  $n$  枚，炮姐和黑子两个人轮流拿，炮姐先拿，每次最少拿 1 枚，最多拿  $k$  枚，拿到最后一枚硬币的人获胜，假设炮姐和黑子都非常聪明，拿硬币的过程中不会出现失误，给 2 个数  $n$  和  $k$ ，问最后谁能赢得比赛。

输入要点：第 1 行，一个数  $t$ ，一共有  $t$  组测试

数据范围：(  $1 \leq t \leq 10000$  )

第 2--> $t+1$  行，每行两个数  $n$ ， $k$  中间用空格分隔

数据范围: ( $1 \leq n, k \leq 10^9$ )

输出要点：共  $t$  行

如果炮姐获胜输出“Misaka Mikoto Win” ( 不带引号 )

如果黑子获胜输出“Shirai Kuroko Win” ( 不带引号 )

题面提示：样例 1：  $n = 3$ ， $k = 2$ 。无论炮姐如何拿，黑子都可以拿到最后一枚硬币

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

- 核心处理采用：取  $1..k$  枚的经典巴什博弈中， $k+1$  的倍数是必败态：无论先手取  $x$ ，后手取  $k+1-x$ ，两人一轮合计取走  $k+1$ ，后手会拿到最后一枚。若  $n$  不是  $k+1$  的倍数，先手先取余数，把局面变成该必败态，之后照互补策略。因此按  $n\%(k+1)$  判断。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long n, k;
10 |         cin >> n >> k;
11 |         if (n % (k + 1) == 0)
12 |             cout << "Shirai Kuroko Win\n";
13 |         else
14 |             cout << "Misaka Mikoto Win\n";
15 |     }
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
4
3 2
4 2
7 3
8 3
```

样例 1 输出

```
Shirai Kuroko Win
Misaka Mikoto Win
Misaka Mikoto Win
Shirai Kuroko Win
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 6.14 | 总 381/526 趣味游戏 ( 四 ) : 蛮王 ( PID 1332 )

出处 : 2017级河南工大新生周赛 ( 四 ) ( D , CID 1031 ) | 训练定位 : 基础提高 | 贪心与博弈

标签 : 排序、贪心、前缀和 | 限制 : 1.000 S / 128 MB | 历史统计 : 36/97 ( 37.1% )

代码口径 : 依据公开题面/解析独立改写 ; 已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要 : 听说你们喜欢玩《英雄联盟》, 但是你们有没有听说过伟大的蛮王泰达米尔? 蛮王从他的父亲手中继承了大剑, 而这把剑也是他父亲从泰达米尔祖父那里继承来的。多年来他一直铭记着父亲的教诲“熟练掌握它, 就能拥有它。”他和寒冰射手艾希缔结了婚姻, 并一起为守护联盟的子民们而战。有一天敌人入侵联盟, 蛮王为了守护联盟和妻子, 英勇地与敌人战斗。经过艰苦地战斗蛮王终于打退了敌人, 战后妻……

输入要点 : 第一行  $T$  表示样例的组数

每组样例第一行  $n$  和  $m$  (  $0 < m < n \leq 10000$  ) 分别表示蛮王在战场上的出刀次数以及暴击次数

每组样例第二行  $n$  个数字, 第  $i$  个数字  $a[i]$  ( $0 < a[i] \leq 1000$ ) 表示蛮王第  $i$  刀打出的基础伤害

输出要点 : 每组样例输出用一个空格隔开的两个整数, 分别表示表示蛮王可能造成伤害的最小值和最大值。

### 零基础先修

- sort 默认升序 ; 排序后应利用相邻性、前缀性质或单调性, 而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优, 常用交换论证或下界与构造相遇。
- $prefix[i]$  表示前  $i$  项之和, 区间  $[l, r]$  的和为  $prefix[r] - prefix[l-1]$ 。

### 思路推导

1. 先按输入说明读取数据, 并用最小样例手算一次, 确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序 ; 若题目规定并列规则, 再加入次关键字。
3. 核心处理采用 : 每一刀基础伤害都必然计入 ; 恰有  $m$  刀暴击, 相当于再额外加上这  $m$  刀各自的一份基础伤害。因此最小总伤害选择最小的  $m$  个数额外加入, 最大总伤害选择最大的  $m$  个数额外加入。排序后用前缀/后缀求和即可。
4. 最后按题目要求输出 ; 提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同, 可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解, 因此算法达到全局最优。

### 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 比较器必须形成严格弱序 ; 相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论 ; 尝试寻找反例或交换论证。
- 按题目上界估算中间量, 相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, crit;
10 |         cin >> n >> crit;
```

```

11 |         vector<long long> damage(n);
12 |         long long baseTotal = 0;
13 |         for (long long& x : damage) {
14 |             cin >> x;
15 |             baseTotal += x;
16 |         }
17 |         sort(damage.begin(), damage.end());
18 |         long long mn = baseTotal;
19 |         long long mx = baseTotal;
20 |         for (int i = 0; i < crit; ++i) {
21 |             mn += damage[i];
22 |             mx += damage[n - 1 - i];
23 |         }
24 |         cout << mn << ' ' << mx << '\n';
25 |     }
26 |     return 0;
27 | }

```

## 公开样例

样例 1 输入

```

2
5 2
2 3 4 5 6
10 2
10 9 8 7 6 5 4 3 2 1

```

样例 1 输出

```

25 31
58 74

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.15 | 总 382/526 趣味游戏 ( 五 ) : 吃鸡 ( PID 1333 )

出处：2017级河南工大新生周赛 ( 四 ) ( E , CID 1031 ) | 训练定位：基础提高 | 贪心与博弈

标签：贪心、排序、字典序目标 | 限制：1.000 S / 128 MB | 历史统计：6/12 ( 50.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，每组样例输出用一个空格隔开的两个整数,第一个整数表示玩家最多可以吃到多少只"鸡"，第二个整数表示玩家吃最多的"鸡"之后最多还能吃下多少单位的食物。

**输入要点：**第一行一个 T 表示测试样例数目；

每组测试样例:

第一行:两个数字 n 和 m ；n 代表玩家最多可以吃下的食物的总单位，m 代表"鸡"的种类数目 ( 0<n<=1000000,0<m<=1000 )

第二行:m 个数字，第 i 个数字 a[i](0<a[i]<=20) 代表第i个类型的"鸡"可以转化成a[i]个单位的食物

第三行:m 个数字，第i个数字b[i](0<b[i]<=50) 表示第i个类型的"鸡"的数目为b[i];

**输出要点：**每组样例输出用一个空格隔开的两个整数,第一个整数表示玩家最多可以吃到多少只"鸡"，第二个整数表示玩家吃最多的"鸡"之后最多还能吃下多少单位的食物。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：目标首先是让“鸡的只数”最大，其次才让剩余胃口最大。因此总应优先吃转化食物单位更少的类型；若跳过一只小鸡去吃更大的，不会增加数量，只会消耗更多容量。按  $a[i]$  升序，对每类可整只吃下  $\min(b[i], \text{剩余容量}/a[i])$  只，更新数量和剩余容量。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(m \log m)$  时间、 $O(m)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         long long capacity;
10 |         int types;
11 |         cin >> capacity >> types;
12 |         vector<int> food(types), count(types);
13 |         for (int& x : food)
14 |             cin >> x;
15 |         for (int& x : count)
16 |             cin >> x;
17 |         vector<pair<int, int>> chicken(types);
18 |         for (int i = 0; i < types; ++i)
19 |             chicken[i] = {food[i], count[i]};
20 |         sort(chicken.begin(), chicken.end());
21 |         long long eaten = 0;
22 |         for (auto [units, available] : chicken) {
23 |             long long take = min<long long>(available, capacity / units);
24 |             eaten += take;
25 |             capacity -= take * units;
26 |         }
27 |         cout << eaten << ' ' << capacity << '\n';
28 |     }
29 |     return 0;
30 | }
```

## 公开样例

样例 1 输入

```
2
600 4
1 2 3 4
50 50 50 50
27 4
4 3 2 1
10 9 8 7
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.16 | 总 383/526 互质 ( PID 1336 )

出处：2017级河南工大新生周赛（五）（B，CID 1033）| 训练定位：基础提高 | 贪心与博弈

标签：最大公约数、贪心、构造 | 限制：1.000 S / 128 MB | 历史统计：20/81 ( 24.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：求插入的数的最少的数量。

输入要点：第一行：整数  $T$ ，测试实例个数。

第一行：一个整数  $n$ ，表示序列元素个数。 ( $1 \leq n \leq 1000$ )

第二行： $n$  个整数  $a_i$ ，用空格隔开，表示序列中的  $n$  个元素。 ( $1 \leq a_i \leq 10^9$ )

输出要点：一个整数，表示最小插入数量。

## 零基础先修

- `std::gcd(a,b)` 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：逐对检查相邻元素。若  $\gcd(a_i, a_{i+1})=1$ ，它们已经互质；否则必须在这两个位置之间至少插入一个数，因为在别处插入不会改变这条相邻关系。插入 1 一定可行，因为 1 与任何正整数互质。因此每个不互质的原相邻对恰贡献一次，统计它们即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log V)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。



- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         vector<long long> a(n);
12 |         for (long long& x : a)
13 |             cin >> x;
14 |         int ans = 0;
15 |         for (int i = 1; i < n; ++i)
16 |             ans += gcd(a[i - 1], a[i]) != 1;
17 |         cout << ans << '\n';
18 |     }
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
2
3
2 7 28
4
1 2 3 4
```

样例 1 输出

```
1
0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.17 | 总 384/526 看医生 ( PID 1343 )

出处：2017级河南工大新生周赛（六）（C，CID 1035）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、等差数列、向上取整 | 限制：2.000 S / 256 MB | 历史统计：9/38 ( 23.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：请你帮小A计算他拜访完所有医生最少需要到第几天（是从第一天开始计算，而不是从拜访第一个医生开始）。

输入要点：第一行：一个整数T，测试实例个数

对于每组测试实例：

第一行：一个整数n ( $1 \leq n \leq 1000$ ) —— 表示小A要拜访的医生的数量。

接下来的n行：每行包括两个整数  $s_i$  和  $d_i$  ( $1 \leq s_i, d_i \leq 1000$ )，含义见上面题目描述。

输出要点：每组测试实例输出一行：包括一个整数——小A拜访完所有医生最少需要到第几天。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：医生必须按编号拜访且一天至多一位，所以第  $i$  位医生要选择严格晚于 上一次拜访日的最早工作日。其工作日为  $s_i + k \cdot d_i$ ：若  $s_i$  已大于当前日，直接选  $s_i$ ；否则把它增加最小个  $d_i$ ，使其严格大于当前日。逐个贪心即可，因为把某次拜访推迟只会压缩后续选择，不会带来好处。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        long long day = 0;
12 |        for (int i = 0; i < n; ++i) {
13 |            long long start, period;
14 |            cin >> start >> period;
15 |            if (start <= day) {
16 |                long long steps = (day - start) / period + 1;
17 |                start += steps * period;
18 |            }
19 |            day = start;
20 |        }
21 |        cout << day << '\n';
22 |    }
23 |    return 0;
24 | }
```

## 公开样例

样例 1 输入

```
2
3
2 2
1 2
2 2
2
10 1
6 5
```

样例 1 输出

```
4
11
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.18 | 总 385/526 括号配对 ( PID 1346 )

出处：2017级河南工大新生周赛 ( 六 ) ( F , CID 1035 ) | 训练定位：基础提高 | 贪心与博弈

标签：括号匹配、贪心、交换 | 限制：1.000 S / 256 MB | 历史统计：5/14 ( 35.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给出一串长度为 $n$ 的括号序列 ( 只包含小括号 )，计算出最少的交换 ( 两两交换 ) 次数，使整个括号序列匹配。

输入要点：第一行：一个整数 $T$ ，表示测试实例个数

对于每组测试实例：

第一行：整数 $n$  (  $0 \leq n \leq 5 \times 10^6$  )，表示括号序列长度

第二行：一个字符串，表示括号

输出要点：每组测试实例输出一行：包含一个整数，表示最少的交换次数

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：先像括号匹配一样扫描：遇到左括号把余额加一；遇到右括号时，若有 未匹配左括号就配掉，否则记一个 unmatchedClose。题目允许交换任意 两个位置且保证可以配平。一次把前面的坏右括号与后面的左括号交换，最多可同时消除两个这种缺口，因此最少交换数为  $\text{ceil}(\text{unmatchedClose}/2)$ ，即  $(\text{unmatchedClose}+1)/2$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

每组  $O(n)$  时间、 $O(1)$  额外空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         string s;
11 |         cin >> n >> s;
12 |         int openCnt = 0;
13 |         int closeCnt = 0;
14 |         for (char c : s) {
15 |             if (c == '(') {
16 |                 ++openCnt;
17 |             } else if (openCnt > 0) {
18 |                 --openCnt;
19 |             } else {
20 |                 ++closeCnt;
21 |             }
22 |         }
23 |         cout << (closeCnt + 1) / 2 << '\n';
24 |     }
25 |     return 0;
26 | }

```

## 公开样例

样例 1 输入

```

2
6
(())(
6
)))(

```

样例 1 输出

```

1
2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.19 | 总 386/526 Choice发糖果 ( PID 1350 )

出处：2017级河南工大新生周赛（七）（D，CID 1036）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、单位价值、排序 | 限制：1.000 S / 128 MB | 历史统计：3/11 ( 27.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出Choice能带走的糖果的最大总价值，保留1位小数。输出占一行。

**输入要点：**输入多组数据。

每组第一组数据输入 $n, w$ 。（ $1 \leq n \leq 1000, 1 \leq w \leq 10000$ ）

接下来 $n$ 行每行输入 $p_i, m_i$ 代表第 $i$ 种糖果的价格和数量。（ $1 \leq p_i, m_i \leq 1e4$ ）

**输出要点：**输出Choice能带走的糖果的最大总价值，保留1位小数。输出占一行。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：同一种糖果每颗价值相同，为  $pi/mi$ 。问题就是按单位价值做“可拆分 背包”：把所有种类按单颗价值从高到低排序，依次拿  $\min(\text{剩余容量}, mi)$  颗，容量用完即停止。交换论证：若方案中拿了较便宜的一颗却漏掉较贵的一颗，交换后价值只会增加，所以该贪心最优。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | struct Candy {
4 |     double unitPrice;
5 |     int count;
6 | };
7 | int main() {
8 |     ios::sync_with_stdio(false);
9 |     cin.tie(nullptr);
10 |     int n, capacity;
11 |     while (cin >> n >> capacity) {
12 |         vector<Candy> candy(n);
13 |         for (Candy& item : candy) {
14 |             int totalPrice, count;
15 |             cin >> totalPrice >> count;
16 |             item = {(double)totalPrice / count, count};
17 |         }
18 |         sort(candy.begin(), candy.end(), [](const Candy& a, const Candy& b) {
19 |             return a.unitPrice > b.unitPrice;
20 |         });
21 |         double ans = 0.0;
22 |         for (const Candy& item : candy) {
23 |             int take = min(capacity, item.count);
24 |             ans += take * item.unitPrice;
25 |             capacity -= take;
26 |             if (capacity == 0)
27 |                 break;
28 |         }
29 |         cout << fixed << setprecision(1) << ans << '\n';
30 |     }
31 |     return 0;
32 | }
```

## 公开样例

样例 1 输入

```
4 20
20 4
40 4
160 8
412 103
```

样例 1 输出

```
236.0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.20 | 总 387/526 大学生活 (一) (PID 1355)

出处：2017级河南工大新生周赛（八）（B，CID 1037）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、区间调度、排序 | 限制：1.000 S / 128 MB | 历史统计：9/28 (32.1%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，每组测试样例，一个输出（理论上最多可以做多少件事）。

输入要点：多实例测试，先是一个正整数  $n$  ( $n \leq 20$ )，每一个  $n$  后面有  $n$  行，每行两个正整数 ( $0 < a < b < 100$ )。

输出要点：每组测试样例，一个输出（理论上最多可以做多少件事）。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
- 核心处理采用：这是经典的不相交区间选择。先按结束时间从早到晚排序，选择第一件事；之后每次选择开始时间不早于上一件结束时间的、结束最早的事情。结束越早，给后续留下的时间越多，交换论证可说明不会比任何最优方案少选。输入为文件尾多组实例。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n) {
8 |         vector<pair<int, int>> event(n);
9 |         for (auto& [start, finish] : event)
```

```

10 |         cin >> start >> finish;
11 |         sort(event.begin(), event.end(), [](const auto& x, const auto& y) {
12 |             if (x.second != y.second)
13 |                 return x.second < y.second;
14 |             return x.first < y.first;
15 |         });
16 |         int count = 0;
17 |         int last = -1;
18 |         for (auto [start, finish] : event) {
19 |             if (start >= last) {
20 |                 ++count;
21 |                 last = finish;
22 |             }
23 |         }
24 |         cout << count << '\n';
25 |     }
26 |     return 0;
27 | }

```

## 公开样例

### 样例 1 输入

```

5
1 4
3 10
5 10
6 11
7 12
5
1 4
5 10
3 10
6 11
7 12
3
1 2
3 10
2 3

```

### 样例 1 输出

```

2
2
3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.21 | 总 388/526 Chomp ( PID 1417 )

出处：2018级河南工大新生周赛总决赛（重现）（A，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（A，CID 1048） | 训练定位：基础提高 | 贪心与博弈

标签：博弈论、策略窃取、观察 | 限制：1.000 S / 32 MB | 历史统计：53/118 ( 44.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，如果最后先手会吃掉那块有毒的巧克力输出First 否则输出Second

**输入要点：**先输入一个数字T，表示T组测试数据（ $T \leq 100$ ）

接下来每行输入两个正整数n, m（ $n, m \leq 100$ ），表示巧克力的尺寸

（虽然这个游戏好像只能进行1局）

输出要点：如果最后先手会吃掉那块有毒的巧克力输出First

否则输出Second

题面提示：当巧克力大小为 $2 \times 2$ 时，先手第一次一定会选择(2,2)，这样后手无论选择(1,2)还是(2,1)，先手只要接下来选择另一块，再轮到后手时就只剩下(1,1)这块巧克力了

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：题目输出的是最终吃到毒巧克力的人，而不是“获胜者”的名字。对  $1 \times 1$  棋盘，先手只能立即吃毒块，所以输出 First。对任何至少有两个格子的矩形 Chomp，经典的策略窃取论证说明先手一定 能把“避免吃毒块”的获胜策略掌握在自己手中：假设先手没有获胜策略，他先吃右下角这一块后，本可属于后手的策略就能被先手照搬；额外少掉的右下角不会让某个合法操作变得非法，产生矛盾。因此最终吃毒块的是 后手，输出 Second。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, m;
10 |        cin >> n >> m;
11 |        cout << (n == 1 && m == 1 ? "First" : "Second") << '\n';
12 |    }
13 |    return 0;
14 | }
```

## 公开样例

样例 1 输入

```
3
2 2
3 3
8 8
```

样例 1 输出

```
Second
Second
Second
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。



- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.22 | 总 389/526 最大的最小区间 ( PID 1442 )

出处：2018级河南工大新生周赛 ( 七 ) @牛寅旭专场 ( F , CID 1046 ) | 训练定位：基础提高 | 贪心与博弈

标签：二分答案、贪心、排序 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：坐标轴上有  $n$  个点，选取  $k$  个点，使这  $k$  个点相邻的区间长度最小值最大

输入要点：第一行有两个整数  $n, k$ 。 ( $1 < k \leq n < 10^5$ )

第二行有  $n$  个正整数 ( $0 < \text{正整数} < \text{int}$ )。

输入坐标无序，且可能重复。

输出要点：输出一个整数，最大的最小区间长度。

### 零基础先修

- 二分答案要求判定函数随答案单调；先写清可行区间与 true/false 的方向。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：排序坐标。给定最小间距  $d$ ，从最左点开始，每次贪心选第一个距离上一已选点至少  $d$  的点，这会为后面留下最大空间；若能选满  $k$  个则  $d$  可行。可行性对  $d$  单调，因此二分最大的可行  $d$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(n \log \text{坐标差} + n \log n)$  时间、 $O(n)$  空间

### 易错点

- 检查左右边界、循环条件和最终返回的是第一个可行还是最后一个不可行。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, k;
7 |     cin >> n >> k;
8 |     vector<long long> point(n);
9 |     for (long long& x : point)
10 |         cin >> x;
11 |     sort(point.begin(), point.end());
12 |     auto possible = [&](long long distance) {
13 |         int selected = 1;
14 |         long long last = point[0];
15 |         for (int i = 1; i < n && selected < k; ++i) {
16 |             if (point[i] - last >= distance) {
17 |                 last = point[i];
18 |                 ++selected;
19 |             }
20 |         }
21 |         return selected >= k;
22 |     };
23 |     long long low = 0, high = point.back() - point.front();
24 |     while (low < high) {
25 |         long long middle = low + (high - low + 1) / 2;
26 |         if (possible(middle))
27 |             low = middle;
28 |         else
29 |             high = middle - 1;
30 |     }
31 |     cout << low << '\n';
32 |     return 0;
33 | }

```

## 公开样例

样例 1 输入

```

5 3
1 2 5 7 8

```

样例 1 输出

```

3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.23 | 总 390/526 相得甚欢 ( PID 1485 )

出处：2019级河南工大新生周赛 ( 二 ) @邹岩专场 ( F , CID 1054 ) | 训练定位：基础提高 | 贪心与博弈

标签：博弈论、奇偶性、结论题 | 限制：1.000 S / 128 MB | 历史统计：97/166 ( 58.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，如果【欢】获胜输出“wula”，如果WX获胜输出“waha”。（输出只有字母不带引号）

输入要点：唯一的一行包含整数 $n$  ( $1 \leq n \leq 10^9$ )，游戏开始时的数字。

输出要点：如果【欢】获胜输出“wula”，如果WX获胜输出“waha”。（输出只有字母不带引号）

### 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：n 为偶数时，欢第一步可以直接取走 n（合法偶数）并获胜；n 为奇数时，无论欢减去哪个偶数，都会留下奇数，WX 可在下一步取走全部剩余数。因此答案只由 n 的奇偶决定。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     cin >> n;
6 |     cout << (n % 2 == 0 ? "wula" : "waha") << '\n';
7 |     return 0;
8 | }
```

### 公开样例

样例 1 输入

1

样例 1 输出

waha

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)

## 6.24 | 总 391/526 虹猫蓝兔AC转 ( PID 1489 )

出处：2019级河南工大新生周赛（三）@邓祎培专场（D，CID 1055）| 训练定位：基础提高 | 贪心与博弈

标签：博弈论、结论题、条件分支 | 限制：1.000 S / 128 MB | 历史统计：55/169 ( 32.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，共一行，输出 "Hongmao" 或 "Lantu"（不输出引号）。

输入要点：共一行，输入一个数

输出要点：共一行，输出 "Hongmao" 或 "Lantu" ( 不输出引号 )。

题面提示：说明：虹猫只能取走1，蓝兔赢

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：n=1 时先手只能取走最后一个数，按反常规则立即输。n>1 时先手 总能通过首步选择把局面留成必败形，公开题解也给出该结论，因此只需 对 n=1 特判。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     cin >> n;
6 |     cout << (n == 1 ? "Lantu" : "Hongmao") << '\n';
7 |     return 0;
8 | }
```

## 公开样例

样例 1 输入

1

样例 1 输出

Lantu

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.25 | 总 392/526 是ZP的步伐 ( PID 1502 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( E , CID 1056 ) | 训练定位：基础提高 | 贪心与博弈

标签：数学、贪心、向上取整 | 限制：1.000 S / 128 MB | 历史统计：234/319 ( 73.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：那么，ZP最少要多少步才能找到WF

输入要点：一个整数 $n$ ， $1 \leq n \leq 1\,000\,000$

输出要点：找到WF的最小步数

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：每步最多走 5 个单位，至少需要  $\text{ceil}(n/5)$  步；先走若干个 5，最后一步走剩余 1..5，故下界可达到。整数向上取整写作  $(n+4)/5$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n;
5 |     cin >> n;
6 |     cout << (n + 4) / 5 << '\n';
7 |     return 0;
8 | }
```

### 公开样例

样例 1 输入

5

样例 1 输出

1

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.26 | 总 393/526 zp的新冒险 ( PID 1506 )

出处：2019级河南工大新生周赛（五）@潘世泽专场（A，CID 1057）| 训练定位：基础提高 | 贪心与博弈

标签：排序、贪心、区间调度 | 限制：1.000 S / 128 MB | 历史统计：31/83 ( 37.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行包含一个整数表示活动个数。

输入要点：第一行一个正整数 $n$  ( $n \leq 50$ )代表活动的个数。第二行到第 $(n + 1)$ 行包含 $n$ 个开始时间和结束时间。开始时间严格小于结束时间，并且时间都是非负整数，小于10000

输出要点：一行包含一个整数表示活动个数。

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：经典活动选择：按结束时间从小到大排序，每次选择开始时间不早于 上一个已选活动结束时间的活动。最早结束给后续留下的可用时间最多，用交换论证可知不会劣于任何最优方案。区间是  $[s, f)$ ，端点相等可无缝衔接。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     vector<pair<int, int>> activity(n);
7 |     for (auto& [finish, start] : activity)
8 |         cin >> start >> finish;
9 |     sort(activity.begin(), activity.end(), [](auto a, auto b) {
10 |         if (a.first != b.first)
11 |             return a.first < b.first;
12 |         return a.second > b.second;
13 |     });
```

```

13 |     });
14 |     int ans = 0, last = -1;
15 |     for (auto [finish, start] : activity) {
16 |         if (start >= last) {
17 |             ++ans;
18 |             last = finish;
19 |         }
20 |     }
21 |     cout << ans << '\n';
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

3
1 2
3 4
2 9

```

样例 1 输出

```

2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.27 | 总 394/526 卷卷的石子 ( PID 1575 )

出处：2022级HAUT新生周赛（零）熟悉周赛规则专场（C，CID 1078）；2021级HAUT新生周赛（一）@张君毅专场（C，CID 1069） | 训练定位：基础提高 | 贪心与博弈

标签：博弈论 | 限制：1.000 S / 128 MB | 历史统计：1031/3196 ( 32.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果卷卷获胜，请输出“%%%聚聚”（不含引号）。如果聚聚获胜，请输出“聚聚NB”（不含引号）。

输入要点：一行一个正整数 $n$ ，表示有 $n$ 个石子。（ $1 \leq n \leq 1000$ ）

输出要点：如果卷卷获胜，请输出“%%%聚聚”（不含引号）。

如果聚聚获胜，请输出“聚聚NB”（不含引号）。

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：卷卷的石子每人每次必须拿走一个，则奇数个石子先手胜，偶数个石子后手胜。注意：输出 '%' 时，需要使用 '%'。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

常数级或一次线性读入；以代码中的循环层数为准

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     // 因为每次必须拿一个，所以判断奇数偶数即可
6 |     if (n % 2 == 0) {
7 |         printf("聚聚NB");
8 |     } else {
9 |         // 输出"%", 需要用"%"
10 |        printf("%%%%%%%%聚聚");
11 |    }
12 |    return 0;
13 | }
```

### 公开样例

样例 1 输入

2

样例 1 输出

聚聚NB

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.28 | 总 395/526 约会？干饭！（PID 1758）

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（E，CID 1085）；2022级HAUT新生周赛（一）@卞子骏专场（E，CID 1079） | 训练定位：基础提高 | 贪心与博弈

标签：博弈论、奇偶性、数组 | 限制：1.000 S / 128 MB | 历史统计：132/436 (30.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，卷卷赢输出"火锅！" 聚聚赢输出"牛排!"（输出不带引号）

输入要点：第一行一个整数  $n$  ( $1 \leq n \leq 100$ )，表示石子堆数

接下来一行  $n$  个数，第  $i$  个数为  $a_i$  ( $1 \leq a_i \leq 1e9$ )，意义如上所述

输出要点：卷卷赢输出"火锅！"

聚聚赢输出"牛排!"



( 输出不带引号 )

题面提示：学会 CTRL + C 和 CTRL + V

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：每次操作一定恰好拿走一颗石子；游戏最终会拿完全部石子，所以总 步数就是石子总数。总数为奇数时先手拿最后一颗并获胜，否则后手获胜。只累计奇偶性即可，避免大数求和。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, parity = 0;
7 |     cin >> n;
8 |     for (int i = 0; i < n; ++i) {
9 |         long long stones;
10 |        cin >> stones;
11 |        parity ^= int(stones & 1LL);
12 |    }
13 |    cout << (parity ? "火锅!" : "牛排!") << '\n';
14 |    return 0;
15 | }
```

## 公开样例

样例 1 输入

```
1
1
```

样例 1 输出

```
火锅！
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.29 | 总 396/526 将故事写成我们 ( PID 1776 )

出处：2022级HAUT新生周赛（三）@焦小航专场（G，CID 1081）| 训练定位：基础提高 | 贪心与博弈

标签：贪心、观察、字符串 | 限制：1.000 S / 128 MB | 历史统计：22/52 (42.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：只有最后两个烟花到达同一位置才视为相汇，即它们的轨迹拼成一个爱心。小坚书直实和一行琉璃正在欣赏烟花，所以他想请你判断烟花的轨迹是否有可能拼成一个爱心。

输入要点：第一行给出两个烟花此时的坐标  $(x_1, y_1)$   $(x_2, y_2)$ 。  $(-25 \leq x_1, x_2, y_1, y_2 \leq 25)$ ，以  $x_1, y_1, x_2, y_2$  顺序输入。

第二行给出指令S。  $(|S| \leq 100000)$ ， $|S|$  为字符串的长度。

字符串S仅包括U（向上走一个单位，即横坐标不变，纵坐标+1），D（向下走一个单位，横坐标不变，即纵坐标-1），L（向左走一个单位，即横坐标-1，纵坐标不变），R（向右走一个单位，即横坐标+1，纵坐标不变）。

输出要点：如果可能相汇碰撞（即轨迹可以拼成爱心）输出"Explosion"，否则输出"Safe"（输出不带" "）

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：两个烟花可以分别选择是否执行每条指令。对任意一条水平方向指令，它对两烟花横坐标差的改变量可为-1、0、1；因此H条水平指令能把横坐标差调整为  $[-H, H]$  内任意整数。纵坐标同理。两个方向相互独立，所以只需同时满足初始横差不超过H、纵差不超过V。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(|S|)$  时间、 $O(1)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int x1, y1, x2, y2;
7 |     string commands;
8 |     cin >> x1 >> y1 >> x2 >> y2 >> commands;
9 |     int horizontal = 0, vertical = 0;
10 |    for (char c : commands) {
```

```

11 |         if (c == 'L' || c == 'R')
12 |             ++horizontal;
13 |         else
14 |             ++vertical;
15 |     }
16 |     bool possible = abs(x1 - x2) <= horizontal && abs(y1 - y2) <= vertical;
17 |     cout << (possible ? "Explosion" : "Safe") << '\n';
18 |     return 0;
19 | }

```

## 公开样例

样例 1 输入

```

-9 20 9 21
DU

```

样例 1 输出

```

Safe

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 6.30 | 总 397/526 金币！（PID 1842）

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（F，CID 1088）| 训练定位：基础提高 | 贪心与博弈

标签：排序、贪心、前缀和 | 限制：2.000 S / 256 MB | 历史统计：6/24 (25.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**卷卷作为本场比赛的讲解，他想要提前了解本场比赛的最终的可能情况，你能帮帮他吗？一共有  $n$  支队伍参加比赛，每个队伍初始时有一些金币。比赛每一轮随机挑两个金币数不为 0 的队伍，然后金币多的队伍获胜，金币少的队伍把金币全部给金币多的（金币数量相同则随机），直到最后只有一个队伍有金币，这个队伍获胜。最终求出有几个队伍是可能获胜的、是哪些队伍。

**输入要点：**第一行包含一个整数  $t(1 \leq t \leq 10^4)$  测试用例的数量。接下来是  $t$  个测试用例。每一个测试用例的第一行由一个正整数  $n(1 \leq n \leq 2 \cdot 10^5)$  组成，代表参加锦标赛的选手人数。第二行包含  $n$  个正整数  $a(1 \leq a \leq 10^9)$ ，代表玩家拥有的代币数量。

**输出要点：**对于每个测试用例，打印有可能赢得冠军的玩家的数目。在第二行下一行按递增顺序打印这些玩家的编号。玩家按照他们出现在输入中的顺序从 1 开始编号，以空格隔开。

## 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- $\text{prefix}[i]$  表示前  $i$  项之和，区间  $[l, r]$  的和为  $\text{prefix}[r] - \text{prefix}[l-1]$ 。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：把队伍按金币升序排序并维护前缀和。若前  $i$  个队的金币总和仍小于第  $i+1$  队，那么前  $i$  个中的任何队都不可能跨过这道断层；答案起点必须移到  $i+1$ 。最后一个断层之后的所有队都可能通过合适的对战顺序获胜，再把它们的原编号升序输出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        vector<pair<long long, int>> player(n);
12 |        for (int i = 0; i < n; ++i) {
13 |            cin >> player[i].first;
14 |            player[i].second = i + 1;
15 |        }
16 |        sort(player.begin(), player.end());
17 |        long long prefix = player[0].first;
18 |        int start = 0;
19 |        for (int i = 0; i + 1 < n; ++i) {
20 |            if (prefix < player[i + 1].first)
21 |                start = i + 1;
22 |            prefix += player[i + 1].first;
23 |        }
24 |        vector<int> ans;
25 |        for (int i = start; i < n; ++i)
26 |            ans.push_back(player[i].second);
27 |        sort(ans.begin(), ans.end());
28 |        cout << ans.size() << '\n';
29 |        for (int i = 0; i < (int)ans.size(); ++i) {
30 |            if (i)
31 |                cout << ' ';
32 |            cout << ans[i];
33 |        }
34 |        cout << '\n';
35 |    }
36 |    return 0;
37 | }
```

## 公开样例

样例 1 输入

```
2
4
1 2 4 3
5
1 1 1 1 1
```

样例 1 输出

```
3
2 3 4
5
1 2 3 4 5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

• 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

• [HAUTOJ 原题](#)

6.31 | 总 398/526 这是一道签到题 ( PID 1871 )

出处：河南工大2024新生周赛 ( 3 ) ——命题人：张宏泽 ( A , CID 1093 ) | 训练定位：基础提高 | 贪心与博弈

标签：字符串、博弈论 | 限制：1.000 S / 128 MB | 历史统计：104/467 ( 22.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，对于每个测试样例，输出一个仅有大写字母构成的字符串，表示胜利者名字的缩写，并输出一个换行。

输入要点：第一行包含一个整数  $t$  (  $1 \leq t \leq 100$  ) - 测试用例数。

每个测试用例由两行描述：

( 1 ) 第一行包含一个整数  $n$  (  $1 \leq n \leq 2000$  ) - 扑克牌的叠数。

( 2 ) 第二行包含  $n$  个整数，其中第  $i$  个数字表示第  $i$  叠扑克牌的数量，保证在  $\text{int}$  的范围内。

输出要点：对于每个测试样例，输出一个仅有大写字母构成的字符串，表示胜利者名字的缩写，并输出一个换行。

题面提示：相信我，这真是一道签到题

零基础先修

- `string` 可按下标访问字符；注意长度、空串、大小写、标点和 `getline` 与 `cin` 混用。
- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：这是一道签到题 题目的背景是基于简单的博弈论Nim游戏，但细心者可以发现小明和小美的名字的首字母一致，所以只需要进行读入，直接输出"XM"即可通过本题
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, x;
3 | void solve() {
4 |     scanf("%d", &n);
5 |     for (int i = 1; i <= n; i++)
6 |         scanf("%d", &x);
```

```
7 |     puts("XM");
8 | }
9 | int main() {
10 |     int T = 1;
11 |     scanf("%d", &T);
12 |     while (T--)
13 |         solve();
14 |     return 0;
15 | }
```

公开样例

样例 1 输入

```
1
2
2 3
```

样例 1 输出

```
XM
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.32 | 总 399/526 Senren Banka ( PID 1882 )

出处：河南工大2024新生周赛 ( 4 ) ——命题人：马贺 ( D , CID 1094 ) | 训练定位：基础提高 | 贪心与博弈

标签：条件分支、数组与序列、贪心 | 限制：1.000 S / 128 MB | 历史统计：46/160 ( 28.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：小丛雨给了你一个数组，你可以对数组中的一些元素施加柚子社之力，但不能对两个相邻的元素施加。你的得分是所有施加柚子社之力的元素其中的最大值加上柚子社之力的数量。请找出能得到的最大分数。

输入要点：第一行一个整数 $n(1 \leq n \leq 100)$ ，数组的长度。

第二行包含 $n$ 个整数 $a_1, a_2, a_3, \dots, a_n, (1 \leq a_i \leq 1000)$ ，给定的数组。

输出要点：输出一个整数：根据声明将某些元素施加柚子社之力后可能得到的最大分数。

题面提示：[图片：https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241116/20241116180548\_85871.jpg]

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：Senren Banka 易推得：要么全部取偶数，要么全部取奇数。故答案为  $\max(\text{oddmax}+\text{oddsum}, \text{evenmax}+\text{evensum})$
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int max(int a, int b) { // 返回两个数中的较大者
3 |     return a > b ? a : b;
4 | }
5 | int main() {
6 |     int arr[105];
7 |     int n;
8 |     scanf("%d", &n);
9 |     int oddmax = 0, evenmax = 0;
10 |    int oddsum = 0, evensum = 0;
11 |    for (int i = 1; i <= n; ++i) {
12 |        scanf("%d", &arr[i]);
13 |        if (i % 2 == 1) {
14 |            oddmax = max(oddmax, arr[i]);
15 |            oddsum++;
16 |        } else {
17 |            evenmax = max(evenmax, arr[i]);
18 |            evensum++;
19 |        }
20 |    }
21 |    printf("%d", max(oddmax + oddsum, evenmax + evensum));
22 |    return 0;
23 | }
```

## 公开样例

样例 1 输入

```
10
3 3 3 3 4 1 2 3 4 5
```

样例 1 输出

```
10
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 6.33 | 总 400/526 命运之桥 ( PID 1906 )

出处：河南工大2024新生周赛（7）——命题人：刘义（G，CID 1098） | 训练定位：基础提高 | 贪心与博弈

标签：贪心、等价变换、最值 | 限制：1.000 S / 128 MB | 历史统计：8/15 ( 53.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你急需计算出，你们全部撤离这座独木桥需要的最大时间，以及最小时间。

输入要点：第一行共一个整数  $L$ ，表示桥的长度。桥上的坐标为  $L_1, L_2, \dots, L_n$ 。

第二行共一个整数  $N$ ，表示初始时留在桥上的旅人数目。

第三行共有  $N$  个整数，分别表示每个旅人的初始坐标。

输出要点：共一行，输出 2 个整数，分别表示旅人撤离独木桥的最大时间和最小时间。2个整数由一个空格符分开。

题面提示：所有初始坐标互不相同； $1 \leq L \leq 5 \times 10^3, 0 \leq N \leq 5 \times 10^3$ ，且  $N \leq L$ 。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：两人相遇后掉头可等价于“彼此穿过并交换身份”。对位置  $x$ ，最快离桥需  $\min(x, L+1-x)$ ，最慢需  $\max(x, L+1-x)$ 。全体离开时间取个人时间的最大值；按题面顺序先输出最大时间，再输出最小时间。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int len, n;
7 |     cin >> len >> n;
8 |     int minimumTime = 0, maximumTime = 0;
9 |     for (int i = 0; i < n; ++i) {
10 |         int pos;
11 |         cin >> pos;
12 |         minimumTime = max(minimumTime, min(pos, len + 1 - pos));
13 |         maximumTime = max(maximumTime, max(pos, len + 1 - pos));
14 |     }
15 |     cout << maximumTime << ' ' << minimumTime << '\n';
16 |     return 0;
17 | }
```

### 公开样例

样例 1 输入



```
4
2
13
```

样例 1 输出

```
42
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.34 | 总 401/526 旅人分宝 ( PID 1907 )

出处：河南工大2024新生周赛（7）——命题人：刘义（B，CID 1098） | 训练定位：基础提高 | 贪心与博弈

标签：条件分支、贪心 | 限制：1.000 S / 128 MB | 历史统计：38/81 ( 46.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：有一天，旅人们得到了一箱宝藏，箱子中有x枚金币。一共有n个旅人，编号从1到n。编号最小的旅人就是旅人老大。根据旅人之间的规则，在分配财宝的时候，将会由旅人老大提出分配方案。在方案提出后，所有旅人对方案进行投票，包括旅人老大在内。每一个旅人都非常的贪婪，渴望得到尽可能多的金币，同时每一个旅人又都非常的狡猾，他们都会选择对自己最有利的情况进行投票。如果一个方案.....

输入要点：每个测试由几组输入数据组成。第一行包含一个整数t(1≤t≤100)—输入数据集的数量。

每组输入数据包含两个整数x,n ( 1≤n≤x≤1000 )

输出要点：请输出一个正整数表示你可以获得的最多金币的数量

题面提示：注意数据的输入

样例输入2：

```
1

100 8
```

样例输出2：

```
97
```

零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：旅人分宝：这是一个经典的海盗分金问题，每个旅人只会选择一个对自己最有利的情况，并且要求50%以上的同意，所以当老大给一半的人每一个人一个金币，他们就会同意了。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

复杂度

通常为  $O(n \log n)$ ，主要来自排序

易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int t;
4 | int n, x;
5 | int main() {
6 |     cin >> t;
7 |     while (t--) {
8 |         cin >> n >> x;
9 |         if (x % 2 == 0) {
10 |             cout << n - x / 2 + 1 << endl;
11 |         } else {
12 |             cout << n - x / 2 << endl;
13 |         }
14 |     }
15 |     return 0;
16 | }
```

公开样例

|            |
|------------|
| 样例 1 输入    |
| 1<br>100 5 |
| 样例 1 输出    |
| 98         |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

6.35 | 总 402/526 直播间红包雨抢券赛 ( PID 1959 )

出处：河南工大2025新生周赛 ( 3 ) ——命题人：路耀华 ( A , CID 1104 ) | 训练定位：基础提高 | 贪心与博弈

标签：条件分支、博弈论 | 限制：1.000 S / 128 MB | 历史统计：225/943 ( 23.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出一行，若能观众获得这张 50 元券，输出 “Got it!”；否则输出 “Miss.”。

输入要点：输入两个整数  $n, k$ ， $1 \leq n \leq 1000000$ ， $1 \leq k \leq n+1$ 。

输出要点：输出一行，若能观众获得这张 50 元券，输出 “Got it!”；否则输出 “Miss.”。

题面提示：注意观察规律

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：直播间红包雨抢券赛 根据规则，若某方领取到陷阱红包则直接判输，因此核心目标是避免领取第  $k$  个红包，即谁能迫使对方领取第  $k$  个红包，谁就获胜。这等价于争夺“领取到第  $k-1$  个红包”的控制权——若能确保自己领取到第  $k-1$  个红包，对方就必然要领取第  $k$  个红包（陷阱），从而获胜。双方每次可领取 1、2 或 3 个红包，且均采用最优策略。观察可知，无论先手（主播）第一次领取 1、2 还是 3 个红包，后手（观众）都能通过对应领取 3、2 或 1 个红包，使两人一轮共领取 4 个红包。依此策略，每轮领取的红包总数始终保持 4 的倍数，形成“4 个一组”的节奏。因此，若  $k-1$  是 4 的倍数，则后手（观众）可通过上述策略确保自己领取到第  $k-1$  个红包，迫使先手领取第  $k$  个陷阱红包，观众获胜；若  $k-1$  不是 4 的倍数，则先手（主播）可先领取  $(k-1)$  除以 4 的余数个红包，随后沿用“每轮共领 4 个”的策略，最终由先手领取第  $k-1$  个红包，迫使观众领取第  $k$  个陷阱红包，观众失败。综上，判断观众能否获胜，只需看  $k-1$  是否为 4 的倍数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int main() {
4 |     int n, k;
5 |     scanf("%d %d", &n, &k);
6 |     k--;
7 |     if (k % 4 == 0)
8 |         printf("Got it!");
9 |     else
10 |         printf("Miss.");
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

7 5

样例 1 输出

Got it!

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.36 | 总 403/526 Flowers ( PID 1227 )

出处：2017级河南工大新生周赛总决赛（重现）（J，CID 1049）；2017级河南工大新生周赛总决赛（重现）（J，CID 1039）；2017级河南工大新生周赛总决赛（J，CID 1038）；2016级河南工大新生周赛（五）（B，CID 1010）| 训练定位：进阶 | 贪心与博弈

标签：贪心、数学、周期性 | 限制：1.000 S / 128 MB | 历史统计：66/999 ( 6.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出答案 $k$ ，表示最多的花束数量。

输入要点：输入一行三个整数，分别表示红，绿，蓝三种颜色的花的数量。（所有数据均在 $\text{int}$ 范围以内）

输出要点：输出答案 $k$ ，表示最多的花束数量。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：一束花要么消耗同色 3 朵，要么三色各 1 朵。设先做  $t$  束混色花，总数为  $t+(a-t)/3+(b-t)/3+(c-t)/3$ 。若  $t$  增加 3，混色束多 3，三种单色束各少 1，总数不变，所以只需尝试  $t=0,1,2$ （且不能超过 最少的颜色数），取最大值即可。这避免了对  $\text{int}$  级数量逐束枚举。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     long long a, b, c;
7 |     cin >> a >> b >> c;
8 |     long long ans = 0;
9 |     for (long long mixed = 0; mixed <= min({2LL, a, b, c}); ++mixed) {
10 |         ans = max(ans, mixed + (a - mixed) / 3 + (b - mixed) / 3 + (c - mixed) / 3);
11 |     }
12 |     cout << ans << '\n';
```

```
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

3 6 9

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.37 | 总 404/526 ykc买零食 ( PID 1231 )

出处：2016级河南工大新生周赛（五）（F，CID 1010）| 训练定位：进阶 | 贪心与博弈

标签：贪心、排序、促销建模 | 限制：1.000 S / 128 MB | 历史统计：0/20 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：ykc的班级准备举行班级聚会，而身为生活委员的ykc要为此准备好零食，这天，ykc来到了学校的新起点超市，在转了3个小时，ykc决定买以下所有的n种零食，其中每种零食的价格可能不一样，而刚好超市有活动，每买m种零食，就可以任选一种不超过k元的零食并免费赠送，而ykc想尽可能的省钱，求ykc的最小花费

输入要点：输入包含多组数据，以EOF结束，

每组首先输入三个正整数，n,m,k，其中(n,m,k<100)

后输入n个数表示每种零食的价格ai(ai<1000)

输出要点：输出一个正整数，表示最小花费

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
- 核心处理采用：买 m 种送 1 种，若最终拿到 n 种，则最多有  $\text{floor}(n/(m+1))$  种 可作为赠品；同时赠品价格必须不超过 k。统计所有合格价格，实际免费数 是它与该上限的较小值。为了最省钱，应让其中价格最高的若干种免费：把合格价格降序排列，从总价中减去前 freeCount 个。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。

- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m, k;
7 |     while (cin >> n >> m >> k) {
8 |         long long total = 0;
9 |         vector<int> eligible;
10 |         for (int i = 0; i < n; ++i) {
11 |             int price;
12 |             cin >> price;
13 |             total += price;
14 |             if (price <= k)
15 |                 eligible.push_back(price);
16 |         }
17 |         sort(eligible.begin(), eligible.end(), greater<int>());
18 |         int freeCount = min<int>(eligible.size(), n / (m + 1));
19 |         for (int i = 0; i < freeCount; ++i)
20 |             total -= eligible[i];
21 |         cout << total << '\n';
22 |     }
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
4 3 2
1 2 3 4
7 3 8
1 2 3 4 5 6 7
```

样例 1 输出

```
8
21
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.38 | 总 405/526 zy最爱的足球赛 ( PID 1249 )

出处：2016级河南工大新生周赛（七）（A，CID 1014） | 训练定位：进阶 | 贪心与博弈

标签：贪心、区间调度、排序 | 限制：1.000 S / 128 MB | 历史统计：6/70 ( 8.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，对于每组输入，输出最多能安排的比赛数量，输出占一行。

输入要点：输入

第一行为一个整数 $t$ 表示有 $t$ 组测试数据。

魅族测试数据第一行为一个整数 $n$  ( $1 < n < 1000$ ) 表示共有 $n$ 场比赛。

随后 $n$ 行，每行有两个整数 $s_i, e_i$  ( $0 \leq s_i, e_i \leq 10000$ ) 表示第 $i$ 个比赛的起始与结束时间。

输出要点：对于每组输入，输出最多能安排的比赛数量，输出占一行。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- `sort` 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：按结束时间从早到晚排序，每次选择开始时间严格晚于上一场结束时间的比赛。严格大于来自样例：一场 1..10 与另一场 10..11 被视为冲突。结束越早为后面留下的可用时间越多；把任一最优方案的第一场替换成结束最早的可选场次，不会减少后续选择，因此贪心成立。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         vector<pair<int, int>> match(n);
12 |         for (auto& [start, finish] : match)
13 |             cin >> start >> finish;
14 |         sort(match.begin(), match.end(), [](const auto& a, const auto& b) {
15 |             if (a.second != b.second)
16 |                 return a.second < b.second;
17 |             return a.first < b.first;
18 |         });
19 |         int ans = 0;
20 |         int last = INT_MIN;
21 |         for (auto [start, finish] : match) {
22 |             if (start > last) {
23 |                 ++ans;
24 |                 last = finish;
25 |             }
26 |         }
```

```
27 |         cout << ans << '\n';
28 |     }
29 |     return 0;
30 | }
```

## 公开样例

样例 1 输入

```
2
2
1 10
10 11
3
1 10
10 11
11 20
```

样例 1 输出

```
1
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.39 | 总 406/526 01串 ( PID 1335 )

出处：2017级河南工大新生周赛 ( 五 ) ( A , CID 1033 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、子序列、游程、区间翻转 | 限制：1.000 S / 128 MB | 历史统计：6/43 ( 14.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，表示翻转后最长子序列的长度。

输入要点：第一行：T，测试实例个数

第二行：一个n表示01串的长度。(1≤n≤10000)

第三行：输入一个01串。

输出要点：一个整数，表示翻转后最长子序列的长度。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：二进制串的最长 0/1 交替子序列长度，恰好等于连续相同字符“段”的数量：每段最多取一个，而每段各取一个就能交替。  
翻转一个区间只会改变 区间左右两个边界是否相等，所以段数最多增加 2；选择合适的单点或区间 又能在尚未全交替时实现这两个新增边界。因此答案是 min(原段数+2,n)。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明



- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n)$  时间、 $O(1)$  额外空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         string s;
11 |         cin >> n >> s;
12 |         int runs = n > 0 ? 1 : 0;
13 |         for (int i = 1; i < n; ++i)
14 |             runs += (s[i] != s[i - 1]);
15 |         cout << min(n, runs + 2) << '\n';
16 |     }
17 |     return 0;
18 | }
```

## 公开样例

样例 1 输入

```
1
8
10000011
```

样例 1 输出

```
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.40 | 总 407/526 排队 ( PID 1339 )

出处：2017级河南工大新生周赛（五）（E，CID 1033） | 训练定位：进阶 | 贪心与博弈

标签：排序、贪心、中位分界 | 限制：1.000 S / 128 MB | 历史统计：38/212 ( 17.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：问是否排成这种队形，如果可以输出“YES”，否则输出“NO”

输入要点：第一行：T，测试实例个数。

第二行：一个整数n。（ $1 \leq n \leq 1000$ ）

第三行：2n个整数， $a_1, a_2, \dots, a_{2n}$ 。分别代表每个人的身高。（ $1 \leq a_i \leq 1000$ ）

输出要点：“YES”或“NO”。

## 零基础先修

- sort 默认排序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：排序后，最合理的分法一定是最矮的  $n$  人在前排、最高的  $n$  人在后排。条件“前排任意人都严格矮于后排任意人”等价于前排最大值  $a[n-1] <$  后排最小值  $a[n]$ 。若这里都不成立，换任何更高的人到前排 只会更差。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         vector<int> height(2 * n);
12 |         for (int& x : height)
13 |             cin >> x;
14 |         sort(height.begin(), height.end());
15 |         cout << (height[n - 1] < height[n] ? "YES" : "NO") << '\n';
16 |     }
17 |     return 0;
18 | }
```

## 公开样例

样例 1 输入

```
2
2
1 2 3 4
1
1 1
```

样例 1 输出

```
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.41 | 总 408/526 Choice and Bramble ( PID 1351 )

出处：2017级河南工大新生周赛 ( 七 ) ( E , CID 1036 ) | 训练定位：进阶 | 贪心与博弈

标签：博弈论、因数、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：0/15 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，如果Choide赢输出“Choice”,如果Bramble赢输出“Bramble”。

输入要点：输入数据有多组。

每组一行输入三个整数 $n, m, k (1 \leq n, m, k \leq 1e9)$

输出要点：如果Choide赢输出“Choice”,如果Bramble赢输出“Bramble”。

题面提示：对于第一个案例Choice先把第一种糖果分成3堆，每堆个数都为5满足要求，因为只有一种糖果，Bramble不能再操作，所以Choice赢了。

### 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：每种糖果至多被操作一次，而且  $n$  种完全相同，因此游戏等价于有  $n$  个可取物品：若这一种能够合法等分，一次操作就取走一个。若可操作数  $n$  为偶数，后手赢；为奇数，先手赢。合法等分要求  $m$  存在一个满足  $k \leq d < m$  的因数  $d$  ( 每堆  $d$  个且至少两堆 )。最大的真因数是  $m$ /最小质因数，找出它后与  $k$  比较即可。不能分时先手直接无步可走。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价值，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组最坏  $O(\sqrt{m})$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | long long div(long long value) {
4 |     for (long long divisor = 2; divisor * divisor <= value; ++divisor)
5 |         if (value % divisor == 0)
6 |             return value / divisor;
7 |     return 1;
8 | }
9 | int main() {
```

```

10 | ios::sync_with_stdio(false);
11 | cin.tie(nullptr);
12 | long long n, m, k;
13 | while (cin >> n >> m >> k) {
14 |     bool splittable = div(m) >= k;
15 |     bool choiceWins = splittable && (n % 2 == 1);
16 |     cout << (choiceWins ? "Choice" : "Bramble") << '\n';
17 | }
18 | return 0;
19 | }

```

## 公开样例

样例 1 输入

```

1 15 4
2 6 3

```

样例 1 输出

```

Choice
Bramble

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 6.42 | 总 409/526 QAQ Games ( PID 1361 )

出处：2017级河南工大新生周赛总决赛（重现）（A，CID 1049）；2017级河南工大新生周赛总决赛（重现）（A，CID 1039）；2017级河南工大新生周赛总决赛（A，CID 1038） | 训练定位：进阶 | 贪心与博弈

标签：博弈论、对称策略、分类讨论 | 限制：1.000 S / 128 MB | 历史统计：3/41 (7.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：可是QAQ还想知道他是否一定能获得游戏的胜利

输入要点：多组输入

每行一个N表示格子的总数（ $1 \leq N \leq 1000$ ）

输出要点：如果QAQ（先手玩家）能通过一定策略必胜，那么输出XD

如果必败，那就真的输出QAQ了

如果必定平局，输出Down

题面提示：[图片：[https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222142304\\_45663.png](https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222142304_45663.png)]

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：任何人都不会被逼“输”：若无法制造 QAQ，最差只是把棋盘填满而平局。当  $n < 7$  时，双方总能及时堵住所有威胁；偶数长度时，后手还可以利用关于中心的对称应对，使先手无法留下一个对方不能同时封堵的双威胁。当  $n \geq 7$  且为奇数，先手先在正中心放 Q。此后利用两端的对称位置制造两个潜在 QAQ，后手一次只能封住一边，先手最终完成另一边。因此只有“奇数且至少 7”是先手必胜 XD，

其余为平局 Down；不会出现必败 QAQ。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     while (cin >> n) {
8 |         cout << (n >= 7 && n % 2 == 1 ? "XD" : "Down") << '\n';
9 |     }
10 |     return 0;
11 | }
```

### 公开样例

样例 1 输入

```
3
4
7
```

样例 1 输出

```
Down
Down
XD
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 6.43 | 总 410/526 Travel ( PID 1364 )

出处：2017级河南工大新生周赛总决赛（重现）（D，CID 1049）；2017级河南工大新生周赛总决赛（重现）（D，CID 1039）；2017级河南工大新生周赛总决赛（D，CID 1038） | 训练定位：进阶 | 贪心与博弈

标签：贪心、区间覆盖、可达性 | 限制：1.000 S / 128 MB | 历史统计：30/157 ( 19.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，Print "YES" if there is a path from DSB's house to his friend's house that uses only teleports, and "NO" otherwise.

输入要点：The first line contains two integers  $n$  and  $m$  ( $1 \leq n \leq 100$ ,  $1 \leq m \leq 100$ ) — the number of teleports and the location of the friend's house.

The next  $n$  lines contain information about teleports.

The  $i$ -th of these lines contains two integers  $a_i$  and  $b_i$  ( $0 \leq a_i \leq b_i \leq m$ ), where  $a_i$  is the location of the  $i$ -th teleport, and  $b_i$  is its limit.

输出要点：Print "YES" if there is a path from DSB's house to his friend's house that uses only teleports, and "NO" otherwise.

题面提示：[图片：[https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222154536\\_17745.png](https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222154536_17745.png)]

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：从 0 出发，若当前能到达整个区间  $[0, \text{reached}]$ ，那么所有起点  $a_i \leq \text{reached}$  的传送器都能使用，并可把右端扩展到  $\max(\text{reached}, b_i)$ 。对传送器反复松弛，直到某一轮不再扩展。 $n$  只有 100，直接做至多  $n$  轮即可；最终  $\text{reached} \geq m$  时仅靠传送器可到朋友家。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n^2)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m;
7 |     cin >> n >> m;
8 |     vector<pair<int, int>> teleport(n);
9 |     for (auto& [a, b] : teleport)
10 |         cin >> a >> b;
11 |     int reached = 0;
12 |     for (int iteration = 0; iteration < n; ++iteration) {
13 |         int old = reached;
14 |         for (auto [a, b] : teleport)
15 |             if (a <= reached)
16 |                 reached = max(reached, b);
17 |         if (reached == old)
18 |             break;
19 |     }
20 |     cout << (reached >= m ? "YES" : "NO") << '\n';
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

样例1：  
3 5

```
0 2
2 4
3 5
```

样例2：  
3 7  
0 4  
2 5  
6 7

样例 1 输出

样例1：  
YES  
  
样例2：  
NO

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 6.44 | 总 411/526 Displaced Plant ( PID 1369 )

出处：2017级河南工大新生周赛总决赛（重现）（I，CID 1049）；2017级河南工大新生周赛总决赛（重现）（I，CID 1039）；2017级河南工大新生周赛总决赛（I，CID 1038） | 训练定位：进阶 | 贪心与博弈

标签：贪心、经典过桥问题、分类讨论 | 限制：1.000 S / 128 MB | 历史统计：4/16 ( 25.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出所有人下楼的最短时间

输入要点：单实例测试

第一行是一个整数 $n$ 表示共有 $n$ 个人（ $1 \leq n \leq 50$ ）

第二行是 $n$ 个整数 $a_i$ ，表示第 $i$ 个人操纵升降机要花的时间（ $1 \leq S_i \leq 100$ ），输入保证有序！

输出要点：输出所有人下楼的最短时间

题面提示：[图片：[https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222181204\\_77887.png](https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222181204_77887.png)]

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：这就是“过桥与手电筒”模型：升降机下楼后必须有人把它送回楼上。时间已排序。每次把当前最慢的两人送下去有两种最优候选：①最快两人配合摆渡，代价  $a_0 + 2a_1 + a_{\text{last}}$ ；②最快者分别陪两名最慢者，代价  $2a_0 + a_{\text{prev}} + a_{\text{last}}$ 。取较小者并删去最慢两人。剩 1、2、3 人时答案分别为  $a_0$ 、 $a_1$ 、 $a_0 + a_1 + a_2$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> a(n);
9 |     for (long long& x : a)
10 |         cin >> x;
11 |     long long ans = 0;
12 |     int people = n;
13 |     while (people > 3) {
14 |         long long planOne = a[0] + 2 * a[1] + a[people - 1];
15 |         long long planTwo = 2 * a[0] + a[people - 2] + a[people - 1];
16 |         ans += min(planOne, planTwo);
17 |         people -= 2;
18 |     }
19 |     if (people == 3)
20 |         ans += a[0] + a[1] + a[2];
21 |     else if (people == 2)
22 |         ans += a[1];
23 |     else if (people == 1)
24 |         ans += a[0];
25 |     cout << ans << '\n';
26 |     return 0;
27 | }
```

## 公开样例

样例 1 输入

```
4
1 1 5 8
```

样例 1 输出

```
12
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)



## 6.45 | 总 412/526 玩玩玩玩石子 ( PID 1392 )

出处：2018级河南工大新生周赛 ( 二 ) @陈润钊专场 ( E , CID 1041 ) | 训练定位：进阶 | 贪心与博弈

标签：数论、最大公约数、博弈论 | 限制：1.000 S / 128 MB | 历史统计：8/60 ( 13.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，如果乌拉拉获胜，输出 wulala 如果挖哈哈获胜，输出 wahaha

输入要点：一行，三个整数  $n, a, b$ 。 $n$  分别表示石子的数量，开局随机取出的两个石子编号。 $(1 < n \leq 20000)$ ,  $(1 \leq a, b \leq n)$  数据保证  $a, b$  不相等

输出要点：如果乌拉拉获胜，输出 wulala

如果挖哈哈获胜，输出 wahaha

题面提示：“万事万物皆有规律”

——— 欧几里得

### 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- std::gcd(a,b) 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。
- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：从已取出的  $a, b$  通过加减能生成的编号恰为  $\gcd(a,b)$  的倍数。1.. $n$  中共有  $\text{floor}(n/g)$  个；开局已移走的两颗不改变剩余数量的奇偶性，所以该值为奇数时先手拿到最后一步获胜，偶数时后手获胜。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(\log \min(a,b))$  时间、 $O(1)$  空间

### 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, a, b;
5 |     cin >> n >> a >> b;
6 |     long long movesParity = (n / gcd(a, b)) % 2;
7 |     cout << (movesParity ? "wulala" : "wahaha") << '\n';
8 |     return 0;
9 | }
```

### 公开样例

样例 1 输入

868

样例 1 输出

wahaha

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.46 | 总 413/526 资源箱 ( PID 1436 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( I , CID 1047 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、模拟、\_\_int128、构造 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：起重机所耗能的最大的和最小的值及其取得最大和最小值的每一步操作(数据保证：只有唯一解)

输入要点：多样例测试：

第一行输入T表示样例数( $1 < T \leq 10$ )

对于每个样例第一行输入n, m ( $0 < n, m < 10^5$ , n表示'A'车上的资源箱的数量， m表示'B'车上的资源箱的数量)

第二行输入n个整数 $a_i$ , 表示'A'车上从下到上的每个资源箱的重量。( $0 < a_i < \text{int}$ )

第三行输入m个整数 $b_i$ , 表示'B'车上从下到上的每个资源箱的重量。( $0 < b_i < \text{int}$ )

输出要点：对于每个样例：

第一行输出所耗能的最大值

接下来输出取得最大值移动的方式：

A>k>B:从'A'车移动 总重量为k的资源箱 到'B'车

B>k>A:从'B'车移动 总重量为k的资源箱 到'A'车

接下来的第一行输出所耗能的最小值

接下来输出取得最小值移动的方式：

A>k>B:从'A'车移动 总重量为k的资源箱 到'B'车

B>k>A:从'B'车移动 总重量为k的资源箱 到'A'车

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- \_\_int128 可保存约 38 位有符号整数，但输入输出需要自行转换。

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：把已经被勾过、目前位于某辆车顶部的箱子看成一个“整体块”。再勾这辆车的下一个未勾箱时，整体块会和新箱一起搬走；连续从同一辆车搬则只新增一个箱子，换边搬才会把已经积累的整体块再次计入耗能。最小耗能一定是把总重量较小的那一堆逐箱搬到另一边：连续同向操作时，每个箱子恰好被搬一次，总耗能就是该堆总重量。最大耗能时应让两边尽量交替，使已勾箱整体被反复搬运。最终留在 B，操作次数分别为 A:n、B:m-1；最终留在 A，则为 A:n-1、B:m。对每种结局，多出来的同向操作必须放在开头，随后严格交替。模拟两种唯一候选方案并取耗能较大者即可。总耗能可能超过 64 位，使用 `__int128`。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n+m)$  时间、 $O(n+m)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | using i128 = __int128_t;
4 | string toString(i128 x) {
5 |     if (x == 0)
6 |         return "0";
7 |     string s;
8 |     while (x > 0) {
9 |         s.push_back(char('0' + x % 10));
10 |        x /= 10;
11 |    }
12 |    reverse(s.begin(), s.end());
13 |    return s;
14 | }
15 | struct Plan {
16 |     i128 cost = 0;
17 |     vector<pair<char, i128>> ops;
18 | };
19 | string alt(int countA, int countB, char last) {
20 |     string labels;
21 |     labels.reserve(countA + countB);
22 |     if (last == 'A') {
23 |         if (countA >= countB) {
24 |             labels.append(countA - countB, 'A');
25 |             for (int i = 0; i < countB; ++i)
26 |                 labels += "BA";
27 |         } else {
28 |             labels.append(countB - countA, 'B');
29 |             for (int i = 0; i < countA; ++i)
30 |                 labels += "BA";
31 |         }
32 |     } else {
33 |         if (countB >= countA) {
34 |             labels.append(countB - countA, 'B');
35 |             for (int i = 0; i < countA; ++i)
36 |                 labels += "AB";
37 |         } else {
38 |             labels.append(countA - countB, 'A');
39 |             for (int i = 0; i < countB; ++i)
40 |                 labels += "AB";
```

```

41 |     }
42 | }
43 | return labels;
44 | }
45 | Plan simulate(const vector<long long>& a, const vector<long long>& b,
46 |             const string& labels) {
47 |     int ia = (int)a.size() - 1;
48 |     int ib = (int)b.size() - 1;
49 |     i128 hookedA = 0, hookedB = 0;
50 |     Plan plan;
51 |     plan.ops.reserve(labels.size());
52 |     for (char side : labels) {
53 |         if (side == 'A') {
54 |             i128 moved = hookedA + a[ia--];
55 |             hookedA = 0;
56 |             hookedB += moved;
57 |             plan.cost += moved;
58 |             plan.ops.push_back({'A', moved});
59 |         } else {
60 |             i128 moved = hookedB + b[ib--];
61 |             hookedB = 0;
62 |             hookedA += moved;
63 |             plan.cost += moved;
64 |             plan.ops.push_back({'B', moved});
65 |         }
66 |     }
67 |     return plan;
68 | }
69 | void printPlan(const Plan& plan) {
70 |     cout << toString(plan.cost) << '\n';
71 |     for (auto [side, weight] : plan.ops) {
72 |         if (side == 'A')
73 |             cout << "A>" << toString(weight) << ">B\n";
74 |         else
75 |             cout << "B>" << toString(weight) << ">A\n";
76 |     }
77 | }
78 | int main() {
79 |     ios::sync_with_stdio(false);
80 |     cin.tie(nullptr);
81 |     int T;
82 |     cin >> T;
83 |     while (T--) {
84 |         int n, m;
85 |         cin >> n >> m;
86 |         vector<long long> a(n), b(m);
87 |         for (long long& x : a)
88 |             cin >> x;
89 |         for (long long& x : b)
90 |             cin >> x;
91 |         Plan endAtB = simulate(a, b, alt(n, m - 1, 'A'));
92 |         Plan endAtA = simulate(a, b, alt(n - 1, m, 'B'));
93 |         printPlan(endAtB.cost > endAtA.cost ? endAtB : endAtA);
94 |         i128 sumA = 0, sumB = 0;
95 |         for (long long x : a)
96 |             sumA += x;
97 |         for (long long x : b)
98 |             sumB += x;
99 |         Plan mn;
100 |         if (sumA < sumB) {
101 |             mn.cost = sumA;
102 |             for (int i = n - 1; i >= 0; --i)
103 |                 mn.ops.push_back({'A', a[i]});
104 |         } else {
105 |             mn.cost = sumB;
106 |             for (int i = m - 1; i >= 0; --i)
107 |                 mn.ops.push_back({'B', b[i]});
108 |         }
109 |         printPlan(mn);
110 |     }
111 |     return 0;
112 | }

```

## 公开样例

### 样例 1 输入

```

1
2 2
2 3
1 1

```

### 样例 1 输出

```

13
A>3>B
B>4>A
A>6>B

```

```
2
B>1>A
B>1>A
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.47 | 总 414/526 想带Ah去浪漫的土耳其 ( PID 1493 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( A , CID 1056 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、计数、分类讨论 | 限制：1.000 S / 128 MB | 历史统计：44/343 ( 12.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：是想和Ah一起的。旅行社为了节约成本，最少要多少辆出租车。

输入要点：第一行输入一个n表示组数 ( $1 \leq n \leq 105$ )

第二行输入n组，每组i人。

输出要点：一个整数，最少该叫多少车。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：分别统计 1、2、3、4 人组。4 人组独占一车；3 人组尽量各配一个 1 人组；2 人组两两配车，若剩一个 2 人组再带最多两个 1 人组；最后剩余 1 人组每四组一车。这个贪心始终优先填满最难安排的大组。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
```

```

5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     int count[5] = {};
9 |     for (int i = 0, size; i < n; ++i) {
10 |         cin >> size;
11 |         ++count[size];
12 |     }
13 |     int ans = count[4];
14 |     ans += count[3];
15 |     count[1] = max(0, count[1] - count[3]);
16 |     ans += count[2] / 2;
17 |     if (count[2] % 2) {
18 |         ++ans;
19 |         count[1] = max(0, count[1] - 2);
20 |     }
21 |     ans += (count[1] + 3) / 4;
22 |     cout << ans << '\n';
23 |     return 0;
24 | }

```

## 公开样例

样例 1 输入

```

5
1 2 4 3 3

```

样例 1 输出

```

4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 6.48 | 总 415/526 村里有个姑娘叫小Jun ( PID 1498 )

出处：2019级河南工大新生周赛 ( 四 ) @胡维强专场 ( C , CID 1056 ) | 训练定位：进阶 | 贪心与博弈

标签：并查集、贪心、区间合并 | 限制：1.000 S / 128 MB | 历史统计：0/32 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数

输入要点：第一行两个整数 $n, m$  ( $n$ 为村庄个数， $m$ 为直连边数)  $3 \leq n \leq 200\,000$  and  $1 \leq m \leq 200\,000$

接下来 $m$ 行，每一行两个整数表示村庄 $a_i$ 和 $a_j$ 直连。

输出要点：一个整数

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：先用并查集连好已有道路，并记录每个连通块内的最大编号。顺序 扫描一个区间  $[left, right]$ ：若扫到的点属于新连通块，为使整个编号 区间和谐就必须加一条边合并它，同时  $right$  可能扩展到该块的最大 编号。直到越过  $right$  后再开始下一个区间。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O((n+m)\alpha(n))$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | struct DSU {
4 |     vector<int> parent, mx;
5 |     DSU(int n) : parent(n + 1), mx(n + 1) {
6 |         iota(parent.begin(), parent.end(), 0);
7 |         iota(mx.begin(), mx.end(), 0);
8 |     }
9 |     int find(int x) {
10 |         return parent[x] == x ? x : parent[x] = find(parent[x]);
11 |     }
12 |     void unite(int a, int b) {
13 |         a = find(a);
14 |         b = find(b);
15 |         if (a == b)
16 |             return;
17 |         parent[b] = a;
18 |         mx[a] = max(mx[a], mx[b]);
19 |     }
20 | };
21 | int main() {
22 |     ios::sync_with_stdio(false);
23 |     cin.tie(nullptr);
24 |     int n, m;
25 |     cin >> n >> m;
26 |     DSU dsu(n);
27 |     while (m--) {
28 |         int a, b;
29 |         cin >> a >> b;
30 |         dsu.unite(a, b);
31 |     }
32 |     int ans = 0;
33 |     for (int left = 1; left <= n; ) {
34 |         int root = dsu.find(left);
35 |         int right = dsu.mx[root];
36 |         for (int point = left + 1; point <= right; ++point) {
37 |             int other = dsu.find(point);
38 |             root = dsu.find(root);
39 |             if (other != root) {
40 |                 right = max(right, dsu.mx[other]);
41 |                 dsu.unite(root, other);
42 |                 root = dsu.find(root);
43 |                 ++ans;
44 |             }
45 |         }
46 |         left = right + 1;
47 |     }
48 |     cout << ans << '\n';
49 |     return 0;
50 | }
```

## 公开样例

### 样例 1 输入

```
14 8
1 2
2 7
3 4
6 3
5 7
3 8
6 8
11 12
```

### 样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.49 | 总 416/526 zp与朋友 (PID 1509)

出处：2019级河南工大新生周赛 (五) @潘世泽专场 (C, CID 1057) | 训练定位：进阶 | 贪心与博弈

标签：字符串、贪心、双指针 | 限制：1.000 S / 128 MB | 历史统计：1/8 (12.5%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**zp的朋友找到了他并且提出了宝可梦对战！对战的时候zp和他的朋友都必须喊出技能的名字，但是技能的名字在神秘力量的作用下成为了被打乱了顺序的字符串，zp必须在经过交换后得到从左往右读和从右往左读是一样的字符串才能释放技能，否则技能会无法释放。请你计算技能能否成功释放，如果能请计算最少的交换次数！交换的定义是：交换两个相邻的字符

**输入要点：**第一行是一个整数N，表示接下来的字符串的长度( $N \leq 500$ )，第二行是一个字符串，长度为N.只包含小写字母。

**输出要点：**如果可能，输出最少的交换次数，否则输出Impossible。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 双指针通过单调移动避免重复枚举；要说明移动哪一端以及为什么不会错过答案。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：先检查字符频次，奇数长度最多一个奇频字符、偶数长度不能有奇频，否则 Impossible。用左右指针：从右端向左找与 `s[left]` 相同的字符，找到后用相邻交换把它移到 right；若直到 left 都找不到，说明当前字符是唯一中间字符，把它向右交换一格后重试。该贪心每步的移动量都是不可避免的最小值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n^2)$  时间、 $O(1)$  额外空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。



## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     string s;
6 |     cin >> n >> s;
7 |     array<int, 26> count{};
8 |     for (char c : s)
9 |         ++count[c - 'a'];
10 |     int odd = 0;
11 |     for (int x : count)
12 |         odd += x % 2;
13 |     if (odd > n % 2) {
14 |         cout << "Impossible\n";
15 |         return 0;
16 |     }
17 |     long long swaps = 0;
18 |     int left = 0, right = n - 1;
19 |     while (left < right) {
20 |         int match = right;
21 |         while (match > left && s[match] != s[left])
22 |             --match;
23 |         if (match == left) {
24 |             swap(s[left], s[left + 1]);
25 |             ++swaps;
26 |         } else {
27 |             while (match < right) {
28 |                 swap(s[match], s[match + 1]);
29 |                 ++match;
30 |                 ++swaps;
31 |             }
32 |             ++left;
33 |             --right;
34 |         }
35 |     }
36 |     cout << swaps << '\n';
37 |     return 0;
38 | }
```

## 公开样例

样例 1 输入

```
5
mamad
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.50 | 总 417/526 zp与草系道馆 ( PID 1512 )

出处：2019级河南工大新生周赛 ( 五 ) @潘世泽专场 ( F , CID 1057 ) | 训练定位：进阶 | 贪心与博弈

标签：博弈论、取石子、取模 | 限制：1.000 S / 128 MB | 历史统计：11/175 ( 6.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果zp获胜那么输出"zp is winner!"，否则输出"curator is winner!"

输入要点：多实例，第一行输入T(T<10),第二行输入草的总单位数n(1<=n<=100)

输出要点：如果zp获胜那么输出"zp is winner!"，否则输出"curator is winner!"

## 零基础先修

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：每次可取 1、2、3，能把 4 的倍数留给对手就是必胜策略。因此  $n\%4=0$  为必败态，其余情况先取  $n\%4$  个，再不断把双方一次取数之和 补成 4。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- C++ 负数取模仍可能为负；减法后补模数。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int T;
5 |     cin >> T;
6 |     while (T--) {
7 |         int n;
8 |         cin >> n;
9 |         cout << (n % 4 ? "zp is winner!" : "curator is winner!") << '\n';
10 |    }
11 |    return 0;
12 | }
```

## 公开样例

样例 1 输入

```
1
5
```

样例 1 输出

```
zp is winner!
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.51 | 总 418/526 不一样的蛋糕 ( PID 1515 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( B , CID 1058 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：0/12 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：EF要过生日了，他请了他的小伙伴来家里聚会，晚餐是EF新学会做的巧克力蛋糕。可是由于EF还不太熟练，于是做出来的蛋糕的大小参差不齐。而EF为了每个小伙伴都有一样的体验，他就准备把  $n$  个蛋糕切成大小为  $k$  的小块分给  $m$  个小伙伴，切出来的蛋糕的形状可以忽略。而且EF有强迫症，他必须给每个人的都是一整块蛋糕，而不是从几块蛋糕上分别切一块再拼起来。而切剩下的.....

输入要点：输入包含两行

第一行包含两个整数  $n$  和  $m$ ，表示蛋糕的数量和小伙伴的个数 ( $1 \leq m, n \leq 104$ )

第二行包含  $n$  个整数  $a[i]$ ，表示第  $i$  块蛋糕的大小 ( $1 \leq a[i] \leq 109$ )

输出要点：输出包含一个数字  $k$ ，数字保留小数点后两位（采用四舍五入略去剩下的的小数）。

题面提示：样例解释：

切出大小为 4.5 的蛋糕块，最多可以切出 6 块，每人拿一块，剩下的扔掉。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：不一样的蛋糕二分基础题 二分切出的蛋糕的大小，然后检验大小是否合格，不断夹逼最优值即可 C++版代码
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <iostream>
2 | #include <cstdio>
3 | #include <vector>
4 | using namespace std;
5 | typedef long long LL;
6 | int n, m;
7 | vector<LL> a;
8 | LL sum;
9 | void init() {
10 |     sum = 0;
```

```

11 |     a.resize(n);
12 | }
13 | bool che(double mid) {
14 |     int num = 0;
15 |     for (int i = 0; i < n; ++i) {
16 |         num += (int)(a[i] / mid);
17 |     }
18 |     return num >= m;
19 | }
20 | double sol() {
21 |     init();
22 |     for (int i = 0; i < n; ++i)
23 |         cin >> a[i], sum += a[i];
24 |     double l = 0, r = sum, ans = r;
25 |     while (r - l > 1e-5) {
26 |         double mid = (l + r) / 2;
27 |         if (che(mid))
28 |             ans = mid, l = mid;
29 |         else
30 |             r = mid;
31 |     }
32 |     return ans;
33 | }
34 | int main() {
35 |     while (cin >> n >> m) {
36 |         printf("%.2f\n", sol());
37 |     }
38 |     return 0;
39 | }

```

## 公开样例

样例 1 输入

```

3 5
9 9 9

```

样例 1 输出

```

4.50

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.52 | 总 419/526 最小的整数 ( PID 1518 )

出处：2019级河南工大新生周赛 ( 六 ) @张伟涛专场 ( D , CID 1058 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、字符串、归并、相对顺序 | 限制：1.000 S / 128 MB | 历史统计：1/23 ( 4.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：那么通过若干次类似的交换（或者不交换），我们能得到的大小最小的数字是什么呢？

输入要点：多组测试数据，每组包含一行，每行包含一个整数 a

输入以 EOF 结束

输出要点：每组测试数据输出包含一行，输出若干次交换后的最小的数字

题面提示：样例解释：

```

4669827589->

```

```

4668927589->

```

4668297589->

4668297859->

4668298759->

4668289759->

4668289759

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：奇数只能与相邻偶数交换，因此所有奇数内部的相对顺序不能改变，所有偶数内部也不能改变；奇偶两条子序列则可以任意穿插。分别提取 两条子序列，每次选择当前队首较小的数字，就得到字典序最小合并。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(|s|)$  时间、 $O(|s|)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     string number;
7 |     while (cin >> number) {
8 |         string evenDigits, oddDigits;
9 |         for (char digit : number) {
10 |             if ((digit - '0') % 2 == 0)
11 |                 evenDigits.push_back(digit);
12 |             else
13 |                 oddDigits.push_back(digit);
14 |         }
15 |         size_t evenIndex = 0, oddIndex = 0;
16 |         string ans;
17 |         while (evenIndex < evenDigits.size() || oddIndex < oddDigits.size()) {
18 |             if (oddIndex == oddDigits.size() ||
19 |                 (evenIndex < evenDigits.size() &&
20 |                  evenDigits[evenIndex] < oddDigits[oddIndex])) {
21 |                 ans.push_back(evenDigits[evenIndex++]);
22 |             } else {
23 |                 ans.push_back(oddDigits[oddIndex++]);
24 |             }
25 |         }
26 |         cout << ans << '\n';
27 |     }
28 |     return 0;
29 | }
```

## 公开样例

样例 1 输入

```
4669827589
```

样例 1 输出

```
4668289759
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.53 | 总 420/526 小青的大学英语 ( PID 1538 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（D，CID 1060）| 训练定位：进阶 | 贪心与博弈

标签：排序、贪心、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：18/137 ( 13.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：众所周知，大学英语的划水率特别高，大部分同学都是在下面玩手机，小青在经历了两周的认真学习之后也加入了玩手机大军。这天，小青突然想起昨晚停电了，手机电量只剩百分之五。教室内有 $k$ 个可以使用的充电插口，本来小青可以快乐的独占一个插口，但由于昨晚停电，班里还有 $m-1$ 名同学手机也没电了，显然有可能无法完成插口的按需分配。于是小青从他的神奇背包里拿出了 $n$ 个插排，现在小青.....

输入要点：多组测试数据，每组第一行输入三个整数 $n, m, k$ 。（ $1 \leq n, m, k \leq 50$ ）

第二行有 $n$ 个正整数，分别表示每个插排上插口的个数，保证每个插排上插口个数小于100。

输出要点：输出最少需要的插排数，如果小青把所有的插排都用上还是有手机无法充上电就输出-1。

每组输出占一行。

### 零基础先修

- sort 默认排序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 多组数据以 EOF 结束时使用 while (cin >> ...)，不要额外输出测试数据。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：一个有  $x$  个插口的插排自身要占用一个现有插口，所以净增加  $x-1$  个可用口。初始有  $k$  个墙上插口；把净增量从大到小使用，首次 令总容量达到  $m$  台手机时就是最少插排数。多组数据读到文件尾。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。

- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- EOF 题不要先读不存在的 T，也不要再在循环结束后多输出内容。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m, k;
7 |     while (cin >> n >> m >> k) {
8 |         vector<int> gain(n);
9 |         for (int& x : gain) {
10 |             cin >> x;
11 |             --x;
12 |         }
13 |         sort(gain.rbegin(), gain.rend());
14 |         int capacity = k, ans = 0;
15 |         while (ans < n && capacity < m)
16 |             capacity += gain[ans++];
17 |         cout << (capacity >= m ? ans : -1) << '\n';
18 |     }
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
3 5 3
3 1 2
4 7 2
3 3 2 4
5 5 1
1 3 1 2 1
```

样例 1 输出

```
1
2
-1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.54 | 总 421/526 后缀自动机next指针dag图上跑SG函数 ( PID 1559 )

出处：2020级HAUT新生周赛（四）@张承树专场（E，CID 1063）| 训练定位：进阶 | 贪心与博弈

标签：贪心、贡献法、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/17 ( 11.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：c++创始人想要把所有的宝石都买光，他发现可以通过一定的购买技巧使得买来所有的宝石花费最小，请你告诉他买完这些宝石的最小花费是多少吧。

输入要点：第一行一个n代表这个商贩有n个宝石

第二行n个数，表示商贩每个宝石的价值

$1 \leq n \leq 1e6$

$|a[i]| \leq 1e6$

输出要点：一个值，表示c++创始人花费的最小金额

题面提示： $1 \leq n \leq 1e6$

$|a[i]| \leq 1e6$

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：无论购买顺序如何，每件宝石至少按位置 1 支付一次。若  $a_i < 0$ ，希望购买它时原来在它前面的  $i-1$  件都尚未删除，使负数乘尽可能大的位置  $i$ ；若  $a_i \geq 0$ ，则希望这些前驱都已删完，使位置为 1。先按原下标从右到左买所有负值，再从左到右买其余值即可同时达到这些最优位置。因而答案为  $\sum(a_i - i, a_i < 0) + \sum(a_i, a_i \geq 0)$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     long long ans = 0;
9 |     for (int i = 1; i <= n; ++i) {
10 |         long long value;
11 |         cin >> value;
12 |         ans += value < 0 ? value * i : value;
13 |     }
14 |     cout << ans << '\n';
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入



```
2
2 3
```

样例 1 输出

```
5
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.55 | 总 422/526 石子游戏 ( PID 1567 )

出处：2020级HAUT新生周赛（五）@许金龙专场（D，CID 1064）| 训练定位：进阶 | 贪心与博弈

标签：位运算、贪心、数论 | 限制：1.000 S / 128 MB | 历史统计：0/45 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现在请绝顶聪明的你判断一下其他玩家是否有可能挑战成功，若能输出 "YES"，否则输出 "NO"；

输入要点：第一行三个整数， $a$ ， $b$ ， $k$ 。

数据范围：

$0 < a, b \leq 1000000$

$0 < k \leq 20$

输出要点：共一行，按照题中描述输出 YES 或 NO

题面提示：样例分析：

初始  $a=8, b=30, k=3$

第一次操作：选择一， $a = a+21, a=10, b=30, k=2$ 。

第二次操作：选择一， $a = a+22, a=14, b=30, k=1$ 。

第三次操作：选择三， $a=14, b=30, k=1$ 。

第四次操作：选择一， $a = a+24, a=30, b=30, k=0$ 。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：第  $i$  次操作的位移是  $2^i$ ，且每个幂最多使用一次；移动较小石子记正号、移动较大石子记负号，问题变成用最少数带符号的二进制幂表示距离差。差为奇数时不可能，因为所有位移均为偶数。差除以 2 后用非相邻形式 NAF：奇数末两位为 01 时减 1，为 11 时加 1，每次选择计一次，所得非零位数最少。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(\log |a-b|)$  时间、 $O(1)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long a, b, k;
5 |     cin >> a >> b >> k;
6 |     long long diff = labs(a - b);
7 |     if (diff % 2) {
8 |         cout << "NO\n";
9 |         return 0;
10 |    }
11 |    long long value = diff / 2;
12 |    int needed = 0;
13 |    while (value > 0) {
14 |        if ((value & 1) == 0) {
15 |            value >>= 1;
16 |        } else {
17 |            ++needed;
18 |            if (value == 1 || value % 4 == 1)
19 |                --value;
20 |            else
21 |                ++value;
22 |        }
23 |    }
24 |    cout << (needed <= k ? "YES" : "NO") << '\n';
25 |    return 0;
26 | }
```

## 公开样例

样例 1 输入

8 30 3

样例 1 输出

YES

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.56 | 总 423/526 放烟花 ( PID 1571 )

出处：2020级HAUT新生周赛（五）@许金龙专场（G，CID 1064）| 训练定位：进阶 | 贪心与博弈

标签：排序、贪心、枚举 | 限制：1.000 S / 128 MB | 历史统计：10/84 ( 11.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，最小花费时间

输入要点：第一行，正整数  $n$  和 非负整数  $m$

第二行  $n$  个正整数。

数据范围：

$1 \leq n \leq 1000$

$0 \leq m \leq 1000$

$1 \leq a[i] \leq 100000$

输出要点：最小花费时间

题面提示：第一根引线连接 1 和 2 花费时间 11

第二根引线连接 1 和 3 花费时间 22

点燃第 1 箱烟花 花费时间 100，第 2、3 箱烟花被瞬间点燃 花费时间 0

点燃第 4 箱烟花 花费时间 100

点燃第 5 箱烟花 花费时间 100

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：使用  $k$  根引线可把  $k$  箱烟花并入其他箱所在连通块，从而少手动点燃  $k$  箱；显然应省下点燃时间最大的  $k$  项。第  $1..k$  根引线总安装时间为  $11 \cdot k(k+1)/2$ 。排序后枚举  $0..min(m, n-1)$  根，取 总点燃时间-前  $k$  大之和+引线时间的最小值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m;
7 |     cin >> n >> m;
8 |     vector<long long> time(n);
9 |     long long total = 0;
10 |     for (long long& x : time) {
11 |         cin >> x;
12 |         total += x;
13 |     }
14 |     sort(time.rbegin(), time.rend());
15 |     long long ans = total, saved = 0;
16 |     for (int k = 1; k <= min(m, n - 1); ++k) {
17 |         saved += time[k - 1];
18 |         long long fuse = 1LL * k * (k + 1) / 2;
19 |         ans = min(ans, total - saved + fuse);
20 |     }
21 |     cout << ans << '\n';
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

5 2
100 100 100 100 100

```

样例 1 输出

```

333

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 6.57 | 总 424/526 完美序列 ( PID 1572 )

出处：2020级HAUT新生周赛（五）@许金龙专场（H，CID 1064）| 训练定位：进阶 | 贪心与博弈

标签：贪心、路径点覆盖、整除 | 限制：1.000 S / 128 MB | 历史统计：17/70 ( 24.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**现在 lcl 有一个序列，他想让你通过更改序列里某些元素的值来实现完美序列。你可以最少更改几个元素来使这个序列变为完美序列？

**输入要点：**第一行一个正整数  $n$

第二行  $n$  个正整数  $a[i]$

**数据范围**

$1 \leq n \leq 1000$

$1 \leq a[i] \leq 1000000$

输出要点：满足题意的最小更改数

题面提示：把 6 改为 8

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：把不满足整除关系的相邻对看成路径上的“坏边”。每条坏边至少要 修改一个端点；反之把选中的端点都改成 1，就能同时修好与它相邻的 两条边。因此问题是路径的最小点覆盖：从左向右遇到尚未覆盖的坏边，修改其右端点并跳过下一条边。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> a(n);
9 |     for (long long& x : a)
10 |         cin >> x;
11 |     int ans = 0;
12 |     for (int i = 1; i < n; i++) {
13 |         bool good = (a[i - 1] % a[i] == 0) || (a[i] % a[i - 1] == 0);
14 |         if (good)
15 |             ++i;
16 |         else {
17 |             ++ans;
18 |             i += 2;
19 |         }
20 |     }
21 |     cout << ans << '\n';
22 |     return 0;
23 | }
```

## 公开样例

样例 1 输入

```
5
2 4 6 32 64
```

样例 1 输出

```
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.58 | 总 425/526 结束了(hard version) ( PID 1724 )

出处：2021级HAUT新生周赛（六）@李志豪专场（H，CID 1074）| 训练定位：进阶 | 贪心与博弈

标签：数组与序列、贪心、数论 | 限制：1.000 S / 128 MB | 历史统计：2/49（4.1%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：“草原上有一个兔子王国，里面有 $n$ 只兔子公民，编号为1到 $n$ ，公民在每年劳作之后要向国王缴纳税款，每只兔子缴纳的税款和自己的编号相同。但是国王的收税库被施了魔法，只能接受能把自己的容量整除的数量。现在国王想尽可能多的收税，但是不想提供过多的容量，请你给出一个最小的容量数吧~”

输入要点：一个整数 $n$ 。（ $1 \leq n \leq 1e8$ ）

输出要点：一个整数表示最小的容量数。由于答案可能过大，请把答案对 $1e9+7$ 取模。

题面提示：小m为了不让小s延毕，热心的另外告诉了一组样例：

输入样例

20

输出样例

232792560

另外还担心小s可能不会线性时间筛素数，直接把线性筛素数的代码也给你了，他真的不想让小s延毕啊延毕了自己又得再延一年才能一起毕业了。

```
//C++ 使用C++的话推荐手动开启O2优化
#pragma GCC optimize("O2")
const int max_n = 1e8 + 100;
int primes[6000000], id = 0;
bool used[max_n];
//primes里面存的素数，used[i] == 0 代表i是素数 id代表素数的数量 运行完init() 函数之后 直接用这几个数组然后提交的时候选择 C++ 就行了
void init(int n)
{
    for (int i = 2; i <= n; i++)
    {
        if (!used[i])
            primes[id++] = i;
        for (int j = 0; j < id && i * primes[j] <= n; j++)
        {
            used[i * primes[j]] = true;
            if (i % primes[j] == 0)
                break;
        }
    }
}
```

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：小m为了不让小s延毕，热心的另外告诉了一组样例：输入样例 20 输出样例 232792560 另外还担心小s可能不会线性时间筛素数，直接把线性筛素数的代码也给你了，他真的不想让小s延毕啊延毕了自己又得再延一年才能一起毕业了。//C++ 使用C++的话推荐手动开启O2优化 #pragma GCC optimize("O2") const int max\_n = 1e8 + 100; int primes[6000000], id = 0; bool used[max\_n]; //primes里面存的素数，used[i] == 0 代表i是素数 id代表素数的数量 运行完init() 函数之后 直接用这几个数组然后提交的时候选择 C++ 就行了 void init(int n) { for (int i = 2; i <= n; i++) { if (!used[i]) primes[id++] = i; for (int j = 0; j < id && i \* primes[j] <= n; j++) { used[i \* primes[j]] = true; if (i % primes[j] == 0) break; } } }
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #pragma GCC optimize("O2")
4 | #define ll long long
5 | #define endl "\n"
6 | const int max_n = 1e8 + 100;
7 | const int inf = 0x3f3f3f3f;
8 | const int mod = 1e9 + 7;
9 | int primes[6000000], id = 0;
10 | bool used[max_n];
11 | void init(int n) {
12 |     for (int i = 2; i <= n; ++i) {
13 |         if (!used[i])
14 |             primes[id++] = i;
15 |         for (int j = 0; j < id && i * primes[j] <= n; ++j) {
16 |             used[i * primes[j]] = true;
17 |             if (i % primes[j] == 0)
18 |                 break;
19 |         }
20 |     }
21 | }
22 | int main() {
23 |     ios::sync_with_stdio(false);
24 |     cin.tie(0);
25 |     cout.tie(0);
26 |     int n;
27 |     cin >> n;
28 |     init(n);
29 |     ll ans = 1;
30 |     for (int i = 0; i < id; ++i) {
31 |         if (primes[i] > n)
32 |             break;
33 |         ll t = 1;
34 |         while (t <= n)
35 |             t *= primes[i];
36 |         t /= primes[i];
37 |         ans = ans * t % mod;
38 |     }
39 |     cout << ans << endl;
40 |     return 0;
41 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.59 | 总 426/526 你是人间四月天 ( PID 1727 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（B，CID 1075）| 训练定位：进阶 | 贪心与博弈

标签：条件分支、字符串、博弈论、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：0/15 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：第三章：给两个玩家相同的卡牌，卡牌上分别带有不同的字符，字符分为'A','B','C','D','E' 五种（双方分配卡牌字符相同），其中A卡牌最大，B卡牌其次，以此类推。每次双方选择一张卡牌，进行大小比对，大的记一分，大小相同都不计分,当所有卡牌比对结束后，分数较大的获胜，分数相同记为房主胜。

输入要点：第一行输入进行的游戏局数count ( 1~1000 )

接下来的count组数据中

第一行输入两个数a,b表示两个玩家的卡牌数 ( 1~100 )

第二行输入两个数c,d表示两个玩家的卡牌数 ( 1~100 )

第三行输入一个数n,表示将要分配的卡牌数 ( 1~100 )

接下来输入一行字符串只含有 "A B C D E" 五个字符

输出要点：根据要求输出count行内容

若晚风能获得胜利输出"Night Breeze" ( 不含引号 )

若四月最终会取得胜利则输出"April Days" ( 不含引号 )

题面提示：-

输入的数据中第一个均表示房主玩家（晚风）

-

第三章的游戏详细说明：双方卡牌数相同和卡牌字符均相同，两人每次选择一个卡牌，四月可以根据晚风所选的卡牌来确定自己要选哪一张，并且四月知道晚风剩下哪些卡牌，因为四月有作弊码

-

第二局第三章样例说明：晚风卡牌ACD，四月卡牌ACD，当晚风出A四月出C，晚风出C四月出D，晚风出D四月出A，即可使本章游戏晚风获胜

数据均为正整数



## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：你是人间四月天博弈 第一章，四月每次将晚风卡牌移到自己这里，晚风将卡牌移到四月的卡牌堆中，晚风在这一章必胜。第二章，只有当两人卡牌都只有一张时，四月后手拿牌，晚风第二回合将无卡牌可拿，晚风会失败，其他情况晚风必胜。第三章，两人卡牌相同，晚风出什么四月就出什么，最终两人分数均为0，晚风胜。综合三局游戏，只要第二章不是两人卡牌都是1，晚风即可获胜 C
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int count;
4 |     int a, b;
5 |     int c, d;
6 |     int n;
7 |     char s[101];
8 |     scanf("%d", &count);
9 |     while (count--) {
10 |         scanf("%d %d", &a, &b);
11 |         scanf("%d %d", &c, &d);
12 |         scanf("%d", &n);
13 |         scanf("%s", s);
14 |         if (c == 1 && d == 1)
15 |             printf("April Days\n");
16 |         else
17 |             printf("Night Breeze\n");
18 |     }
19 | }
```

## 公开样例

样例 1 输入

```
2
1 1
1 3
3
BCD
2 1
1 2
3
ACD
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 6.60 | 总 427/526 我什么都愿意做 ( PID 1765 )

出处：2022级HAUT新生周赛（二）@武其轩专场（D，CID 1080）| 训练定位：进阶 | 贪心与博弈

标签：排序、贪心、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：58/278 ( 20.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：喝完幸运药水之后，你获得了 $n$ 点幸运值，现在你想要消耗掉这 $n$ 点幸运值并使其收益最大化。你有 $n$ 个活动可以同时参加（每项活动只能参加一次），参加不同的活动会获得不同数量的收益，且每个活动会消耗一点幸运值，你该如何安排你的活动使得到的收益最大呢？

输入要点：一个整数 $n(1 \leq n \leq 100)$ ，代表你的幸运值。

接下来一个整数 $t(1 \leq t \leq 10000)$ 表示可以参加的活动数量，随后 $t$ 个不超过 $1e8$ （即 $100000000$ ）的正整数，分别代表第1到第 $t$ 个活动会获得的收益。

输出要点：一行一个整数，输入最大收益。

题面提示：建议使用较快的输入输出

### 零基础先修

- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：每个活动消耗完全相同的一点幸运值，且最多参加 $n$ 个，因此应优先选择收益最大的 $n$ 个活动。把全部收益降序排序，累加前 $\min(n, t)$ 个；收益总和可能达到 $1e10$ ，必须用 long long。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(t \log t)$  时间、 $O(t)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, t;
7 |     cin >> n >> t;
8 |     vector<long long> gain(t);
9 |     for (long long& x : gain)
10 |         cin >> x;
11 |     sort(gain.rbegin(), gain.rend());
12 |     long long ans = 0;
13 |     for (int i = 0; i < min(n, t); ++i)
14 |         ans += gain[i];
15 |     cout << ans << '\n';
16 |     return 0;
17 | }
```

## 公开样例

样例 1 输入

```
3
4 1 2 3 4
```

样例 1 输出

```
9
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 6.61 | 总 428/526 一花依世界 ( PID 1789 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（D，CID 1083）| 训练定位：进阶 | 贪心与博弈

标签：贪心、博弈论、大整数 | 限制：1.000 S / 128 MB | 历史统计：1/22 ( 4.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入， $t$ 行，每行一个整数，小梦能吃到的包子的最大数量。

输入要点：第一行一个整数  $t$ ，表示测试用例的个数。

接下来  $t$  行每行一个整数  $n$ ，表示一笼有  $n$  个小笼包。

$(1 \leq n \leq 109)(1 \leq t \leq 105)$

输出要点： $t$ 行，每行一个整数，小梦能吃到的包子的最大数量。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。
- 估算最大值：若乘积可能超过约  $9 \times 10^{18}$ ，就需 `__int128` 或高精度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：一花依世界 题意：计算出按照题目所给规则所能吃到的最大包子数。解法：首先，可能有很多同学选择优先吃一半的方法，但显然不对，如果吃一半后仍为偶数相当于给了对手一半包子所以不是最优选择。所以应该在对手的角度考虑，如何让对手吃的最少，自己就吃的最多。显然让对手吃的最少情况是，每次轮到对手来取的时候都是奇数，对手只能吃一个，对手吃过以后是偶数，所以自己可以吃一半。所以最优策略是如果吃过一半后是偶数，选择吃一个，这样对手也只能吃一个，如果吃一半后是奇数，这时自己得到一半的包子，而对手只能选择吃一个，如果剩4个包子是例外（这时按照上述策略不是最优）。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int t, n;
4 |     scanf("%d", &t);
5 |     while (t--) {
6 |         scanf("%d", &n);
7 |         int ans = 0, now = 1; // now用来记录目前是哪个人在吃包子, 1即为自己0则是对手
8 |         while (n > 0) {
9 |             int cnt = 0;
10 |             if (!(n & 1) && n / 2 % 2 == 1) {
11 |                 n >>= 1;
12 |                 cnt = n;
13 |             } else if (n == 4) {
14 |                 n = 1;
15 |                 cnt = 3;
16 |             } else {
17 |                 cnt = 1;
18 |                 n--;
19 |             }
20 |             ans += cnt * now;
21 |             now = !now;
22 |         }
23 |         printf("%d\n", ans);
24 |     }
25 |     return 0;
26 | }
```

## 公开样例

样例 1 输入

```
2
5
6
```

样例 1 输出

```
2
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.62 | 总 429/526 糖果之“战” ( PID 1837 )

出处：2023级HAUT新生周赛（三）@22级学姐专场（杨焱，刘振歌，周欣滢）（A，CID 1088）| 训练定位：进阶 | 贪心与博弈

标签：贪心、最大值与次大值、构造判定 | 限制：1.000 S / 128 MB | 历史统计：11/93 ( 11.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入， $t$ 行，每一行都包含输入的相应测试用例的答案。如果可以按计划拿糖果，输出“YES”，否则输出“NO”。

输入要点：输入第一行包含一个整数 $t(1 \leq t \leq 10^4)$ ，输入测试用例的数量。每组数据第一行包含一个整数 $n(1 \leq n \leq 2 \cdot 10^5)$ ，糖的种类数，第二行为 $n$ 个数 $a(1 \leq a \leq 10^9)$

输出要点： $t$ 行，每一行都包含输入的相应测试用例的答案。如果可以按计划拿糖果，输出“YES”，否则输出“NO”。

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：题目强制每次从“当前数量最多”的种类中拿一颗。若最大数量比第二大数量多至少 2，第一次拿最大类后它仍严格最多，下一次只能拿同类，立即违反要求；反之最大值至多比次大值多 1，可以始终在并列最大类之间交替。把不存在的第二类视为数量 0。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O(\Sigma n)$  时间、 $O(1)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
11 |         long long largest = 0, sec = 0;
12 |         for (int i = 0; i < n; ++i) {
```

```
13 |         long long count;
14 |         cin >> count;
15 |         if (count >= largest) {
16 |             sec = largest;
17 |             largest = count;
18 |         } else if (count > sec) {
19 |             sec = count;
20 |         }
21 |     }
22 |     cout << (largest <= sec + 1 ? "YES" : "NO") << '\n';
23 | }
24 | return 0;
25 | }
```

公开样例

样例 1 输入

```
6
2
2 3
1
2
5
1 6 2 4 3
4
2 2 2 1
3
1 1000000000 999999999
1
1
```

样例 1 输出

```
YES
NO
NO
YES
YES
YES
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

6.63 | 总 430/526 尖塔第四强的糕手 ( PID 1896 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭钰 ( J , CID 1095 ) | 训练定位：进阶 | 贪心与博弈

标签：条件分支、博弈论 | 限制：1.000 S / 128 MB | 历史统计：12/75 ( 16.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：现在需要你计算在  $t$  个回合内，故障机器人能否战胜boss，如果可以，请输入最少需要的回合数，否则输出-1；

输入要点：输入分为三行

第一行为一个整数  $t$ ，表示  $t$  个回合 (  $1 \leq t \leq 100$  )

第二行为四个实数  $n, m, k, q$ ，(  $1 \leq n, m, k, q \leq 1e5$  ) 分别表示故障机器人的血量，BOSS的血量，BOSS的攻击力以及BOSS攻击力的成长值。

第三行为两个实数  $s, d$ ，(  $1 \leq s, d \leq 5000$  ) 分别为古老的光明之魂与古老的黑暗之魂的数值

输出要点：输出只有一行

如果可以击败BOSS，输出最少需要的回合数

否则输出 -1

题面提示：注意数据类型

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 先确定终止态和每步改变的量，再判断奇偶、不变量或必胜/必败状态。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：尖塔第四强的糕手 模拟即可，注意参数有可能是浮点型
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

常数级或一次线性读入；以代码中的循环层数为准

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int t;
4 | double n, m, k, q; // 实数 -> double型
5 | double s, d;
6 | // 判断t个回合中是否会被boss打死
7 | bool if_alive(int res) {
8 |     // t个回合，每个回合boss的攻击力上升q，
9 |     // 则t个回合boss的总伤害可以用等差数列求和公式计算
10 |    if (n <= (k + (k + q * 1.0 * res)) * 1.0 * res / 2.0)
11 |        return false;
12 |    else
13 |        return true;
14 | }
15 | int main() {
16 |    cin >> t;
17 |    cin >> n >> m >> k >> q;
18 |    cin >> s >> d;
19 |    double res = 0.0;
20 |    // 判断使用哪种攻击方式能在最少的回合内击败boss
21 |    if (2.0 * s >= d)
22 |        res = 1.0 * m / s / 2.0;
23 |    else
24 |        res = 1.0 * m / d;
25 |    // 如果res是小数，向上取整才能使得boss生命值小于等于0
26 |    // 由于是我们先手进攻，所以不用在boss伤害处进行修改
27 |    if (res - (int)res > 0) {
28 |        res = (int)(res + 1);
29 |    }
30 |    // 如果会被boss击杀或击杀boss的时间超出t回合
31 |    if (!if_alive(res) || res > t)
32 |        res = -1;
33 |    cout << res << endl;
34 |    return 0;
35 | }
```

## 公开样例

样例 1 输入

```
10
40 100 10 1
15 40
```

样例 1 输出

```
3
```

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 6.64 | 总 431/526 Two movies影评 ( PID 1905 )

出处：河南工大2024新生周赛（6）——命题人：魏方（H，CID 1097） | 训练定位：进阶 | 贪心与博弈

标签：贪心、枚举分配、分类讨论 | 限制：1.000 S / 128 MB | 历史统计：5/19 ( 26.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：公司的评分是两部电影评分的最小值。您的任务是计算公司可能获得的最高评分。

输入要点：第一行包含一个整数 $t$  ( $1 \leq t \leq 104$ ) - 测试用例数。

每个测试用例的第一行包含一个整数 $n$  ( $1 \leq n \leq 2 \cdot 105$ )。

第二行包含  $n$  个整数  $a_1, a_2, \dots, a_n$  ( $-1 \leq a_i \leq 1$ )，其中如果  $a_i$  等于  $-1$ ，第一部电影被  $i$ -th 观众不喜欢；等于  $1$ ，如果第一部电影被喜欢； $0$ ，如果态度中立。

第三行包含  $n$  个整数  $b_1, b_2, \dots, b_n$  ( $-1 \leq b_i \leq 1$ )，其中如果  $b_i$  等于  $-1$ ，第二部电影被  $i$ -th 个观众不喜欢；等于  $1$  如果第二部电影被喜欢； $0$ ，如果态度中立。

输入的附加限制：所有测试案例的  $n$ -之和不超过  $2 \cdot 105$ 。

输出要点：为每个测试用例打印一个整数--公司可能获得的最高评分，如果为每个人选择哪部电影留下评论

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：先处理两部电影评价不同的位置：把评价为  $1$  的那一部对应得分加一；此时得到基础分  $x, y$ 。评价都为  $-1$  的位置有  $neg$  个，都为  $1$  的位置有  $pos$  个。枚举把多少个这类“共同评价”分给第一部电影，第二部得到 剩余部分，取  $\min(x, y)$  的最大值。 $0$  对答案没有影响。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。



- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(\sum n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        vector<int> first(n), second(n);
12 |        for (int& value : first)
13 |            cin >> value;
14 |        for (int& value : second)
15 |            cin >> value;
16 |        int x = 0, y = 0, positive = 0, negative = 0;
17 |        for (int i = 0; i < n; ++i) {
18 |            if (first[i] > second[i])
19 |                x += first[i];
20 |            else if (second[i] > first[i])
21 |                y += second[i];
22 |            else if (first[i] == 1)
23 |                ++positive;
24 |            else if (first[i] == -1)
25 |                ++negative;
26 |        }
27 |        int ans = INT_MIN;
28 |        for (int assigned = -negative; assigned <= positive; ++assigned) {
29 |            int other = positive - negative - assigned;
30 |            ans = max(ans, min(x + assigned, y + other));
31 |        }
32 |        cout << ans << '\n';
33 |    }
34 |    return 0;
35 | }
```

## 公开样例

### 样例 1 输入

```
4
2
-1 1
-1 -1
1
-1
-1
5
0 -1 1 0 1
-1 1 0 0 1
4
-1 -1 -1 1
-1 1 1 1
```

### 样例 1 输出

```
0
-1
1
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1

6.65 | 总 432/526 梦应归于何处 ( PID 1950 )

出处：河南工大2025新生周赛（4）——命题人：张梦淇（B，CID 1105） | 训练定位：进阶 | 贪心与博弈

标签：贪心、数组与序列、区间 | 限制：1.000 S / 128 MB | 历史统计：64/504（12.7%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：已知dream到阿斯德纳星系的路径可视为一条长为L的轴，n个传送站分散在数轴上(可能有多个传送站在相同位置).dream的初始位置为0(可视为第0个传送站),请判断他最多能到达第几个传送站.

输入要点：第1行为一个非负整数n,表示共n个传送站, $n \leq 1e4$

第2行有n个整数的 $a_i$ ,分别表示第i个传送站所在数轴的位置, $a_i \leq 1e9$ (题目保证单调且不降)

第3行有n个整数的 $b_i$ ,分别表示第i个传送站的可以传送的有效距离, $b_i \leq 1e9$

输出要点：输出他最远能到达的传送站,若他无法到达任何传送站,输出0.

题面提示：提示：dream无法走出传送站。

零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：把当前位置记为 current。第 i 个站的有效范围是  $[a_i - b_i, a_i + b_i]$ ；若  $current \geq a_i - b_i$ ，由于站点位置不降且只能由前向后，就能进入该站并把 current 更新为  $a_i$ 。第一次不满足时，后续站也无法跳过到达，直接结束。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

复杂度

$O(n)$  时间、 $O(n)$  空间

易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
```

```

5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> pos(n), radius(n);
9 |     for (long long& x : pos)
10 |         cin >> x;
11 |     for (long long& x : radius)
12 |         cin >> x;
13 |     long long cur = 0;
14 |     int ans = 0;
15 |     for (int i = 0; i < n; ++i) {
16 |         if (pos[i] - radius[i] > cur)
17 |             break;
18 |         cur = pos[i];
19 |         ans = i + 1;
20 |     }
21 |     cout << ans << '\n';
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

5
3 7 12 18 25
5 3 6 5 4

```

样例 1 输出

```

1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 6.66 | 总 433/526 幽蝶能留一缕芳 ( PID 1953 )

出处：河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇 ( F , CID 1105 ) | 训练定位：进阶 | 贪心与博弈

标签：排序、贪心、枚举 | 限制：1.000 S / 128 MB | 历史统计：9/75 ( 12.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数表示所有大阵的最大作用总和。

输入要点：对于每个测试点：

在第一行分别输入两个非负整数 $n$ 和 $m$ ，分别表示大阵种类与点位数( $n \leq 2 \times 10^3, n \leq m \leq 10^9$ )。

接下来的 $n$ 行，第 $i$ 行输入两个整数 $a_i$ 和 $b_i$ ( $1 \leq a_i, b_i \leq 1e9$ )，表示第 $i$ 个大阵旁边在有、无大阵时可以发挥的作用。

输出要点：输出一个整数表示所有大阵的最大作用总和。

## 零基础先修

- sort 默认排序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：若一个阵旁有别的阵，它贡献  $a_i$ ，否则贡献  $b_i$ 。设有  $k$  个阵属于至少含两阵的连续块，则  $k$  不能等于 1；它们至少占  $k$  个相邻点，其余  $n-k$  个孤立阵各需一个空点隔开，故需要  $k+2(n-k)$  个点。先把所有阵按孤立贡献  $b_i$  计入，再按  $a_i-b_i$  从大到小选择改为相邻状态，并枚举所有几何上可行的  $k$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long m;
8 |     cin >> n >> m;
9 |     vector<long long> diff(n);
10 |    long long isolated = 0;
11 |    for (int i = 0; i < n; ++i) {
12 |        long long a, b;
13 |        cin >> a >> b;
14 |        isolated += b;
15 |        diff[i] = a - b;
16 |    }
17 |    sort(diff.rbegin(), diff.rend());
18 |    long long ans = LLONG_MIN;
19 |    if (2LL * n - 1 <= m)
20 |        ans = isolated;
21 |    long long cur = isolated;
22 |    for (int k = 1; k <= n; ++k) {
23 |        cur += diff[k - 1];
24 |        if (k >= 2 && k + 2LL * (n - k) <= m) {
25 |            ans = max(ans, cur);
26 |        }
27 |    }
28 |    cout << ans << '\n';
29 |    return 0;
30 | }
```

## 公开样例

样例 1 输入

```
4 5
1 100
100 1
100 1
100 1
```

样例 1 输出

```
400
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

6.67 | 总 434/526 数数数数数组 ( PID 1979 )

出处：河南工大2025新生周赛 ( 8 ) ——命题人：庞贺航、高旭 ( B , CID 1110 ) | 训练定位：进阶 | 贪心与博弈

标签：贪心、奇偶性、数组 | 限制：1.000 S / 128 MB | 历史统计：66/429 ( 15.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：请找出所需要的最少操作次数。

输入要点：每个测试包含一个测试用例。

每个测试用例包含两行：

第一行是整数  $n$  (  $1 \leq n \leq 1000$  )，表示数组长度。

第二行包含  $n$  个整数  $a_1$  到  $a_n$ ，(  $-1e18 \leq a_i \leq 1e18$  )。

输出要点：对于每个测试用例，输出一个整数，表示使数组中所有元素的乘积严格为正的最小操作次数。

零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 只关心奇偶时可把数化为  $x \% 2$ ；和的奇偶等于所有项奇偶的异或。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：每个 0 至少加一次才能使乘积非零，先把这些代价计入答案。若负数 个数为偶数，非零乘积已经为正；若为奇数，必须把一个负数一路加到 +1，选最接近 0 的负数代价最小，代价为  $1-x$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

复杂度

$O(n)$  时间、 $O(1)$  空间

易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
```

```

3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, negCnt = 0;
7 |     long long ans = 0, neg = LLONG_MIN;
8 |     cin >> n;
9 |     for (int i = 0; i < n; ++i) {
10 |         long long value;
11 |         cin >> value;
12 |         if (value == 0)
13 |             ++ans;
14 |         else if (value < 0) {
15 |             ++negCnt;
16 |             neg = max(neg, value);
17 |         }
18 |     }
19 |     if (negCnt % 2 == 1)
20 |         ans += 1 - neg;
21 |     cout << ans << '\n';
22 |     return 0;
23 | }

```

## 公开样例

样例 1 输入

```

3
-1 0 1

```

样例 1 输出

```

3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“贪心与博弈”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 7 | 数据结构

当普通数组无法高效维护动态信息时，选择栈、队列、集合、哈希、堆、链表或线段树。重点是明确每个结构保存什么以及操作复杂度。

本模块共 31 道唯一题。

## 7.1 | 总 435/526 愉快序列 ( PID 1431 )

出处：2018级河南工大新生周赛 ( 八 ) @李维航专场 ( D , CID 1047 ) | 训练定位：入门 | 数据结构

标签：双指针与滑动窗口、哈希与集合、最长区间 | 限制：1.000 S / 128 MB | 历史统计：0/0 ( 无公开统计 )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**根据给定输入，对于一个序列，输出它最长的"愉快序列"的长度。

**输入要点：**多样例测试

第一行输入一个T表示样例数( $1 \leq T \leq 200$ )

对于每个样例：

第一行输入一个整数n ( $1 \leq n \leq 10^5$ )

第二行输入n个整数 $a_i$  ( $0 \leq a_i \leq 10^5$ )

输出要点：对于一个序列，输出它最长的"愉快序列"的长度。

## 零基础先修

- 窗口维护一段连续区间；右端扩展、左端收缩时同步维护统计量。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：滑动窗口维护区间内元素互不重复。记录每个值上次出现位置；读到  $a[\text{right}]$  时，把 left 推到  $\max(\text{left}, \text{last}[a]+1)$ ，于是窗口重新合法，再用  $\text{right}-\text{left}+1$  更新答案。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

每组  $O(n + \text{值域初始化})$  时间、 $O(\text{值域})$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, left = 0, ans = 0;
10 |        cin >> n;
11 |        vector<int> last(100001, -1);
12 |        for (int right = 0; right < n; ++right) {
13 |            int value;
14 |            cin >> value;
15 |            left = max(left, last[value] + 1);
16 |            last[value] = right;
17 |            ans = max(ans, right - left + 1);
18 |        }
19 |        cout << ans << '\n';
20 |    }
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
3
1
1
3
1 3 1
5
4 5 6 7 4
```

样例 1 输出

```
1
2
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.2 | 总 436/526 最大增区间 (二) (PID 1439)

出处：2018级河南工大新生周赛（七）@牛寅旭专场（C，CID 1046）| 训练定位：入门 | 数据结构

标签：线段树、枚举、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：0/0（无公开统计）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：求最大增区间的长度。

输入要点：第一行一个数字  $n$ 。

第二行为  $n$  个数。

输出要点：输出一行，最大增区间的长度。

题面提示：样例中调换 6,4 的位置。

增区间中数字可以相等

### 零基础先修

- 线段树把区间递归拆分，支持  $O(\log n)$  单点修改和区间查询；先定义节点维护量。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
3. 核心处理采用：把相邻关系  $a[i] \leq a[i+1]$  记为 1，最长非递减段长度就是最长连续 1 的数量加 1。交换  $i$ 、 $j$  只会改变至多四条相邻边；用线段树维护 01 串的最长连续 1，枚举  $O(n^2)$  对交换时只点改这些边、读取根答案，再回滚。这样保留了题解的“枚举交换”，但避免  $O(n^3)$  重扫。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

### 复杂度

$O(n^2 \log n)$  时间、 $O(n)$  空间

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。



- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | struct Node {
4 |     int len = 0, prefix = 0, suffix = 0, best = 0;
5 | };
6 | Node mergeNode(const Node& left, const Node& right) {
7 |     Node res;
8 |     res.len = left.len + right.len;
9 |     res.prefix = left.prefix == left.len ? left.len + right.prefix : left.prefix;
10 |    res.suffix = right.suffix == right.len ? right.len + left.suffix : right.suffix;
11 |    res.best = max({left.best, right.best, left.suffix + right.prefix});
12 |    return res;
13 | }
14 | struct SegmentTree {
15 |     int size = 1;
16 |     vector<Node> tree;
17 |     SegmentTree(const vector<int>& value) {
18 |         while (size < (int)value.size())
19 |             size <<= 1;
20 |         tree.resize(2 * size);
21 |         for (int i = 0; i < size; ++i) {
22 |             int bit = i < (int)value.size() ? value[i] : 0;
23 |             tree[size + i] = {1, bit, bit, bit};
24 |         }
25 |         for (int i = size - 1; i; --i)
26 |             tree[i] = mergeNode(tree[i << 1], tree[i << 1 | 1]);
27 |     }
28 |     void update(int pos, int bit) {
29 |         int node = size + pos;
30 |         tree[node] = {1, bit, bit, bit};
31 |         for (node >= 1; node; node >>= 1)
32 |             tree[node] = mergeNode(tree[node << 1], tree[node << 1 | 1]);
33 |     }
34 |     int best() const {
35 |         return tree[1].best;
36 |     }
37 | };
38 | int main() {
39 |     ios::sync_with_stdio(false);
40 |     cin.tie(nullptr);
41 |     int n;
42 |     cin >> n;
43 |     vector<long long> a(n);
44 |     for (long long& x : a)
45 |         cin >> x;
46 |     if (n == 1) {
47 |         cout << 1 << '\n';
48 |         return 0;
49 |     }
50 |     vector<int> good(n - 1);
51 |     for (int i = 0; i + 1 < n; ++i)
52 |         good[i] = a[i] <= a[i + 1];
53 |     SegmentTree tree(good);
54 |     int ans = tree.best() + 1;
55 |     for (int i = 0; i < n; ++i) {
56 |         for (int j = i + 1; j < n; ++j) {
57 |             vector<int> edges = {i - 1, i, j - 1, j};
58 |             sort(edges.begin(), edges.end());
59 |             edges.erase(unique(edges.begin(), edges.end()), edges.end());
60 |             swap(a[i], a[j]);
61 |             for (int edge : edges) {
62 |                 if (edge >= 0 && edge + 1 < n)
63 |                     tree.update(edge, a[edge] <= a[edge + 1]);
64 |             }
65 |             ans = max(ans, tree.best() + 1);
66 |             swap(a[i], a[j]);
67 |             for (int edge : edges) {
68 |                 if (edge >= 0 && edge + 1 < n)
69 |                     tree.update(edge, good[edge]);
70 |             }
71 |         }
72 |     }
73 |     cout << ans << '\n';
74 |     return 0;
75 | }
```

## 公开样例

### 样例 1 输入

```
10
15364891573
```

样例 1 输出

5

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.3 | 总 437/526 区间和 ( PID 1551 )

出处：2020级HAUT新生周赛（三）@张雍蕤专场（E，CID 1062） | 训练定位：基础提高 | 数据结构

标签：前缀和与差分、鸽巢原理、哈希与集合 | 限制：1.000 S / 128 MB | 历史统计：15/66 ( 22.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定一个长度为 $n$ 的序列 $a$ 试求出 $a$ 的一段非空子区间,使得区间内所有元素的和为 $n$ 的整数倍

输入要点：第一行为 $n$ ，代表序列的长度

接下来一行 $n$ 个数，代表 $a[i]$ 的值

$1 \leq n \leq 5000$

$1 \leq a[i] \leq 5000$

输出要点：如果存在这样的子区间，输出它的左右端点(下标从1开始)

如果存在多个满足条件的区间，输出字典序最小的

如果不存在满足条件的区间，输出-1

题面提示：样例输入2：

3

3 3 3

样例输出2：

1 1

样例输入3：

6

3 3 3 3 3 3

样例输出3：

1 2

### 零基础先修

- 前缀和让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。

- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：令  $\text{prefix}[i]$  为前  $i$  项和对  $n$  的余数。区间  $[l, r]$  的和能被  $n$  整除，当且仅当  $\text{prefix}[l-1] = \text{prefix}[r]$ 。记录每个余数首次出现的前缀下标；再次出现时就得到以该首次位置为左端的最早候选，再按  $(l, r)$  比较即可得到字典序最小答案。 $n+1$  个前缀余数落入  $n$  类，所以按题目范围一定存在重复。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> first(n, -1);
9 |     first[0] = 0;
10 |     long long prefix = 0;
11 |     pair<int, int> best = {n + 1, n + 1};
12 |     for (int i = 1; i <= n; ++i) {
13 |         long long x;
14 |         cin >> x;
15 |         prefix = (prefix + x) % n;
16 |         int residue = static_cast<int>(prefix);
17 |         if (first[residue] == -1)
18 |             first[residue] = i;
19 |         else
20 |             best = min(best, make_pair(first[residue] + 1, i));
21 |     }
22 |     if (best.first == n + 1)
23 |         cout << -1 << '\n';
24 |     else
25 |         cout << best.first << ' ' << best.second << '\n';
26 |     return 0;
27 | }
```

## 公开样例

样例 1 输入

```
3
1 2 3
```

样例 1 输出

```
1 2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 7.4 | 总 438/526 新建 Microsoft PowerPoint 演示文稿.pptx ( PID 1560 )

出处：2020级HAUT新生周赛（四）@张承树专场（F，CID 1063）| 训练定位：基础提高 | 数据结构

标签：模拟、集合、字符串格式 | 限制：1.000 S / 128 MB | 历史统计：14/60 ( 23.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：输入2：删除一个编号为  $x$  的文档，反馈文档是否删除成功

输入要点：第一行一个  $n$  代表操作次数

接下来  $n$  行。

如果该行第一个数字是1，则表示为新建。

如果为2，则表示删除，后面再跟一个数  $x$ ，表示删除一个编号为  $x$  的文档。

输出要点：如果操作是1，输出新建文档的名称。

如果操作是2 如果存在这个文件，则删除成功，如果不存在，则删除失败

成功输出"Successful"，失败输出"Failed"

题面提示： $1 < n < 1000$

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- 集合用于去重和成员查询；输出有序结果时优先使用 set 或最后排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：用布尔数组记录编号是否存在。新建时从 1 起寻找最小未占用编号；编号 1 使用无括号的基础文件名，编号大于 1 才加  $(x)$ 。删除时按当前占用状态分别反馈 Successful 或 Failed。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

最坏  $O(n^2)$  时间、 $O(n)$  空间； $n < 1000$

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
```

```

3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<bool> used(n + 2, false);
9 |     while (n--) {
10 |         int type;
11 |         cin >> type;
12 |         if (type == 1) {
13 |             int id = 1;
14 |             while (used[id])
15 |                 ++id;
16 |             used[id] = true;
17 |             cout << "新建 Microsoft PowerPoint 演示文稿";
18 |             if (id > 1)
19 |                 cout << '(' << id << ')';
20 |             cout << ".pptx\n";
21 |         } else {
22 |             int id;
23 |             cin >> id;
24 |             if (id < (int)used.size() && used[id]) {
25 |                 used[id] = false;
26 |                 cout << "Successful\n";
27 |             } else {
28 |                 cout << "Failed\n";
29 |             }
30 |         }
31 |     }
32 |     return 0;
33 | }

```

## 公开样例

### 样例 1 输入

```

5
1
1
1
2 2
1

```

### 样例 1 输出

```

新建 Microsoft PowerPoint 演示文稿.pptx
新建 Microsoft PowerPoint 演示文稿(2).pptx
新建 Microsoft PowerPoint 演示文稿(3).pptx
Successful
新建 Microsoft PowerPoint 演示文稿(2).pptx

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.5 | 总 439/526 奇迹魔法都是存在的 (PID 1737)

出处：2021级HAUT新生周赛（八）@孔维斌专场（D，CID 1076）| 训练定位：基础提高 | 数据结构

标签：数组与序列、字符串、哈希与集合 | 限制：1.000 S / 128 MB | 历史统计：6/26 (23.1%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：例如{1, 2, 3}的子序列是{1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3}，至于空集合由于元素异或值是0所以不影响结果，无论是否算上空集合。

输入要点：第一行一个整数n

第二行n个整数,s1,s2...,si,...,sn 代表小圆面临的n个魔法值。

数据范围  $1 \leq n \leq 11, 1 \leq s_i \leq 1e4$

输出要点：一个整数，集合所有子序列的异或的和

题面提示：样例解释

样例子序列有  $\{1\}, \{2\}, \{3\}, \{1, 2\}, \{1, 3\}, \{2, 3\}, \{1, 2, 3\}$ ,

每个子序列的异或值分布为  $1, 2, 3, 3, 2, 1, 0$ ，所以答案就是  $1 + 2 + 3 + 3 + 2 + 1 = 12$ 。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：奇迹魔法都是存在的(传送门)题目大意，得到这个集合的所有子序列的异或的和 由于这个是数据比较弱，那么  $n$  是小于等于 11 的，那么我们可以自然的想到枚举所有的子序列，然后暴力求出答案即可。那么如何进行枚举所有的集合，可以使用二进制枚举的方式，或者使用 dfs 的方式来枚举（就是每个是选是或者否）。二进制枚举就算由于集合中每个数只有选或者不选两种状态，发现和二进制很想，那么可以使用二进制的 0 代表不选，1 代表选，那么任意一个子序列都可以以一个二进制数来表示，而且这个二进制数是小于  $2^n$ ，因为只需要把  $n$  个数在  $0 \sim n-1$  位表示即可，那么我们就可以使用一个循环来枚举所有的子序列。 $2^n$  时间复杂度: 参考代码  $O(2^n)O(2n)O(2n)$
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(2^n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[110];
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     for (int i = 0; i < n; i++)
7 |         scanf("%d", &a[i]); // 读入所有的数
8 |     int ans = 0;
9 |     for (int i = 0; i < (1 << n); i++) { // 进行二进制枚举
10 |         int x = 0;
11 |         for (int j = 0; j < n; j++) { // 枚举n位选择情况，i就代表当前的子序列选择情况
12 |             if (i >> j & 1) { // 如果i的第j位是1代表选第j个元素
13 |                 x ^= a[j];
14 |             }
15 |         }
16 |         ans += x; // 统计答案即可
17 |     }
18 |     printf("%d\n", ans);
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
3
1 2 3
```

样例 1 输出

```
12
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 7.6 | 总 440/526 万分之一的光(easy version) ( PID 1787 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（B，CID 1083）| 训练定位：基础提高 | 数据结构

标签：栈与队列 | 限制：1.000 S / 128 MB | 历史统计：17/36 ( 47.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，亮着的路灯的数量。

输入要点：第一行两个整数  $n, k$ ，表示网格边长和亮着灯的大楼的数量。

接下来  $k$  行每行两个整数  $x, y$ ，表示大楼的坐标。

$(1 \leq n, k \leq 103)$

输出要点：一个整数，亮着的路灯的数量。

### 零基础先修

- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：万分之一的光(easy version) 题意：计算给出所有方格之间相邻的边的数量乘以4。解法：因为是easy version所以解法有很多，可以每两个方格都对比一下，看是否有边相邻，也可以使用hard version的方法（下面再讲），但使用简单的排序算法。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$ ；若循环嵌套或迭代范围不同，应按代码重新核算

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```

1 | #include <stdio.h>
2 | // 大楼结构体，包括两个成员记录纵横坐标
3 | struct build {
4 |     int x, y;
5 | } a[1005];
6 | int main() {
7 |     int n, k;
8 |     // 读入数据
9 |     scanf("%d%d", &n, &k);
10 |    for (int i = 0; i < k; i++)
11 |        scanf("%d%d", &a[i].x, &a[i].y);
12 |    // ans记录相邻的边的数量
13 |    int ans = 0;
14 |    // 按照先横再纵的方式排序
15 |    for (int i = 0; i < k; i++)
16 |        for (int j = i + 1; j < k; j++)
17 |            if (a[i].x != a[j].x) {
18 |                if (a[i].x > a[j].x) {
19 |                    struct build temp = a[j];
20 |                    a[j] = a[i];
21 |                    a[i] = temp;
22 |                }
23 |            } else {
24 |                if (a[i].y > a[j].y) {
25 |                    struct build temp = a[j];
26 |                    a[j] = a[i];
27 |                    a[i] = temp;
28 |                }
29 |            }
30 |    // 如果两栋楼横坐标相同且纵坐标相差1(说明两建筑相邻)，答案+1
31 |    for (int i = 1; i < k; i++)
32 |        if ((a[i].y == a[i - 1].y + 1 && a[i].x == a[i - 1].x))
33 |            ans++;
34 |    // 先纵后横，与上面同理
35 |    for (int i = 0; i < k; i++)
36 |        for (int j = i + 1; j < k; j++)
37 |            if (a[i].y != a[j].y) {
38 |                if (a[i].y > a[j].y) {
39 |                    struct build temp = a[j];
40 |                    a[j] = a[i];
41 |                    a[i] = temp;
42 |                }
43 |            } else {
44 |                if (a[i].x > a[j].x) {
45 |                    struct build temp = a[j];
46 |                    a[j] = a[i];
47 |                    a[i] = temp;
48 |                }
49 |            }
50 |    for (int i = 1; i < k; i++)
51 |        if ((a[i].x == a[i - 1].x + 1 && a[i].y == a[i - 1].y))
52 |            ans++;
53 |    // 输出答案
54 |    printf("%d", ans * 4);
55 | }

```

## 公开样例

### 样例 1 输入

```

6 12
1 1
2 1
2 2
1 4
3 3
4 3
4 4
3 4
3 6
4 6
5 6
6 6

```

### 样例 1 输出

```

36

```

## 训练动作



- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.7 | 总 441/526 冥界的人员统计 ( PID 1963 )

出处：河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇 ( G , CID 1105 ) | 训练定位：基础提高 | 数据结构

标签：哈希与集合、字符串、模拟 | 限制：1.000 S / 128 MB | 历史统计：49/113 ( 43.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**新的逝者注册时，他将向系统发送一则内容为其名称的请求，如果该名称尚未存在于系统数据库内，则将该名称插入数据库，同时得到回应信息"OK"表示其已经成功登记。如果他请求的用户名已经存在于数据库内，那么系统将产生一个新的名称并将其加入数据库。新名称由逝者请求的名称与正整数*i*构成，*i*为使"名称"尚未存在于数据库内的最小的*i*。

**输入要点：**第一行一个整数  $n(1 \leq n \leq 103)$ 。接下来  $n$  行，每行表示逝者向系统发出的一则请求。每行内容均非空且均为由至多 32 个小写拉丁字母组成的字符串。

**输出要点：**共  $n$  行，每行表示系统对一则请求做出的回应。如果该名称尚未存在于系统数据库内，则输出 OK。如果用户请求的用户名已经被注册，则输出依照规则生成的新名。

### 零基础先修

- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：哈希表 `count[name]` 既表示名字已注册，也保存下次应尝试的后缀。首次出现输出 OK 并把计数设为 1；再次出现输出 `name+count`，再把计数 加一。按该题的标准数据，系统生成名不会与未来原始名产生冲突；若要 做通用版，可把生成名也写入表中。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

期望  $O(\text{总字符数})$  时间、 $O(n)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     unordered_map<string, int> count;
9 |     while (n--) {
10 |         string name;
11 |         cin >> name;
12 |         if (!count.count(name)) {
13 |             cout << "OK\n";
14 |             count[name] = 1;
15 |         } else {
16 |             string created = name + to_string(count[name]++);
17 |             while (count.count(created)) {
18 |                 created = name + to_string(count[name]++);
19 |             }
20 |             cout << created << '\n';
21 |             count[created] = 1;
22 |         }
23 |     }
24 |     return 0;
25 | }

```

## 公开样例

样例 1 输入

```

4
abacaba
acaba
abacaba
acab

```

样例 1 输出

```

OK
OK
abacaba1
OK

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.8 | 总 442/526 Simple学长整理咖啡 ( PID 1239 )

出处：2016级河南工大新生周赛（六）（D，CID 1012） | 训练定位：进阶 | 数据结构

标签：优先队列、Huffman、贪心 | 限制：1.000 S / 128 MB | 历史统计：3/62 ( 4.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：前面说到Simple学长为了炫耀他的咖啡，把他的咖啡都放到箱子里摆成一行。然而寝室的空间是有限的，他的室友现在警告他了，赶快把这些箱子都处理掉，不然就把这些箱子扔出去或者把Simple学长扔出去。可怜的Simple学长现在要把所有的咖啡都合并到一个箱子里，合并两个咖啡数为a和b的两个箱子需要花费a+b的体力，Simple学长整理完这些咖啡还要留一些体力去敲代……

输入要点：输入一个T，表示有T组数据 (  $T \leq 10$  )

每组数据一个行有一个数n，表示有n个箱子 (  $n \leq 1000$  )

第二行有n个数， $a_i$ 表示第i个箱子里有 $a_i$ 杯咖啡( $a_i \leq 100$ )

输出要点：输出一个数，表示最少需要花费的体力。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：每次合并两个箱子的代价为咖啡数之和，新箱子的咖啡数也等于该和。为避免大箱子反复参与产生高代价，应每次取当前最小的两个箱子合并，把新箱子放回。这是 Huffman 合并模型，用小根堆可在  $O(n \log n)$  内完成；交换论证说明任一最优合并树最深处可以安排两个最小权值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        priority_queue<long long, vector<long long>, greater<long long>> heap;
12 |        for (int i = 0; i < n; ++i) {
13 |            long long count;
14 |            cin >> count;
15 |            heap.push(count);
16 |        }
17 |        long long ans = 0;
18 |        while (heap.size() > 1) {
19 |            long long a = heap.top();
20 |            heap.pop();
21 |            long long b = heap.top();
22 |            heap.pop();
23 |            ans += a + b;
24 |            heap.push(a + b);
25 |        }
26 |        cout << ans << '\n';
27 |    }
28 |    return 0;
29 | }
```

## 公开样例

样例 1 输入

```
2
3
8 5 8
3
1 1 1
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.9 | 总 443/526 最小的数 ( PID 1344 )

出处：2017级河南工大新生周赛 ( 六 ) ( D , CID 1035 ) | 训练定位：进阶 | 数据结构

标签：贪心、集合、分类讨论 | 限制：1.000 S / 256 MB | 历史统计：20/160 ( 12.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给你两个一维数组（都为大于等于1且小于等于9的数），从第一个数组中取至少一个数字，再从第二个数组中取至少一个数字，用你选取的数字组成一个整数，求能组成的最小整数。

输入要点：第一行：一个整数T，表示测试实例个数

对于每组测试实例：

第一行：包含两个整数n 和 m ( $1 \leq n, m \leq 9$ ) —— 分别表示两个数组的大小

第二行：包含n个整数a1, a2, ..., an ( $1 \leq a_i \leq 9$ ) —— 第一个数组

第三行：包含m个整数 b1, b2, ..., bm ( $1 \leq b_i \leq 9$ ) 第二个数组

输出要点：每组测试实例输出一行：包含一个整数 —— 能组成的最小整数

### 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 集合用于去重和成员查询；输出有序结果时优先使用 set 或最后排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：若两个数组有公共数字 d，可以从两边都选 d 且只保留一个，最小答案 就是最小公共数字（一个一位数一定优于任何两位数）。若没有公共数字，必须各选一个；显然应选两数组各自最小值，并把较小者放十位，组成  $10 \cdot \min(x,y) + \max(x,y)$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

每组  $O(n+m)$  时间、 $O(n+m)$  空间

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n, m;
10 |         cin >> n >> m;
11 |         vector<int> a(n), b(m);
12 |         array<bool, 10> inA{}, inB{};
13 |         for (int& x : a) {
14 |             cin >> x;
15 |             inA[x] = true;
16 |         }
17 |         for (int& x : b) {
18 |             cin >> x;
19 |             inB[x] = true;
20 |         }
21 |         int common = -1;
22 |         for (int digit = 1; digit <= 9; ++digit)
23 |             if (inA[digit] && inB[digit]) {
24 |                 common = digit;
25 |                 break;
26 |             }
27 |         if (common != -1) {
28 |             cout << common << '\n';
29 |         } else {
30 |             int x = *min_element(a.begin(), a.end());
31 |             int y = *min_element(b.begin(), b.end());
32 |             cout << min(x, y) * 10 + max(x, y) << '\n';
33 |         }
34 |     }
35 |     return 0;
36 | }
```

## 公开样例

样例 1 输入

```
2
23
42
576
88
12345678
87654321
```

样例 1 输出

```
25
1
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.10 | 总 444/526 男欢女爱 ( PID 1480 )

出处：2019级河南工大新生周赛（二）@邹岩专场（A，CID 1054）| 训练定位：进阶 | 数据结构

标签：字符串、哈希与集合、计数 | 限制：1.000 S / 128 MB | 历史统计：94/504（18.7%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，如果检查为女性，请打印"CHAT WITH HER!"（不带引号），否则，打印"IGNORE HIM!"。（不带引号）。

输入要点：第一行包含一个非空字符串，该字符串仅包含小写英文字母-用户名。此字符串最多包含100个字母。

输出要点：如果检查为女性，请打印"CHAT WITH HER!"（不带引号），否则，打印"IGNORE HIM!"。（不带引号）。

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：统计用户名中不同小写字母的数量；偶数按题意判为女性，奇数判为 男性。26 位布尔数组即可去重。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(|name|)$  时间、 $O(1)$  空间

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     string name;
5 |     cin >> name;
6 |     array<bool, 26> exists{};
7 |     for (char c : name)
8 |         exists[c - 'a'] = true;
9 |     int distinct = count(exists.begin(), exists.end(), true);
10 |     cout << (distinct % 2 == 0 ? "CHAT WITH HER!" : "IGNORE HIM!") << '\n';
11 |     return 0;
12 | }
```

### 公开样例

样例 1 输入

wjmbzmr

样例 1 输出

CHAT WITH HER!

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.11 | 总 445/526 zp与火系道馆 ( 对小19的再次关怀 ) ( PID 1511 )

出处：2019级河南工大新生周赛 ( 五 ) @潘世泽专场 ( E , CID 1057 ) | 训练定位：进阶 | 数据结构

标签：构造、哈希与集合、序列还原 | 限制：1.000 S / 128 MB | 历史统计：0/5 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在给你经过若干次1, 2变换后的这n-3个四元组，请还原这个序列.如果有多个答案，请输出字典序最小的答案。

输入要点：样例数T( $1 \leq T \leq 10$ )

每个样例第一行输出一个n (  $7 \leq n \leq 200$  )。

接下来n-3个四元组。

输出要点：满足条件的序列 ( 如果有多个答案请输出字典序最小的答案 )

### 零基础先修

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：原序列端点只出现在一个四元组中，次端点出现两次，第三个位置 出现三次，内部位置出现四次。选择两个频次为 1 的数中较小者作为  $p_1$ ，可在正序与逆序中取得字典序更小者；其唯一四元组内其余三个数 按全局频次 2、3、4 排成  $p_2, p_3, p_4$ 。之后每次寻找尚未使用且包含 最近三个数的四元组，第四个数就是下一项。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

### 复杂度

每组  $O(n^2)$  时间、 $O(n)$  空间

### 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |         cin >> n;
```

```

11 |         int groups = n - 3;
12 |         vector<array<int, 4>> tuple(groups);
13 |         vector<int> freq(n + 1, 0);
14 |         for (auto& group : tuple) {
15 |             for (int& x : group) {
16 |                 cin >> x;
17 |                 ++freq[x];
18 |             }
19 |         }
20 |         int first = n + 1;
21 |         for (int value = 1; value <= n; ++value) {
22 |             if (freq[value] == 1)
23 |                 first = min(first, value);
24 |         }
25 |         int initial = -1;
26 |         for (int i = 0; i < groups; ++i) {
27 |             if (find(tuple[i].begin(), tuple[i].end(), first) != tuple[i].end()) {
28 |                 initial = i;
29 |                 break;
30 |             }
31 |         }
32 |         vector<int> ans = {first};
33 |         vector<int> rest;
34 |         for (int x : tuple[initial])
35 |             if (x != first)
36 |                 rest.push_back(x);
37 |         sort(rest.begin(), rest.end(), [&](int a, int b) {
38 |             if (freq[a] != freq[b])
39 |                 return freq[a] < freq[b];
40 |             return a < b;
41 |         });
42 |         ans.insert(ans.end(), rest.begin(), rest.end());
43 |         vector<bool> used(groups, false);
44 |         used[initial] = true;
45 |         while ((int)ans.size() < n) {
46 |             array<int, 3> last = {ans[ans.size() - 3], ans[ans.size() - 2],
47 |                                   ans[ans.size() - 1]};
48 |             for (int i = 0; i < groups; ++i) {
49 |                 if (used[i])
50 |                     continue;
51 |                 bool containsAll = true;
52 |                 for (int x : last) {
53 |                     if (find(tuple[i].begin(), tuple[i].end(), x) == tuple[i].end()) {
54 |                         containsAll = false;
55 |                     }
56 |                 }
57 |                 if (!containsAll)
58 |                     continue;
59 |                 for (int x : tuple[i]) {
60 |                     if (x != last[0] && x != last[1] && x != last[2]) {
61 |                         ans.push_back(x);
62 |                         break;
63 |                     }
64 |                 }
65 |                 used[i] = true;
66 |                 break;
67 |             }
68 |         }
69 |         for (int i = 0; i < n; ++i) {
70 |             if (i)
71 |                 cout << ' ';
72 |             cout << ans[i];
73 |         }
74 |         cout << '\n';
75 |     }
76 |     return 0;
77 | }

```

## 公开样例

### 样例 1 输入

```

1
7
2 5 4 6
5 4 6 7
4 6 7 3
1 2 4 5

```

### 样例 1 输出

```
1 2 5 4 6 7 3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。



- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.12 | 总 446/526 小青的女装生涯 ( PID 1536 )

出处：2020级HAUT新生周赛（一）@李晨龙专场（F，CID 1060）| 训练定位：进阶 | 数据结构

标签：哈希与集合、贪心、计数 | 限制：1.000 S / 128 MB | 历史统计：5/23 ( 21.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：小青上大学后发现自己逐渐喜欢上了女装，恰好小青的好朋友四火也准备尝试一下女装，双十一小青在某宝某家店购买了 $n$ 件女装，四火也在这家店购买了 $m$ 件女装，每件衣服都有自己的漂亮值 $A_i$ ，同时我们保证仅有相同的衣服漂亮值才相同。小青和四火是好兄弟，所以他们决定把衣服放在一个衣柜里，从此四火的衣服就是小青的，小青的衣服还是小青的。大学一件很令人苦恼的事就是撞衫，很不幸的是……

输入要点：第一行两个整数 $n, m$ 分别为小青和四火购买女装的数量（ $1 \leq n, m \leq 100$ ）

第二行为  $n$  个正整数，为小青购买的女装的漂亮值

第三行为  $m$  个正整数，为四火购买的女装的漂亮值

题目保证每件女装的漂亮值 $A_i$ 满足  $1 \leq A_i \leq 100$

输出要点：一个整数，表示小青至少需要换货的次数

题面提示：小青需要将两件漂亮值为1的女装邮寄过去换回一件漂亮值为 $1+1=2$ 的女装和一件漂亮值为1的女装，即可满足所有衣服各不相同

### 零基础先修

- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：若共有  $total$  件衣服、 $distinct$  种漂亮值，就有  $total - distinct$  件是重复副本。一次换货最多把一件重复衣服改成和，从而少一个重复；反过来可不断选择重复副本与足够大的衣服，使新值和继续增大且不与现值冲突，所以这个下界可以达到。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

$O((n+m) \log(n+m))$  时间、 $O(n+m)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n, m;
5 |     cin >> n >> m;
6 |     set<int> values;
7 |     for (int i = 0, x; i < n + m; ++i) {
8 |         cin >> x;
9 |         values.insert(x);
10 |     }
11 |     cout << n + m - static_cast<int>(values.size()) << '\n';
12 |     return 0;
13 | }

```

## 公开样例

样例 1 输入

```

3 2
1 1 3
5 7

```

样例 1 输出

```

1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.13 | 总 447/526 阿正的英语阅读 ( PID 1543 )

出处：2020级HAUT新生周赛（二）@杨哲专场（E，CID 1061） | 训练定位：进阶 | 数据结构

标签：字符串、哈希与集合、词频统计 | 限制：1.000 S / 128 MB | 历史统计：7/45 ( 15.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出出现次数最多的英语单词，若出现次数最多的单词不唯一，请输出出现最早的。

**输入要点：**一段符合题目要求的英语文本，单词之间以空格分隔。

**输出要点：**输出出现次数最多的英语单词，若出现次数最多的单词不唯一，请输出出现最早的。

**题面提示：**在样例中单词"Azheng"出现的次数最多，一共出现了2次。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：逐词读取并用哈希表计数。某个词的新计数严格大于当前最大值时才 更新答案；相等时不更新，便自然保留最早出现的那个词。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

期望  $O(\text{单词数})$  时间、 $O(\text{不同单词数})$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     unordered_map<string, int> count;
7 |     string word, ans;
8 |     int best = 0;
9 |     while (cin >> word) {
10 |         int now = ++count[word];
11 |         if (now > best) {
12 |             best = now;
13 |             ans = word;
14 |         }
15 |     }
16 |     cout << ans << '\n';
17 |     return 0;
18 | }
```

## 公开样例

样例 1 输入

Azheng have read tenet recently Azheng love science fiction

样例 1 输出

Azheng

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.14 | 总 448/526 替换 ( PID 1549 )

出处：2020级HAUT新生周赛 ( 三 ) @张雍蕻专场 ( C , CID 1062 ) | 训练定位：进阶 | 数据结构

标签：哈希与集合、计数、模拟 | 限制：1.000 S / 128 MB | 历史统计：4/117 ( 3.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入， $q$ 行，每行一个整数，表示每次操作后整个序列的和

输入要点：第一行两个整数 $n, q$ 代表序列长度和操作次数

第二行 $n$ 个整数,代表序列中的元素

接下来两个整数 $a, b$ , 表示将序列中的所有 $a$ 替换为 $b$

$1 \leq n, q \leq 1e5$

1 <= a,b <= 1e5

输出要点：q行，每行一个整数，表示每次操作后整个序列的和

## 零基础先修

- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：维护每个值的出现次数 count 和当前总和 sum。把所有 a 换成 b 时，总和改变量为  $\text{count}[a] \cdot (b - a)$ ，随后把  $\text{count}[a]$  合并进  $\text{count}[b]$ 。a=b 时序列不变，必须单独跳过，避免把计数错误清零。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n+q)$  时间、 $O(\text{值域})$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, q;
7 |     cin >> n >> q;
8 |     const int LIMIT = 100000;
9 |     vector<long long> count(LIMIT + 1, 0);
10 |     long long sum = 0;
11 |     for (int i = 0, x; i < n; ++i) {
12 |         cin >> x;
13 |         ++count[x];
14 |         sum += x;
15 |     }
16 |     while (q--) {
17 |         int a, b;
18 |         cin >> a >> b;
19 |         if (a != b) {
20 |             sum += count[a] * (b - a);
21 |             count[b] += count[a];
22 |             count[a] = 0;
23 |         }
24 |         cout << sum << '\n';
25 |     }
26 |     return 0;
27 | }
```

## 公开样例

样例 1 输入

```
10 10
2 3 7 9 2 6 3 9 8 3
2 6
9 1
1 3
1 8
```

```
7 10
10 1
3 1
1 3
6 8
3 4
```

样例 1 输出

```
60
44
48
48
51
42
32
44
50
56
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.15 | 总 449/526 区间和 ( 加强版 ) ( PID 1552 )

出处：2020级HAUT新生周赛（三）@张雍薛专场（F，CID 1062）| 训练定位：进阶 | 数据结构

标签：前缀和与差分、鸽巢原理、哈希与集合 | 限制：1.000 S / 128 MB | 历史统计：1/15 ( 6.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定一个长度为 $n$ 的序列 $a$ 试求出 $a$ 的一段非空子区间,使得区间内所有元素的和为 $n$ 的整数倍

输入要点：第一行为 $n$ ，代表序列的长度

接下来一行 $n$ 个数，代表 $a[i]$ 的值

$1 \leq n \leq 1e5$

$1 \leq a[i] \leq 1e6$

输出要点：如果存在这样的子区间，输出它的左右端点(下标从1开始)

如果存在多个满足条件的区间，输出字典序最小的

如果不存在满足条件的区间，输出-1

题面提示：样例输入2：

3

3 3 3

样例输出2：

1 1

样例输入3：

6

3 3 3 3 3 3

样例输出3：

1 2

## 零基础先修

- 前缀和让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。
- `set` 有序且去重，`unordered_set` 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：令 `prefix[i]` 为前  $i$  项和对  $n$  的余数。区间  $[l, r]$  的和能被  $n$  整除，当且仅当 `prefix[l-1]=prefix[r]`。记录每个余数首次出现的前缀下标；再次出现时就得到以该首次位置为左端的最早候选，再按  $(l, r)$  比较即可得到字典序最小答案。 $n+1$  个前缀余数落入  $n$  类，所以按题目范围一定存在重复。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> first(n, -1);
9 |     first[0] = 0;
10 |     long long prefix = 0;
11 |     pair<int, int> best = {n + 1, n + 1};
12 |     for (int i = 1; i <= n; ++i) {
13 |         long long x;
14 |         cin >> x;
15 |         prefix = (prefix + x) % n;
16 |         int residue = static_cast<int>(prefix);
17 |         if (first[residue] == -1)
18 |             first[residue] = i;
19 |         else
20 |             best = min(best, make_pair(first[residue] + 1, i));
21 |     }
22 |     if (best.first == n + 1)
23 |         cout << -1 << '\n';
24 |     else
25 |         cout << best.first << ' ' << best.second << '\n';
26 |     return 0;
27 | }
```

## 公开样例

样例 1 输入

```
3
1 2 3
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.16 | 总 450/526 会写脚本的月月鸟 ( PID 1555 )

出处：2020级HAUT新生周赛（四）@张承树专场（A，CID 1063）| 训练定位：进阶 | 数据结构

标签：线段树、位运算、区间查询 | 限制：2.000 S / 256 MB | 历史统计：0/16 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，输出 $m$ 行 每行一个数代表 $a[l]\&a[l+1]\&a[l+2]\dots\dots a[r-1]\&a[r]$ 的值

**输入要点：**一个 $n, m$

第二行 $n$ 个数

下面 $m$ 行，每行两个数 $l, r$

**输出要点：**输出 $m$ 行

每行一个数代表 $a[l]\&a[l+1]\&a[l+2]\dots\dots a[r-1]\&a[r]$ 的值

题面提示： $n$ 小于等于 $2e5$

$m$ 小于等于 $1e5$

保证数据大小都在 $1e9$ 范围内

由于数据量较大建议使用scanf或者更快的输入方式

## 零基础先修

- 线段树把区间递归拆分，支持  $O(\log n)$  单点修改和区间查询；先定义节点维护量。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：区间按位与满足结合律，可用迭代线段树维护。叶子存原数组，父节点存两个子区间的按位与；查询时把被覆盖的节点逐个与到答案。按位与的单位元可取所有位均为 1 的  $\sim 0$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

建树  $O(n)$ ，每次询问  $O(\log n)$ ，空间  $O(n)$

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m;
7 |     cin >> n >> m;
8 |     int size = 1;
9 |     while (size < n)
10 |         size <= 1;
11 |     vector<int> tree(2 * size, ~0);
12 |     for (int i = 0; i < n; ++i)
13 |         cin >> tree[size + i];
14 |     for (int i = size - 1; i; --i)
15 |         tree[i] = tree[i << 1] & tree[i << 1 | 1];
16 |     while (m--) {
17 |         int l, r;
18 |         cin >> l >> r;
19 |         --l;
20 |         int ans = ~0;
21 |         for (l += size, r += size; l < r; l >= 1, r >= 1) {
22 |             if (l & 1)
23 |                 ans &= tree[l++];
24 |             if (r & 1)
25 |                 ans &= tree[--r];
26 |         }
27 |         cout << ans << '\n';
28 |     }
29 |     return 0;
30 | }
```

## 公开样例

### 样例 1 输入

```
5 2
1 2 3 4 5
1 2
4 5
```

### 样例 1 输出

```
0
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 7.17 | 总 451/526 禁止复读 ( PID 1561 )

出处：2020级HAUT新生周赛（四）@张承树专场（G，CID 1063）| 训练定位：进阶 | 数据结构

标签：字符串、连续段、集合、模拟 | 限制：1.000 S / 128 MB | 历史统计：3/90 (3.3%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，第一行一个k输出被禁言人的个数。第二行由小到大输出k个整数，表示被禁言的人是谁。如果k=0，则输出空行即可



输入要点：第一行一个 $n$ ，代表群里的消息条数。

第2到 $n+1$ 行为聊天记录，每行为一个数字 $x$ 和一个字符串 $s$ ， $x$ 表示发言的人的编号， $s$ 表示这个人说的话。

$1 \leq n \leq 1000, 1 \leq |s| \leq 50$

$0 \leq x \leq 1000$

输出要点：第一行一个 $k$ 输出被禁言人的个数。

第二行由小到大输出 $k$ 个整数，表示被禁言的人是谁。

如果 $k=0$ ，则输出空行即可

题面提示：2复读1说的话发起了'lcltql'的复读，3并没有发起复读，4一个人发起了'%%%lcl'的复读，因此2和4将被禁言

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 集合用于去重和成员查询；输出有序结果时优先使用 set 或最后排序。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：“发起复读”只指一段相同消息中的第二条，而不是第三条、第四条。因此当前消息等于上一条、且上一条不是其前一条的延续时，才把当前发言者加入集合。set 同时完成去重和编号升序。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

## 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<string> messages(n);
9 |     set<int> banned;
10 |    for (int i = 0; i < n; ++i) {
11 |        int user;
12 |        cin >> user >> messages[i];
13 |        bool same = i >= 1 && messages[i] == messages[i - 1];
14 |        bool prevDup = i >= 2 && messages[i - 1] == messages[i - 2];
15 |        if (same && !prevDup)
16 |            banned.insert(user);
17 |    }
18 |    cout << banned.size() << '\n';
19 |    bool first = true;
20 |    for (int user : banned) {
```

```

21 |         if (!first)
22 |             cout << ' ';
23 |         cout << user;
24 |         first = false;
25 |     }
26 |     cout << '\n';
27 |     return 0;
28 | }

```

## 公开样例

样例 1 输入

```

5
1 lcltql
2 lcltql
3 lcltql
4 %%%lcl
4 %%%lcl

```

样例 1 输出

```

2
2 4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)
- [题面图片 3](#)

## 7.18 | 总 452/526 平衡区间（削弱版）（PID 1566）

出处：2020级HAUT新生周赛（五）@许金龙专场（C，CID 1064）| 训练定位：进阶 | 数据结构

标签：前缀和与差分、哈希与集合、最长区间 | 限制：1.000 S / 128 MB | 历史统计：5/72（6.9%）

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

**题意摘要：**已知一个长度为  $n$  的数组  $a$ ，给定两个数  $x, y$ 。对于数组  $a$  的子区间  $\{a_l, a_{l+1}, \dots, a_{r-1}, a_r\}$ ，若满足区间内  $x$  的个数等于  $y$  的个数，则被称为平衡区间(可以是空区间)。试求所能得到的平衡区间的长度的最大值。

**输入要点：**第一行一个正整数  $n$

第二行  $n$  个正整数  $a[i]$

第三行 两个正整数  $x, y$

**数据范围：**

$n \leq 3000$ ,

$a[i], x, y \leq n$

**输出要点：**输出一个整数，表示答案。

**题面提示：**样例解析：

所能得到的平衡区间有：

{1 1 2 2 3 3}, {1 2 2 3}, {2}, {2 2}, {2}

答案为 6

## 零基础先修

- 前缀和让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把  $x$  记为  $+1$ 、 $y$  记为  $-1$ 、其他值记为  $0$ 。一个区间内两者数量相等，恰好等价于该区间映射后的和为  $0$ ；记录每个前缀和值第一次出现的位置，再次出现时两位置之间就是平衡区间，取最大距离。若  $x=y$ ，则任意区间天然平衡，答案直接为  $n$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> a(n);
9 |     for (int& value : a)
10 |         cin >> value;
11 |     int x, y;
12 |     cin >> x >> y;
13 |     if (x == y) {
14 |         cout << n << '\n';
15 |         return 0;
16 |     }
17 |     vector<int> first(2 * n + 1, -2);
18 |     int prefix = 0, ans = 0;
19 |     first[n] = 0;
20 |     for (int i = 1; i <= n; ++i) {
21 |         if (a[i - 1] == x)
22 |             ++prefix;
23 |         else if (a[i - 1] == y)
24 |             --prefix;
25 |         int& pos = first[prefix + n];
26 |         if (pos == -2)
27 |             pos = i;
28 |         else
29 |             ans = max(ans, i - pos);
30 |     }
31 |     cout << ans << '\n';
32 |     return 0;
33 | }
```

## 公开样例

样例 1 输入

```
7
1 1 2 2 3 3 3
1 3
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.19 | 总 453/526 平衡区间 ( PID 1568 )

出处：2020级HAUT新生周赛（五）@许金龙专场（E，CID 1064）| 训练定位：进阶 | 数据结构

标签：前缀和与差分、哈希与集合、最长区间 | 限制：1.000 S / 128 MB | 历史统计：0/10 ( 0.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：已知一个长度为  $n$  的数组  $a$ ，给定两个数  $x, y$ 。对于数组  $a$  的子区间  $\{a_l, a_{l+1}, \dots, a_{r-1}, a_r\}$ ，若满足区间内  $x$  的个数等于  $y$  的个数，则被称为平衡区间(可以是空区间)。试求所能得到的平衡区间的长度的最大值。

输入要点：第一行一个正整数  $n$

第二行  $n$  个正整数  $a[i]$

第三行 两个正整数  $x, y$

数据范围：

$n \leq 1000000$ ，

$a[i], x, y \leq n$

输出要点：输出一个整数，表示答案。

题面提示：样例解析：

所能得到的平衡区间有：

$\{1\ 1\ 2\ 2\ 3\ 3\}$ ， $\{1\ 2\ 2\ 3\}$ ， $\{2\}$ ， $\{2\ 2\}$ ， $\{2\}$

答案为 6

## 零基础先修

- 前缀和让区间和  $O(1)$  查询；差分让批量区间增减转成端点修改。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：把  $x$  记为  $+1$ 、 $y$  记为  $-1$ 、其他值记为  $0$ 。一个区间内两者数量相等，恰好等价于该区间映射后的和为  $0$ ；记录每个前缀和值第一次出现的位置，再次出现时两位置之间就是平衡区间，取最大距离。若  $x=y$ ，则任意区间天然平衡，答案直接为  $n$ 。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(n)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> a(n);
9 |     for (int& value : a)
10 |         cin >> value;
11 |     int x, y;
12 |     cin >> x >> y;
13 |     if (x == y) {
14 |         cout << n << '\n';
15 |         return 0;
16 |     }
17 |     vector<int> first(2 * n + 1, -2);
18 |     int prefix = 0, ans = 0;
19 |     first[n] = 0;
20 |     for (int i = 1; i <= n; ++i) {
21 |         if (a[i - 1] == x)
22 |             ++prefix;
23 |         else if (a[i - 1] == y)
24 |             --prefix;
25 |         int& pos = first[prefix + n];
26 |         if (pos == -2)
27 |             pos = i;
28 |         else
29 |             ans = max(ans, i - pos);
30 |     }
31 |     cout << ans << '\n';
32 |     return 0;
33 | }
```

## 公开样例

样例 1 输入

```
7
1 1 2 2 3 3 3
13
```

样例 1 输出

```
6
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 7.20 | 总 454/526 牛牛mex函数 ( PID 1580 )

出处：2021级HAUT新生周赛（二）@李亚凯专场（B，CID 1070）| 训练定位：进阶 | 数据结构

标签：mex、布尔标记、集合 | 限制：1.000 S / 128 MB | 历史统计：26/315 ( 8.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：牛牛最近在组合游戏中经常会遇到求mex，于是他查百度得到：在组合游戏中计算状态的SG值时，我们常常会遇到mex函数。mex(S)的值为集合S中没有出现过的最小自然数。例如， $\text{mex}(\{1,2\}) = 0$ 、 $\text{mex}(\{0,1,2,3\}) = 4$ 。

输入要点：第一行输入一个整数n，表示集合S有n个元素。 $(1 \leq n \leq 100)$

第二行依次输入 $a_1, a_2, \dots, a_n$ , n个整数，代表集合S的元素。 $(0 \leq a_i \leq 1000)$

输出要点：输出集合的mex ( S )。

### 零基础先修

- 集合用于去重和成员查询；输出有序结果时优先使用 set 或最后排序。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：用布尔数组标记集合中出现过的非负整数。n 个元素最多只能覆盖  $0..n-1$ ，所以 mex 不会超过 n；从 0 向上找第一个未标记值即可。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(n+V)$  时间、 $O(V)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<bool> present(1002, false);
9 |     for (int i = 0; i < n; ++i) {
10 |         int value;
11 |         cin >> value;
12 |         if (value < (int)present.size())
13 |             present[value] = true;
14 |     }
15 |     int mex = 0;
16 |     while (present[mex])
17 |         ++mex;
18 |     cout << mex << '\n';
19 |     return 0;
20 | }
```

### 公开样例

5  
0 1 2 4 5

3

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

- HAUTOJ 原题
- 公开题解/参考来源 1

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：怎么可能会后悔(传送门)这道题是D的加强数据的版本，数据范围不一样，由于n的范围比较大，是，所以不能使用二进制枚举的方法了，那么就需要挖掘一些性质。 $10^5 105 105$ 对于每个数我们单独考虑第i位，那么一共n个数，设第i位是0的个数是x，是1的个数是y，那么第i为对最后答案的贡献是什么？简单的推理得到贡献是，那么考虑所有位，就可以得出答案是所有的数进行或的值记为 $2^{i_1} + 2^{i_2} + \dots + 2^{i_n}$ 。那么第i为对最后答案的贡献是什么？简单的推理得到贡献是，那么考虑所有位，就可以得出答案是所有的数进行或的值记为 $2^{i_1} + 2^{i_2} + \dots + 2^{i_n}$ 。

$$C_{-}\{n\}^{\wedge}\{0\} + C_{-}\{n\}^{\wedge}\{1\} + C_{-}\{n\}^{\wedge}\{2\} + C_{-}\{n\}^{\wedge}\{3\} + \dots + C_{-}\{n\}^{\wedge}\{k\} \times x^{\wedge}k + \dots + C_{-}\{n\}^{\wedge}\{n\} \times x^{\wedge}n = 2^{\wedge}n(1+1)n = (Cn0+Cn1+Cn2+Cn3+\dots+Cnkxk+\dots+Cnnxn) = 2n(1+1)n = (Cn0+Cn1+Cn2+Cn3+\dots+Cnkxk+\dots+Cnnxn) = 2n \ 2^{\wedge}n - 0 = 2^{\ast}(C_{-}\{n\}^{\wedge}\{1\} + C_{-}\{n\}^{\wedge}\{3\} + \dots + C_{-}\{n\}^{\wedge}\{k\} \times x^{\wedge}k) = 2^{\wedge}n2n-0=2^{\ast}(Cn1+Cn3+\dots+Cnkxk)=2n2n-0=2^{\ast}(Cn1+Cn3+\dots+Cnkxk)=2n \ C_{-}\{n\}^{\wedge}\{1\} + C_{-}\{n\}^{\wedge}\{3\} + \dots + C_{-}\{n\}^{\wedge}\{k\} \times x^{\wedge}k = 2^{\wedge}\{n-1\}Cn1+Cn3+\dots+Cnkxk=2n-1Cn1+Cn3+\dots+Cnkxk=2n-1 \ 2^{\wedge}x \ 2^{\ast}\{y-1\} \ 2^{\ast}b = 2^{\wedge}\{x+y-1+b\} = 2^{\wedge}\{n+b-1\} = 2^{\wedge}\{n-1\} \ 2^{\ast}b2x2y-12b=2x+y-1b=2n+b-1=2n-12b2x2y-12b=2x+y-1b=2n+b-1=2n-12b \ x(101010) = 2^{\wedge}\{n-1\} (1 \ 2^{\ast}A5 + 0 \ 2^{\ast}A4 + 1 \ 2^{\ast}A3 + 0 \ 2^{\ast}A2 + 1 \ 2^{\ast}A1 + 0 \ 2^{\ast}A0) \ x(101010)=2n-1(125+024+123+022+121+020)x(101010)=2n-1(125+024+123+022+121+020) 时间复杂度：O(n)O(n)O(n)参考代码$$

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

 $O(n)$ 

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```

1 | #include <stdio.h>
2 | typedef long long ll; // def一下long long
3 | const ll mod = 998244353; // 模数
4 | ll fast_pow(ll a, ll n, ll mod) { // 快速幂
5 |     ll ans = 1;
6 |     while (n) {
7 |         if (n & 1)
8 |             ans = ans * a % mod;
9 |         a = a * a % mod;
10 |        n >>= 1;
11 |    }
12 |    return ans;
13 | }
14 | int main() {
15 |     int n;
16 |     scanf("%d", &n);
17 |     ll ans = 0;
18 |     for (int i = 0; i < n; i++) {
19 |         ll x;
20 |         scanf("%lld", &x);
21 |         ans |= x;
22 |     }
23 |     printf("%lld\n", ans * fast_pow(2, n - 1, mod) % mod);
24 |     return 0;
25 | }

```

## 公开样例

### 样例 1 输入

3  
1 2 3

### 样例 1 输出

12

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1
- 题面图片 1

## 7.22 | 总 456/526 赶上世界崩坏的速度，将你拯救 ( PID 1771 )

出处：2022级HAUT新生周赛（三）@焦小航专场（B，CID 1081）| 训练定位：进阶 | 数据结构

标签：哈希与集合、贪心、计数 | 限制：1.000 S / 128 MB | 历史统计：26/153 ( 17.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出至少多少次操作后存在一种排序使得剩下的数排序后，在数值上严格递增。

输入要点：第一行给出  $t$  (  $1 \leq t \leq 1000$  )

接下来  $t$  组，每组包括两行，每组的第一行给出一个  $n$  (  $1 \leq n \leq 50$  )，代表数的数量。第二行给出 数值  $a_i$  (  $0 < a_i \leq 10000$  )。

输出要点：输出至少多少次操作后存在一种排序使得剩下的数排序后，在数值上严格递增。

### 零基础先修

- set 有序且去重，unordered\_set 平均  $O(1)$ ；使用前明确是否需要顺序。
- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：最后能严格递增等价于“剩余元素互不相同”。设  $n$  个数中有  $u$  个不同值，则至少要删掉  $d=n-u$  个重复副本。一次恰好删两个数，所以至少  $\text{ceil}(d/2)$  次；若  $d$  为奇数，最后一次顺带删掉一个目前唯一的数也不会制造重复，因而这个下界总能达到。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

### 复杂度

每组  $O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         int n;
10 |        cin >> n;
11 |        set<int> distinct;
12 |        for (int i = 0, x; i < n; ++i) {
13 |            cin >> x;
```

```

14 |         distinct.insert(x);
15 |     }
16 |     int duplicates = n - static_cast<int>(distinct.size());
17 |     cout << (duplicates + 1) / 2 << '\n';
18 | }
19 | return 0;
20 | }

```

## 公开样例

样例 1 输入

```

2
5
1 2 3 4 4
6
1 2 3 2 2 2

```

样例 1 输出

```

1
2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 7.23 | 总 457/526 万分之一的光(hard version) ( PID 1788 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（C，CID 1083）| 训练定位：进阶 | 数据结构

标签：栈与队列、排序 | 限制：1.000 S / 128 MB | 历史统计：4/73 ( 5.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，亮着的路灯的数量。

输入要点：第一行两个整数  $n, k$ ，表示网格边长和亮着灯的大楼的数量。

接下来  $k$  行每行两个整数  $x, y$ ，表示大楼的坐标。

( $1 \leq n, k \leq 105$ )

输出要点：一个整数，亮着的路灯的数量。

## 零基础先修

- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：万分之一的光(hard version) 题意：同上 解法：相比于easy version增加了数据范围。对楼房先按横坐标排序，然后判断排序后的楼之间是否相邻（优先按照纵坐标排序，再按照横坐标排序，这样对比数组中相邻的楼房是否相邻即可），然后按照纵坐标排序，再判断排序后的楼之间是否相邻（同理），由于每条相邻的边上都有4栈路灯，所以只要将答案乘4即可。（使用快速排序，时间复杂度（ $n \cdot \log n$ ））

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int MAXN = 1e5 + 5;
4 | struct lou {
5 |     int x, y;
6 | } a[MAXN];
7 | int n, k, ans;
8 | bool cmp1(lou a, lou b) {
9 |     if (a.x != b.x)
10 |         return a.x < b.x;
11 |     else
12 |         return a.y < b.y;
13 | }
14 | bool cmp2(lou a, lou b) {
15 |     if (a.y != b.y)
16 |         return a.y < b.y;
17 |     else
18 |         return a.x < b.x;
19 | }
20 | int main() {
21 |     scanf("%d%d", &n, &k);
22 |     for (int i = 1; i <= k; i++) {
23 |         int x, y;
24 |         scanf("%d%d", &x, &y);
25 |         a[i].x = x;
26 |         a[i].y = y;
27 |     }
28 |     sort(a + 1, a + k + 1, cmp1);
29 |     for (int i = 1; i < k; i++)
30 |         if (a[i].x == a[i + 1].x && a[i].y == a[i + 1].y - 1)
31 |             ans++;
32 |     sort(a + 1, a + k + 1, cmp2);
33 |     for (int i = 1; i < k; i++)
34 |         if (a[i].y == a[i + 1].y && a[i].x == a[i + 1].x - 1)
35 |             ans++;
36 |     printf("%d", ans * 4);
37 |     return 0; // qwq
38 | }
```

## 公开样例

样例 1 输入

```
6 12
1 1
2 1
2 2
1 4
3 3
4 3
4 4
3 4
3 6
4 6
5 6
6 6
```

样例 1 输出

```
36
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。

- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.24 | 总 458/526 堆栈的奇妙世界 ( PID 1816 )

出处：2023级HAUT新生周赛（零）熟悉周赛规则专场（J，CID 1085） | 训练定位：进阶 | 数据结构

标签：数组与序列、栈与队列 | 限制：1.000 S / 128 MB | 历史统计：23/64 ( 35.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在给出所有的操作，请你输出最终的数列（从头至尾），每个数字用空格隔开。

输入要点：第一行一个整数 $n$ ， $1 \leq n \leq 10000$ 。

然后接下来 $n$ 行每行一个操作 $x$ ， $1 \leq x \leq 3$ 。

输出要点：一行整数，用空格隔开

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：堆栈的奇妙世界 按照题目描述进行操作即可，注意记录数列的长度变化。c c++
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | int a[10005], idx;
3 | int main() {
4 |     int n;
5 |     scanf("%d", &n);
6 |     for (int i = 0; i < n; i++) {
7 |         int op;
8 |         scanf("%d", &op);
9 |         if (op == 1) {
10 |             a[idx] = 1;
11 |             idx++;
12 |         } else if (op == 2) {
```

```
13 |         a[idx] = a[idx - 1];
14 |         idx++;
15 |     } else {
16 |         a[idx - 2] += a[idx - 1];
17 |         idx--;
18 |     }
19 | }
20 | for (int i = 0; i < idx; i++)
21 |     printf("%d ", a[i]);
22 | return 0;
23 | }
```

公开样例

样例 1 输入

```
5
1
1
3
2
2
```

样例 1 输出

```
2 2 2
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

7.25 | 总 459/526 阿拉德的冒险者 ( PID 1887 )

出处：河南工大2024新生周赛 ( 5 ) ——命题人：刘庭铄 ( A , CID 1095 ) | 训练定位：进阶 | 数据结构

标签：栈与队列、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：5/237 ( 2.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，输出包含一行，共一个整数，表示你能畅游的国家的个数。

输入要点：输入包含 2 行。

第一行三个正整数  $n, m, k$  (  $1 \leq n \leq 2e4, 1 \leq m \leq 1e9, 0 \leq k \leq 2e4$  )，

分别代表区域的个数，你拥有的初始生命力，你可以释放能量的次数。

第二行包含  $n$  个正整数，第  $i$  个正整数  $a_i$  (  $1 \leq a_i \leq 1e9$  ) 代表你不释放能量帮助第  $i$  个区域需要消耗的生命力大小

输出要点：输出包含一行，共一个整数，表示你能畅游的国家的个数。

题面提示：请注意看主页通知下面的提示

零基础先修

- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：阿拉德的冒险者 遍历的路线已经确定（ $1 \sim n$ ），只需要从 1 到  $n$  遍历一遍，将各自需要消耗的生命力排序后放入另一数组中，由于数据较小，每次重新排序并不会超时，排序后在头尾设置指针，需要消耗生命力最小的不释放能量，大的释放能量即可 stl 小根堆版本
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

通常为  $O(n)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | const int N = 2e5 + 10;
5 | ll n, m, k;
6 | ll sum;
7 | ll s[N];
8 | priority_queue<ll, vector<ll>, greater<ll>> q; // 小根堆，确保队头为最小的元素值
9 | int main() {
10 |     cin >> n >> m >> k;
11 |     for (int i = 1; i <= n; i++)
12 |         cin >> s[i];
13 |     for (int i = 1; i <= n; i++) {
14 |         q.push(s[i]); // 将该地区入队
15 |         if (q.size() > k) // 如果队中元素大于k
16 |             {
17 |                 sum += q.top(); // 第一个地区，即消耗生命力最小的地区不释放能量，选择消耗生命
18 |                 q.pop();
19 |             }
20 |         if (sum >= m) {
21 |             cout << i - 1; // 在当前区域消耗完生命力意味着没有畅游该国家
22 |             return 0;
23 |         }
24 |     }
25 |     cout << n;
26 |     return 0;
27 | }
```

## 公开样例

样例 1 输入

```
3 10 1
11 2 1
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.26 | 总 460/526 千年暗室，一灯即明！（PID 1900）

出处：河南工大2024新生周赛（6）——命题人：魏方（C，CID 1097）| 训练定位：进阶 | 数据结构

标签：数组与序列、栈与队列 | 限制：1.000 S / 128 MB | 历史统计：14/120（11.7%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：小扣初始血量为 1，且无上限。假定小扣原计划按房间编号升序访问所有房间补血/打怪，为保证血量始终为正值，wf需要一个小桂，需对房间访问顺序进行调整，每次仅能将一个大妖房间（负数的房间）调整至访问顺序末尾。请返回wf最少需要调整几次，才能顺利访问所有房间。若调整顺序也无法访问完全部房间，请返回 -1。

输入要点：输入一个整数 $n$ ，表示总共有 $n$ 个房间( $1 \leq n \leq 105$ )

接下来输入 $n$ 个数(每个数的值 $x, 1 \leq x \leq 105$ )，表示血量增加或者减少的数值

输出要点：输出一个整数，表示保证wf能顺利通关的最小开桂次数，若是无论如何都无法通关，则输出-1

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：千年暗室，一灯即明！方法一：贪心 + 优先队列 思路 我们按照原计划访问所有房间。当访问到第  $i$  个房间时，如果生命值小于等于 0，那么我们必须要对房间顺序进行调整：显然选择第  $i$  个房间之后的房间是没有意义的，它并不会改变当前的生命值；因此我们只能选择第  $i$  个房间及之前的房间。对于所有可选的房间，无论将哪个房间调整至末尾，都不会改变最终的生命值（因为数组 `nums` 的和不会变化）。由于我们希望调整的次数最少，因此应当贪心地选择最小的那个 `nums[j]` 调整至末尾，使得当前的生命值尽可能高。算法：在遍历房间的过程中，如果 `nums[i]` 为负数，我们将其放入一个小根堆（优先队列）中。当计算完第  $i$  个房间的生命值影响后，如果生命值小于等于 0，那么我们取出堆顶元素，表示将该房间调整至末尾，并将其补回生命值中。由于一定会从小根堆中取出一个小于等于 `nums[i]` 的值，因此调整完成后，生命值一定大于 0。当所有房间遍历完成后，我们还需要将所有从堆中取出元素的和重新加入生命值，如果生命值小于等于 0，说明无解，则输出-1。本题使用c++解题可以直接使用c++标准库中的stl方法，省去自行实现栈和优先级队列的方法，建议同学们下去可自行学习stl方法，可以省去大量的时间。栈的本质是一个先进先出的结构，优先级队列是一个大根或者小根堆，插入元素后会自行调整插入位置，使得从优先级队列出来中的数值是最大或者最小的，vector容器就相当于c语言中的数组，但大小可伸缩即变长数组。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <iostream>
2 | #include <vector>
3 | #include <queue>
```

```

4 | using namespace std;
5 | int magic(vector<int>& nums) {
6 |     priority_queue<int, vector<int>, greater<int>> q;
7 |     int ans = 0;
8 |     long long hp = 1, delay = 0;
9 |     for (int num : nums) {
10 |         if (num < 0) {
11 |             q.push(num);
12 |         }
13 |         hp += num;
14 |         if (hp <= 0) {
15 |             ++ans;
16 |             delay += q.top();
17 |             hp -= q.top();
18 |             q.pop();
19 |         }
20 |     }
21 |     hp += delay;
22 |     return hp <= 0 ? -1 : ans;
23 | }
24 | int main() {
25 |     int n;
26 |     cin >> n;
27 |     vector<int> nums(n);
28 |     for (int i = 0; i < n; i++)
29 |         cin >> nums[i];
30 |     int ans = magic(nums);
31 |     cout << ans << endl;
32 |     return 0;
33 | }

```

## 公开样例

样例 1 输入

```

10
100 100 100 -250 -60 -140 -50 -50 100 150

```

样例 1 输出

```

1

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.27 | 总 461/526 Set ( PID 1901 )

出处：河南工大2024新生周赛 ( 6 ) ——命题人：魏方 ( D , CID 1097 ) | 训练定位：进阶 | 数据结构

标签：数组与序列、哈希与集合 | 限制：1.000 S / 128 MB | 历史统计：8/51 ( 15.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：求可以进行的最大操作数。

输入要点：每个测试包含多个测试用例。输入的第一行包含一个整数  $t$  ( $1 \leq t \leq 104$ ) - 测试用例的数量。测试用例说明如下。

每个测试用例的唯一一行包含三个整数  $l, r$  和  $k$  ( $1 \leq l \leq r \leq 109$ ), ( $1 \leq k \leq r - l + 1$ ) -- 最小整数，最大整数和参数  $k$ 。

输出要点：对于每个测试用例，输出一个整数，可执行的最大操作数。

题面提示：注

在第一个测试用例中，最初是  $S = \{3, 4, 5, 6, 7, 8, 9\}$ 。一个可能的最佳操作顺序是



-  
选择  $x=4$

进行第一次操作，因为在  $S$  中有两个 4 的倍数：4 和 8。 $S$  就等于  $\{3,5,6,7,8,9\}$

；

-  
选择  $x=3$

进行第二次运算，因为  $S$  中有 3 的三个倍数：3、6 和 9。 $S$  就等于  $\{5,6,7,8,9\}$ 。

在第二个测试案例中，起初是  $S=\{4,5,6,7,8,9\}$ 。一个可能的最佳操作序列是

-  
选择  $x=5$ ,  $S$  变为等于  $\{4,6,7,8,9\}$ ；

-  
选择  $x=6$ ,  $S$  等于  $\{4,7,8,9\}$ ；

-  
选择  $x=4$ ,  $S$  变为  $\{7,8,9\}$ ；

-  
选择  $x=8$ ,  $S$  就等于  $\{7,9\}$ ；

-  
选择  $x=7$ ,  $S$  变为等于  $\{9\}$ ；

-  
选择  $x=9$ ,  $S$  就等于  $\{\}$ 。

在第三个测试案例中，最初选择  $S=\{7,8,9\}$ 。对于  $S$  中的每一个  $x$ ， $S$  中除了  $x$  本身之外，找不到  $x$  的倍数。自  $k=2$  起，无法进行任何运算。

在第四个测试用例中，最初是  $S=\{2,3,4,5,6,7,8,9,10\}$ 。一个可能的最佳操作序列是

-  
选择  $x=2$ ,  $S$  变为等于  $\{3,4,5,6,7,8,9,10\}$ ；

-  
选择  $x=4$

,  $S$  等于  $\{3,5,6,7,8,9,10\}$

-  
选择  $x=3$ ,  $S$  就等于  $\{5, \dots$

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- `set` 有序且去重，`unordered_set` 平均  $O(1)$ ；使用前明确是否需要顺序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：Set 我们可以从小到大删除数字，因此，之前删除的数字并不会影响未来的选择(如果  $x < y$ , 那么  $x$  不可能是  $y$  的倍数)，因此，当且仅当  $k \cdot x \leq r$ , 即  $x \leq \lceil r/k \rceil$  时，整数  $x (1 \leq x \leq r)$  可以被整除，答案是  $\max(\lceil r/j \rceil - 1, 0)$ ; `cin` 对应 `c` 语言中的 `scanf`，`cout` 对应 `printf`。`max` 是 `c++` 标准库中的取最大值函数
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。

- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

通常为  $O(n)$

易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int T;
4 | void solve() {
5 |     int l, r, k;
6 |     cin >> l >> r >> k;
7 |     cout << max(r / k - l + 1, 0) << endl;
8 |     return;
9 | }
10 | int main() {
11 |     cin >> T;
12 |     while (T--) {
13 |         solve();
14 |     }
15 | }
```

公开样例

样例 1 输入

```
8
3 9 2
4 9 1
7 9 2
2 10 2
154 220 2
147 294 2
998 24435 3
1 1000000000 2
```

样例 1 输出

```
2
6
0
4
0
1
7148
500000000
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

7.28 | 总 462/526 恶魔 ( PID 1910 )

出处：河南工大2024新生周赛 ( 7 ) ——命题人：刘义 ( H , CID 1098 ) | 训练定位：进阶 | 数据结构

标签：贪心、栈与队列、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/12 ( 25.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：现在，每个恶魔都想要尽可能多地杀掉其他恶魔。给定每个恶魔的初始血量值，你需要计算出在最优策略下，每个恶魔最多可以杀掉多少个同伴。

输入要点：第一行包含一个整数 $n(1 \leq n \leq 2 \times 10^5)$

第二行包含 $n$ 个整数 $a[1], a[2], \dots, a[n](0 \leq a[i] \leq 10^9)$

输出要点：一行，用空格隔开的  $n$  个整数。第  $i$  个整数表示第  $i$  个恶魔最多可以杀掉多少个同伴。

题面提示：第一个恶魔：无法杀死任何恶魔，所以杀0个；

第二个恶魔：先杀第三个恶魔后，变为2血量，再杀掉第一个恶魔，所以杀2个；

第三个恶魔：当第二个恶魔先杀掉第一个恶魔，血量变为1，第三个恶魔再杀掉第二的恶魔，所以杀1个；

第四个恶魔：第四个恶魔先杀掉第三的恶魔，血量变2，然后让第二个恶魔杀掉第一个恶魔，血量变为1，然后第四个恶魔再杀掉第二恶魔，所以总共杀了2个。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
- 核心处理采用：恶魔更小的第一个位置，右侧同理 `#include <bits/stdc++.h>`
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int N = 2e5 + 10;
4 | int n;
5 | int a[N], l[N], r[N];
6 | int main() {
7 |     cin >> n;
8 |     for (int i = 1; i <= n; i++)
9 |         cin >> a[i];
10 |     stack<int> s;
11 |     for (int i = 1; i <= n; i++) // 左边
12 |     {
13 |         while (!s.empty() && a[s.top()] > a[i])
14 |             s.pop();
15 |         if (!s.empty())
16 |             l[i] = l[s.top()] + 1;
17 |         else
18 |             l[i] = 0;
19 |         s.push(i);
20 |     }
21 |     reverse(a + 1, a + 1 + n); // 反转一次，开始做右边
22 |     stack<int> s1;
```

```

23 |     for (int i = 1; i <= n; i++) // 右边
24 |     {
25 |         while (!s1.empty() && a[s1.top()] > a[i])
26 |             s1.pop();
27 |         if (!s1.empty())
28 |             r[i] = r[s1.top()] + 1;
29 |         else
30 |             r[i] = 0;
31 |         s1.push(i);
32 |     }
33 |     for (int i = 1; i <= n; i++) {
34 |         cout << l[i] + r[n - i + 1] << " ";
35 |     }
36 |     return 0;
37 | }

```

## 公开样例

样例 1 输入

```

4
1 4 2 4

```

样例 1 输出

```

0 2 1 2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 7.29 | 总 463/526 合并果子 ( PID 1968 )

出处：河南工大2025新生周赛 ( 5 ) ——命题人：袁林坤 ( H , CID 1106 ) | 训练定位：进阶 | 数据结构

标签：贪心、栈与队列、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：28/66 ( 42.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**因为还要花大力气把这些果子搬回家，所以多多在合并果子时要尽可能地节省体力。假定每个果子重量都为 1，并且已知果子的种类数和每种果子的数目，你的任务是设计出合并的次序方案，使多多耗费的体力最少，并输出这个最小的体力耗费值。

**输入要点：**共两行。

第一行是一个整数  $n(1 \leq n \leq 10000)$ ，表示果子的种类数。

第二行包含  $n$  个整数，用空格分隔，第  $i$  个整数  $a_i(1 \leq a_i \leq 20000)$  是第  $i$  种果子的数目。

**输出要点：**一个整数，也就是最小的体力耗费值。输入数据保证这个值小于 231。

**题面提示：**对于全部的数据，保证有  $n \leq 10000$ 。

## 零基础先修

- 贪心需要证明局部选择能延伸到全局最优，常用交换论证或下界与构造相遇。
- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 确定每一步的贪心选择，并用交换论证或下界证明该选择不会损失最优解。
3. 核心处理采用：合并果子 贪心加堆，可用两个数组模拟顺序队列，每次最小的两个相加，加完重新插入排序，重复过程到只剩一个元素，总值即为答案。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 题解选择在当前局面最有利且不占用额外未来资源的方案。
- 若某个最优解首个选择不同，可把它与本算法的选择交换而不使答案变差。
- 反复交换可得到与算法完全相同的最优解，因此算法达到全局最优。

## 复杂度

$O(n)$

## 易错点

- 没有证明的“看起来最优”不算结论；尝试寻找反例或交换论证。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define endl '\n'
4 | #define ull unsigned long long
5 | #define ll long long
6 | #define PII pair<int, int>
7 | const ll N = 1e5 + 10;
8 | const ll Mod = 1e9 + 7;
9 | const int P = 1;
10 | void solve() {
11 |     priority_queue<int, vector<int>, greater<int>> pq;
12 |     int n;
13 |     cin >> n;
14 |     for (int i = 0; i < n; i++) {
15 |         int x;
16 |         cin >> x;
17 |         pq.push(x);
18 |     }
19 |     int sum = 0;
20 |     while (pq.size() > 1) {
21 |         int a = pq.top();
22 |         pq.pop();
23 |         int b = pq.top();
24 |         pq.pop();
25 |         int temp = a + b;
26 |         sum += temp;
27 |         pq.push(temp);
28 |     }
29 |     cout << sum << endl;
30 | }
31 | int main() {
32 |     ios::sync_with_stdio(0);
33 |     cin.tie(0);
34 |     cout.tie(0);
35 |     int T = 1;
36 |     // cin >> T;
37 |     while (T--)
38 |         solve();
39 |     return 0;
40 | }
```

## 公开样例

样例 1 输入

```
3
1 2 9
```

样例 1 输出

```
15
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.30 | 总 464/526 数组切割 ( PID 1977 )

出处：河南工大2025新生周赛 ( 6 ) ——命题人：张康发 ( H , CID 1107 ) | 训练定位：进阶 | 数据结构

标签：数组与序列、字符串、哈希与集合 | 限制：1.000 S / 128 MB | 历史统计：13/55 ( 23.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定一个包含 $2n$ 个整数的序列 $a$ 。记 $f(b)$ 表示序列 $b$ 中出现次数为奇数的不同元素的数量。你需要将给定的数组拆分为两个不相交的子序列 $p$ 和 $q$  ( 每个子序列的长度均为 $n$  )，使得 $f(p)+f(q)$ 的值最大化。输出这个最大值。

输入要点：第一行输入一个整数 $n(1 \leq n \leq 10^3)$ ；

第二行输入 $2n$ 个整数 $a_1, a_2, \dots, a_{2n}(1 \leq a_i \leq 2n)$ 。

输出要点：输出 $f(p)+f(q)$ 的最大值。

题面提示：注意分析题目。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- `string` 可按下标访问字符；注意长度、空串、大小写、标点和 `getline` 与 `cin` 混用。
- `set` 有序且去重，`unordered_set` 平均  $O(1)$ ；使用前明确是否需要顺序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：数组切割 根据题目可以分析出，出现次数为奇数的不同元素一定有偶数个。根据元素的特点分类讨论：对于出现次数为偶数且 $/2$ 为奇数的，直接分开放到两个序列中，之后不再考虑；对于出现次数为偶数且 $/2$ 为偶数的，交替放到两个序列中，并且放入两个序列的个数之差为2，以保证答案最大化；对于出现次数为奇数的，分为一奇一偶放入两个序列。注意特例的判断。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰处理好一次，累计状态即为所求。

### 复杂度

通常为  $O(n)$

### 易错点

- 确认  $0/1$  起下标、首尾元素和  $n=1$ 。
- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | typedef long long ll;
4 | const int P = 1e9 + 7;
5 | const int N = 4e5 + 10;
6 | int a[N];
7 | void solve() {
8 |     int n;
9 |     cin >> n;
10 |    unordered_map<int, int> mp;
11 |    vector<int> v1; // 偶数且/2为偶数
12 |    vector<int> v2; // 偶数且/2为奇数
13 |    vector<int> v3; // 奇数 一定有偶数个
14 |    for (int i = 1; i <= 2 * n; i++) {
15 |        cin >> a[i];
16 |        mp[a[i]]++; // 存储元素出现的次数
17 |    }
18 |    int cnt1 = 0, cnt2 = 0; // cnt1,cnt2分别表示序列p, q的元素数量
19 |    int ans = 0;
20 |    for (auto t : mp) {
21 |        int w = t.first, v = t.second;
22 |        if (v % 2 == 0 && (v / 2) % 2 == 0)
23 |            v1.push_back(w);
24 |        if (v % 2 == 0 && (v / 2) % 2 == 1)
25 |            v2.push_back(w);
26 |        if (v % 2 == 1)
27 |            v3.push_back(w);
28 |    }
29 |    ans += v2.size() * 2;
30 |    if (v3.size() != 0) {
31 |        for (int i = 0; i < v1.size(); i++) {
32 |            int t = mp[v1[i]];
33 |            if (cnt1 > cnt2)
34 |                cnt1 += (t / 2) - 1, cnt2 += (t / 2) + 1;
35 |            else
36 |                cnt1 += (t / 2) + 1, cnt2 += (t / 2) - 1;
37 |            ans += 2;
38 |        }
39 |        ans += v3.size();
40 |    } else {
41 |        if (v1.size() % 2 == 1)
42 |            ans += (v1.size() - 1) * 2;
43 |        else
44 |            ans += v1.size() * 2;
45 |    }
46 |    cout << ans << endl;
47 | }
48 | int main() {
49 |     int t;
50 |     // cin>>t;
51 |     t = 1;
52 |     while (t--) {
53 |         solve();
54 |     }
55 |     return 0;
56 | }
```

## 公开样例

样例 1 输入

```
2
2 2 2 2
```

样例 1 输出

```
0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 7.31 | 总 465/526 编程课 ( PID 1987 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（G，CID 1110） | 训练定位：进阶 | 数据结构

标签：优先队列、双向链表、模拟、延迟删除 | 限制：1.000 S / 128 MB | 历史统计：1/9 ( 11.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：任务是模拟以上过程，确定做题的配对及顺序。

输入要点：第一行一个正整数  $n$  表示队伍中的人数。（ $1 \leq n \leq 200000$ ）

第二行包含  $n$  个字符 B 或者 G，B 代表男，G 代表女。

第三行为  $n$  个整数  $a_i$ 。所有信息按照从左到右的顺序给出。（ $1 \leq a_i \leq 50000$ ）

输出要点：第一行一个整数表示出列的总对数  $k$ 。

接下来  $k$  行，每行是两个整数。按做题顺序输出，两个整数代表这一对码伴的编号（按输入顺序从左往右 1 至  $n$  编号）。请先输出较小的整数，再输出较大的整数。

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：用 left/right 数组把当前仍在队列中的人维护成双向链表。优先队列 保存所有相邻且性别不同的候选对，按能力差、左端编号排序。弹出时若两人已不相邻就丢弃；否则记录并删除二人，再把新形成的边界候选入堆。这是“堆延迟删除 + 数组链表”的典型组合。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | struct Candidate {
4 |     int diff, left, right;
5 |     bool operator>(const Candidate& other) const {
6 |         return tie(diff, left, right) > tie(other.diff, other.left, other.right);
7 |     }
8 | };
9 | int main() {
10 |     ios::sync_with_stdio(false);
11 |     cin.tie(nullptr);
12 |     int n;
13 |     cin >> n;
14 |     string gender;
15 |     cin >> gender;
16 |     gender = " " + gender;
17 |     vector<int> skill(n + 1), left(n + 1), right(n + 1);
18 |     for (int i = 1; i <= n; ++i) {
```



```

19 |         cin >> skill[i];
20 |         left[i] = i - 1;
21 |         right[i] = i + 1;
22 |     }
23 |     priority_queue<Candidate, vector<Candidate>, greater<Candidate>> heap;
24 |     auto addCandidate = [&](int a, int b) {
25 |         if (a >= 1 && b <= n && gender[a] != gender[b]) {
26 |             heap.push({abs(skill[a] - skill[b]), a, b});
27 |         }
28 |     };
29 |     for (int i = 1; i < n; ++i)
30 |         addCandidate(i, i + 1);
31 |     vector<pair<int, int>> ans;
32 |     while (!heap.empty()) {
33 |         Candidate cur = heap.top();
34 |         heap.pop();
35 |         int x = cur.left, y = cur.right;
36 |         if (right[x] != y || left[y] != x)
37 |             continue;
38 |         ans.push_back({x, y});
39 |         int nl = left[x], nr = right[y];
40 |         if (nl >= 1)
41 |             right[nl] = nr;
42 |         if (nr <= n)
43 |             left[nr] = nl;
44 |         right[x] = left[y] = -1;
45 |         addCandidate(nl, nr);
46 |     }
47 |     cout << ans.size() << '\n';
48 |     for (auto [l, r] : ans) {
49 |         cout << l << ' ' << r << '\n';
50 |     }
51 |     return 0;
52 | }

```

## 公开样例

### 样例 1 输入

```

4
BGBG
4 2 4 3

```

### 样例 1 输出

```

2
3 4
1 2

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“数据结构”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 模块 8 | 动态规划

动态规划的四件事是状态、转移、初值和答案。零基础阶段先写出状态含义，再考虑维度压缩，避免只背代码。

本模块共 21 道唯一题。

### 8.1 | 总 466/526 盖瓦 ( PID 1565 )

出处：2020级HAUT新生周赛（五）@许金龙专场（B，CID 1064）| 训练定位：入门 | 动态规划

标签：动态规划、递推、斐波那契 | 限制：1.000 S / 128 MB | 历史统计：46/76 ( 60.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

#### 题意与输入输出拆解

题意摘要：根据给定输入，每行输出对应的  $f(n)$

输入要点：第一行输入一个正整数  $T$  ( $T \leq 20$ ) 表示样例个数

接下来  $T$  行每行输入一个正整数  $n$  ( $n \leq 20$ )

输出要点：每行输出对应的  $f(n)$

题面提示：试着列出前几项

#### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

#### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：观察最左侧：竖放一块后剩下  $2 \times (n-1)$ ，横放则必须上下各一块，剩下  $2 \times (n-2)$ ，故  $f(n)=f(n-1)+f(n-2)$ ，且  $f(0)=f(1)=1$ 。预处理到 20 后逐询问输出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

#### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

#### 复杂度

预处理  $O(20)$ ，每组  $O(1)$ ， $O(1)$  空间

#### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

#### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long f[21] = {};
5 |     f[0] = f[1] = 1;
6 |     for (int i = 2; i <= 20; ++i)
7 |         f[i] = f[i - 1] + f[i - 2];
8 |     int T;
9 |     cin >> T;
10 |    while (T--) {
11 |        int n;
12 |        cin >> n;
13 |        cout << f[n] << '\n';
    }
```

```
14 |     }  
15 |     return 0;  
16 | }
```

## 公开样例

样例 1 输入

```
3  
2  
4  
7
```

样例 1 输出

```
2  
5  
21
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 8.2 | 总 467/526 体育课 (一) (PID 1359)

出处：2017级河南工大新生周赛（八）（F，CID 1037）| 训练定位：基础提高 | 动态规划

标签：动态规划、数字三角形、滚动数组 | 限制：1.000 S / 128 MB | 历史统计：3/3 (100.0%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出 XX 他用尽全力耗尽人品能得到的最高分。

输入要点：第一行一个整数  $n$  ( $0 < n < 50$ )。

接下来  $n$  个俯视图（每个图 18 行）。

正式的游戏都是 18 行，一共  $1 + 2 + \dots + 18$  个点。

每个点的分值都是  $s$ 。（ $0 \leq s \leq 1000$ ）

输出要点：输出 XX 他用尽全力耗尽人品能得到的最高分。

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：每幅图固定是 18 行数字三角形。令  $dp[j]$  表示到当前行第  $j$  个点的最高得分；为了不覆盖上一行仍要使用的值，从右向左更新： $dp[j] = \max(dp[j-1], dp[j]) + \text{value}$ 。左右边界只来自一个父节点。读完 18 行后取最后一行 dp 的最大值。也可以从底向上把每个点加上两个孩子的较大值。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。

- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

每组  $O(18^2)$  时间、 $O(18)$  空间

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         const long long NEGATIVE = -(1LL << 60);
10 |         vector<long long> dp(18, NEGATIVE);
11 |         for (int row = 0; row < 18; ++row) {
12 |             vector<long long> value(row + 1);
13 |             for (long long& x : value)
14 |                 cin >> x;
15 |             for (int column = row; column >= 0; --column) {
16 |                 long long bestParent = NEGATIVE;
17 |                 if (column < row)
18 |                     bestParent = max(bestParent, dp[column]);
19 |                 if (column > 0)
20 |                     bestParent = max(bestParent, dp[column - 1]);
21 |                 if (row == 0)
22 |                     bestParent = 0;
23 |                 dp[column] = bestParent + value[column];
24 |             }
25 |         }
26 |         cout << *max_element(dp.begin(), dp.end()) << '\n';
27 |     }
28 |     return 0;
29 | }
```

## 公开样例

### 样例 1 输入

```
1
61
694 185
439 919 869
539 735 363 96
711 583 963 479 121
295 519 603 332 919 547
443 502 272 550 917 896 829
3 728 748 434 317 143 128 451
791 864 394 777 194 814 804 41 658
676 506 493 963 164 153 415 839 255 754
130 908 109 688 347 412 23 207 609 445 483
725 441 290 527 981 949 97 746 89 406 703 536
72 479 553 732 377 660 772 483 967 683 903 926 389
497 95 688 284 368 819 705 730 184 746 138 421 476 986
434 985 318 906 753 390 133 466 667 349 46 129 895 812 961
399 968 233 382 237 363 787 784 185 538 847 920 166 793 546 661
908 231 234 578 828 144 202 184 14 832 437 126 678 450 442 774 860
802 937 267 468 40 166 240 118 214 107 50 774 779 523 352 974 689 532
```

### 样例 1 输出

```
11949
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.3 | 总 468/526 表面兄弟 ( PID 1362 )

出处：2017级河南工大新生周赛总决赛（重现）（B，CID 1049）；2017级河南工大新生周赛总决赛（重现）（B，CID 1039）；2017级河南工大新生周赛总决赛（B，CID 1038） | 训练定位：基础提高 | 动态规划

标签：网格动态规划、曼哈顿距离、滚动数组 | 限制：1.000 S / 128 MB | 历史统计：16/69 ( 23.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在他将考试座位表给你，想问你他是否有可能在考试中有充足时间作弊并拿到100分

输入要点：单实例测试

输入第一行两个数 $n, m$ 表示班级考试座位 ( $2 \leq n, m \leq 1000$ )

之后 $n$ 行，每行 $m$ 个数，为1表示这个位置上的同学愿意帮忙传纸条

为0表示不愿意帮忙传纸条

其中左上角是LBW，右下角是QAQ，所以这两个位置一定是1

(若数组过大，请定义成全局变量)

输出要点：如果QAQ能作弊成功并且抄完，输出YES

否则输出LBWNB

题面提示：[图片：[https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222150101\\_54595.png](https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222150101_54595.png)]

### 零基础先修

- 动态规划的四件事是状态、转移、初值和答案。零基础阶段先写出状态含义，再考虑维度压缩，避免只背代码。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。

3核心处理采用：左上角到右下角的曼哈顿距离恰好是 $n+m-2$ 。题目不允许多花一秒，所以合法路线不能向上或向左，只能始终向右或向下。  
做一个布尔动态规划：当前座位愿意传纸条且它的上方或左方可达时，当前格可达。最后检查右下角。这比普通 BFS 更准确地利用了严格时间上限。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(nm)$  时间、 $O(m)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。

- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m;
7 |     cin >> n >> m;
8 |     vector<char> reachable(m, false);
9 |     for (int i = 0; i < n; ++i) {
10 |         for (int j = 0; j < m; ++j) {
11 |             int willing;
12 |             cin >> willing;
13 |             if (!willing) {
14 |                 reachable[j] = false;
15 |             } else if (i == 0 && j == 0) {
16 |                 reachable[j] = true;
17 |             } else {
18 |                 bool fromUp = reachable[j];
19 |                 bool fromLeft = j > 0 && reachable[j - 1];
20 |                 reachable[j] = fromUp || fromLeft;
21 |             }
22 |         }
23 |     }
24 |     cout << (reachable[m - 1] ? "YES" : "LBWNB") << '\n';
25 |     return 0;
26 | }
```

## 公开样例

### 样例 1 输入

```
样例1：
7 8
1 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0
1 0 0 1 1 1 1 1
1 1 1 1 0 0 0 1
1 0 0 0 0 0 0 1
1 0 0 0 0 0 0 1
```

```
样例2：
4 4
1 0 0 0
0 0 0 0
0 0 0 0
0 0 0 1
```

### 样例 1 输出

```
样例1：
LBWNB

样例2：
LBWNB
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 8.4 | 总 469/526 阿正的学期准备 ( PID 1540 )

出处：2020级HAUT新生周赛（二）@杨哲专场（B，CID 1061） | 训练定位：基础提高 | 动态规划

标签：枚举、状态压缩、贪心验证 | 限制：1.000 S / 128 MB | 历史统计：85/361 ( 23.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行一个整数，代表阿正购物卡中最多的金币数量。

输入要点：一行一个非负整数，代表阿正打算充值的金币数；

输入保证所有数据都在整形范围内。

输出要点：一行一个整数，代表阿正购物卡中最多的金币数量。

题面提示：阿正可以将200个金币按照以下方法分多次进行充值以获得最大的优惠：

第一次充值6枚，享受第一种优惠，此时阿正的购物卡中共有7枚金币；

第二次充值36枚，享受第二种优惠，此时阿正的购物卡中共有49枚金币；

第三次充值158枚，享受第三种优惠，此时阿正的购物卡中共有225枚金币。

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。
- 当状态由少量二元选择组成时，可用位掩码表示集合；第  $i$  位表示第  $i$  个对象是否选中。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：享受某个优惠至少要为它单独充值对应门槛金额；按门槛从小到大 充值即可同时享受选中的多项优惠。因此枚举 5 种优惠的 32 个子集，若门槛总和不超过本金，就用其奖金总和更新最大值。剩余金币并入任意一次充值，不会减少已经得到的优惠。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

$O(2^5 \cdot 5)$  时间、 $O(1)$  空间

### 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long money;
5 |     cin >> money;
6 |     const long long need[5] = {6, 36, 108, 324, 648};
7 |     const long long bonus[5] = {1, 6, 18, 54, 108};
8 |     long long bestBonus = 0;
9 |     for (int mask = 0; mask < (1 << 5); ++mask) {
10 |         long long cost = 0, gain = 0;
11 |         for (int i = 0; i < 5; ++i) {
12 |             if (mask >> i & 1) {
13 |                 cost += need[i];
14 |                 gain += bonus[i];
```

```

15 |         }
16 |     }
17 |     if (cost <= money)
18 |         bestBonus = max(bestBonus, gain);
19 |     }
20 |     cout << money + bestBonus << '\n';
21 |     return 0;
22 | }

```

## 公开样例

样例 1 输入

200

样例 1 输出

225

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 8.5 | 总 470/526 能量分配 ( PID 1943 )

出处：河南工大2025新生周赛（7）——命题人：牛见青（G，CID 1109） | 训练定位：基础提高 | 动态规划

标签：动态规划、数组与序列 | 限制：1.000 S / 32 MB | 历史统计：9/36 ( 25.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**任务是从这些模块中选择若干，使得总能量消耗不超过核心容量，同时总战斗效益最大化。

**输入要点：**第一行包含两个整数  $E$  ( $1 \leq E \leq 100$ ) 和  $N$  ( $1 \leq N \leq 1000$ )，分别表示初号机能量核心容量与可选模块数。

接下来的  $N$  行，每行包含两个整数  $w_i$  和  $v_i$ ，表示第  $i$  个模块的能量消耗与战斗效益。

**输出要点：**可能有多组测试数据，对于每组数据，

输出只包括一行，这一行只包含一个整数，表示在不超过能量上限的情况下，初号机能获得的最大战斗效益。

## 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：能量分配 本题为动态规划问题中经典的01背包问题。状态转移方程为  $dp[j] = \max(dp[j], dp[j - w[i]] + v[i])$  for (int  $i = 1$ ;  $i \leq N$ ;  $++i$ ) { for (int  $j = E$ ;  $j \geq w[i]$ ;  $--j$ ) {  $dp[j] = \max(dp[j], dp[j - w[i]] + v[i]);$  } } AC代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。



- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(1)$

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int E, N;
7 |     while (cin >> E >> N) {
8 |         if (E == 0 && N == 0)
9 |             break;
10 |         vector<int> dp(E + 1, 0);
11 |         for (int i = 0; i < N; ++i) {
12 |             int w, v;
13 |             cin >> w >> v;
14 |             for (int j = E; j >= w; --j) {
15 |                 dp[j] = max(dp[j], dp[j - w] + v);
16 |             }
17 |         }
18 |         cout << dp[E] << '\n';
19 |     }
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
42 6
1 35
25 70
59 79
65 63
46 6
28 82
962 6
43 96
37 28
5 92
54 3
83 93
17 22
0 0
```

样例 1 输出

```
117
334
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.6 | 总 471/526 非常会伪装的金钱树 ( PID 1983 )

出处：河南工大2025新生周赛（8）——命题人：庞贺航、高旭（E，CID 1110） | 训练定位：基础提高 | 动态规划

标签：动态规划、最长非降子序列、二分 | 限制：1.000 S / 128 MB | 历史统计：36/60 ( 60.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给定一组节点，每个节点有一个权值，要求以如下规则建出一颗高度最大的树，输出该树的高度。（树的高度详见提示）

输入要点：输入共两行。

第一行一个整数 $n$ 。表示共 $n$ 个节点。

第二行共 $n$ 个整数。第 $i$ 个整数表示第 $i$ 个节点的值。

输出要点：输出一个整数。即能建出的树的最大高度。

题面提示： $0 < n \leq 1000$   $0 < n_i \leq 1000$

树的高度：树从根结点开始往下数，叶子结点所在的最大层数称为树的高度

树上的节点有上下两层与其直接相连的节点，在该节点的上层且有唯一与其相连的节点称为该节点的父节点，在该节点的下层且允许有多个与其相连的节点称为该节点的子节点

[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251215/20251215153128\\_40038.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20251215/20251215153128_40038.png)]

/\*这个金钱树真的很会伪装\*/

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：维护  $tails[len-1]$ ：长度为  $len$  的非降子序列目前能取得的最小末尾。当前值可接在所有  $\leq$  它的末尾之后，因此用  $upper\_bound$  找第一个大于当前值的位置并替换；找不到就扩展答案。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

$O(n \log n)$  时间、 $O(n)$  空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回  $int$ 。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
```

```

8 |     vector<long long> tails;
9 |     for (int i = 0; i < n; ++i) {
10 |         long long value;
11 |         cin >> value;
12 |         auto pos = upper_bound(tails.begin(), tails.end(), value);
13 |         if (pos == tails.end())
14 |             tails.push_back(value);
15 |         else
16 |             *pos = value;
17 |     }
18 |     cout << tails.size() << '\n';
19 |     return 0;
20 | }

```

## 公开样例

样例 1 输入

```

6
3 1 2 7 2 4

```

样例 1 输出

```

4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 8.7 | 总 472/526 最大连续子段和 ( PID 1218 )

出处：2016级河南工大新生周赛 ( 三 ) ( E , CID 1008 ) | 训练定位：进阶 | 动态规划

标签：动态规划、Kadane、最大子段和 | 限制：1.000 S / 128 MB | 历史统计：16/119 ( 13.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：给出  $n$  个数，求出  $n$  个数中最大连续子段和。

输入要点：输入分两行，第一行为一个正整数  $n$ ，第二行输入  $n$  个整数  $x_i$ ，

如果最大子段和为负，输出 0。

$2 \leq n \leq 100000$ ， $-1000 \leq x \leq 1000$ 。

输出要点：输出最大连续子段和。

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：Kadane 算法维护“以当前位置结尾、值得继续保留的最大和” current。加入  $x$  后若  $current < 0$ ，就把它重置为 0，因为负前缀只会拖累以后任意子段；每步用 current 更新 answer。answer 初始为 0，恰好满足全负数时输出 0 的特殊要求。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

$O(n)$  时间、 $O(1)$  空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     long long cur = 0, ans = 0;
9 |     for (int i = 0; i < n; ++i) {
10 |         long long value;
11 |         cin >> value;
12 |         cur = max(0LL, cur + value);
13 |         ans = max(ans, cur);
14 |     }
15 |     cout << ans << '\n';
16 |     return 0;
17 | }
```

### 公开样例

|               |
|---------------|
| 样例 1 输入       |
| 5<br>12 4 2 3 |
| 样例 1 输出       |
| 5             |

### 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.8 | 总 473/526 Simple学长取咖啡 ( PID 1234 )

出处：2016级河南工大新生周赛 ( 六 ) ( E , CID 1012 ) | 训练定位：进阶 | 动态规划

标签：动态规划、递推、预处理 | 限制：1.000 S / 128 MB | 历史统计：21/131 ( 16.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个数字表示有多少种方法可以取到咖啡。（最后的数字有可能很大哦）

输入要点：输入一个T，表示有T组数据

每一组数据输入一个数字 $n$ ，表示Simple学长的咖啡在第 $n$ 阶梯子上 ( $n \leq 40$ )

输出要点：输出一个数字表示有多少种方法可以取到咖啡。（最后的数字有可能很大哦）

## 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：到第  $n$  阶的最后一步只能从  $n-1$ 、 $n-2$  或  $n-3$  阶跳来，因此  $ways[n]=ways[n-1]+ways[n-2]+ways[n-3]$ 。设  $ways[0]=1$  表示站在起点的一种空方案，负下标为 0，就得到  $ways[1]=1$ 、 $ways[2]=2$ 、 $ways[3]=4$ 。预处理到 40 后逐询问输出。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

预处理  $O(40)$ ，每次询问  $O(1)$ ，空间  $O(40)$

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     array<unsigned long long, 41> ways{};
7 |     ways[0] = 1;
8 |     for (int stair = 1; stair <= 40; ++stair)
9 |         for (int jump = 1; jump <= 3; ++jump)
10 |             if (stair >= jump)
11 |                 ways[stair] += ways[stair - jump];
12 |     int T;
13 |     cin >> T;
14 |     while (T--) {
15 |         int n;
16 |         cin >> n;
17 |         cout << ways[n] << '\n';
18 |     }
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
3
1
2
3
```

样例 1 输出

```
1
2
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 8.9 | 总 474/526 趣味游戏 ( 六 ) : 爬楼梯 ( PID 1334 )

出处：2017级河南工大新生周赛 ( 四 ) ( F , CID 1031 ) | 训练定位：进阶 | 动态规划

标签：动态规划、斐波那契推广、取模 | 限制：1.000 S / 128 MB | 历史统计：5/39 ( 12.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组样例输出一个整数 表示走到第  $n$  层楼可以有多少种不同的走法。

输入要点：第一行  $T$  表示样例组数

每组样例第一行两个整数  $n$  和  $m$ ,  $n$  表示要去的楼层,  $m$  表示第一层楼到第  $n$  层楼之间有  $m$  阶台阶 (  $0 < n \leq 100, 0 < m \leq 10000$  )

输出要点：每组样例输出一个整数 表示走到第  $n$  层楼可以有多少种不同的走法。

题面提示：<1> $m$  阶台阶可以看成是连续的。

<2>结果对 1000000007 取模。

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 取模用于周期或大数；减法后可写  $(x-y+\text{mod})\% \text{mod}$  防止出现负数。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 3 核心处理采用：到达一段连续  $n$  阶楼梯的最后位置，每一步可走 1、2、3 阶。设  $\text{ways}[i]$  为恰好走完阶的方案数，最后一步分别来自  $i-1$ 、 $i-2$ 、 $i-3$ ，所以  $\text{ways}[i] = \text{ways}[i-1] + \text{ways}[i-2] + \text{ways}[i-3]$ ， $\text{ways}[0] = 1$ ，负下标视为 0。楼层编号  $n$  只提供背景，实际方案只由台阶数  $m$  决定。结果每步取模。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

每组  $O(m)$  时间、 $O(m)$  空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- C++ 负数取模仍可能为负；减法后补模数。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     const long long MOD = 1000000007LL;
```

```

7 |     int T;
8 |     cin >> T;
9 |     while (T--) {
10 |         int floorNumber, stairs;
11 |         cin >> floorNumber >> stairs;
12 |         vector<long long> ways(stairs + 1, 0);
13 |         ways[0] = 1;
14 |         for (int i = 1; i <= stairs; ++i) {
15 |             for (int step = 1; step <= 3; ++step)
16 |                 if (i >= step)
17 |                     ways[i] = (ways[i] + ways[i - step]) % MOD;
18 |             cout << ways[stairs] << '\n';
19 |         }
20 |     }
21 |     return 0;
22 | }

```

## 公开样例

样例 1 输入

```

2
4 10
4 8

```

样例 1 输出

```

274
81

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 8.10 | 总 475/526 Lighting ( PID 1367 )

出处：2017级河南工大新生周赛总决赛（重现）（G，CID 1049）；2017级河南工大新生周赛总决赛（重现）（G，CID 1039）；2017级河南工大新生周赛总决赛（G，CID 1038） | 训练定位：进阶 | 动态规划

标签：状态压缩、位枚举、模拟 | 限制：1.000 S / 128 MB | 历史统计：5/55 (9.1%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，如果能把灯全部关掉输出YES 否则输出QAQ

输入要点：输入一个数n表示一排n个灯（ $1 \leq n \leq 16$ ）

接下来n个数(0或1)，0表示灯是关的，1表示灯是亮着的

输出要点：如果能把灯全部关掉输出YES

否则输出QAQ

题面提示：[图片：[https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222165420\\_48542.png](https://acm.haut.edu.cn/JudgeOnline/upload/image/20171222/20171222165420_48542.png)]

## 零基础先修

- 当状态由少量二元选择组成时，可用位掩码表示集合；第 i 位表示第 i 个对象是否选中。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。

3. 核心处理采用：同一个按钮按两次会抵消，所以每个按钮只需考虑按或不按，共  $2^n$  种方案。枚举一个二进制掩码，复制初始灯态；若第  $i$  位为 1，就翻转  $i-1$ 、 $i$ 、 $i+1$  三个仍在范围内的灯。只要某个方案得到全 0 就输出 YES。 $n \leq 16$ ，最多约一百万次翻转，足够快，也很适合作为位枚举入门。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(n \cdot 2^n)$  时间、 $O(n)$  空间

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<int> initial(n);
9 |     for (int i = 0; i < n; ++i) {
10 |         char bit;
11 |         cin >> bit;
12 |         initial[i] = bit - '0';
13 |     }
14 |     for (int mask = 0; mask < (1 << n); ++mask) {
15 |         vector<int> light = initial;
16 |         for (int i = 0; i < n; ++i) {
17 |             if (!(mask >> i & 1))
18 |                 continue;
19 |             for (int j = max(0, i - 1); j <= min(n - 1, i + 1); ++j)
20 |                 light[j] ^= 1;
21 |         }
22 |         if (all_of(light.begin(), light.end(), [](int x) {
23 |             return x == 0;
24 |         })) {
25 |             cout << "YES\n";
26 |             return 0;
27 |         }
28 |     }
29 |     cout << "NO\n";
30 |     return 0;
31 | }
```

## 公开样例

样例 1 输入

```
8
10001011
```

样例 1 输出

```
YES
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 8.11 | 总 476/526 元气满满 ( PID 1405 )

出处：2018级河南工大新生周赛 ( 四 ) @杜锐专场 ( D , CID 1043 ) | 训练定位：进阶 | 动态规划

标签：状态压缩、枚举、字典序 | 限制：1.000 S / 128 MB | 历史统计：4/41 ( 9.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，每组数据对应一行输出。一行输出是7个整数，0或1，0表示当天不买，1表示当天买。有多种情况时输出二进制数最大的一组情况 ( 如1100000 > 1010000 )

输入要点：第一行是一个整数N，其后有N行。

每行的第一个数是整数S (  $0 \leq S \leq 10$  )，代表卡特莉这个星期的零花钱。其后有7个大写英文字母 ( A、B或者C )，表示当天售卖的点心品类。

输出要点：每组数据对应一行输出。

一行输出是7个整数，0或1，0表示当天不买，1表示当天买。

有多种情况时输出二进制数最大的一组情况 ( 如1100000 > 1010000 )

### 零基础先修

- 当状态由少量二元选择组成时，可用位掩码表示集合；第  $i$  位表示第  $i$  个对象是否选中。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：三种甜品的“元气时长/价格”都相同，因此最长总时长等价于在预算内花掉尽可能多的钱。把价格乘4化为  $A=4$ 、 $B=2$ 、 $C=3$  的整数，枚举7天的128个购买子集；花费更大优先，花费相同则把第1天作为最高二进制位比较，选择字典序最大的0/1方案。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

每组  $O(2^7 \cdot 7)$  时间、 $O(1)$  空间

### 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

### C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int T;
5 |     cin >> T;
6 |     while (T--) {
7 |         int budget;
8 |         cin >> budget;
9 |         array<char, 7> type{};
10 |        for (char& c : type)
11 |            cin >> c;
12 |        int bestCost = -1, bestCode = -1;
13 |        for (int mask = 0; mask < 128; ++mask) {
14 |            int cost = 0, code = 0;
15 |            for (int day = 0; day < 7; ++day) {
16 |                bool buy = mask >> day & 1;
17 |                code = code * 2 + buy;
18 |                if (buy)
19 |                    cost += type[day] == 'A' ? 4 : type[day] == 'B' ? 2 : 3;
20 |            }
21 |            if (cost <= 4 * budget &&
22 |                (cost > bestCost || (cost == bestCost && code > bestCode))) {
23 |                bestCost = cost;
24 |                bestCode = code;
25 |            }
26 |        }
27 |        for (int day = 6; day >= 0; --day) {
28 |            if (day != 6)
29 |                cout << ' ';
30 |            cout << (bestCode >> day & 1);
31 |        }
32 |        cout << '\n';
33 |    }
34 |    return 0;
35 | }

```

## 公开样例

样例 1 输入

```

3
10ABCCABBB
3AABBCACB
3BCBBAAC

```

样例 1 输出

```

11111111
1110001
1111001

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 8.12 | 总 477/526 买东西 (PID 1527)

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 (G, CID 1059) | 训练定位：进阶 | 动态规划

标签：条件分支、排序、动态规划、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/70 (2.9%)

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：某天，商城中有  $n$  种商品出售，每种商品可购买一次。已知每种商品的价格以及卡上的余额，问最少可使卡上的余额为多少。

输入要点：有多组数据。对于每组数据：

第一行只有一个正整数  $n$ ，表示商品的数量。 $n \leq 1000$ 。

然后第二行包括 $n$ 个正整数，表示每种商品的价格。价格不超过50。

最后一行只有一个正整数 $m$ ，表示卡上的余额。 $m \leq 1000$ 。

$n=0$ 表示数据结束。

输出要点：对于每组输入,输出一行,包含一个整数，表示卡上可能的最小余额。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：买东西  $n$  道菜，选出最贵的，放到最后买，然后在保留5元的情况下，用 $n-5$ 元买剩下的才转化为背包问题。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。
- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | const int maxn = 1e4 + 7;
4 | const int INF = 0x3f3f3f3f;
5 | int value[maxn], size1[maxn], dp[maxn];
6 | int main() {
7 |     int n;
8 |     while (cin >> n && n != 0) {
9 |         memset(dp, 0, sizeof(dp));
10 |         int x;
11 |         for (int i = 1; i <= n; i++)
12 |             cin >> value[i];
13 |         cin >> x;
14 |         if (x < 5) {
15 |             cout << x << endl;
16 |             continue;
17 |         }
18 |         sort(value + 1, value + 1 + n);
19 |         for (int i = 1; i < n; i++) {
20 |             for (int j = x - 5; j >= value[i]; j--)
21 |                 dp[j] = max(dp[j], dp[j - value[i]] + value[i]);
22 |             }
23 |         cout << x - dp[x - 5] - value[n] << endl;
24 |     }
25 |     return 0;
26 | }
```

## 公开样例

样例 1 输入

```
1
50
5
10
1 2 3 2 1 1 2 3 2 1
50
0
```

样例 1 输出

```
-45
32
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 8.13 | 总 478/526 选择 ( PID 1554 )

出处：2020级HAUT新生周赛（三）@张雍薜专场（H，CID 1062） | 训练定位：进阶 | 动态规划

标签：状态压缩、枚举、子集和 | 限制：1.000 S / 128 MB | 历史统计：1/40 ( 2.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：你的目标是使得这s个元素的和的绝对值最小。

输入要点：第一行为n，代表序列的长度

接下来一行n个整数，代表序列的元素

$1 \leq n \leq 20$

$-1e6 \leq a[i] \leq 1e6$

输出要点：一个整数，代表最小的绝对值

题面提示：选择第一个和第四个元素

## 零基础先修

- 当状态由少量二元选择组成时，可用位掩码表示集合；第i位表示第i个对象是否选中。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用： $n \leq 20$ ，可以枚举除空集外的全部  $2^n - 1$  个子集。用 lowbit 找到 新增的最低位，并由去掉该位后的子集和递推当前和，可把总复杂度从  $O(n2^n)$  降为  $O(2^n)$ 。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(2^n)$  时间、 $O(2^n)$  空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     vector<long long> a(n);
7 |     for (long long& x : a)
8 |         cin >> x;
9 |     int total = 1 << n;
10 |    vector<long long> subsetSum(total, 0);
11 |    long long ans = LLONG_MAX;
12 |    for (int mask = 1; mask < total; ++mask) {
13 |        int bit = __builtin_ctz(mask);
14 |        int pre = mask & (mask - 1);
15 |        subsetSum[mask] = subsetSum[pre] + a[bit];
16 |        ans = min(ans, labs(subsetSum[mask]));
17 |    }
18 |    cout << ans << '\n';
19 |    return 0;
20 | }
```

## 公开样例

样例 1 输入

```
5
-2 5 12 4 -40
```

样例 1 输出

```
2
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 8.14 | 总 479/526 界面越花 编程越拉 ( PID 1717 )

出处：2021级HAUT新生周赛（六）@李志豪专场（A，CID 1074） | 训练定位：进阶 | 动态规划

标签：数论、动态规划、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：16/132 ( 12.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：现给出小m的初始华丽值n，请输出使华丽值变为0的方案数。由于方案数可能过多，所以请对最终答案取模 $1e9+7$ 。

输入要点：一个正整数n。(n <= 1e5)

输出要点：一个整数。

## 零基础先修

- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：界面越花 编程越拉 标签:动态规划 mid题目大意:给你一个数  $n$ ， $n$  可以从  $n$  的状态转移过来，也可以用  $n - n - 1$  的状态转移过来，问从 0 开始能有多少种方案。 $n \geq 2$  思路：如果能够把题意理解到上述意思，其实解法就很明确了，就是一个简单的递推，每个值由两个状态转移得到。程序表达如下： $dp[i] = dp[i - 1] + dp[i / 2]$ 同样可以使用记忆化搜索来解决这个问题（不使用记忆化的话会因为数据量过大而 T）DP 的标程如下
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(n)$

## 易错点

- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。
- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define INF 0x3f3f3f3f
4 | #define ll long long
5 | #define endl "\n"
6 | const int max_n = 1e5 + 100;
7 | const int mod = 1e9 + 7;
8 | int ans = 0;
9 | int dp[max_n]; // dp[i]为初始值为i的方案数
10 | int main() {
11 |     ios::sync_with_stdio(false);
12 |     cin.tie(0);
13 |     cout.tie(0);
14 |     int n;
15 |     cin >> n;
16 |     dp[1] = 2;
17 |     for (int i = 2; i <= n; i++)
18 |         dp[i] = (dp[i - 1] + dp[i / 2]) % mod;
19 |     cout << dp[n] << endl;
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

3

样例 1 输出

6

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.15 | 总 480/526 小C爱编程 ( PID 1744 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（B，CID 1077）| 训练定位：进阶 | 动态规划

标签：动态规划 | 限制：1.000 S / 128 MB | 历史统计：2/37 ( 5.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在，小C手头一共还有 $n$ 项作业没写完，他知道完成每项作业所需的时间，请问，他最少还需要多长时间才能去打代码（即写完这些作业所需的最短时间）？

输入要点：第一行输入一个整数 $n(1 \leq n \leq 1000)$ ，即现在小C现在有 $n$ 项作业需要完成

第二行包含 $n$ 个整数，即小C完成每一项作业所需的时间。对于每份作业的完成时间 $t_i(1 \leq t_i \leq 200)$

输出要点：输出一个整数，即小C在同一时间能写两份作业的情况下，写完这些作业所需的最短时间

题面提示：对于样例数据，小C的左手写第1、3、4份作业；右手写第2、5份作业，即可在18分钟完成所有作业

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：小C爱编程这道题是一道背包题（刚发现这题与昨天郑轻的10星题口嘴猫玩卡牌是差不多的）。也许这题会有同学用贪心写，即先将作业时间按大到小排序，然后再依次将作业分给左右手中目前写作业耗时少的那只手，最后求出答案。这样的方法只能求出近似最优解，并不是完全正确的，这里提供一个错误样例：6 105 88 118 36 70 77 对于这个数据，那种方法是分为118、77、36；105、88、70两组，算出答案为263但正确答案应当分为105、77、70；118、88、36两组，答案为252对于这题，我们可以用背包统计出左手分配到不同作业耗时的所有情况，然后再用写作业的总时长减去当前左手当前情况的时间，即可得到右手写作业的时间。对于每一种情况，对两只手取最大值，计算最大值最小即可。那么，最重要的还是怎么用背包来表示出左手分配到作业的所有情况。我们用  $dp[i][j]$  表示前 $i$ 份作业中，有没有组成 $j$ 分钟的作业组合，有则是1，没有则是0。 $dp[i][j]$ 初始情况是，没有作业的情况下，耗时是0分钟的可以实现的。 $dp[0][0]=1, dp[0][j]=0$ 。后面要考虑怎么状态转移了，这里就直接把转移的式子给出来： $dp[0][0]=1; for(int i=1; i \leq n; i++) for(int j=0; j \leq sum; j++) //sum指的是所有作业所需的总时间 { if(dp[i-1][j]==1) //表示前i-1份作业，存在可以组成j分钟的情况 {  $dp[i][j]=1$ ; //如果当前这份作业我们不用左手写，即代表前i份，也可以组成j分钟的情况  $dp[i][j+a[i]]=1$ ; //如果当前这一份作业用左手写，那么代表前i份作业，可以组成 $j+a[i]$ 分钟的这种情况 //（ $a[i]$ 是当前这份作业所用的时间） } }（这里如果不理解可以自己拿笔模拟一下题目样例的过程）通过二维的转移方程了解完怎么状态转移后，我们需要知道的是，其实二维数组在这题并不可行：本题需要的数组是很容易会出现内存超限的情况的。 $dp[1005][200005]$ 通过观察我们可以发现，其实这个转移方程，每次转移，也就只用了第 $i-1$ 行和第 $i$ 行，最后我们需要的也只有第 $n$ 行的数据，所以，我们可以试图将其降维，降成只有两行的二维数组，这样就不会有问题了：//第0行即代表前面的前 $i-1$ 份作业能产生的所有情况，第1行即代表前面的前 $i$ 份作业能产生的所有情况  $dp[0][0]=1; for(int i=1; i \leq n; i++) { for(int j=0; j \leq sum; j++) { if(dp[0][j]==1) {  $dp[1][j]=1$ ;  $dp[1][j+a[i]]=1$ ; } } for(int j=0; j \leq sum; j++) //对于下一行来说，当前行的结果则是其上一行的结果  $dp[0][j]=dp[1][j];$  } }要再进行优化，我们可以将数组降成一维：我们可以将第二层循环倒过来，即从 $sum$ 循环到0： $dp[0]=1; for(i=1; i \leq n; i++) for(j=sum; j \geq a[i]; j--) { if(dp[j-a[i]]==1)  $dp[j]=1$ ; } }这样做，第 $j$ 位后面的都是当前行的结果，而前面则仍是上一行的结果，那我们前面的  $if(dp[j-a[i]]==1) dp[j]=1$ ; 在这就可以写成这样了： $if(dp[j-a[i]]==1) //代表着前i-1份作业能组成j-a[i]分钟这种情况 dp[j]=1; //即前i份作业能组成j分钟这种情况$  前面这一大段，在试图讲清背包和背包是如何降维的 如果没弄懂，可以去听听y总讲的：视频定位15分04秒到这，我们也算是已经用背包把左手的所有情况都找出来了，那对于每种情况，我们用  $dp[j]$  即可表示出右手写作业需要的时间了，那么两者取较大的那个则是当前情况下，小C写作业需要的时间了。 $sum-isum-isum-i$ 对于每种情况，取写作业时间最短的时间即可 上代码$$$

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

由状态数  $\times$  单状态转移数决定，需结合题目范围核算

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, sum = 0, ans = 0x3f3f3f3f, a[1005], dp[200005];
3 | int main() {
4 |     scanf("%d", &n);
5 |     for (int i = 1; i <= n; i++) {
6 |         scanf("%d", &a[i]);
7 |         sum += a[i];
8 |     }
9 |     dp[0] = 1;
10 |    for (int i = 1; i <= n; i++)
11 |        for (int j = sum; j >= a[i]; j--) {
12 |            if (dp[j - a[i]])
13 |                dp[j] = 1;
14 |        }
15 |    for (int i = 1; i <= sum; i++) {
16 |        if (dp[i]) {
17 |            int l = i, r = sum - i, k; // 左右手当前情况写作业的时间
18 |            if (l > r)
19 |                k = l; // k为写作业的时间，即左右手写作业时间的较大者
20 |            else
21 |                k = r;
22 |            if (ans > k)
23 |                ans = k; // 每种情况都对答案进行更新
24 |        }
25 |    }
26 |    printf("%d", ans);
27 |    return 0;
28 | }
```

## 公开样例

样例 1 输入

```
5
2 8 15 1 10
```

样例 1 输出

```
18
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)



## 8.16 | 总 481/526 Stay with you ( PID 1772 )

出处：2022级HAUT新生周赛（三）@焦小航专场（C，CID 1081）| 训练定位：进阶 | 动态规划

标签：动态规划、字符串、排序 | 限制：3.000 S / 128 MB | 历史统计：4/64 ( 6.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出最长单词链的长度

输入要点：第一行给出  $n$  (  $1 \leq n \leq 2000$  )

接下来  $n$  行，每行一个字符串  $S$  (  $1 \leq |S| \leq 20$  )， $|S|$  代表字符串的长度。

输出要点：输出最长单词链的长度

题面提示：给出的单词表不一定是按字典序排序，可能是无序的。

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- string 可按下表访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：把单词按长度从短到长排序后，任何可能的前驱都已出现在当前词之前。令  $dp[i]$  为以第  $i$  个词结尾的最长链；枚举所有更短或等长的旧词  $j$ ，若  $word[j]$  是  $word[i]$  的前缀，则可用  $dp[j]+1$  更新。相同单词也允许相邻，题面示例明确给出了重复 staywith。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

$O(n^2L)$  时间， $L \leq 20$ ； $O(n)$  额外空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<string> word(n);
9 |     for (string& s : word)
10 |         cin >> s;
11 |     stable_sort(word.begin(), word.end(), [](const string& a, const string& b) {
12 |         return a.size() < b.size();
13 |     });
14 |     vector<int> dp(n, 1);
15 |     int ans = 1;
16 |     for (int i = 0; i < n; ++i) {
```

```

17 |         for (int j = 0; j < i; ++j) {
18 |             if (word[j].size() <= word[i].size() &&
19 |                 word[i].compare(0, word[j].size(), word[j]) == 0) {
20 |                 dp[i] = max(dp[i], dp[j] + 1);
21 |             }
22 |         }
23 |         ans = max(ans, dp[i]);
24 |     }
25 |     cout << ans << '\n';
26 |     return 0;
27 | }

```

## 公开样例

样例 1 输入

```

5
stay
staywith
staywithy
staywithyo
staywithyou

```

样例 1 输出

```

5

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 8.17 | 总 482/526 签到题 ( PID 1779 )

出处：2022级HAUT新生周赛（四）@张子豪专场（D，CID 1082）| 训练定位：进阶 | 动态规划

标签：动态规划、最大公约数、状态压缩思想 | 限制：1.000 S / 128 MB | 历史统计：3/23 ( 13.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**现在请你求出通过任意次上述操作使数组中所有数字的最大公约数等于1的最小成本。(当数组只有一个数时，所有数字的最大公约数按这个数字本身算)

**输入要点：**第一行，一个正整数 $n$ ，数组元素数量。  $1 \leq n \leq 20$

**第二行，** $n$ 个正整数，  $1 \leq a[i] \leq 1e9$

**输出要点：**一行，一个正整数，所需要的最小成本

**题面提示：**样例解释：

在样例中，你可以对 $a[4]$ 和 $a[5]$ 进行操作，在两次操作后数组变为  $[120,60,80,4,5]$ ，此时数组中所有元素的最大公约为1，需要的成本为3。

## 零基础先修

- 先用一句话定义  $dp$  状态，再写转移来源、初值、遍历顺序和最终答案。
- `std::gcd(a,b)` 返回最大公约数，并满足  $\gcd(a,b)=\gcd(b,a \bmod b)$ 。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义  $dp$  状态的精确含义，列出初值，并只从已经合法的状态转移。

- 核心处理采用：对每个下标  $i$  只有“不操作”和“把  $a[i]$  变成  $\gcd(a[i], i)$ ”两种 有意义的选择，重复操作不会再改变它。动态规划的状态是已处理前缀的 总  $\gcd$ ，值是最小成本；加入当前元素时分别执行两种转移。最终  $\gcd=1$  的最小成本就是答案。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(n \cdot S \cdot \log S)$  时间、 $O(S)$  空间， $S$  为过程中不同  $\gcd$  的数量

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回  $\text{int}$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     cin >> n;
8 |     vector<long long> value(n + 1);
9 |     for (int i = 1; i <= n; ++i)
10 |         cin >> value[i];
11 |     map<long long, long long> dp;
12 |     dp[0] = 0;
13 |     for (int i = 1; i <= n; ++i) {
14 |         map<long long, long long> next;
15 |         auto relax = [&](long long divisor, long long cost) {
16 |             auto pos = next.find(divisor);
17 |             if (pos == next.end() || cost < pos->second) {
18 |                 next[divisor] = cost;
19 |             }
20 |         };
21 |         for (auto [g, currentCost] : dp) {
22 |             relax(gcd(g, value[i]), currentCost);
23 |             long long changed = gcd(value[i], static_cast<long long>(i));
24 |             relax(gcd(g, changed), currentCost + (n - i + 1));
25 |         }
26 |         dp.swap(next);
27 |     }
28 |     cout << dp[1] << '\n';
29 |     return 0;
30 | }
```

## 公开样例

样例 1 输入

```
5
120 60 80 40 80
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.18 | 总 483/526 勇士的末路 ( PID 1795 )

出处：2022级HAUT新生周赛（六）@张鑫专场（B，CID 1084） | 训练定位：进阶 | 动态规划

标签：动态规划、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：1/25 ( 4.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：城堡是一个矩形并且由 $n$ 行 $m$ 列个 $1*1$ 的小房间组成， $(x,y)$ 表示第 $x$ 行 $y$ 列的房间，入口位置在 $(1,1)$ 位置，出口在 $(n,m)$ 位置。哈利因为和巨龙长时间战斗身上的能量只剩下 $w$ 个了，而每进入一个房间都需要消耗单位1个能量（包括出入口）且哈利只能向行数或列数增加的方向移动，每个房间都有 $a$ 枚金币，哈利需要收集尽量多的金币为了保证有足够的的路费回家。请问哈利是否能逃出……

输入要点：第一行三个正整数 $n,m,w$ 。接下来 $n$ 行每行有 $m$ 个整数表示对应位置房间的金币数。

输出要点：如果可以逃出城堡输出收集的最大金币数，反之输出-1。

题面提示： $1 \leq n,m \leq 1000$ ， $1 \leq w \leq 5000$ ， $1 \leq x \leq n$ ， $1 \leq y \leq m$ ， $0 \leq a \leq 1e9$ ;

（结果可能需要开longlong）

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：勇士的末路（递推/动态规划）去到某个房间的上一个房间只能由上方或左侧的房间进入，所以每个房间的最大金币数为  $dp[i][j] = \max(dp[i-1][j], dp[i][j-1]) + x$ ， $i,j$ 分别为房间所在的行和列， $x$ 为当前房间金币数。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

由状态数  $\times$  单状态转移数决定，需结合题目范围核算

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | typedef long long int LL;
3 | LL dp[1010][1010] = {0}; // 明显会爆int所以开longlong
4 | LL max(LL a, LL b) {
5 |     if (a > b)
6 |         return a;
7 |     else
8 |         return b;
9 | }
10 | int main() {
11 |     int n, m, w;
12 |     scanf("%d %d %d", &n, &m, &w);
```

```

13 |     for (int i = 1; i <= n; i++) {
14 |         for (int j = 1; j <= m; j++) {
15 |             int x;
16 |             scanf("%d", &x);
17 |             dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) + x; // 状态转移方程
18 |             // 每个房间的最大金币数只能由上或左的房间得到
19 |         }
20 |     }
21 |     if (w < n + m - 1)
22 |         printf("-1\n"); // 消耗能量固定为n+m-1
23 |     else
24 |         printf("%lld\n", dp[n][m]);
25 |     return 0;
26 | }

```

## 公开样例

### 样例 1 输入

```

2 5 100
1 2 3 4 5
1 2 3 4 5

```

### 样例 1 输出

```

20

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 8.19 | 总 484/526 大富翁！启动！（PID 1849）

出处：2023级HAUT新生周赛（四）@牛浩然专场（E，CID 1089）| 训练定位：进阶 | 动态规划

标签：动态规划、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：3/33（9.1%）

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，一个整数，表示小A能在该局游戏中获得的最大收益。

**输入要点：**第一行一个整数，代表一共n个格子（ $0 < n \leq 100000$ ）。

第二行n个整数 $a_1 a_2 a_3 a_4 \dots a_n$ ，第i个数代表第i个格子的收益（ $-1000000 \leq a_i \leq 1000000$ ）。

**输出要点：**一个整数，表示小A能在该局游戏中获得的最大收益。

题面提示：获得的金币可以为负

## 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：大富翁！启动！影响最终收益的有当前所处的位置和当前已经获取的金币。考虑动态规划，dp[i]表示到第i个点时小A可以获取多少收益。那么dp[i]的最大值会从dp[i-1], dp[i-2], dp[i-3], dp[i-4], dp[i-5], dp[i-6]中的最大值+第i格获取的金币。dp[n+1]即为终点。注意，由于获取的金币可以为负数，需要初始化dp[i]为最小值。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

由状态数  $\times$  单状态转移数决定，需结合题目范围核算

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | long long arr[110000];
3 | long long dp[110000];
4 | int main() {
5 |     int n;
6 |     scanf("%d", &n);
7 |     for (int i = 1; i <= n; ++i) {
8 |         scanf("%lld", &arr[i]);
9 |     }
10 |    for (int i = 1; i <= n + 10; ++i) {
11 |        dp[i] = -1e12;
12 |    }
13 |    for (int i = 1; i <= n + 1; ++i) {
14 |        for (int j = i - 1; j >= 0 && j >= i - 6; --j) {
15 |            if (dp[j] > dp[i])
16 |                dp[i] = dp[j];
17 |        }
18 |        dp[i] += arr[i];
19 |    }
20 |    printf("%lld\n", dp[n + 1]);
21 |    return 0;
22 | }
```

## 公开样例

样例 1 输入

```
5
1 2 -10 3 4
```

样例 1 输出

```
10
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 8.20 | 总 485/526 建房子 ( PID 1909 )

出处：河南工大2024新生周赛（7）——命题人：刘义（F，CID 1098） | 训练定位：进阶 | 动态规划

标签：0/1背包、动态规划、最小代价 | 限制：1.000 S / 128 MB | 历史统计：11/82 ( 13.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：判断LY能把房子建好吗？如果能，判断LY还剩下的最大体力。

输入要点：输入文件的第一行是三个整数：v n c。

从第二行到第 n+1 行分别为每块石头的体积和把它搬到施工场地需要的体力。

输出要点：输出文件只有一行，如果LY能把房子建完，则输出LY还剩下的最大体力，否则输出 Impossible（不带引号）

题面提示：样例输入2：

```
10 2 1
50 5
10 2
```

样例输出2：

```
Impossible
```

$0 < n \leq 104$ ，所有读入的数均属于  $[0, 104]$ ，最后答案不大于 c。

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：把“已消耗体力”当作 0/1 背包容量，dp[j] 表示恰好在不超过 j 体力限制下可搬到的最大体积。每块石头倒序更新，之后找到使体积至少为 v 的最小消耗 j，最大剩余体力就是 c-j；找不到则输出 Impossible。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

$O(nc)$  时间、 $O(c)$  空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int need, n, stamina;
7 |     cin >> need >> n >> stamina;
8 |     vector<int> mx(stamina + 1, 0);
9 |     for (int i = 0; i < n; ++i) {
10 |         int volume, cost;
11 |         cin >> volume >> cost;
```

```
12 |         for (int used = stamina; used >= cost; --used) {
13 |             mx[used] = max(mx[used], mx[used - cost] + volume);
14 |         }
15 |     }
16 |     for (int used = 0; used <= stamina; ++used) {
17 |         if (mx[used] >= need) {
18 |             cout << stamina - used << '\n';
19 |             return 0;
20 |         }
21 |     }
22 |     cout << "Impossible\n";
23 |     return 0;
24 | }
```

## 公开样例

样例 1 输入

```
100 2 10
50 5
50 5
```

样例 1 输出

```
0
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 8.21 | 总 486/526 耐久魔法 ( PID 1946 )

出处：河南工大2025新生周赛（7）——命题人：牛见青（H，CID 1109）| 训练定位：进阶 | 动态规划

标签：动态规划、数学、数论、排序、数组与序列 | 限制：4.000 S / 256 MB | 历史统计：0/22 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：有一天，海拉鲁的女神给了林克一个神秘的“强化试炼”——每次试炼，林克都会随机选择一段连续的武器（编号为  $[l, r]$ ），并用其中最结实的一把来强化整个区间，使得这段区间内所有武器的耐久都变成该区间的最大耐久值。

输入要点：输入共两行：

-

第一行包含两个正整数  $n, q$ ，分别表示武器的数量和强化试炼的次数。

-

第二行包含  $n$  个非负整数  $a_1, a_2, \dots, a_n$ ，表示每把武器的初始耐久度。

输出要点：输出一行，包含  $n$  个整数，第  $i$  个数表示第  $i$  把武器的答案，即期望耐久度乘上  $(n(n+1)/2)q$  并对  $10^9+7$  取模的结果。

题面提示：-

$1 \leq n \leq 400$

-

$1 \leq q \leq 400$

-

$0 \leq a_i \leq 10^9$

-



每次操作选择的区间  $[l, r]$  在所有  $n(n+1)/2$  个区间中等概率随机选取。

## 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 数论常用整除、gcd/lcm、质数、同余；乘法前要判断是否需要 long long 或 \_\_int128。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：耐久魔法 本题考查线段树 `#include <bits/stdc++.h>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

通常为  $O(n \log n)$ ，主要来自排序

## 易错点

- 初值不能默认全 0 可达；0/1 背包要倒序遍历容量。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- gcd/lcm 的 0 值、负数和乘法溢出都要单独核对。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | constexpr int N = 405;
4 | constexpr int MOD = 1000000007;
5 | struct Node {
6 |     int id;
7 |     int w;
8 | };
9 | int n, m;
10 | int ans[N];
11 | int dp[N][N];
12 | int s1[N][N], s2[N][N];
13 | bool used[N];
14 | Node a[N];
15 | inline int addMod(int x, int y) {
16 |     x += y;
17 |     return (x >= MOD) ? x - MOD : x;
18 | }
19 | inline void incMod(int& x, int y) {
20 |     x += y;
21 |     if (x >= MOD)
22 |         x -= MOD;
23 | }
24 | int qPow(int x, int y) {
25 |     int res = 1;
26 |     while (y) {
27 |         if (y & 1)
28 |             res = 1LL * res * x % MOD;
29 |         x = 1LL * x * x % MOD;
30 |         y >>= 1;
31 |     }
32 |     return res;
33 | }
34 | inline bool cmpNode(const Node& x, const Node& y) {
35 |     return x.w < y.w;
36 | }
37 | // 原 f(l, r) 函数
```

```

38 | inline int F(int l, int r) {
39 |     return (l * (l + 1) + (n - r + 1) * (n - r + 2) + (r - l - 1) * (r - l)) / 2;
40 | }
41 | int main() {
42 |     if (scanf("%d %d", &n, &m) != 2) {
43 |         return 0;
44 |     }
45 |     used[n + 1] = true;
46 |     for (int i = 1; i <= n; ++i) {
47 |         scanf("%d", &a[i].w);
48 |         a[i].id = i;
49 |     }
50 |     sort(a + 1, a + n + 1, cmpNode);
51 |     // 预处理 dp
52 |     for (int i = n, last; i >= 1; --i) {
53 |         last = 0;
54 |         used[a[i].id] = true;
55 |         for (int j = 1; j <= n + 1; ++j) {
56 |             if (used[j]) {
57 |                 dp[last][j] += a[i].w - a[i - 1].w;
58 |                 last = j;
59 |             }
60 |         }
61 |     }
62 |     ans[1] = 1LL * a[n].w * qPow(n * (n + 1) / 2, m) % MOD;
63 |     for (int i = 2; i <= n; ++i) {
64 |         ans[i] = ans[1];
65 |     }
66 |     // 做 m 次转移
67 |     for (int step = 1; step <= m; ++step) {
68 |         // 计算前缀和 s1, s2
69 |         for (int j = 0; j <= n; ++j) {
70 |             for (int k = n + 1; k > j + 1; --k) {
71 |                 int val = dp[j][k];
72 |                 s1[j][k] = addMod(j ? s1[j - 1][k] : 0, 1LL * val * j % MOD);
73 |                 s2[j][k] = addMod(k <= n ? s2[j][k + 1] : 0, 1LL * val * (n - k + 1) % MOD);
74 |             }
75 |         }
76 |         for (int j = 0; j <= n; ++j) {
77 |             for (int k = j + 2; k <= n + 1; ++k) {
78 |                 int val = 1LL * dp[j][k] * F(j, k) % MOD;
79 |                 if (j)
80 |                     incMod(val, s1[j - 1][k]);
81 |                 if (k <= n)
82 |                     incMod(val, s2[j][k + 1]);
83 |                 dp[j][k] = val;
84 |             }
85 |         }
86 |     }
87 |     // 统计答案
88 |     for (int i = 0; i <= n; ++i) {
89 |         for (int j = i + 2; j <= n + 1; ++j) {
90 |             for (int k = i + 1; k < j; ++k) {
91 |                 incMod(ans[k], MOD - dp[i][j]);
92 |             }
93 |         }
94 |     }
95 |     for (int i = 1; i <= n; ++i) {
96 |         printf("%d ", ans[i]);
97 |     }
98 |     return 0;
99 | }

```

## 公开样例

### 样例 1 输入

```

5 5
1 5 2 3 4

```

### 样例 1 输出

```

3152671 3796875 3692207 3623487 3515626

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“动态规划”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 9 | 搜索、图论与树

把对象抽象为点、关系抽象为边，再决定 BFS、DFS、最短路或树上遍历。必须处理访问标记、连通性和复杂度。

本模块共 25 道唯一题。

## 9.1 | 总 487/526 获得自由？( PID 1564 )

出处：2020级HAUT新生周赛（五）@许金龙专场（A，CID 1064）| 训练定位：入门 | 搜索、图论与树

标签：图论、二分图、奇偶性 | 限制：1.000 S / 128 MB | 历史统计：78/125 ( 62.4% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，若他能获得自由，则输出“YES”，否则输出“NO”。

输入要点：输入一个正整数  $n$  ( $n \leq 8$ )。

输出要点：若他能获得自由，则输出“YES”，否则输出“NO”。

题面提示：题面示例包含图片，完整示意请在原网页查看。

### 零基础先修

- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 只关心奇偶时可把数化为  $x\%2$ ；和的奇偶等于所有项奇偶的异或。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：网格图是黑白二分图，每走一步颜色必然改变。右下角与左上角颜色相同；若  $n$  为偶数， $n^2$  个格子的路径含奇数条边，两个端点应异色，矛盾。若  $n$  为奇数，则可按蛇形路线遍历所有格子并从右下到左上，所以答案恰为  $n$  是否为奇数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 核对有向/无向、重边、自环、不可达点和点编号范围。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     cout << (n % 2 ? "YES" : "NO") << '\n';
7 |     return 0;
}
```

## 公开样例

样例 1 输入

3

样例 1 输出

YES

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)

## 9.2 | 总 488/526 迷宫探险 ( PID 1404 )

出处：2018级河南工大新生周赛（四）@杜锐专场（C，CID 1043）| 训练定位：基础提高 | 搜索、图论与树

标签：BFS、网格、搜索 | 限制：1.000 S / 128 MB | 历史统计：34/139 ( 24.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，若可以走通，输出yes；否则输出no。

输入要点：输入是一个只有零和一的矩阵。起点在左下，终点在右上。起点和终点一定为1。其他数据随机分布

输出要点：若可以走通，输出yes；否则输出no。

### 零基础先修

- BFS 按层扩展，在无权图中第一次到达某点就是最短步数。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：在固定 5×5 网格上从左下角做 BFS，只扩展上下左右且值为 1 的 未访问格子。最终查看右上角是否访问到，并按题目小写格式输出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

O(25) 时间、O(25) 空间

### 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int grid[5][5];
5 |     for (auto& row : grid)
6 |         for (int& x : row)
7 |             cin >> x;
8 |     bool visited[5][5] = {};
9 |     queue<pair<int, int>> q;
10 |    q.push({4, 0});
11 |    visited[4][0] = true;
12 |    const int dx[4] = {1, -1, 0, 0};
13 |    const int dy[4] = {0, 0, 1, -1};
14 |    while (!q.empty()) {
15 |        auto [x, y] = q.front();
16 |        q.pop();
17 |        for (int d = 0; d < 4; ++d) {
18 |            int nx = x + dx[d], ny = y + dy[d];
19 |            if (nx >= 0 && nx < 5 && ny >= 0 && ny < 5 && grid[nx][ny] &&
20 |                !visited[nx][ny]) {
21 |                visited[nx][ny] = true;
22 |                q.push({nx, ny});
23 |            }
24 |        }
25 |    }
26 |    cout << (visited[0][4] ? "yes" : "no") << '\n';
27 |    return 0;
28 | }
```

## 公开样例

样例 1 输入

```
0 0 0 0 1
0 0 0 1 1
0 0 1 1 0
0 0 1 0 0
1 1 1 0 0
```

样例 1 输出

```
yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 9.3 | 总 489/526 想象之中 ( PID 1726 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（A，CID 1075）| 训练定位：基础提高 | 搜索、图论与树

标签：条件分支、搜索 | 限制：1.000 S / 128 MB | 历史统计：15/55 ( 27.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，判断每一关是否可以成功通过，如果可以通过输出"YES"，否则输出"No"

输入要点：第一行输入一个T代表有T组测试样例。(1<= T <= 100)

每组样例有三行数据

第一行 输入三个数地图大小0<map<=1000,初始等级0<level<100,升级所需经验10<=require<=500

第二行 输入map个数,输入1表示起点，2表示怪物，3表示宝箱，4表示终点

第三行 输入map-2个数，输入顺序对应与第二行的怪物或者宝箱，若对应于怪物，数据为小于1000的自然数，对应与宝箱，则数据为1~4对应四种类型的经验瓶。

输出要点：判断每一关是否可以成功通过，如果可以通过输出"YES"，否则输出"No"

题面提示：输出注意换行

样例解释

起点在3处，终点在5处

4处有一个血量为10的怪物，当前玩家等级为5级，对应攻击力为50，可以击败怪物，获得100点经验，经验条填充30点可填满并升级，共升三级，经验条剩余10点经验，到达5号终点，输出YES。

## 零基础先修

- if/else 用来按互斥条件选择处理路径；先覆盖特殊边界，再处理一般情况。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：想象之中注意读题，如果无法击败怪物，怪物会逃跑，所以一定能到终点，直接输出YES即可 C
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 检查条件是否互斥且覆盖所有边界，尤其是等号落在哪一侧。
- 入队/递归时及时标记访问，避免重复状态和死循环。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int n;
4 |     scanf("%d", &n);
5 |     while (n-->0)
6 |         printf("YES\n");
7 | }
```

## 公开样例

样例 1 输入

```
1
10 5 30
3 2 1 2 4 3 3 2 2 2
3 100 10 1 2 30 20 23
```

样例 1 输出

```
YES
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题
- 公开题解/参考来源 1
- 题面图片 1

## 9.4 | 总 490/526 似乎在梦里见过那样 ( PID 1734 )

出处：2021级HAUT新生周赛（八）@孔维斌专场（A，CID 1076）| 训练定位：基础提高 | 搜索、图论与树

标签：图论、数学、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：45/101 ( 44.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，多边形的面积，输出保留两位小数。

输入要点：第一行给出多边形的顶点数 $n(3 \leq n \leq 100)$ 。接下来的几行每行给出多边形一个顶点的坐标值 $X$ 和 $Y$ (都为整数并且用空格隔开)。

顶点按逆时针方向逐个给出。并且多边形的每一个顶点的坐标值 $-200 \leq x, y \leq 200$ 。

多边形最后是靠从最后一个顶点到第一个顶点画一条边来封闭的。

输出要点：多边形的面积，输出保留两位小数。

题面提示：[图片：[https://acm.haut.edu.cn/upload/image/20211205/20211205184952\\_37626.png](https://acm.haut.edu.cn/upload/image/20211205/20211205184952_37626.png)]

样例图示

### 零基础先修

- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：似乎在梦里见过那样（传送门）题目大意是求出一个凸多边形的面积。我们根据高中知识可以得出，在面对求一个多边形面积时，如果多边形是三角形或者四边形可以直接使用公式来进行计算，那么遇见一个多边形一般是将一个多边形划分成为多个三角形。那么我们只需求出划分出来的三角形的面积，之后再进一步求和即可。那么我们再想一下如何划分三角形，由于题目给出了是凸多边形，发现只要固定一个点，剩下的两个点在多边形的逆时针的点上依次选取即可，如图所示。样例的图经过这样划分后就划分成为5个多边形，这样求出5个三角形就求出凸多边形面积了。那么这样分为什么是正确的？如果换成凹多边形是否正确？由于是凸多边形，那么这样划分的下一个三角形不会和上一个三角形有交集，而这样划分的三角形又可以覆盖整个多边形，所以是正确的。但是如果是凹多边形就会出现错误，会出现重复的部分，并且会出现三角形外部的部分，如：凹多边形的面积可以使用容斥原理，就是一些划分三角形面积记位正值，一些记位负值，这样就可以求出了。这里不过多涉及，但是标程代码可以实现求出凹多边形的面积 还有一点精度问题，这里使用叉乘的方式求出面积，就是三角形两个边的向量的叉乘结果是三角形面积两倍。时间复杂度  $O(n)O(n)O(n)$
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

$O(n)$

## 易错点

- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | typedef struct Node { // 定义一个结构体即表示点，又表示向量
4 |     double x, y;
5 | } node;
6 | node p[110];          // 因为最多100个
7 | double across(node a, node b) { // 返回两个向量的叉乘
8 |     return a.x * b.y - a.y * b.x;
9 | }
10 | double area(node a, node b, node c) { // 将三个点转化为两条边的向量
11 |     node x, y;
12 |     x.x = b.x - a.x;
13 |     x.y = b.y - a.y;
14 |     y.x = c.x - a.x;
15 |     y.y = c.y - a.y;
16 |     return across(x, y);
17 | }
18 | int main() {
19 |     int n;
20 |     scanf("%d", &n);
21 |     for (int i = 0; i < n; i++) {
22 |         scanf("%lf%lf", &p[i].x, &p[i].y);
23 |     }
24 |     double ans = 0;
25 |     for (int i = 1; i + 1 < n; i++) {
26 |         ans += area(
27 |             p[0], p[i],
28 |             p[i +
29 |                 1]); // 划分的三角形第一个点固定为p[0],剩下的从两个点为i和i+1，这样来枚举所有三角形。
30 |     }
31 |     printf("%.2f\n", ans / 2);
32 |     return 0;
33 | }
```

## 公开样例

样例 1 输入

```
7
4 2
2 6
-2 8
-6 6
-8 4
-6 0
-2 -2
```

样例 1 输出

```
74.00
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)



## 9.5 | 总 491/526 九九八十一 ( PID 1790 )

出处：2022级HAUT新生周赛（五）@李昊阳专场（E，CID 1083）| 训练定位：基础提高 | 搜索、图论与树

标签：数组与序列、搜索 | 限制：1.000 S / 128 MB | 历史统计：3/6 ( 50.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一个整数表示孙悟空需要火的数量。

输入要点：第一行两个整数  $n, m$ ，(  $1 \leq n, m \leq 1000$  )

接下来  $n$  行每行  $m$  列是盘丝洞的地图。

输出要点：一个整数表示孙悟空需要火的数量。

### 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
- 核心处理采用：九九八十一 题意：计算出地图上蜘蛛网形成的块的数量。解法：一道dfs算法的入门题（也可以拿bfs算法做），遍历每个点，如果这个点是蜘蛛网且还没有被烧掉，则对这个格子进行dfs，把所有与这个格子相连的蜘蛛网都标记为已经烧掉，答案+1。当对一个点dfs时，首先把自身标记为已经烧掉，然后判断它周围的四个格子，如果有格子是蜘蛛网却还未被烧掉，则再对这个点dfs。直到所有与原先蜘蛛网链接的蜘蛛网被标记为烧掉。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

### 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 入队/递归时及时标记访问，避免重复状态和死循环。

### C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <stdlib.h>
3 | int n, m;
4 | char dt[1005][1005];
5 | int vis[1005][1005];
6 | int dx[4] = {0, 1, 0, -1};
7 | int dy[4] = {1, 0, -1, 0};
8 | void dfs(int x, int y) {
9 |     // 标记自身为烧掉的状态
10 |    vis[x][y] = 1;
11 |    for (int i = 0; i < 4; i++) {
12 |        int xx = x + dx[i], yy = y + dy[i];
13 |        // 如果它周围的某个方向的格子是蜘蛛网且还未被烧掉，则dfs那个格子
14 |        if (dt[xx][yy] == '#' && vis[xx][yy] == 0)
15 |            dfs(xx, yy);
16 |    }
17 | }
18 | int main() {
19 |    scanf("%d%d", &n, &m);
```

```

20 |     for (int i = 1; i <= n; i++)
21 |         scanf("%s", dt[i] + 1);
22 |     int ans = 0;
23 |     for (int i = 1; i <= n; i++) {
24 |         for (int j = 1; j <= m; j++) {
25 |             // 如果这个格子是蜘蛛网且还未被烧掉
26 |             if (dt[i][j] == '#' && vis[i][j] == 0) {
27 |                 dfs(i, j);
28 |                 ans++;
29 |             }
30 |         }
31 |     }
32 |     printf("%d", ans);
33 |     return 0;
34 | }

```

## 公开样例

样例 1 输入

```

3 4
##.
.##
##.

```

样例 1 输出

```

3

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.6 | 总 492/526 打印图形 ( PID 1799 )

出处：2022级HAUT新生周赛（六）@张鑫专场（F，CID 1084）| 训练定位：基础提高 | 搜索、图论与树

标签：字符串、搜索、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：5/9 ( 55.6% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

**题意摘要：**根据给定输入，第一行输入一个正整数 $t$ 表示有 $t$ 组测试样例，接下来 $t$ 行每行一个正整数 $n$ 表示“类X形”的阶数和一个英文字符 $ch$ 。

**输入要点：**第一行输入一个正整数 $t$ 表示有 $t$ 组测试样例，接下来 $t$ 行每行一个正整数 $n$ 表示“类X形”的阶数和一个英文字符 $ch$ 。

**输出要点：**第一行输入一个正整数 $t$ 表示有 $t$ 组测试样例，接下来 $t$ 行每行一个正整数 $n$ 表示“类X形”的阶数和一个英文字符 $ch$ 。

**题面提示：** $1 \leq n \leq 4$ ， $t \leq 10$ （数据很小）

对于每一行输出后面必须有空格直到和对应图形的第一行长度相当，每行末应该有回车。

## 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。

- 核心处理采用：打印图形（递归每次由中心向四个角和中心递归直到阶数为1，3的阶数减1次方为“类X形”边长（数据不大也可以直接输出，其实这题本身就是让大家输出四种情况的，下面是进阶做法）。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 `cin` 留下的换行。
- 入队/递归时及时标记访问，避免重复状态和死循环。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | char a[3000][3000], ch;
4 | int xx[10] = {1, -1, 1, -1, 0}, yy[5] = {1, 1, -1, -1, 0}; // 方便递归“类X型”的5个方向
5 | int func(int x) // 计算3的x次方并返回
6 | {
7 |     int ans = 1;
8 |     while (x--)
9 |         ans *= 3;
10 |    return ans;
11 | }
12 | void dfs(int step, int x, int y) // step记录对应的边长为3的step次方，x,y为当前位置
13 | {
14 |     if (step == 1) // 当step==1时达到递归终点，结束
15 |     {
16 |         a[x][y] = ch; // 标记位置
17 |         return;
18 |     }
19 |     int m = func(step - 2);
20 |     for (int i = 0; i < 5; i++)
21 |         dfs(step - 1, x + xx[i] * m,
22 |             y + yy[i] * m); // 分别向四个角的下一级中心点和当前点递归
23 | }
24 | int main() {
25 |     int t;
26 |     scanf("%d", &t);
27 |     while (t--) {
28 |         memset(a, '\0', sizeof(a)); // 每次把a重置
29 |         int n, m;
30 |         scanf("%d %c", &n, &ch); // 前面加一个空格吞回车
31 |         m = func(n - 1); // m为类X型的边长
32 |         dfs(n, m / 2 + 1, m / 2 + 1); // 从中心点开始递归
33 |         for (int i = 1; i <= m; i++) {
34 |             for (int j = 1; j <= m; j++)
35 |                 if (a[i][j] != ch)
36 |                     printf(" "); // 无标记输出空格
37 |             else
38 |                 printf("%c", ch);
39 |             printf("\n");
40 |         }
41 |     }
42 |     return 0;
43 | }
```

## 公开样例

样例 1 输入

```
2
2*
4O
```

样例 1 输出

```
**
*
```

```

**
00 00    00 00
0 0      0 0
00 00    00 00
  00      00
  0        0
  00      00
00 00    00 00
0 0      0 0
00 00    00 00
  00 00
    0 0
    00 00
      00
      0
      00
    00 00
    0 0
    00 00
00 00    00 00
0 0      0 0
00 00    00 00
  00      00
  0        0
  00      00
00 00    00 00
0 0      0 0
00 00    00 00

```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

9.7 | 总 493/526 破碎的未来 ( PID 1857 )

出处：2023级HAUT新生周赛（五）@陈兰锴专场（E，CID 1090） | 训练定位：基础提高 | 搜索、图论与树

标签：搜索、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：2/3 ( 66.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：根据给定输入，一个整数，最少有多少地图残片

输入要点：n , m

接下来n行，每行m个数

输出要点：一个整数，最少有多少地图残片

题面提示：提示  $n \leq 1000, m \leq 1000$ ;

零基础先修

- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。

3. 核心处理采用：破碎的未来 中等题 对于地图上的每一个点都对其进行标记，如果四个方向中有一个点和其颜色相同，并且未被标记，将递归进行标记。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

$O(\text{false})$

## 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int a[1100][1100];
4 | int n, m;
5 | void dfs(int i, int j, int op) {
6 |     if (i < 1 || i > n || j < 1 || j > m)
7 |         return;
8 |     if (a[i][j] == -1)
9 |         return;
10 |    if (a[i][j] != op)
11 |        return;
12 |    a[i][j] = -1;
13 |    dfs(i + 1, j, op);
14 |    dfs(i - 1, j, op);
15 |    dfs(i, j + 1, op);
16 |    dfs(i, j - 1, op);
17 | }
18 | void solve() {
19 |     cin >> n >> m;
20 |     for (int i = 1; i <= n; ++i) {
21 |         for (int j = 1; j <= m; ++j) {
22 |             cin >> a[i][j];
23 |         }
24 |     }
25 |     int ans = 0;
26 |     for (int i = 1; i <= n; ++i) {
27 |         for (int j = 1; j <= m; ++j) {
28 |             if (a[i][j] != -1) {
29 |                 dfs(i, j, a[i][j]);
30 |                 ans++;
31 |             }
32 |         }
33 |     }
34 |     cout << ans;
35 | }
36 | int main() {
37 |     ios::sync_with_stdio(false);
38 |     cin.tie(0);
39 |     cout.tie(0);
40 |     int test = 1;
41 |     // cin >> test;
42 |     while (test--) {
43 |         solve();
44 |     }
45 |     return 0;
46 | }
```

## 公开样例

样例 1 输入

```
4 4
0 0 0 0
1 1 1 1
0 1 0 0
1 0 0 0
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.8 | 总 494/526 BOX ( PID 1876 )

出处：河南工大2024新生周赛 ( 3 ) ——命题人：张宏泽 ( F , CID 1093 ) | 训练定位：基础提高 | 搜索、图论与树

标签：字符串、图论 | 限制：1.000 S / 128 MB | 历史统计：56/158 ( 35.4% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在zhz有  $q$  个查询。对于每个查询，他都提供了一个点  $(x_i, y_i, z_i)$  并想知道该点是否在盒子内(边界也被视为在盒子内)。

输入要点：第一行包含六个整数  $z_0, h, u_0, v_0, u_1, v_1$  ( $0 \leq z_0, h \leq 10^6, -10^6 \leq u_0, v_0, u_1, v_1 \leq 10^6$ )。

第二行包含一个整数  $q$  ( $1 \leq q \leq 10^3$ )，表示查询次数。

接下来的  $q$  行分别包含三个整数  $x_i, y_i, z_i$  ( $-10^6 \leq x_i, y_i, z_i \leq 10^6$ ) 表示  $i$  个查询。

输出要点：输出  $q$  行。在第  $i$  行中，输出字符串 YES 或 NO。YES 表示查询点在盒子内，NO 表示查询点不在盒子内。

题面提示： $u_0$  和  $u_1$  以及  $v_0$  和  $v_1$  的大小关系并不确定

### 零基础先修

- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
- 核心处理采用：BOX 简单的几何问题，如果给定点的三个坐标值都在长方体的三个坐标值确定的范围之内，则该点在盒子中。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

### 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 核对有向/无向、重边、自环、不可达点和点编号范围。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | class Point // 笔者写几何类题目的时候比较喜欢定义一个类 Point 表示点
3 | {
4 |     public:
5 |         int x, y, z;
6 |         void read() {
7 |             scanf("%d%d%d", &x, &y, &z);
8 |         } // 定义一个函数使得读入更方便
9 | };
10 | int z0, h, u0, v0, u1, v1;
11 | int min(int a, int b) {
12 |     return a > b ? b : a;
13 | }
14 | int max(int a, int b) {
15 |     return a > b ? a : b;
16 | }
17 | bool check(Point p) {
18 |     if (p.z >= z0 && p.z <= z0 + h && p.x >= min(u0, u1) && p.x <= max(u0, u1) &&
19 |         p.y >= min(v0, v1) && p.y <= max(v0, v1))
20 |         return 1;
21 |     return 0;
22 | }
23 | void solve() {
24 |     scanf("%d%d%d%d%d", &z0, &h, &u0, &v0, &u1, &v1);
25 |     int q;
26 |     scanf("%d", &q);
27 |     while (q--) {
28 |         Point p;
29 |         p.read(); // 读入点的坐标;
30 |         puts(check(p) ? "YES" : "NO"); // 如果满足在盒子内部则输出"YES"
31 |                                     // 这里的语法可能用的比较繁琐, 能理解就行
32 |     }
33 | }
34 | int main() {
35 |     int T = 1;
36 |     while (T--)
37 |         solve();
38 |     return 0;
39 | }
```

## 公开样例

### 样例 1 输入

```
1 1 -1 -1 1 1
3
-1 -1 1
0 0 2
1 2 2
```

### 样例 1 输出

```
YES
YES
NO
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.9 | 总 495/526 离散数学 ( PID 1890 )

出处：河南工大2024新生周赛（5）——命题人：刘庭钰（D，CID 1095） | 训练定位：基础提高 | 搜索、图论与树

标签：图论 | 限制：1.000 S / 128 MB | 历史统计：12/26 ( 46.2% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：定义这个图的复杂度是：你有任意次操作，每次操作合并两个联通块，合并后联通块大小为二者之和，最后剩下两个联通块大小的乘积为此图的复杂度，若只有一个联通块则复杂度为0。你需要最大化此图复杂度

输入要点：第一行包包含两个整数  $n$  和  $m$ ，图中顶点的数量和边的数量。

接下来的每  $m$  行包含两个整数  $u$  和  $v$ ，表示图中顶点  $u$  和  $v$  之间有一条无向边

( $0 < n \leq 1e6, 0 \leq m \leq 1e6, 0 < u, v \leq n$ )

输出要点：输出一个整数表示最大代价

题面提示：本题不涉及数据结构中图的内容，请大家认真读题

样例图如下

[图片：[https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241126/20241126194857\\_96048.png](https://acm.haut.edu.cn/upload/acm.haut.edu.cn/image/20241126/20241126194857_96048.png)]

## 零基础先修

- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：离散数学 题目中说明：拥有无数次删除操作 则直接将图删除为  $n$  个点，由基本不等式  $a^2 + b^2 \geq 2ab$  得 当  $a$  与  $b$  尽量接近时，求得的复杂度最大
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 按题目上界估算中间量，相关变量不要退回 `int`。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, m, u, v;
5 |     long long val;
6 |     cin >> n >> m;
7 |     for (int i = 1; i <= m; i++) {
8 |         cin >> u >> v;
9 |     }
10 |    if (n == 1) // 如果只有1个点时，只能构成1个连通块，复杂度为 0
11 |    {
12 |        cout << 0;
13 |        return 0;
14 |    } else {
15 |        // 如果n为偶数，复杂度最大值即为 (n/2)^2;
16 |        if (n % 2 == 0)
17 |            val = (n / 2) * (n / 2);
18 |        // 如果n为奇数，复杂度最大值即为 n/2 * (n+1)/2;
19 |        else
20 |            val = n / 2 * (n + 1) / 2;
21 |    }
22 |    cout << val;
23 |    return 0;
24 | }
```



公开样例

样例 1 输入

```
4 3
1 2
2 3
3 4
```

样例 1 输出

```
4
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

9.10 | 总 496/526 世界线跃迁 ( PID 1944 )

出处：河南工大2025新生周赛 ( 7 ) ——命题人：牛见青 ( F , CID 1109 ) | 训练定位：基础提高 | 搜索、图论与树

标签：数组与序列、图论 | 限制：1.000 S / 32 MB | 历史统计：6/14 ( 42.9% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：如果某个世界线无法到达，请输出 -1。

输入要点：第一行包含两个正整数  $n$  和  $s$ ：

-

$n$  表示世界线的数量 ( 编号为 0 到  $n-1$  ) ；

-

$s$  表示当前所在的源世界线编号。

保证  $n \leq 50$ ，且  $0 \leq s < n$ 。

接下来的  $n$  行，每行包含  $n$  个用空格隔开的整数，表示跃迁代价矩阵。

-

若第  $i$  行第  $j$  个数 大于 0，则表示存在一条从世界线  $i$  跃迁到世界线  $j$  的有向通道，能量代价为该数。

-

若该数 等于 0，则表示不存在从  $i$  到  $j$  的通道。

-

保证对角线元素 (  $i == j$  ) 均为 0。

输出要点：输出一行，共有  $n-1$  个整数，依次表示从源世界线  $s$  到其余每一个世界线的最小能量代价。

若某个世界线无法到达，输出  $-1$ 。

行尾需输出换行。

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：世界线跃迁 本题考察单源最短路，使用 Dijkstra 算法即可满足题目复杂度。AC 代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

$O(\text{false})$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 核对有向/无向、重边、自环、不可达点和点编号范围。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, s;
7 |     if (!(cin >> n >> s))
8 |         return 0;
9 |     vector<vector<int>> g(n, vector<int>(n));
10 |    for (int i = 0; i < n; ++i) {
11 |        for (int j = 0; j < n; ++j) {
12 |            cin >> g[i][j];
13 |        }
14 |    }
15 |    const int INF = 0x3f3f3f3f;
16 |    vector<int> dist(n, INF);
17 |    vector<bool> vis(n, false);
18 |    dist[s] = 0;
19 |    for (int i = 0; i < n; ++i) {
20 |        int u = -1, mind = INF;
21 |        for (int j = 0; j < n; ++j) {
22 |            if (!vis[j] && dist[j] < mind) {
23 |                mind = dist[j];
24 |                u = j;
25 |            }
26 |        }
27 |        if (u == -1)
28 |            break;
29 |        vis[u] = true;
30 |        for (int v = 0; v < n; ++v) {
31 |            if (g[u][v] > 0 && dist[u] + g[u][v] < dist[v]) {
32 |                dist[v] = dist[u] + g[u][v];
33 |            }
34 |        }
35 |    }
36 |    bool first = true;
37 |    for (int i = 0; i < n; ++i) {
```

```

38 |         if (i == s)
39 |             continue;
40 |         if (!first)
41 |             cout << ' ';
42 |         first = false;
43 |         if (dist[i] == INF)
44 |             cout << -1;
45 |         else
46 |             cout << dist[i];
47 |     }
48 |     cout << '\n';
49 |     return 0;
50 | }

```

## 公开样例

样例 1 输入

```

4 1
0 3 0 1
0 0 4 0
2 0 0 0
0 0 1 0

```

样例 1 输出

```

6 4 7

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.11 | 总 497/526 萤火虫大战真蛰虫 ( PID 1952 )

出处：河南工大2025新生周赛（4）——命题人：张梦淇（E，CID 1105） | 训练定位：基础提高 | 搜索、图论与树

标签：栈与队列、BFS、网格模拟 | 限制：1.000 S / 128 MB | 历史统计：44/115 ( 38.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：请你判断：萤火虫们能否消灭完所有真蛰虫，守护格拉默！

输入要点：第1行有2个整数 $n, m$  ( $1 \leq n \leq 100, m \leq 100000$ )。

第2~ $n+1$ 行，每行有 $n$ 个整数，表示这个位置的真蛰虫HP ( $HP \leq 100000$ )，若为0，则此位置不存在真蛰虫。

接下来 $m$ 行，每行有2个整数 $x, y$ 表示萨姆传送的位置。 ( $1 \leq x, y \leq n$ )

输出要点：在成功清除所有真蛰虫时输出"YES", 失败时输出"NO" (大小写敏感)。

### 零基础先修

- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- BFS 按层扩展，在无权图中第一次到达某点就是最短步数。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。

3. 核心处理采用：把一次  $3 \times 3$  伤害封装为函数。生命值从正数恰好降到 0 的虫子入队，随后按 BFS 顺序爆炸并继续给邻域减血；每只虫只会在首次归零时入队，所以不会重复爆炸。所有萤火虫攻击处理完且连锁队列清空后，检查是否还有生命值大于 0 的格子。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

$O(n^2+m+\text{爆炸影响总量})$ ，每次攻击或爆炸检查 9 格； $O(n^2)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, m;
7 |     cin >> n >> m;
8 |     vector<vector<int>> hp(n + 2, vector<int>(n + 2, 0));
9 |     int alive = 0;
10 |    for (int i = 1; i <= n; ++i) {
11 |        for (int j = 1; j <= n; ++j) {
12 |            cin >> hp[i][j];
13 |            if (hp[i][j] > 0)
14 |                ++alive;
15 |        }
16 |    }
17 |    queue<pair<int, int>> explosions;
18 |    auto damage = [&](int x, int y) {
19 |        for (int i = x - 1; i <= x + 1; ++i) {
20 |            for (int j = y - 1; j <= y + 1; ++j) {
21 |                if (i < 1 || i > n || j < 1 || j > n || hp[i][j] <= 0)
22 |                    continue;
23 |                --hp[i][j];
24 |                if (hp[i][j] == 0) {
25 |                    --alive;
26 |                    explosions.push({i, j});
27 |                }
28 |            }
29 |        }
30 |    };
31 |    for (int shot = 0; shot < m; ++shot) {
32 |        int x, y;
33 |        cin >> x >> y;
34 |        damage(x, y);
35 |        while (!explosions.empty()) {
36 |            auto [a, b] = explosions.front();
37 |            explosions.pop();
38 |            damage(a, b);
39 |        }
40 |    }
41 |    cout << (alive == 0 ? "YES" : "NO") << '\n';
42 |    return 0;
43 | }
```

## 公开样例

样例 1 输入

```
3 5
2 2 4
1 1 4
3 4 3
1 3
3 1
2 2
2 2
3 3
```

样例 1 输出

YES

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.12 | 总 498/526 领导领导 ( PID 1965 )

出处：河南工大2025新生周赛 ( 5 ) ——命题人：袁林坤 ( F , CID 1106 ) | 训练定位：基础提高 | 搜索、图论与树

标签：数学、搜索、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：24/50 ( 48.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出包含 M 行，每行包含一个正整数，依次为每一个查询的结果。

输入要点：第一行包含三个正整数 N、M、S，分别表示企业的员工总数、查询次数、CEO 的员工编号。

接下来 N-1 行，每行包含两个正整数 x、y，表示员工 x 和员工 y 是直属上下级关系。

接下来 M 行，每行包含两个正整数 a、b，表示查询员工 a 和员工 b 的最近共同领导。

输出要点：输出包含 M 行，每行包含一个正整数，依次为每一个查询的结果。

题面提示：对于 100% 的数据， $N \leq 1000$ ， $M \leq 2000$ 。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：领导领导 lca 板子题，有兴趣可以去了解，这里提供一个倍增法。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

$O(0)$

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 入队/递归时及时标记访问，避免重复状态和死循环。
- 确认 0/1 起下标、首尾元素和 n=1。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```

1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | #define endl '\n'
4 | #define ull unsigned long long
5 | #define ll long long
6 | #define PII pair<int, int>
7 | const ll N = 1e4 + 10;
8 | const int mod = 80112002;
9 | const int P = 1;
10 | int n, m, s, a, b;
11 | vector<int> e[N];
12 | int dep[N], fa[N][22];
13 | void dfs(int u, int father) {
14 |     dep[u] = dep[father] + 1;
15 |     fa[u][0] = father;
16 |     for (int i = 1; i <= 20; i++)
17 |         fa[u][i] = fa[fa[u][i - 1]][i - 1];
18 |     for (int v : e[u])
19 |         if (v != father)
20 |             dfs(v, u);
21 | }
22 | int lca(int u, int v) {
23 |     if (dep[u] < dep[v])
24 |         swap(u, v);
25 |     for (int i = 20; i >= 0; i--)
26 |         if (dep[fa[u][i]] >= dep[v])
27 |             u = fa[u][i];
28 |     if (u == v)
29 |         return v;
30 |     for (int i = 20; i >= 0; i--) {
31 |         if (fa[u][i] != fa[v][i])
32 |             u = fa[u][i], v = fa[v][i];
33 |     }
34 |     return fa[u][0];
35 | }
36 | void solve() {
37 |     cin >> n >> m >> s;
38 |     for (int i = 1; i < n; i++) {
39 |         int x, y;
40 |         cin >> x >> y;
41 |         e[x].push_back(y);
42 |         e[y].push_back(x);
43 |     }
44 |     dfs(s, 0);
45 |     while (m--) {
46 |         int u, v;
47 |         cin >> u >> v;
48 |         cout << lca(u, v) << endl;
49 |     }
50 | }
51 | int main() {
52 |     ios::sync_with_stdio(0);
53 |     cin.tie(0);
54 |     cout.tie(0);
55 |     int T = 1;
56 |     // cin >> T;
57 |     while (T--)
58 |         solve();
59 |     return 0;
60 | }

```

## 公开样例

### 样例 1 输入

```

5 5 4
3 1
2 4
5 1
1 4
2 4
3 2
3 5
1 2
4 5

```

样例 1 输出

```
4
4
1
4
4
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 9.13 | 总 499/526 疲惫的一天 ( PID 1221 )

出处：2016级河南工大新生周赛 ( 四 ) ( B , CID 1009 ) | 训练定位：进阶 | 搜索、图论与树

标签：枚举方案、绝对值距离、最短路思想 | 限制：1.000 S / 128 MB | 历史统计：13/186 ( 7.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个整数，占一行，表示cds需要步行的最短路程

输入要点：4个正整数x1, y1, x2, y2,数据保证x1<y1, x2<y2, 且都在int范围内，

输出要点：一个整数，占一行，表示cds需要步行的最短路程

## 零基础先修

- 把对象抽象为点、关系抽象为边，再决定 BFS、DFS、最短路或树上遍历。必须处理访问标记、连通性和复杂度。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：cds 从点 0 步行，学校在 1000。每辆车只能在起点 x 上车、终点 y 下车，且中途不能下。最优方案只可能是：都不坐、只坐 1、只坐 2、先 1 后 2、先 2 后 1。分别把各段步行距离的绝对值相加，取五个候选 的最小值；坐车路段不计步行。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 |     long long x1, y1, x2, y2;
5 |     cin >> x1 >> y1 >> x2 >> y2;
6 |     long long ans = 1000;
7 |     ans = min(ans, labs(x1) + labs(1000 - y1));
8 |     ans = min(ans, labs(x2) + labs(1000 - y2));
9 |     ans = min(ans, labs(x1) + labs(y1 - x2) + labs(1000 - y2));
10 |    ans = min(ans, labs(x2) + labs(y2 - x1) + labs(1000 - y1));
11 |    cout << ans << '\n';
12 |    return 0;
13 | }

```

## 公开样例

样例 1 输入

300 1000 50 301

样例 1 输出

51

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 9.14 | 总 500/526 欢迎来到实力至上主义的OJ ( PID 1328 )

出处：2017级河南工大新生周赛（三）（A，CID 1030）| 训练定位：进阶 | 搜索、图论与树

标签：动态规划、最短路、状态转移 | 限制：1.000 S / 128 MB | 历史统计：116/548 ( 21.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一整数，走过的最短距离

输入要点：第一行：n-->总共经过顶点的次数

第二行：dis1--> a 顶点到 b 顶点的距离

第三行：dis2--> a 顶点到 c 顶点的距离

第四行：dis3--> b 顶点到 c 顶点的距离

数据范围均为 (  $1 \leq n, dis1, dis2, dis3 \leq 100$  )

输出要点：一整数，走过的最短距离

题面提示：在实力至上主义的OJ时间代替点数成为了最重要的东西，所以，请不要粗心大意，努力避免罚时，尽快AK吧！

## 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：默认已经在顶点 a，并把它算作经过的第 1 个顶点，所以还需走 n-1 条边。顶点只有三个，做小型动态规划：dp[step][v] 表示共经过 step 个顶点且停在 v 的最短路程；从另外两个顶点转移并加相应边长。允许 重复经过顶点，但一次移动必须沿三角形的一条边。



最后取三个终点最小值。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n;
7 |     long long ab, ac, bc;
8 |     cin >> n >> ab >> ac >> bc;
9 |     long long distance[3][3] = {{0, ab, ac}, {ab, 0, bc}, {ac, bc, 0}};
10 |     const long long INF = (1LL << 60);
11 |     array<long long, 3> dp{0, INF, INF};
12 |     for (int visited = 2; visited <= n; ++visited) {
13 |         array<long long, 3> next{INF, INF, INF};
14 |         for (int from = 0; from < 3; ++from)
15 |             for (int to = 0; to < 3; ++to)
16 |                 if (from != to)
17 |                     next[to] = min(next[to], dp[from] + distance[from][to]);
18 |         dp = next;
19 |     }
20 |     cout << *min_element(dp.begin(), dp.end()) << '\n';
21 |     return 0;
22 | }
```

## 公开样例

样例 1 输入

```
3
2
2
1
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.15 | 总 501/526 Despair ( PID 1418 )

出处：2018级河南工大新生周赛总决赛（重现）（B，CID 1050）；2018级河南工大新生周赛总决赛@陈丁山大神专场（B，CID 1048） | 训练定位：进阶 | 搜索、图论与树

标签：动态规划、最短路思想、绝对值距离 | 限制：1.000 S / 128 MB | 历史统计：1/13 ( 7.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：你已经饿得没有体力了，那么如何以最小的移动距离触发 $m$ 次机关以点亮走廊的灯呢？

输入要点：先输入一个数 $T$ 表示有 $T$ 组测试数据（ $T \leq 20$ ）

每组测试数据先输入三个数 $n$ 和 $m, x$ ，表示机关的数量和你要触发的机关次数（ $2 \leq n \leq 100, 1 \leq m \leq 100$ ），以及你所在的位置（ $0 \leq x \leq 1000$ ）

接下来输入 $n$ 个数，第 $i$ 个数字 $a_i$ 表示第 $i$ 个机关所在位置（ $0 \leq a_i \leq 1000$ ），保证所有数字独一无二，且都不等于上面的 $x$ ，输入保证有序

输出要点：输出最小的移动距离

题面提示：对于第一个样例：有2个机关，位置分别是10和11，它们挨在一起，你一开始在位置100

你需要先跑到位置11触发第2个机关，在走到位置10触发第1个机关，然后再去位置11触发第2个机关，再回到位置10触发第1个机关……依次往复直到触发10次为止，总共需要移动98步

对于第2个样例：有5个机关，位置分别是1,2,7,8,9，你一开始在位置10

触发机关顺序：5→4→3→4，也就是说前两个机关不需要去触发

### 零基础先修

- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
- 核心处理采用：把“已经触发了  $k$  次、最后一次在机关  $i$ ”作为状态。第一次到  $i$  的代价是  $|x - a_i|$ ；之后不能连续触发同一机关，所以从任意  $j \neq i$  转移到  $i$ ，新增距离  $|a_j - a_i|$ 。逐层做  $m$  次转移，最后一层的最小值 就是答案。这个动态规划直接覆盖“先走到某对相邻机关再反复横跳”等所有可能方案，不需要猜测最优的那一对。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

每组  $O(mn^2)$  时间、 $O(n)$  空间

### 易错点

- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 按题目上界估算中间量，相关变量不要退回 int。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
```

```

8 |     const long long INF = (1LL << 60);
9 |     while (T--) {
10 |         int n, m, x;
11 |         cin >> n >> m >> x;
12 |         vector<int> a(n);
13 |         for (int& v : a)
14 |             cin >> v;
15 |         vector<long long> dp(n), nextDp(n, INF);
16 |         for (int i = 0; i < n; ++i)
17 |             dp[i] = abs(x - a[i]);
18 |         for (int step = 2; step <= m; ++step) {
19 |             fill(nextDp.begin(), nextDp.end(), INF);
20 |             for (int from = 0; from < n; ++from) {
21 |                 for (int to = 0; to < n; ++to) {
22 |                     if (from == to)
23 |                         continue;
24 |                     nextDp[to] = min(nextDp[to], dp[from] + abs(a[from] - a[to]));
25 |                 }
26 |             }
27 |             dp.swap(nextDp);
28 |         }
29 |         cout << *min_element(dp.begin(), dp.end()) << '\n';
30 |     }
31 |     return 0;
32 | }

```

## 公开样例

### 样例 1 输入

```

2
2 10 100
10 11
5 4 10
1 2 7 8 9

```

### 样例 1 输出

```

98
4

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 9.16 | 总 502/526 zp与水系道馆 ( PID 1513 )

出处：2019级河南工大新生周赛 ( 五 ) @潘世泽专场 ( G , CID 1057 ) | 训练定位：进阶 | 搜索、图论与树

标签：BFS、网格、搜索 | 限制：1.000 S / 128 MB | 历史统计：10/55 ( 18.2% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：zp来到了水系道馆，在挑战馆主之前要先进行一个解谜游戏——那就是走迷宫，由于每次迷宫都会变，只有入口和出口是固定的，所以zp无法知道迷宫是否有出口，请你帮助他判断！0代表水柱，1代表可以走的路！

输入要点：输入n (  $5 \leq n \leq 50$  )，迷宫为n\*n的正方形迷宫，入口一定在左下角，出口一定在右上角，其余的数据随机分布！

输出要点：输出若可以走通输出"Yes"，否则输出"No"。

## 零基础先修

- BFS 按层扩展，在无权图中第一次到达某点就是最短步数。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：从左下角入口对值为 1 的格子做 BFS，四个方向扩展且每格只访问一次；若能访问右上角出口输出 Yes，否则 No。入口或出口若为 0，也会自然判为不可达。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

$O(n^2)$  时间、 $O(n^2)$  空间

## 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     int n;
5 |     cin >> n;
6 |     vector<vector<int>> grid(n, vector<int>(n));
7 |     for (auto& row : grid)
8 |         for (int& x : row)
9 |             cin >> x;
10 |     queue<pair<int, int>> q;
11 |     vector<vector<bool>> visited(n, vector<bool>(n, false));
12 |     if (grid[n - 1][0]) {
13 |         q.push({n - 1, 0});
14 |         visited[n - 1][0] = true;
15 |     }
16 |     const int dx[4] = {1, -1, 0, 0};
17 |     const int dy[4] = {0, 0, 1, -1};
18 |     while (!q.empty()) {
19 |         auto [x, y] = q.front();
20 |         q.pop();
21 |         for (int d = 0; d < 4; ++d) {
22 |             int nx = x + dx[d], ny = y + dy[d];
23 |             if (nx >= 0 && nx < n && ny >= 0 && ny < n && grid[nx][ny] &&
24 |                 !visited[nx][ny]) {
25 |                 visited[nx][ny] = true;
26 |                 q.push({nx, ny});
27 |             }
28 |         }
29 |     }
30 |     cout << (visited[0][n - 1] ? "Yes" : "No") << '\n';
31 |     return 0;
32 | }
```

## 公开样例

样例 1 输入

```
5
00001
00011
00110
00100
11100
```

样例 1 输出

```
Yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- HAUTOJ 原题

## 9.17 | 总 503/526 抓牛 ( PID 1522 )

出处：2019级河南工大新生周赛总决赛@王嘉豪专场 ( D , CID 1059 ) | 训练定位：进阶 | 搜索、图论与树

标签：栈与队列、搜索、字符串、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：4/24 ( 16.7% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：在一个数轴上有N和K两个点，CXK在N( $0 \leq N \leq 100000$ ) 这个位置，奶牛在K( $0 \leq K \leq 100000$ ) 那个位置，CXK要抓住奶牛，有三种移动方法：1、向前走一步，耗时一分钟。2、向后走一步，耗时一分钟。3、向前移动到当前位置的两倍，耗时一分钟。问CXK抓到奶牛的最少时间。PS：奶牛是不会动的。

输入要点：输入两个数N,K

输出要点：输出最短时间

题面提示：5-10-9-18-17

### 零基础先修

- 栈后进先出，队列先进先出；BFS 使用队列，括号/单调性问题常使用栈。
- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。
- string 可按下标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：抓牛 BFS的基础题 `#include <iostream> #include <algorithm> #include <cmath> #include <ctype.h> #include <cstring> #include <cstdio> #include <sstream> #include <cstdlib> #include <iomanip> #include <string> #include <queue>`
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

### 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。
- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 确认 0/1 起下标、首尾元素和  $n=1$ 。

### C++17 参考实现

```
1 | #include <iostream>
2 | #include <algorithm>
3 | #include <cmath>
4 | #include <ctype.h>
5 | #include <cstring>
6 | #include <cstdio>
```

```

7 | #include <sstream>
8 | #include <cstdlib>
9 | #include <iomanip>
10 | #include <string>
11 | #include <queue>
12 | using namespace std;
13 | const int inf = 0x3f3f3f3f;
14 | int n, k;
15 | queue<int> q;
16 | int flag[100002];
17 | int bfs() {
18 |     q.push(n), flag[n] = 1;
19 |     while (!q.empty()) {
20 |         int date = q.front();
21 |         int b;
22 |         q.pop();
23 |         for (int i = 1; i <= 3; i++) {
24 |             if (i == 1)
25 |                 b = date + 1;
26 |             else if (i == 2)
27 |                 b = date - 1;
28 |             else if (i == 3)
29 |                 b = date * 2;
30 |             if (b >= 0 && b <= 100001 && !flag[b]) {
31 |                 q.push(b);
32 |                 flag[b] = flag[date] + 1;
33 |                 if (b == k)
34 |                     return flag[b] - 1;
35 |             }
36 |         }
37 |     }
38 | }
39 | int main() {
40 |     while (~scanf("%d %d", &n, &k)) {
41 |         if (n >= k)
42 |             cout << n - k << endl;
43 |         else
44 |             cout << bfs() << endl;
45 |     }
46 |     return 0;
47 | }

```

## 公开样例

样例 1 输入

5 17

样例 1 输出

4

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [公开题解/参考来源 2](#)

## 9.18 | 总 504/526 lwyj的最短路径 ( PID 1715 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（G，CID 1073）| 训练定位：进阶 | 搜索、图论与树

标签：模拟、图论、数学 | 限制：1.000 S / 128 MB | 历史统计：2/14 ( 14.3% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：这天lwyj走在一个类似棋盘的地面上(如下图所示)，分别给出lwyj的起点坐标和终点坐标，现在要找出从起点到终点的最短距离，lwyj一眼就看出了答案然后想要考考你，lwyj知道你也肯定很聪明，请你找出从起点到终点最短路径的长度及最短路径的走法并按相应的

格式输出(路径最短的同时优先斜着走)

输入要点：第一行是lwjy的起点坐标，第二行是lwjy的终点坐标。每个坐标由横向坐标A-H和纵向坐标1-8组成(中间没有空格)

输出要点：在第一行中输出一个正整数n，其中n为lwjy从起点走到终点的最小步数。然后在接下来的n行中输出lwjy的动作。每个动作都用8个动作中的一个来描述:L, R, U, D, LU, LD, RU或RD。

L、R、U、D分别代表左、右、上、下移动，2个字母组合代表对角线移动，路径最短的同时优先斜着走

题面提示：根据样例看图模拟一下过程即可。

## 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。
- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：lwjy的最短路径 思路：本题采用贪心的思想，也就是说可以斜着走就斜着走，因为这样其实是一步走两个格子，当不能斜着走的时候再水平移动或者竖直移动。 具体分析：首先要明确一点，从起点到终点本质上的区别是竖直方向距离和水平方向距离的区别，那么走一步产生的影响有三种，水平方向间距减少1，竖直方向间距减少1，水平方向和竖直方向的间距同时减少1。那么很显然最少的操作数就是水平方向间距的差值和竖直方向间距的差值的最大值(由起点到终点的最理想状态)，分析完了这个之后我们可以直接写代码了。第一种写法：从起点到终点的横纵坐标的差值有四种情况(根据正负的组合即可得出有四种)，那么我们可以分开讨论再模拟走的过程并输出相应的操作即可
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。
- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <math.h>
3 | int main() {
4 |     char x;
5 |     int y;
6 |     char a;
7 |     int b;
8 |     scanf("%c%d %c%d", &x, &y, &a, &b);
9 |     int ans1 = a - x, ans2 = b - y;
10 |    printf("%d\n", abs(ans1) > abs(ans2) ? abs(ans1) : abs(ans2)); // 最短路径
11 |    if (ans1 >= 0 && ans2 <= 0) // 需要往右下走
12 |    {
13 |        while (ans1 > 0 && ans2 < 0) {
14 |            ans1--;
15 |            ans2++;
16 |            puts("RD");
17 |        }
18 |        if (ans1 == 0) {
19 |            while (ans2 < 0) {
20 |                ans2++;
```

```

21 |         puts("D");
22 |     }
23 | } else if (ans2 == 0) {
24 |     while (ans1 > 0) {
25 |         ans1--;
26 |         puts("R");
27 |     }
28 | }
29 | } else if (ans1 >= 0 && ans2 >= 0) // 需要往右上走
30 | {
31 |     while (ans1 > 0 && ans2 > 0) {
32 |         ans1--;
33 |         ans2--;
34 |         puts("RU");
35 |     }
36 |     if (ans1 == 0) {
37 |         while (ans2 > 0) {
38 |             ans2--;
39 |             puts("U");
40 |         }
41 |     } else if (ans2 == 0) {
42 |         while (ans1 > 0) {
43 |             ans1--;
44 |             puts("R");
45 |         }
46 |     }
47 | } else if (ans1 <= 0 && ans2 >= 0) // 需要往左上走
48 | {
49 |     while (ans1 < 0 && ans2 > 0) {
50 |         ans1++;
51 |         ans2--;
52 |         puts("LU");
53 |     }
54 |     if (ans1 == 0) {
55 |         while (ans2 > 0) {
56 |             ans2--;
57 |             puts("U");
58 |         }
59 |     } else if (ans2 == 0) {
60 |         while (ans1 < 0) {
61 |             ans1++;
62 |             puts("L");
63 |         }
64 |     }
65 | } else if (ans1 <= 0 && ans2 <= 0) // 需要往左下走
66 | {
67 |     while (ans1 < 0 && ans2 < 0) {
68 |         ans1++;
69 |         ans2++;
70 |         puts("LD");
71 |     }
72 |     if (ans1 == 0) {
73 |         while (ans2 < 0) {
74 |             ans2++;
75 |             puts("D");
76 |         }
77 |     } else if (ans2 == 0) {
78 |         while (ans1 < 0) {
79 |             ans1++;
80 |             puts("L");
81 |         }
82 |     }
83 | }
84 | }

```

## 公开样例

### 样例 1 输入

```

G6
D5

```

### 样例 1 输出

```

3
LD
L
L

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。



## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)

## 9.19 | 总 505/526 cgg学长的致富之路 ( PID 1716 )

出处：2021级HAUT新生周赛（五）@常浩锋专场（H，CID 1073）| 训练定位：进阶 | 搜索、图论与树

标签：搜索、动态规划、排序、数组与序列 | 限制：1.000 S / 128 MB | 历史统计：14/45 ( 31.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：这天，cgg学长来到了一个金库迷宫中，他在这个迷宫里面发现了m个小金库，每个小金库里面有ai个装有bi块金子的袋子，幸运的是这些袋子大小都是一样的，不考虑袋子变形的因素，而cgg学长的背包只能装下n个袋子，因此本题需要你帮他求出一个最优的致富方案。

输入要点：输入的第一行包含整数n( $1 \leq n \leq 2 \cdot 10^8$ )和整数m( $1 \leq m \leq 20$ )。第i + 1行包含一对数字ai和bi( $1 \leq a_i \leq 10^8, 1 \leq b_i \leq 10$ )。所有输入的数字都是整数。

输出要点：输出一个整数代表最优致富方案的大小。

题面提示：7个袋子选第一个金库的五个袋子和第三个金库的两个袋子可以获得最大财富值 $5 \cdot 10 + 6 \cdot 2 = 62$ ；cgg学长绝对聪明哦。

### 零基础先修

- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。
- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- sort 默认升序；排序后应利用相邻性、前缀性质或单调性，而不只是机械排序。
- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把需要比较的记录按主关键字排序；若题目规定并列规则，再加入次关键字。
3. 核心处理采用：cgg学长的致富之路思路：这道题的要求已经比较明确了，就是找最优的财富方案所得到的财富的总数，需要特殊注意的一点就是袋子都是相同的，那么我们只需要用现有的袋子优先去装金钱多的袋子即可。 具体分析：对金库每个袋子里面金币数量进行一个降序排序再去判断就可以了。参考代码如下： 写法1 暴力模拟
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

### 复杂度

通常为  $O(n \log n)$ ，主要来自排序

### 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。
- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。
- 比较器必须形成严格弱序；相等时按题目指定的次关键字处理。

## C++17 参考实现

```
1 | #include <algorithm>
2 | #include <iostream>
3 | using namespace std;
4 | typedef struct jg {
5 |     int first, second;
6 | } ff;
7 | ff a[50];
8 | bool cmp(ff q, ff w) {
9 |     return q.second > w.second;
10 | }
11 | int main() {
12 |     int n, m, x, y;
13 |     scanf("%d %d", &n, &m);
14 |     for (int i = 1; i <= m; i++) {
15 |         scanf("%d %d", &a[i].first, &a[i].second);
16 |     }
17 |     sort(a + 1, a + m + 1, cmp); // 使用sort
18 |     int ans = 0;
19 |     for (int i = 1; i <= m; i++) {
20 |         if (n >= a[i].first) {
21 |             ans += a[i].first * a[i].second;
22 |             n -= a[i].first;
23 |         } else {
24 |             ans += n * a[i].second;
25 |             break;
26 |         }
27 |     }
28 |     printf("%d\n", ans);
29 | }
```

## 公开样例

### 样例 1 输入

```
7 3
5 10
2 5
3 6
```

### 样例 1 输出

```
62
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.20 | 总 506/526 年少有为 ( PID 1731 )

出处：2021级HAUT新生周赛（七）@王佳琪专场（F，CID 1075）| 训练定位：进阶 | 搜索、图论与树

标签：数组与序列、图论、动态规划、字符串 | 限制：1.000 S / 128 MB | 历史统计：0/4 ( 0.0% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：设17673学长在1处，C+++创始人在n处，在1~n的路上共有1,2,3...,n个车站，从车站i到j所需的路费为 $r(i,j)$  ( $1 \leq i < j \leq n$ )。17673学长正在研究哈密顿路径，没有时间规划怎么换乘车辆，需要你来规划路费最少的路线，但是17673学长只喜欢最短路，所以要求你规划的路线路程最短。

输入要点：第一行一个正整数n，表示n个公交站

接下来n-1行是一个半矩阵 $r(i,j)$  ( $1 \leq i < j \leq n$ )

输出要点：输出计算从17673学长的位置1到C+++创始人家n所需的最少车费

题面提示：n≤200，保证计算过程中任何时刻数值都不超过  $10^7$ 。

样例中的输入矩阵如下

- 5 15

-- 7

---

## 零基础先修

- 数组下标必须在合法范围内；扫描时明确当前位置、前缀信息和边界元素。
- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 先用一句话定义 dp 状态，再写转移来源、初值、遍历顺序和最终答案。
- string 可按标访问字符；注意长度、空串、大小写、标点和 getline 与 cin 混用。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 定义 dp 状态的精确含义，列出初值，并只从已经合法的状态转移。
3. 核心处理采用：年少有为动态规划，搜索，最短路 动态规划法从起点递推从起点到这个点的最短距离，推到终点后直接输出结果即可 C
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 状态含义覆盖了当前前缀/阶段的全部必要信息，初始状态与空选择一致。
- 每次转移枚举当前决策的所有合法来源，因此不会漏掉可行方案；取最优值也不会保留劣解。
- 处理完全部输入后，目标状态恰好对应题目要求，所以读出该状态即为答案。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 确认 0/1 起下标、首尾元素和  $n=1$ 。
- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 初值不能默认全 0 都可达；0/1 背包要倒序遍历容量。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | #include <string.h>
3 | int min(int a, int b) {
4 |     if (a > b)
5 |         return b;
6 |     else
7 |         return a;
8 | }
9 | int main() {
10 |     int n;
11 |     int a[201][201], dp[201];
12 |     scanf("%d", &n);
13 |     memset(a, 127, sizeof(a));
14 |     for (int i = 1; i < n; i++) {
15 |         for (int j = i + 1; j <= n; j++) {
16 |             scanf("%d", &a[i][j]);
17 |         }
18 |     }
19 |     for (int i = 1; i <= n; i++) {
20 |         dp[i] = a[1][i];
21 |     }
22 |     for (int i = 2; i <= n; i++) {
23 |         for (int j = 1; j < i; j++) {
24 |             dp[i] = min(dp[i], dp[j] + a[j][i]);
25 |         }
26 |     }
```

```
27 |     printf("%d", dp[n]);
28 | }
```

## 公开样例

样例 1 输入

```
3
5 15
7
```

样例 1 输出

```
12
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.21 | 总 507/526 小C刷跑 ( PID 1743 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（A，CID 1077）| 训练定位：进阶 | 搜索、图论与树

标签：计算几何、最短路、分类讨论 | 限制：1.000 S / 128 MB | 历史统计：22/192 ( 11.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出小C还需要跑的最短距离，结果保留两位小数

输入要点：输入三个整数：x, y, R (  $1 \leq x, y \leq 1e9$ ,  $4R^2 < x^2 + y^2$  )

输出要点：输出小C还需要跑的最短距离，结果保留两位小数

题面提示：小C先从1号点 ( 0, 0 ) 跑到 ( 4, 3 )，此时距离2号点距离为1，在打卡范围内，完成2号点打卡；

随后小C再从 ( 4, 3 ) 跑到 ( 7.2, 0.6 )，此时距3号点距离为1，在打卡范围内，完成3号点打卡；

打卡完成，一共跑了9.00的距离

### 零基础先修

- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：若  $R \geq y$ ，沿 x 轴直跑就进入 2 号点打卡圆，最后到距 3 号点 R 处停止，距离  $2x - R$ 。否则 2 号打卡点最优取 (x,y-R)，到起点和 3 号点的距离相等，跑程为两倍该距离，再减去最后无需进入 3 号圆心的 R。平方使用 long double 防溢出。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long double x, y, radius;
5 |     cin >> x >> y >> radius;
6 |     long double ans;
7 |     if (radius >= y) {
8 |         ans = 2 * x - radius;
9 |     } else {
10 |         long double vertical = y - radius;
11 |         ans = 2 * sqrt(x * x + vertical * vertical) - radius;
12 |     }
13 |     cout << fixed << setprecision(2) << (double)ans << '\n';
14 |     return 0;
15 | }
```

## 公开样例

样例 1 输入

4 4 1

样例 1 输出

9.00

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 9.22 | 总 508/526 小C的选择 ( PID 1745 )

出处：2021级HAUT新生周赛（寒假专场）@苏锦鹏专场（C，CID 1077）| 训练定位：进阶 | 搜索、图论与树

标签：搜索 | 限制：1.000 S / 128 MB | 历史统计：6/35 ( 17.1% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个整数，表示小C为了获得刚好8个积分有多少种不同的任务选择

输入要点：第一行输入一个整数 $n$  ( $3 \leq n \leq 25$ )，表示目前剩余未完成任务数量

第二行输入 $n$ 个整数，表示每个任务的分值 ( $1 \leq a[i] \leq 5$ )

输出要点：输出一个整数，表示小C为了获得刚好8个积分有多少种不同的任务选择

题面提示：对于样例中的任务，小C分别选择第1、2个任务；第1、3个任务；第2、3个任务，一共3种选择方案

## 零基础先修

- 搜索必须有终止条件和访问标记；先估算状态数，避免指数爆炸。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：小C的选择剥去背景，这题其实也就是求选x个数，让其总和为8，有多少种选法，是一道深搜题 我们可以定义dfs函数为，x则表示当前这次选择选到哪一个任务，sum则表示当前已选的任务的总分为多少dfs(x,sum)dfs(x,sum)dfs(x,sum)那么，对于每一个任务我都可以有选和不选两种情况，对此，我们都得将其考虑进去，然后再通过递归调用进入下一个任务选择判断，如果当前的所选总分已经达到8分，则进行计数并返回即可 上代码
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 入队/递归时及时标记访问，避免重复状态和死循环。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int n, ans, a[30];
3 | void dfs(int x, int sum) {
4 |     if (sum == 8) // 总和等于8，即可技术并返回
5 |     {
6 |         ans++;
7 |         return;
8 |     }
9 |     if (x > n || sum > 8)
10 |         return;
11 |     dfs(x + 1, sum + a[x]); // 选择当前的任务，任务总分加上a[x]
12 |     dfs(x + 1, sum);        // 不选当前的任务，任务总分不变
13 | }
14 | int main() {
15 |     scanf("%d", &n);
16 |     for (int i = 1; i <= n; i++)
17 |         scanf("%d", &a[i]);
18 |     dfs(1, 0);
19 |     printf("%d", ans);
20 |     return 0;
21 | }
```

## 公开样例

样例 1 输入

```
3
4 4 4
```

样例 1 输出

```
3
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 9.23 | 总 509/526 约好了下午三点，在记忆博物馆 ( PID 1768 )

出处：2022级HAUT新生周赛（二）@武其轩专场（A，CID 1080）| 训练定位：进阶 | 搜索、图论与树

标签：图论、函数图、模拟 | 限制：1.000 S / 128 MB | 历史统计：14/240 ( 5.8% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：工作人员告诉你：记忆博物馆只能通过魔法列车（魔法列车一共有12个站点，每条运行的魔法列车只有起点和终点且为单向行驶，不会停靠在别的站点，随时都有车次）到达，你现在所处的位置为1号站点，而记忆博物馆在n号站点（ $2 \leq n \leq 12$ ），并且给了你目前开通的t（ $1 \leq t \leq 12$ ）条魔法列车线路及其运行一次所用的时间。机智的你发现，对于每个站点，最多只能成为一次起点。现……

输入要点：第一行输入两个整数n和t。

随后t行，每行输入三个整数a, b, x，分别表示该条列车线路的起点、终点和耗时（单位为分钟）。

输出要点：两行，如果能按时抵达输出"yes"（不含引号，下同），并在第二行输出最少的用时（整数，单位分钟，下同）。否则输出"No"，如果能够抵达在第二行输出最少用时，如果无法抵达则输出"0"。

题面提示： $2 \leq n \leq 12$ ,

$1 \leq t \leq 12$ ,

$1 \leq a, b \leq 12, a \neq b$ ,

$1 \leq x \leq 120$ 。

补充样例

输入：

```
4 3
1 3 20
2 3 30
3 2 10
```

输出：

```
No
0
```

### 零基础先修

- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：每个站点最多只有一条出边，因此从 1 号站出发不存在路线选择，只需沿唯一后继一直走。用 visited 防止线路形成环；累加沿途耗时。到达 n 后按 120 分钟判断 yes/No，遇到无出边或环则说明不能到达。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。
- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

### 复杂度

$O(t+n)$  时间、 $O(n)$  空间

## 易错点

- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, t;
7 |     cin >> n >> t;
8 |     vector<int> nextStation(13, -1), cost(13, 0);
9 |     for (int i = 0; i < t; ++i) {
10 |         int a, b, x;
11 |         cin >> a >> b >> x;
12 |         nextStation[a] = b;
13 |         cost[a] = x;
14 |     }
15 |     vector<bool> visited(13, false);
16 |     int cur = 1, total = 0;
17 |     while (cur != n && cur != -1 && !visited[cur]) {
18 |         visited[cur] = true;
19 |         if (nextStation[cur] == -1) {
20 |             cur = -1;
21 |             break;
22 |         }
23 |         total += cost[cur];
24 |         cur = nextStation[cur];
25 |     }
26 |     if (cur == n) {
27 |         cout << (total <= 120 ? "yes" : "No") << '\n' << total << '\n';
28 |     } else {
29 |         cout << "No\n0\n";
30 |     }
31 |     return 0;
32 | }
```

## 公开样例

样例 1 输入

```
4 3
1 3 20
2 4 30
3 2 10
```

样例 1 输出

```
yes
60
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)



## 9.24 | 总 510/526 关键词 ( PID 1775 )

出处：2022级HAUT新生周赛（三）@焦小航专场（F，CID 1081）| 训练定位：进阶 | 搜索、图论与树

标签：观察、网格、模拟 | 限制：1.000 S / 128 MB | 历史统计：17/106 ( 16.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，最后涂画的颜色是哪种颜色，如果是红色输出R，否则输出B

输入要点：第一行给出一个  $t$ ，代表有  $t$  组询问。（ $1 \leq t \leq 4000$ ）

每组包括一个  $8 \times 8$  的网格。每组之间有个空格

输出要点：最后涂画的颜色是哪种颜色，如果是红色输出R，否则输出B

### 零基础先修

- 模拟就是维护题面中的状态，并严格按事件顺序更新；建议先列出状态变量和更新时刻。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
- 核心处理采用：最后若画的是红色，最终网格一定还保留一整行 R；若不存在整行 R，最后一次只能是覆盖某列的蓝色。枚举 8 行，找到全 R 行输出 R，否则输出 B。这是由“后画覆盖先画”推出的判定，不必还原全部操作。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 初始变量与题面初始状态相同。
- 假设某一步前程序状态正确，本步按相同规则和顺序更新，所以更新后仍与题面过程一致。
- 由归纳法，全部步骤结束时程序状态正确，输出也正确。

### 复杂度

每组  $O(64)$  时间、 $O(64)$  空间

### 易错点

- 按题面规定的先后顺序更新，避免本轮新状态被同一轮重复使用。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         vector<string> grid(8);
10 |         for (string& row : grid)
11 |             cin >> row;
12 |         bool red = false;
13 |         for (const string& row : grid) {
14 |             if (row == "RRRRRRRR")
15 |                 red = true;
16 |         }
17 |         cout << (red ? 'R' : 'B') << '\n';
18 |     }
19 |     return 0;
20 | }
```

### 公开样例

样例 1 输入

```
1
....B...
....B...
....B...
RRRRRRRR
....B...
....B...
....B...
....B...
```

样例 1 输出

```
R
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 9.25 | 总 511/526 我要成为复数高手 ( PID 1845 )

出处：2023级HAUT新生周赛（四）@牛浩然专场（A，CID 1089）| 训练定位：进阶 | 搜索、图论与树

标签：图论、数学 | 限制：1.000 S / 128 MB | 历史统计：76/461 ( 16.5% )

代码口径：公开题解代码整理；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：小A在学习复数后认为它没有什么用处，但万能的小精灵告诉他复数是很重要的，小精灵告诉他：“复数是平面上点和另一平面上的点的一个变换，复数能表示平移，旋转，镜射，伸缩在几何和图形处理有极为重要的应用。电磁波信号就是通过付里叶变换和逆变换实现，它们就是一对复变函数。当今的量子力学的最基本方程，薛定谔方程是由复数来建立。量子力学的理论是基于复变量的希尔伯特空间实现的……”

输入要点：第一行一个整数 $T$  ( $0 < T < 1000$ )。

接下来 $T$ 行，每行四个整数 $a_1, b_1, a_2, b_2$  ( $-100000 < a_1, b_1, a_2, b_2 < 100000$ )。

输出要点：每一行输出 $a_1 + b_1i$ 与 $a_2 + b_2i$ 的和，并且以 $a+bi$ 的形式输出，若可以简写，输出简写形式。

题面提示：注意 $a+bi$ 的简写形式。

### 零基础先修

- 建图前明确点、边和方向；邻接表适合稀疏图，遍历复杂度通常为  $O(V+E)$ 。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把对象建成点和边，初始化访问/距离信息，再按 BFS、DFS 或题解给出的图算法扩展。
3. 核心处理采用：我要成为复数高手 按照复数加法的运算规则运算即可，注意一些复数有简写形式：1.当虚数的系数为1/-1时系数省略，2.当实数部分的系数为0时，实数部分省略。3.当虚数部分的系数为0时，虚数部分省略。5.当虚数部分的符号为负数时，加号省略。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 建图把题面中的每个合法状态/关系一一映射为点和边。

- 访问标记保证每个状态按算法规则处理；扩展操作覆盖所有且仅覆盖合法转移。
- 因此遍历结束后的可达、距离或累计结果与原问题完全对应。

## 复杂度

按一次完整遍历估算，通常为  $O(n+m)$

## 易错点

- 核对有向/无向、重边、自环、不可达点和点编号范围。
- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

## C++17 参考实现

```
1 | #include <stdio.h>
2 | int main() {
3 |     int t;
4 |     scanf("%d", &t);
5 |     while (t--) {
6 |         int a, b, c, d, a1, a2;
7 |         scanf("%d %d %d %d", &a, &b, &c, &d);
8 |         a1 = a + c;
9 |         a2 = b + d;
10 |         if (a1 == 0) // 实数部分系数为0
11 |         {
12 |             if (a2 == 1) {
13 |                 printf("i\n");
14 |             } else if (a2 == -1) {
15 |                 printf("-i\n");
16 |             } else if (a2 == 0) // 虚数系数的特殊情况。
17 |             {
18 |                 printf("0\n");
19 |             } else {
20 |                 printf("%di\n", a2);
21 |             }
22 |         } else // 实数部分不为0
23 |         {
24 |             printf("%d", a1);
25 |             if (a2 == 0) {
26 |                 printf("\n");
27 |             } else if (a2 == -1) {
28 |                 printf("-i\n");
29 |             } else if (a2 == 1) {
30 |                 printf("+i\n");
31 |             } // 虚数系数的特殊情况。
32 |             else
33 |                 printf("%+di\n", a2);
34 |         }
35 |     }
36 |     return 0;
37 | }
```

## 公开样例

样例 1 输入

```
5
1 2 1 -3
1 1 4 5
1 4 -1 -4
6 7 6 -7
4 5 4 -4
```

样例 1 输出

```
2-i
5+6i
0
12
8+i
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“搜索、图论与树”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 模块 10 | 计算几何、概率与综合

这类题通常综合公式、精度、建模与实现。先确定数学模型，再决定是否需要浮点、期望线性性、距离公式或综合数据结构。

本模块共 15 道唯一题。

## 10.1 | 总 512/526 Add Water ( PID 1212 )

出处：2016级河南工大新生周赛（二）（A，CID 1007）| 训练定位：入门 | 计算几何、概率与综合

标签：空间几何、欧氏距离、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：176/293 ( 60.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，一行距离，保留三位小数

输入要点：第一行三个数表示第一个点的x y z坐标

第二行三个数表示第二个点的x y z坐标

输出要点：一行距离，保留三位小数

题面提示：输入的每个数字绝对值小于<10000

### 零基础先修

- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：三维欧氏距离为  $\sqrt{(x1-x2)^2+(y1-y2)^2+(z1-z2)^2}$ 。先用 `double` 计算三个坐标差，平方求和后开根号，保留三位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double x1, y1, z1, x2, y2, z2;
5 |     cin >> x1 >> y1 >> z1;
```

```
6 |     cin >> x2 >> y2 >> z2;
7 |     double dx = x1 - x2;
8 |     double dy = y1 - y2;
9 |     double dz = z1 - z2;
10 |     cout << fixed << setprecision(3) << sqrt(dx * dx + dy * dy + dz * dz) << '\n';
11 |     return 0;
12 | }
```

公开样例

样例 1 输入

```
1 1 1
1 1 1
```

样例 1 输出

```
0.000
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 10.2 | 总 513/526 挂谷集 ( PID 1941 )

出处：河南工大2025新生周赛 ( 7 ) ——命题人：牛见青 ( B , CID 1109 ) | 训练定位：入门 | 计算几何、概率与综合

标签：数学、计算几何、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：81/150 ( 54.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：挂谷转针问题，询问在平面上是否有一个面积最小的区域 D，在其中针可以旋转 360 度。挂谷宗一于 1917 年首次对于凸集提出此问题。

输入要点：一个实数x，满足 $0 < x \leq 1e9$

输出要点：一个实数y，保留两位小数

零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 浮点题按误差要求输出足够位数，通常使用 fixed 与 setprecision。

思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：题面给出长为 1 的针所需最小凸集面积为  $1/\sqrt{3}$ 。长度按 x 缩放时，所有线性尺度乘 x，面积因此乘  $x^2$ ，所以答案为  $x^2/\sqrt{3}$ 。用 fixed 与 setprecision(2) 完成四舍五入到两位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

O(1) 时间、O(1) 空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long double x;
5 |     cin >> x;
6 |     long double area = x * x / sqrtl(3.0L);
7 |     cout << fixed << setprecision(2) << area << '\n';
8 |     return 0;
9 | }
```

## 公开样例

样例 1 输入

1

样例 1 输出

0.58

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 10.3 | 总 514/526 三角形面积 ( PID 1216 )

出处：2016级河南工大新生周赛 ( 三 ) ( C , CID 1008 ) | 训练定位：基础提高 | 计算几何、概率与综合

标签：计算几何、叉积、浮点误差 | 限制：1.000 S / 128 MB | 历史统计：298/806 ( 37.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：给出三个点的二维坐标，判断这三个点是否能构成三角形，如果能，输出三角形的面积，否则输出No answer。

输入要点：输入为一行六个实数，每两个表示一个点的坐标。 $-100 \leq x, y \leq 100$ 。

输出要点：如果能构成三角形，则输出三角形面积，结果保留两位小数。

如果不能，输出No answer。

## 零基础先修

- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 浮点相等应使用允许误差；比较前确认题目要求的绝对/相对误差。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。

2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：从第一个点出发构造向量  $u=(x_2-x_1, y_2-y_1)$  与  $v=(x_3-x_1, y_3-y_1)$ 。二维叉积  $u \times v = y_2 y_3 - y_1 y_3$  的绝对值是三角形面积的两倍。若其绝对值接近 0，三点共线，输出 No answer；否则除以 2 并保留两位小数。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double x1, y1, x2, y2, x3, y3;
5 |     cin >> x1 >> y1 >> x2 >> y2 >> x3 >> y3;
6 |     double twiceArea = (x2 - x1) * (y3 - y1) - (y2 - y1) * (x3 - x1);
7 |     if (fabs(twiceArea) < 1e-10) {
8 |         cout << "No answer\n";
9 |     } else {
10 |         cout << fixed << setprecision(2) << fabs(twiceArea) / 2.0 << '\n';
11 |     }
12 |     return 0;
13 | }
```

## 公开样例

样例 1 输入

```
0 0 3 0 0 4
```

样例 1 输出

```
6.00
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 10.4 | 总 515/526 Beru-taxi ( PID 1224 )

出处：2016级河南工大新生周赛（四）（E，CID 1009）| 训练定位：基础提高 | 计算几何、概率与综合

标签：欧氏距离、浮点数、最值 | 限制：1.000 S / 128 MB | 历史统计：24/62 ( 38.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，Print a single real value — the minimum time Vasily needs to get in any of the Beru-taxi cars. You answer will be considered correct if its absolute or relative error does.....

输入要点：The first line of the input contains two integers  $a$  and  $b$  ( $-100 \leq a, b \leq 100$ ) — coordinates of Vasiliy's home.

The second line contains a single integer  $n$  ( $1 \leq n \leq 1000$ ) — the number of available Beru-taxi cars nearby.

The  $i$ -th of the following  $n$  lines contains three integers  $x_i, y_i$  and  $v_i$  ( $-100 \leq x_i, y_i \leq 100, 1 \leq v_i \leq 100$ ) — the coordinates of the  $i$ -th car and its speed.

It's allowed that several cars are lo.....

输出要点：Print a single real value — the minimum time Vasiliy needs to get in any of the Beru-taxi cars. Your answer will be considered correct if its absolute or relative error does not exceed  $10^{-6}$ .

Namely: let's assume that your answer is  $a$ , and the answer of the jury is  $b$ . The checker program will consider your answer corr.....

题面提示：In the first sample, first taxi will get to Vasiliy in time 2, and second will do this in time 1, therefore 1 is the answer.

In the second sample, cars 2 and 3 will arrive simultaneously.

## 零基础先修

- 这类题通常综合公式、精度、建模与实现。先确定数学模型，再决定是否要浮点、期望线性性、距离公式或综合数据结构。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：第  $i$  辆车直线驶向家，路程由欧氏距离公式  $\sqrt{(x_i-a)^2+(y_i-b)^2}$  给出，时间是路程除以速度  $v_i$ 。遍历所有车 取最小时间，输出足够多的小数位以满足  $1e-6$  误差要求。使用 `hypot` 可直接、稳定地计算二维距离。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(n)$  时间、 $O(1)$  空间

## 易错点

- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     double homeX, homeY;
7 |     int n;
8 |     cin >> homeX >> homeY >> n;
9 |     double ans = numeric_limits<double>::infinity();
10 |    for (int i = 0; i < n; ++i) {
11 |        double x, y, speed;
12 |        cin >> x >> y >> speed;
13 |        ans = min(ans, hypot(x - homeX, y - homeY) / speed);
14 |    }
15 |    cout << fixed << setprecision(10) << ans << '\n';
16 |    return 0;
17 | }
```

## 公开样例

样例 1 输入



```
0 0
2
2 0 1
0 2 2
```

样例 1 输出

```
1.0000
```

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

10.5 | 总 516/526 数学 ( PID 1310 )

出处：2017级河南工大新生周赛 ( 一 ) ( C , CID 1028 ) | 训练定位：基础提高 | 计算几何、概率与综合

标签：几何、三角形不等式、余弦定理 | 限制：1.000 S / 128 MB | 历史统计：182/790 ( 23.0% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：给你三根木棍，问能不能组成钝角三角形，能组成请输出“YES”,反之输出“NO”(引号不用输出)，木棍当然不可以折断。

输入要点：三个正整数，每根木棍的长度：a, b, c(0<a,b,c<20)

输出要点：YES or NO

零基础先修

- 这类题通常综合公式、精度、建模与实现。先确定数学模型，再决定是否需要浮点、期望线性性、距离公式或综合数据结构。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：把三边排序为  $x \leq y \leq z$ 。首先必须满足三角形不等式  $x+y>z$ ；在能组成 三角形的前提下，最大角对着 z，根据余弦定理，当  $x^2+y^2<z^2$  时该角为钝角。等号是直角，不能输出 YES。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

O(1) 时间、O(1) 空间

易错点

- 按题目上界估算中间量，相关变量不要退回 int。

C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
```

```

4 |     array<long long, 3> side{};
5 |     cin >> side[0] >> side[1] >> side[2];
6 |     sort(side.begin(), side.end());
7 |     bool triangle = side[0] + side[1] > side[2];
8 |     bool obtuse = side[0] * side[0] + side[1] * side[1] < side[2] * side[2];
9 |     cout << (triangle && obtuse ? "YES" : "NO") << '\n';
10 |     return 0;
11 | }

```

## 公开样例

样例 1 输入

3 4 5

样例 1 输出

NO

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 10.6 | 总 517/526 Takeaway Stealer ( PID 1365 )

出处：2017级河南工大新生周赛总决赛（重现）（E，CID 1049）；2017级河南工大新生周赛总决赛（重现）（E，CID 1039）；2017级河南工大新生周赛总决赛（E，CID 1038） | 训练定位：基础提高 | 计算几何、概率与综合

标签：枚举、字符串、概率、去重 | 限制：1.000 S / 128 MB | 历史统计：11/28 ( 39.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：根据给定输入，输出一个百分数，精确到小数点后2位，表示概率

输入要点：单实例测试

第一行输一个数字  $n$  (  $0 \leq n \leq 50$  )

之后  $n$  行每行一个4位数代表手机号码尾号 ( 0000~9999 )

提示：可以用 `sprintf(temp, "%04d", i);` 的方法将数字转成长度为4的字符串尾号，其中  $i$  是数字

例如  $i=15$ ，那么 `temp = "0015"`

输出要点：输出一个百分数，精确到小数点后2位，表示概率

题面提示：你只要报如下数字：\*714 \*724 6\*14 6\*24 67\*4 671\* 672\* 中其中一个就可以领到外卖 ( 其中\*为任意一个数字 )

### 零基础先修

- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。
- string 可按标访问字符；注意长度、空串、大小写、标点和 `getline` 与 `cin` 混用。
- 概率题先定义事件；期望常用线性性把总量拆成每个对象的贡献。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。

- 核心处理采用：随机号码只有 0000..9999 共一万种，规模极小，直接枚举最稳妥。对每个候选号码，与每份订单逐位比较；只要某一份订单有至少三个对应位置相同，就把该候选计入有利结果且立即停止，避免“可领多份”时重复计数。概率为有利号码数/10000，再转成百分数两位小数。读取订单时保留为字符串并左侧补 0。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

$O(10000 \cdot n \cdot 4)$  时间、 $O(n)$  空间

## 易错点

- 避免越界；若要读取含空格整行，先处理前一次 cin 留下的换行。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | string fourDigits(int value) {
4 |     string s = to_string(value);
5 |     return string(4 - s.size(), '0') + s;
6 | }
7 | int main() {
8 |     ios::sync_with_stdio(false);
9 |     cin.tie(nullptr);
10 |     int n;
11 |     cin >> n;
12 |     vector<string> orders(n);
13 |     for (string& s : orders) {
14 |         cin >> s;
15 |         if (s.size() < 4)
16 |             s = string(4 - s.size(), '0') + s;
17 |     }
18 |     int favorable = 0;
19 |     for (int value = 0; value < 10000; ++value) {
20 |         string guess = fourDigits(value);
21 |         bool canTake = false;
22 |         for (const string& order : orders) {
23 |             int eq = 0;
24 |             for (int i = 0; i < 4; ++i)
25 |                 eq += (guess[i] == order[i]);
26 |             if (eq >= 3) {
27 |                 canTake = true;
28 |                 break;
29 |             }
30 |         }
31 |         favorable += canTake;
32 |     }
33 |     cout << fixed << setprecision(2) << favorable / 100.0 << "%\n";
34 |     return 0;
35 | }
```

## 公开样例

样例 1 输入

```
2
6714
6724
```

样例 1 输出

```
0.64%
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 10.7 | 总 518/526 阿正的平面行进 ( PID 1542 )

出处：2020级HAUT新生周赛（二）@杨哲专场（D，CID 1061） | 训练定位：基础提高 | 计算几何、概率与综合

标签：数学、曼哈顿距离、文件尾输入 | 限制：1.000 S / 128 MB | 历史统计：126/410 ( 30.7% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在阿正一天有若干节课，假设每次阿正和他索要前往的教室都在同一水平面上，给出阿正同学与教室在平面上的坐标，你的任务是算出阿正学会穿墙术后每次最少需要走过多少个单位的距离才能达到教室。

输入要点：多组测试数据。

每组第一行两个非负整数，分别代表阿正在平面上的横坐标与纵坐标；

每组第二行两个非负整数，分别代表教室在平面上的横坐标与纵坐标；

输入保证所有数据都在整形范围内。

输出要点：每组输出一行一个整数代表阿正与教室间的曼哈顿距离。

题面提示：在第一组数据中阿正需要先沿X轴行进10个单位，再沿Y轴行进10个单位才能到达教室。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 多组数据以 EOF 结束时使用 `while (cin >> ...)`，不要额外输出测试数据。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：只能沿坐标轴移动时，最短路就是横坐标差绝对值与纵坐标差绝对值 之和。每组连续读取四个坐标，直到文件尾。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 `long long` 保存乘积；除法前处理除数为 0 和整数截断。
- EOF 题不要先读不存在的 T，也不要循环结束后多输出内容。
- 按题目上界估算中间量，相关变量不要退回 `int`。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
```

```

5 |     cin.tie(nullptr);
6 |     long long x1, y1, x2, y2;
7 |     while (cin >> x1 >> y1 >> x2 >> y2) {
8 |         cout << llabs(x1 - x2) + llabs(y1 - y2) << '\n';
9 |     }
10 |     return 0;
11 | }

```

## 公开样例

样例 1 输入

```

0 0
10 10
5 5
38 79
10005493 841265
99874514 6654283

```

样例 1 输出

```

20
107
95682039

```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

# 10.8 | 总 519/526 光的反射 ( PID 1550 )

出处：2020级HAUT新生周赛（三）@张雍蕤专场（D，CID 1062）| 训练定位：基础提高 | 计算几何、概率与综合

标签：计算几何、反射法、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：110/231 ( 47.6% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，一个实数，代表在x轴上的位置，结果保留六位小数

输入要点：四个实数，分别代表A点坐标x1, y1,B点坐标x2, y2

$-1e6 \leq x1, x2 \leq 1e6$

$0 \leq y1, y2 \leq 1e6$

输出要点：一个实数，代表在x轴上的位置，结果保留六位小数

题面提示：下图是样例的解释

[图片：[https://acm.haut.edu.cn/upload/image/20201213/20201213135727\\_98595.png](https://acm.haut.edu.cn/upload/image/20201213/20201213135727_98595.png)]

## 零基础先修

- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
3. 核心处理采用：把 B 关于 x 轴反射到 B'(x2,-y2)，则 A→O→B 的反射路径变成 A→O→B' 的直线。用线性插值求这条线在 y=0 时的横坐标： $(x1 \cdot y2 + x2 \cdot y1) / (y1 + y2)$ 。两点都在轴上时取 x1 即可。

4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long double x1, y1, x2, y2;
5 |     cin >> x1 >> y1 >> x2 >> y2;
6 |     long double ans;
7 |     if (y1 + y2 == 0)
8 |         ans = x1;
9 |     else
10 |         ans = (x1 * y2 + x2 * y1) / (y1 + y2);
11 |     cout << fixed << setprecision(6) << ans << '\n';
12 |     return 0;
13 | }
```

## 公开样例

样例 1 输入

1 1 7 2

样例 1 输出

3.000000

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [题面图片 1](#)
- [题面图片 2](#)

## 10.9 | 总 520/526 唯一的道标 (PID 1769)

出处：2022级HAUT新生周赛（二）@武其轩专场（G，CID 1080）| 训练定位：基础提高 | 计算几何、概率与综合

标签：数学、计算几何、枚举 | 限制：1.000 S / 128 MB | 历史统计：142/493 (28.8%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出离你位置最近的一个出口的坐标（整数），例如：“(2, 0)”（不包含双引号）。

输入要点：第一行包含  $n, m$  ( $1 \leq n, m \leq 100$ ,  $n$  和  $m$  都是 2 的倍数) 两个正整数，分别代表矩形的长宽（或宽长）。

第二行给出两个整数 $a, b (0 \leq a \leq n, 0 \leq b \leq m)$ ，表示你当前位置坐标 $(a, b)$ 。

输出要点：输出离你位置最近的一个出口的坐标（整数），例如：“(2, 0)”（不包含双引号）。

## 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 枚举要明确搜索空间、判断条件和不会漏解的理由，再用数据范围核对复杂度。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 确定完整且可承受的枚举范围，对每个候选严格检查全部约束。
3. 核心处理采用：把四个出口坐标列出来，比较当前位置到它们的距离平方。平方距离与真实欧氏距离的大小关系相同，却不需要开方，也不会引入浮点误差。题目保证最近出口唯一，因此只保留严格更小的候选即可。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 枚举范围包含每一个可能答案，没有候选被遗漏。
- 判断条件与题目约束等价，所以只接受合法候选。
- 按题意计数、取最优或依序输出这些候选即可得到正确答案。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 按题目上界估算中间量，相关变量不要退回 int。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long long n, m, x, y;
5 |     cin >> n >> m >> x >> y;
6 |     vector<pair<long long, long long>> exits = {
7 |         {n / 2, 0}, {n, m / 2}, {n / 2, m}, {0, m / 2}};
8 |     int best = 0;
9 |     auto distance2 = [&](int i) {
10 |         long long dx = x - exits[i].first;
11 |         long long dy = y - exits[i].second;
12 |         return dx * dx + dy * dy;
13 |     };
14 |     for (int i = 1; i < 4; ++i) {
15 |         if (distance2(i) < distance2(best))
16 |             best = i;
17 |     }
18 |     cout << '(' << exits[best].first << ", " << exits[best].second << ")\n";
19 |     return 0;
20 | }
```

## 公开样例

样例 1 输入

```
4 4
1 2
```

样例 1 输出

```
(0, 2)
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。

- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

### 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 10.10 | 总 521/526 Starlight ( PID 1942 )

出处：河南工大2025新生周赛（7）——命题人：牛见青（D，CID 1109） | 训练定位：基础提高 | 计算几何、概率与综合

标签：数学、计算几何、浮点误差 | 限制：1.000 S / 128 MB | 历史统计：52/133 ( 39.1% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：现在小光和华恋各自随机选择一个整数 $x$ 、 $y$ ，对应直角坐标上的点 $(x, y)$ ；再由隔壁班的奈奈同学确定尺寸参数 $a$ ，请你帮她们确定这个点是否在 $a$ 决定的星形线内。

输入要点：一行，三个整数 $x$ 、 $y$ 、 $a$ ，由空格隔开，其中 $a > 0$ 。

输出要点：一行，若在星形线围成的图形内输出“Position Zero!”，若在线外输出“Fly Me to the Star”。

特殊地，在边上（误差小于 $1e-6$ ），输出“Non Non Da Yo”。

### 零基础先修

- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。
- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 浮点相等应使用允许误差；比较前确认题目要求的绝对/相对误差。

### 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
3. 核心处理采用：星形线边界满足 $|x|^{2/3} + |y|^{2/3} = a^{2/3}$ 。计算左右两边的差：绝对值小于 $1e-6$ 判为边界，左边更小在内部，否则在外部。对负坐标必须先取绝对值，否则分数幂会产生NaN。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long double x, y, a;
5 |     cin >> x >> y >> a;
6 |     long double left = powl(fabsl(x), 2.0L / 3.0L) + powl(fabsl(y), 2.0L / 3.0L);
7 |     long double right = powl(a, 2.0L / 3.0L);
8 |     if (fabsl(left - right) < 1e-6L)
9 |         cout << "Non Non Da Yo\n";
```



```

10 |         else if (left < right)
11 |             cout << "Position Zero!\n";
12 |         else
13 |             cout << "Fly Me to the Star!\n";
14 |         return 0;
15 |     }

```

## 公开样例

样例 1 输入

0 0 1

样例 1 输出

Position Zero!

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)
- [题面图片 1](#)
- [题面图片 2](#)
- [题面图片 3](#)

# 10.11 | 总 522/526 圆的交点 ( PID 1201 )

出处：2016级河南工大新生周赛（一）（D，CID 1006） | 训练定位：进阶 | 计算几何、概率与综合

标签：几何、圆的位置关系、浮点误差 | 限制：1.000 S / 128 MB | 历史统计：130/798 ( 16.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：输入两个圆的坐标和半径，判断它们是否有交点

输入要点：第一行输入三个实数  $x_1, y_1, r$  表示第一个圆的坐标和半径，同理第二行输入三个数表示第二个圆的坐标和半径

输出要点：如果有交点输出 "Yes"，否则输出 "No"（不带双引号）

题面提示：注意：相切也算有交点

## 零基础先修

- 浮点相等应使用允许误差；比较前确认题目要求的绝对/相对误差。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：设圆心距为  $d$ 。两圆相离时  $d > r_1 + r_2$ ，内含且不接触时  $d < |r_1 - r_2|$ ；其余情况都有交点，包括外切、内切以及完全重合。因此判断  $|r_1 - r_2| \leq d \leq r_1 + r_2$ ，浮点比较加入很小的  $\text{eps}$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。

- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

## 复杂度

$O(1)$  时间、 $O(1)$  空间

## 易错点

- 核对读入顺序、变量类型和输出格式。
- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double x1, y1, r1, x2, y2, r2;
5 |     cin >> x1 >> y1 >> r1;
6 |     cin >> x2 >> y2 >> r2;
7 |     double distance = hypot(x1 - x2, y1 - y2);
8 |     const double EPS = 1e-10;
9 |     bool intersects = distance + EPS >= fabs(r1 - r2) && distance <= r1 + r2 + EPS;
10 |     cout << (intersects ? "Yes" : "No") << '\n';
11 |     return 0;
12 | }
```

## 公开样例

样例 1 输入

```
0 0 5
10 0 5
```

样例 1 输出

```
Yes
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

## 10.12 | 总 523/526 Run out of Prison ( PID 1368 )

出处：2017级河南工大新生周赛总决赛（重现）（H，CID 1049）；2017级河南工大新生周赛总决赛（重现）（H，CID 1039）；2017级河南工大新生周赛总决赛（H，CID 1038） | 训练定位：进阶 | 计算几何、概率与综合

标签：概率、构造、最优性证明 | 限制：1.000 S / 128 MB | 历史统计：99/174 ( 56.9% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出一个小数表示答案，精确到小数点后5位

输入要点：先输入一个数T，表示T组测试数据（ $1 \leq T \leq 2018$ ）

之后T行，每行一个数n表示袋子里球的个数（ $1 \leq n \leq 1000000007$ ）

输出要点：输出一个小数表示答案，精确到小数点后5位

题面提示：只有1个球，那么他们肯定能100%活下来，有两个球时，他们只要猜自己摸出的球就可以使概率从1/4提高到1/2

## 零基础先修

- 概率题先定义事件；期望常可用线性性把总量拆成每个对象的贡献。

- 构造题输出不唯一；答案必须逐条满足长度、取值、互异、和或相邻关系等全部约束。

## 思路推导

1. 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
2. 列出状态变量，严格按题面时间顺序更新；构造题同时维护每条输出约束。
3. 核心处理采用：两人可以事先约定：看到编号  $i$  就猜对方也是  $i$ 。两次独立摸球共有  $n^2$  个等可能有序对，只有  $(1,1),(2,2),\dots,(n,n)$  这  $n$  对会使两人同时猜中，所以成功率为  $1/n$ 。任何确定性策略都对应两张从编号到猜测的映射；每个自己摸到的编号至多与一个对方编号组成双方同时正确的配对，有利有序对不可能超过  $n$ ，因此  $1/n$  已是最优。
4. 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 构造过程只使用题目允许的元素与操作。
- 逐步维护的长度、取值、相邻关系或和等不变量保证每条约束始终成立。
- 构造结束时规模达到题目要求，所以输出是一组完整合法答案。

## 复杂度

每组  $O(1)$  时间、 $O(1)$  空间

## 易错点

- 样例只是一个合法答案；本地应写检查器验证约束，而非逐字比较。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int T;
7 |     cin >> T;
8 |     while (T--) {
9 |         double n;
10 |         cin >> n;
11 |         cout << fixed << setprecision(5) << 1.0 / n << '\n';
12 |     }
13 |     return 0;
14 | }
```

## 公开样例

样例 1 输入

```
2
1
2
```

样例 1 输出

```
1.00000
0.50000
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 10.13 | 总 524/526 外卖距离 ( PID 1397 )

出处：2018级河南工大新生周赛 ( 三 ) @陈帅专场 ( B , CID 1042 ) | 训练定位：进阶 | 计算几何、概率与综合

标签：计算几何、点到矩形距离、浮点输出 | 限制：1.000 S / 128 MB | 历史统计：43/212 ( 20.3% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

### 题意与输入输出拆解

题意摘要：最近CS很喜欢点外卖。但是CS发现选择不同的收货地址，最后下单的配送费不一样。CS研究了一下，觉得部分外卖机构的配送费是与店家到收货地址之间的距离有关。店家在平面上可以看成是一个点，如果选择的收货地址在空间概念上可看作一个点，则店家到收货地址之间的距离就是这两点之间的直线。但如果收货地址可看成是一个面，则店家到收货地址之间的距离为店家到这个面上所有点的距离的最小……

输入要点：输入一行，依次包含 ( 平面直角坐标系上 ) 店家的坐标，学校的左下角坐标，学校的长 ( 与X轴正向平行 )、宽 ( 与Y轴正向平行 )。所有输入数据都是整数

输出要点：输出店家到学校的距离 ( 小数点后保留两位 )，占一行。

### 零基础先修

- 几何计算要明确坐标系、距离和边界；平方距离比较可避免不必要的开方。
- 浮点题按误差要求输出足够位数，通常使用 `fixed` 与 `setprecision`。

### 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：矩形横坐标范围为 `[left, left+length]`、纵坐标范围为 `[bottom, bottom+width]`。点在某一维区间内时该维距离为 0，否则为 到最近端点的差；二维直线距离用 `hypot(dx, dy)` 合成。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

### 正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处一次，累计状态即为所求。

### 复杂度

$O(1)$  时间、 $O(1)$  空间

### 易错点

- 不要用字符串精确比较小数样例；按误差要求保留足够精度。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

### C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     double x, y, left, bottom, len, width;
5 |     cin >> x >> y >> left >> bottom >> len >> width;
6 |     double right = left + len, top = bottom + width;
7 |     double dx = max({left - x, 0.0, x - right});
8 |     double dy = max({bottom - y, 0.0, y - top});
9 |     cout << fixed << setprecision(2) << hypot(dx, dy) << '\n';
10 |     return 0;
11 | }
```

### 公开样例

样例 1 输入

```
0 0 1 1 1000 2500
```

样例 1 输出

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)

## 10.14 | 总 525/526 zp与宝可梦 (题面已修改) (PID 1510)

出处：2019级河南工大新生周赛 (五) @潘世泽专场 (D, CID 1057) | 训练定位：进阶 | 计算几何、概率与综合

标签：概率、期望、数学 | 限制：1.000 S / 128 MB | 历史统计：1/11 (9.1%)

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

## 题意与输入输出拆解

题意摘要：根据给定输入，输出概率，如果绝对误差或相对误差小于或等于 $10e-9$ ，则认为答案正确。

输入要点：一行输入 $n$ 个精灵球， $m$ 个人 ( $1 \leq n, m \leq 100000$ )

输出要点：输出概率，如果绝对误差或相对误差小于或等于 $10e-9$ ，则认为答案正确。

## 零基础先修

- 概率题先定义事件；期望常可用线性性把总量拆成每个对象的贡献。
- 把操作过程转成公式前，先在小数据上验证，再证明公式覆盖所有情况。

## 思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 从定义推导公式或不变量，并用小数据验证边界与奇偶/整除关系。
- 核心处理采用：对任意一个精灵球，一名参与者不选它的概率是 $1-1/n$ ， $m$ 人都不选的概率为 $(1-1/n)^m$ ，因此它被拿走的概率为一减该值。利用期望的线性性乘 $n$ ，得到 $n \cdot (1-(1-1/n)^m)$ 。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

## 正确性说明

- 推导把原操作量化为题解中的公式/不变量，两者在每个合法输入上等价。
- 算法直接计算该等价量，并对边界与数值范围使用合适的数据类型。
- 因此所得数值正是题目定义的答案。

## 复杂度

$O(\log m)$  量级幂运算时间、 $O(1)$  空间

## 易错点

- 先用 long long 保存乘积；除法前处理除数为 0 和整数截断。
- 小数位数服务于误差要求；样例位数不同不代表数值答案错误。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     long double n, m;
5 |     cin >> n >> m;
6 |     long double ans = n * (1.0L - powl(1.0L - 1.0L / n, m));
7 |     cout << fixed << setprecision(12) << ans << '\n';
```

```
8 |     return 0;
9 | }
```

公开样例

|             |
|-------------|
| 样例 1 输入     |
| 5 7         |
| 样例 1 输出     |
| 3.951424000 |

训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

链接与来源

- [HAUTOJ 原题](#)

10.15 | 总 526/526 周长之和 ( PID 1864 )

出处：河南工大2024新生周赛（2）——命题人：马家硕（H，CID 1092） | 训练定位：进阶 | 计算几何、概率与综合

标签：最值、几何、扫描 | 限制：1.000 S / 128 MB | 历史统计：55/111 ( 49.5% )

代码口径：依据公开题面/解析独立改写；已通过 C++17 编译与公开样例复核

题意与输入输出拆解

题意摘要：求所有图形周长之和的最小值。

输入要点：第一行一个整数  $n$ ，表示矩形的个数。 $1 \leq n \leq 100$ 。

接下来  $n$  行，每行两个整数  $w$  和  $h$ ，代表矩形的宽度和高度。 $1 \leq w \leq 100, 1 \leq h \leq 100$ 。不可旋转。

输出要点：所有图形周长之和的最小值。

零基础先修

- 这类题通常综合公式、精度、建模与实现。先确定数学模型，再决定是否需要浮点、期望线性性、距离公式或综合数据结构。

思路推导

- 先按输入说明读取数据，并用最小样例手算一次，确认每个变量代表什么。
- 把题目条件翻译成分支或线性扫描，不引入题目没有要求的额外状态。
- 核心处理采用：所有矩形允许重叠时，把它们左下角对齐可让并集落在宽为最大  $w$ 、高为 最大  $h$  的外接矩形内，并达到最小周长  $2(\max W + \max H)$ 。扫描两个最大值。
- 最后按题目要求输出；提交前检查最小值、最大值、空/单元素、重复值和格式边界。

正确性说明

- 扫描前保存的信息与已经处理的输入完全对应。
- 每读入一个新元素，分支覆盖它的全部可能情况，并正确更新累计状态。
- 扫描结束时所有输入恰好处理一次，累计状态即为所求。

复杂度

$O(n)$  时间、 $O(1)$  空间

易错点

- 核对读入顺序、变量类型和输出格式。

- 至少自测最小规模、最大边界和一个一般样例。

## C++17 参考实现

```
1 | #include <bits/stdc++.h>
2 | using namespace std;
3 | int main() {
4 |     ios::sync_with_stdio(false);
5 |     cin.tie(nullptr);
6 |     int n, maxWidth = 0, maxHeight = 0;
7 |     cin >> n;
8 |     while (n--) {
9 |         int width, height;
10 |         cin >> width >> height;
11 |         maxWidth = max(maxWidth, width);
12 |         maxHeight = max(maxHeight, height);
13 |     }
14 |     cout << 2 * (maxWidth + maxHeight) << '\n';
15 |     return 0;
16 | }
```

## 公开样例

样例 1 输入

```
3
3 1
4 1
2 2
```

样例 1 输出

```
12
```

## 训练动作

- 先遮住代码，用 3~5 句话复述状态、步骤和复杂度，再独立实现。
- 自行补至少三个边界：最小规模、全部相等/极端值、恰好卡在条件等号处。
- 把本题归档到“计算几何、概率与综合”，一周后限时重做；若 20 分钟仍无方向，只看思路不看代码。

## 链接与来源

- [HAUTOJ 原题](#)
- [公开题解/参考来源 1](#)

# 按周赛反查题目

## 河南工大2025新生周赛 ( 8 ) ——命题人：庞贺航、高旭

- [比赛页](#)

A 签到题？ ( 1980 )；B 数数数数数组 ( 1979 )；C 编程霸总强制爱 ( 1981 )；D 一觉醒来成为算法高手 ( 1982 )；E 非常会伪装的金钱树 ( 1983 )；F 这是啥杯？ ( 1984 )；G 编程课 ( 1987 )；H 进制转换 ( 1988 )

## 河南工大2025新生周赛 ( 7 ) ——命题人：牛见青

- [比赛页](#)

A 回声 ( 1985 )；B 挂谷集 ( 1941 )；C 身份证校验 ( 1986 )；D Starlight ( 1942 )；E 小圈子 ( 1945 )；F 世界线跃迁 ( 1944 )；G 能量分配 ( 1943 )；H 耐久魔法 ( 1946 )

## 河南工大2025新生周赛 ( 6 ) ——命题人：张康发

- [比赛页](#)

A 这是一道签到题吗？ ( 1974 )；B 这是一道签到题 ( 1975 )；C 这真是一道签到题吗？ ( 1971 )；D Hello,World! ( 1955 )；E 数组求和 ( 1978 )；F 加 and 乘 ( 1969 )；G 多项式相加 ( 多实例 ) ( 1199 )；H 数组切割 ( 1977 )

## 河南工大2025新生周赛 ( 5 ) ——命题人：袁林坤

- [比赛页](#)

A A-B ( 1957 ) ; B 圣诞节前前夕 ( 1947 ) ; C NASA的食物补给 ( 1960 ) ; D 接水 ( 1964 ) ; E 阳台接雨 ( 1966 ) ; F 领导领导 ( 1965 ) ; G 校园文化节宣传标语 ( 1967 ) ; H 合并果子 ( 1968 )

## 河南工大2025新生周赛 ( 4 ) ——命题人：张梦淇

- [比赛页](#)

A 编程大蛇dream ( 1948 ) ; B 梦应归于何处 ( 1950 ) ; C 卡厄斯兰娜的33550336次轮回 ( 1951 ) ; D AK路上的绊脚山 ( 1954 ) ; E 萤火虫大战真蛰虫 ( 1952 ) ; F 幽蝶能留一缕芳 ( 1953 ) ; G 冥界的人员统计 ( 1963 ) ; H Crychic的解散 ( 1949 )

## 河南工大2025新生周赛 ( 3 ) ——命题人：路耀华

- [比赛页](#)

A 直播间红包雨抢券赛 ( 1959 ) ; B 时空快递的“跨天送达” ( 1961 ) ; C 双 11 红包大派送 ( 1939 ) ; D 银行账单的数字美化 ( 1930 ) ; E 数字躲猫猫 ( 1928 ) ; F 机器人的寻宝路径 ( 1929 ) ; G 密码锁的旋转密码 ( 1926 ) ; H 魔法水晶的三角形测试 ( 1940 ) ; I 日历格式化输出 ( 1931 ) ; J 魔法天平的反向称重 ( 1927 )

## 河南工大2025新生周赛 ( 2 ) ——命题人：王伟立

- [比赛页](#)

A 小唐的签到 ( 1936 ) ; B 小唐的日历 ( 1938 ) ; C 小唐的密码 ( 1919 ) ; D 小唐的运气 ( 1932 ) ; E 小唐的真名 ( 1933 ) ; F 小唐的工作 ( 1934 ) ; G 小唐的饼干 ( 1935 ) ; H 小唐的升级 ( 1937 )

## 河南工大2025新生周赛 ( 1 ) ——命题人：程立老师，马家硕

- [比赛页](#)

A 诚信参赛 ( 1924 ) ; B 计算罚时 ( 1923 ) ; C 踱步 ( 1869 ) ; D 长跑评级 ( 1868 ) ; E 找坐标 ( 1867 ) ; F 连续五天早八 ( 1866 ) ; G 求序列和 ( 1865 ) ; H ReLU 函数 ( 1863 )

## 河南工大2024新生周赛 ( 7 ) ——命题人：刘义

- [比赛页](#)

A 圆 ( 1908 ) ; B 旅人分宝 ( 1907 ) ; C  $A*B$  ( 1913 ) ; D 时间加速器 ( 1911 ) ; E 切巧克力 ( 1912 ) ; F 建房子 ( 1909 ) ; G 命运之桥 ( 1906 ) ; H 恶魔 ( 1910 )

## 河南工大2024新生周赛 ( 6 ) ——命题人：魏方

- [比赛页](#)

A 光 ( 1899 ) ; B 天杀的二进制！ ( 1897 ) ; C 千年暗室，一灯即明！ ( 1900 ) ; D Set ( 1901 ) ; E you love 1203! ( 1902 ) ; F 幂幂幂 ( 1903 ) ; G 山雨欲来！ ( 1904 ) ; H Two movies影评 ( 1905 )

## 河南工大2024新生周赛 ( 5 ) ——命题人：刘庭铄

- [比赛页](#)

A 阿拉德的冒险者 ( 1887 ) ; B 大大大 ( 1888 ) ; C 玫瑰花的葬礼 ( 1889 ) ; D 离散数学 ( 1890 ) ; E 简单的运算 ( 1891 ) ; F 哥德巴赫猜想 ( 1892 ) ; G 分解质因数 ( 1893 ) ; H 倾盆的雨下了一整夜 ( 1894 ) ; I 神奇的数字 ( 1895 ) ; J 尖塔第四强的糕手 ( 1896 )

## 河南工大2024新生周赛 ( 4 ) ——命题人：马贺

- [比赛页](#)

A Doki Doki Literature Club! ( 1879 ) ; B 君と彼女と彼女の恋 ( 1880 ) ; C Raging Loop ( 1881 ) ; D Senren Banka ( 1882 ) ; E Haut校草 ( 1883 ) ; F 9nine~九次九日九重色 ( 1884 ) ; G 星空列车与白的旅行 ( 1885 ) ; H NEEDY GIRL OVERDOSE ( 1886 )



## 河南工大2024新生周赛 ( 3 ) ——命题人：张宏泽

• [比赛页](#)

A 这是一道签到题 ( 1871 ) ; B ACMer ( 1872 ) ; C Stock Exchange(Easy Version) ( 1873 ) ; D Stock Exchange(Hard Version) ( 1874 ) ; E  $\wedge$  ( 1875 ) ; F BOX ( 1876 ) ; G 数学考试 ( 1877 ) ; H 200个数字 ( 1878 )

## 河南工大2024新生周赛 ( 2 ) ——命题人：马家硕

• [比赛页](#)

A ReLU 函数 ( 1863 ) ; B 求序列和 ( 1865 ) ; C 连续五天早八 ( 1866 ) ; D 找坐标 ( 1867 ) ; E 长跑评级 ( 1868 ) ; F 踱步 ( 1869 ) ; G 进位 ( 1870 ) ; H 周长之和 ( 1864 )

## 河南工大2024新生周赛 ( 1 ) ——命题人：程立老师，陈兰锴

• [比赛页](#)

A 我爱编程，编程使我快乐。( 1817 ) ; B 我要看奥运!!! ( 1862 ) ; C 二进制翻转 ( 1818 ) ; D 小明的数字 ( 1828 ) ; E 小明的打怪游戏 ( 1829 ) ; F 寝室的宝箱 ( 简单版 ) ( 1821 ) ; G 寝室的宝箱 ( 困难版 ) ( 1822 ) ; H 小明的午餐难题 ( 1826 ) ; I 历经千辛万苦，只为得到你 ( 1850 ) ; J 元素中和之力 ( 1854 )

## 2023级HAUT新生周赛 ( 五 ) @陈兰锴专场

• [比赛页](#)

A 旅行者的抛弃 ( 1853 ) ; B 元素中和之力 ( 1854 ) ; C 旅行传送门 ( 1855 ) ; D 卡萨的阻拦 ( 1856 ) ; E 破碎的未来 ( 1857 ) ; F 怎么了，你累啦？ ( 1858 ) ; G 是心碎的声音 ( 1859 ) ; H 你就是被上天选中的人 ( 1860 )

## 2023级HAUT新生周赛 ( 四 ) @牛浩然专场

• [比赛页](#)

A 我要成为复数高手 ( 1845 ) ; B 依稀记得，那是一个烟雨蒙蒙的清晨 ( 1846 ) ; C 现在是，幻想时刻！ ( 1847 ) ; D 命运之夜 ( 1848 ) ; E 大富翁！启动！ ( 1849 ) ; F 历经千辛万苦，只为得到你 ( 1850 ) ; G 火柴的奇怪用途 ( 1851 ) ; H 奇怪的魔法书 ( 1852 )

## 2023级HAUT新生周赛 ( 三 ) @22级学姐专场 ( 杨焱，刘振歌，周欣滢 )

• [比赛页](#)

A 糖果之“战” ( 1837 ) ; B 简单统计数字 ( 1838 ) ; C 奖学金 ( 1839 ) ; D 派蒙爱吃素 ( 1840 ) ; E 神奇的谕示裁定枢机 ( 1841 ) ; F 金币！ ( 1842 ) ; G 输出WAGD ( 1843 ) ; H 纵向排版 ( 1844 )

## 2023级HAUT新生周赛 ( 二 ) @曹瑞峰专场

• [比赛页](#)

A 小明的人生难题 ( 1825 ) ; B 小明的午餐难题 ( 1826 ) ; C 小明的内卷日 ( 1827 ) ; D 小明的数字 ( 1828 ) ; E 小明的打怪游戏 ( 1829 ) ; F 小明的出题日 ( 1830 ) ; G 小明的数学游戏 ( 1833 ) ; H 小明的幸运数 ( 1832 )

## 2023级HAUT新生周赛 ( 一 ) @21级学长专场 ( 张子豪，张鑫，李昊阳 )

• [比赛页](#)

A 我爱编程，编程使我快乐。( 1817 ) ; B 二进制翻转 ( 1818 ) ; C 二进制扩列 ( 1819 ) ; D 睡过去的jxh仍在学习 ( 1820 ) ; E 寝室的宝箱 ( 简单版 ) ( 1821 ) ; F 寝室的宝箱 ( 困难版 ) ( 1822 ) ; G 排队 ( 1823 ) ; H jxh和lhy的决斗 ( 1824 )

## 2023级HAUT新生周赛 ( 零 ) 熟悉周赛规则专场

• [比赛页](#)

A I really love haut! ( 1754 ) ; B 工作罢了 ( 1755 ) ; C 挑选礼物，但是收礼人 ( 1756 ) ; D 表白 ( 1757 ) ; E 约会？干饭！ ( 1758 ) ; F 好数 ( 1760 ) ; G glycoris ( 1761 ) ; H 比赛模拟 ( 1753 ) ; I 我的矩形 ( 1815 ) ; J 堆栈的奇妙世界 ( 1816 )

## 2022级HAUT新生周赛（六）@张鑫专场

- [比赛页](#)

A 寻找相似度 ( 1794 ) ; B 勇士的末路 ( 1795 ) ; C 数字统计 ( 1796 ) ; D 最大数 ( 1797 ) ; E 宝石 ( 1798 ) ; F 打印图形 ( 1799 ) ; G 比较大小 ( 1800 ) ; H 握手 ( 1801 )

## 2022级HAUT新生周赛（五）@李昊阳专场

- [比赛页](#)

A 夏虫 ( 1786 ) ; B 万分之一的光(easy version) ( 1787 ) ; C 万分之一的光(hard version) ( 1788 ) ; D 一花依世界 ( 1789 ) ; E 九九八十一 ( 1790 ) ; F 达拉崩吧 ( 1791 ) ; G 夜间出租车 ( 1792 ) ; H 一半一半 ( 1793 )

## 2022级HAUT新生周赛（四）@张子豪专场

- [比赛页](#)

A 调查员三宝之朋友 ( 1780 ) ; B 调查员三宝之撬棍 ( 1781 ) ; C 调查员三宝之旧印 ( 1782 ) ; D 签到题 ( 1779 ) ; E 真·签到题 ( 1778 ) ; F 最大值 ( 1783 ) ; G 棋子反转 ( 1784 ) ; H 神之天平 ( 1785 )

## 2022级HAUT新生周赛（三）@焦小航专场

- [比赛页](#)

A 我愿付出一切，只为再次看到你的笑容 ( 1770 ) ; B 赶上世界崩坏的速度，将你拯救 ( 1771 ) ; C Stay with you ( 1772 ) ; D 为了再见你一面，我愿意重启整个世界 ( 1773 ) ; E 我愿意为你跨越世界的尽头 ( 1774 ) ; F 关键词 ( 1775 ) ; G 将故事写成我们 ( 1776 ) ; H 即使世界毁灭，我也想再见你一面 ( 1777 )

## 2022级HAUT新生周赛（二）@武其轩专场

- [比赛页](#)

A 约好了下午三点，在记忆博物馆 ( 1768 ) ; B 这就是绝交证明书 ( 1763 ) ; C 我们之间没有你插手的余地哦？ ( 1764 ) ; D 我什么都愿意做 ( 1765 ) ; E 绝对不能回邮件哦 ( 1766 ) ; F 我的名字 ( 1767 ) ; G 唯一的道标 ( 1769 ) ; H 喂喂，你知道魔法少女的这个传闻吗？ ( 1762 )

## 2022级HAUT新生周赛（一）@卞子骏专场

- [比赛页](#)

A I really love haut! ( 1754 ) ; B 工作罢了 ( 1755 ) ; C 挑选礼物，但是收礼人 ( 1756 ) ; D 表白 ( 1757 ) ; E 约会？干饭！ ( 1758 ) ; F 免费餐食 ^o^y ( 1759 ) ; G 好数 ( 1760 ) ; H lycoris ( 1761 )

## 2022级HAUT新生周赛（零）熟悉周赛规则专场

- [比赛页](#)

A 卷卷签到成功 ( 1573 ) ; B 卷卷的空间限制 ( 1574 ) ; C 卷卷的石子 ( 1575 ) ; D 卷卷今天休息(?) ( 1576 ) ; E 聚聚的小游戏 ( 1577 ) ; F 聚聚的幸运数字 ( 1578 ) ; G 比赛模拟 ( 1753 )

## 2021级HAUT新生周赛（寒假专场）@苏锦鹏专场

- [比赛页](#)

A 小C刷题 ( 1743 ) ; B 小C爱编程 ( 1744 ) ; C 小C的选择 ( 1745 ) ; D 小C买泡芙 ( easy version ) ( 1746 ) ; E 小C买泡芙 ( hard version ) ( 1747 ) ; F 小C的快乐寒假 ( 1751 ) ; G 零食采购员小C ( 1748 ) ; H 小C和反转数字 ( 1752 ) ; I 小C的回文矩阵 ( 1750 ) ; J 小C的又一个矩阵 ( 1742 ) ; K 小C的cf上分历程 ( 1749 )

## 2021级HAUT新生周赛（八）@孔维飒专场

- [比赛页](#)

A 似乎在梦里见过那样 ( 1734 ) ; B 那真是太令人高兴了 ( 1735 ) ; C 已经没有什么好怕的了 ( 1736 ) ; D 奇迹魔法都是存在的 ( 1737 ) ; E 怎么可能会后悔 ( 1738 ) ; F 这种事绝对很奇怪啊 ( 1739 ) ; G 你能面对真正的内心吗 ( 1740 ) ; H 我，真是笨蛋 ( 1741 )

## 2021级HAUT新生周赛（七）@王佳琪专场

- [比赛页](#)

A 想象之中 ( 1726 ) ; B 你是人间四月天 ( 1727 ) ; C 关键词 ( 1728 ) ; D 爱情废柴 ( 1729 ) ; E 错位时空 ( 1730 ) ; F 年少有为 ( 1731 ) ; G 交换余生 ( 1732 ) ; H 突然之间 ( 1733 )

## 2021级HAUT新生周赛（六）@李志豪专场

- [比赛页](#)

A 界面越花 编程越拉 ( 1717 ) ; B 你是个好人 ( 1718 ) ; C 爱你呦 ( 1719 ) ; D 爬山 ( 1721 ) ; E 游戏 ( 1722 ) ; F 你又是个好人 ( 1723 ) ; G 结束了 (easy version) ( 1725 ) ; H 结束了(hard version) ( 1724 )

## 2021级HAUT新生周赛（五）@常浩锋专场

- [比赛页](#)

A 爱因斯坦光电效应 ( 1709 ) ; B 图书馆信息管理系统 ( 1710 ) ; C 这年头难道有人不喜欢签到吗? ( 1711 ) ; D Isjp和他的小伙伴们 ( 1712 ) ; E lwjq与国际象棋 ( 1713 ) ; F lzlif的三角形绝技 ( 1714 ) ; G lwyj的最短路径 ( 1715 ) ; H cgg学长的致富之路 ( 1716 )

## 2021级HAUT新生周赛（四）@王一杰专场

- [比赛页](#)

A 小锋的书 ( 1700 ) ; B 大海我来了 ( 1701 ) ; C 拔河比赛 ( 1702 ) ; D 子序列和 ( 1707 ) ; E 博学的学长 ( 1704 ) ; F 人生的抉择 ( 1705 ) ; G 小锋卖瓜 ( 1706 ) ; H 对称括号序列 ( 1708 )

## 2021级HAUT新生周赛（三）@李晨曦专场

- [比赛页](#)

A 卷王之争 ( 1589 ) ; B 纸牌游戏 ( 1590 ) ; C 签到时间 ( 1591 ) ; D 干饭时间 ( 1592 ) ; E 游戏时间 ( 1593 ) ; F 周赛榜单 ( 1596 ) ; G 没有名字 ( 1597 ) ; H 三角图形 ( 1598 )

## 2021级HAUT新生周赛（二）@李亚凯专场

- [比赛页](#)

A 牛牛的签到题 ( 1579 ) ; B 牛牛 mex 函数 ( 1580 ) ; C 牛牛的战斗力 ( 1594 ) ; D 牛牛的x数 ( 1582 ) ; E 牛牛的素数判断 ( 1583 ) ; F 牛牛的导数 ( 1584 ) ; G 牛牛的循环次数 ( 1585 ) ; H 牛牛的数独 ( 1595 )

## 2021级HAUT新生周赛（一）@张君毅专场

- [比赛页](#)

A 卷卷签到成功 ( 1573 ) ; B 卷卷的空间限制 ( 1574 ) ; C 卷卷的石子 ( 1575 ) ; D 卷卷今天休息(?) ( 1576 ) ; E 聚聚的小游戏 ( 1577 ) ; F 聚聚的幸运数字 ( 1578 ) ; G 聚聚的子串 ( 1587 ) ; H 聚聚的Rating目标 ( 1588 )

## 2020级HAUT新生周赛（五）@许金龙专场

- [比赛页](#)

A 获得自由? ( 1564 ) ; B 盖瓦 ( 1565 ) ; C 平衡区间 ( 削弱版 ) ( 1566 ) ; D 石子游戏 ( 1567 ) ; E 平衡区间 ( 1568 ) ; F 数字钟 ( 1570 ) ; G 放烟花 ( 1571 ) ; H 完美序列 ( 1572 )

## 2020级HAUT新生周赛（四）@张承树专场

- [比赛页](#)

A 会写脚本的月月鸟 ( 1555 ) ; B 别看了 这是水题 ( 1556 ) ; C ACM脱单大法 ( 1557 ) ; D 我劝你们这些年轻人，不要轻易碰此题 ( 1558 ) ; E 后缀自动机next指针dag图上跑SG函数 ( 1559 ) ; F 新建 Microsoft PowerPoint 演示文稿.pptx ( 1560 ) ; G 禁止复读 ( 1561 ) ; H 吓得我赶紧出了个签到题 ( 1562 )

## 2020级HAUT新生周赛（三）@张雍蘩专场

- [比赛页](#)

A 递归式 ( 1547 ) ; B 旅行 ( 1548 ) ; C 替换 ( 1549 ) ; D 光的反射 ( 1550 ) ; E 区间和 ( 1551 ) ; F 区间和 ( 加强版 ) ( 1552 ) ; G 求和 ( 1553 ) ; H 选择 ( 1554 )

## 2020级HAUT新生周赛（二）@杨哲专场

- [比赛页](#)

A 阿正的忐忑不安 ( 1539 ) ; B 阿正的学期准备 ( 1540 ) ; C 阿正的快乐源泉 ( 1541 ) ; D 阿正的平面行进 ( 1542 ) ; E 阿正的英语阅读 ( 1543 ) ; F 阿正的子母序列 ( 1544 ) ; G 阿正的排球测试 ( 1545 ) ; H 阿正的换位排序 ( 1546 )

## 2020级HAUT新生周赛（一）@李晨龙专场

- [比赛页](#)

A 小青的班委竞选 ( 1531 ) ; B 小青的报道 ( 1529 ) ; C 小青的传统艺能 ( 1530 ) ; D 小青的大学英语 ( 1538 ) ; E 小青的高等数学 ( 1532 ) ; F 小青的女装生涯 ( 1536 ) ; G 小青的宿舍 ( 1534 ) ; H 小青的线性代数 ( 1533 ) ; I 小青和四火的肥宅生活 ( 1537 ) ; J 小青与nana的不解之谜 ( 1535 )

## 2019级河南工大新生周赛总决赛@王嘉豪专场

- [比赛页](#)

A CXK的篮球数 ( 加强版 ) ( 1504 ) ; B CXK想要篮球 ( 1505 ) ; C Help Me ! ! ! ! ! ( 1520 ) ; D 抓牛 ( 1522 ) ; E 游戏 ( 1526 ) ; F 读书 ( 1523 ) ; G 买东西 ( 1527 ) ; H x的n次幂 ( 1528 )

## 2019级河南工大新生周赛（六）@张伟涛专场

- [比赛页](#)

A 棋盘上的米粒 ( 1514 ) ; B 不一样的蛋糕 ( 1515 ) ; C HJ病毒 ( 1516 ) ; D 最小的整数 ( 1518 ) ; E 第k大分数 ( 简单版 ) ( 1519 ) ; F 第k大分数 ( 加强版 ) ( 1517 )

## 2019级河南工大新生周赛（五）@潘世泽专场

- [比赛页](#)

A zp的新冒险 ( 1506 ) ; B zp与车费 ( 1508 ) ; C zp与朋友 ( 1509 ) ; D zp与宝可梦 ( 题面已修改 ) ( 1510 ) ; E zp与火系道馆 ( 对小19的再次关怀 ) ( 1511 ) ; F zp与草系道馆 ( 1512 ) ; G zp与水系道馆 ( 1513 )

## 2019级河南工大新生周赛（四）@胡维强专场

- [比赛页](#)

A 想带Ah去浪漫的土耳其 ( 1493 ) ; B 如果能重来我要选Jinx ( 1496 ) ; C 村里有个姑娘叫小Jun ( 1498 ) ; D 对小19的亲切关怀 ( 1501 ) ; E 是ZP的步伐 ( 1502 ) ; F 欢迎来到LXD广场 ( 1503 )

## 2019级河南工大新生周赛（三）@邓祎培专场

- [比赛页](#)

A 那年那题那些AC ( 1486 ) ; B 你的AC。 ( 1495 ) ; C AC之子 ( 1487 ) ; D 虹猫蓝兔AC转 ( 1489 ) ; E 成龙AC记 ( 1490 ) ; F AC小将 ( 1491 )

## 2019级河南工大新生周赛（二）@邹岩专场

- [比赛页](#)

A 男欢女爱 ( 1480 ) ; B 欢天喜地 ( 1481 ) ; C 郁郁寡欢 ( 1482 ) ; D 寻欢作乐 ( 1483 ) ; E 悲欢离合 ( 1484 ) ; F 相得甚欢 ( 1485 )

## 2019级河南工大新生周赛（一）@陶福专场

- [比赛页](#)

A cxx想打篮球 ( 1479 ) ; B cxx的苹果总数 ( 1476 ) ; C cxx下棋 ( 1475 ) ; D cxx的篮球数 ( 1477 ) ; E cxx的进制转换问题 ( 1478 ) ; F cxx的数学难题 ( 1474 )

## 2018级河南工大新生周赛总决赛 ( 重现 )

- [比赛页](#)

A Chomp ( 1417 ) ; B Despair ( 1418 ) ; C Balance ( 1419 ) ; D Gaming ( 1420 ) ; E Invictus ( 1421 ) ; F Math ( 1422 ) ; G Minecraft ( 1423 ) ; H Greedy ( 1424 ) ; I Nineteen ( 1425 ) ; J Poem ( 1426 ) ; K River ( 1427 )

## 2018级河南工大新生周赛总决赛@陈丁山大神专场

- [比赛页](#)

A Chomp ( 1417 ) ; B Despair ( 1418 ) ; C Balance ( 1419 ) ; D Gaming ( 1420 ) ; E Invictus ( 1421 ) ; F Math ( 1422 ) ; G Minecraft ( 1423 ) ; H Greedy ( 1424 ) ; I Nineteen ( 1425 ) ; J Poem ( 1426 ) ; K River ( 1427 )

## 2017级河南工大新生周赛总决赛 ( 重现 )

- [比赛页](#)

A QAQ Games ( 1361 ) ; B 表面兄弟 ( 1362 ) ; C Hearthstone ( 1363 ) ; D Travel ( 1364 ) ; E Takeaway Stealer ( 1365 ) ; F 辛普森积分与球面公式 ( 1366 ) ; G Lighting ( 1367 ) ; H Run out of Prison ( 1368 ) ; I Displaced Plant ( 1369 ) ; J Flowers ( 1227 )

## 2018级河南工大新生周赛 ( 八 ) @李维航专场

- [比赛页](#)

A 签到题 ( 1428 ) ; B 迷宫 ( 1429 ) ; C 战舰 ( 1430 ) ; D 愉快序列 ( 1431 ) ; E 矿产含量 ( 1432 ) ; F 数学家 ( 1433 ) ; G 优化算法 ( 1434 ) ; H 文件 ( 1435 ) ; I 资源箱 ( 1436 )

## 2018级河南工大新生周赛 ( 七 ) @牛寅旭专场

- [比赛页](#)

A 数字分组 ( 1437 ) ; B 最大增区间 ( 一 ) ( 1438 ) ; C 最大增区间 ( 二 ) ( 1439 ) ; D 取物品 ( 1440 ) ; E 选数字 ( 1441 ) ; F 最大的最小区间 ( 1442 )

## 2018级河南工大新生周赛 ( 六 ) @李欣欣专场

- [比赛页](#)

官网要求竞赛密码，无法公开核验题目。

## 2018级河南工大新生周赛 ( 五 ) @董业伟专场

- [比赛页](#)

官网要求竞赛密码，无法公开核验题目。

## 2018级河南工大新生周赛 ( 四 ) @杜锐专场

- [比赛页](#)

A 文物鉴定 ( 1402 ) ; B 决胜的抓硬币 ! ( 1403 ) ; C 迷宫探险 ( 1404 ) ; D 元气满满 ( 1405 ) ; E 寻找遗迹 ( 1406 ) ; F 神秘代码 ( 1407 )

## 2018级河南工大新生周赛 ( 三 ) @陈帅专场

- [比赛页](#)

A 存储单位进率 ( 1396 ) ; B 外卖距离 ( 1397 ) ; C 容斥原理 ( 1398 ) ; D 逻辑判断 ( 1399 ) ; E three numbers and two operators ( 1400 ) ; F 点赞分能量 ( 1401 )

## 2018级河南工大新生周赛（二）@陈润钊专场

- [比赛页](#)

A 铺铺铺铺地板 ( 1388 ) ; B 曼曼曼曼曼曼 ( 1389 ) ; C 学学学学学素数 ( 1390 ) ; D 超超超超越数 ( 1391 ) ; E 玩玩玩玩石子 ( 1392 ) ; F 大大大大扫除 ( 1393 ) ; G 签签签签到题 ( 1394 )

## 2018级河南工大新生周赛（一）@徐向阳专场

- [比赛页](#)

A 友好的第一题 ( 1382 ) ; B 大学生活 ( 二 ) ( 1383 ) ; C 数学 ( 二 ) ( 1384 ) ; D 午饭问题 ( 二 ) ( 1385 ) ; E 加密工作 ( 二 ) ( 1386 ) ; F 体育课 ( 二 ) ( 1387 )

## 2017级河南工大新生周赛总决赛（重现）

- [比赛页](#)

A QAQ Games ( 1361 ) ; B 表面兄弟 ( 1362 ) ; C Hearthstone ( 1363 ) ; D Travel ( 1364 ) ; E Takeaway Stealer ( 1365 ) ; F 辛普森积分与球面公式 ( 1366 ) ; G Lighting ( 1367 ) ; H Run out of Prison ( 1368 ) ; I Displaced Plant ( 1369 ) ; J Flowers ( 1227 )

## 2017级河南工大新生周赛总决赛

- [比赛页](#)

A QAQ Games ( 1361 ) ; B 表面兄弟 ( 1362 ) ; C Hearthstone ( 1363 ) ; D Travel ( 1364 ) ; E Takeaway Stealer ( 1365 ) ; F 辛普森积分与球面公式 ( 1366 ) ; G Lighting ( 1367 ) ; H Run out of Prison ( 1368 ) ; I Displaced Plant ( 1369 ) ; J Flowers ( 1227 )

## 2017级河南工大新生周赛（八）

- [比赛页](#)

A 可以说是签到题 ( 1354 ) ; B 大学生活 ( 一 ) ( 1355 ) ; C 数学 ( 一 ) ( 1356 ) ; D 午饭问题 ( 一 ) ( 1357 ) ; E 加密工作 ( 一 ) ( 1360 ) ; F 体育课 ( 一 ) ( 1359 ) ; G 时光跑得太快，溜了溜了 ( 1358 )

## 2017级河南工大新生周赛（七）

- [比赛页](#)

A Choice的加法 ( 1347 ) ; B Choice的心情 ( 1348 ) ; C Choice的箱子 ( 1349 ) ; D Choice发糖果 ( 1350 ) ; E Choice and Bramble ( 1351 ) ; F Choice的字符串 ( 1352 ) ; G N ! ( 1353 )

## 2017级河南工大新生周赛（六）

- [比赛页](#)

A 比赛 ( 1341 ) ; B 消灭怪物 ( 1342 ) ; C 看医生 ( 1343 ) ; D 最小的数 ( 1344 ) ; E QAQ ( 1345 ) ; F 括号配对 ( 1346 )

## 2017级河南工大新生周赛（五）

- [比赛页](#)

A 01串 ( 1335 ) ; B 互质 ( 1336 ) ; C 偶数 ( 1337 ) ; D 多米诺骨牌 ( 1338 ) ; E 排队 ( 1339 ) ; F 移动坐标 ( 1340 )

## 2017级河南工大新生周赛（四）

- [比赛页](#)

A 趣味游戏 ( 一 ) : 喝饮料 ( 1329 ) ; B 趣味游戏 ( 二 ) : 铺地板 ( 1330 ) ; C 趣味游戏 ( 三 ) : 剑圣 ( 1331 ) ; D 趣味游戏 ( 四 ) : 蛮王 ( 1332 ) ; E 趣味游戏 ( 五 ) : 吃鸡 ( 1333 ) ; F 趣味游戏 ( 六 ) : 爬楼梯 ( 1334 )

## 2017级河南工大新生周赛（三）

- [比赛页](#)

A 欢迎来到实力至上主义的OJ ( 1328 ) ; B 崩崩崩 ( 1322 ) ; C 和小埋一起做准备运动 ( 1324 ) ; D 魔法卡题少女 ( 1323 ) ; E Fate/Accepted ( 1325 ) ; F 炮姐的硬币 ( 1326 )

## 2017级河南工大新生周赛 ( 二 )

- [比赛页](#)

A Choice学姐买糖果 I ( 1314 ) ; B Choice学姐买糖果 II ( 1320 ) ; C Choice学姐买糖果III ( 1316 ) ; D Choice学姐买糖果 IV ( 1317 ) ; E Choice学姐买糖果 V ( 1318 ) ; F Choice学姐的众数问题 ( 1321 )

## 2017级河南工大新生周赛 ( 一 )

- [比赛页](#)

A 第一个程序 ( 1308 ) ; B 大学生活 ( 1309 ) ; C 数学 ( 1310 ) ; D 午饭问题 ( 1311 ) ; E 加密工作 ( 1312 ) ; F 体育课 ( 1313 )

## 2016级河南工大新生周赛总决赛

- [比赛页](#)

A Simple学长玩游戏 ( 1250 ) ; B Simple学长的史诗级难题 ( 1251 ) ; C Simple学长的数字游戏 ( 1252 ) ; D Simple学长翻硬币 ( 1253 ) ; E Simple学长的礼物 ( 1254 ) ; F Simple学长的数学题 ( 1255 ) ; G Simple学长的a+b ( 1256 )

## 2016级河南工大新生周赛 ( 七 )

- [比赛页](#)

A zy最爱的足球赛 ( 1249 ) ; B XXX班的团事活动 ( 1241 ) ; C 最小的距离之和 ( 1248 ) ; D ykc's easy exam ( 1246 ) ; E cds的大大阶乘 ( 1245 ) ; F 又是郊游 ( 1242 ) ; G 简单的数学问题 ( 1243 ) ; H 神奇的井 ( 1244 )

## 2016级河南工大新生周赛 ( 六 )

- [比赛页](#)

A 此为背景 ( 1236 ) ; B Simple学长买咖啡 ( 1238 ) ; C Simple学长数咖啡 ( 1237 ) ; D Simple学长整理咖啡 ( 1239 ) ; E Simple学长取咖啡 ( 1234 ) ; F Simple学长喝咖啡 ( 1235 ) ; G Simple学长发咖啡 ( 1240 )

## 2016级河南工大新生周赛 ( 五 )

- [比赛页](#)

A Crazy Computer ( 1226 ) ; B Flowers ( 1227 ) ; C 数字游戏 ( 1228 ) ; D 等式判断 ( 1229 ) ; E 矩阵乘法 ( 1230 ) ; F ykc买零食 ( 1231 ) ; G 手机剩余电量 ( 1232 )

## 2016级河南工大新生周赛 ( 四 )

- [比赛页](#)

A ykc的简单数学题 ( 1220 ) ; B 疲惫的一天 ( 1221 ) ; C 1的个数 ( 1222 ) ; D 约瑟夫环 ( 1223 ) ; E Beru-taxi ( 1224 ) ; F 零钱问题 ( 1225 )

## 2016级河南工大新生周赛 ( 三 )

- [比赛页](#)

A Meeting Friends ( 1213 ) ; B 蚂蚁 ( 1215 ) ; C 三角形面积 ( 1216 ) ; D 最大公约数 ( 1217 ) ; E 最大连续子段和 ( 1218 ) ; F 求和 ( 1219 )

## 2016级河南工大新生周赛 ( 二 )

- [比赛页](#)

A Add Water ( 1212 ) ; B A Simple Math Problem ( 1211 ) ; C lolizlm的数字 ( 1207 ) ; D txt的号码 ( 1208 ) ; E Let's play ( 1210 ) ; F kx的压缩 ( 1209 )

## 2016级河南工大新生周赛 ( 一 )

- [比赛页](#)

A 多项式相加 ( 多实例 ) ( 1199 ) ; B 平均数计算 ( 1200 ) ; C 四则运算 ( 1037 ) ; D 圆的交点 ( 1201 ) ; E 韩信点兵 ( 1202 ) ; F 取石子游戏 ( 1203 )

## 最后提醒

真正面向选拔的完成标准是：陌生题能从范围判断模型、能独立写出并调通、能解释为什么正确、能在一周后不看答案重做。当年具体选拔时间、名额和规则应以学院或校队最新通知为准。